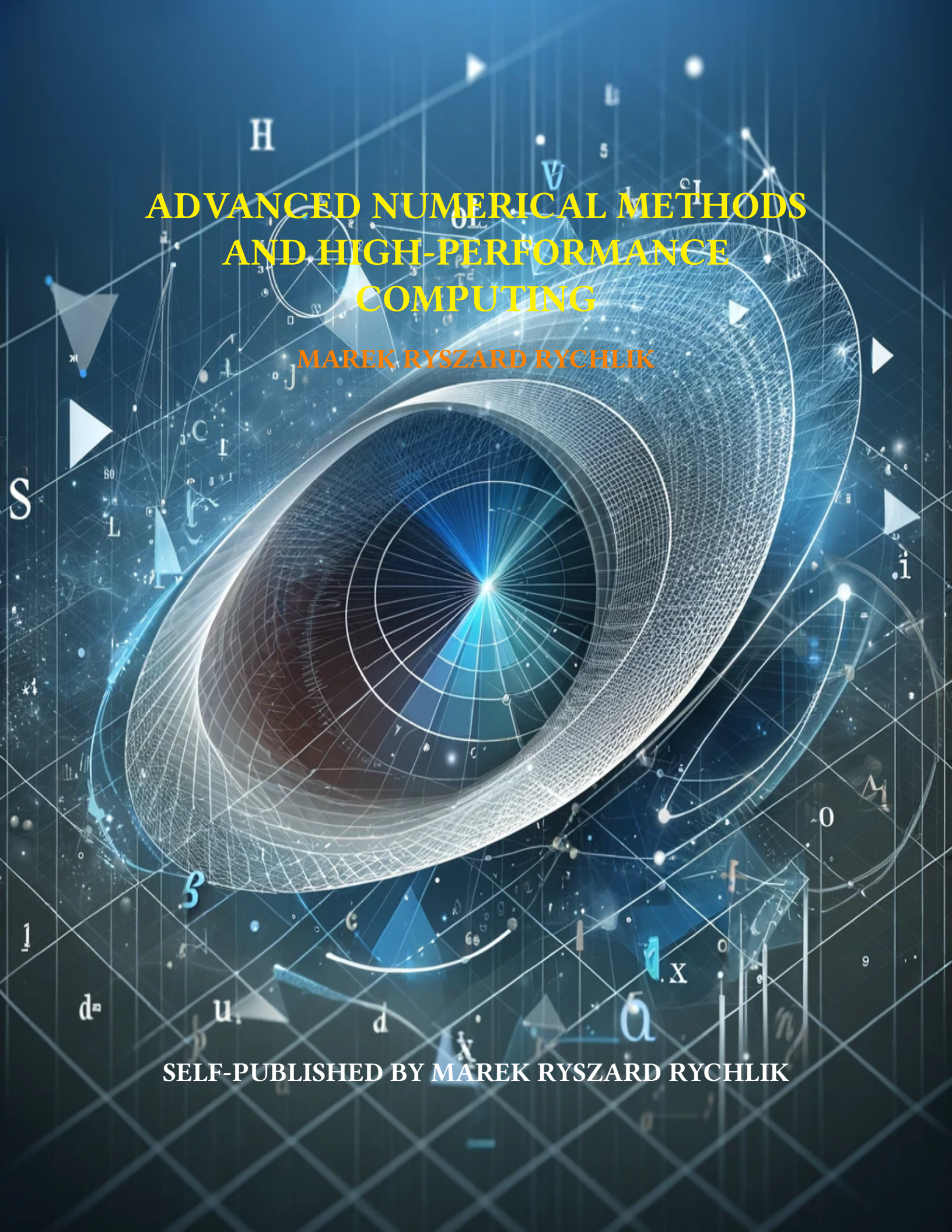




ADVANCED NUMERICAL METHODS AND HIGH-PERFORMANCE COMPUTING

MAREK RYSZARD RYCHLIK

SELF-PUBLISHED BY MAREK RYSZARD RYCHLIK

Advanced Numerical Methods And High-Performance Computing

Copyright © 2024 Marek Ryszard Rychlik

All rights reserved.

No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without the prior written permission of the copyright owner.

First published in 2024 by
Self-published by Marek Ryszard Rychlik
Tucson, Arizona, United States of America

Printed in the United States of America

Disclaimer: The author and publisher have made every effort to ensure that the information in this book is accurate and up-to-date. However, they make no warranties or representations as to its accuracy and disclaim any liability for any errors or omissions.

Contents

Prologue	17
The Role of Automatic Differentiation in Numerical Methods	17
Computational Graphs and the Stack	17
Operator overloading in Python	19
Conclusion	19
Exercises	19
1 Introduction to Numerical Methods and HPC	21
1.1 Motivation and Applications	21
1.1.1 Real-World Applications	21
1.2 Overview of High-Performance Computing (HPC)	21
1.2.1 HPC Hardware Architectures	21
1.2.2 Parallel Computing Models	22
1.2.3 Programming for HPC	22
1.2.4 Importance of HPC in Numerical Methods	22
1.3 Structure of the Book	22
2 Error Analysis	25
2.1 Computer Arithmetic	25
2.1.1 Floating-Point Representation	25
2.1.2 Notation for integers in arbitrary base	25
2.1.3 How to access the bits of a IEEE 754 float in Python?	27
2.1.4 Definition of Machine Numbers	28
2.1.5 Precision and Rounding	29
2.1.6 Machine Epsilon	29
2.1.7 Machine Epsilon in Accuracy Analysis	30
2.1.8 A Theorem on Accuracy of Naive Summation	32
2.1.9 Errors in Numerical Computations	35
2.1.10 Examples of Floating-Point Arithmetic in C++	37
2.1.11 Mitigating Numerical Errors	37
2.1.12 Case Study: Summing a Large Array	37
2.2 Unit in the Last Place (ulp)	38
2.2.1 Definition of ulp	38
2.2.2 Significance of ulp	38
2.2.3 0.5 ulp Guarantee	38
2.2.4 Implications for Numerical Computing	39
2.3 Minifloats and Floating-Point Arithmetic	39
2.3.1 Floating-Point Representation	39
2.3.2 The (1.4.3) Minifloat Format	39
2.3.3 Normalized and Subnormal Numbers	40

2.3.4	Special Values	40
2.3.5	Arithmetic Example: Linear Function $f(x) = 3x + 1$	40
2.3.6	Arithmetic Example: Quadratic Function $f(x) = (x - 1)^2$	40
2.3.7	Key Observations	40
2.3.8	Exercises for Students	41
2.3.9	Pedagogical Role	41
2.4	Types of Errors	41
2.4.1	Round-Off Errors	41
2.4.2	Truncation Errors	41
2.4.3	Absolute and Relative Errors	41
2.5	Error Propagation	41
2.5.1	Propagation of Errors in Arithmetic Operations	41
2.5.2	Conditioning of Problems	42
2.6	Stability and Convergence	42
2.6.1	Stability	42
2.6.2	Convergence	42
2.6.3	Consistency	42
2.7	Error Analysis in Numerical Methods	42
2.7.1	Taylor Series Expansion	42
2.7.2	Backward Error Analysis	42
2.7.3	Forward Error Analysis	42
2.8	Practical Considerations for Error Control	43
2.8.1	Adaptive Methods	43
2.8.2	Error Estimation and Tolerance	43
2.8.3	Validation and Verification	43
2.9	Exercises	43
2.9.1	Floating-Point Representation	43
2.9.2	Precision and Rounding	43
2.9.3	Errors in Numerical Computations	43
2.9.4	Unit in the Last Place (ulp)	44
2.9.5	Mitigating Numerical Errors	44
2.9.6	Backward Stability	44
3	Root-Finding Algorithms	47
3.1	Introduction to Root-Finding	47
3.2	Bisection Method	47
3.2.1	Algorithm	47
3.2.2	About Algorithms and Pseudo-Code	48
3.2.3	Error Analysis	49
3.3	Fixed-Point Iteration	49
3.3.1	Algorithm	50
3.3.2	Convergence Criteria	51
3.3.3	Proof of convergence	51
3.4	The Two-Ladder Problem	51
3.4.1	The Classical Two-Ladder Problem	51
3.4.2	The Modified Two-Ladder Problem	52
3.4.3	The dimensionless form of the ladder equation	54
3.5	Newton-Raphson Method	54
3.5.1	Algorithm	55
3.5.2	Rationale	55
3.5.3	Error Analysis	56
3.5.4	A proof (idea only)	56

3.6	Secant Method	56
3.6.1	Algorithm	56
3.6.2	Error Analysis	57
3.6.3	Secant Method as Fixed Point Iteration	57
3.6.4	The Secant Method and Its Order of Convergence	57
3.7	False Position Method	58
3.7.1	Algorithm	59
3.7.2	Error Analysis	59
3.8	Comparisons and Practical Considerations	59
3.8.1	Convergence Rates	59
3.8.2	Choosing the Right Method	59
3.8.3	Implementation and Efficiency	60
3.9	Automatic Differentiation	60
3.9.1	Automatic differentiation in Python and Syntactic Sugar	60
3.9.2	How does automatic differentiation work?	60
3.10	Exercises	61
4	Numerical Linear Algebra	63
4.1	Introduction to Numerical Linear Algebra	63
4.2	Example: Principal Component Analysis (PCA)	63
4.2.1	Motivation	63
4.2.2	PCA Algorithm	63
4.2.3	Example Implementation	64
4.2.4	Discussion	64
4.3	Outline of the Chapter	64
4.4	Direct Methods for Linear Systems	65
4.4.1	Gaussian Elimination	65
4.4.2	Gaussian Elimination with Partial Pivoting	65
4.4.3	Gaussian Elimination with Complete Pivoting	65
4.4.4	Back Substitution	65
4.5	LU Decomposition	68
4.5.1	Gaussian Elimination with Simulated Row Exchanges	68
4.5.2	PA=LU Decomposition	70
4.6	Normed Vector Spaces and the Condition Number	73
4.6.1	On Frobenius norm	73
4.6.2	Condition number of a matrix	74
4.6.3	The fundamental inequality	74
4.6.4	Exercises	75
4.7	Least Squares Approximation and Hilbert Matrices	75
4.7.1	Least Squares Criterion	75
4.7.2	Normal Equations	75
4.7.3	Matrix Formulation	76
4.7.4	Hilbert Matrix Connection	76
4.8	Iterative Methods	77
4.8.1	Jacobi Method and Spectral Radius	78
4.8.2	Analysis of the Fixed-Point Equation	78
4.8.3	Jacobi Method	79
4.8.4	Gauss-Seidel Method	79
4.8.5	Successive Over-Relaxation (SOR)	80
4.9	Poisson Equation with a Point Charge	80
4.9.1	On the sign of the Laplace operator	80
4.9.2	Point Charge Source Term	81

4.9.3	Boundary Conditions	81
4.9.4	Discretization and Sparse Linear System	81
4.9.5	Solution Using Iterative Methods	81
4.9.6	Exercises on solving Poisson Equation	81
4.10	Eigenvalue Problems	84
4.10.1	Power Method	84
4.11	Introduction to QR Decomposition	87
4.12	Definition and Notation	87
4.13	Gram-Schmidt Orthogonalization	87
4.13.1	The Classical Gram-Schmidt (CGS) Process	87
4.13.2	Modified Gram-Schmidt (MGS)	88
4.13.3	A numerical example	88
4.13.4	Some Applications of QR Decomposition	91
4.13.5	Linear Systems	91
4.13.6	Least Squares	91
4.14	Solving a Least Squares Problem without Differentiation	91
4.14.1	Problem Formulation	91
4.14.2	Adjoint Operator	92
4.14.3	Solution to the Least Squares Problem	92
4.14.4	The Pythagorean Theorem and Sufficiency	93
4.14.5	Computing Pseudo-inverse and Orthogonal Projection using QR-decomposition	93
4.15	Finding Complex Dominant Eigenvalues Using a Modified Power Method	94
4.15.1	Detailed pseudo-code	96
4.15.2	Retrieval of the Complex Eigenvector	96
4.15.3	Summary of the Approach	97
4.16	QR Algorithm	98
4.16.1	A discussion of the QR algorithm	99
4.17	Advanced QR Algorithm	100
4.17.1	Reduction to Tridiagonal Form	100
4.18	QR Factorization by Householder Reflections	103
4.19	Wilkinson Shift	106
4.19.1	Context	106
4.19.2	Motivation for the Shift	106
4.19.3	Definition of the Wilkinson Shift	106
4.19.4	Algorithm	106
4.19.5	Benefits	106
4.19.6	Applications	106
4.19.7	QR Algorithm with Shifts	107
4.19.8	Inverse Power Method with Shifts	107
4.19.9	Combining Methods	107
4.20	Singular Value Decomposition (SVD) from Symmetric Matrices	108
4.20.1	Connection to Symmetric Matrices	108
4.20.2	Relationship Between Eigenvalues and Singular Values	108
4.20.3	Constructing the SVD	108
4.20.4	Final Result	108
4.20.5	Representation of a matrix as a Sum of Rank One Matrices	109
4.20.6	A formal proof	109
4.20.7	Alternative forms of SVD	110
4.20.8	Summary	110
4.21	Comparisons and Practical Considerations	110
4.21.1	Choosing the Right Method	110
4.21.2	Implementation and Efficiency	110

4.22	Exercises	111
4.23	Application-Oriented Exercises for SVD	115
5	Multi-Dimensional Root Finding	117
5.1	Example: Mathematical Theory of GPS Positioning	117
5.1.1	Satellites and Signals	117
5.1.2	Time-of-Flight Calculation	117
5.1.3	Distance Spheres	117
5.1.4	Trilateration	117
5.1.5	Mathematical Formulation	118
5.2	Numerical Example	118
5.2.1	Example Scenario	118
5.2.2	Satellite Positions and Signal Travel Times	118
5.2.3	Calculating Distances	118
5.2.4	Setting Up the System of Equations	119
5.2.5	Solving the System of Equations	119
5.2.6	Example Solution	119
5.2.7	Conclusion	119
5.3	Fixed Point Iteration	119
5.4	Banach Contraction Principle	119
5.4.1	Statement of the Banach Contraction Principle	119
5.4.2	Proof of the Banach Contraction Principle	120
5.5	Exercises	121
5.5.1	Algorithm	121
5.5.2	Example	121
5.6	Newton's Method	122
5.6.1	Algorithm	122
5.6.2	An Example	122
5.7	Proof of Convergence of Newton's Method	123
5.7.1	Theorem	124
5.7.2	Proof	124
5.8	Sherman-Morrison Formula	125
5.9	Broyden's Good Method	125
5.9.1	The Purpose of the Update Rule for B	125
5.9.2	Example	126
5.10	Proof of Convergence of Broyden's Good Method	127
5.10.1	Theorem	128
5.10.2	Proof	128
5.11	Kantorovich Theorem on Newton's Method	128
5.11.1	Example Applications of Kantorovich Theorem	131
5.12	Exercises	134
6	Unconstrained and Constrained Optimization	137
6.1	Linearly Constrained Quadratic Optimization in Portfolio Selection	137
6.1.1	Application in Markowitz Portfolio Optimization	138
6.1.2	A More Interesting Optimization Problem	139
6.1.3	Lagrange multipliers	139
6.1.4	Conclusion	139
6.2	Gradient Descent	139
6.3	Newton's Method for Optimization	140
6.4	BFGS Quasi-Newton Method and Its Limited-Memory Variant	140
6.4.1	BFGS Algorithm	140

6.4.2	Sherman-Morrison-Woodbury Formula	141
6.5	Rationale of the BFGS Method	142
6.5.1	Limited-memory BFGS (L-BFGS) Algorithm	144
6.5.2	Comparison Between BFGS and L-BFGS	145
6.5.3	A numerical example	145
6.5.4	Conclusion	147
6.6	Backtracking in Gradient Descent	147
6.6.1	The Armijo Rule	147
6.6.2	Wolfe Conditions	147
6.6.3	Strong Wolfe Conditions	147
6.6.4	Goldstein Conditions	148
6.7	Stochastic Gradient Descent (SGD)	148
6.7.1	Definition	148
6.7.2	Advantages and Challenges	148
6.7.3	Applications	148
6.7.4	Variants of SGD	149
6.8	Gradient Projection Method	149
6.8.1	Algorithm	149
6.8.2	Example Problem	150
6.8.3	Illustrative Exercise	150
6.9	Computed Tomography	151
6.9.1	Problem Formulation	151
6.9.2	Tikhonov Regularization	152
6.9.3	Total Variation (TV) Regularization	152
6.9.4	Optimization Methods: BFGS and L-BFGS	152
6.9.5	Datasets and References	152
6.10	Exercises	153
7	Interpolation and Polynomial Approximation	155
7.1	Introduction to Interpolation	155
7.2	Lagrange Interpolation	155
7.2.1	Lagrange Polynomial	155
7.2.2	Algorithm	155
7.2.3	Error Analysis	155
7.3	Newton Interpolation	156
7.3.1	Newton's Divided Differences	156
7.3.2	Newton Interpolating Polynomial	156
7.3.3	Algorithm	156
7.3.4	Lagrange Remainder Formula	156
7.3.5	Error Analysis	157
7.3.6	Chebyshev Nodes	157
7.4	Algorithm to Construct the Divided Difference Table	157
7.5	Spline Interpolation	159
7.5.1	An Overview of Cubic Splines	159
7.5.2	Algorithm	159
7.5.3	Error Analysis	159
7.5.4	Proof of the Constant 384	159
7.5.5	A Detailed Algorithm for Cubic Splines	160
7.5.6	Pseudocode	162
7.5.7	Conclusion	162
7.6	Piecewise Interpolation	162
7.6.1	Algorithm	162

7.7	Finite Difference Formulas	162
7.7.1	Forward Difference Formula	162
7.7.2	Backward Difference Formula	162
7.7.3	Central (Symmetric) Difference Formula	163
7.7.4	Newton's Forward and Backward Difference Formulas	163
7.7.5	Central Difference Formula	164
7.8	Verification of the Third Derivative Approximation Using Taylor Expansion	165
7.8.1	Verification Using Python and SymPy	165
7.8.2	An Alternative Central Difference Third Derivative Approximation	166
7.8.3	Basic Example	166
7.9	Stirling's Central Difference Interpolation Formula	166
7.9.1	Numerical Example	167
7.10	Finite Difference Calculus in Terms of Operator Theory	168
7.10.1	Shift Operator and Difference Operators	168
7.10.2	Higher-Order Differences and Binomial Expansion	169
7.10.3	Discrete Laplacian and Exponential Representation	169
7.11	Mean Value Theorem for Divided Differences	169
7.11.1	Proof	170
7.11.2	References	170
7.12	Exercises	170
8	Numerical Integration	173
8.1	Introduction	173
8.1.1	Trapezoidal Rule	173
8.1.2	Derivation of the Single-Interval Trapezoidal Rule Error Formula	173
8.1.3	Single-Step Simpson's Rule and Its Error Term	175
8.2	Composite Rules for Adaptive Integration	177
8.2.1	Composite Trapezoidal Rule	177
8.2.2	Composite Simpson's Rule	177
8.2.3	Gaussian Quadrature	178
8.2.4	Legendre Polynomials	178
8.2.5	Derivation of Gaussian Quadrature Weights	180
8.2.6	The error bound	180
8.2.7	Exactness for Polynomials up to Degree $2n - 1$	180
8.2.8	Example: Gaussian Quadrature for $n = 2$	180
8.2.9	References	181
8.3	Richardson Extrapolation	181
8.3.1	Motivation and Basic Idea	181
8.3.2	Two-Level Extrapolation Procedure	181
8.3.3	General Framework	182
8.3.4	Why It Works	182
8.3.5	Summary	182
8.4	Romberg Integration	182
8.4.1	Algorithm	182
8.4.2	Example	182
8.5	Newton-Cotes Formulas and Gaussian Quadrature	183
8.5.1	Newton-Cotes Quadrature	184
8.5.2	Gaussian Quadrature	184
8.5.3	Gaussian Quadrature Beyond Legendre Polynomials	184
8.5.4	Summary of Legendre, Laguerre and Hermite Polynomials	185
8.6	Exercises	185

9	Initial Value Problems for Ordinary Differential Equations (ODEs)	189
9.1	Introduction	189
9.2	Euler's Method	189
9.2.1	Algorithm	189
9.2.2	Example	190
9.3	Runge-Kutta Methods	190
9.3.1	Fourth-Order Runge-Kutta Method (RK4)	191
9.3.2	A 2-dimensional example — the pendulum	193
9.4	Multistep Methods	195
9.4.1	Adams-Bashforth Method	195
9.4.2	Adams-Moulton Method	196
9.5	Stability and Convergence	196
9.5.1	Stability	196
9.5.2	Convergence	197
9.5.3	Example: Stability Analysis of Euler's Method	197
9.6	More on Runge-Kutta methods	197
9.6.1	Runge-Kutta 2nd Order Method (RK2)	197
9.6.2	Runge-Kutta 3rd Order Method (RK3)	197
9.6.3	Runge-Kutta 4th Order Method (RK4)	198
9.6.4	RK(F)45	198
9.6.5	How does RK45 work?	199
9.6.6	Summary	200
9.7	The Taylor Method	200
9.7.1	Taylor Series Expansion	200
9.7.2	Deriving the 3rd-Order Taylor Method	201
9.7.3	3rd-Order Taylor Method Formula	201
9.7.4	Calculating Higher Order Derivatives	201
9.7.5	Conclusion	201
9.7.6	A higher-order calculation	201
9.7.7	Derivation of Runge-Kutta of order 2	202
9.7.8	A proof that RK2 is not order 3	202
9.8	Example: Solving an IVP using the Second-Order Taylor Method	203
9.9	The General Form of Explicit Runge-Kutta Methods	204
9.10	Equations of Motion for a Pendulum with an Elastic Rod	205
9.10.1	System Description	205
9.10.2	Kinetic Energy	205
9.10.3	Potential Energy	205
9.10.4	Lagrangian and Equations of Motion	205
9.10.5	Final Equations of Motion	206
9.10.6	Solving an IVP — vectorized version	206
9.10.7	References	208
9.11	The Lorenz System and Its Poincaré Section	208
9.11.1	Trajectory of the Lorenz System	209
9.11.2	Computing the Poincaré Section	210
9.11.3	Simulation and Results	211
9.12	Calculating Lyapunov Exponents	212
9.12.1	Introduction	212
9.12.2	First Variation Equations Method	213
9.12.3	Poincaré Section Method	214
9.12.4	Direct Method: Trajectory Separation	214
9.12.5	Example: The Lyapunov Exponent of the Lorenz System	214
9.12.6	Computing the Derivative of the Poincaré Map	216

9.12.7	Conclusion	217
9.13	Step Size Restriction for Convergence of RK4 on a Stiff System	217
9.14	Example: A Stiff 2D System with Non-Linearity	220
9.15	Exercises	228
10	Boundary Value Problems for ODEs	231
10.1	Introduction	231
10.2	Shooting Method	231
10.2.1	Algorithm	231
10.2.2	Example	231
10.3	Finite Difference Method	231
10.3.1	Algorithm	231
10.3.2	Example	233
10.4	Exercises	233
11	Control Theory	235
11.1	Introduction	235
11.2	Pontryagin Maximum Principle	235
11.3	On the Mechanics of Applying PMP	236
11.4	Optimal Control of a Swinging Pendulum	239
11.5	Optimal Synthesis and Phase Diagrams for a Nonlinear Pendulum	240
11.6	Optimal Control with Bounded Torque	244
11.7	Linear-Quadratic Regulator (LQR)	245
11.7.1	Solution of LQR in 1D by using PMP	245
11.8	The Direct method	247
11.9	Additional Control Theory Background	251
11.10	Symplecticity	254
11.11	Pendulum on a Cart	256
11.12	Inverted Pendulum on a Cart: Controller Design	257
11.13	Choosing Q and R in LQR Control	258
11.14	Hamilton-Jacobi-Bellman Equations	259
11.15	Solving the Pendulum Control Problem	260
11.16	Energy-Realistic Control of a Swinging Pendulum	260
11.17	Derivation of the Hamilton-Jacobi-Bellman (HJB) Equation	261
11.18	Derivation of Pontryagin Maximum Principle	263
11.19	Derivation of PMP via regularization	264
11.20	Optimal Control with Saturated Input: Scalar Linear System	266
11.21	Controlling Mathematical Pendulum	268
11.22	Exercises	272
11.22.1	Linear-Quadratic Regulator (LQR)	272
11.22.2	Pontryagin Maximum Principle (PMP)	273
11.22.3	Hamilton-Jacobi-Bellman (HJB) Equation	274
12	Partial Differential Equations (PDEs)	275
12.1	Finite Difference Method	275
12.1.1	Derivation of Finite Differences	275
12.1.2	Application to PDEs	275
12.2	Image Restoration Using Total Variation	277
12.2.1	Discretization Model for Real Images	277
12.2.2	Euler-Lagrange PDE for Regularized ROF Model	278
12.2.3	A constant-free version of the problem	278
12.2.4	1D Image Restoration with Squared Gradient Norm (Tikhonov Regularization)	279

12.2.5	The difference scheme	279
12.2.6	Numerical Implementation	279
12.2.7	Python Code for Image Restoration	280
12.2.8	Smoothing, Denoising and the Total Variation Functional	281
12.2.9	A Discrete Variant of TV denoising	282
12.3	Finite Element Method (FEM)	283
12.3.1	Basic Concept	283
12.3.2	Weak Formulation	283
12.4	Finite Volume Method (FVM)	284
12.4.1	Conservation Law Formulation	284
12.4.2	Discretization	284
12.5	Solving Burgers' Equation: Methods and Sample Code	284
12.6	Multigrid Methods	288
12.6.1	Basic Idea	288
12.6.2	Multigrid Cycle	289
12.6.3	Applications and Advantages	289
12.7	Exercises	289
13	Numerical Optimization	291
13.1	Introduction	291
13.2	Constrained Optimization	291
13.2.1	Linear Programming	291
13.2.2	Nonlinear Programming	292
13.3	Exercises	293
14	Numerical Methods in HPC	295
14.1	Parallel Computing Fundamentals	295
14.1.1	Parallel Computing Models	295
14.1.2	Parallel Programming Basics	295
14.2	Programming for HPC	295
14.2.1	OpenMP	295
14.2.2	MPI (Message Passing Interface)	296
14.3	GPU Programming	296
14.3.1	CUDA	296
14.3.2	OpenCL	297
14.4	Parallel Numerical Algorithms	298
14.4.1	Parallel Root-Finding Algorithms	298
14.4.2	Parallel Matrix Operations	298
14.4.3	Parallel ODE/PDE Solvers	299
14.5	Exercises	299
15	Monte Carlo Methods	301
15.1	Random Sampling	301
15.1.1	Monte Carlo Integration	301
15.1.2	Solving Problems Involving Randomness	302
15.2	Markov Chain Monte Carlo (MCMC)	302
15.2.1	Introduction to Markov Chains	302
15.3	Reversibility and Detailed Balance	304
15.3.1	Reversibility	304
15.3.2	Detailed Balance	304
15.3.3	Example: Simple Random Walk on a Graph	305
15.3.4	Applications in MCMC	305

15.4	Global Minimization Using Metropolis-Hastings	309
15.5	Exercises	311
15.6	Gibbs Sampling	312
15.6.1	Algorithm	312
15.6.2	Example	312
15.7	Importance Sampling	314
15.7.1	Motivation	314
15.7.2	Basic Idea	315
15.7.3	How is the variance reduced?	315
15.7.4	Choosing the Importance Distribution	315
15.7.5	Variance Reduction	315
15.7.6	Example	316
15.7.7	Applications	318
15.8	Exercises	318
16	Advanced Topics in HPC	321
16.1	Sparse Matrix Techniques	321
16.1.1	Storage Formats	321
16.1.2	Sparse Matrix-Vector Multiplication	321
16.2	Advanced Eigenvalue Problems	321
16.2.1	Arnoldi Method	322
16.2.2	Lanczos Method	322
16.3	Preconditioning Techniques	322
16.3.1	Jacobi Preconditioner	322
16.3.2	Incomplete LU (ILU) Preconditioner	322
16.4	Monte Carlo and Stochastic Methods	322
16.4.1	Monte Carlo Integration	322
16.4.2	Stochastic Differential Equations	323
16.5	Big Data Techniques	323
16.5.1	MapReduce	323
16.5.2	Apache Spark	323
16.6	Deep Learning on GPUs	323
16.6.1	CUDA for Deep Learning	323
16.6.2	TensorFlow and PyTorch	323
16.7	Emerging Trends and Research Topics in HPC	323
16.7.1	Quantum Computing	323
16.7.2	Exascale Computing	324
16.8	Exercises	324
17	Case Studies and Applications	325
17.1	Scientific Computing Applications	325
17.1.1	Case Study: Simulating Molecular Dynamics	325
17.1.2	Case Study: Climate Modeling	326
17.2	Optimization in Machine Learning	326
17.2.1	Case Study: Gradient Descent in Deep Learning	326
17.2.2	Case Study: Hyperparameter Tuning for Support Vector Machines	327
17.3	Real-World Problem Solving	327
17.3.1	Case Study: Optimizing Supply Chain Management	327
17.3.2	Case Study: Data-Driven Healthcare Solutions	328
17.3.3	Case Study: Financial Portfolio Optimization	328

18 Practical Considerations and Software	331
18.1 Programming Languages and Tools	331
18.1.1 Python	331
18.1.2 C/C++	331
18.1.3 MATLAB	332
18.2 Numerical Libraries	332
18.2.1 NumPy and SciPy	332
18.2.2 BLAS and LAPACK	333
18.2.3 TensorFlow and PyTorch	333
18.3 Optimization Techniques for HPC	334
18.3.1 Parallel Computing	334
18.3.2 Vectorization	335
18.3.3 Memory Optimization	335
19 Final Project Guidelines	337
19.1 Project Selection	337
19.1.1 Identify Your Interests	337
19.1.2 Literature Review	337
19.1.3 Define Clear Objectives	337
19.1.4 Feasibility Assessment	337
19.1.5 Consultation with Advisors	337
19.1.6 Project Proposal	338
19.2 Implementation and Documentation	338
19.2.1 Development Environment	338
19.2.2 Modular Development	338
19.2.3 Testing and Validation	338
19.2.4 Documentation	338
19.2.5 Ethical Considerations	338
19.3 Presentation and Evaluation	339
19.3.1 Preparation	339
19.3.2 Visual Aids	339
19.3.3 Rehearsal	339
19.3.4 Delivery	339
19.3.5 Evaluation Criteria	339
19.3.6 Feedback	339
A Leveraging AI Tools for Studying Numerical Methods	341
A.1 Getting Assistance with Coding	341
A.1.1 Explaining Concepts	341
A.1.2 Debugging Code	341
A.1.3 Code Examples and Templates	342
A.2 Locating Resources	342
A.2.1 Finding Tutorials and Documentation	342
A.2.2 Research Papers and Articles	342
A.3 Advice on Programming Languages and Tools	342
A.3.1 Selecting a Programming Language	342
A.3.2 Exploring Software Tools and Libraries	343
A.4 Ethical Considerations and Best Practices	343
A.4.1 Avoiding Plagiarism	343
A.4.2 Validating Information	343
A.4.3 Continuous Learning	343
A.4.4 Data Privacy	343

A.5	Case Studies: Using ChatGPT for Numerical Methods	343
A.5.1	Case Study: Solving Differential Equations	343
A.5.2	Case Study: Matrix Operations	344
A.6	Conclusion	344
B	IEEE 754 Floating-Point Standard: A Detailed Overview	345
C	Reading GPS Data in Python	349
C.1	Connect the GPS Module	349
C.2	Install Required Libraries	349
C.2.1	Read and Parse GPS Data	349
D	Python Virtual Environments	351
D.1	Introduction	351
D.2	Rationale for Using Virtual Environments	351
D.3	Creating and Using Virtual Environments	351
D.3.1	Using venv	351
D.3.2	Using virtualenv	352
D.4	Example: Setting Up a Virtual Environment for Numerical Methods	352
D.5	Conclusion	352
D.6	Further Reading	353
E	The Rosenbrock Function and Its Computational Graph	355
F	An automatic differentiation module in Python	357
G	QR Decomposition using Maxima	361
H	Horner’s Scheme and Synthetic division	363
I	The Modified Power Method on the Grassmannian Manifold	365
I.1	The Algorithm	365
I.2	Geometric Interpretation	365
I.3	A Proof of Convergence	366
I.4	Flag manifolds and QR algorithm	368
I.5	Conclusion	369
I.6	Exercises	369
J	Spectral Decomposition and Resolvent of a Matrix	371
K	Theory of Support Vector Machines	375
L	Lagrange Multipliers	381
M	Envelope Theorem (Milgrom–Segal formulation)	391
M.1	Setup	391
M.2	The Milgrom–Segal envelope theorem	391
M.3	Interpretation	392
M.4	Multidimensional parameter version	392
M.4.1	Regularity assumptions	392
M.4.2	Directional envelope theorem	392
M.4.3	Gradient form	393
M.5	Notes and related references	393

N	Constrained Minimization with Inequality Constraints	395
N.1	Problem formulation	395
N.2	Convex and nonconvex settings	395
N.2.1	Convex problems	395
N.2.2	Nonconvex problems	396
N.3	Lagrangian and KKT conditions	396
N.4	Box constraints and componentwise optimality	396
N.5	Active-set methods	396
N.6	Interior-point and barrier methods	397
N.7	Projected gradient methods	397
N.8	Worked two-dimensional example	398
N.8.1	Karush–Kuhn–Tucker conditions	398
N.9	Coordinate descent and quasi-Newton methods	399
N.10	Sequential quadratic programming	399
N.11	Duality and primal–dual methods	399
N.12	A cone-separation view	399
N.12.1	The implication and its conic certificate	399
N.12.2	Gordan’s theorem (theorem of the alternative)	399
N.12.3	Farkas’ lemma (classical cone form)	400
N.12.4	Summary	401
N.13	Summary	401
O	Markowitz Portfolio Theory: Data Preparation and Implementation	403
P	Differential Geometry and Gradient	407
P.1	Exercises	408
Q	Green’s Identities	409
R	The Error Bound in Simpson’s Rule	411
S	Symmetry Reduction of Burgers’ Equation via Jet Coordinates	415
S.0.1	Formal Definitions: Prolongation and Projection	415
S.0.2	Galilean Symmetry Reduction of Burgers’ Equation	420
T	An Analysis of Kahan’s Algorithm	425
U	Conditional Expectation and Related Concepts	427
U.1	What Can We Condition On?	427
U.2	Conditional Expectation with Respect to a σ -Algebra	427
U.3	Finite Partition Case	428
U.4	General Case via Approximation	428
U.5	Conditioning on a Random Variable	428
U.6	Density-Based Formulas	428
U.7	Conditional Probability as Conditional Expectation	429
U.8	Key Properties	429
U.9	Hilbert Space Viewpoint (Optional but Insightful)	429
U.10	Connection to Markov Processes	429
U.11	Path Space and Shift Map	430
U.12	Closing Remark	430
U.13	Worked Examples	430
U.14	Exercises	432

V	Answers and Hints For Selected Exercises	433
	List of Algorithms	448

Prologue

Like every long journey, a book project starts with the first step. The first paragraphs should explain where we are going with our exposition of the subject. This book is about algorithms, and the first algorithm we discuss is Automatic Differentiation (AD). It is not an algorithm to be found in traditional numerical analysis books, which focus on influential algorithms of the past, such as the Simplex Method and the Fast Fourier Transform (FFT). AD is not only the object of our studies, but is behind the AI tools used in the creation of this book. The implementation of AD in Python, included in the next section, is a product of interaction with ChatGPT 4 and 4o, the most popular AI tools of the day [16].

The Role of Automatic Differentiation in Numerical Methods

The dawn of the machine learning era has brought about profound advancements in artificial intelligence, making systems like large language models (LLMs) possible. Central to these achievements is the ability to perform automatic differentiation (AD) efficiently. AD, a method for evaluating the derivative of a function specified by a computer program, is pivotal not only in machine learning but also in many numerical methods encountered in applied mathematics.

Consider a graduate student embarking on a journey through numerical methods. The ability to differentiate a function programmatically, without manual intervention, opens doors to solving complex optimization problems, sensitivity analysis, and even training deep neural networks. AD, particularly in the context of machine learning, allows for the backpropagation algorithm to train models by efficiently computing gradients, which are essential for adjusting model parameters.

Computational Graphs and the Stack

The essence of AD can be captured using the concept of a **computational graph**. Each operation in a function corresponds to a node in this graph, with edges representing the flow of data. The forward pass computes the function's output, while the backward pass, leveraging the chain rule, computes the derivative. This process is critical in both forward and reverse mode AD, the latter being particularly powerful for functions with many inputs and few outputs, such as in deep learning.

The computational graph is often represented as a sequence of operations that are executed in a particular order. During function calls, the **stack** plays a crucial role. The stack is a data structure used to store information about active subroutines or functions within a program. When a function is called, a new **frame** (a contiguous range of memory) is created on the stack, which contains the function's parameters, local variables, and return address. As functions are called and return, these frames are pushed onto and popped off the stack, respectively.

In the context of AD, particularly the forward mode, the stack is used to store intermediate variables, sometimes referred to as 'u'-variables, such as u_{-1} , u_0 , u_1 . These variables represent the values and derivatives of expressions as they are computed and are essential for backpropagation or computing derivatives efficiently.

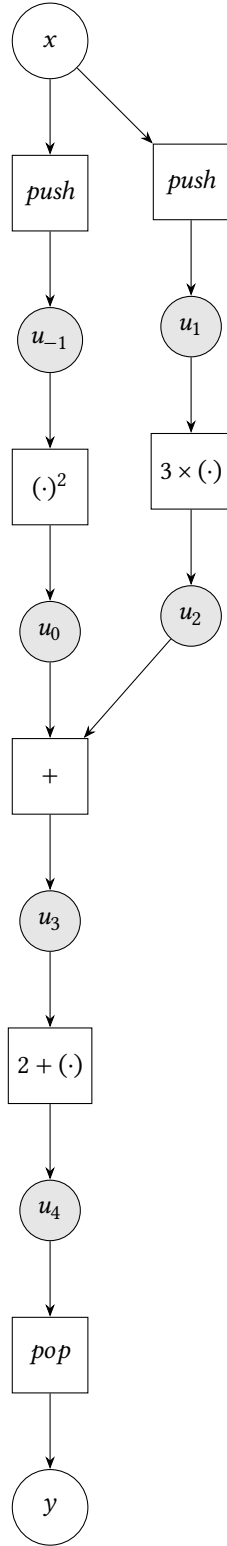


Figure 1: The computational graph for the expression $y = x^2 + 3x + 2$ (`y = x**2 + 3*x + 2` in Python). The round nodes represent memory locations. The square nodes represent operations.

Operator Overloading in Python

Operator overloading in Python allows custom classes to define the behavior of operators like '+', '-', '**', and '/'. This feature is a form of *syntactic sugar* that makes code involving custom types more readable and intuitive. For example, by overloading the '+' operator in a class, instances of that class can be added together using the '+' symbol, just like built-in types.

In Python, operator overloading is implemented by defining special methods within a class. For instance, the '__add__' method is used to overload the '+' operator, '__mul__' for '*', and so on. Additionally, Python provides the ability to overload operators for reflected operations—where the left operand does not support the operation—using methods like '__radd__' and '__rmul__'.

In Listing 1 there is an example in Python, illustrating how operator overloading facilitates automatic differentiation.

In this code, the 'Variable' class represents a scalar value and its derivative. The operations '+', '**', and '***' are overloaded so that when these operators are used in expressions involving 'Variable' objects, they automatically compute the resulting value and derivative. This approach effectively turns the code into a symbolic differentiation engine.

During execution, the stack is employed to store intermediate variables (referred to as 'u'-variables, such as u_{-1} , u_0 , u_1), which are necessary for backpropagation or computing derivatives in the forward mode. This is where the computational graph becomes invaluable: each node in the graph corresponds to an operation, and by traversing this graph, derivatives can be computed efficiently.

Operator overloading not only enhances the readability of the code but also abstracts away the complexities involved in derivative computation. This retrospection, or automatic "looking back" over the computation to compute derivatives, is what enables powerful tools like MATLAB's automatic differentiation capabilities, as detailed in their official documentation [13].

Conclusion

In conclusion, the ability to programmatically differentiate functions is a cornerstone of modern numerical methods and machine learning. As you delve deeper into the world of applied mathematics, understanding and utilizing tools like AD will be essential in tackling the challenges ahead. With operator overloading and computational graphs, you gain not just a method but a paradigm that bridges the gap between symbolic mathematics and numerical computation.

An exercise

When you evaluate a derivative using the chain rule, additional stack space is used during the calls to functions that perform the chain rule. Find the computational graph including those extra variables. How big is the resulting computational tree, in terms of the node and operation counts? Estimate the excess computational cost of automatic differentiation vs. the cost of evaluating the function, by counting the extra operations and memory used.

HINT: A function (or a method) is supposed to clean up after itself, that is, upon return, the stack shrinks to the state upon the entry. Thus, the effects of the extra stack space needed during derivative evaluation go away when a method completes. This observation makes the analysis of computational cost straightforward.

```

1 import numpy as np
2
3 class Variable:
4     def __init__(self, value, derivative=1.0):
5         self.value = value
6         self.derivative = derivative
7
8     def __add__(self, other):
9         if isinstance(other, Variable):
10             return Variable(self.value + other.value, self.derivative + other.derivative)
11         else:
12             return Variable(self.value + other, self.derivative)
13
14     def __radd__(self, other):
15         return self.__add__(other)
16
17     def __mul__(self, other):
18         if isinstance(other, Variable):
19             return Variable(self.value * other.value, self.value * other.derivative + self.derivative * other.
20 value)
21         else:
22             return Variable(self.value * other, self.derivative * other)
23
24     def __rmul__(self, other):
25         return self.__mul__(other)
26
27     def __truediv__(self, other):
28         if isinstance(other, Variable):
29             return Variable(self.value / other.value,
30 (self.derivative * other.value - self.value * other.derivative) / (other.value ** 2)
31 )
32         else:
33             return Variable(self.value / other, self.derivative / other)
34
35     def __rtruediv__(self, other):
36         return Variable(other / self.value, -other * self.derivative / (self.value ** 2))
37
38     def __pow__(self, power):
39         return Variable(self.value ** power, power * self.value ** (power - 1) * self.derivative)
40
41     def sin(self):
42         return Variable(np.sin(self.value), np.cos(self.value) * self.derivative)
43
44     def cos(self):
45         return Variable(np.cos(self.value), -np.sin(self.value) * self.derivative)
46
47     def exp(self):
48         return Variable(np.exp(self.value), np.exp(self.value) * self.derivative)
49
50 # Example usage:
51 x = Variable(2.0)
52 y = x**2 + 3*x + 2
53
54 print(f"Value of expression: {y.value}")
55 print(f"Derivative of expression: {y.derivative}")

```

Listing 1: Automatic Differentiation in Python

Chapter 1

Introduction to Numerical Methods and HPC

1.1 Motivation and Applications

Numerical methods are a cornerstone of applied mathematics, providing powerful tools to solve complex problems that arise in science, engineering, and other fields. These methods allow us to approximate solutions to equations that cannot be solved analytically, optimize functions, and model physical phenomena with high precision.

In recent years, the advent of high-performance computing (HPC) has revolutionized the field of numerical methods. HPC leverages advanced computational resources, such as multi-core processors, graphics processing units (GPUs), and distributed computing systems, to perform large-scale simulations and data analysis tasks that were previously infeasible.

This book aims to equip readers with a comprehensive understanding of numerical methods and HPC, bridging the gap between theoretical concepts and practical implementation.

1.1.1 Real-World Applications

Numerical methods and HPC are used in a wide range of applications, including:

- **Engineering:** Finite element analysis, computational fluid dynamics, and structural optimization.
- **Physics:** Simulation of physical systems, solving differential equations, and modeling particle interactions.
- **Biology:** Bioinformatics, population modeling, and systems biology.
- **Finance:** Option pricing, risk management, and portfolio optimization.
- **Climate Science:** Weather prediction, climate modeling, and environmental monitoring.

1.2 Overview of High-Performance Computing (HPC)

High-performance computing involves the use of supercomputers and parallel processing, techniques to solve complex computational problems. HPC systems are designed to perform a large number of calculations at high speed, making them ideal for tasks that require significant computational power.

1.2.1 HPC Hardware Architectures

HPC systems are built using various hardware architectures, including:

- **Multi-Core Processors:** CPUs with multiple processing cores that can execute multiple threads simultaneously.

- **Graphics Processing Units (GPUs):** Specialized hardware designed for parallel processing, widely used in scientific computing and machine learning.
- **Distributed Computing Systems:** Clusters of interconnected computers that work together to perform parallel computations.

1.2.2 Parallel Computing Models

Parallel computing involves dividing a problem into smaller tasks that can be solved concurrently. The main parallel computing models include:

- **Shared Memory Model:** Multiple processors access a common memory space. Examples include OpenMP and pthreads.
- **Distributed memory Model:** Each processor has its own local memory, and processors communicate by passing messages. Examples include MPI (Message Passing Interface).

1.2.3 Programming for HPC

Programming for HPC requires specialized tools and libraries to efficiently utilize the available computational resources. Key programming models and languages for HPC include:

- **OpenMP:** An API for multi-platform shared memory parallel programming in C, C++, and Fortran.
- **MPI (Message Passing Interface):** A standardized and portable message-passing system designed for distributed computing.
- **CUDA:** A parallel computing platform and programming model developed by NVIDIA for general computing on GPUs.
- **OpenCL:** An open standard for parallel programming of heterogeneous systems, allowing code to run on CPUs, GPUs, and other processors.

1.2.4 Importance of HPC in Numerical Methods

The integration of HPC with numerical methods has opened up new possibilities for solving large-scale and high-dimensional problems. It allows researchers and engineers to:

- Perform large-scale simulations with high accuracy and resolution.
- Analyze and visualize massive datasets.
- Develop more sophisticated models and algorithms.
- Reduce computation time for complex problems.

1.3 Structure of the Book

This book is structured to provide a comprehensive understanding of both numerical methods and high-performance computing. The chapters are organized as follows:

- **Chapter 1: Introduction to Numerical Methods and HPC:** Overview and motivation for studying numerical methods and HPC.
- **Chapter 2: Error Analysis:** Types of errors, error propagation, and stability.
- **Chapter 3: Root-Finding Algorithms:** Techniques for finding roots of equations.
- **Chapter 4: Numerical Linear Algebra:** Solving linear systems, eigenvalue problems, and matrix factorizations.
- **Chapter 5: Multi-Dimensional Root Finding:** Newton-type methods and related algorithms for systems of nonlinear equations.
- **Chapter 6: Unconstrained and Constrained Optimization:** Finding minima and maxima of functions with and without constraints.

- **Chapter 7: Interpolation and Polynomial Approximation:** Methods for fitting and approximating functions.
- **Chapter 8: Numerical Integration:** Techniques for numerical integration.
- **Chapter 9: Initial Value Problems for Ordinary Differential Equations (ODEs):** Solving ordinary differential equations using numerical methods.
- **Chapter 10: Boundary Value Problems for ODEs:** Methods for solving boundary value problems.
- **Chapter 11: Control Theory:** Pontryagin Maximum Principle, Linear Quadratic Regulator problems, and related computational methods.
- **Chapter 12: Partial Differential Equations (PDEs):** Numerical methods for solving PDEs.
- **Chapter 13: Numerical Optimization:** Additional techniques for optimizing functions.
- **Chapter 14: Numerical Methods in HPC:** Applying numerical methods in high-performance computing environments.
- **Chapter 15: Monte Carlo Methods:** Random sampling methods for integration, simulation, and probabilistic computation.
- **Chapter 16: Advanced Topics in HPC:** Advanced HPC topics and research trends.
- **Chapter 17: Case Studies and Applications:** Real-world applications of numerical methods and HPC.
- **Chapter 18: Practical Considerations and Software:** Tools, libraries, and best practices for numerical computing.
- **Chapter 19: Final Project Guidelines:** Guidelines for selecting, implementing, and presenting a final project.

Supplemental chapters and appendices are organized as follows:

- **Chapter A: Leveraging AI Tools for Studying Numerical Methods:** Using AI tools while studying numerical methods and programming.
- **Chapter B: IEEE 754 Floating-Point Standard: A Detailed Overview:** A detailed overview of the IEEE 754 floating-point standard.
- **Chapter C: Reading GPS Data in Python:** Python code for reading data from a GPS sensor.
- **Chapter D: Python Virtual Environments:** Python virtual environments for managing module versions and reproducible examples.
- **Chapter E: The Rosenbrock Function and Its Computational Graph:** Computational graphs through the Rosenbrock function.
- **Chapter F: An automatic differentiation module in Python:** A compact Python implementation of automatic differentiation.
- **Chapter G: QR Decomposition using Maxima:** Exact QR decomposition using symbolic computation in Maxima.
- **Chapter H: Horner's Scheme and Synthetic division:** Classical algorithms for evaluating and dividing polynomials.
- **Chapter I: The Modified Power Method on the Grassmannian Manifold:** A power method variant for finding multiple eigenvalues and eigenvectors.
- **Chapter J: Spectral Decomposition and Resolvent of a Matrix:** Spectral decomposition and resolvents in relation to shifted inverse iteration.
- **Chapter K: Theory of Support Vector Machines:** The quadratic programming problem behind support vector machines.
- **Chapter L: Lagrange Multipliers:** Theory of Lagrange multipliers, including second derivative tests.
- **Chapter M: Envelope Theorem (Milgrom–Segal formulation):** The Milgrom–Segal envelope theorem for arbitrary choice sets.
- **Chapter N: Constrained Minimization with Inequality Constraints:** Inequality-constrained optimization and optimality conditions.
- **Chapter O: Markowitz Portfolio Theory: Data Preparation and Implementation:** Preparing financial data for Markowitz portfolio optimization.
- **Chapter P: Differential Geometry and Gradient:** Gradients and metrics from a differential-geometric viewpoint.

- **Chapter Q: Green’s Identities:** Green identities and related variational calculations.
- **Chapter R: The Error Bound in Simpson’s Rule:** A proof of the classical error bound in Simpson’s rule.
- **Chapter S: Symmetry Reduction of Burgers’ Equation via Jet Coordinates:** Symmetry reduction of Burgers’ equation using jet coordinates.
- **Chapter T: An Analysis of Kahan’s Algorithm:** Theory and analysis of Kahan summation.
- **Chapter U: Conditional Expectation and Related Concepts:** Conditional expectation, conditional probability, and related probability concepts.

By the end of this book, readers will have a solid foundation in numerical methods and high-performance computing, enabling them to tackle complex computational problems in various scientific and engineering disciplines.

The book has a number of exercises. Some of them are marked with the “dangerous bend” sign invented by Donald Knuth in his book on $\text{\TeX}$, like this:

Exercise 1.1. This exercises will challenge you more than an average exercise in the book, and may require extra time. $\square$



Or exercises bordering on a research project may be marked as this:

Exercise 1.2. This exercises will take genius to solve and may result in a research paper or a dissertation. $\square$



Chapter 2

Error Analysis

2.1 Computer Arithmetic

Computer arithmetic is the foundation of numerical computing. It involves the representation and manipulation of numbers in a computer system. Understanding computer arithmetic is crucial for developing efficient and accurate numerical algorithms.

2.1.1 Floating-Point Representation

Most numerical computations in computers use floating-point arithmetic. A floating-point number is represented as:

$$x = \pm m \times b^e$$

where:

- m is the significand (or mantissa)
- b is the base (or radix), typically 2 for binary systems
- e is the exponent

The IEEE 754 standard is widely used for floating-point arithmetic. It defines formats for representing floating-point numbers, including single precision (32-bit) and double precision (64-bit). In Chapter B a brief account of the standard are provided.

2.1.2 Notation for integers in arbitrary base

Once we agree upon the base (radix) of a representation, integers admit a base b representation in terms of its base b digits:

$$n = \sum_{j=0}^k d_j \cdot b^j,$$
$$0 \leq d_j \leq b - 1.$$

The numbers d_j are the *digits* in base b . This representation is unique.

The abbreviated notation is:

$$n = (d_k d_{k-1} \dots d_0)_b.$$

Example 2.1 (Equivalence of binary and decimal representations). The following identity amounts to expressing 23 (base 10 implied) in base 2 notation.

$$(10111)_2 = (23)_{10}.$$

To convert $(10111)_2$ to decimal notation we evaluate

$$1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 = 23$$

In this calculation, we use the familiar rules of arithmetic in base 10. To convert 23 to binary, we may repeatedly divide by 2 and note the remainders, which are the binary digits:

$$23 = 2 \cdot 11 + \mathbf{1},$$

$$11 = 2 \cdot 5 + \mathbf{1},$$

$$5 = 2 \cdot 2 + \mathbf{1},$$

$$2 = 2 \cdot 1 + \mathbf{0},$$

$$1 = 2 \cdot 0 + \mathbf{1}.$$

We stop when the quotient is 0. We note that the binary digits of $23 = (23)_{10}$ are obtained in reverse order, the least significant digit first. $\square$

Exercise 2.1. Write $n = 23$ in base 3. $\square$

Commonly used representations are: octal (base 8) and hexadecimal (base 16). For these two bases, special notations were developed:

Example 2.2 (Octal representation of 23). We have, after padding with a leading 0:

$$(23)_{10} = (010\ 111)_2 = \backslash o27.$$

The digits of the octal notation are 0, 1, 2, ..., 7. They can be easily obtained from the binary representation by considering groups of 3 binary digits, padded with leading 0's if necessary:

$$(111)_2 = 7, \quad (010)_2 = 2.$$

$\square$

Example 2.3 (Hexadecimal representation of 23). We have:

$$(23)_{10} = (0001\ 0111)_2 = \backslash x17.$$

The digits of the hexadecimal notation are 0, 1, 2, ..., 7, 8, 9, A, B, C, D, E, F. Because we run out the decimal digits at 9, we use the initial letters of the alphabet (upper or lower case) to represent the digits of the hexadecimal notation.

Hexadecimal digits are obtained from the binary representation we obtain considering groups of 4 binary digits, padded with leading 0's if necessary:

$$(0111)_2 = 7, \quad (0001)_2 = 1.$$

We also have these examples:

$$(10)_{10} = \backslash xA,$$

$$(15)_{10} = \backslash xF,$$

$$(127)_{10} = (0111\ 1111)_2 = \backslash x7F,$$

$$(255)_{10} = (1111\ 1111)_2 = \backslash xFF.$$

$\square$

Example 2.4 (Python in hexadecimal (“hex”) notation). Keyboard letters and other symbols are mapped to *bytes*, which are digits of the base 256 numeral system. However, since 256 symbols for digits from 0 to 255 are not practical, a byte is divided into two hex digits called *nibbles*, each representing 4 bits (half a byte).

The oldest assignment of numbers to keyboard symbols is called ASCII, and the number assigned is called the *ASCII code*. For example, the character ‘A’ is assigned the ASCII code 65_{10} , so:

$$'A' \mapsto (65)_{10}.$$

We can represent this in binary, octal, and hexadecimal as:

$$(65)_{10} = (0100\ 0001)_2 = \backslash o101 = \backslash x41.$$

The string “Python” as a byte sequence is:

$$\backslash x50\ \backslash x79\ \backslash x74\ \backslash x68\ \backslash x6f\ \backslash x6e.$$

In some applications, such as machine learning, it may be beneficial to consider the above expression as an integer written in base 256. On a 64-bit computer, 8-byte integers are standard, and our integer has only 6 bytes. The integer representing the string is (in decimal):

$$\begin{aligned} & (5 \cdot 16 + 0) \cdot 256^5 + (7 \cdot 16 + 9) \cdot 256^4 \\ & + (7 \cdot 16 + 4) \cdot 256^3 + (6 \cdot 16 + 8) \cdot 256^2 \\ & + (6 \cdot 16 + 15) \cdot 256 + (6 \cdot 16 + 14) = 88461099429742. \end{aligned}$$

This value is correct if the byte sequence is in network (“big-endian”) order, where the most significant byte (MSB) comes first. The alternative order, called “little-endian” (LSB first), is used by Intel processors, while RISC processors tend to use network order. Endianness affects how sequences of bytes are interpreted when grouped into larger data types like 32-bit or 64-bit integers.

The term “network order” originates from the standardization of big-endian byte order for internet data transmission. $\square$

2.1.3 How to access the bits of a IEEE 754 float in Python?

The example below does this. This code is somewhat “magical”, as it is not very natural to operate on raw memory in Python. The exact explanation can be found in the documentation of the Python package `struct`, and, of course, by reading the source code.¹ Underneath, there is code written in C (very good at this task).

```

1  import struct
2
3  def binary(num):
4      # Pack the float into 4 bytes (32 bits)
5      packed = struct.pack('!f', num)
6      # Convert each byte to its binary representation and join them
7      return ''.join('{:0>8b}'.format(c) for c in packed)
8
9  def split_ieee754(num):
10     # Get the full 32-bit binary representation
11     bits = binary(num)
12     # Split into sign, exponent, and mantissa
13     sign = bits[0]          # 1 bit
14     exponent = bits[1:9]    # 8 bits
15     mantissa = bits[9:]     # 23 bits
16     return sign, exponent, mantissa
17
18  def print_num_digits(x):
19     sign, exponent, mantissa = split_ieee754(x)
20     print('\begin{verbatim}')
21     print(f'x = {x}')
22     print('+' + '-'*7 + '+' + '-'*11 + '+' + '-'*26 + '+')
```

¹A once-popular saying in code wizard circles: “Luke, use the force: read the source!”

```

23 print(f'| Sign | Exponent | Mantissa |')
24 print('+ ' + '-'*7 + ' + ' + '-'*11 + ' + ' + '-'*26 + ' + ')
25 print(f'| {sign} | {exponent} | {mantissa} |')
26 print('+ ' + '-'*7 + ' + ' + '-'*11 + ' + ' + '-'*26 + ' + ')
27 print('\end{verbatim}')
28
29 # Example usage:
30 print_num_digits(3.14)

```

The printed result (when included in a $\LaTeX$ file):

```

x = 3.14
+-----+-----+-----+
| Sign | Exponent | Mantissa |
+-----+-----+-----+
| 0 | 10000000 | 10010001111010111000011 |
+-----+-----+-----+

```

The exponent is $2^7 = 128$ in decimal. This is because exponent is an 8-bit integer, ranging from 0 to $2^8 - 1 = 255$, but offset by $2^7 - 1 = 127$ in the negative direction, so that exponent 0 is represented by $2^7 - 1 = 127$. The number 127 is called the *bias*, and is chosen to be the middle of the exponent range.

Thus numbers in the range $[1, 2)$ have exponent 0, but the bit pattern representing the exponent is:

$$(01111111)_2 = (127)_{10}.$$

For Example 3.14/2 = 1.57 has the following representation:

```

x = 1.57
+-----+-----+-----+
| Sign | Exponent | Mantissa |
+-----+-----+-----+
| 0 | 01111111 | 10010001111010111000011 |
+-----+-----+-----+

```

The mantissa for 1.57 and 3.14 is exactly the same, but the exponents of 3.14 is by 1 higher. The exponent of 3.14 is 1, but after the offset is applied, it is $127 + 1 = 128$ which has the bit pattern:

$$(10000000)_2 = (128)_{10}.$$

2.1.4 Definition of Machine Numbers

Machine numbers are the set of numbers that can be exactly represented within a computer's floating-point system, such as those defined by the IEEE 754 standard.

For a number to qualify as a machine number, it must have an exact binary representation within the constraints of the floating-point format (e.g., single-precision, double-precision). This means that the number can be precisely stored and manipulated without any approximation.

Example 2.5 (Representation of 3.14). The decimal number 3.14 cannot be exactly represented in the binary floating-point format used by computers. The closest approximation in IEEE 754 single-precision floating-point representation will be a number very close to 3.14, but not exactly 3.14.

Therefore, 3.14 is not a machine number. The actual value stored in the floating-point system is a nearby approximation, not the exact value of 3.14. $\square$

It is important to note that this explanation pertains to the binary representation of numbers and is unrelated to the fact that 3.14 is a rounded approximation of π to two decimal places or three significant digits.

Exercise 2.2. Provide precise criteria for determining whether a real number (in the mathematical sense) can be considered a machine number. Apply these criteria to evaluate whether 3.14 qualifies as a machine number. $\square$

Exercise 2.3. When you input 3.14 into Python, it is, by default, parsed as a *double-precision* floating-point number. To obtain a *single-precision* floating-point number, you can use the following code fragment:

```
1 import numpy as np
2 x = np.float32(3.14)
3 print(type(x))
```

Demonstrate that the discussion about 3.14 being a machine number or not holds regardless of whether it is represented as a 32-bit or 64-bit float. Assume that Python approximates non-machine numbers by their closest machine number approximation. $\square$

Exercise 2.4. Clearly, all machine numbers are rational numbers. Prove that if a rational number $\frac{m}{n}$, where m and n are coprime, is a machine number, then n must be a divisor of b^k for some non-negative integer k , where b is the base (radix) of the representation. This is a *necessary* condition. Can you derive a condition that is both necessary and sufficient? $\square$

2.1.5 Precision and Rounding

Precision refers to the number of digits used to represent a number. In floating-point arithmetic, precision is limited, leading to rounding errors. There are several rounding modes defined by IEEE 754:

- **Round to nearest:** Rounds to the nearest representable value.
- **Round toward zero:** Rounds toward zero (truncation).
- **Round toward positive infinity:** Rounds toward positive infinity.
- **Round toward negative infinity:** Rounds toward negative infinity.

Example 2.6 (Banker's Rounding). Banker's rounding, also known as round half to even, is a midpoint rounding scheme where numbers equidistant from the two nearest integers are rounded to the nearest even integer. This method is particularly useful in reducing cumulative rounding errors in repeated calculations.

In fact, Python 3 employs this rounding scheme in its built-in `round()` function. $\square$

Example 2.7 (MATLAB Rounding). MATLAB uses a rounding method that differs from Banker's rounding. Specifically, MATLAB's `round()` function rounds numbers that are exactly halfway between two integers away from zero. This means that numbers with a fractional part of exactly 0.5 are rounded to the nearest integer in the direction away from zero.

- `round(2.5)` returns 3
- `round(3.5)` returns 4
- `round(-2.5)` returns -3
- `round(-3.5)` returns -4

This differs from Python 3, which uses Banker's rounding, where numbers equidistant from the two nearest integers are rounded to the nearest even integer. $\square$

2.1.6 Machine Epsilon

Machine epsilon (ϵ_{mach}) is the smallest positive number that, when added to 1, yields a result different from 1 in the computer's arithmetic. It provides a measure of the precision of the floating-point representation.

For a given floating-point system, machine epsilon can be determined by:

$$\epsilon_{\text{mach}} = b^{1-p}$$

where b is the base and p is the precision (number of significant digits).

Example in C++

```
1  #include <iostream>
2  #include <cmath>
3
4  int main() {
5      float epsilon = 1.0f;
6      while ((1.0f + epsilon / 2.0f) != 1.0f) {
7          epsilon /= 2.0f;
8      }
9      std::cout << "Machine epsilon (float): " << epsilon << std::endl;
10
11     double epsilon_double = 1.0;
12     while ((1.0 + epsilon_double / 2.0) != 1.0) {
13         epsilon_double /= 2.0;
14     }
15     std::cout << "Machine epsilon (double): " << epsilon_double << std::endl;
16
17     return 0;
18 }
```

Listing 2.1: Calculating Machine Epsilon in C++

2.1.7 Machine Epsilon in Accuracy Analysis

Definition 1 (Machine epsilon). The *machine epsilon*, denoted $\varepsilon_{\text{mach}}$, is the smallest positive floating-point number such that

$$1 + \varepsilon_{\text{mach}} > 1.$$

□

In IEEE double precision arithmetic this value is approximately $2^{-53} \approx 1.11 \times 10^{-16}$. For IEEE single precision arithmetic it is $2^{-23} \approx 1.192 \times 10^{-7}$. It measures the relative precision of floating-point representation near 1, and sets a fundamental limit on how close two distinct floating-point numbers can be.

Error model. A standard model for rounding is

$$\text{fl}(a \circ b) = (a \circ b)(1 + \delta), \quad |\delta| \leq \varepsilon_{\text{mach}},$$

where $\circ$ denotes one of the basic arithmetic operations. Thus every elementary operation incurs a relative error bounded by machine epsilon.

Backward error viewpoint. Numerical analysis often interprets computed solutions as exact solutions to perturbed problems. If an algorithm is *backward stable*, the perturbation is proportional to $\varepsilon_{\text{mach}}$. Forward error bounds then involve the problem's condition number, which can amplify or damp these perturbations.

Error Analysis, Example 1: Summation Order

Consider the sum

$$S = (1 + 10^{-16}) - 1 = 10^{-16}.$$

Computing left-to-right,

$$s_1 = \text{fl}(1 + 10^{-16}) \approx 1, \quad s_2 = \text{fl}(s_1 - 1) = 0,$$

so the small term is lost. If grouped as $(1 - 1) + 10^{-16}$, the result is exactly 10^{-16} .

In general, for naive summation of n terms, which are assumed to be machine numbers,

$$|\hat{S} - S| \leq \gamma_{n-1} \sum_{i=1}^n |a_i|,$$

where

$$\gamma_n := \frac{n \varepsilon_{\text{mach}}}{1 - n \varepsilon_{\text{mach}}}.$$

Thus the relative error can grow on the order of $n\varepsilon_{\text{mach}}$. Compensated summation (e.g. Kahan's algorithm, Algorithm 2.1) reduces the error to $O(\varepsilon_{\text{mach}})$ independent of n . The algorithm has been a subject of extensive studies. The mathematical result and references to the literature are included in Chapter T.

Algorithm 2.1 Kahan's Algorithm

Require: A sequence $x_n, n = 1, 2, \dots, N$ of real numbers

Ensure: An accurate approximation to $\sum_{n=1}^N x_n$

- 1: Initialize the sum $S = 0$ and a compensation variable $c = 0$.
 - 2: **for** each element x_n in the sequence **do**
 - 3: Calculate the temporary sum $t = x_n - c$.
 - 4: Update the sum S by adding t .
 - 5: Update the compensation c as the difference between $(S - \text{previous } S)$ and t .
 - 6: **end for**{After all terms are added, S contains the accurate sum with reduced round-off error.}
-

Error Analysis, Example 2: Solving Linear Systems

Suppose we solve $Ax = b$ with a backward-stable algorithm. Then the computed solution $\hat{x}$ satisfies

$$(A + \Delta A)\hat{x} = b, \quad \frac{\|\Delta A\|}{\|A\|} = O(\varepsilon_{\text{mach}}).$$

The forward error is bounded by

$$\frac{\|\hat{x} - x\|}{\|x\|} \lesssim \kappa(A) \varepsilon_{\text{mach}}, \quad \kappa(A) = \|A\| \|A^{-1}\|.$$

Example 2.8 (Machine epsilon, condition number). Let

$$A = \begin{bmatrix} 1 & 0 \\ 0 & \varepsilon \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ \varepsilon \end{bmatrix}, \quad x = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Here $\kappa(A) = 1/\varepsilon$. If $\varepsilon = 10^{-8}$, the bound gives a forward error $\sim 10^{-8}$. If $\varepsilon = 10^{-16}$, then $\kappa(A) \sim 10^{16}$ and the forward error can be $O(1)$: no digits of accuracy can be guaranteed. This loss of accuracy stems from the problem's conditioning, not from algorithmic instability. $\square$

Remark 2.1 (Machine epsilon vs. unit roundoff). Let b denote the floating-point base and p the precision (number of base- b digits in the significand). Two related but distinct quantities are common in the literature:

- The *machine epsilon*, defined as the distance between 1 and the next larger representable floating-point number:

$$\varepsilon_{\text{mach}} = b^{1-p}.$$

In IEEE double precision ($b = 2, p = 53$), this gives $\varepsilon_{\text{mach}} = 2^{-52} \approx 2.22 \times 10^{-16}$.

- The *unit roundoff*, defined as the maximum relative error when rounding to nearest:

$$u = \frac{1}{2} b^{1-p}.$$

For IEEE double, $u = 2^{-53} \approx 1.11 \times 10^{-16}$.

Error analysis usually employs the unit roundoff u , because it directly bounds relative rounding error. Some texts, however, use the term “machine epsilon” to refer to u . It is important to check the author's convention. $\square$

²The bound $\prod_{i=1}^n (1 + \delta_i) = 1 + \theta_n$ with $|\delta_i| \leq u = \varepsilon_{\text{mach}}$ leads to $|\theta_n| \leq \gamma_n$. For small nu , $\gamma_n \approx nu$, so γ_n is a convenient shorthand for accumulated rounding error.

2.1.8 A Theorem on Accuracy of Naive Summation

We analyze the accuracy of numerically finding partial sums of an infinite series:

$$S_n = \sum_{i=1}^n a_i$$

We *do not* assume that a_n are machine numbers, but they are in the normal range, so that the standard model of rounding holds. Let $\hat{S}_n$ be the computed value of the sum S_n . That is,

$$\hat{S}_n = \text{fl}(\hat{S}_{n-1} + \text{fl}(a_n))$$

where fl is the customary model of floating point arithmetic:

1. Each mathematically exact summand is rounded to machine precision.
2. Exact arithmetic is performed on machine numbers.
3. The result of exact arithmetic is rounded to machine precision.

Theorem 2.1 (On accuracy of naive summation).

$$|\hat{S}_n - S_n| \leq \rho_n \sum_{i=1}^n |a_i|$$

where

$$\rho_n := (1 + \varepsilon_{mach})^n - 1.$$

Furthermore, $\rho_n \leq \gamma_n$, where γ_n is defined as follows:

$$\gamma_n := \frac{n\varepsilon_{mach}}{1 - n\varepsilon_{mach}}.$$

□

Proof. Let $\varepsilon = \varepsilon_{mach}$. The proof is by induction on n , starting with $n = 1$. For $n = 1$

$$S_1 = a_1, \quad \hat{S}_1 = \text{fl}(a_1) = (1 + \delta)a_1$$

where $|\delta| \leq \varepsilon$. Thus

$$|\hat{S}_1 - S_1| = |\delta||S_1| = |\delta||a_1| \leq \varepsilon|a_1| = \rho_1|S_1|$$

Let us assume that the statement is valid all n smaller than a fixed $n \geq 2$. We consider the recurrence relation:

$$\hat{S}_n = \text{fl}(\hat{S}_{n-1} + \text{fl}(a_n)).$$

It follows that, for $n \geq 2$:

$$\begin{aligned} \hat{S}_n - S_n &= \text{fl}(\hat{S}_{n-1} + \text{fl}(a_n)) - S_n \\ &= (1 + \delta_1)(\hat{S}_{n-1} + \text{fl}(a_n)) - S_{n-1} - a_n \\ &= (\hat{S}_{n-1} - S_{n-1}) + (\text{fl}(a_n) - a_n) + \delta_1 (\hat{S}_{n-1} + \text{fl}(a_n)). \end{aligned}$$

Using the induction hypothesis, and with some values δ_1, δ_2 such that $|\delta_j| \leq \varepsilon$ for $j = 1, 2$:

$$\begin{aligned}
|\hat{S}_n - S_n| &\leq \rho_{n-1} \sum_{i=1}^{n-1} |a_i| + \delta_2 |a_n| + \delta_1 \left((1 + \rho_{n-1}) \sum_{i=1}^{n-1} |a_i| + (1 + \delta_2) |a_n| \right) \\
&\leq \rho_{n-1} \sum_{i=1}^{n-1} |a_i| + \varepsilon |a_n| + \varepsilon \left((1 + \rho_{n-1}) \sum_{i=1}^{n-1} |a_i| + (1 + \varepsilon) |a_n| \right) \\
&= (\rho_{n-1} + \varepsilon(1 + \rho_{n-1})) \sum_{i=1}^{n-1} |a_i| + (\varepsilon + \varepsilon(1 + \varepsilon)) |a_n|. \\
&= ((1 + \varepsilon)\rho_{n-1} + \varepsilon) \sum_{i=1}^{n-1} |a_i| + ((1 + \varepsilon)^2 - 1) |a_n|.
\end{aligned}$$

Let us postulate that for every $n \geq 2$:

$$\rho_n \geq (1 + \varepsilon)\rho_{n-1} + \varepsilon, \quad (2.1a)$$

$$\rho_n \geq (1 + \varepsilon)^2 - 1. \quad (2.1b)$$

Under these assumptions,

$$|\hat{S}_n - S_n| \leq \rho_n \sum_{i=1}^{n-1} |a_i| + \rho_n |a_n| = \rho_n \sum_{i=1}^n |a_i|.$$

Thus, if the postulates are satisfied then the proof is complete. To satisfy the postulates, let us solve the recurrence relation:

$$\rho_n = \rho_{n-1} + \varepsilon(1 + \rho_{n-1})$$

or

$$\rho_n = (1 + \varepsilon)\rho_{n-1} + \varepsilon.$$

By the theory of linear recurrence relations, the solution is

$$\rho_n = (1 + \varepsilon)^n A + B$$

where A, B are undetermined coefficients. We can satisfy arbitrary 2 constraints: $\rho_1 = \varepsilon$ and $\rho_2 = (1 + \varepsilon)^2 - 1$. We obtain a system of equations for A and B :

$$\rho_1 = \varepsilon = (1 + \varepsilon)A + B, \quad \rho_2 = (1 + \varepsilon)^2 - 1 = (1 + \varepsilon)^2 A + B.$$

Eliminating B , we obtain

$$\varepsilon(1 + \varepsilon)A = (1 + \varepsilon)^2 - 1 - \varepsilon = \varepsilon(1 + \varepsilon).$$

which yields $A = 1$. Subsequently, $B = \varepsilon - (1 + \varepsilon)A = -1$. Hence

$$\rho_n = (1 + \varepsilon)^n - 1.$$

It is necessary to check that $\rho_n \geq (1 + \varepsilon)^2 - 1 = \rho_2$ for all $n \geq 2$. However, this is obvious because ρ_n increases with n , so $\rho_n \geq \rho_2$.

Finally, let us prove the inequality $\rho_n \leq \gamma_{n+1}$, which is tantamount to showing that for every $n \geq 1$:

$$(1 + \varepsilon)^n - 1 \leq \frac{n\varepsilon}{1 - n\varepsilon}.$$

We note the inequality $e^{-x} \geq 1 - x$ for $x \geq 0$. Thus

$$\begin{aligned} (1 + \varepsilon)^n - 1 &= \exp(n \log(1 + \varepsilon)) - 1 \leq \exp(n\varepsilon) - 1 \\ &= \frac{1}{\exp(-n\varepsilon)} - 1 \leq \frac{1}{1 - n\varepsilon} - 1 \\ &= \frac{n\varepsilon}{1 - n\varepsilon}. \end{aligned}$$

□

Remark 2.2 (On naive summation of machine numbers). The same argument can be repeated for the case where a_n are already machine numbers. The inner fl goes away, leading to

$$\hat{S}_n = \text{fl}(\hat{S}_{n-1} + a_n).$$

This recurrence relation leads to a slightly better estimate:

$$|\hat{S}_n - S_n| \leq \rho_{n-1} \sum_{i=1}^n |a_i| \leq \gamma_{n-1} \sum_{i=1}^n |a_i|.$$

There is no roundoff error when $n = 1$. □

Remark 2.3 (On computer assisted proofs). Theorem 2.1 can be used in a class of computer assisted proofs which rely upon finding a rigorous bound on the value of a sum. Note that $S_n = \sum_{i=1}^n a_i$ is the *mathematically exact* value, and so is $\sum_{i=1}^n |a_i|$, which may also be bound by $(1 + \varepsilon_{\text{mach}}) \sum_{i=1}^n |\text{fl}(a_i)|$ which is the computed value (a machine number). Hence the theorem provides means of finding rigorous bounds on exact mathematical quantities. □

Example 2.9 (Estimating π with a rational alternating series). From

$$\frac{\pi}{6} = \arctan\left(\frac{1}{\sqrt{3}}\right) = \sum_{k=0}^{\infty} \frac{(-1)^k}{(2k+1)(\sqrt{3})^{2k+1}},$$

multiplying by $\sqrt{3}$ gives

$$\frac{\pi\sqrt{3}}{6} = \sum_{k=0}^{\infty} \frac{(-1)^k}{(2k+1)3^k} = 1 + \sum_{k=1}^{\infty} \frac{(-1)^k}{(2k+1)3^k}.$$

Define

$$S_N := \sum_{k=1}^N \frac{(-1)^k}{(2k+1)3^k}, \quad S_{\infty} := \sum_{k=1}^{\infty} \frac{(-1)^k}{(2k+1)3^k} = \frac{\pi\sqrt{3}}{6} - 1 \approx -0.0931.$$

Truncation error. Since the terms decrease in magnitude and alternate,

$$|S_{\infty} - S_N| \leq \frac{1}{(2N+3)3^{N+1}}.$$

Roundoff error (naive running sum, with inputs rounded). Using the standard model with unit roundoff ε and the bound $\gamma_N = \frac{N\varepsilon}{1-N\varepsilon}$,

$$|\hat{S}_N - S_N| \leq \gamma_N \sum_{k=1}^N \frac{1}{(2k+1)3^k} \leq \gamma_N \sum_{k=1}^{\infty} \frac{1}{3 \cdot 3^k} = \frac{\gamma_N}{6}.$$

Total error. Combining,

$$E_N \leq \frac{1}{6} \frac{N\varepsilon}{1-N\varepsilon} + \frac{1}{(2N+3)3^{N+1}}.$$

This exhibits the tradeoff: truncation error decays like $3^{-(N+1)}$, while roundoff grows roughly like $N\varepsilon$; the optimal N balances the two.

Error balance and optimal N . With round-to-nearest unit roundoff ε and naive running sum (inputs rounded), the total error for

$$S_N = \sum_{k=1}^N \frac{(-1)^k}{(2k+1)3^k}$$

obeys

$$E_N \leq \underbrace{\frac{1}{(2N+3)3^{N+1}}}_{\text{truncation}} + \underbrace{\frac{1}{6} \frac{N\varepsilon}{1-N\varepsilon}}_{\text{roundoff}}.$$

The truncation part decays geometrically, while the roundoff part grows roughly linearly in N . For IEEE double ($\varepsilon = 2^{-53}$) the bound is minimized at

$$N_\star = 30,$$

with neighboring $N = 29, 31$ essentially tied. (If compensated/Kahan summation is used, the roundoff term becomes $O(\varepsilon)$ independent of N , so it is best to take N as large as needed to drive the truncation term below the desired tolerance.)

A program used in the calculations is presented in Listing 2.2. The program prints the following table:

N	trunc_bound	round_bound	total_bound	S_N (approx)
25	7.4229E-15	4.6259E-16	7.8855E-15	-0.09310031788290
26	2.3843E-15	4.8110E-16	2.8654E-15	-0.09310031788289
27	7.6688E-16	4.9960E-16	1.2665E-15	-0.09310031788289
28	2.4696E-16	5.1810E-16	7.6507E-16	-0.09310031788289
29	7.9622E-17	5.3661E-16	6.1623E-16	-0.09310031788289
30	2.5698E-17	5.5511E-16	5.8081E-16	-0.09310031788289
31	8.3025E-18	5.7362E-16	5.8192E-16	-0.09310031788289
32	2.6849E-18	5.9212E-16	5.9480E-16	-0.09310031788289
33	8.6902E-19	6.1062E-16	6.1149E-16	-0.09310031788289
34	2.8151E-19	6.2913E-16	6.2941E-16	-0.09310031788289
35	9.1267E-20	6.4763E-16	6.4772E-16	-0.09310031788289

□

Practical implications.

- Iterative methods should stop when residuals are within a constant multiple of $\varepsilon_{\text{mach}}$.
- Reformulations that avoid cancellation can retain accuracy down to the $\varepsilon_{\text{mach}}$ level.
- Stable algorithmic variants (compensated summation, Householder QR, etc.) keep backward error proportional to $\varepsilon_{\text{mach}}$.

Summary. Machine epsilon sets the per-operation accuracy limit. A stable algorithm ensures backward error $\sim \varepsilon_{\text{mach}}$, while the problem's condition number determines whether the forward error remains small or is magnified.

2.1.9 Errors in Numerical Computations

Errors in numerical computations arise due to the limitations of computer arithmetic. Common types of errors include:

- **Round-off Error:** Caused by rounding during arithmetic operations.
- **Truncation Error:** Caused by approximating a mathematical procedure (e.g., using a finite number of terms in a series).
- **Overflow and Underflow:** Occur when a computed value exceeds the range of representable values.

```

1  #!/usr/bin/env python3
2  from decimal import Decimal, getcontext
3  from fractions import Fraction
4
5  getcontext().prec = 80
6
7  def truncation_bound(N: int) -> Decimal:
8      return Decimal(1) / Decimal((2*N + 3) * (3 ** (N+1)))
9
10 def roundoff_bound(N: int, eps: Decimal) -> Decimal:
11     one = Decimal(1)
12     gamma = (Decimal(N+1) * eps) / (one - Decimal(N+1) * eps)
13     return gamma / Decimal(6)
14
15 def total_bound(N: int, eps: Decimal) -> Decimal:
16     return truncation_bound(N) + roundoff_bound(N, eps)
17
18 def find_optimal_N(eps: Decimal, Nmax: int = 200) -> int:
19     bestN = 0
20     bestVal = total_bound(0, eps)
21     for N in range(1, Nmax+1):
22         tb = total_bound(N, eps)
23         if tb < bestVal:
24             bestVal = tb
25             bestN = N
26     return bestN
27
28 def partial_sum_fraction(N: int) -> Fraction:
29     s = Fraction(0,1)
30     pow3 = 3
31     for k in range(1, N+1):
32         term = Fraction(1, (2*k+1) * pow3)
33         if k % 2 == 1:
34             s -= term
35         else:
36             s += term
37     pow3 *= 3
38     return s
39
40 def main(bits=53):
41     eps = Decimal(2) ** Decimal(-bits)
42     Nopt = find_optimal_N(eps, Nmax=200)
43     print(f"Unit roundoff eps = 2^-{bits} ≈ {eps}")
44     print(f"Optimal N ≈ {Nopt}")
45     header = f"{'N':>4}   {'trunc_bound':>14}   {'round_bound':>14}   {'total_bound':>14}   {'S_N (approx)':>20}"
46     print(header)
47     print("-"*len(header))
48     for N in range(max(0, Nopt-5), Nopt+6):
49         tb = truncation_bound(N)
50         rb = roundoff_bound(N, eps)
51         eb = tb + rb
52         sN = partial_sum_fraction(N)
53         sN_dec = Decimal(sN.numerator) / Decimal(sN.denominator)
54         print(f"{N:4d}   {float(tb):14.4E}   {float(rb):14.4E}   {float(eb):14.4E}   {float(sN_dec):20.14f}")
55
56 if __name__ == "__main__":
57     import sys
58     bits = int(sys.argv[1]) if len(sys.argv) > 1 else 53
59     main(bits)
60

```

Listing 2.2: Estimating π using naive summation

2.1.10 Examples of Floating-Point Arithmetic in C++

```
1 #include <iostream>
2 #include <iomanip>
3 #include <cmath>
4
5 int main() {
6     // Single precision (float)
7     float a = 1.0e20f;
8     float b = a + 1.0f;
9     std::cout << std::setprecision(8) << "a = " << a << std::endl;
10    std::cout << std::setprecision(8) << "b - a = " << b - a << std::endl; // Should be 1.0, but it's 0.0 due
    to round-off error
11
12    // Double precision (double)
13    double x = 1.0e20;
14    double y = x + 1.0;
15    std::cout << std::setprecision(16) << "x = " << x << std::endl;
16    std::cout << std::setprecision(16) << "y - x = " << y - x << std::endl; // Should be 1.0, but it's 0.0 due
    to round-off error
17
18    return 0;
19 }
```

Listing 2.3: Floating-Point Representation and Arithmetic in C++

2.1.11 Mitigating Numerical Errors

To mitigate numerical errors, consider the following strategies:

- **Avoid Subtraction of Nearly Equal Numbers:** This can lead to significant loss of precision.
- **Use Higher Precision:** When possible, use double precision instead of single precision.
- **Rearrange Computations:** Change the order of operations to minimize error accumulation.
- **Use Stable Algorithms:** Choose numerical algorithms that are less sensitive to errors.

2.1.12 Case Study: Summing a Large Array

Summing a large array of floating-point numbers can illustrate the impact of numerical errors. Consider the following example:

```
1 #include <iostream>
2 #include <vector>
3 #include <numeric>
4
5 int main() {
6     std::vector<float> vec(1000000, 0.0001f); // 1 million elements, each 0.0001
7     float sum = std::accumulate(vec.begin(), vec.end(), 0.0f);
8     std::cout << "Sum = " << sum << std::endl; // Expected 100.0, but due to round-off error, the result may
    differ
9
10    return 0;
11 }
```

Listing 2.4: Summing a Large Array of Floating-Point Numbers

In this example, the expected sum is 100.0, but due to round-off errors, the actual result may be slightly different.

Understanding computer arithmetic and its limitations is essential for developing reliable and accurate numerical algorithms in high-performance computing.

2.2 Unit in the Last Place (ulp)

The concept of a unit in the last place (ulp) is fundamental in understanding the precision of floating-point arithmetic. An ulp represents the smallest difference between two distinct floating-point numbers.

2.2.1 Definition of ulp

For a given floating-point number x , the unit in the last place (ulp) is defined as the distance between x and the next larger representable floating-point number. Formally, if x and x_{next} are two consecutive floating-point numbers, then:

$$\text{ulp}(x) = |x_{\text{next}} - x|$$

The value of an ulp depends on the position of the floating-point number in the number line. For normalized floating-point numbers, the ulp of x can be computed as:

$$\text{ulp}(x) = b^{e-p+1}$$

where b is the base, e is the exponent of x , and p is the precision (number of significant digits).

2.2.2 Significance of ulp

Understanding ulp is crucial for assessing the accuracy and precision of numerical computations. It provides a measure of the rounding error introduced during arithmetic operations. When a floating-point number is rounded to the nearest representable value, the error is typically within 0.5 ulp.

2.2.3 0.5 ulp Guarantee

The 0.5 ulp guarantee is a promise made by the IEEE 754 standard for floating-point arithmetic. It ensures that the result of a floating-point operation is as close as possible to the true mathematical result, with an error of at most 0.5 ulp. This means that the computed result is the correctly rounded value.

Example

Consider the floating-point numbers $x = 1.0$ and $y = 1.0000001$ in single precision (32-bit). The ulp of x is:

$$\text{ulp}(1.0) = 2^{-23} \approx 1.1920929 \times 10^{-7}$$

When performing the addition $x + y$, the true mathematical result is:

$$1.0 + 1.0000001 = 2.0000001$$

The nearest representable floating-point number to 2.0000001 is 2.0, with an error of 0.5 ulp. Thus, the 0.5 ulp guarantee ensures that the result is correctly rounded.

Example in C++

```
1  #include <iostream>
2  #include <cmath>
3  #include <limits>
4
5  float next_float(float x) {
6      return std::nextafter(x, std::numeric_limits<float>::infinity());
7  }
8
9  int main() {
10     float x = 1.0f;
11     float y = 1.0000001f;
```

```

12     float result = x + y;
13
14     float ulp_x = std::fabs(next_float(x) - x);
15     std::cout << "ulp(1.0) = " << ulp_x << std::endl;
16     std::cout << "x + y = " << result << std::endl;
17     std::cout << "Error = " << std::fabs(result - (x + y)) << " (should be <= 0.5 ulp)" << std::endl;
18
19     return 0;
20 }

```

Listing 2.5: Calculating ulp and Demonstrating the 0.5 ulp Guarantee in C++

In this example, we compute the ulp of 1.0 and demonstrate the 0.5 ulp guarantee by adding two floating-point numbers and checking the error.

2.2.4 Implications for Numerical Computing

The concept of ulp and the 0.5 ulp guarantee have significant implications for numerical computing:

- **Error Analysis:** Understanding ulp helps in analyzing and predicting the errors introduced in floating-point computations.
- **Algorithm Design:** Numerical algorithms can be designed to minimize the error by considering the ulp of intermediate and final results.
- **Precision Management:** Programmers can manage precision more effectively by understanding the behavior of ulp in different parts of the number line.

Understanding ulp and the 0.5 ulp guarantee is essential for developing reliable and accurate numerical algorithms in high-performance computing.

2.3 Minifloats and Floating-Point Arithmetic

Floating-point representations allow us to approximate real numbers on a digital computer with finite memory. In this section, we study an especially compact floating-point format called the *minifloat*, which provides an ideal environment for understanding floating-point principles. We also perform arithmetic using minifloats and highlight critical phenomena such as overflow, underflow, rounding, and loss of significance.

2.3.1 Floating-Point Representation

In the IEEE 754 standard, a floating-point number is represented as:

$$x = (-1)^S \cdot M \cdot 2^{E-\text{bias}}$$

where S is the sign bit, E is the biased exponent, and M is the significand (or mantissa). Normalized numbers have the form $M = 1.f$, where f is the fractional part encoded in the mantissa bits. Subnormal (denormalized) numbers have no implicit leading 1 and instead use $M = 0.f$.

2.3.2 The (1.4.3) Minifloat Format

The minifloat format we study uses 8 bits arranged as follows:

- 1 sign bit S
- 4 exponent bits E with a bias of 7
- 3 fraction bits FFF for the mantissa

The bit layout is:

$$[S \mid EEEE \mid FFF]$$

The value of the number is determined by the exponent:

- If $E = 0$ and $FFF = 000$, the number is ± 0
- If $E = 0$ and $FFF \neq 000$, the number is subnormal: $(-1)^S \cdot (0.f) \cdot 2^{-6}$
- If $1 \leq E \leq 14$, the number is normalized: $(-1)^S \cdot (1.f) \cdot 2^{E-7}$
- If $E = 15$ and $FFF = 000$, the number is $\pm\infty$
- If $E = 15$ and $FFF \neq 000$, the number is NaN

2.3.3 Normalized and Subnormal Numbers

In normalized form, the mantissa has an *implicit leading 1*. For example, if $FFF = 100$, then

$$1.f = 1.100_2 = 1 + 0.5 + 0 + 0 = 1.5$$

For subnormal numbers ($E = 0$), the implicit 1 is *not* included. So $FFF = 001$ is interpreted as

$$0.f = 0.001_2 = 1/8$$

and the actual value is $\pm(1/8) \cdot 2^{-6} = 1/512$.

2.3.4 Special Values

- **Zero:** Encoded with $E = 0$, $FFF = 000$.
- **Subnormals:** Encoded with $E = 0$, $FFF \neq 000$. Smallest positive subnormal is $0.001 \cdot 2^{-6} = 1/512$.
- **Normalized:** $1 \leq E \leq 14$. For example, $E = 7$ and $FFF = 000$ yields $1.0 \cdot 2^0 = 1.0$.
- **Infinities:** $E = 15$, $FFF = 000$.
- **NaNs:** $E = 15$, $FFF \neq 000$.

2.3.5 Arithmetic Example: Linear Function $f(x) = 3x + 1$

Let us evaluate $f(x) = 3x + 1$ for various values of x using minifloat arithmetic:

- $x = 1.0$: exact. $f(x) = 3.0 + 1 = 4.0$.
- $x = 0.001$: too small, rounds to smallest subnormal. $f(x) \approx 1.00586 \rightarrow$ rounded to 1.0.
- $x = 80$: $f(x) = 241$ exceeds max representable (240) $\Rightarrow$ overflow $\rightarrow \infty$.

Exercise. Find the largest x such that $3x + 1 < 240$.

Solution: $x < 79.666 \dots \Rightarrow x = 79.625$ is safe.

2.3.6 Arithmetic Example: Quadratic Function $f(x) = (x - 1)^2$

This is useful for observing loss of significance and underflow:

- $x = 1.125$: $f = 0.125^2 = 0.015625$, representable as normalized.
- $x = 1.0625$: $f = 0.00390625$, subnormal, but representable.
- $x = 1.0078125$: $f = (1.0078125 - 1)^2 = 6.1035 \cdot 10^{-5}$, underflow to zero.

Exercise. Find smallest $x > 1$ such that $(x - 1)^2$ is nonzero.

Solution: Requires $(x - 1)^2 \geq 0.001953125 \Rightarrow x \geq 1.0625$.

2.3.7 Key Observations

- Minifloats support subnormals, infinity, NaNs, and signed zeros like IEEE 754.
- Implicit leading 1 in normalized numbers boosts precision.
- Subnormals bridge the gap between 0 and smallest normal number.
- Arithmetic can lead to overflow ($\rightarrow \infty$) or underflow ($\rightarrow 0$).
- Functions like $f(x) = (x - 1)^2$ reveal loss of significance.

2.3.8 Exercises for Students

1. Decode the minifloat 0 0111 101. Determine if it is zero, subnormal, normal, infinity, or NaN, and compute its decimal value.
2. For $f(x) = 3x + 1$, find the largest minifloat x such that $f(x)$ does not overflow. Give the bitstring.
3. For $f(x) = (x - 1)^2$, list all minifloats x such that $f(x)$ is subnormal but nonzero.
4. For $f(x) = x^2$, determine the smallest positive minifloat x such that $f(x) = 0$ due to underflow.

2.3.9 Pedagogical Role

This analysis of minifloat arithmetic mirrors the behavior of IEEE 754 formats in a pedagogically accessible environment. Because the (1.4.3) minifloat has so few representable values, it is possible to tabulate them all, reason about rounding explicitly, and demonstrate core phenomena that arise in scientific computation: overflow, underflow, subnormals, NaNs, and the effects of rounding. This makes minifloats a powerful teaching tool in a graduate-level course on numerical methods.

2.4 Types of Errors

Understanding the types of errors that can arise in numerical computations is crucial for developing reliable and accurate numerical methods. The primary types of errors encountered in numerical methods are:

2.4.1 Round-Off Errors

Round-off errors occur due to the finite precision with which computers represent numbers. These errors arise because digital computers can only store a finite number of digits. When a number with more digits is approximated by a number with fewer digits, the difference is the round-off error.

2.4.2 Truncation Errors

Truncation errors result from approximating a mathematical process by a finite number of steps. For instance, when a Taylor series is truncated to a finite number of terms, the error introduced is called the truncation error. These errors can often be reduced by increasing the number of terms or steps in the approximation.

2.4.3 Absolute and Relative Errors

Absolute Error The difference between the true value and the approximate value. If x is the true value and $\hat{x}$ is the approximate value, the absolute error is given by $|x - \hat{x}|$.

Relative Error The absolute error divided by the true value. It provides a measure of the error in relation to the size of the true value: $\frac{|x - \hat{x}|}{|x|}$.

2.5 Error Propagation

Errors in numerical computations can propagate and amplify through subsequent calculations. Understanding how errors propagate is essential for developing stable numerical methods.

2.5.1 Propagation of Errors in Arithmetic Operations

When performing arithmetic operations, errors can propagate in different ways:

Addition and Subtraction: The absolute error in addition or subtraction is the sum of the absolute errors of the operands.

Multiplication and Division: The relative error in multiplication or division is the sum of the relative errors of the operands.

2.5.2 Conditioning of Problems

A problem is said to be *well-conditioned problem* if a small change in the input results in a small change in the output. Conversely, a problem is *ill-conditioned problem* if a small change in the input leads to a large change in the output. The conditioning of a problem can significantly affect the propagation of errors.

2.6 Stability and Convergence

Stability and convergence are fundamental concepts in the analysis of numerical methods. A numerical method must be both stable and convergent to be considered reliable.

2.6.1 Stability

Stability refers to the behavior of errors during the execution of a numerical algorithm. A numerical method is stable if errors do not grow uncontrollably as the computation progresses. Stability is particularly important in iterative methods and the solution of differential equations.

2.6.2 Convergence

A numerical method is convergent if the approximate solution approaches the exact solution as the step size or discretization parameter is refined. For example, in the context of iterative methods, convergence means that the sequence of iterates approaches the true solution.

2.6.3 Consistency

Consistency is a property related to the local error of a numerical method. A method is consistent if the local error tends to zero as the step size tends to zero. Consistency is a necessary condition for convergence.

2.7 Error Analysis in Numerical Methods

To ensure the reliability of numerical methods, it is essential to perform error analysis. This involves estimating and controlling errors to achieve the desired accuracy.

2.7.1 Taylor Series Expansion

The Taylor series expansion is a powerful tool for analyzing truncation errors. By expanding a function in a Taylor series, we can approximate it using a finite number of terms and quantify the truncation error.

2.7.2 Backward Error Analysis

Backward error analysis involves determining the perturbation in the input that would result in the computed output. This approach is useful for understanding the sensitivity of numerical algorithms to input errors.

2.7.3 Forward Error Analysis

Forward error analysis involves estimating the error in the output based on the errors in the input. This method is useful for predicting the accuracy of numerical solutions.

2.8 Practical Considerations for Error Control

In practical numerical computations, various techniques are employed to control and minimize errors:

2.8.1 Adaptive Methods

Adaptive methods adjust the step size or the number of terms in an approximation dynamically based on the estimated error. These methods are commonly used in numerical integration and solving differential equations.

2.8.2 Error Estimation and Tolerance

Setting appropriate error tolerances and using error estimation techniques can help ensure that numerical solutions meet the desired accuracy. This involves balancing the trade-off between computational cost and accuracy.

2.8.3 Validation and Verification

Validation involves comparing numerical results with analytical solutions or experimental data to ensure correctness. Verification involves ensuring that the numerical implementation correctly solves the mathematical problem it is intended to solve.

By understanding and controlling errors, we can develop robust numerical methods that provide accurate and reliable solutions to complex problems.

2.9 Exercises

2.9.1 Floating-Point Representation

Exercise 2.5. Convert the decimal number 45.6875 into its IEEE 754 single-precision floating-point representation. Show all steps involved in the conversion. $\square$

Exercise 2.6. Given the IEEE 754 single-precision floating-point representation 01000010101101100000000000000000, determine the decimal value it represents. Show all steps involved in the conversion. $\square$

2.9.2 Precision and Rounding

Exercise 2.7. Given the decimal number 3.141592653589793, round it to four significant digits using each of the following rounding methods:

- Round to nearest
- Round toward zero
- Round toward positive infinity
- Round toward negative infinity

$\square$

Exercise 2.8. Calculate the rounding error when the number 0.123456789 is rounded to three significant digits using the round to nearest method. $\square$

2.9.3 Errors in Numerical Computations

Exercise 2.9. Compute the relative error and absolute error when approximating π by 3.14. $\square$

Exercise 2.10. Consider the quadratic equation $ax^2 + bx + c = 0$ with $a = 1$, $b = 1 \times 10^6$, and $c = 1$. Solve the equation using the quadratic formula and analyze the numerical stability of the solution. $\square$

2.9.4 Unit in the Last Place (ulp)

Exercise 2.11. Calculate the unit in the last place (ulp) for the floating-point number $x = 1.5$ in IEEE 754 single-precision representation. $\square$

Exercise 2.12. Given two floating-point numbers $x = 1.0$ and $y = 1.0000001$ in IEEE 754 single-precision, determine the number of ulps between x and y . $\square$

Exercise 2.13. Write a Python function to compute the ulp of a given floating-point number. Use this function to compute the ulp of $x = 10.0$ and $x = 1.0$. $\square$

Exercise 2.14. Explain the significance of the 0.5 ulp guarantee in the context of floating-point arithmetic. Provide an example where this guarantee ensures the correct rounding of a floating-point operation. $\square$

2.9.5 Mitigating Numerical Errors

Exercise 2.15. Rearrange the following expression to minimize numerical error:

$$f(x) = \frac{1 - \cos(x)}{x^2}.$$

$\square$

Exercise 2.16. Consider the summation of a large array of floating-point numbers. Compare the numerical stability of summing the numbers in their given order versus summing them in order of increasing magnitude. $\square$

2.9.6 Backward Stability

Exercise 2.17 (Backward stability of 2 determinant). Let

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

be a 2 matrix and let us compute its determinant directly from the formula:

$$\det A = a \times d - b \times c.$$

Prove that this algorithm is backward stable, i.e., the computed value

$$\widehat{\det A} = \det(A + \Delta A)$$

where all entries of ΔA are $\leq Cu$ where u is the unit roundoff. Find the optimal C . $\square$

Exercise 2.18 (Backward stability of Cramer's rule for 2×2 systems). Consider the linear system



$$Ax = b, \quad A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, \quad b = \begin{bmatrix} f \\ g \end{bmatrix}.$$

Cramer's rule gives

$$x_1 = \frac{\det \begin{bmatrix} f & b \\ g & d \end{bmatrix}}{\det A}, \quad x_2 = \frac{\det \begin{bmatrix} a & f \\ c & g \end{bmatrix}}{\det A}.$$

1. Show that computing a 2×2 determinant $\det \begin{bmatrix} u & v \\ w & z \end{bmatrix} = uz - vw$ in floating point arithmetic can be modeled as

$$\widehat{\det} = \det \left(\begin{bmatrix} u & v \\ w & z \end{bmatrix} + \Delta \right),$$

where the entries of Δ are bounded in relative size by $O(u)$, with u the unit roundoff. Conclude that a single determinant is computed backward stably.

2. Using this model, argue that the computed numerators and denominators in Cramer's rule are exact determinants of perturbed 2×2 matrices with $O(u)$ relative perturbations.
3. Show that the final quotient $\hat{x}_i = \widehat{\det}_i / \widehat{\det A}$ is also backward stable, since floating-point division has a similar error model.
4. Conclude that the computed solution $\hat{x}$ satisfies $(A + \Delta A)\hat{x} = b$ for some perturbation ΔA with $\|\Delta A\|/\|A\| = O(u)$. Hence Cramer's rule for 2×2 systems is backward stable.
5. (Discussion) Why does this argument not extend efficiently to larger $n \times n$ systems solved by Cramer's rule?

□



Exercise 2.19 (Backward stability of LU factorization for 2×2 matrices). Consider the matrix

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix},$$

with $a \neq 0$ so that no pivoting is required. The LU decomposition algorithm computes

$$\ell = \frac{c}{a}, \quad L = \begin{bmatrix} 1 & 0 \\ \ell & 1 \end{bmatrix}, \quad U = \begin{bmatrix} a & b \\ 0 & d - \ell b \end{bmatrix}.$$

1. Show that in exact arithmetic $A = LU$.
2. Suppose floating-point arithmetic with unit roundoff u is used. Derive error bounds for the computed $\hat{\ell}, \hat{L}, \hat{U}$.
3. Prove that the computed factors satisfy

$$\hat{L}\hat{U} = A + \Delta A,$$

with $\|\Delta A\|/\|A\| = O(u)$ provided no severe cancellation occurs.

4. Conclude that the algorithm mapping $A \mapsto (L, U)$ is backward stable in this 2×2 case.
5. (Discussion) Why does the situation become more subtle for larger matrices, leading to the need for partial pivoting?

□

Chapter 3

Root-Finding Algorithms

3.1 Introduction to Root-Finding

Root-finding algorithms are essential numerical methods used to determine the roots of a function, i.e., the values of x for which $f(x) = 0$. These algorithms are widely used in various scientific and engineering applications where analytical solutions are not feasible.

3.2 Bisection Method

The Bisection Method is a simple and robust root-finding algorithm that works on continuous functions. It is based on the Intermediate Value Theorem, which states that if a continuous function changes sign over an interval, then it has at least one root in that interval.

3.2.1 Algorithm

Algorithm 3.1 Bisection Method

Require: A continuous function f and an interval $[a, b]$ such that $f(a) \cdot f(b) < 0$

Ensure: An approximation to the root of f

```
1: while the interval  $[a, b]$  is not sufficiently small do
2:   Compute the midpoint  $c = \frac{a+b}{2}$ 
3:   if  $f(c) = 0$  then
4:      $c$  is the root
5:     return  $c$ 
6:   else if  $f(a) \cdot f(c) < 0$  then
7:     Set  $b = c$ 
8:   else
9:     Set  $a = c$ 
10:  end if
11: end while
12: return  $\frac{a+b}{2}$ 
```

If $f(a) \cdot f(b) < 0$, i.e., the signs of $f(a)$ and $f(b)$ differ, the interval $[a, b]$ is called a *bracket*. Many root-finding algorithms in the real domain rely upon the existence of a bracket. The algorithms themselves do not find a bracket. Finding a bracket is more of an art, and depends on insights regarding the behavior of a function, or using trial-and-error approach.

3.2.2 About Algorithms and Pseudo-Code

Algorithm 3.1 is the first algorithm in this book, presented in the form of pseudo-code. While pseudo-code is not a programming language, it is meant to provide concise notation that a competent programmer can transform into executable program in any programming language. While pseudo-code has many flavors, and the choice of the particular variation is somewhat of a personal preference, we write all pseudo-code using the `algorithmic` package of L^AT_EX, as it provides uniformity of presentation, and is used in most computer science articles.

Pseudo-code is a mathematical object and allows one to treat an algorithm as a mathematical object. The typical issues raised are:

- **Termination** Does the algorithm terminate in a finite number of steps?
- **Correctness** Does the algorithm compute what it claims?
- **Computational Complexity** How much time and memory will the program consume when run on inputs of a particular size?

The bisection algorithm allows one to discuss the method used in studying algorithms formally. Many proofs involving algorithms involve mathematical induction on the value of the loop counter. We immediately observe that our algorithm does not count the number of executions of the loop.

To make reasoning about the algorithm, we modify the pseudo-code to have an explicit loop counter. We also disambiguate some of the language by introducing tolerance. The modified pseudo-code is Algorithm 3.2.

Algorithm 3.2 Bisection Method with Explicit Loop Counter and Tolerance

Require: A continuous function f and an interval $[a, b]$ such that $f(a) \cdot f(b) < 0$ and tolerance $tol > 0$

Ensure: An approximation to the root of f

```

1: Set counter to 0.
2: while  $|b - a| \geq tol$  do
3:   Set counter to  $1 + counter$ 
4:   Compute the midpoint  $c = \frac{a+b}{2}$ 
5:   if  $f(c) = 0$  then
6:      $c$  is the root
7:     return  $c$ 
8:   else if  $f(a) \cdot f(c) < 0$  then
9:     Set  $b = c$ 
10:  else
11:    Set  $a = c$ 
12:  end if
13: end while
14: return  $\frac{a+b}{2}$ 

```

We could proof by induction on the loop counter that

$$|b_n - a_n| \leq \frac{|b_0 - a_0|}{2^n} \quad (3.1)$$

where n is synonymous with *counter*, and a_n, b_n denote the consecutive values of variables a and b , respectively, after n passes through the loop body (the pseudo-code between **while** and **end while**). A statement that remains true regardless of the value of the loop counter is called a *loop invariant*. Thus, inequality (3.1) is an example of a loop invariant. The proof looks like this:

1. **Base Case:** When $n = 0$, the inequality obviously holds.
2. **Inductive Hypothesis:** The inequality holds for $n = k$.
3. **Inductive Step:** The inequality holds for $n = k + 1$ because with every iteration

$$|a_{k+1} - b_{k+1}| = \frac{|a_k - b_k|}{2}$$

as either a_k or b_k is replaced by the midpoint of the interval $[a_k, b_k]$.

4. **Conclusion** The inequality 3.1 holds for every n that was attained during consecutive passes through the loop body. Note that we must break out of the loop, because the inequality $|b_n - a_n| < tol$ will be satisfied after a finite number of steps.

After we have the loop invariant, we can easily address the issue of termination. The inequality

$$|b_n - a_n| \leq tol$$

is attained within n iterations where

$$n \leq \left\lceil \log_2 \frac{|a_0 - b_0|}{tol} \right\rceil$$

Thus, the loop cannot continue indefinitely. These considerations are mathematically rigorous, and provide a framework for algorithm analysis, but in a book on numerical methods one typically focuses on error analysis, and discusses other issues less formally.

3.2.3 Error Analysis

The error in the Bisection Method decreases by a factor of 2 with each iteration. The method converges linearly, and the absolute error after n iterations is given by:

$$|c_n - c| \leq \frac{b - a}{2^n}$$

where c is one of the roots of the equation $f(c) = 0$.

Exercise 3.1. Consider $tol = 0$ in Algorithm 3.1.

1. Does the algorithm terminate in this case?
2. A theorem in topology says that a descending sequence of compact sets, e.g. closed intervals, has a non-empty intersection. Use this theorem to prove that a_n and b_n converge to a root of the equation $f(c) = 0$.
3. Do a_n and b_n form a *Cauchy sequence*?

□



Exercise 3.2. The equation $f(c) = 0$ can have an infinite or even uncountable set of solutions. One example is a function called the *Devil's Staircase* or *Cantor (ternary) function* [11] (see Figure 3.1). This is a function $F : [0, 1] \rightarrow [0, 1]$ which is continuous, monotonically increasing, surjective (“onto”) and differentiable everywhere, except for the middle-third Cantor set.

1. Write a program which calculates $F(x)$ within given tolerance tol .
2. Use the bisection algorithm to solve the equation $F(x) = 1 - x^2$ (which must have a solution!). Does the algorithm converge to the exact solution in a finite number of steps, when the round-off error is ignored?
3. Consider applying the bisection method to solving the equation $F(x) = y$ for $y \in [0, 1]$. Prove that the solution is unique, except for a countable, dense set of $y \in [0, 1]$. Characterize those special values of y .

□

3.3 Fixed-Point Iteration

Fixed-Point Iteration is a method for finding the root of a function by rewriting the equation $f(x) = 0$ in the form $x = g(x)$ and iterating:

$$x_{n+1} = g(x_n)$$

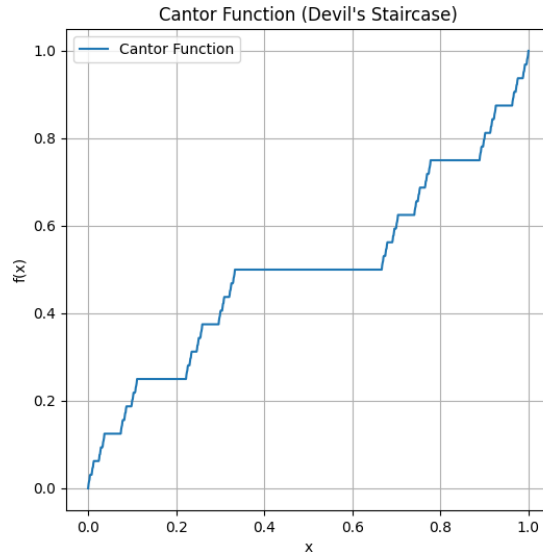


Figure 3.1: The Cantor (ternary) function.

3.3.1 Algorithm

See Algorithm 3.3.

Note that in addition to tolerance, we also impose a bound on the number of iterations, as otherwise the algorithm may not terminate.

Algorithm 3.3 Fixed-Point Iteration

Require: A continuous function g and an initial guess x_0 ; tolerance $tol > 0$; maximum number of iterations N

Ensure: An approximation to the fixed point of g

```

1: for  $n = 1$  to  $N$  do
2:   Compute the next iterate  $x_1 = g(x_0)$ 
3:   if  $|x_1 - x_0| < tol$  then
4:     return  $x_1$ 
5:   end if
6:   Update  $x_0 = x_1$ 
7: end for
8: Print 'Tolerance not met'
```

The stopping condition $|x_1 - x_0| < tol$ is geared towards achieving a small absolute error. We could replace it with

$$\frac{|x_1 - x_0|}{|x_0|} < tol$$

if we wanted to achieve a small relative error. The stopping condition may be even more nuanced, to achieve application-specific goals.

3.3.2 Convergence Criteria

Invariance Condition

We require that $g : [a, b] \rightarrow \mathbb{R}$ and that $g([a, b]) \subseteq [a, b]$, i.e., that the interval $[a, b]$ is *invariant*. The function $f(x) = g(x) - x$ thus changes sign on $[a, b]$ as $g(a) \geq a$ and $g(b) \leq b$, or one of the endpoints is a fixed point. Thus, given that g is continuous, there exists a point x such that $f(x) = g(x) - x = 0$, i.e., there exists a fixed point.

Contraction Condition

Given that $g : [a, b] \rightarrow [a, b]$, the Fixed-Point Iteration converges if $g(x)$ is a contraction, i.e., there exists a constant $0 \leq k < 1$ such that:

$$\forall x \in [a, b] : |g'(x)| \leq k.$$

3.3.3 Proof of convergence

Let $x_\infty \in [a, b]$ be a fixed point. If g is a contraction then for every $x \in [a, b]$

$$|g(x) - x_\infty| = |g(x) - g(x_\infty)| = |g'(\theta)| |x - x_\infty| \leq k |x - x_\infty|$$

for some $\theta \in [x_\infty, x]$, by the Mean Value Theorem. In particular

$$|x_{n+1} - x_\infty| = |g(x_n) - x_\infty| \leq k |x_n - x_\infty|$$

and by induction

$$\forall n : |x_n - x_\infty| \leq k^n |x_0 - x_\infty|.$$

In particular,

$$\lim_{n \rightarrow \infty} x_n = x_\infty$$

exponentially fast, which is called *linear convergence*.

3.4 The Two-Ladder Problem

We study the classical two-ladder problem with an easy solution, and a modified version which makes a case for using numerical methods.

3.4.1 The Classical Two-Ladder Problem

Problem 1 (The Classical Two-Ladder Problem). Two ladders, one of length L_1 and the other of length L_2 , lean against opposite walls of a narrow alley of width w . The ladders cross each other at a height h above the ground. The goal is to determine the height h . $\square$

The problem is solved easily using Thales Theorem, by setting up a system of equations for the variables x and h , the coordinates of the point of intersection (x, h) :

$$\begin{cases} \frac{h}{x} &= \frac{\sqrt{L_1^2 - w^2}}{w} \\ \frac{h}{w-x} &= \frac{\sqrt{L_2^2 - w^2}}{w} \end{cases}$$

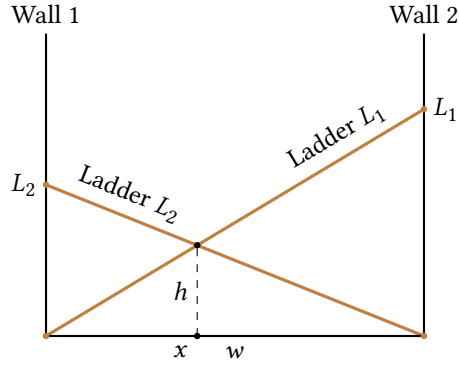


Figure 3.2: Two ladders in an alley, unknown is the intersection point.

We can eliminate h , dividing side-wise, and solve for x :

$$\begin{aligned}\frac{w-x}{x} &= \frac{\sqrt{L_1^2 - w^2}}{\sqrt{L_2^2 - w^2}}, \\ \frac{w}{x} - 1 &= \frac{\sqrt{L_1^2 - w^2}}{\sqrt{L_2^2 - w^2}}, \\ \frac{w}{x} &= 1 + \frac{\sqrt{L_1^2 - w^2}}{\sqrt{L_2^2 - w^2}}, \\ x &= \frac{w}{1 + \frac{\sqrt{L_1^2 - w^2}}{\sqrt{L_2^2 - w^2}}}.\end{aligned}$$

Having x , h is obtained easily, e.g., using the first equation:

$$\begin{aligned}h &= x \frac{\sqrt{L_1^2 - w^2}}{w} \\ &= \frac{\sqrt{L_1^2 - w^2}}{1 + \frac{\sqrt{L_1^2 - w^2}}{\sqrt{L_2^2 - w^2}}} \\ &= \frac{1}{\frac{1}{\sqrt{L_1^2 - w^2}} + \frac{1}{\sqrt{L_2^2 - w^2}}}.\end{aligned}$$

3.4.2 The Modified Two-Ladder Problem

Problem 2 (The Modified Two-Ladder Problem). Two buildings are separated by an alley. Two ladders are placed so that the base of each ladder is against one of the buildings and reaches the top of the other building. The two ladders are $L_1 = 40$ feet and $L_2 = 30$ feet long. Further, they cross at a point $h = 10$ feet from the ground. How wide is the alley? $\square$

Below we use a more general approach to a variety of problems in point geometry, where a system of equations is build for the unknowns from basic principles.

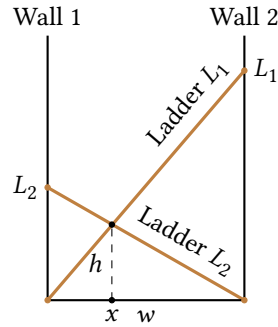


Figure 3.3: Two ladders in an alley, unknown is the width of the alley.

Formulating the Equations

As before, we set up a system of equations, this time for the variables x and w .

$$\begin{cases} \frac{h}{x} = \frac{\sqrt{L_1^2 - w^2}}{w}, \\ \frac{h}{w-x} = \frac{\sqrt{L_2^2 - w^2}}{w}. \end{cases}$$

We solve both systems for x and compare.

$$\begin{aligned} x &= \frac{hw}{\sqrt{L_1^2 - w^2}}, \\ x &= w - \frac{hw}{\sqrt{L_2^2 - w^2}}, \\ \frac{hw}{\sqrt{L_1^2 - w^2}} &= w - \frac{hw}{\sqrt{L_2^2 - w^2}}, \\ \frac{h}{\sqrt{L_1^2 - w^2}} &= 1 - \frac{h}{\sqrt{L_2^2 - w^2}}, \\ \frac{h}{\sqrt{L_1^2 - w^2}} + \frac{h}{\sqrt{L_2^2 - w^2}} &= 1. \end{aligned}$$

This results in a quartic (degree 4) polynomial equation in w that is challenging to solve analytically.

Numerical Solution Using the Bisection Method

To solve this problem numerically (in Python), we use the bisection method to approximate the width w :

Solving the Two Ladders Problem by Bisection

```

1 import numpy as np
2
3 def f(w, L1, L2, h):
4     u1 = h / np.sqrt(L1**2 - w**2)
5     u2 = h / np.sqrt(L2**2 - w**2)
6     return u1 + u2 - 1
7
8 def bisection_method(func, a, b, tol):
```



```

9      while (b - a) / 2.0 >= tol:
10         midpoint = (a + b) / 2.0
11         if func(midpoint) == 0:
12             return midpoint
13         elif func(midpoint) * func(a) < 0:
14             b = midpoint
15         else:
16             a = midpoint
17         return (a + b) / 2.0
18
19     # Given values
20     L1 = 40
21     L2 = 30
22     h = 10
23
24     # Initial guesses
25     a = 0.0
26     b = min(L1, L2)
27
28     # Tolerance level
29     tol = 1e-6
30
31     # Solve for w
32     func = lambda w : f(w, L1, L2, h)
33     w = bisection_method(func, a, b, tol)
34     print(f"The width of the alley is approximately {w:.6f}")

```

This width of the alley is $w = 22.5$.

3.4.3 The dimensionless form of the ladder equation

The main equation of the two-ladder problem was written as:

$$\frac{h}{\sqrt{L_1^2 - w^2}} + \frac{h}{\sqrt{L_2^2 - w^2}} = 1. \quad (3.2)$$

Introducing variables $\xi = (w/h)^2$, $a = (L_1/h)^2$ and $b = (L_2/h)^2$, we can write this equation as

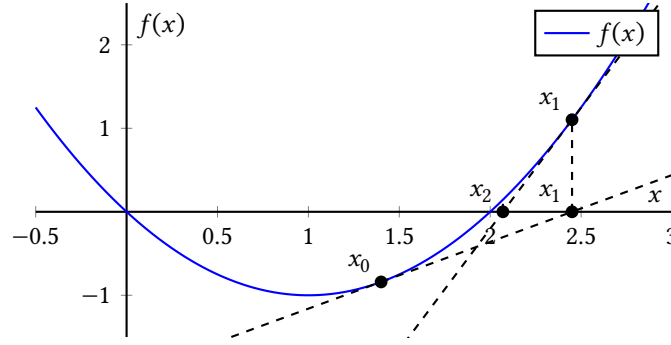
$$\frac{1}{\sqrt{a - \xi}} + \frac{1}{\sqrt{b - \xi}} = 1. \quad (3.3)$$

We notice that all quantities are dimensionless¹. This equation is equivalent to a 4th degree polynomial equation, and as such it has a solution in terms of radicals. Generally speaking, this fact can be exploited to solve the equation very efficiently, as it is reduced to finding the radicals, which is highly optimized in either hardware or library codes. The solution of equation (3.3) is subject of Exercise 3.23.

3.5 Newton-Raphson Method

The Newton-Raphson Method is a powerful root-finding algorithm that uses the derivative of the function to achieve rapid convergence. It is particularly effective when the initial guess is close to the true root.

¹https://en.wikipedia.org/wiki/Dimensionless_quantity



3.5.1 Algorithm

It is features as Algorithm 3.4.

Algorithm 3.4 Newton-Raphson Method

Require: A differentiable function f and an initial guess x_0 ; tolerance $tol > 0$; maximum number of iterations N

Ensure: An approximation to the root of f

```

1: for  $n = 1$  to  $N$  do
2:   Set  $x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$ 
3:   if  $|x_1 - x_0| < tol$  then
4:     return  $x_1$ 
5:   end if
6:   Set  $x_0 = x_1$ 
7: end for
8: Print 'Tolerance not met'
```

3.5.2 Rationale

Newton-Raphson algorithm is derived by *linearization* of a typically non-linear function $f(x)$ /

The equation $f(x) = 0$ with initial approximation x_0 is linearized by replacing $f(x)$ with the linear function $\ell(x) = f(x_0) + f'(x_0)(x - x_0)$. This is the function whose graph is tangent to the graph of $f(x)$ at $(x_0, f(x_0))$. Then we solve the equation $L(x) = 0$.

$$f(x_0) + f'(x_0)(x - x_0) = 0 \implies x = x_0 - \frac{f(x_0)}{f'(x_0)}.$$

It is worthwhile to note that Newton-Raphson iteration is equivalent to fixed point iteration $x_{n+1} = g(x_n)$, where

$$g(x) = x - \frac{f(x)}{f'(x)}.$$

The condition of convergence is *contraction*:

$$|g'(x)| \leq k < 1.$$

This leads to considering

$$g'(x) = 1 - \frac{(f'(x))^2 - f(x)f''(x)}{f'(x)^2} = \frac{f(x)f''(x)}{f'(x)^2}.$$

If either $f(x_0) \approx 0$ or the $f(x)$ is close to linear ($f''(x)$ small) then contraction will hold and the method will converge to a solution.

3.5.3 Error Analysis

The Newton-Raphson Method converges quadratically if the initial guess is sufficiently close to the true root x and $f'(x) \neq 0$. The error at the n -th iteration is approximately:

$$|x_{n+1} - x| \approx \frac{|f''(x)|}{2|f'(x)|} |x_n - x|^2 \quad (3.4)$$

Kantorovich theorem² refines this inequality to guarantee the existence of solution to $f(x) = 0$ and quadratic convergence to it without explicitly knowing x .

3.5.4 A proof (idea only)

To prove bound (3.4), we note that

$$0 = f(x) = f(x_n) + f'(x_n)(x - x_n) + \frac{1}{2}f''(\theta)(x - x_n)^2$$

where $\theta \in [x_n, x]$. On the other hand,

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

yields

$$f'(x_n)(x_{n+1} - x_n) = -f(x_n)$$

and by second order Taylor expansion at x_n :

$$f'(x_n)(x_{n+1} - x) = -f'(x_n)(x - x_n) - f(x_n) = \frac{1}{2}f''(\theta)(x - x_n)^2$$

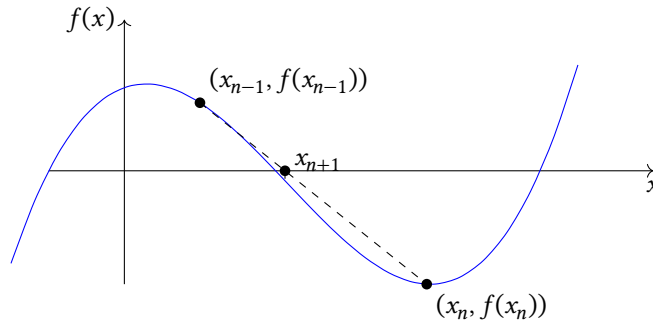
Hence

$$x_{n+1} - x = \frac{1}{2} \frac{f''(\theta)}{f'(x_n)} (x - x_n)^2$$

which is close enough to (3.4).

3.6 Secant Method

The Secant Method is similar to the Newton-Raphson Method but does not require the computation of the derivative. Instead, it uses a finite difference approximation to estimate the derivative.



3.6.1 Algorithm

²https://en.wikipedia.org/wiki/Kantorovich_theorem

Algorithm 3.5 Secant Method

Require: A continuous function f and two initial guesses x_0 and x_1 ; tolerance $tol > 0$; maximum number of iterations N

Ensure: An approximation to the root of f

- 1: **for** $n = 1$ to N **do**
- 2: Compute the next iterate using the formula:

$$x_2 = x_1 - f(x_1) \frac{x_1 - x_0}{f(x_1) - f(x_0)}$$

- 3: **if** $|x_1 - x_0| < tol$ **then**
 - 4: **return** x_1
 - 5: **end if**
 - 6: Update $x_0 = x_1$ and $x_1 = x_2$
 - 7: **end for**
 - 8: Print 'Tolerance not met'
-

3.6.2 Error Analysis

The Secant Method converges superlinearly with an order of convergence of approximately 1.618 (the golden ratio). The error at the n -th iteration is approximately:

$$|x_{n+1} - x| \approx C|x_n - x|^\phi$$

where $\phi \approx 1.618$ is the *golden ratio* and C is a constant.

3.6.3 Secant Method as Fixed Point Iteration

The Secant Method can be regarded as fixed point iteration, for $F : \mathbb{R}^2 \rightarrow \mathbb{R}^2$, where

$$F(x_0, x_1) = (x_1, x_2).$$

Thus,

$$F(x, y) = \left(y, y - f(y) \frac{y - x}{f(y) - f(x)} \right).$$

The mapping can be extended to the diagonal $\Delta = \{(x, y) \in \mathbb{R}^2 : x = y\}$ by taking a limit as $y \rightarrow x$:

$$F(x, y) = \begin{cases} \left(y, y - f(y) \frac{y - x}{f(y) - f(x)} \right), & x \neq y, \\ \left(x, x - \frac{f(x)}{f'(x)} \right), & x = y. \end{cases}$$

The second formula has the Newton-Raphson formula embedded in it. The stopping condition $|x_1 - x_0| < tol$ measures the distance of (x_0, x_1) from the diagonal, which is an *invariant set*, i.e., $F(\Delta) \subseteq \Delta$.

3.6.4 The Secant Method and Its Order of Convergence

The secant method is a classical root-finding algorithm that replaces Newton's derivative evaluation with a finite-difference approximation. Its iterates are defined by

$$x_{k+1} = x_k - f(x_k) \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}, \quad k \geq 1,$$

starting from two initial guesses x_0, x_1 .

The convergence rate of the secant method is *superlinear* but not quadratic. Remarkably, its order of convergence is the golden ratio.

Theorem 3.1 (Convergence order of the secant method). Let $f \in C^2$ in a neighborhood of a simple root α , i.e. $f(\alpha) = 0$, $f'(\alpha) \neq 0$. If the secant method is initiated sufficiently close to α , then the iterates $x_k \rightarrow \alpha$ and the error sequence $e_k := x_k - \alpha$ satisfies

$$|e_{k+1}| \sim A |e_k|^p, \quad k \rightarrow \infty,$$

where

$$p = \frac{1 + \sqrt{5}}{2} \approx 1.618, \quad A = \left| \frac{f''(\alpha)}{2f'(\alpha)} \right|^{1/p}.$$

Thus, the secant method converges with order p , the golden ratio. $\square$

Proof. Expand f around α :

$$f(x) = f'(\alpha)(x - \alpha) + \frac{1}{2}f''(\alpha)(x - \alpha)^2 + O((x - \alpha)^3).$$

For the iterates $x_k = \alpha + e_k$, this yields

$$f(x_k) = f'(\alpha)e_k + \frac{1}{2}f''(\alpha)e_k^2 + O(e_k^3).$$

Similarly for $f(x_{k-1})$. The secant denominator then expands as

$$\frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}} = f'(\alpha) + \frac{1}{2}f''(\alpha)(e_k + e_{k-1}) + O(e_k^2 + e_{k-1}^2).$$

Inserting these expansions into the secant update and simplifying gives the two-step error relation

$$e_{k+1} = \frac{f''(\alpha)}{2f'(\alpha)} e_k e_{k-1} + O(e_k e_{k-1} (e_k + e_{k-1})).$$

Let $C := \frac{f''(\alpha)}{2f'(\alpha)}$.

Assume the method converges with order $p > 1$, i.e.

$$|e_k| \sim A |e_{k-1}|^p, \quad k \rightarrow \infty.$$

Then

$$|e_{k+1}| \sim A |e_k|^p \sim A (A |e_{k-1}|^p)^p = A^{p+1} |e_{k-1}|^{p^2}.$$

On the other hand, from the recurrence,

$$|e_{k+1}| \sim |C| |e_k| |e_{k-1}| \sim |C| A |e_{k-1}|^{p+1}.$$

Matching exponents gives $p^2 = p + 1$, whose positive solution is

$$p = \frac{1 + \sqrt{5}}{2}.$$

Matching coefficients then forces $A^p = |C|$, hence $A = |C|^{1/p}$. This completes the proof. $\square$

3.7 False Position Method

The False Position Method (Regula Falsi), featured as Algorithm 3.6, combines the ideas of the Bisection Method and the Secant Method to achieve reliable convergence by maintaining a bracketing interval.

Algorithm 3.6 False Position Method

Require: A continuous function f and an interval $[a, b]$ such that $f(a) \cdot f(b) < 0$; tolerance $tol > 0$

Ensure: An approximation to the root of f

- 1: **while true do**
- 2: Compute the next iterate using the formula:

$$c = b - f(b) \frac{b - a}{f(b) - f(a)}$$

- 3: **if** $|f(c)| < tol$ **then**
 - 4: **return** c $\{c$ is the approximate root $\}$
 - 5: **else if** $f(a) \cdot f(c) < 0$ **then**
 - 6: Set $b = c$
 - 7: **else**
 - 8: Set $a = c$
 - 9: **end if**
 - 10: **end while**
-

3.7.1 Algorithm

Note that the interval $[a_n, b_n]$ is not expected to shrink, e.g., when the function is convex. Instead, a_n converges to the root, while b_n remains constant. For that reason, we must use a different termination condition from the Bisection Method (Algorithm 3.1). From the several possible ways to terminate the algorithm we chose the variant where $|f(c)|$ is supposed to become small.

The convergence of the method is only linear, i.e., for sufficiently large n

$$|c_{n+1} - x| \leq k|c_n - x|$$

where k is constant and $0 \leq k < 1$.

3.7.2 Error Analysis

The False Position Method improves the convergence of the Bisection Method while maintaining its robustness. The convergence rate is generally slower than the Secant Method but faster than the Bisection Method.

3.8 Comparisons and Practical Considerations

3.8.1 Convergence Rates

- **Bisection Method:** Linear convergence.
- **Fixed-Point Iteration:** Linear convergence, dependent on $g'(x)$.
- **Newton-Raphson Method:** Quadratic convergence, requires derivative.
- **Secant Method:** Superlinear convergence, does not require derivative.
- **False Position Method:** Linear convergence, maintains bracketing interval.

3.8.2 Choosing the Right Method

The choice of root-finding method depends on various factors, including:

- Availability of the derivative.
- Initial guesses.
- Desired accuracy.
- Computational efficiency.

```

1 from autodiff import autodiff
2
3 @autodiff
4 def h(x):
5     y = x**2-3*x+1
6     return y
7
8 y, dy = h(2.0)
9
10 print(f"Value: {y}, Derivative:{dy}")

```

Listing 3.1: Automatic Differentiation in Python

3.8.3 Implementation and Efficiency

When implementing root-finding algorithms, consider:

- Stopping criteria: Define a tolerance for the convergence check.
- Handling special cases: Ensure the method can handle cases where the function is flat or has multiple roots.
- Robustness: Choose methods that are less sensitive to initial guesses for critical applications.

In conclusion, root-finding algorithms are fundamental tools in numerical analysis, and selecting the appropriate method can significantly impact the efficiency and accuracy of solving practical problems.

3.9 Automatic Differentiation

As we mentioned in the Prologue, the technique of automatic differentiation gained prominence, due in part to the growing complexity of the models considered, such as neural network models, which are equivalent to deeply nested functions with millions of variables. Methods requiring evaluation of derivatives, such as Newton-Raphson, become impractical if the calculation of the derivative would to be done by hand. That is, if the derivative needed to be calculated symbolically by the chain rule. Symbolic computation simulates by-hand calculation using the chain rule, but the size of the symbolic derivative is prohibitively expensive for even modest problems. This is when automatic differentiation comes in.

The main idea of automatic differentiation is to eagerly substitute numerical values of variables, while executing the chain rule. This leads to great savings in terms of time and memory, making the calculation of derivatives comparable in computational cost to the calculation of the value of the function.

3.9.1 Automatic differentiation in Python and Syntactic Sugar

We will show a simple implementation of automatic differentiation in Python. Additionally, we will use the decorator facility of Python to create functions that “differentiate themselves”. This makes the code in Listing 3.1 possible. The code prints “Value: -1.0, Derivative:1.0”, although we never told the function `h` how to calculate the derivative. Magically, the function knows to return the derivative as the second value. The term *syntactic sugar* is used to describe such unexpected benefits of language features, in this case, decorators.

We should quickly observe that the function `h` is ready to be passed to the Newton-Raphson algorithm, thus achieving quadratic convergence with minimal effort.

The full implementation of our Python package `autodiff` is presented in Chapter F.

3.9.2 How does automatic differentiation work?

The idea is simple. Variables store a quantity (the “value”). Variables are augmented with the rate of change (“derivative”). The augmented variable named `var` for example, stores both, i.e. is a pair:

$$x = (x, c) = x + c \cdot dx$$

where the right-hand side is a more mnemonic mathematical notation. The factor c is called the *derivative* (note that there is no function!).

Now we can easily implement the Chain Rule on the augmented variables. The value variable is $y + dy$, where e is to be found. If f is a function then

$$y + dy = f(x + dx) = f(x) + f'(x) dx + \text{higher order terms.}$$

Thus, to reflect the meaning of calculus, we must set

$$dy = f'(x) dx$$

Thus, we augment the rule for evaluating the function f on an augmented variable as follows:

$$f(x + c dx) = f(x) + c f'(x) dx = (f(x), f'(x)c)$$

Finally, we realize that we may get rid of c as the only thing that matters is the coefficient in front of it. Therefore, we set $c = 1$ and get just the coefficient.

So, *mere evaluation* of f on the augmented variable has the effect of evaluating the derivative $f'(x)$ built into it! The crux is a new rule for evaluating functions on the augmented variables, which we just discovered. This automatically extends to compositions, like $z = g(f(x))$, which will automatically perform the Chain Rule (correctly), and functions of many variables, e.g.,

$$w = h(g(x, y), h(y)).$$

A point perhaps somewhat lost in the notation is that if we want to track the change of some complex composition under the changes of x , on the onset we augment x with $c = 1$, i.e. the augmented variable $(x, 1)$. This is reflected in the constructor of the `Variable` class, where the derivative field is initialized to 1 by default. Another important initialization is with a 0, which means that a variable is considered constant in the calculations. This is useful in evaluating partial derivatives.

3.10 Exercises

Exercise 3.3. Use the Bisection Method to find the root of the function $f(x) = x^3 - 4x^2 + 6$ within the interval $[1, 2]$ with a tolerance of 10^{-4} . $\square$

Exercise 3.4. Given the function $g(x) = \sqrt{4 - x^2}$, use Fixed-Point Iteration to find the fixed point starting with the initial guess $x_0 = 1.5$. Use a tolerance of 10^{-4} . $\square$

Exercise 3.5. Apply the Newton-Raphson Method to find the root of $f(x) = e^x - 3x$ starting with an initial guess of $x_0 = 0.5$. Use a tolerance of 10^{-5} . $\square$

Exercise 3.6. Use the Secant Method to find the root of the function $f(x) = \ln(x) - 1$ with initial guesses $x_0 = 2$ and $x_1 = 1.5$. Use a tolerance of 10^{-5} . $\square$

Exercise 3.7. Find the root of the equation $f(x) = x^2 - 2$ using the False Position Method within the interval $[1, 2]$. Use a tolerance of 10^{-6} . $\square$

Exercise 3.8. Find the root of $f(x) = \cos(x) - x$ in the interval $[0, 1]$ using the Bisection Method with a tolerance of 10^{-4} . $\square$

Exercise 3.9. For the function $g(x) = \frac{x+2}{3}$, perform Fixed-Point Iteration starting at $x_0 = 2$ and find the fixed point with a tolerance of 10^{-4} . $\square$

Exercise 3.10. Use the Newton-Raphson Method to find the root of $f(x) = x^2 - \sin(x) - 1$ starting with $x_0 = 1$. Use a tolerance of 10^{-5} . $\square$

Exercise 3.11. Apply the Secant Method to find the root of $f(x) = x^3 - x - 1$ with initial guesses $x_0 = 1$ and $x_1 = 1.5$. Use a tolerance of 10^{-6} . $\square$

Exercise 3.12. Solve $f(x) = \tan(x) - x$ in the interval $[4, 5]$ using the False Position Method with a tolerance of 10^{-5} . $\square$

Exercise 3.13. Implement a hybrid method combining Bisection and Newton-Raphson to find the root of $f(x) = x^2 - 4$ starting with $x_0 = 3$. Use a tolerance of 10^{-6} . $\square$

Exercise 3.14. Prove that the Fixed-Point Iteration $g(x) = \frac{1}{3} \sin(x) + \frac{2}{3}$ converges for x in the interval $[0, 1]$. $\square$

Exercise 3.15. Use the Newton-Raphson Method to find the root of $f(x) = (x - 1)^2$. Discuss how the method behaves differently for this function compared to a simple root. $\square$

Exercise 3.16. Compare the convergence rates of the Bisection Method and the Secant Method for $f(x) = x^3 - 6x^2 + 11x - 6$ starting with appropriate intervals or initial guesses. $\square$

Exercise 3.17. Use symbolic differentiation to derive the Newton-Raphson update formula for $f(x) = x^2 \cos(x) - 1$, then find its root. $\square$

Exercise 3.18. Analyze the error propagation in the Bisection Method for $f(x) = e^x - 2$ and determine the number of iterations required to achieve a tolerance of 10^{-6} . $\square$

Exercise 3.19. Give an example of a function $f(x)$ and an initial guess where the Newton-Raphson Method fails to converge. Explain why the method fails. $\square$

Exercise 3.20. Write a program to implement the Secant Method and use it to find the root of $f(x) = xe^x - 2$. Compare the result with the analytical solution. $\square$

Exercise 3.21. Use the Newton-Raphson Method to find the maximum of $f(x) = x^3 - 3x^2 + 4$ by solving $f'(x) = 0$. $\square$

Exercise 3.22. Use the False Position Method to find all roots of $f(x) = x^4 - 10x^2 + 9$ within the interval $[-3, 3]$. $\square$

Exercise 3.23. Solve the following equation in radicals, where a and b are positive parameters, for the unknown ξ :

$$\frac{1}{\sqrt{a-\xi}} + \frac{1}{\sqrt{b-\xi}} = 1$$

The solution of interest satisfies $\xi > 0$.

1. Prove that a solution exists iff $1/a + 1/b < 1$. Prove that the solution is unique.
2. Reduce the equation to a quartic equation

$$\xi^4 + A\xi^3 + B\xi^2 + C\xi + D = 0.$$

You may consider using a CAS or Python package *Sympy*.

3. Using on-line resources, AI assistance or otherwise, express the solution in terms of radicals. The only radicals you need are $\sqrt{\cdot}$ and $\sqrt[3]{\cdot}$.
4. Implement the solution in software and compare run times to the bisection method and Regula Falsi, when solving the equation for 1000 randomly chosen parameters a and b .

$\square$

Chapter 4

Numerical Linear Algebra

4.1 Introduction to Numerical Linear Algebra

Numerical linear algebra involves the development and analysis of algorithms for performing linear algebra computations, such as solving systems of linear equations, computing eigenvalues and eigenvectors, and performing matrix factorizations. These algorithms are fundamental to many applications in science, engineering, and computer science.

4.2 Example: Principal Component Analysis (PCA)

To illustrate the importance and impact of numerical linear algebra, let's begin with an interesting example involving the analysis of large-scale data using principal component analysis (PCA). This example is aspirational in nature, and we will spend the remainder of this chapter introducing the specific techniques involved.

Principal Component Analysis (PCA) is a powerful technique used in data analysis and machine learning for reducing the dimensionality of large datasets while preserving as much information as possible. It achieves this by transforming the original data into a new set of orthogonal variables called principal components, which are ordered by the amount of variance they capture from the data.

4.2.1 Motivation

Consider a dataset consisting of measurements from several sensors in a large industrial plant. Each sensor records data every second, resulting in a high-dimensional dataset with thousands of variables. Analyzing such a dataset can be challenging due to its high dimensionality and the presence of correlations among the variables.

4.2.2 PCA Algorithm

The goal of PCA is to identify the directions (principal components) in which the data varies the most and project the data onto these directions. This reduces the dimensionality of the data while retaining the most important features. The steps involved in PCA are as follows:

1. **Standardize the Data:** Subtract the mean and divide by the standard deviation for each variable to obtain a dataset with zero mean and unit variance.
2. **Compute the Covariance Matrix:** Calculate the covariance matrix of the standardized data.
3. **Eigenvalue Decomposition:** Perform eigenvalue decomposition of the covariance matrix to obtain the eigenvalues and eigenvectors.
4. **Select Principal Components:** Choose the top k eigenvectors corresponding to the largest eigenvalues to form the principal components.

5. **Transform the Data:** Project the original data onto the principal components to obtain the reduced-dimensional dataset.

4.2.3 Example Implementation

Let's illustrate PCA with a simple example. Consider a dataset with two variables x and y :

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Generate a synthetic dataset
5 np.random.seed(0)
6 x = np.random.normal(0, 1, 100)
7 y = 2 * x + np.random.normal(0, 1, 100)
8 data = np.vstack((x, y)).T
9
10 # Standardize the data
11 data_mean = np.mean(data, axis=0)
12 data_std = np.std(data, axis=0)
13 data_standardized = (data - data_mean) / data_std
14
15 # Compute the covariance matrix
16 cov_matrix = np.cov(data_standardized, rowvar=False)
17
18 # Perform eigenvalue decomposition
19 eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)
20
21 # Select the top principal component
22 principal_component = eigenvectors[:, np.argmax(eigenvalues)]
23
24 # Project the data onto the principal component
25 data_pca = data_standardized @ principal_component
26
27 # Plot the original data and the principal component
28 plt.scatter(data[:, 0], data[:, 1], label='Original Data')
29 plt.quiver(0, 0, principal_component[0], principal_component[1],
30 angles='xy', scale_units='xy', scale=1, color='r', label='Principal Component')
31 plt.xlabel('x')
32 plt.ylabel('y')
33 plt.legend()
34 plt.title('PCA on 2D Data')
35 plt.show()
```

Listing 4.1: PCA Implementation in Python

4.2.4 Discussion

In this example, we generated a synthetic dataset with two correlated variables and applied PCA to identify the principal component. The plot shows the original data along with the principal component, which captures the direction of maximum variance in the data.

This example demonstrates how numerical linear algebra techniques, such as eigenvalue decomposition, are essential for data analysis tasks like PCA. In the following sections, we will explore the fundamental concepts and algorithms of numerical linear algebra that underpin many such applications.

4.3 Outline of the Chapter

This chapter will cover the following topics:

- Vector and Matrix Norms

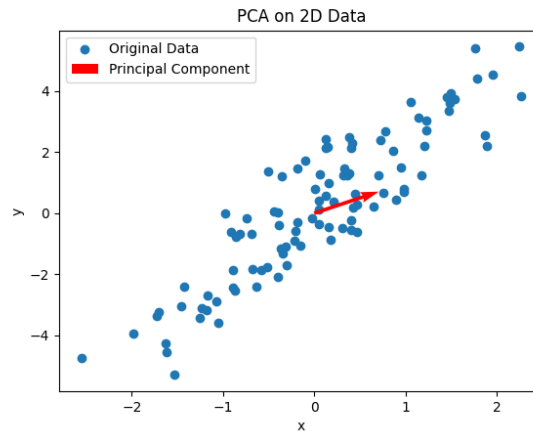


Figure 4.1: PCA — the principal component.

- Solving Linear Systems
- Eigenvalue Problems
- Singular Value Decomposition (SVD)
- Iterative Methods
- Applications in Data Science and Machine Learning

By the end of this chapter, you will have a solid understanding of the key numerical linear algebra techniques and their applications, enabling you to tackle a wide range of computational problems.

4.4 Direct Methods for Linear Systems

Direct methods provide exact solutions to systems of linear equations (up to round-off error) in a finite number of operations. The most common direct methods include Gaussian Elimination and LU Decomposition.

4.4.1 Gaussian Elimination

Gaussian Elimination is a method for solving a system of linear equations by transforming the system's matrix into an upper triangular form.

4.4.2 Gaussian Elimination with Partial Pivoting

Partial Pivoting involves swapping rows to ensure the pivot element is the largest in the column, which improves numerical stability.

4.4.3 Gaussian Elimination with Complete Pivoting

Complete Pivoting involves swapping both rows and columns to ensure the pivot element is the largest in the remaining submatrix, which further improves numerical stability.

4.4.4 Back Substitution

After transforming the matrix A into an upper triangular form, back substitution is used to solve for the unknowns.

Algorithm 4.1 Gaussian Elimination without Pivoting

Require: A matrix A and a vector b such that $A\mathbf{x} = b$

Ensure: Solution vector $\mathbf{x}$

```
1: for  $k = 1$  to  $n-1$  do
2:   for  $i = k+1$  to  $n$  do
3:     Compute the multiplier  $m_{ik} = \frac{A_{ik}}{A_{kk}}$ 
4:     for  $j = k$  to  $n$  do
5:       Update  $A_{ij} = A_{ij} - m_{ik}A_{kj}$ 
6:     end for
7:     Update  $b_i = b_i - m_{ik}b_k$ 
8:   end for
9: end for
10: for  $i = n$  down to  $1$  do
11:   Compute  $x_i = \frac{b_i - \sum_{j=i+1}^n A_{ij}x_j}{A_{ii}}$ 
12: end for
13: return  $\mathbf{x}$ 
```

Algorithm 4.2 Gaussian Elimination with Partial Pivoting

Require: A matrix A and a vector b such that $A\mathbf{x} = b$

Ensure: Solution vector $\mathbf{x}$

```
1: for  $k = 1$  to  $n-1$  do
2:   Find the index  $p$  of the maximum element in the column  $k$  from row  $k$  to  $n$ 
3:   Swap rows  $k$  and  $p$  in  $A$  and  $b$ 
4:   for  $i = k+1$  to  $n$  do
5:     Compute the multiplier  $m_{ik} = \frac{A_{ik}}{A_{kk}}$ 
6:     for  $j = k$  to  $n$  do
7:       Update  $A_{ij} = A_{ij} - m_{ik}A_{kj}$ 
8:     end for
9:     Update  $b_i = b_i - m_{ik}b_k$ 
10:   end for
11: end for
12: for  $i = n$  down to  $1$  do
13:   Compute  $x_i = \frac{b_i - \sum_{j=i+1}^n A_{ij}x_j}{A_{ii}}$ 
14: end for
15: return  $\mathbf{x}$ 
```

Algorithm 4.3 Gaussian Elimination with Complete Pivoting

Require: A matrix A and a vector b such that $A\mathbf{x} = b$

Ensure: Solution vector $\mathbf{x}$

```
1: Initialize permutation vectors  $P$  and  $Q$  to track row and column swaps
2: for  $k = 1$  to  $n-1$  do
3:   Find the index  $(p, q)$  of the maximum element in the submatrix  $A_{k:n, k:n}$ 
4:   Swap rows  $k$  and  $p$ , and columns  $k$  and  $q$  in  $A$ 
5:   Swap elements  $b_k$  and  $b_p$ 
6:   Update permutation vectors  $P$  and  $Q$ 
7:   for  $i = k+1$  to  $n$  do
8:     Compute the multiplier  $m_{ik} = \frac{A_{ik}}{A_{kk}}$ 
9:     for  $j = k$  to  $n$  do
10:      Update  $A_{ij} = A_{ij} - m_{ik}A_{kj}$ 
11:    end for
12:    Update  $b_i = b_i - m_{ik}b_k$ 
13:  end for
14: end for
15: for  $i = n$  down to  $1$  do
16:   Compute  $x_i = \frac{b_i - \sum_{j=i+1}^n A_{ij}x_j}{A_{ii}}$ 
17: end for
18: Apply the inverse permutations from  $P$  and  $Q$  to the solution vector  $\mathbf{x}$ 
19: return  $\mathbf{x}$ 
```

Algorithm 4.4 Back Substitution

Require: An upper triangular matrix U and a vector b

Ensure: Solution vector $\mathbf{x}$

```
1: for  $i = n$  down to  $1$  do
2:   Compute  $x_i = \frac{b_i - \sum_{j=i+1}^n U_{ij}x_j}{U_{ii}}$ 
3: end for
4: return  $\mathbf{x}$ 
```

4.5 LU Decomposition

LU Decomposition factorizes a matrix A into the product of a lower triangular matrix L and an upper triangular matrix U . This decomposition simplifies solving systems of linear equations.

Algorithm 4.5 LU Decomposition

Require: A matrix A such that $A = LU$

Ensure: Lower triangular matrix L and upper triangular matrix U

```
1: for  $k = 1$  to  $n$  do
2:   for  $i = k$  to  $n$  do
3:     Compute  $U_{ki} = A_{ki} - \sum_{j=1}^{k-1} L_{kj}U_{ji}$ 
4:   end for
5:   for  $i = k+1$  to  $n$  do
6:     Compute  $L_{ik} = \frac{A_{ik} - \sum_{j=1}^{k-1} L_{ij}U_{jk}}{U_{kk}}$ 
7:   end for
8: end for
9: return  $L$  and  $U$ 
```

4.5.1 Gaussian Elimination with Simulated Row Exchanges

Gaussian Elimination is a method for solving systems of linear equations. It transforms a given system into an upper triangular form using a sequence of row operations. Simulated row exchanges, also known as partial pivoting, enhance the numerical stability of the algorithm by keeping track of row permutations instead of physically swapping rows. This is especially useful in the context of $PA = LU$ decomposition.

Algorithm

The Gaussian Elimination algorithm with simulated row exchanges can be described as follows:

Example

Consider the system of equations:

$$2x + 3y + z = 1$$

$$4x + y + 2z = 2$$

$$3x + 2y + 3z = 3$$

Step-by-step Gaussian Elimination with simulated row exchanges:

1. Initial matrix and vector:

$$A = \begin{pmatrix} 2 & 3 & 1 \\ 4 & 1 & 2 \\ 3 & 2 & 3 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}, \quad P = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}$$

2. For $k = 1$, find the pivot element:

$$\text{pivot} = \max(|4|, |2|, |3|) = 4$$

Pivot row index is 2. Swap permutation entries $P[1]$ and $P[2]$:

$$P = \begin{pmatrix} 2 \\ 1 \\ 3 \end{pmatrix}$$

Algorithm 4.6 Gaussian Elimination with Simulated Row Exchanges

Require: A square matrix A of size $n \times n$ and a vector b of size n

Ensure: The solution vector x of the system $Ax = b$

1: Initialize the permutation array P to $[1, 2, \dots, n]$

2: **for** $k = 1$ **to** $n-1$ **do**

3: Find the pivot element:

$$\text{pivot} = \max_{i=k}^n |A_{P[i],k}|$$

4: Find the row r containing the pivot element and swap the permutation entries $P[k]$ and $P[r]$

5: **for** $i = k+1$ **to** n **do**

6: Compute the multiplier:

$$m_{P[i],k} = \frac{A_{P[i],k}}{A_{P[k],k}}$$

7: **for** $j = k+1$ **to** n **do**

8: Update the matrix A :

$$A_{P[i],j} = A_{P[i],j} - m_{P[i],k} A_{P[k],j}$$

9: **end for**

10: Update the vector b :

$$b_{P[i]} = b_{P[i]} - m_{P[i],k} b_{P[k]}$$

11: **end for**

12: **end for**

13: Perform back substitution to find the solution vector x :

14: Set $x_n = \frac{b_{P[n]}}{A_{P[n],n}}$

15: **for** $i = n-1$ **down to** 1 **do**

16: Compute:

$$x_i = \frac{b_{P[i]} - \sum_{j=i+1}^n A_{P[i],j} x_j}{A_{P[i],i}}$$

17: **end for**

18: **return** x

3. Eliminate the entries below the pivot element: For $i = 2$:

$$m_{P[2],1} = \frac{A_{1,1}}{A_{2,1}} = \frac{2}{4} = 0.5$$

Update row 2 (represented by $P[2] = 1$):

$$A_1 = A_1 - 0.5 \times A_2 = \begin{pmatrix} 2 & 3 & 1 \end{pmatrix} - 0.5 \times \begin{pmatrix} 4 & 1 & 2 \end{pmatrix} = \begin{pmatrix} 0 & 2.5 & 0 \end{pmatrix}$$

Update b :

$$b_{P[2]} = b_1 - 0.5 \times b_2 = 1 - 0.5 \times 2 = 0$$

For $i = 3$:

$$m_{P[3],1} = \frac{A_{3,1}}{A_{2,1}} = \frac{3}{4} = 0.75$$

Update row 3 (represented by $P[3] = 3$):

$$A_3 = A_3 - 0.75 \times A_2 = \begin{pmatrix} 3 & 2 & 3 \end{pmatrix} - 0.75 \times \begin{pmatrix} 4 & 1 & 2 \end{pmatrix} = \begin{pmatrix} 0 & 1.25 & 1.5 \end{pmatrix}$$

Update b :

$$b_{P[3]} = b_3 - 0.75 \times b_2 = 3 - 0.75 \times 2 = 1.5$$

4. For $k = 2$, find the pivot element:

$$\text{pivot} = \max(|2.5|, |1.25|) = 2.5$$

No need to swap rows. Eliminate the entry below the pivot element:

$$m_{P[3],2} = \frac{A_{3,2}}{A_{1,2}} = \frac{1.25}{2.5} = 0.5$$

Update row 3:

$$A_3 = A_3 - 0.5 \times A_1 = \begin{pmatrix} 0 & 1.25 & 1.5 \end{pmatrix} - 0.5 \times \begin{pmatrix} 0 & 2.5 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 1.5 \end{pmatrix}$$

Update b :

$$b_{P[3]} = b_3 - 0.5 \times 0 = 1.5$$

5. Perform back substitution:

$$x_3 = \frac{b_{P[3]}}{A_{P[3],3}} = \frac{1.5}{1.5} = 1$$

$$x_2 = \frac{b_{P[2]} - A_{P[2],3}x_3}{A_{P[2],2}} = \frac{0 - 0 \times 1}{2.5} = 0$$

$$x_1 = \frac{b_{P[1]} - A_{P[1],2}x_2 - A_{P[1],3}x_3}{A_{P[1],1}} = \frac{2 - 1 \times 0 - 2 \times 1}{4} = 0$$

Thus, the solution vector is:

$$x = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

Gaussian Elimination with simulated row exchanges (partial pivoting) helps improve the stability and accuracy of the solution, particularly for matrices that are close to singular or have large variations in their elements.

4.5.2 PA=LU Decomposition

The $PA = LU$ decomposition is an extension of the LU decomposition that includes partial pivoting to improve numerical stability. In this decomposition, P is a permutation matrix, L is a lower triangular matrix with unit diagonal elements, and U is an upper triangular matrix such that $PA = LU$.

Algorithm 4.7 PA=LU Decomposition

Require: A square matrix A of size $n \times n$

Ensure: Matrices P , L , and U such that $PA = LU$

1: Initialize P to the identity matrix of size $n \times n$

2: Initialize L to a zero matrix of size $n \times n$

3: Initialize U to the matrix A

4: **for** $k = 1$ **to** $n-1$ **do**

5: Find the pivot element:

$$\text{pivot} = \max_{i=k}^n |U_{i,k}|$$

6: Find the row r containing the pivot element and swap rows k and r in U

7: Swap rows k and r in P

8: Swap rows k and r in L (only columns 1 to $k-1$)

9: **for** $i = k+1$ **to** n **do**

10: Compute the multiplier:

$$L_{i,k} = \frac{U_{i,k}}{U_{k,k}}$$

11: **for** $j = k$ **to** n **do**

12: Update the matrix U :

$$U_{i,j} = U_{i,j} - L_{i,k}U_{k,j}$$

13: **end for**

14: **end for**

15: **end for**

16: Set the diagonal elements of L to 1

17: **return** Matrices P , L , and U

Algorithm

The $PA = LU$ decomposition algorithm can be described as follows:

Example

Consider the matrix:

$$A = \begin{pmatrix} 2 & 3 & 1 \\ 4 & 1 & 2 \\ 3 & 2 & 3 \end{pmatrix}$$

Step-by-step $PA = LU$ decomposition:

1. Initial matrices:

$$P = I = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad L = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}, \quad U = \begin{pmatrix} 2 & 3 & 1 \\ 4 & 1 & 2 \\ 3 & 2 & 3 \end{pmatrix}$$

2. For $k = 1$, find the pivot element:

$$\text{pivot} = \max(|2|, |4|, |3|) = 4$$

Pivot row index is 2. Swap rows 1 and 2 in U and P :

$$P = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad U = \begin{pmatrix} 4 & 1 & 2 \\ 2 & 3 & 1 \\ 3 & 2 & 3 \end{pmatrix}$$

3. Eliminate the entries below the pivot element: For $i = 2$:

$$L_{2,1} = \frac{U_{2,1}}{U_{1,1}} = \frac{2}{4} = 0.5$$

Update row 2 of U :

$$U_{2,j} = U_{2,j} - 0.5 \times U_{1,j} \quad \text{for } j = 1, 2, 3$$

$$U_2 = (2 \quad 3 \quad 1) - 0.5 \times (4 \quad 1 \quad 2) = (0 \quad 2.5 \quad 0)$$

For $i = 3$:

$$L_{3,1} = \frac{U_{3,1}}{U_{1,1}} = \frac{3}{4} = 0.75$$

Update row 3 of U :

$$U_{3,j} = U_{3,j} - 0.75 \times U_{1,j} \quad \text{for } j = 1, 2, 3$$

$$U_3 = (3 \quad 2 \quad 3) - 0.75 \times (4 \quad 1 \quad 2) = (0 \quad 1.25 \quad 1.5)$$

4. For $k = 2$, find the pivot element:

$$\text{pivot} = \max(|2.5|, |1.25|) = 2.5$$

No need to swap rows. Eliminate the entry below the pivot element:

$$L_{3,2} = \frac{U_{3,2}}{U_{2,2}} = \frac{1.25}{2.5} = 0.5$$

Update row 3 of U :

$$U_{3,j} = U_{3,j} - 0.5 \times U_{2,j} \quad \text{for } j = 2, 3$$

$$U_3 = (0 \quad 1.25 \quad 1.5) - 0.5 \times (0 \quad 2.5 \quad 0) = (0 \quad 0 \quad 1.5)$$

5. Set the diagonal elements of L to 1:

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 0.5 & 1 & 0 \\ 0.75 & 0.5 & 1 \end{pmatrix}$$

Thus, the matrices P , L , and U are:

$$P = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad L = \begin{pmatrix} 1 & 0 & 0 \\ 0.5 & 1 & 0 \\ 0.75 & 0.5 & 1 \end{pmatrix}, \quad U = \begin{pmatrix} 4 & 1 & 2 \\ 0 & 2.5 & 0 \\ 0 & 0 & 1.5 \end{pmatrix}$$

This shows that $PA = LU$, where P is the permutation matrix, L is the lower triangular matrix, and U is the upper triangular matrix.

4.6 Normed Vector Spaces and the Condition Number

A vector space V can be endowed with a variety of norms, i.e., functions $\|\cdot\| : V \rightarrow \mathbb{R}$ satisfying these properties:

1. $\|x\| \geq 0$ and $\|x\| = 0$ iff $x = 0$ (**non-negativity** and **non-degeneracy**);
2. If a is a scalar then $\|a x\| = |a| \|x\|$ (**homogeneity**);
3. $\|x + y\| \leq \|x\| + \|y\|$ (**triangle inequality**).

We say that $(V, \|\cdot\|)$ is a *normed space*.

The Euclidean space $V = \mathbb{R}^n$ may also be endowed with a multitude of useful norms, but the most common is the ℓ^p -norm:

$$\|x\|_p = \left(\sum_{j=1}^n |x_j|^p \right)^{1/p}$$

and the most common choice of p is 1, 2, ∞ , where $p = \infty$ is the limit:

$$\|x\|_\infty = \max_{j: 1 \leq j \leq n} |x_j|.$$

Given normed spaces $(V, \|\cdot\|_V)$ and $(W, \|\cdot\|_W)$, we define the *operator norm* of a linear mapping $A : V \rightarrow W$, where V, W are vector spaces, as

$$\|A\|_{V,W} = \sup_{x \in V \setminus \{0\}} \frac{\|Ax\|_W}{\|x\|_V}.$$

Using the subscripts V and W disambiguates the norms in use. Even if $V = W$, two distinct norms may be used in this definition.

4.6.1 On Frobenius norm

The *Frobenius norm* of a matrix A is defined by

$$\|A\|_F = \sqrt{\sum_{i,j} |a_{ij}|^2}$$

We compare this to the *Euclidean norm* of a vector. $\|x\|_2 = \left(\sum_j |x_j|^2 \right)^{1/2}$.

It is obvious that this norm is the usual Euclidean norm of a column vector obtained by *stacking the columns* of A . It is also the Euclidean norm of a row-vector resulting from *linearizing* the matrix, i.e., writing its rows horizontally, next to each other.

The norm of a matrix A with respect to the Euclidean norm.

As $\|A\|$ is hard to find, as is $\sqrt{\lambda_{\max}(A^\top A)}$, the square root of maximal eigenvalue of $A^\top A$, we often use the inequality:

$$\|A\| \leq \|A\|_F$$

to estimate the Euclidean norm of the matrix A .

4.6.2 Condition number of a matrix

Definition 2 (Condition number of a matrix). Let $A \in L(\mathbb{R}^n, \mathbb{R}^m) = \mathbb{R}^{m \times n}$ be an $m \times n$ real matrix ($\mathbb{R}$ can be replaced with $\mathbb{C}$ with no changes). The condition number is:

$$K(A) = \|A\| \|A^{-1}\|$$

where $\|A\|$ is the *matrix (operator) norm* of A :

$$\|A\| = \sup_{x \neq 0} \frac{\|Ax\|}{\|x\|}.$$

□

4.6.3 The fundamental inequality

Using the condition number we can evaluate the impact of the error in the matrix and in the right-hand side of the linear system $Ax = b$ on the solution x .

Theorem 4.1 (Relative error of linear equation solving). If we have an approximated linear system

$$(A + \delta A)(x + \delta x) = b + \delta b$$

then

$$\frac{\|\delta x\|}{\|x\|} \leq \frac{K(A)}{1 - K(A) \frac{\|\delta A\|}{\|A\|}} \left(\frac{\|\delta b\|}{\|b\|} + \frac{\|\delta A\|}{\|A\|} \right). \quad (4.1)$$

□

Proof. Using $Ax = b$ and other manipulations we obtain:

$$\begin{aligned} 0 &= (A + \delta A)(x + \delta x) - b - \delta b \\ &= Ax + \delta A x + A \delta x + \delta A \delta x - b - \delta b \\ &= \delta A x + A \delta x + \delta A \delta x - \delta b, \\ (A + \delta A)\delta x &= \delta b - \delta A x, \\ (I + A^{-1}\delta A)\delta x &= A^{-1}\delta b - A^{-1}\delta A x \\ (1 - \|A^{-1}\delta A\|)\|\delta x\| &\leq \|A^{-1}\|\|\delta b\| + \|A^{-1}\delta A\|\|x\|, \\ \frac{\|\delta x\|}{\|x\|} &\leq \frac{1}{1 - \|A^{-1}\delta A\|} \left(\|A^{-1}\| \frac{\|\delta b\|}{\|x\|} + \|A^{-1}\delta A\| \right). \end{aligned}$$

We also have $\|b\| = \|Ax\| \leq \|A\|\|x\|$, so $\|x\| \geq \|A\|^{-1}\|b\|$. Hence, substituting this bound for $\|x\|$ we obtain:

$$\frac{\|\delta x\|}{\|x\|} \leq \frac{1}{1 - \|A^{-1}\delta A\|} \left(\|A^{-1}\| \|A\| \frac{\|\delta b\|}{\|b\|} + \|A^{-1}\delta A\| \right).$$

Finally,

$$\|A^{-1}\delta A\| \leq \|A^{-1}\|\|\delta A\| = \|A^{-1}\| \|A\| \frac{\|\delta A\|}{\|A\|} = K(A) \frac{\|\delta A\|}{\|A\|}.$$

Now routine manipulations conclude the proof. □

4.6.4 Exercises

Exercise 4.1 (Basic Condition Number Calculation). Let A be the matrix

$$A = \begin{pmatrix} 4 & 1 \\ 2 & 3 \end{pmatrix}.$$

1. Compute the Euclidean (2-norm) condition number of A .
2. Compute the Frobenius norm condition number of A .

□

Exercise 4.2 (Sensitivity of Solutions). Consider the linear system $Ax = b$, where

$$A = \begin{pmatrix} 3 & 1 \\ 1 & 2 \end{pmatrix}, \quad b = \begin{pmatrix} 5 \\ 5 \end{pmatrix}.$$

1. Solve the system for x .
2. Now, suppose the matrix A is perturbed by a small amount:

$$\delta A = \begin{pmatrix} 0.01 & 0 \\ 0 & 0.01 \end{pmatrix}.$$

Using the condition number, estimate the relative error in the solution due to this perturbation.

□

4.7 Least Squares Approximation and Hilbert Matrices

The goal of least squares approximation is to approximate a given function $f(x)$ over an interval $[a, b]$ by a polynomial $p(x)$ of degree n :

$$p(x) = c_0 + c_1x + c_2x^2 + \cdots + c_nx^n.$$

4.7.1 Least Squares Criterion

To find the best approximation, we minimize the squared error:

$$\min_{c_0, c_1, \dots, c_n} \int_a^b (f(x) - p(x))^2 dx.$$

This integral represents the squared difference between $f(x)$ and $p(x)$ over the interval $[a, b]$, and minimizing it determines the coefficients $\{c_k\}$ of the polynomial that provides the best least squares approximation.

4.7.2 Normal Equations

The least squares condition requires that the error be orthogonal to the space of polynomials of degree n . This leads to setting the partial derivatives of the error functional with respect to each coefficient c_k to zero:

$$\frac{\partial}{\partial c_k} \int_a^b (f(x) - p(x))^2 dx = 0, \quad \text{for } k = 0, 1, \dots, n.$$

These conditions yield the *normal equations*:

$$\sum_{j=0}^n c_j \int_a^b x^{i+j} dx = \int_a^b f(x) x^i dx, \quad \text{for } i = 0, 1, \dots, n.$$

4.7.3 Matrix Formulation

Let's introduce the *moment matrix* M , whose entries are the moments of the monomials over the interval $[a, b]$:

$$M_{ij} = \int_a^b x^{i+j} dx.$$

This matrix M is symmetric and its size is $(n+1) \times (n+1)$. The right-hand side vector $\mathbf{f}$ contains the projections of $f(x)$ onto the monomials:

$$\mathbf{f}_i = \int_a^b f(x)x^i dx, \quad \text{for } i = 0, 1, \dots, n.$$

The normal equations can be written in matrix form as:

$$M\mathbf{c} = \mathbf{f},$$

where $\mathbf{c}$ is the vector of coefficients $[c_0, c_1, \dots, c_n]^\top$.

4.7.4 Hilbert Matrix Connection

When the interval is $[0, 1]$, the entries of the moment matrix M are given by:

$$M_{ij} = \int_0^1 x^{i+j} dx = \frac{1}{i+j+1}.$$

This matrix is known as the *Hilbert matrix* H , with entries:

$$H_{ij} = \frac{1}{i+j+1}, \quad \text{for } i, j = 0, 1, \dots, n.$$

This formula assumes that the matrices are *zero-based*, like in programming languages C and Python. MATLAB arrays are *one-based* and the Hilbert matrix formula is modified accordingly.

$$H_{ij} = \frac{1}{i+j-1}, \quad \text{for } i, j = 1, 2, \dots, n.$$

Hilbert matrices are known for being notoriously ill-conditioned, making the least squares approximation problem numerically challenging, especially as the degree n of the polynomial increases. This ill-conditioning is a consequence of the closely spaced *singular values* of the Hilbert matrix, which make solving the normal equations sensitive to errors in the data or numerical computations.

Exercise 4.3 (Condition Number of a Hilbert Matrix). The Hilbert matrix H_n of size $n \times n$ is defined by:

$$H_n(i, j) = \frac{1}{i+j-1}.$$

1. Write down the Hilbert matrix H_3 explicitly.
2. Compute the condition number of H_3 and H_5 in the 2-norm.
3. Comment on how the condition number changes as the matrix size n increases.

□

Exercise 4.4 (Hilbert Matrix in Least Squares Problems). Consider the overdetermined system $H_6x = b$, where H_6

is the 6×6 Hilbert matrix, and $b = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}$.

1. Compute the least squares solution x .
2. What is the condition number of H_6 , and how does it affect the accuracy of the least squares solution?
3. Compare this with the solution to the same problem using a 6×6 random matrix (with entries uniformly distributed between 0 and 1) instead of the Hilbert matrix.

□

Exercise 4.5 (Exploring the Condition Number of Larger Hilbert Matrices).

1. Generate Hilbert matrices of sizes 10×10 , 20×20 , and 50×50 .
2. Compute the condition numbers of these matrices in the 2-norm.
3. How does the condition number behave as the matrix size increases?
4. Discuss why Hilbert matrices are considered ill-conditioned and how this affects numerical computations involving these matrices.

□

Exercise 4.6 (Condition Number and Random Matrices).

1. Generate a random 3×3 matrix A with entries from a uniform distribution in the range $[-1, 1]$.
2. Compute the condition number of this random matrix and compare it to the condition number of a 3×3 Hilbert matrix.
3. Repeat the process for larger matrix sizes, such as 5×5 and 10×10 .
4. Discuss how the condition numbers of Hilbert matrices compare to random matrices and how this affects the stability of linear systems involving these matrices.

□

Exercise 4.7 (Ill-conditioned Matrix). Consider the matrix

$$A = \begin{pmatrix} 1 & 1.0001 \\ 1 & 1 \end{pmatrix}.$$

1. Compute the condition number of A in the Euclidean norm.
2. Solve $Ax = b$, where $b = \begin{pmatrix} 2 \\ 2 \end{pmatrix}$, and compare the sensitivity of the solution to small changes in b .
3. Comment on how the condition number affects the reliability of the solution.

□

Exercise 4.8 (Using Condition Numbers to Assess Numerical Stability). Consider the matrix $A = \begin{pmatrix} 1 & 2 \\ 2 & 3 \end{pmatrix}$ and solve the system $Ax = b$ for $b = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$.

1. Compute the condition number of A .
2. Now, suppose there is an error in b such that $b_{\text{perturbed}} = \begin{pmatrix} 1.001 \\ 1.001 \end{pmatrix}$. Solve the system for the perturbed b .
3. Compare the solutions for the original and perturbed b , and explain how the condition number affects the sensitivity of the solution.

□

4.8 Iterative Methods

Iterative methods are used to solve large systems of linear equations by iteratively refining an initial guess. These methods are particularly useful for sparse or structured matrices.

4.8.1 Jacobi Method and Spectral Radius

- Split A into
 - D - the diagonal part
 - L - the lower-triangular part
 - U - the upper-triangular part
- Transform the equation to fixed-point form:

$$\begin{aligned}
 Ax = b &\implies (D + L + U)x = b \\
 &\implies Dx = b - (L + U)x \\
 &\implies x = D^{-1}b - D^{-1}(L + U)x
 \end{aligned}$$

4.8.2 Analysis of the Fixed-Point Equation

Consider the fixed-point equations $x = c - Bx$ leading to

$$x^{(n+1)} = c - Bx^{(n)}$$

First few iterations:

$$\begin{aligned}
 x^{(0)} &= 0, \\
 x^{(1)} &= c, \\
 x^{(2)} &= c - Bc, \\
 x^{(3)} &= c - B(c - Bc) = c - Bc + B^2c.
 \end{aligned}$$

We deduce the general form:

$$x^{(n)} = \sum_{k=0}^{n-1} (-1)^k B^k c$$

We propose the solution:

$$x^{(\infty)} = \sum_{k=0}^{\infty} (-1)^k B^k c = \left(\sum_{k=0}^{\infty} (-1)^k B^k \right) c$$

The matrix (operator) series is the *Neumann series*:

$$(I + B)^{-1} = \sum_{k=0}^{\infty} (-1)^k B^k, \quad (I - B)^{-1} = \sum_{k=0}^{\infty} B^k$$

The sufficient condition of convergence is that the *spectrum* (the set of eigenvalues) of B lie within the unit circle:

$$\sigma(B) \subseteq \{\lambda \in \mathbb{C} : |\lambda| < 1\}$$

The *spectral radius* is defined as

$$\rho(B) = \max\{|\lambda| : \lambda \in \sigma(B)\}$$

It can be shown that $\rho(B)$ is the rate of convergence of this series:

$$\rho(B) = \lim_{n \rightarrow \infty} \|B^n\|^{1/n} = \inf_{n \geq 0} \|B^n\|^{1/n}$$

The condition of convergence is: $\rho(B) < 1$.

Algorithm 4.8 Jacobi Method

Require: A matrix A , a vector b , and an initial guess $\mathbf{x}^{(0)}$

Ensure: Solution vector $\mathbf{x}$

```
1: for  $k = 0$  to  $\text{max\_iter}$  do
2:   for  $i = 1$  to  $n$  do
3:     Compute  $x_i^{(k+1)} = \frac{1}{A_{ii}} \left( b_i - \sum_{j \neq i} A_{ij} x_j^{(k)} \right)$ 
4:   end for
5:   if convergence criterion is met then
6:     break
7:   end if
8: end for
9: return  $\mathbf{x}$ 
```

4.8.3 Jacobi Method

The Jacobi Method is an iterative algorithm for solving a system of linear equations. Each iteration updates the solution based on the values from the previous iteration.

4.8.4 Gauss-Seidel Method

The Gauss-Seidel Method given as Algorithm 4.9 is an improvement over the Jacobi Method that uses the most recent updates for faster convergence. In terms of matrix algebra, the Gauss-Seidel Method can be explained as the following transformation of the equation $Ax = b$ to a fixed-point form:

$$\begin{aligned} Ax = b &\implies (D + L + U)x = b \\ &\implies (D + L)x = b - Ux \\ &\implies x = (D + L)^{-1}(b - Ux) \end{aligned}$$

As this is again an equation of the form $x = c - Bx$, the condition of convergence is $\rho(B) < 1$, or more explicitly

$$\rho((D + L)^{-1}U) < 1.$$

As $D + L$ is a lower-triangular matrix, its inversion is implemented on the fly, and it has the effect of using the most recent updates of the components of x .

Algorithm 4.9 Gauss-Seidel Method

Require: A matrix A , a vector b , and an initial guess $\mathbf{x}^{(0)}$

Ensure: Solution vector $\mathbf{x}$

```
1: for  $k = 0$  to  $\text{max\_iter}$  do
2:   for  $i = 1$  to  $n$  do
3:     Compute  $x_i^{(k+1)} = \frac{1}{A_{ii}} \left( b_i - \sum_{j < i} A_{ij} x_j^{(k+1)} - \sum_{j > i} A_{ij} x_j^{(k)} \right)$ 
4:   end for
5:   if convergence criterion is met then
6:     break
7:   end if
8: end for
9: return  $\mathbf{x}$ 
```

4.8.5 Successive Over-Relaxation (SOR)

The Successive Over-Relaxation Method (SOR), presented as Algorithm 4.10 is an extension of the Gauss-Seidel Method that accelerates convergence by introducing a relaxation factor. The equation $Ax = b$ is re-written as

$$(D + \omega L)x = \omega b - [\omega U + (\omega - 1)D]x. \quad (4.2)$$

The form of this equation may be puzzling at first, but the main idea is splitting D between the left- and right-hand side of the equation. We write

$$D = \frac{1}{\omega}D + \left(1 - \frac{1}{\omega}\right)D.$$

This leads to rewriting $(L + D + U)x = b$ as

$$\left(L + \frac{1}{\omega}D\right)x = b - \left[\left(1 - \frac{1}{\omega}\right)D + U\right]x$$

Multiplying by ω we obtain equation (4.2).

The spectral condition of convergence is

$$\rho((D + \omega L)^{-1}[\omega U + (\omega - 1)D]) < 1.$$

The crucial feature of the algorithm is that the parameter ω is *tunable*. The best value of ω yields the fastest convergence and is obtained by minimizing the spectral radius. This can be achieved either experimentally or theoretically, as several papers address the issue of determining optimum value of ω . For most applications $\omega \in (1, 2)$.

Algorithm 4.10 Successive Over-Relaxation (SOR)

Require: A matrix A , a vector b , an initial guess $\mathbf{x}^{(0)}$, and a relaxation factor ω

Ensure: Solution vector $\mathbf{x}$

```

1: for k = 0 to max_iter do
2:   for i = 1 to n do
3:     Compute  $x_i^{(k+1)} = (1 - \omega)x_i^{(k)} + \frac{\omega}{A_{ii}} \left( b_i - \sum_{j < i} A_{ij}x_j^{(k+1)} - \sum_{j > i} A_{ij}x_j^{(k)} \right)$ 
4:   end for
5:   if convergence criterion is met then
6:     break
7:   end if
8: end for
9: return  $\mathbf{x}$ 
```

4.9 Poisson Equation with a Point Charge

Consider the two-dimensional Poisson equation defined on a square domain $\Omega = [0, 1] \times [0, 1]$:

$$-\Delta u(x, y) = f(x, y),$$

where $\Delta u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}$ is the Laplace operator, and $f(x, y)$ is a source term.

4.9.1 On the sign of the Laplace operator

In many texts Poisson equation is written as $\Delta u(x, y) = f(x, y)$, i.e., there is no “minus” on Δ . We will follow other texts partial toward the operator $-\Delta$, as it is *positive definite*. To show this fact, we need the *First Greene’s Identity*, specialized to 2 dimensions:

$$\iint_{\Omega} (u \Delta v + \nabla u \cdot \nabla v) dA = \oint_{\partial\Omega} u(\nabla v \cdot \mathbf{n}) d\ell = \oint_{\partial\Omega} u \nabla v \cdot d\mathbf{t}$$

where $dA = dx dy$ is the *area element*, where $d\ell$ is the *length element* along the boundary, and where $\mathbf{n}$ is the normal vector to the boundary. The *vector length element* is defined by

$$d\ell = \mathbf{n} \cdot d\mathbf{\ell}$$

and is *normal* to the boundary.

Therefore,

$$\iint_{\Omega} (-\Delta u) \cdot u \, dA = \iint_{\Omega} \|\nabla u\|^2 \, dA > 0,$$

provided that $u|_{\partial\Omega} = 0$, and $u \not\equiv 0$.

4.9.2 Point Charge Source Term

A physically relevant choice for $f(x, y)$ is a point charge located at a position (x_0, y_0) within Ω . This can be modeled by setting

$$f(x, y) = \delta(x - x_0, y - y_0),$$

where δ is the Dirac delta function, representing an infinitely concentrated source. The resulting solution $u(x, y)$ represents the potential field generated by the point charge.

4.9.3 Boundary Conditions

To ensure a well-posed problem, we impose homogeneous *Dirichlet boundary conditions*:

$$u(x, y) = 0 \quad \text{on} \quad \partial\Omega.$$

4.9.4 Discretization and Sparse Linear System

The continuous problem is discretized using a *finite difference scheme*. Let the domain Ω be divided into a *uniform grid* of spacing $h = \frac{1}{N+1}$ (see Figure 4.2). The discrete approximation of the (negative) Laplace operator (Laplacian) at each interior point (i, j) of the grid is given by a *finite difference approximation*

$$-\Delta u \approx \frac{4u_{i,j} - u_{i-1,j} - u_{i+1,j} - u_{i,j-1} - u_{i,j+1}}{h^2}.$$

This leads to a sparse linear system of equations:

$$A\mathbf{u} = \mathbf{f},$$

where A is a sparse matrix with a structure reflecting the 5-point stencil of the finite difference approximation.

4.9.5 Solution Using Iterative Methods

The resulting linear system can be efficiently solved using iterative methods such as Jacobi, Gauss-Seidel, or Successive Over-Relaxation (SOR), which exploit the sparsity of A . The point of interest in the solution is at (x_0, y_0) , where the point charge is located, leading to localized behavior in the potential field $u(x, y)$.

4.9.6 Exercises on solving Poisson Equation

In these exercises you will approximate the solution to the Poisson equation on the unit square $\Omega = [0, 1]^2$. You will consider the *discretized Poisson equation*:

$$\frac{4u_{i,j} - u_{i-1,j} - u_{i+1,j} - u_{i,j-1} - u_{i,j+1}}{h^2} = f_{i,j} \quad (4.3)$$

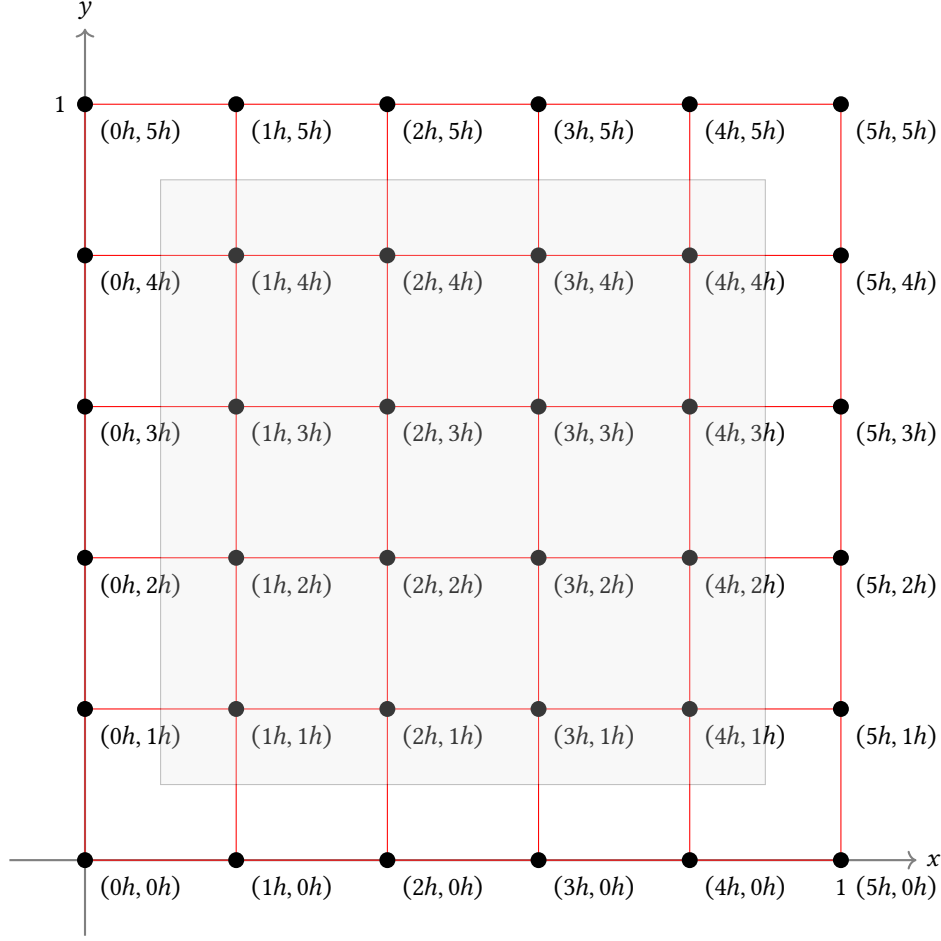


Figure 4.2: The grid for $N = 4$. The lightly shaded area is the portion of the grid to which we attach variables $u_{i,j}$. The remainder of the grid corresponds to the boundary $\partial\Omega$ and under Dirichlet boundary conditions, it does not participate in calculations.

where $u_{i,j}$ is the unknown potential at $x = ih$, $y = jh$, $1 \leq i, j \leq N$ and $h = 1/(N + 1)$. The Dirichlet boundary condition is implemented by imposing the constraints:

$$\begin{aligned} u_{0,j} = u_{N+1,j} &= 0, & \text{if } 0 \leq j \leq N + 1, \\ u_{i,0} = u_{i,N+1} &= 0, & \text{if } 0 \leq i \leq N + 1. \end{aligned}$$

We note that although the values $u_{i,j}$ along the boundary are known, they participate in the value of $\Delta(x, y)$ inside Ω . But since they are 0, we simply drop them.

As the first step, we must *flatten* or *linearize* the two-dimensional matrix $u_{i,j}$, representing it by a flat vector $\mathbf{u} \in \mathbb{R}^{N^2}$ where

$$u_{i,j} = u_{iN+j}, \quad 1 \leq i, j \leq N.$$

In this version, we lay out $\mathbf{u}$ in memory by the row, which works best with programming languages that naturally support this way of storing matrices (notably, C/C++ and Python). MATLAB and Fortran lay out matrices by the column, and the scheme for linearizing should be changed accordingly.

Similarly, the source function is represented by a flat vector $\mathbf{f}$:

$$f_{i,j} = f_{iN+j}, \quad 0 \leq i, j \leq N.$$

The left-hand side of the equation is of the form $A\mathbf{u} = \mathbf{f}$, where A is an $N^2 \times N^2$ matrix, with entries given by this equation

$$A_{iN+j,kN+l} = \begin{cases} \frac{4}{h^2} & \text{if } i = k \text{ and } j = l, \\ -\frac{1}{h^2} & \text{if } |i - k| = 1 \text{ and } j = l, \\ -\frac{1}{h^2} & \text{if } i = k \text{ and } |j - l| = 1, \\ 0 & \text{otherwise.} \end{cases}$$

The matrix A has a specific look when written out in full. E.g., for $N = 4$ (see Figure 4.2) the 16×16

$$A = \begin{pmatrix} a & b & . & . & b & . & . & . & . & . & . & . & . & . & . & . \\ b & a & b & . & . & b & . & . & . & . & . & . & . & . & . & . \\ . & b & a & b & . & . & b & . & . & . & . & . & . & . & . & . \\ . & . & b & a & . & . & . & b & . & . & . & . & . & . & . & . \\ b & . & . & . & a & b & . & . & b & . & . & . & . & . & . & . \\ . & b & . & . & b & a & b & . & . & b & . & . & . & . & . & . \\ . & . & b & . & . & b & a & b & . & . & b & . & . & . & . & . \\ . & . & . & b & . & . & b & a & . & . & b & . & . & . & . & . \\ . & . & . & . & b & . & . & . & a & b & . & . & b & . & . & . \\ . & . & . & . & . & b & . & . & b & a & b & . & . & b & . & . \\ . & . & . & . & . & . & b & . & . & . & a & b & . & . & . & b \\ . & . & . & . & . & . & . & b & . & . & . & a & b & . & . & . \\ . & . & . & . & . & . & . & . & b & . & . & b & a & b & . & . \\ . & . & . & . & . & . & . & . & . & b & . & . & b & a & b & . \\ . & . & . & . & . & . & . & . & . & . & b & . & . & b & a & b \end{pmatrix} \quad (4.4)$$

where we replaced the zero entries with “.” and used two constants

$$a = \frac{4}{h^2}, \quad \text{and} \quad b = -\frac{1}{h^2}$$

to avoid clutter.

This type of matrix is called a *band matrix*. Clearly, one identifies 3 bands, i.e., groups of diagonals, where the non-zero entries are positioned. The total number of diagonals is 5 which allows us to store this matrix as a $N^2 \times 5$ -dimensional, sparse matrix, reducing the number of entries from $(N^2)^2 = N^4$ to merely $5N^2$. This is important when N is large, but relatively modest, e.g., $N = 1000$, when storing the full matrix A is not possible on even a formidable computer, as it requires $\approx 10^{12} \times 8 \text{ bytes} = 8 \text{ terabytes}$ of memory to store. On the other hand, the sparse form requires $\approx 5 \cdot 10^6 \text{ bytes} = 5 \text{ megabytes}$ of memory, which is typically available in a laptop computer.

The matrix has a *block structure*, i.e., it is composed from smaller matrices. Let us define two $N \times N$ matrices:

$$J = \begin{pmatrix} 4 & -1 & . & . \\ -1 & 4 & -1 & . \\ . & -1 & 4 & -1 \\ . & . & -1 & 4 \end{pmatrix}, \quad S = \begin{pmatrix} -1 & . & . & . \\ . & -1 & . & . \\ . & . & -1 & . \\ . & . & . & 1 \end{pmatrix} = -I_4,$$

where I_4 is the 4×4 identity matrix. Let $I = I_4$. Then

$$A = \frac{1}{h^2} \begin{pmatrix} J & -I & . & . \\ -I & J & -I & . \\ . & -I & J & -I \\ . & . & -I & J \end{pmatrix} \quad (4.5)$$

is the block matrix representation of A . This construction generalizes to an arbitrary grid size N , where A is represented by an $N \times N$ block matrix with $N \times N$ blocks, with $N + 2(N - 1)$ non-zero blocks.

Exercise 4.9 (Positive definiteness of the discretized negative Laplacian). Prove that the matrix A is positive definite, by showing that for all $\mathbf{u} \in \mathbb{R}^{N^2}$



$$\mathbf{u}^T A \mathbf{u} \geq 0$$

and “=” only holds when $\mathbf{u} = \mathbf{0}$, by deriving a discrete version of the Greene’s first identity. $\square$

Exercise 4.10 (A dense matrix approach to Poisson Equation). Write a program that builds the matrix A and the vector b , using the standard, dense representation of matrices. Solve the system using Gaussian Elimination. Analyze the memory usage and run time of the algorithm as $N \rightarrow \infty$. $\square$

Exercise 4.11 (Iterative solution to Poisson Equation using sparse representation). Implement iterative methods for solving the Poisson equation. To implement an iterative method, you may implement matrix multiplication utilizing this representation. However, the recommended solution does not store the matrix A at all! The matrix A is replaced with a *function* that multiplies A by a given vector x . You will need only to store to constants: $a = 4/h^2$ and $b = -1/h^2$ where $h = 1/(N + 1)$. In linear algebra, A is identified with the linear operator it represents, and the recommended approach stays close to this idea. $\square$



Exercise 4.12 (Poisson Equation in 3D). The Laplacian in three dimensions is



$$\Delta u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2}.$$

1. Write down the discrete Laplacian on the cube $[0, 1]^3$ with an $N \times N \times N$ grid.
2. Describe in detail the matrix A such that $-\Delta u \approx Au$, where a function u is identified with a 1D column vector with N^3 entries.

$\square$

4.10 Eigenvalue Problems

Eigenvalue problems involve finding the eigenvalues and eigenvectors of a matrix. These are important in many applications, such as stability analysis and vibration analysis.

4.10.1 Power Method

The Power Method is an iterative algorithm for finding the dominant eigenvalue and its corresponding eigenvector of a matrix.

Algorithm 4.11 Power Method

Require: A square $N \times N$ matrix A and an initial vector $\mathbf{x}^{(0)}$

Ensure: Dominant eigenvalue λ and eigenvector $\mathbf{v}$

- 1: **for** $k = 0$ **to** max_iter **do**
 - 2: Compute $\mathbf{y} = A\mathbf{x}^{(k)}$
 - 3: Update $\mathbf{x}^{(k+1)} = \frac{\mathbf{y}}{\|\mathbf{y}\|}$
 - 4: Estimate the eigenvalue $\lambda = \mathbf{x}^{(k+1)T} A \mathbf{x}^{(k+1)}$
 - 5: **if** convergence criterion is met **then**
 - 6: **break**
 - 7: **end if**
 - 8: **end for**
 - 9: **return** λ and $\mathbf{v} = \mathbf{x}^{(k+1)}$
-

The power method is closely related to fixed point iteration. Consider the pair of transformations:

$$\begin{aligned}\mathcal{A} &: \mathbf{x} \mapsto \mathbf{y} = A\mathbf{x} \\ \mathcal{N} &: \mathbf{y} \mapsto \frac{\mathbf{y}}{\|\mathbf{y}\|}\end{aligned}$$

Let $\mathcal{F} = \mathcal{N} \circ \mathcal{A}$ be the composition of the two maps: a linear map $\mathcal{A}$ and the normalization map $\mathcal{N}$. The algorithm defines the sequence

$$\mathbf{x}^{(k+1)} = \mathcal{F}(\mathbf{x}^{(k)})$$

and the crucial point is that this sequence converges to a fixed point which is an eigenvector of A corresponding to the dominant eigenvalue.

Theorem 4.2 (Convergence of the Power Method). Let A be an $N \times N$ diagonalizable matrix, and let $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_N$ be the linearly independent, unit eigenvectors corresponding to (not necessarily distinct) eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_N$. Let

$$|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_N| \geq 0.$$

Let

$$\mathbf{x}^{(0)} \notin \text{span}\{\mathbf{v}_2, \mathbf{v}_3, \dots, \mathbf{v}_N\}.$$

Let $\mathbf{x}^{(k)} \in \mathbb{R}^N$ be the sequence defined by

$$\mathbf{x}^{(k+1)} = \mathcal{F}(\mathbf{x}^{(k)}), \quad k = 0, 1, \dots \quad (4.6)$$

Then:

1. $\mathbf{x}^{(k+1)}$ converges to $\mathbf{v}_1$;
2. The $\lambda^{(k)} = \mathbf{x}^{(k)T} A \mathbf{x}^{(k)}$ converges to λ_1 .

□

Proof. Let $\mathbf{x}^{(0)} = \sum_{j=1}^N c_j \mathbf{v}_j$. Then up to a constant factor $\mathbf{x}^{(k)} = \sum_{j=1}^N c_j \lambda_j^k \mathbf{v}_j$. Dividing by λ_1^k we obtain

$$\lambda_1^{-k} \mathbf{x}^{(k)} = c_1 \mathbf{v}_1 + \sum_{j=2}^N c_j \left(\frac{\lambda_j}{\lambda_1} \right)^k \mathbf{v}_j.$$

Therefore $\lim_{k \rightarrow \infty} \lambda_1^{-k} \mathbf{x}^{(k)} = c_1 \mathbf{v}_1$ where $c_1 \neq 0$. In particular, for large k

$$\lambda_1^{-2k} \mathbf{x}^{(k)T} A \mathbf{x}^{(k)} \approx c_1^2 \|\mathbf{v}_1\|^2 = c_1^2 \quad (4.7)$$

and

$$\lambda_1^{-2k} \mathbf{x}^{(k)T} A \mathbf{x}^{(k)} \approx c_1 \mathbf{v}_1^T c_1 \lambda_1 \mathbf{v}_1 = \lambda_1 c_1^2 \|\mathbf{v}_1\|^2 = \lambda_1 c_1^2. \quad (4.8)$$

Dividing (4.8) by (4.7) we obtain

$$\frac{\mathbf{x}^{(k)T} A \mathbf{x}^{(k)}}{\mathbf{x}^{(k)T} \mathbf{x}^{(k)}} \approx \lambda_1$$

and the difference goes away as $k \rightarrow \infty$. We also have $\mathbf{x}^{(k)T} \mathbf{x}^{(k)} = 1$, by definition. Therefore

$$\lim_{k \rightarrow \infty} \mathbf{x}^{(k)T} A \mathbf{x}^{(k)} = \lambda_1.$$

□

We note that convergence holds for *almost every* choice of the initial condition $\mathbf{x}^{(0)}$. The convergence is exponentially fast when there is a *spectral gap*:

$$\rho = \frac{\max_{j \in \{2, \dots, N\}} |\lambda_j|}{|\lambda_1|} < 1$$

The error size is $\sim \rho^n$. If there is a spectral gap, λ_1 is called the *dominant eigenvalue*.

If there is no dominant eigenvalue, i.e., when $|\lambda_1| = |\lambda_2|$, the Power Method may not converge. If $|\lambda_2| > |\lambda_3|$ and if the ratio $\frac{\lambda_2}{\lambda_1} = e^{i\theta}$ is not a real number, the method diverges. A more variant of the Power Method is presented in Section 4.15.

What if A is not diagonalizable, the reader may ask. The theorem still holds by replacing diagonalization with the *Jordan canonical form*.

Exercise 4.13 (Convergence of the Power Method for non-diagonalizable matrices). Generalize Theorem 4.2 to the case when A is non-diagonalizable. Recall that every matrix is similar to a matrix in a Jordan canonical form: a block form which looks like this:

$$A \sim \begin{pmatrix} J_1 & 0 & 0 & \cdots & 0 \\ 0 & J_2 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & 0 \\ 0 & 0 & \cdots & 0 & J_r \end{pmatrix}$$

where

$$J_m = \begin{pmatrix} \lambda_m & 1 & 0 & \cdots & 0 \\ 0 & \lambda_m & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \vdots & \lambda_m & 1 \\ 0 & 0 & \cdots & 0 & \lambda_m \end{pmatrix}.$$

An essential part of the proof is to find the form of J_m^k . $\square$

Limitations of the Power Method

- The Power Method only finds the eigenvalue with the largest magnitude.
- It may converge slowly if $|\lambda_1|$ and $|\lambda_2|$ are close in magnitude.

Deflation Technique for Finding the Second Largest Eigenvalue

Once the largest eigenvalue λ_1 is found using the Power Method, we can use a **deflation technique** to modify the matrix A and find the second largest eigenvalue λ_2 .

Deflation Procedure

1. **Let** $\mathbf{v}_1$ be the eigenvector corresponding to λ_1 .
2. **Modify the matrix** A by subtracting the contribution of λ_1 as follows:

$$A_1 = A - \lambda_1 \mathbf{v}_1 \mathbf{v}_1^\top.$$

3. **Apply the Power Method** to A_1 to find the largest eigenvalue of A_1 , which corresponds to the second largest eigenvalue λ_2 of A .

This technique effectively “removes” the effect of λ_1 from the matrix, allowing the Power Method to converge to λ_2 . The eigenvector of λ_1 becomes a part of the nullspace of A , $\mathcal{N}(A)$. Assuming WLOG that $\mathbf{v}_1^\top \mathbf{v}_1 = \|\mathbf{v}_1\| = 1$

$$A_1 \mathbf{v}_1 = A \mathbf{v}_1 - \lambda_1 \mathbf{v}_1 \mathbf{v}_1^\top \mathbf{v}_1 = \lambda_1 \mathbf{v}_1 - \lambda_1 \mathbf{v}_1 = 0.$$

Also, for $j = 2, \dots, N$.

$$A_1 v_j = A v_j - \lambda_1 v_1 v_1^\top v_j = \lambda_j v_j - \lambda_1 v_1^\top v_j v_1.$$

Thus, no longer v_j is an eigenvector (unless $v_1^\top v_j = 0$), but

$$v'_j = v_j - \frac{\lambda_1}{\lambda_j} v_1^\top v_j v_1$$

is an eigenvector of A_1 , thanks to the fact that not v_1 is in $\mathcal{N}(A_1)$. Indeed, by the above,

$$A_1 v'_j = A v_j = \lambda_j v'_j.$$

4.11 Introduction to QR Decomposition

In numerical linear algebra, matrix decompositions are powerful tools for simplifying complex problems. One essential decomposition is the *QR decomposition*, which factors a given matrix A into the product of an orthogonal matrix Q and an upper triangular matrix R . This section provides a thorough exploration of the QR decomposition, covering its theoretical foundations, numerical methods, and practical applications.

The primary objectives of this section are:

- To provide a rigorous explanation of the Gram-Schmidt orthogonalization method.
- To explore numerical stability and alternative methods such as Householder reflections and Givens rotations.
- To discuss the applications of QR decomposition in solving linear systems, least squares problems, and eigenvalue computations.

4.12 Definition and Notation

Let $A \in \mathbb{R}^{m \times n}$ with $m \geq n$. The *QR decomposition* of A is defined as:

$$A = QR,$$

where:

- $Q \in \mathbb{R}^{m \times m}$ is an orthogonal matrix, i.e., $Q^\top Q = I$, where I denotes the $m \times m$ identity matrix.
- $R \in \mathbb{R}^{m \times n}$ is an upper triangular matrix, i.e., $r_{ij} = 0$ for $i > j$.

This decomposition is of significant importance due to the stability properties of orthogonal matrices and the simplicity of working with triangular matrices.

4.13 Gram-Schmidt Orthogonalization

The **Gram-Schmidt orthogonalization** process is a classical method for transforming a set of linearly independent vectors into an orthonormal basis. Given a matrix $A = [a_1, a_2, \dots, a_n]$, where a_i are the linearly independent columns of A , the goal is to construct a set of orthonormal vectors $\{q_1, q_2, \dots, q_n\}$, which will form the columns of the matrix Q .

4.13.1 The Classical Gram-Schmidt (CGS) Process

The classical Gram-Schmidt (CGS) process works as follows:

★★★ **Step 1: Initialization** ★★★ Start with the first column of A :

$$q_1 = \frac{a_1}{\|a_1\|}.$$

*** **Step 2: Orthogonalization** *** For each subsequent column a_k , orthogonalize a_k against all previously computed q_i (for $i = 1, 2, \dots, k-1$):

$$\tilde{q}_k = a_k - \sum_{i=1}^{k-1} \langle q_i, a_k \rangle q_i.$$

*** **Step 3: Normalization** *** Normalize $\tilde{q}_k$ to obtain q_k :

$$q_k = \frac{\tilde{q}_k}{\|\tilde{q}_k\|}.$$

*** **Step 4: Construction of R** *** The entries of R are given by:

$$r_{ij} = \langle q_i, a_j \rangle, \quad \text{for } i \leq j.$$

While CGS is simple and intuitive, it suffers from numerical instability due to finite precision arithmetic, which can lead to loss of orthogonality.

4.13.2 Modified Gram-Schmidt (MGS)

The **Modified Gram-Schmidt** (MGS) algorithm addresses the stability issues of CGS by updating each column incrementally:

1. **Initialization:** Set $q_1 = a_1 / \|a_1\|$.
2. **For** $k = 2, 3, \dots, n$:
 - Orthogonalize a_k against the previously computed q_i :

$$a_k = a_k - \langle q_i, a_k \rangle q_i, \quad \text{for each } i < k.$$

- Normalize a_k to obtain q_k .

This small change in order improves numerical stability significantly. MGS is a more reliable approach to constructing the QR decomposition than CGS, particularly for ill-conditioned matrices.

4.13.3 A numerical example

Example 4.1 (QR Decomposition using Classical and Modified Gram-Schmidt). Let the matrix A be given as:

$$A = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}.$$

□

Classical Gram-Schmidt (CGS) Process

Step 1: Compute q_1

Start with the first column vector a_1 :

$$a_1 = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \quad \|a_1\| = \sqrt{1^2 + 1^2 + 0^2} = \sqrt{2}.$$

$$q_1 = \frac{a_1}{\|a_1\|} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \\ 0 \end{bmatrix}.$$

Step 2: Compute q_2

Next, orthogonalize a_2 against q_1 :

$$a_2 = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}, \quad \langle q_1, a_2 \rangle = \left\langle \begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \right\rangle = \frac{1}{\sqrt{2}}.$$

$$\tilde{q}_2 = a_2 - \langle q_1, a_2 \rangle q_1 = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} - \frac{1}{\sqrt{2}} \begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \\ 0 \end{bmatrix} = \begin{bmatrix} \frac{1}{2} \\ -\frac{1}{2} \\ 1 \end{bmatrix}.$$

Now, normalize $\tilde{q}_2$:

$$\|\tilde{q}_2\| = \sqrt{\left(\frac{1}{2}\right)^2 + \left(-\frac{1}{2}\right)^2 + 1^2} = \sqrt{\frac{3}{2}}.$$

$$q_2 = \frac{\tilde{q}_2}{\|\tilde{q}_2\|} = \frac{1}{\sqrt{\frac{3}{2}}} \begin{bmatrix} \frac{1}{2} \\ -\frac{1}{2} \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{1}{\sqrt{6}} \\ -\frac{1}{\sqrt{6}} \\ \sqrt{\frac{2}{3}} \end{bmatrix}.$$

Step 3: Compute q_3

Now, orthogonalize a_3 against q_1 and q_2 :

$$a_3 = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}.$$

$$\langle q_1, a_3 \rangle = \left\langle \begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} \right\rangle = \frac{1}{\sqrt{2}}.$$

$$\tilde{q}_3 = a_3 - \langle q_1, a_3 \rangle q_1 = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} - \frac{1}{\sqrt{2}} \begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \\ 0 \end{bmatrix} = \begin{bmatrix} -\frac{1}{2} \\ \frac{1}{2} \\ 1 \end{bmatrix}.$$

$$\langle q_2, \tilde{q}_3 \rangle = \left\langle \begin{bmatrix} \frac{1}{\sqrt{6}} \\ -\frac{1}{\sqrt{6}} \\ \sqrt{\frac{2}{3}} \end{bmatrix}, \begin{bmatrix} -\frac{1}{2} \\ \frac{1}{2} \\ 1 \end{bmatrix} \right\rangle = 0.$$

Thus, $\tilde{q}_3$ is already orthogonal to q_2 . Normalize $\tilde{q}_3$ to get q_3 :

$$\|\tilde{q}_3\| = \sqrt{\left(-\frac{1}{2}\right)^2 + \left(\frac{1}{2}\right)^2 + 1^2} = \sqrt{\frac{3}{2}}.$$

$$q_3 = \frac{\tilde{q}_3}{\|\tilde{q}_3\|} = \frac{1}{\sqrt{\frac{3}{2}}} \begin{bmatrix} -\frac{1}{2} \\ \frac{1}{2} \\ 1 \end{bmatrix} = \begin{bmatrix} -\frac{1}{\sqrt{6}} \\ \frac{1}{\sqrt{6}} \\ \sqrt{\frac{2}{3}} \end{bmatrix}.$$

Construct R

The entries of R are given by $r_{ij} = \langle q_i, a_j \rangle$. Thus,

$$R = \begin{bmatrix} \langle q_1, a_1 \rangle & \langle q_1, a_2 \rangle & \langle q_1, a_3 \rangle \\ 0 & \langle q_2, a_2 \rangle & \langle q_2, a_3 \rangle \\ 0 & 0 & \langle q_3, a_3 \rangle \end{bmatrix} = \begin{bmatrix} \sqrt{2} & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ 0 & \sqrt{\frac{3}{2}} & \sqrt{\frac{2}{3}} \\ 0 & 0 & \sqrt{\frac{3}{2}} \end{bmatrix}.$$

Final Result

The QR decomposition using the Classical Gram-Schmidt process yields:

$$Q = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{6}} & -\frac{1}{\sqrt{6}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{6}} & \frac{1}{\sqrt{6}} \\ 0 & \sqrt{\frac{2}{3}} & \sqrt{\frac{2}{3}} \end{bmatrix}, \quad R = \begin{bmatrix} \sqrt{2} & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ 0 & \sqrt{\frac{3}{2}} & \sqrt{\frac{2}{3}} \\ 0 & 0 & \sqrt{\frac{3}{2}} \end{bmatrix}.$$

Modified Gram-Schmidt (MGS) Process

In the Modified Gram-Schmidt process, we incrementally orthogonalize each vector a_i by subtracting its projections onto the previously computed q_j vectors.

Step 1: Compute q_1

As before,

$$q_1 = \begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \\ 0 \end{bmatrix}.$$

Step 2: Orthogonalize a_2

$$\text{Projection of } a_2 \text{ onto } q_1 = \langle q_1, a_2 \rangle q_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \\ 0 \end{bmatrix}.$$

$$a_2 = a_2 - \text{Projection of } a_2 = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} - \begin{bmatrix} \frac{1}{2} \\ \frac{1}{2} \\ 0 \end{bmatrix} = \begin{bmatrix} \frac{1}{2} \\ -\frac{1}{2} \\ 1 \end{bmatrix}.$$

Normalize to obtain q_2 :

$$q_2 = \begin{bmatrix} \frac{1}{\sqrt{6}} \\ -\frac{1}{\sqrt{6}} \\ \sqrt{\frac{2}{3}} \end{bmatrix}.$$

Step 3: Orthogonalize a_3

$$a_3 = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}.$$

Subtract the projections onto q_1 and q_2 :

$$a_3 = a_3 - \langle q_1, a_3 \rangle q_1 - \langle q_2, a_3 \rangle q_2.$$

$$a_3 = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} - \frac{1}{\sqrt{2}} \begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \\ 0 \end{bmatrix} - 0 \times \begin{bmatrix} \frac{1}{\sqrt{6}} \\ -\frac{1}{\sqrt{6}} \\ \sqrt{\frac{2}{3}} \end{bmatrix} = \begin{bmatrix} -\frac{1}{2} \\ \frac{1}{2} \\ 1 \end{bmatrix}.$$

Normalize to obtain q_3 :

$$q_3 = \begin{bmatrix} -\frac{1}{\sqrt{6}} \\ \frac{1}{\sqrt{6}} \\ \sqrt{\frac{2}{3}} \end{bmatrix}.$$

Construct R

The matrix R is the same as in the CGS process.

Final Result

The QR decomposition using the Modified Gram-Schmidt process yields the same Q and R as the Classical Gram-Schmidt process.

Solution using the Maxima Computer Algebra System (CAS)

The computations to obtain exact QR decomposition make a case for using *symbolic computation*. Software capable of symbolic computation is typically known as a Computer Algebra System (CAS). One such CAS is Maxima. The automated calculations in Maxima are included in Appendix G. The Python package *Sympy* is a viable alternative, although its functionality as a general purpose CAS are still limited.

4.13.4 Some Applications of QR Decomposition

4.13.5 Linear Systems

Solve $Ax = b$ efficiently using the QR decomposition:

$$Ax = b \implies QRx = b \implies Rx = Q^T b.$$

4.13.6 Least Squares

In minimizing $\|Ax - b\|_2$, QR decomposition provides a numerically stable alternative to the normal equations:

$$Rx = Q^T b.$$

4.14 Solving a Least Squares Problem without Differentiation

4.14.1 Problem Formulation

In this section, we explore the most general inner product defined on $\mathbb{R}^m$ and $\mathbb{R}^n$:

$$\begin{aligned} \langle x, y \rangle &= x^T B y, & x, y \in \mathbb{R}^m, \\ \langle x, y \rangle &= x^T C y, & x, y \in \mathbb{R}^n, \end{aligned}$$

where B and C are symmetric, positive definite matrices. This setup generalizes the standard inner products with $B = I_m$ and $C = I_n$, and can be extended to Hilbert spaces.

Consider a linear system given by

$$Ax = b, \quad A \in \mathbb{R}^{m \times n}, \quad m \geq n.$$

We aim to solve the following unconstrained optimization problem:

$$\text{minimize: } \|Ax - b\| \quad \text{over all } x \in \mathbb{R}^n.$$

4.14.2 Adjoint Operator

The *adjoint operator* of A is denoted as $A^* \in \mathbb{R}^{n \times m}$, satisfying the relation

$$\forall x \in \mathbb{R}^n, \forall y \in \mathbb{R}^m : \langle Ax, y \rangle = \langle x, A^*y \rangle.$$

In the case of Euclidean spaces, this condition transforms into a matrix condition:

$$(Ax)^\top By = x^\top CA^*y,$$

which can be rephrased as

$$x^\top (A^\top B)y = x^\top CA^*y \implies x^\top (A^\top B - CA^*)y = 0.$$

From this, we derive the critical equation:

$$A^\top B = CA^*,$$

leading to the conclusion:

$$\boxed{A^* = C^{-1}A^\top B}.$$

In particular, under standard inner products, we have:

$$A^* = A^\top.$$

4.14.3 Solution to the Least Squares Problem

The essence of our derivative-free approach lies in geometric intuition. We seek the point $z = Ax \in \text{range}(A)$ that is closest to b . The solution can be found by identifying y as the orthogonal projection of b onto $\text{range}(A)$, satisfying:

$$\forall y \in \mathbb{R}^n : \quad b - z \perp Ay.$$

This translates mathematically to:

$$\langle b - Ax, Ay \rangle = 0 \quad \text{or} \quad \langle A^*(b - Ax), y \rangle = 0.$$

Consequently, we arrive at the fundamental equation:

$$\boxed{A^*Ax = A^*b}.$$

Thus, we can express x as:

$$x = (A^*A)^{-1}A^*b, \quad z = Ax = A(A^*A)^{-1}A^*b.$$

Defining two linear operators, we have:

$$A^\dagger = (A^*A)^{-1}A^* \quad (\text{pseudo-inverse of } A),$$

$$P = AA^\dagger = A(A^*A)^{-1}A^* \quad (\text{orthogonal projection onto } \text{range}(A)).$$

It is important to note that our formula for the pseudo-inverse is valid under the condition that A^*A is invertible, which holds if $\text{nullspace}(A) = \{0\}$. In particular, we find:

$$\|Ax\|^2 = \langle Ax, Ax \rangle = \langle A^*Ax, x \rangle,$$

indicating that A^*A is non-negative definite, and $\text{nullspace}(A^*A) = \text{nullspace}(A)$. Thus, when $\text{nullspace}(A) = \{0\}$, the matrix A^*A is indeed invertible.

Our pseudo-inverse formula typically applies to over-determined systems, which are the most common scenarios in the Least Squares Method. The condition for nullspace directly relates to the *full rank* condition, $\text{rank}(A) = n$, for $m \geq n$.

4.14.4 The Pythagorean Theorem and Sufficiency

In Hilbert spaces, the general Pythagorean Theorem states that if $x_1, x_2, \dots, x_k$ form an *orthogonal set*, then:

$$\left\| \sum_{i=1}^k x_i \right\|^2 = \sum_{i=1}^k \|x_i\|^2.$$

Lemma 4.1 (Necessary and Sufficient Condition of Optimality). *Let $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$. A vector $x \in \mathbb{R}^n$ minimizes $\|Ax - b\|$ if and only if Ax is the orthogonal projection of b onto $\text{range}(A)$. $\square$*

Proof. $\implies$: If $\|Ax - b\|^2$ is minimized by x , then for all y ,

$$\|Ay - b\|^2 \geq \|Ax - b\|^2.$$

This leads to:

$$\begin{aligned} \|Ax - b\|^2 &= \|A(x - y) + (Ay - b)\|^2 \\ &= \|A(x - y)\|^2 + 2\langle A(x - y), Ay - b \rangle + \|Ay - b\|^2 \\ &\leq \|Ay - b\|^2. \end{aligned}$$

Let $u = x - y$. Thus, for all u :

$$\|Au\|^2 + 2\langle Au, Ax - b \rangle \leq 0.$$

Since this holds for $-u$ as well:

$$\|Au\|^2 - 2\langle Au, Ax - b \rangle \leq 0.$$

Adding these two inequalities yields:

$$2\|Au\|^2 \leq 0,$$

which implies $Au = 0$ and thus $x = y$.

$\impliedby$: If Ax is the orthogonal projection of b onto $\text{range}(A)$, we have:

$$A(y - x) \perp b - Ax,$$

leading to the Pythagorean Theorem:

$$\|b - Ay\|^2 = \|(b - Ax) + A(x - y)\|^2 = \|b - Ax\|^2 + \|A(x - y)\|^2.$$

Therefore, $\|b - Ay\| \geq \|b - Ax\|$, unless $y - x \in \text{nullspace}(A)$, in which case $Ay = Ax$ is also the orthogonal projection. $\square$

Corollary 4.1 (Generalization to the Least Squares Problem). Our analysis relies solely on the existence of an orthogonal projection of a point onto the range of a linear operator. This approach seamlessly extends to the Hilbert space context and applies to broader classes of linear operators, including differential operators. The pivotal lemma asserts the existence of the closest vector to a given vector in a *closed subspace* of a Hilbert space. $\square$

4.14.5 Computing Pseudo-inverse and Orthogonal Projection using QR-decomposition

Let $A \in \mathbb{R}^{m \times n}$, $m \geq n$, and $A = QR$, where $Q \in \mathbb{R}^{m \times n}$ and $R \in \mathbb{R}^{n \times n}$ possess the following properties:

1. Q has columns that form an orthonormal set, satisfying $Q^\top Q = I_n$.
2. R is an upper-triangular matrix.

While $P = QQ^\top$ is an *orthogonal projection matrix*, it is not necessarily the identity matrix when $m > n$. This projection matrix is characterized by:

1. Symmetry: $P^\top = P$,
2. Idempotence: $P^2 = P$.

This outcome typically arises from applying the Gram-Schmidt process to the coefficient matrix of an over-determined linear system $Ax = b$ where A has full rank. We derive:

$$\begin{aligned} A^\dagger &= (A^\top A)^{-1} A^\top = (R^\top Q^\top Q R)^{-1} R^\top Q^\top \\ &= (R^\top R)^{-1} R^\top Q^\top = R^{-1} (R^\top)^{-1} R^\top Q^\top \\ &= R^{-1} Q^\top. \end{aligned}$$

Consequently, the solution to the Least Squares Problem $Ax = b$ is expressed as:

$$x = R^{-1} Q^\top b.$$

Moreover,

$$Pb = Ax = QRR^{-1}Q^\top b = QQ^\top b.$$

Thus, the orthogonal projection operator onto $\text{range}(A)$ is given by:

$$P = QQ^\top.$$

4.15 Finding Complex Dominant Eigenvalues Using a Modified Power Method

Introduction

When dealing with matrices or real polynomials, the eigenvalues can be real or occur in complex conjugate pairs. If a matrix has a pair of complex conjugate eigenvalues that are dominant, the standard **Power Method** is insufficient to capture them accurately. The method we discuss here, often referred to as a **2D Power Method** or **Complex Conjugate Shifted Power Method**, is designed to approximate a 2D invariant subspace that captures these eigenvalues.

Problem with the Standard Power Method for Complex Eigenvalues

The standard Power Method iteratively multiplies a matrix A by a vector x to find the dominant eigenvector corresponding to a real eigenvalue. If A has complex eigenvalues $\lambda_1 = a + bi$ and $\lambda_2 = a - bi$, the standard Power Method struggles to converge to a clear eigenvalue because the iterates oscillate in a pattern induced by the complex eigenvalues. In effect, the standard Power Method fails to isolate complex eigenvalues directly.

Complex Conjugate Shifted Power Method (2D Power Method)

The Complex Conjugate Shifted Power Method leverages the fact that real matrices with complex eigenvalues will have those eigenvalues occur as conjugate pairs. This method identifies the 2D subspace that approximates the span of the real and imaginary parts of the eigenvectors corresponding to these complex eigenvalues.

Algorithm Outline

1. **Initialize with Two Linearly Independent Vectors:** Choose two linearly independent initial vectors x_1 and x_2 .
2. **Apply Matrix Multiplication:** Multiply each of these initial vectors by the matrix A . Compute:

$$y_1 = Ax_1, \quad y_2 = Ax_2.$$

3. **Orthogonalize:** Perform an orthogonalization step (such as Gram-Schmidt) on y_1 and y_2 to obtain an orthogonal basis $\{q_1, q_2\}$.
4. **Form the 2D Projection Matrix:** Construct a 2×2 projection matrix B by projecting A onto the subspace spanned by $\{q_1, q_2\}$. The elements of B are defined as:

$$B = \begin{bmatrix} q_1^\top A q_1 & q_1^\top A q_2 \\ q_2^\top A q_1 & q_2^\top A q_2 \end{bmatrix}.$$

5. **Calculate the Eigenvalues of B :** The 2×2 matrix B will have two eigenvalues, which are approximations of the complex conjugate eigenvalues of A .

Example: Applying the Method to a Matrix

Suppose we have a matrix A :

$$A = \begin{bmatrix} 4 & -5 \\ 2 & -3 \end{bmatrix}.$$

Step 1: Initialize Vectors

Choose two linearly independent vectors:

$$x_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad x_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Step 2: Multiply the Vectors by A

$$y_1 = Ax_1 = \begin{bmatrix} 4 \\ 2 \end{bmatrix}, \quad y_2 = Ax_2 = \begin{bmatrix} -5 \\ -3 \end{bmatrix}.$$

Step 3: Orthogonalize y_1 and y_2

Perform the Gram-Schmidt orthogonalization on y_1 and y_2 :

$$q_1 = \frac{y_1}{\|y_1\|}, \quad q_2 = \frac{y_2 - \langle q_1, y_2 \rangle q_1}{\|y_2 - \langle q_1, y_2 \rangle q_1\|}.$$

Step 4: Form the 2×2 Matrix B

The matrix B is defined by projecting A onto the subspace spanned by $\{q_1, q_2\}$. Calculate the entries of B :

$$B_{11} = q_1^\top A q_1, \quad B_{12} = q_1^\top A q_2, \quad B_{21} = q_2^\top A q_1, \quad B_{22} = q_2^\top A q_2.$$

Step 5: Calculate the Eigenvalues of B

The eigenvalues of the 2×2 matrix B are given by:

$$\lambda_{\pm} = \frac{B_{11} + B_{22}}{2} \pm \sqrt{\left(\frac{B_{11} - B_{22}}{2}\right)^2 + B_{12}B_{21}}.$$

These eigenvalues will be approximations of the complex conjugate eigenvalues of A .

4.15.1 Detailed pseudo-code

The purpose is to approximate the two largest eigenvalues of matrix A using two initial vectors. Algorithm 4.12 contains all steps to implement the method. Complex numbers are likely to occur (eventhough the matrix is real!), when finding the values of $r_{\pm}$. Software such as MATLAB, FORTRAN and Python's *NumPy* have native complex arithmetic. Other software, such as C or C++ based programs, would use readily available libraries.

Algorithm 4.12 Power Method in 2D

Require: Matrix A , vectors $\mathbf{x}_1$ and $\mathbf{x}_2$ (random if not provided), maximum iterations maxiter , tolerance ϵ

Ensure: Approximated eigenvalues r_+ , r_- , and eigenvectors $\mathbf{x}_1$, $\mathbf{x}_2$

```

1: Set  $n \leftarrow$  number of rows of  $A$ 
2: if  $\mathbf{x}_1$  is not provided then
3:   Set  $\mathbf{x}_1$  as a random vector of length  $n$ 
4: end if
5: if  $\mathbf{x}_2$  is not provided then
6:   Set  $\mathbf{x}_2$  as a random vector of length  $n$ 
7: end if
8: Initialize  $r_+^{(\text{old})} \leftarrow 0, r_-^{(\text{old})} \leftarrow 0$ 
9: for  $i = 0$  to  $\text{maxiter}$  do
10:  Compute  $\mathbf{y}_1 \leftarrow A \cdot \mathbf{x}_1$ 
11:  Compute  $\mathbf{y}_2 \leftarrow A \cdot \mathbf{x}_2$ 
12:  Normalize  $\mathbf{y}_1$  to obtain  $\mathbf{q}_1 \leftarrow \mathbf{y}_1 / \|\mathbf{y}_1\|$ 
13:   $\mathbf{q}_2 \leftarrow \mathbf{y}_2 - (\mathbf{q}_1^\top \mathbf{y}_2) \cdot \mathbf{q}_1$  { Orthogonalize  $\mathbf{y}_2$  with respect to  $\mathbf{q}_1$  }
14:  Normalize  $\mathbf{q}_2 \leftarrow \mathbf{q}_2 / \|\mathbf{q}_2\|$ 
15:  Compute  $B_{11} \leftarrow \mathbf{q}_1^\top (A \cdot \mathbf{q}_1)$ 
16:  Compute  $B_{12} \leftarrow \mathbf{q}_1^\top (A \cdot \mathbf{q}_2)$ 
17:  Compute  $B_{21} \leftarrow \mathbf{q}_2^\top (A \cdot \mathbf{q}_1)$ 
18:  Compute  $B_{22} \leftarrow \mathbf{q}_2^\top (A \cdot \mathbf{q}_2)$ 
19:  Compute  $\text{Trace} \leftarrow 0.5 \times (B_{11} + B_{22})$ 
20:  Compute  $\Delta \leftarrow (0.5 \times (B_{11} - B_{22}))^2 + B_{12} \times B_{21}$ 
21:  Compute  $r_+ \leftarrow \text{Trace} + \sqrt{\Delta}$ 
22:  Compute  $r_- \leftarrow \text{Trace} - \sqrt{\Delta}$ 
23:  if  $i > 0$  and  $\left| \frac{r_+ - r_+^{(\text{old})}}{r_+} \right| < \epsilon$  and  $\left| \frac{r_- - r_-^{(\text{old})}}{r_-} \right| < \epsilon$  then
24:    break
25:  end if
26:  Update  $r_+^{(\text{old})} \leftarrow r_+$ 
27:  Update  $r_-^{(\text{old})} \leftarrow r_-$ 
28:  Update  $\mathbf{x}_1 \leftarrow \mathbf{q}_1$ 
29:  Update  $\mathbf{x}_2 \leftarrow \mathbf{q}_2$ 
30: end for
31: if iterations reached  $\text{maxiter}$  then
32:   raise error: "Tolerance not met within  $\text{maxiter}$  iterations"
33: end if
34: return  $r_+, r_-, \mathbf{x}_1, \mathbf{x}_2$ 

```

4.15.2 Retrieval of the Complex Eigenvector

We should comment on the fact that the vectors $\mathbf{x}_1$ and $\mathbf{x}_2$ returned by the algorithm are not eigenvectors in the usual sense because a matrix $A \in \mathbb{R}^{n \times n}$ cannot have a real eigenvector with a complex eigenvalue. If $\lambda = \xi + i\eta$ is a complex eigenvalue then there is *complex* eigenvector $\mathbf{v} \in \mathbb{C}^n$, satisfying the equation

$$A\mathbf{v} = \lambda\mathbf{v}$$

We can decompose this vector into its real and imaginary part coordinatewise:

$$\mathbf{v} = \mathbf{v}_1 + i\mathbf{v}_2, \quad \mathbf{v}_1, \mathbf{v}_2 \in \mathbb{R}^n.$$

We also have

$$A(\mathbf{v}_1 + i\mathbf{v}_2) = (\xi + i\eta)(\mathbf{v}_1 + i\mathbf{v}_2) = (\xi\mathbf{v}_1 - \eta\mathbf{v}_2) + i(\eta\mathbf{v}_1 + \xi\mathbf{v}_2)$$

Taking real and imaginary parts of both sides we obtain:

$$\begin{cases} A\mathbf{v}_1 = \xi\mathbf{v}_1 - \eta\mathbf{v}_2, \\ A\mathbf{v}_2 = \eta\mathbf{v}_1 + \xi\mathbf{v}_2. \end{cases} \quad (4.9)$$

This equation can be written in matrix form, with $V = [\mathbf{v}_1 | \mathbf{v}_2]$:

$$AV = V\Lambda$$

where

$$\Lambda = \begin{pmatrix} \xi & -\eta \\ \eta & \xi \end{pmatrix}. \quad (4.10)$$

Algorithm 4.12 finds the 2-dimensional subspace $\text{span}\{\mathbf{v}_1, \mathbf{v}_2\}$, which is the same as $\text{span}\{\mathbf{x}_1, \mathbf{x}_2\}$, where $\mathbf{x}_1$ and $\mathbf{q}_2$ are the two orthonormal vectors returned by the algorithm. If $Q = [\mathbf{q}_1 | \mathbf{q}_2]$, we have

$$AQ = QB$$

where

$$B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}.$$

Any (real or complex) eigenvector of B becomes an eigenvector of A . Indeed, if $B\mathbf{u} = \lambda\mathbf{u}$, $\mathbf{u} \in \mathbb{C}^2$ then

$$AQ\mathbf{u} = QB\mathbf{u} = Q\lambda\mathbf{u} = \lambda Q\mathbf{u}.$$

This means that

$$\mathbf{v} = Q\mathbf{u}$$

is an eigenvector of A with eigenvalue λ . If λ is complex then $\sigma(B) = \{\lambda, \bar{\lambda}\}$ and it has the *real-diagonal form*, i.e., for some real invertible matrix $S \in \mathbb{R}^2$:

$$S^{-1}BS = \Lambda$$

where $\Lambda \in \mathbb{R}^2$ is given by (4.10). We find that

$$AQS = QBS = Q\Lambda S$$

Writing $QS = [\mathbf{v}_1 | \mathbf{v}_2]$, i.e., defining $\mathbf{v}_1$ and $\mathbf{v}_2$ as the columns of the matrix QS , we obtain (4.9). In summary, to retrieve the complex eigenvector of A , we need to

1. Diagonalize B .
2. Find QS , and retrieve its columns $\mathbf{v}_1$ and $\mathbf{v}_2$
3. Define the eigenvector as

$$\mathbf{v} = \mathbf{v}_1 + i\mathbf{v}_2.$$

Exercise 4.14. Modify Algorithm 4.12 to find a true (complex) eigenvector of a real matrix A . $\square$

4.15.3 Summary of the Approach

The Complex Conjugate Shifted Power Method is a modification of the standard Power Method designed to capture pairs of complex conjugate eigenvalues. It works by finding a 2D invariant subspace that approximates the eigenvectors associated with the complex conjugate eigenvalues.

4.16 QR Algorithm

The QR Algorithm is a more advanced method for computing all eigenvalues of a matrix. It iteratively decomposes the matrix into orthogonal and upper triangular matrices.

Algorithm 4.13 QR Algorithm

Require: A matrix A

Ensure: Eigenvalues of A

```
1: Set  $A_0 = A$ 
2: for  $k = 0$  to  $\text{max\_iter}$  do
3:   Compute the QR decomposition  $A_k = Q_k R_k$ 
4:   Update  $A_{k+1} = R_k Q_k$ 
5:   if convergence criterion is met then
6:     break
7:   end if
8: end for
9: return Diagonal elements of  $A_k$  (approximations of the eigenvalues)
```

A numerical example of QR algorithm

Below is a demo of QR algorithm; if you set $A[2,0]=10$, failure to converge due to complex eigenvalues

QR Algorithm

```
1 import numpy as np
2 from numpy.linalg import norm, qr, eig
3 from numpy import diag
4
5 A0 = np.array([[3, 1, 0], [1, 2, 1], [0, 1, 1]])
6 tol = 1e-7
7 A = A0
8 maxiter = 100
9 iter = 0
10 while iter < maxiter:
11     iter += 1
12     Q, R = qr(A)
13     Anew = R @ Q
14     if norm(A-Anew, 'fro') < tol:
15         break
16     A = Anew
17
18 # Estimated eigenvalues
19 eigs1 = diag(A)
20
21 # Norm of the off-diagonal part
22 err = norm(A-diag(diag(A)), 'fro')
23
24 # Calculated eigenvalues by built-in function
25 eigs2 = eig(A0)[0]
26
27 # Round the matrix for presentation
28 A = np.round(A,6)
```

The algorithm terminates after 27 iterations, and the final matrix A rounded to 6 decimal places is:

$$\begin{pmatrix} 3.732051 & 0.0 & -0.0 \\ 0.0 & 2.0 & -0.0 \\ 0.0 & 0.0 & 0.267949 \end{pmatrix}$$

and it is nearly diagonal. The norm of the off-diagonal part is $1.62\text{e-}07$. The estimated eigenvalues are

	Eigenvalues		
Estimated	3.7320508075688754	2.0000000000000044	0.26794919243112264
Calculated	3.732050807568877	1.9999999999999991	0.2679491924311226

The fact that the final matrix is diagonal is due to the initial matrix being symmetric. When we break the symmetry, for instance, by setting $A_{20} = 1$, the final matrix is nearly upper-triangular. The triangular form occurs when the initial matrix is real and has real spectrum with simple eigenvalues.

When complex eigenvalues occur, our basic QR algorithm fails to converge.

4.16.1 A discussion of the QR algorithm

The main observation is that the matrices A_k have the same spectrum (thus the spectrum is a loop invariant). This is because

$$A_{k+1} = R_k Q_k = (Q_k^\top Q_k) R_k Q_k = Q_k^\top A_k Q_k.$$

As $Q_k^\top = Q_k^{-1}$, this equation can be rewritten as a *matrix similarity relationship*:

$$A_{k+1} = Q_k^{-1} A_k Q_k.$$

It is well known that similar matrices have the same spectrum. Furthermore, they have the same Jordan canonical form (or diagonal form). Furthermore,

$$A_{k+1} = (Q_0 Q_1 Q_2 \cdots Q_k)^{-1} A_0 (Q_0 Q_1 Q_2 \cdots Q_k)$$

The matrix

$$Q_0 Q_1 Q_2 \cdots Q_k$$

is orthogonal. If it happens that A_{k+1} converges to A_∞ rapidly enough, then $Q_0 Q_1 Q_2 \cdots Q_k$ converges to an orthogonal matrix Q_∞ and

$$Q_\infty^{-1} A_0 Q_\infty = A_\infty.$$

Moreover, Q_k must converge to I_n . Therefore

$$A_\infty = R_\infty,$$

i.e., the limit is an upper-triangular matrix. Thus,

$$Q^{-1} A_0 Q = R_\infty.$$

Thus, Q is the *similarity matrix* which turns A_0 into an upper-triangular matrix. In particular, we have shown

Theorem 4.3 (Consequences of Convergence of QR). If A is a matrix for which the QR algorithm converges then

1. The limit $A_\infty = R_\infty = \lim_{k \rightarrow \infty} A_k$ is an upper triangular matrix.
2. The limit $Q_\infty = \lim_{k \rightarrow \infty} \prod_{r=0}^k Q_r$ exists and is an orthogonal matrix.
3. The following similarity relation holds:

$$Q_\infty^\top A Q_\infty = R_\infty.$$

A sufficient condition for the QR algorithm to converge is that A has all real eigenvalues with distinct magnitudes:

$$|\lambda_1| > |\lambda_2| > \dots > |\lambda_n| \geq 0.$$

The eigenvalues appear on the diagonal of R_∞ in this precise order:

$$R_\infty = \begin{pmatrix} \lambda_1 & * & * & \dots & * \\ 0 & \lambda_2 & * & \dots & * \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & \lambda_{n-1} & * \\ 0 & 0 & \dots & 0 & \lambda_n \end{pmatrix}.$$

□

If A is symmetric, R_∞ is a diagonal matrix. This implies that Q_∞ diagonalizes A , therefore its columns are the eigenvectors of A .

QR algorithm generalizes to a complex matrix $A \in \mathbb{C}^{n \times n}$, but one must substitute orthogonal matrices with *unitary* matrices, i.e., matrices $Q \in \mathbb{C}^{n \times n}$ such that $Q^H Q = I_n$. Here Q^H is the *Hermitian transpose* of Q . By definition

$$(Q^H)_{i,j} = \overline{Q_{j,i}}$$

where $\bar{z}$ is the complex conjugate of z . When applying QR method to complex matrices, we must replace the inner product with the *Hermitian inner product* defined by

$$\langle \mathbf{x}, \mathbf{y} \rangle = \sum_{j=1}^n \bar{x}_j y_j.$$

4.17 Advanced QR Algorithm

The QR algorithm is a powerful method for computing the eigenvalues of a matrix. To improve its efficiency, especially for large matrices, it is often beneficial to first reduce the matrix to a tridiagonal form. This section covers the process of transforming a matrix to tridiagonal form using Householder transformations or Givens rotations and then using the QR algorithm with shifts and the inverse power method to approximate the eigenvalues.

4.17.1 Reduction to Tridiagonal Form

Reducing a matrix to tridiagonal form simplifies the subsequent steps of the QR algorithm. This can be achieved using Householder transformations or Givens rotations.

Householder Transformations

Householder transformations are used to zero out specific elements of a matrix, effectively reducing it to tridiagonal form.

Algorithm 4.14 Householder Transformation

Require: A symmetric matrix A

Ensure: Tridiagonal matrix T similar to A

- 1: **for** $k = 1$ **to** $n-2$ **do**
 - 2: Compute the Householder vector $\mathbf{v}$ such that $(I - 2\mathbf{v}\mathbf{v}^\top)A(I - 2\mathbf{v}\mathbf{v}^\top)$ zeroes out subdiagonal elements below $A_{k+1,k}$
 - 3: Update $A = (I - 2\mathbf{v}\mathbf{v}^\top)A(I - 2\mathbf{v}\mathbf{v}^\top)$
 - 4: **end for**
 - 5: **return** Tridiagonal matrix T
-

When the algorithm is applied to a non-symmetric matrix, the matrix T becomes a *Hessenberg matrix*, i.e., a matrix whose all subdiagonal entries below the (-1) st diagonal are 0. The value of this form is somewhat limited. But for a symmetric matrix the Hessenberg matrix (which is also symmetric), must be tridiagonal. This allows for rapid calculation of T^n .

Constructing the Householder Vector $\mathbf{v}$ to Zero Out Entries Below $A_{2,1}$

To construct a vector $\mathbf{v}$ such that the Householder transformation $H = I - 2\mathbf{v}\mathbf{v}^\top$ zeroes the entries below $A_{2,1}$ in a matrix A , follow these steps:

1. **Identify the target subvector:** Let $\mathbf{x}$ be the subvector of the first column of A starting from the second element downward:

$$\mathbf{x} = \begin{bmatrix} 0 \\ a_{2,1} \\ a_{3,1} \\ \vdots \\ a_{n,1} \end{bmatrix}.$$

2. **Compute the norm:** Calculate the Euclidean norm of $\mathbf{x}$:

$$\|\mathbf{x}\| = \sqrt{\sum_{i=2}^n a_{i,1}^2}.$$

3. **Determine the sign factor:** Define α as:

$$\alpha = -\operatorname{sgn}(a_{2,1}) \cdot \|\mathbf{x}\|,$$

where sgn denotes the sign function.

4. **Construct the vector $\mathbf{u}$:** Define $\mathbf{u}$ as:

$$\mathbf{u} = \mathbf{x} - \alpha \mathbf{e}_1,$$

where $\mathbf{e}_1$ is the unit vector $[1, 0, \dots, 0]^\top$ of appropriate dimension.

5. **Normalize $\mathbf{u}$:** Compute $\mathbf{v}$ by normalizing $\mathbf{u}$:

$$\mathbf{v} = \frac{\mathbf{u}}{\|\mathbf{u}\|}.$$

The vector $\mathbf{v}$ obtained through this procedure ensures that the Householder transformation $H = I - 2\mathbf{v}\mathbf{v}^\top$ will zero out the entries below $A_{2,1}$ in the matrix A .

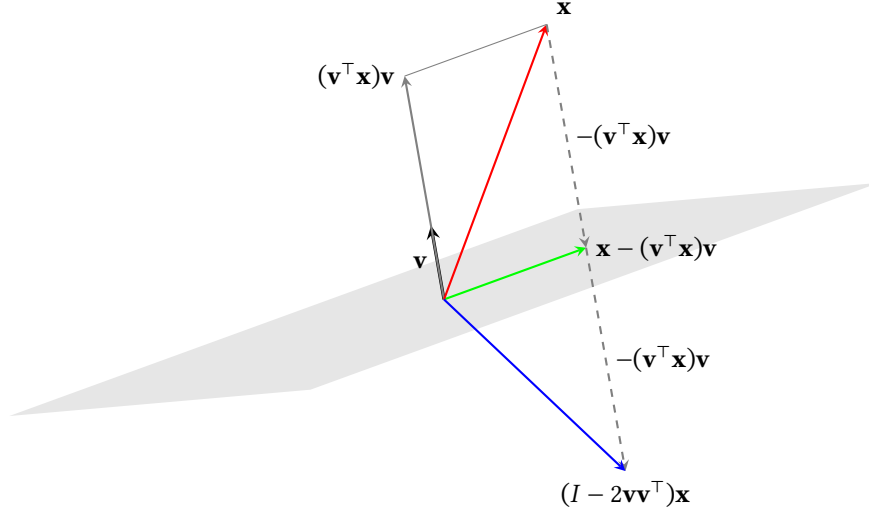
A numerical example is listed below:

```

1 load(linearalgebra);
2 A : matrix(
3   [1,2,2],
4   [3,1,2],
5   [4,1,3]);
6
7 x : submatrix(1,col(A,1));
8 norm_x : mat_norm(x,frobenius);
9 alpha : -signum(A[2,1])*norm_x;
10 u : x-alpha*col(identfor(x.transpose(x)),1);
11 v : u/mat_norm(u,frobenius);
12 v : addrow(matrix([0]),v);
13 B : v.transpose(v);
14 B : identfor(B)-2*B;
15 B.A.B;
```

Listing 4.2: A step of Householder algorithm in Maxima

The resulting matrix has its $(3, 1)$ entry zeroed out.



Geometry of Householder transformations

Geometrically, Householder transformation performs a reflection of a vector $\mathbf{x}$ in the hyperplane orthogonal to vector $\mathbf{v}$. The orthogonal projection onto the vector $\mathbf{v}$ is

$$P(\mathbf{x}) = (\mathbf{v}^T \mathbf{x})\mathbf{v} = \mathbf{v}(\mathbf{v}^T \mathbf{x}).$$

The ability to exchange the order of the factors is due to the fact that $\mathbf{v}^T \mathbf{x}$ is a scalar. Re-distributing the terms we obtain

$$P(\mathbf{x}) = (\mathbf{v}\mathbf{v}^T)\mathbf{x}.$$

This means that the projection is performed by a matrix P :

$$P = \mathbf{v}\mathbf{v}^T.$$

Matrices of this form are orthogonal projections, and are characterized by two properties:

$$P^2 = P, \quad P^T = P.$$

The matrix $I - P$ is another projection matrix: the projection onto the hyperplane:

$$\mathbf{v}^\perp = \{\mathbf{x} \in \mathbb{R}^n : \langle \mathbf{v}, \mathbf{x} \rangle = 0\}$$

The Householder transformation is defined as

$$H = I - 2P$$

where P is any orthogonal projection onto a unit vector $\mathbf{v}$.

Givens Rotations

Givens rotations are used to zero out individual elements of a matrix, making them useful for reducing a matrix to tridiagonal form. The Givens rotation matrix $G(i, j, \theta)$ is defined to zero out the (i, j) -th element of a matrix.

For a given matrix element A_{ij} where $i < j$, the Givens rotation matrix $G(i, j, \theta)$ has the form:

$$G(i, j, \theta) = \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & \cos(\theta) & \cdots & -\sin(\theta) \\ & & \sin(\theta) & \cdots & \cos(\theta) \\ & & & & \ddots \end{pmatrix}$$

Where θ is chosen such that:

$$\cos(\theta) = \frac{a}{\sqrt{a^2 + b^2}}, \quad \sin(\theta) = \frac{b}{\sqrt{a^2 + b^2}}$$

And $a = A_{ii}$, $b = A_{ij}$.

Algorithm 4.15 Givens Rotation

Require: A symmetric matrix A

Ensure: Tridiagonal matrix T similar to A

```

1: for  $i = 1$  to  $n-1$  do
2:   for  $j = i+1$  to  $n$  do
3:     if  $A_{ij} \neq 0$  then
4:       Compute  $r = \sqrt{A_{ii}^2 + A_{ij}^2}$ 
5:       Set  $c = \frac{A_{ii}}{r}$  and  $s = \frac{A_{ij}}{r}$ 
6:       Form the Givens rotation matrix  $G$ 
7:       Update  $A = G^T A G$ 
8:     end if
9:   end for
10: end for
11: return Tridiagonal matrix  $T$ 

```

4.18 QR Factorization by Householder Reflections

Given a real matrix $A \in \mathbb{R}^{m \times n}$ with $m \geq n$, the Householder method computes an orthogonal $Q \in \mathbb{R}^{m \times m}$ and an upper triangular $R \in \mathbb{R}^{m \times n}$ such that

$$A = QR, \quad Q^T Q = I, \quad R = \begin{bmatrix} \text{upper-triangular } n \times n \\ 0 \end{bmatrix}.$$

It proceeds by successively zeroing the subdiagonal parts of each column using orthogonal reflections

$$H_k = I - 2 \frac{v_k v_k^T}{v_k^T v_k},$$

where v_k is chosen so that H_k maps the current column segment to a multiple of e_1 . Householder QR is backward stable and is the workhorse for least squares with full-rank design matrices.

Algorithm 4.16 Householder QR (thin Q)

Require: $A \in \mathbb{R}^{m \times n}$ with $m \geq n$

Ensure: $Q \in \mathbb{R}^{m \times n}$ with orthonormal columns, $R \in \mathbb{R}^{n \times n}$ upper triangular, and $A = QR$

```
1:  $R \leftarrow A$ 
2: Initialize an empty list of Householder vectors  $\{v_k\}_{k=1}^n$ 
3: for  $k = 1, 2, \dots, n$  do
4:    $x \leftarrow R_{k:m, k} \in \mathbb{R}^{m-k+1}$  {the active column segment}
5:   if  $x = 0$  then
6:     continue
7:   end if
8:    $\alpha \leftarrow \|x\|_2 = \sqrt{x^T x}$ 
9:    $s \leftarrow \begin{cases} +1, & \text{if } x_1 \geq 0, \\ -1, & \text{otherwise} \end{cases}$ 
10:   $u \leftarrow x + s \alpha e_1$ 
11:   $v \leftarrow u / \|u\|_2$  {Normalize so that  $v^T v = 1$ }
12:  Store  $v$  (or its tail) for later application
13:  Apply  $H_k = I - 2vv^T$  to rows  $k:m$  of  $R$ :  $R_{k:m, k:n} \leftarrow R_{k:m, k:n} - 2v(v^T R_{k:m, k:n})$ 
14: end for
15: (Forming  $Q$  explicitly, if desired)
16:  $Q \leftarrow$  zero  $m \times n$  matrix
17: for  $j = 1, 2, \dots, n$  do
18:    $q \leftarrow e_j \in \mathbb{R}^m$  {Start from standard basis}
19:   for  $k = n, n-1, \dots, 1$  do
20:     Lift  $v$  to  $\tilde{v} \in \mathbb{R}^m$  by padding zeros above row  $k$ 
21:      $q \leftarrow q - 2\tilde{v}(\tilde{v}^T q)$ 
22:   end for
23:   Set column  $j$  of  $Q$  to  $q$ 
24: end for
25: return ( $Q, R_{1:n, 1:n}$ )
```

Remarks.

- In high-quality implementations, one stores the Householder vectors v_k in the lower-triangular part of R (overwriting A) together with diagonal scalars $\tau_k = 2/(v_k^T v_k)$, and applies them via Level-2/3 BLAS without forming Q explicitly.
- Cost: $2mn^2 - \frac{2}{3}n^3 + \mathcal{O}(mn)$ flops to obtain R and the compact Householder data. Forming Q explicitly costs an extra $\sim 2mn^2$.
- For least squares $\min_x \|Ax - b\|_2$ with full column rank, compute R and the Householder vectors; apply the reflectors to b to get $c = Q^T b$, then solve $R_{1:n, 1:n} x = c_{1:n}$.

By-Hand Example (all rational entries)

Consider

$$A = \begin{bmatrix} 2 & 2 & 1 \\ 2 & 3 & 2 \\ 1 & 2 & 2 \end{bmatrix}.$$

We construct three reflectors H_1, H_2, H_3 so that $R = H_3 H_2 H_1 A$ is upper triangular, and $Q = H_1^T H_2^T H_3^T = H_1 H_2 H_3$ (since $H_k^T = H_k$).

★★★ **Step 1:** Zero subdiagonal of column 1 ★★★ Take $x = (2, 2, 1)^T$, $\|x\|_2 = 3$, $s = +1$.

$$u = x + 3e_1 = \begin{bmatrix} 5 \\ 2 \\ 1 \end{bmatrix}, \quad v = \frac{u}{\|u\|_2} = \frac{1}{\sqrt{30}} \begin{bmatrix} 5 \\ 2 \\ 1 \end{bmatrix}.$$

The reflector $H_1 = I - 2vv^T$ simplifies to the rational matrix

$$H_1 = \begin{bmatrix} -\frac{2}{3} & -\frac{2}{3} & -\frac{1}{3} \\ -\frac{2}{3} & \frac{11}{15} & -\frac{2}{15} \\ -\frac{1}{3} & -\frac{2}{15} & \frac{14}{15} \end{bmatrix},$$

and

$$H_1 A = \begin{bmatrix} -3 & -4 & -\frac{8}{3} \\ 0 & \frac{3}{5} & \frac{8}{15} \\ 0 & \frac{4}{5} & \frac{19}{15} \end{bmatrix}.$$

★★★ **Step 2:** Zero subdiagonal of column 2 in rows 2–3 ★★★ Work on the 2×2 trailing block. With

$$x = \begin{bmatrix} \frac{3}{5} \\ \frac{4}{5} \\ \frac{1}{5} \end{bmatrix}, \quad \|x\|_2 = 1, \quad s = +1, \quad u = x + e_1, \quad v = \frac{u}{\|u\|_2} = \frac{1}{\sqrt{5}} \begin{bmatrix} 2 \\ 1 \\ 1 \end{bmatrix},$$

the embedded reflector

$$H_2 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -\frac{3}{5} & -\frac{4}{5} \\ 0 & -\frac{4}{5} & \frac{3}{5} \end{bmatrix}$$

yields

$$H_2(H_1 A) = \begin{bmatrix} -3 & -4 & -\frac{8}{3} \\ 0 & -1 & -\frac{4}{3} \\ 0 & 0 & \frac{1}{3} \end{bmatrix}.$$

★★★ **Step 3:** Fix the sign of r_{33} ★★★ Finally, take $H_3 = \text{diag}(1, 1, -1)$ to make the last diagonal entry positive:

$$R = H_3 H_2 H_1 A = \begin{bmatrix} -3 & -4 & -\frac{8}{3} \\ 0 & -1 & -\frac{4}{3} \\ 0 & 0 & \frac{1}{3} \end{bmatrix}.$$

(If you prefer a strictly positive main diagonal, flip the overall signs of corresponding columns of Q and rows of R .)

Assemble Q . Since each H_k is symmetric and orthogonal, $Q = H_1 H_2 H_3$. Multiplying the three reflectors gives

$$Q = \begin{bmatrix} -\frac{2}{3} & \frac{2}{3} & -\frac{1}{3} \\ -\frac{2}{3} & -\frac{1}{3} & \frac{2}{3} \\ -\frac{1}{3} & -\frac{2}{3} & -\frac{2}{3} \end{bmatrix}, \quad R = \begin{bmatrix} -3 & -4 & -\frac{8}{3} \\ 0 & -1 & -\frac{4}{3} \\ 0 & 0 & -\frac{1}{3} \end{bmatrix}.$$

One readily checks $Q^T Q = I$ and $A = QR$ exactly, using only rational arithmetic.

Notes for hand computation. To keep arithmetic simple when teaching:

1. Choose the sign s so that $u = x + s\|x\|_2 e_1$ avoids cancellation; this also makes the first entry of $H_k x$ equal to $-s\|x\|_2$.
2. In many small examples, the final reflectors simplify to rational matrices even if v has square roots.
3. To solve least squares, never form Q ; instead apply the stored v_k to b in sequence to obtain $Q^T b$.

4.19 Wilkinson Shift

The Wilkinson shift is a technique used to accelerate the convergence of iterative algorithms, such as the QR algorithm, when computing the eigenvalues of a symmetric tridiagonal matrix. Named after James H. Wilkinson, it plays a crucial role in numerical linear algebra.

4.19.1 Context

In the QR algorithm, a matrix A is decomposed into the product of an orthogonal matrix Q and an upper triangular matrix R , forming a new matrix $A' = RQ$. This process is repeated iteratively, and under suitable conditions, the sequence of matrices converges to a diagonal matrix, whose diagonal entries are the eigenvalues of A .

4.19.2 Motivation for the Shift

The convergence of the QR algorithm can be slow, particularly when the eigenvalues of the matrix are not well-separated. Introducing a shift value can significantly accelerate convergence by strategically modifying the matrix before applying the QR decomposition.

4.19.3 Definition of the Wilkinson Shift

The Wilkinson shift is a specific choice of shift designed to improve the convergence rate. For a symmetric tridiagonal matrix T , the Wilkinson shift is based on the eigenvalues of the bottom 2×2 submatrix. Suppose the bottom 2×2 submatrix is given by

$$\begin{pmatrix} a & b \\ b & c \end{pmatrix}.$$

The eigenvalues of this submatrix are

$$\lambda_{\pm} = \frac{a+c}{2} \pm \sqrt{\left(\frac{a-c}{2}\right)^2 + b^2}.$$

The Wilkinson shift, μ , is chosen as the eigenvalue $\lambda_{\pm}$ that is closer to c , the bottom-right element of the submatrix.

4.19.4 Algorithm

1. **Shift the Matrix:** Subtract the Wilkinson shift μ from the diagonal elements of the matrix T : $T - \mu I$.
2. **Perform QR Decomposition:** Apply the QR decomposition to $T - \mu I$ to obtain matrices Q and R .
3. **Form the New Matrix:** Compute $T' = RQ + \mu I$. Adding the shift μ back ensures the matrix remains similar to the original T .
4. **Iterate:** Repeat this process until the matrix converges to a form from which the eigenvalues can be easily extracted.

4.19.5 Benefits

The Wilkinson shift typically improves the rate of convergence of the QR algorithm, especially when the eigenvalues are closely spaced or clustered. It helps deflate the matrix more efficiently, facilitating faster separation of eigenvalues compared to the unshifted QR algorithm.

4.19.6 Applications

The Wilkinson shift is widely implemented in numerical software libraries, such as LAPACK, due to its efficiency and reliability in computing the eigenvalues of symmetric matrices.

4.19.7 QR Algorithm with Shifts

Once the matrix is in tridiagonal form, the QR algorithm with shifts can be used to approximate the eigenvalues efficiently.

Algorithm 4.17 QR Algorithm with Shifts

Require: A tridiagonal matrix T

Ensure: Eigenvalues of T

```
1: Initialize  $T_0 = T$ 
2: for  $k = 0$  to  $\text{max\_iter}$  do
3:   Compute a shift  $\mu$  (e.g., using the Wilkinson shift)
4:   Perform the QR decomposition  $T_k - \mu I = Q_k R_k$ 
5:   Update  $T_{k+1} = R_k Q_k + \mu I$ 
6:   if off-diagonal elements of  $T_{k+1}$  are sufficiently small then
7:     break
8:   end if
9: end for
10: return Diagonal elements of  $T_k$  (approximations of the eigenvalues)
```

4.19.8 Inverse Power Method with Shifts

The inverse power method with shifts can be used to refine the approximations of eigenvalues, especially for finding the smallest eigenvalues.

Algorithm 4.18 Inverse Power Method with Shifts

Require: A tridiagonal matrix T , initial guess for an eigenvalue λ_0 , and shift μ

Ensure: Refined eigenvalue λ

```
1: Initialize  $\mathbf{x}_0$  (e.g., a random vector)
2: for  $k = 0$  to  $\text{max\_iter}$  do
3:   Solve  $(T - \mu I)\mathbf{y}_k = \mathbf{x}_k$ 
4:   Normalize  $\mathbf{y}_k$  to obtain  $\mathbf{x}_{k+1}$ 
5:   Update the eigenvalue approximation  $\lambda_{k+1} = \mathbf{x}_{k+1}^\top T \mathbf{x}_{k+1}$ 
6:   if convergence criterion is met then
7:     break
8:   end if
9: end for
10: return  $\lambda$ 
```

4.19.9 Combining Methods

Combining the reduction to tridiagonal form, the QR algorithm with shifts, and the inverse power method with shifts provides a robust approach for approximating the eigenvalues of a matrix efficiently.

By first reducing the matrix to tridiagonal form, we simplify the structure of the matrix, making the QR algorithm more efficient. Using shifts in the QR algorithm accelerates convergence, and applying the inverse power method with shifts allows for accurate refinement of eigenvalue approximations.

In summary, the advanced QR algorithm techniques provide powerful tools for eigenvalue computation, essential for many applications in numerical linear algebra, including stability analysis, vibration analysis, and more.

4.20 Singular Value Decomposition (SVD) from Symmetric Matrices

The Singular Value Decomposition (SVD) of a real $m \times n$ matrix A is expressed as:

$$A = U\Sigma V^T$$

where:

- U is an $m \times m$ orthogonal matrix,
- Σ is an $m \times n$ diagonal matrix with non-negative real numbers (the singular values) on the diagonal,
- V is an $n \times n$ orthogonal matrix.

4.20.1 Connection to Symmetric Matrices

To derive the SVD, we use the fact that symmetric matrices can be diagonalized using orthogonal matrices. Consider the following symmetric matrices:

$$A^T A \quad (\text{an } n \times n \text{ symmetric matrix}) \quad \text{and} \quad AA^T \quad (\text{an } m \times m \text{ symmetric matrix}).$$

Since both $A^T A$ and AA^T are symmetric, they can be diagonalized as:

$$A^T A = V\Lambda V^T \quad \text{and} \quad AA^T = U\Gamma U^T,$$

where:

- V is an $n \times n$ orthogonal matrix,
- U is an $m \times m$ orthogonal matrix,
- Λ and Γ are diagonal matrices containing the eigenvalues of $A^T A$ and AA^T , respectively.

4.20.2 Relationship Between Eigenvalues and Singular Values

The eigenvalues of $A^T A$ and AA^T are non-negative, and the square roots of these eigenvalues are the singular values of A . This relationship is crucial for constructing the SVD.

4.20.3 Constructing the SVD

1. Find Eigenvalues and Eigenvectors:

- Compute the eigenvalues of $A^T A$ and arrange them in descending order. The corresponding eigenvectors form the columns of V .
- Compute the eigenvalues of AA^T to obtain the columns of U .

2. Form the Diagonal Matrix Σ :

- The diagonal entries of Σ are the square roots of the eigenvalues of $A^T A$ (or equivalently, AA^T). These entries are the singular values of A .

3. Establish the Orthogonal Matrices:

- U is constructed from the eigenvectors of AA^T .
- V is constructed from the eigenvectors of $A^T A$.

4.20.4 Final Result

Combining all components, we obtain:

$$A = U\Sigma V^T,$$

where U and V are orthogonal matrices, and Σ is the diagonal matrix of singular values.

4.20.5 Representation of a matrix as a Sum of Rank One Matrices

The equation $A = U\Sigma V^\top$ can be written as

$$A = \sum_{i=1}^n \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$$

where σ_i is the i th singular value of A . Singular values are typically listed in the order

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_p$$

where $p = \min(m, n)$. The matrix $\mathbf{u}_i \mathbf{v}_i^\top$ is a *rank-one matrix*. Every rank one matrix can be written in this form. Additionally, $\mathbf{u}_i$ and $\mathbf{v}_i$ are unit vectors, which implies that the ℓ^2 -norm of this matrix is exactly 1. If $r = \text{rank } A$ then exactly r singular values are nonzero and $\sigma_{r+1} = \sigma_{r+2} = \dots = \sigma_p = 0$. Thus

$$A = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^\top.$$

Many applications of SVD rely upon the observation that only a few singular values significantly different from 0. One typically studies the sequence of approximations to A :

$$A^{(k)} = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^\top, \quad (k \leq r).$$

Equivalently

$$A^{(k)} = U \Sigma^{(k)} V^\top$$

where we zero out $\sigma_{k+1}, \sigma_{k+2}, \dots, \sigma_r$. Furthermore, one can write $A^{(k)}$ in a more economical way, because columns $k+1, k+2, \dots, r$ of U and V can be dropped. The resulting form is

$$A^{(k)} = \underbrace{U^{(k)}}_{m \times k} \underbrace{\Sigma^{(k)}}_{k \times k} \underbrace{V^{(k)\top}}_{k \times n}.$$

The art of using SVD relies upon a good heuristic (supported by a numerical experiment) choosing k . The main idea is that $A^{(k)}$ is a reflection of the dominant features of the underlying data (e.g., in PCA), while the discarded singular values reflect noise which can be discarded without affecting the interpretation of the data.

4.20.6 A formal proof

Theorem 4.4 (The existence of SVD). Let $A \in \mathbb{R}^{m \times n}$ and let $r = \text{rank } A$. Then there exist numbers (called the *singular values*) $\sigma_1, \sigma_2, \dots, \sigma_r > 0$ and orthonormal sets $\{\mathbf{u}_i\}_{i=1}^r \subset \mathbb{R}^m$ and $\{\mathbf{v}_i\}_{i=1}^r \subset \mathbb{R}^n$ such that

$$A = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^\top,$$

The numbers σ_i are unique. $\square$

Proof. Let $\mathbf{v}_i, i = 1, 2, \dots, r$, be the an orthonormal eigenbasis of $A^\top A$ with real, non-negative eigenvalues λ_i , ordered in descending order:

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_r > 0 = \lambda_{r+1} = \dots = \lambda_n$$

If $\lambda_i > 0$ then multiplying the equation $A^\top A \mathbf{v}_i = \lambda_i \mathbf{v}_i$ by A on the left we obtain $A(A^\top A) \mathbf{v}_i = \lambda_i A \mathbf{v}_i$ and redistributing terms:

$$(AA^\top)(A \mathbf{v}_i) = \lambda_i (A \mathbf{v}_i).$$

Hence $A\mathbf{v}_i$ is an eigenvector of $AA^\top$ with eigenvalue $\lambda_i \geq 0$. Furthermore, we claim that $\{A\mathbf{v}_i\}_{i=1}^n$ is an orthogonal set. Indeed,

$$(A\mathbf{v}_i)^\top(A\mathbf{v}_j) = \mathbf{v}_i^\top(A^\top A)\mathbf{v}_j = \mathbf{v}_i^\top \lambda_j \mathbf{v}_j = \lambda_j \mathbf{v}_i^\top \mathbf{v}_j = \lambda_j \delta_{ij} = \begin{cases} \lambda_j & \text{if } i = j, \\ 0 & \text{if } i \neq j. \end{cases}$$

The claim is shown. Let $\sigma_i \stackrel{\text{def}}{=} \sqrt{\lambda_i}$. It follows that $\|A\mathbf{v}_i\| = \sigma_i$. Let

$$\mathbf{u}_i \stackrel{\text{def}}{=} \sigma_i^{-1} A\mathbf{v}_i, \quad i = 1, 2, \dots, r.$$

It also follows that $\{\mathbf{u}_i\}_{i=1}^r$ is an orthonormal set and

$$A\mathbf{v}_i = \sigma_i \mathbf{u}_i, \quad i = 1, 2, \dots, r.$$

We complete this set to an orthonormal basis of $\mathbb{R}^m$. It is clear that $\mathbf{u}_i$ for $i = r+1, \dots, m$ must be orthogonal to $\mathcal{R}(A) = \text{span}\{\mathbf{u}_i\}_{i=1}^r$, i.e., must be chosen to be an orthonormal basis of $\mathcal{N}(A^\top)$, due to a well known and easy to prove identity $\mathcal{N}(A^\top) = \mathcal{R}(A)^\perp$. Similarly, by definition

$$A\mathbf{v}_i = 0, \quad i = r+1, \dots, n$$

which means that $\{\mathbf{v}_i\}_{i=r+1}^n$ is an orthonormal basis of $\mathcal{N}(A)$.

All other outstanding claims follow easily. □

4.20.7 Alternative forms of SVD

We note that the alternative form $A = U\Sigma V^\top$ may be preferred in some situations. It comes in two flavors:

1. *full form*, where $U \in \mathbb{R}^{m \times n}$ and $V \in \mathbb{R}^{n \times n}$ are square matrices and $\Sigma \in \mathbb{R}^{m \times n}$;
2. *compact (or economy) form*, where $U \in \mathbb{R}^{m \times r}$, $V \in \mathbb{R}^{n \times r}$ are rectangular and $\Sigma \in \mathbb{R}^{r \times r}$ is square.

The compact form is obtained by keeping only first $r = \text{rank}(A)$ basis vectors of both bases, which may result in significant data reduction if $r \ll \min(m, n)$.

4.20.8 Summary

The symmetric matrices $A^\top A$ and $AA^\top$ have the same non-zero eigenvalues, which are the squares of the singular values of A . By diagonalizing these symmetric matrices, we extract the orthogonal matrices U and V , along with the singular values in Σ , thus constructing the SVD.

4.21 Comparisons and Practical Considerations

4.21.1 Choosing the Right Method

The choice of method for solving linear systems or eigenvalue problems depends on several factors:

- **Matrix Size and Structure:** Direct methods are suitable for small to medium-sized dense matrices, while iterative methods are preferred for large sparse matrices.
- **Accuracy and Convergence:** Iterative methods may require preconditioning to improve convergence.
- **Computational Resources:** Consider memory usage and computational complexity.

4.21.2 Implementation and Efficiency

When implementing numerical linear algebra algorithms, consider:

- **Precision:** Ensure that the algorithm maintains numerical stability and minimizes round-off errors.
- **Parallelization:** Take advantage of parallel computing techniques to speed up computations.

- **Software Libraries:** Utilize optimized numerical libraries such as LAPACK, BLAS, and Eigen for efficient implementation.

In conclusion, numerical linear algebra is a critical area of study in numerical methods, providing essential tools for solving linear systems and eigenvalue problems. Understanding and choosing the appropriate algorithms can significantly enhance the efficiency and accuracy of computations in scientific and engineering applications.

4.22 Exercises

Exercise 4.15. Use Gaussian Elimination without pivoting to solve the system of equations:

$$\begin{cases} 2x + 3y - z &= 5 \\ 4x + y + 2z &= 6 \\ -2x + 7y + 2z &= 3 \end{cases}$$

□

Exercise 4.16. Solve the following system of equations using Gaussian Elimination with partial pivoting:

$$\begin{cases} 3x - y + 2z &= 4 \\ 2x + y - 2z &= 1 \\ x + 3y + z &= 5 \end{cases}$$

□

Exercise 4.17. Apply Gaussian Elimination with complete pivoting to solve the system:

$$\begin{cases} 2x_1 + 4x_2 - 2x_3 &= 4 \\ 4x_1 + 9x_2 - 3x_3 &= 8 \\ -2x_1 - x_2 + 7x_3 &= 10 \end{cases}$$

□

Exercise 4.18. Use LU Decomposition to solve the system of equations:

$$\begin{cases} x + y + z &= 6 \\ 2x + 3y + 4z &= 20 \\ 3x + 5y + 7z &= 40 \end{cases}$$

□

Exercise 4.19. Implement the Jacobi Method to solve the following linear system. Perform 10 iterations starting from the initial guess $\mathbf{x}^{(0)} = (0, 0, 0)^\top$:

$$\begin{cases} 4x + y + z &= 5 \\ 2x + 5y + 2z &= 15 \\ x + 2y + 4z &= 8 \end{cases}$$

□

Exercise 4.20. Solve the system of equations using the Gauss-Seidel Method. Perform 10 iterations starting from the initial guess $\mathbf{x}^{(0)} = (1, 1, 1)^\top$:

$$\begin{cases} 4x_1 + x_2 - x_3 &= 3 \\ x_1 + 4x_2 + x_3 &= 4 \\ -x_1 + x_2 + 4x_3 &= 5 \end{cases}$$

□

Exercise 4.21. Apply the Successive Over-Relaxation (SOR) Method with a relaxation factor $\omega = 1.25$ to solve:

$$\begin{cases} 3x + y + z &= 5 \\ x + 4y + 2z &= 6 \\ x + y + 5z &= 8 \end{cases}$$

Perform 10 iterations starting from $\mathbf{x}^{(0)} = (0, 0, 0)^\top$. $\square$

Exercise 4.22. Use the Power Method to find the dominant eigenvalue and corresponding eigenvector of the matrix:

$$A = \begin{pmatrix} 4 & 1 & 2 \\ 1 & 3 & 1 \\ 2 & 1 & 3 \end{pmatrix}$$

Perform 10 iterations starting from $\mathbf{x}^{(0)} = (1, 1, 1)^\top$. $\square$

Exercise 4.23. Compute the LU Decomposition of the matrix:

$$A = \begin{pmatrix} 2 & -1 & 1 \\ 3 & 3 & 9 \\ 3 & 3 & 5 \end{pmatrix}$$

Verify the decomposition by showing $A = LU$. $\square$

Exercise 4.24. Apply the QR Algorithm to find the eigenvalues of the matrix:

$$A = \begin{pmatrix} 5 & 4 & 2 \\ 4 & 5 & 4 \\ 2 & 4 & 5 \end{pmatrix}$$

Perform 5 iterations. $\square$

Exercise 4.25. Determine if the Jacobi Method will converge for the following system. Justify your answer:

$$\begin{cases} 10x_1 + x_2 + x_3 &= 7 \\ x_1 + 10x_2 + x_3 &= 8 \\ x_1 + x_2 + 10x_3 &= 9 \end{cases}$$

$\square$

Exercise 4.26. Show that the Gauss-Seidel Method converges for the system:

$$\begin{cases} 4x + y + z &= 5 \\ x + 5y + z &= 6 \\ x + y + 6z &= 7 \end{cases}$$

$\square$

Exercise 4.27. Use Gaussian Elimination with partial pivoting to solve the system:

$$\begin{cases} 2x + 3y - z &= 1 \\ 4x + 6y - 2z &= 2 \\ -2x + y + z &= 3 \end{cases}$$

Comment on the numerical stability of the solution. $\square$

Exercise 4.28. Write a program to implement the Gauss-Seidel Method and use it to solve:

$$\begin{cases} 10x_1 - 2x_2 + x_3 &= 6 \\ -2x_1 + 10x_2 - 2x_3 &= 5 \\ x_1 - 2x_2 + 10x_3 &= 6 \end{cases}$$

Compare the results with those obtained using LU Decomposition. $\square$

Exercise 4.29. Using the matrix:

$$A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$$

perform 10 iterations of the Power Method starting from $\mathbf{x}^{(0)} = (1, 0)^\top$. Find the dominant eigenvalue and eigenvector. $\square$

Exercise 4.30. Analyze the convergence rate of the Jacobi Method for the system:

$$\begin{cases} 4x + y &= 5 \\ x + 3y &= 4 \end{cases}$$

Explain why the method converges or does not converge. $\square$

Exercise 4.31. Solve the system of equations using the Successive Over-Relaxation (SOR) Method with $\omega = 1.1$:

$$\begin{cases} 3x_1 + x_2 - x_3 &= 1 \\ x_1 + 4x_2 + x_3 &= 2 \\ x_1 + x_2 + 5x_3 &= 3 \end{cases}$$

Perform 15 iterations starting from $\mathbf{x}^{(0)} = (0, 0, 0)^\top$. $\square$

Exercise 4.32. Consider the matrix:

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 0 & 4 & 5 \\ 0 & 0 & 6 \end{pmatrix}$$

Is the Gauss-Seidel Method applicable to solve $A\mathbf{x} = \mathbf{b}$ for any vector $\mathbf{b}$? Justify your answer. $\square$

Exercise 4.33. Prove that if A is a strictly diagonally dominant matrix, then the Jacobi Method converges. $\square$

Exercise 4.34. Implement the LU Decomposition with partial pivoting in a programming language of your choice and use it to solve:

$$\begin{cases} 3x + 2y - z &= 1 \\ 2x - 3y + 4z &= 2 \\ -x + 4y + 3z &= 3 \end{cases}$$

Compare your results with those obtained from a standard numerical library. $\square$

Exercise 4.35. Use Householder transformations to reduce the following matrix to tridiagonal form:

$$A = \begin{pmatrix} 4 & 1 & 2 & 3 \\ 1 & 3 & 4 & 2 \\ 2 & 4 & 5 & 1 \\ 3 & 2 & 1 & 2 \end{pmatrix}$$

Show each step of the transformation. $\square$

Exercise 4.36. Apply Givens rotations to zero out the subdiagonal elements of the following matrix and reduce it to tridiagonal form:

$$A = \begin{pmatrix} 4 & 3 & 2 & 0 \\ 3 & 5 & 1 & 2 \\ 2 & 1 & 6 & 4 \\ 0 & 2 & 4 & 3 \end{pmatrix}$$

Show the Givens rotation matrices used at each step. $\square$

Exercise 4.37. Given the tridiagonal matrix:

$$T = \begin{pmatrix} 2 & 1 & 0 & 0 \\ 1 & 2 & 1 & 0 \\ 0 & 1 & 2 & 1 \\ 0 & 0 & 1 & 2 \end{pmatrix}$$

use the QR algorithm with shifts to approximate the eigenvalues. Perform 5 iterations and report the intermediate matrices. $\square$

Exercise 4.38. Consider the matrix:

$$A = \begin{pmatrix} 10 & 1 & 2 \\ 1 & 8 & 3 \\ 2 & 3 & 6 \end{pmatrix}$$

Reduce A to tridiagonal form using Householder transformations, then use the inverse power method with a shift to find the smallest eigenvalue. Start with an initial guess of $\lambda_0 = 5$ and perform 10 iterations. $\square$

Exercise 4.39. Implement the QR algorithm with shifts in a programming language of your choice. Use your implementation to find the eigenvalues of the matrix:

$$A = \begin{pmatrix} 4 & 1 & 0 & 0 \\ 1 & 3 & 2 & 0 \\ 0 & 2 & 1 & 1 \\ 0 & 0 & 1 & 2 \end{pmatrix}$$

Compare your results with the eigenvalues computed using a standard numerical library. $\square$

Exercise 4.40. Show that the Givens rotation matrix $G(i, j, \theta)$ is orthogonal. Then, apply a series of Givens rotations to the matrix:

$$A = \begin{pmatrix} 5 & 4 & 0 \\ 4 & 5 & 4 \\ 0 & 4 & 5 \end{pmatrix}$$

to reduce it to tridiagonal form. List each rotation matrix and the resulting matrices. $\square$

Exercise 4.41. Use the inverse power method with a shift to find the largest eigenvalue of the matrix:

$$A = \begin{pmatrix} 3 & 1 & 0 \\ 1 & 4 & 2 \\ 0 & 2 & 5 \end{pmatrix}$$

Start with an initial guess of $\lambda_0 = 4$ and perform 10 iterations. $\square$

Exercise 4.42. Prove that reducing a symmetric matrix to tridiagonal form using Householder transformations preserves its eigenvalues. Then, apply this result to the matrix:

$$A = \begin{pmatrix} 1 & 2 & 0 & 0 \\ 2 & 3 & 2 & 0 \\ 0 & 2 & 4 & 2 \\ 0 & 0 & 2 & 5 \end{pmatrix}$$

$\square$

Exercise 4.43. Explain the role of the Wilkinson shift in the QR algorithm with shifts. Use the Wilkinson shift to find the eigenvalues of the following tridiagonal matrix:

$$T = \begin{pmatrix} 3 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 1 \end{pmatrix}$$

Perform 5 iterations and discuss the convergence. $\square$

4.23 Application-Oriented Exercises for SVD

To gain a deeper understanding of the practical uses of Singular Value Decomposition (SVD), consider the following application-oriented exercises:

Exercise 4.44 (Image Compression). Take an image (in grayscale or RGB format) and express it as a matrix (or a set of matrices for RGB channels). Use SVD to decompose the image matrix and reconstruct the image using only a subset of the largest singular values. Experiment with different numbers of singular values to observe the trade-off between compression and image quality.

Objective: Understand how SVD is used for data compression and the impact of retaining only the most significant singular values on image storage and quality.

Bonus: Calculate the compression ratio and the mean squared error between the original and compressed images.

$\square$

Exercise 4.45 (Principal Component Analysis (PCA)). Given a dataset X with n samples and m features, use SVD to perform Principal Component Analysis (PCA). Center the dataset and use SVD to identify the principal components. Project the data onto the top k principal components and visualize the reduced-dimensional data.

Objective: Demonstrate how SVD is applied for dimensionality reduction while capturing as much variance as possible. Discuss the significance of the principal components.

Bonus: Compare the results with a PCA implementation from a standard machine learning library and explain any differences.

$\square$

Exercise 4.46 (Low-Rank Matrix Approximation). For a large, noisy data matrix A , use SVD to create a low-rank approximation A_k by retaining only the top k singular values and their corresponding singular vectors. Compare A_k with the original matrix in terms of the Frobenius norm and discuss the approximation quality for different k values.

Objective: Explore the concept of low-rank approximation and its applications in simplifying datasets, denoising, and data analysis.

Bonus: Apply this technique to a real-world dataset, such as a user-item ratings matrix in a recommendation system, and discuss the implications.

$\square$

Exercise 4.47 (Latent Semantic Analysis (LSA) for Text Analysis). Given a term-document matrix A representing a collection of text documents, perform SVD to decompose A into $U\Sigma V^T$. Use this decomposition to identify latent topics in the corpus and discuss the interpretability of the singular vectors.

Objective: Apply SVD to extract latent semantic structures from text data and understand its use in natural language processing (NLP) for tasks such as topic modeling.

Bonus: Use the SVD-based representation to compare document similarities or perform clustering.

□

Exercise 4.48 (Data Denoising). Consider a data matrix A corrupted by noise. Use SVD to denoise the data by retaining only the top singular values and setting smaller ones to zero. Reconstruct the denoised matrix and compare it with the original noisy matrix.

Objective: Learn how SVD can be used for noise removal and the balance between denoising and information loss.

Bonus: Analyze the effectiveness of denoising for different noise levels.

□

Exercise 4.49 (SVD-Based Solution to Least Squares Problems). Given an overdetermined system $A\mathbf{x} = \mathbf{b}$, where A is an $m \times n$ matrix with $m > n$, use SVD to find the least-squares solution. Discuss the advantages of using SVD over the normal equation method, particularly for ill-conditioned matrices.

Objective: Understand how SVD provides a stable approach to solving least-squares problems and the role of singular values in system conditioning.

Bonus: Apply this method to a real-world dataset, such as fitting a model to noisy measurements, and analyze the robustness of the solution.

□

Exercise 4.50 (Recommender System Using SVD). Given a user-item rating matrix R , use SVD to build a collaborative filtering recommendation system. Decompose R using SVD, reconstruct a low-rank approximation, and predict missing ratings. Use these predictions to recommend items to users.

Objective: Explore how SVD is applied in recommendation systems and how it addresses sparsity in user-item matrices.

Bonus: Evaluate the system using metrics like root mean squared error (RMSE) and discuss how the choice of rank k influences recommendations.

□

Chapter 5

Multi-Dimensional Root Finding

Multi-dimensional root finding involves solving systems of nonlinear equations. These methods extend one-dimensional root finding techniques to higher dimensions, providing powerful tools for various scientific and engineering applications.

5.1 Example: Mathematical Theory of GPS Positioning

Global Positioning System (GPS) data are used to determine one's position through a process known as trilateration, which involves calculating distances from multiple satellites to pinpoint a specific location on Earth.

5.1.1 Satellites and Signals

The GPS system consists of at least 24 satellites orbiting the Earth, each transmitting signals that include the satellite's position and the precise time the signal was sent.

5.1.2 Time-of-Flight Calculation

A GPS receiver on Earth captures these signals. By knowing the exact time the signal was sent and the time it was received, the receiver can calculate the time-of-flight (Δt) for the signal. The speed of light (c) is then used to convert this time difference into a distance:

$$d = c \cdot \Delta t$$

5.1.3 Distance Spheres

Each distance measurement corresponds to a sphere centered at the satellite, with a radius equal to the calculated distance (d). Therefore, the receiver lies somewhere on the surface of this sphere.

5.1.4 Trilateration

To determine a precise location, the receiver needs distance measurements from at least four satellites. This requirement accounts for the three spatial dimensions and the receiver's clock error. The intersections of these spheres provide the receiver's position:

- With the first satellite, the receiver is on a sphere's surface.
- With the second satellite, the receiver is on the intersection circle of two spheres.
- With the third satellite, the receiver is at one of the two points where the second circle intersects a third sphere.
- The fourth satellite resolves the ambiguity, providing the exact location and correcting any clock errors.

5.1.5 Mathematical Formulation

Let (x_i, y_i, z_i) be the coordinates of the i -th satellite and (x, y, z) be the coordinates of the receiver. The distance from the receiver to the i -th satellite is:

$$(x - x_i)^2 + (y - y_i)^2 + (z - z_i)^2 = (c \cdot \Delta t_i)^2$$

These equations form a system of nonlinear equations, which can be solved using numerical methods to find the coordinates (x, y, z) .

5.2 Numerical Example

Let's go through a numerical example to illustrate how GPS data are used to determine a position using trilateration.

5.2.1 Example Scenario

Imagine a GPS receiver gets signals from four satellites. We'll assume the positions of the satellites and the time it takes for the signals to travel from the satellites to the receiver. We'll then determine the receiver's position.

5.2.2 Satellite Positions and Signal Travel Times

- Satellite 1: $(x_1, y_1, z_1) = (15600 \text{ km}, 7540 \text{ km}, 20140 \text{ km})$
 - Signal travel time: $t_1 = 0.0700$ seconds
- Satellite 2: $(x_2, y_2, z_2) = (18760 \text{ km}, 2750 \text{ km}, 18610 \text{ km})$
 - Signal travel time: $t_2 = 0.0720$ seconds
- Satellite 3: $(x_3, y_3, z_3) = (17610 \text{ km}, 14630 \text{ km}, 13480 \text{ km})$
 - Signal travel time: $t_3 = 0.0760$ seconds
- Satellite 4: $(x_4, y_4, z_4) = (19170 \text{ km}, 610 \text{ km}, 18390 \text{ km})$
 - Signal travel time: $t_4 = 0.0710$ seconds

5.2.3 Calculating Distances

Using the speed of light $c = 299,792.458 \text{ km/s}$, we calculate the distances from each satellite to the receiver:

- Distance to Satellite 1:

$$d_1 = c \cdot t_1 = 299,792.458 \text{ km/s} \cdot 0.0700 \text{ s} = 20,985.47206 \text{ km}$$

- Distance to Satellite 2:

$$d_2 = c \cdot t_2 = 299,792.458 \text{ km/s} \cdot 0.0720 \text{ s} = 21,584.25696 \text{ km}$$

- Distance to Satellite 3:

$$d_3 = c \cdot t_3 = 299,792.458 \text{ km/s} \cdot 0.0760 \text{ s} = 22,383.82848 \text{ km}$$

- Distance to Satellite 4:

$$d_4 = c \cdot t_4 = 299,792.458 \text{ km/s} \cdot 0.0710 \text{ s} = 21,285.86898 \text{ km}$$

5.2.4 Setting Up the System of Equations

We now have four spheres with known centers and radii. We need to solve the following system of nonlinear equations for (x, y, z) :

$$\begin{cases} (x - 15600)^2 + (y - 7540)^2 + (z - 20140)^2 = (20,985.47206)^2 \\ (x - 18760)^2 + (y - 2750)^2 + (z - 18610)^2 = (21,584.25696)^2 \\ (x - 17610)^2 + (y - 14630)^2 + (z - 13480)^2 = (22,383.82848)^2 \\ (x - 19170)^2 + (y - 610)^2 + (z - 18390)^2 = (21,285.86898)^2 \end{cases}$$

5.2.5 Solving the System of Equations

This system is typically solved using numerical methods such as the Newton-Raphson method. For simplicity, let's outline the steps without delving into numerical computations:

- **Linearization:** Linearize the equations using Taylor series expansion around an initial guess for (x, y, z) .
- **Iteration:** Iteratively solve the linearized system, updating the guess for (x, y, z) in each iteration until convergence.
- **Correction:** Apply corrections for any clock errors and atmospheric effects.

5.2.6 Example Solution

Assume the numerical methods provide the following solution:

$$(x, y, z) = (17800 \text{ km}, 9700 \text{ km}, 21000 \text{ km})$$

This would be the receiver's position in the Earth-Centered Earth-Fixed (ECEF) coordinate system (Figure 5.1).

5.2.7 Conclusion

By solving the system of equations derived from the distances to at least four satellites, we can determine the GPS receiver's position. The calculations involve converting signal travel times to distances, setting up the corresponding geometric equations, and using numerical methods to find the intersection point of the spheres representing these distances.

5.3 Fixed Point Iteration

Fixed point iteration is an iterative method for finding a point $\mathbf{x}$ such that $\mathbf{x} = \mathbf{g}(\mathbf{x})$, where $\mathbf{g}$ is a given function.

5.4 Banach Contraction Principle

The Banach Contraction Principle, also known as the Contraction Mapping Theorem, is a fundamental result in fixed-point theory. It provides a sufficient condition for the existence and uniqueness of fixed points of certain mappings in complete metric spaces.

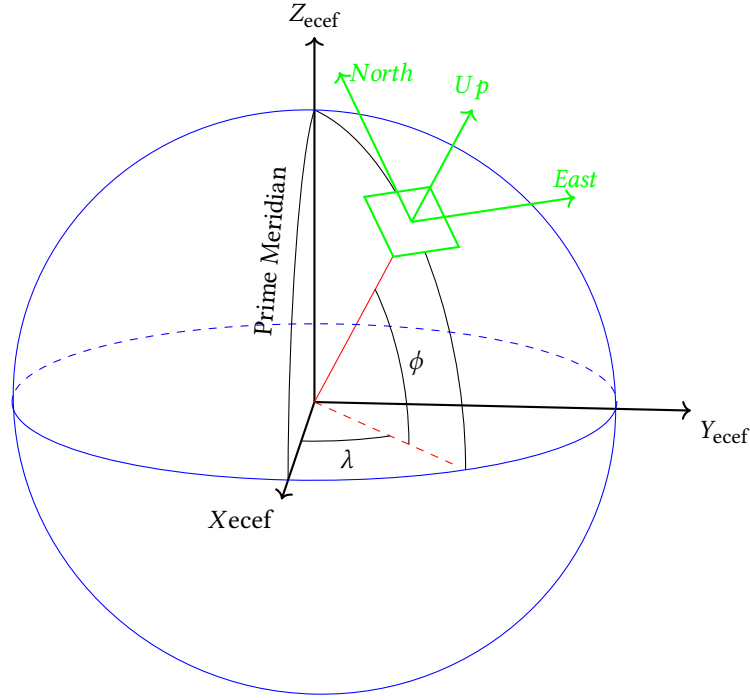
5.4.1 Statement of the Banach Contraction Principle

Let (X, d) be a complete metric space. A mapping $T : X \rightarrow X$ is called a contraction if there exists a constant $0 \leq k < 1$ such that for all $x, y \in X$,

$$d(T(x), T(y)) \leq k \cdot d(x, y).$$

The Banach Contraction Principle states that if T is a contraction mapping on a complete metric space (X, d) , then T has a unique fixed point $x^* \in X$, i.e., $T(x^*) = x^*$. Moreover, for any initial point $x_0 \in X$, the sequence defined by $x_{n+1} = T(x_n)$ converges to the fixed point x^* .

Figure 5.1: the Earth-Centered Earth-Fixed (ECEF) coordinate system



5.4.2 Proof of the Banach Contraction Principle

Proof. Let $x_0 \in X$ be an arbitrary point, and define a sequence $\{x_n\}$ by $x_{n+1} = T(x_n)$ for $n \geq 0$. First, we show that $\{x_n\}$ is a Cauchy sequence. For $m > n$, we have

$$d(x_m, x_n) = d(T(x_{m-1}), T(x_{n-1})).$$

Applying the contraction property of T ,

$$d(T(x_{m-1}), T(x_{n-1})) \leq k \cdot d(x_{m-1}, x_{n-1}).$$

Using the contraction property repeatedly, we obtain

$$d(x_m, x_n) \leq k^{m-n} \cdot d(x_n, x_{n-1}).$$

Consider the distances $d(x_{n+1}, x_n)$:

$$d(x_{n+1}, x_n) = d(T(x_n), T(x_{n-1})) \leq k \cdot d(x_n, x_{n-1}).$$

Thus, we have

$$d(x_{n+1}, x_n) \leq k^n \cdot d(x_1, x_0).$$

Summing up the distances for $n = 0, 1, 2, \dots, N-1$, we get

$$d(x_m, x_n) \leq \sum_{i=n}^{m-1} d(x_{i+1}, x_i) \leq \sum_{i=n}^{m-1} k^i \cdot d(x_1, x_0).$$

Since $0 \leq k < 1$, the sum $\sum_{i=n}^{\infty} k^i$ converges and is bounded by $\frac{k^n}{1-k}$. Therefore,

$$d(x_m, x_n) \leq \frac{k^n}{1-k} \cdot d(x_1, x_0).$$

As $n \rightarrow \infty$, $\frac{k^n}{1-k} \rightarrow 0$, which implies $d(x_m, x_n) \rightarrow 0$ as $n, m \rightarrow \infty$. Thus, $\{x_n\}$ is a Cauchy sequence. Since X is a complete metric space, $\{x_n\}$ converges to some limit $x^* \in X$. To show that x^* is a fixed point of T , we note that T is continuous (as it is a contraction), so

$$T(x^*) = T\left(\lim_{n \rightarrow \infty} x_n\right) = \lim_{n \rightarrow \infty} T(x_n) = \lim_{n \rightarrow \infty} x_{n+1} = x^*.$$

Finally, to prove the uniqueness of the fixed point, suppose y^* is another fixed point of T . Then,

$$d(x^*, y^*) = d(T(x^*), T(y^*)) \leq k \cdot d(x^*, y^*).$$

Since $0 \leq k < 1$, this implies $d(x^*, y^*) = 0$, so $x^* = y^*$.

Thus, T has a unique fixed point x^* , and the sequence $\{x_n\}$ converges to x^* for any initial point $x_0 \in X$. □

The Banach Contraction Principle is a powerful tool in numerical analysis and has applications in solving non-linear equations, differential equations, and optimization problems.

5.5 Exercises

Exercise 5.1. Let $X = [0, 1]$ with the standard Euclidean distance $d(x, y) = |x - y|$. Consider the function $T(x) = \frac{1}{2}x + \frac{1}{2}$. Show that T is a contraction mapping and find its fixed point. □

Exercise 5.2. Let $X = \mathbb{R}^2$ with the Euclidean distance. Consider the mapping $T(x, y) = \left(\frac{1}{2}x, \frac{1}{3}y\right)$. Prove that T is a contraction mapping and find its fixed point. □

Exercise 5.3. Consider the mapping $T : \mathbb{R} \rightarrow \mathbb{R}$ defined by $T(x) = \frac{x}{2}$. Show that T is a contraction mapping on $\mathbb{R}$ and find its fixed point. What happens if we consider T on the interval $[1, 2]$? □

5.5.1 Algorithm

The fixed point iteration algorithm can be described as follows:

Algorithm 5.1 Fixed Point Iteration

Require: A function $g(\mathbf{x})$, initial guess $\mathbf{x}_0$, and tolerance ϵ

Ensure: Approximate solution $\mathbf{x}$

- 1: Initialize $\mathbf{x} = \mathbf{x}_0$
 - 2: **while** $\|g(\mathbf{x}) - \mathbf{x}\| > \epsilon$ **do**
 - 3: Update $\mathbf{x} \leftarrow g(\mathbf{x})$
 - 4: **end while**
 - 5: **return** $\mathbf{x}$
-

5.5.2 Example

Consider the system of equations:

$$\begin{cases} x_1 = \cos(x_2) \\ x_2 = \sin(x_1) \end{cases}$$

Using fixed point iteration with an initial guess $\mathbf{x}_0 = (0.5, 0.5)$, we approximate the solution.

5.6 Newton's Method

Newton's method for multi-dimensional root finding uses the Jacobian matrix to iteratively improve the solution to a system of nonlinear equations $\mathbf{f}(\mathbf{x}) = \mathbf{0}$.

5.6.1 Algorithm

The Newton's method algorithm can be described as follows:

Algorithm 5.2 Newton's Method

Require: A function $\mathbf{f}(\mathbf{x})$, its Jacobian $J(\mathbf{x})$, initial guess $\mathbf{x}_0$, and tolerance ϵ

Ensure: Approximate solution $\mathbf{x}$

```
1: Initialize  $\mathbf{x} = \mathbf{x}_0$ 
2: while  $\|\mathbf{f}(\mathbf{x})\| > \epsilon$  do
3:   Compute  $\Delta\mathbf{x} = -J(\mathbf{x})^{-1}\mathbf{f}(\mathbf{x})$ 
4:   Update  $\mathbf{x} \leftarrow \mathbf{x} + \Delta\mathbf{x}$ 
5: end while
6: return  $\mathbf{x}$ 
```

5.6.2 An Example

Example 5.1 (Finding root by Newton's Method). Consider the system of equations:

$$\begin{cases} x_1^2 + x_2^2 - 1 = 0 \\ x_1^2 - x_2 = 0 \end{cases}$$

Using Newton's method with an initial guess $\mathbf{x}_0 = (0.5, 0.5)$, we approximate the solution. $\square$

Solution By-Hand

We define a mapping $f : \mathbb{R}^2 \rightarrow \mathbb{R}^2$:

$$f(x_1, x_2) \stackrel{\text{def}}{=} (x_1^2 + x_2^2 - 1, x_1^2 - x_2).$$

We find the Jacobian of f :

$$J(x_1, x_2) = \begin{bmatrix} 2x_1 & 2x_2 \\ 2x_1 & -1 \end{bmatrix}.$$

Newton's iteration scheme requires the inverse:

$$J(x_1, x_2)^{-1} = \frac{1}{-2x_1 - 2x_1x_2} \begin{bmatrix} -1 & -2x_2 \\ -2x_1 & 2x_1 \end{bmatrix}.$$

Newton's Method is simply the fixed point method applied to the map $g : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ where

$$g\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right) \stackrel{\text{def}}{=} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} - \frac{1}{-2x_1 - 2x_1x_2} \begin{bmatrix} -1 & -2x_2 \\ -2x_1 & 2x_1 \end{bmatrix} \begin{bmatrix} x_1^2 + x_2^2 - 1 \\ x_1^2 - x_2 \end{bmatrix}$$

An implementation in Python

```
1 import numpy as np
2
3 def f(x):
4     return np.array([x[0]**2 + x[1]**2 - 1, x[0]**2 - x[1]])
5
```

```

6 def jacobian(x):
7     return np.array([[2*x[0], 2*x[1]], [2*x[0], -1]])
8
9 def newton_method(x0, tol=1e-6, max_iter=100):
10    x = x0
11    for _ in range(max_iter):
12        J = jacobian(x)
13        F = f(x)
14        delta_x = np.linalg.solve(J, -F)
15        x = x + delta_x
16        if np.linalg.norm(F) < tol:
17            break
18        return x
19
20 x0 = np.array([0.5, 0.5])
21 solution = newton_method(x0)
22 print("Solution:", solution)

```

Listing 5.1: Newton's Method in Python

This book executes the Python directly, using *Pythontex*. Here is the code without the print statements. The results are made available to L^AT_EX, and they are printed and commented upon after the code.

————— Newton's Method —————

```

1 import numpy as np
2
3 def f(x):
4     return np.array([x[0]**2 + x[1]**2 - 1, x[0]**2 - x[1]])
5
6 def jacobian(x):
7     return np.array([[2*x[0], 2*x[1]], [2*x[0], -1]])
8
9 def newton_method(x0, tol=1e-6, max_iter=100):
10    x = x0
11    for _ in range(max_iter):
12        J = jacobian(x)
13        F = f(x)
14        delta_x = np.linalg.solve(J, -F)
15        x = x + delta_x
16        if np.linalg.norm(F) < tol:
17            break
18        return x
19
20 x0 = np.array([0.5, 0.5])
21 solution = newton_method(x0)

```

The code produces solution: [0.78615138 0.61803399]

5.7 Proof of Convergence of Newton's Method

Newton's method is a powerful iterative technique for finding the roots of a system of nonlinear equations. In the multi-dimensional case, the method involves the Jacobian matrix of the system. We will present a theorem on the convergence of Newton's method and provide a proof.

5.7.1 Theorem

Theorem 5.1. Let $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^n$ be continuously differentiable. Suppose $\mathbf{x}^* \in \mathbb{R}^n$ is a solution to $\mathbf{f}(\mathbf{x}^*) = \mathbf{0}$, and the Jacobian matrix $J(\mathbf{x}^*) = \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\mathbf{x}^*}$ is nonsingular. If $\mathbf{x}_0$ is sufficiently close to $\mathbf{x}^*$, then the sequence $\{\mathbf{x}_k\}$ defined by

$$\mathbf{x}_{k+1} = \mathbf{x}_k - J(\mathbf{x}_k)^{-1} \mathbf{f}(\mathbf{x}_k)$$

converges to $\mathbf{x}^*$, and the convergence is quadratic. $\square$

5.7.2 Proof

Proof. Let $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^n$ be twice continuously differentiable, and suppose $\mathbf{x}^*$ is a solution to $\mathbf{f}(\mathbf{x}^*) = \mathbf{0}$. Let $J(\mathbf{x})$ denote the Jacobian matrix of $\mathbf{f}$ at $\mathbf{x}$. Since $J(\mathbf{x}^*)$ is nonsingular, there exists an inverse $J(\mathbf{x}^*)^{-1}$.

We use the Taylor series expansion of $\mathbf{f}$ around $\mathbf{x}^*$:

$$\mathbf{f}(\mathbf{x}) = \mathbf{f}(\mathbf{x}^*) + J(\mathbf{x}^*)(\mathbf{x} - \mathbf{x}^*) + \mathbf{R}(\mathbf{x}),$$

where $\mathbf{R}(\mathbf{x})$ is the remainder term. Since $\mathbf{f}(\mathbf{x}^*) = \mathbf{0}$, we have

$$\mathbf{f}(\mathbf{x}) = J(\mathbf{x}^*)(\mathbf{x} - \mathbf{x}^*) + \mathbf{R}(\mathbf{x}).$$

Newton's method updates the iterate $\mathbf{x}_k$ as follows:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - J(\mathbf{x}_k)^{-1} \mathbf{f}(\mathbf{x}_k).$$

Let $\mathbf{e}_k = \mathbf{x}_k - \mathbf{x}^*$ be the error at the k -th iteration. Then,

$$\mathbf{f}(\mathbf{x}_k) = J(\mathbf{x}^*)(\mathbf{x}_k - \mathbf{x}^*) + \mathbf{R}(\mathbf{x}_k) = J(\mathbf{x}^*)\mathbf{e}_k + \mathbf{R}(\mathbf{x}_k).$$

Using the update formula,

$$\mathbf{x}_{k+1} = \mathbf{x}_k - J(\mathbf{x}_k)^{-1} \mathbf{f}(\mathbf{x}_k).$$

Substituting $\mathbf{f}(\mathbf{x}_k)$,

$$\mathbf{x}_{k+1} = \mathbf{x}_k - J(\mathbf{x}_k)^{-1} (J(\mathbf{x}^*)\mathbf{e}_k + \mathbf{R}(\mathbf{x}_k)).$$

We can write this as:

$$\mathbf{e}_{k+1} = \mathbf{x}_{k+1} - \mathbf{x}^* = \mathbf{e}_k - J(\mathbf{x}_k)^{-1} J(\mathbf{x}^*)\mathbf{e}_k - J(\mathbf{x}_k)^{-1} \mathbf{R}(\mathbf{x}_k).$$

Since $J(\mathbf{x}^*)$ is nonsingular and $\mathbf{x}_k$ is sufficiently close to $\mathbf{x}^*$, $J(\mathbf{x}_k)$ is also invertible. Using the continuous differentiability of J at $\mathbf{x}^*$,

$$J(\mathbf{x}_k) = J(\mathbf{x}^*) + \mathbf{E}_k,$$

where $\mathbf{E}_k = \mathcal{O}(\|\mathbf{e}_k\|)$ as $\mathbf{x}_k \rightarrow \mathbf{x}^*$.

Thus,

$$\mathbf{e}_{k+1} = \mathbf{e}_k - (J(\mathbf{x}^*) + \mathbf{E}_k)^{-1} (J(\mathbf{x}^*)\mathbf{e}_k + \mathbf{R}(\mathbf{x}_k)).$$

Using the approximation $(I + \mathbf{E}_k J(\mathbf{x}^*)^{-1})^{-1} = I - \mathbf{E}_k J(\mathbf{x}^*)^{-1} + \mathcal{O}(\|\mathbf{E}_k\|^2)$ for small $\mathbf{E}_k$,

$$\mathbf{e}_{k+1} \approx \mathbf{e}_k - (I - \mathbf{E}_k J(\mathbf{x}^*)^{-1}) (J(\mathbf{x}^*)\mathbf{e}_k + \mathbf{R}(\mathbf{x}_k)).$$

Simplifying,

$$\mathbf{e}_{k+1} \approx \mathbf{e}_k - \mathbf{e}_k - J(\mathbf{x}_k)^{-1} \mathbf{R}(\mathbf{x}_k) + \mathbf{E}_k J(\mathbf{x}^*)^{-1} J(\mathbf{x}^*)\mathbf{e}_k.$$

$$\mathbf{e}_{k+1} \approx -J(\mathbf{x}_k)^{-1} \mathbf{R}(\mathbf{x}_k).$$

Since $\mathbf{R}(\mathbf{x}_k) = \mathcal{O}(\|\mathbf{e}_k\|^2)$, we have

$$\mathbf{e}_{k+1} = \mathcal{O}(\|\mathbf{e}_k\|^2).$$

Thus, the convergence is quadratic, and the sequence $\{\mathbf{x}_k\}$ converges to the unique fixed point $\mathbf{x}^*$ if the initial guess $\mathbf{x}_0$ is sufficiently close to $\mathbf{x}^*$. $\square$

5.8 Sherman-Morrison Formula

The Sherman-Morrison formula is a mathematical result that provides a way to efficiently compute the inverse of a rank-1 update to a matrix. If $A \in \mathbb{R}^{n \times n}$ is an invertible matrix and $u, v \in \mathbb{R}^n$, then the rank-1 update $A + uv^\top$ is also invertible provided $1 + v^\top A^{-1}u \neq 0$. The formula for the inverse of this updated matrix is:

$$(A + uv^\top)^{-1} = A^{-1} - \frac{A^{-1}uv^\top A^{-1}}{1 + v^\top A^{-1}u}.$$

Key Properties and Insights

- **Efficiency:** Computing the inverse of $A + uv^\top$ directly would require $\mathcal{O}(n^3)$ operations, but using the Sherman-Morrison formula reduces this to $\mathcal{O}(n^2)$.
- **Conditions:** The formula is valid only if $1 + v^\top A^{-1}u \neq 0$. This ensures the denominator is well-defined and the updated matrix is invertible.
- **Rank-1 Updates:** The formula is particularly useful in iterative optimization algorithms, such as quasi-Newton methods, where rank-1 updates are common.

Example

Let $A = I$, $u = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$, and $v = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$. Compute the inverse of $A + uv^\top$ using the Sherman-Morrison formula. Verify your result by direct computation.

5.9 Broyden's Good Method

Broyden's Good Method featured in Algorithm 5.9 is a quasi-Newton method that updates an approximate *inverse* Jacobian matrix instead of computing it directly, making the method more efficient for large systems. It achieves it by avoiding the computation of the Jacobian and solving a linear system upon every iteration.

5.9.1 The Purpose of the Update Rule for B

The Algorithm 5.9 provides detailed pseudo-code of the method.

The main idea of the algorithm is to initialize and update a matrix $\mathbf{B}$ which serves as a proxy for $\mathbf{J}(\mathbf{x}^*)$, where $\mathbf{x}^*$ is the solution to the equation $\mathbf{f}(\mathbf{x}) = 0$.

The update formula for $\mathbf{B}$ is:

$$\mathbf{B}_{\text{new}} = \mathbf{B} + \frac{(\Delta \mathbf{x} - \mathbf{B} \cdot \Delta \mathbf{F}) \cdot \Delta \mathbf{x}^\top \cdot \mathbf{B}}{\Delta \mathbf{x}^\top \cdot \mathbf{B} \cdot \Delta \mathbf{F}}$$

where

$$\Delta \mathbf{x} = \mathbf{x}_{\text{new}} - \mathbf{x}$$

and

$$\Delta \mathbf{F} = \mathbf{f}(\mathbf{x}_{\text{new}}) - \mathbf{f}(\mathbf{x})$$

represent the changes in the solution estimate and function values, respectively.

This update adjusts $\mathbf{B}$ to better reflect the local behavior of $\mathbf{f}$ around the current estimate, using the latest information from changes in $\mathbf{x}$ and $\mathbf{f}(\mathbf{x})$.

The key aspects of the update rule are:

- Ensures efficiency by avoiding direct Jacobian computation, using the secant condition.
- Fulfills the *secant condition*

$$\mathbf{B}_{\text{new}} \cdot \Delta \mathbf{F} = \Delta \mathbf{x} \tag{5.1}$$

crucial for convergence.

- Incorporates the most recent information ($\Delta \mathbf{x}$ and $\Delta \mathbf{f}$) to refine $\mathbf{B}$.
- Designed to preserve inverse properties, ensuring $\mathbf{B}_{\text{new}}$ continues to approximate the inverse Jacobian well.

Algorithm 5.3 Broyden's Good Method

Require: Starting point $\mathbf{x}_0$, function $\mathbf{f}(\mathbf{x})$, maximum iterations *maxIter*, tolerance *tol*

Ensure: Approximation of $\mathbf{x}$ such that $\mathbf{f}(\mathbf{x}) \approx \mathbf{0}$

- 1: Initialize $\mathbf{x} \leftarrow \mathbf{x}_0$
- 2: Compute $\mathbf{F} \leftarrow \mathbf{f}(\mathbf{x})$
- 3: Approximate initial Jacobian $\mathbf{J}$ at $\mathbf{x}_0$, or use identity matrix. Compute $\mathbf{B} \leftarrow \mathbf{J}^{-1}$.
- 4: **for** $k = 1$ to *maxIter* **do**
- 5: Calculate step direction $\Delta \mathbf{x} \leftarrow -\mathbf{B} \cdot \mathbf{F}$
- 6: Update solution estimate $\mathbf{x}_{\text{new}} \leftarrow \mathbf{x} + \Delta \mathbf{x}$
- 7: Compute new function value $\mathbf{F}_{\text{new}} \leftarrow \mathbf{f}(\mathbf{x}_{\text{new}})$
- 8: **if** $\|\mathbf{F}_{\text{new}}\| < \text{tol}$ **then**
- 9: Print "Solution found"
- 10: **return** $\mathbf{x}_{\text{new}}$
- 11: **end if**
- 12: Calculate $\Delta \mathbf{F} \leftarrow \mathbf{F}_{\text{new}} - \mathbf{F}$
- 13: Update $\mathbf{B}$ using Broyden's formula:

$$\mathbf{B}_{\text{new}} \leftarrow \mathbf{B} + \frac{(\Delta \mathbf{x} - \mathbf{B} \cdot \Delta \mathbf{F}) \cdot \Delta \mathbf{x}^\top \cdot \mathbf{B}}{\Delta \mathbf{x}^\top \cdot \mathbf{B} \cdot \Delta \mathbf{F}}$$

- 14: $\mathbf{x} \leftarrow \mathbf{x}_{\text{new}}, \mathbf{B} \leftarrow \mathbf{B}_{\text{new}}, \mathbf{F} \leftarrow \mathbf{F}_{\text{new}}$
 - 15: **end for**
 - 16: Print "Maximum iterations exceeded"
 - 17: **return** the last computed $\mathbf{x}$
-

5.9.2 Example

Consider the same system of equations:

$$\begin{cases} x_1^2 + x_2^2 - 1 = 0 \\ x_1^2 - x_2 = 0 \end{cases}$$

Using Broyden's Good Method with an initial guess $\mathbf{x}_0 = (0.5, 0.5)$ and an initial Jacobian approximation $B_0 = I$ (identity matrix), we approximate the solution.

```
1 import numpy as np
2
3 def f(x):
4     return np.array([x[0]**2 + x[1]**2 - 1, x[0]**2 - x[1]])
5
6 def broyden_good_method(x0, B0, tol=1e-6, max_iter=100):
7     status = False
8     x = x0
9     B = B0
10    F = f(x)
11    for _ in range(max_iter):
12        delta_x = np.dot(B, -F)
13        x_new = x + delta_x
14        F_new = f(x_new)
15        err = np.linalg.norm(F)
16        print(err)
17        if err < tol:
18            status = True
19            break
20        delta_F = F_new - F
21        B = B + ( np.outer((delta_x - B @ delta_F), delta_x) @ B) / \
22            (delta_x @ B @ delta_F)
```

```

23     x = x_new
24     F = F_new
25     return x, status
26
27 x0 = np.array([0.5, 0.5])
28 B0 = np.eye(2)
29 solution, status = broyden_good_method(x0, B0)
30 if not status:
31     print("Maximum iterations exceeded.")
32
33 print("Solution:", solution)

```

Listing 5.2: Broyden's Good Method in Python

This book executes the Python directly, using *Pythontex*. Here is the code without the print statements. The results are made available to L^AT_EX, and they are printed and commented upon after the code.

Broyden's Good Method

```

1  import numpy as np
2
3  def f(x):
4      return np.array([x[0]**2 + x[1]**2 - 1, x[0]**2 - x[1]])
5
6  def broyden_good_method(x0, B0, tol=1e-6, max_iter=100):
7      status = False
8      x = x0
9      B = B0
10     F = f(x)
11     for _ in range(max_iter):
12         delta_x = np.dot(B, -F)
13         x_new = x + delta_x
14         F_new = f(x_new)
15         err = np.linalg.norm(F)
16         if err < tol:
17             status = True
18             break
19         delta_F = F_new - F
20         B = B + ( np.outer((delta_x - B @ delta_F), delta_x) @ B) / \
21             (delta_x @ B @ delta_F)
22         x = x_new
23         F = F_new
24     return x, status
25
26
27 x0 = np.array([0.5, 0.5])
28 B0 = np.eye(2)
29 solution, status = broyden_good_method(x0, B0)

```

The solution found is: [0.5 0.5]. The status is: False

5.10 Proof of Convergence of Broyden's Good Method

Broyden's Good Method is a quasi-Newton method for solving systems of nonlinear equations. It updates an approximate Jacobian matrix instead of computing it directly, making it efficient for large systems. We will present a theorem on the convergence of Broyden's Good Method and provide a proof.

5.10.1 Theorem

Theorem 5.2. Let $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^n$ be continuously differentiable, and let $\mathbf{x}^* \in \mathbb{R}^n$ be a solution to $\mathbf{f}(\mathbf{x}^*) = \mathbf{0}$. Assume that the Jacobian $J(\mathbf{x}^*) = \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\mathbf{x}^*}$ is nonsingular. If the initial guess $\mathbf{x}_0$ and the initial Jacobian approximation B_0 are sufficiently close to $\mathbf{x}^*$ and $J(\mathbf{x}^*)^{-1}$, respectively, then the sequence $\{\mathbf{x}_k\}$ defined by

$$\mathbf{x}_{k+1} = \mathbf{x}_k - B_k \mathbf{f}(\mathbf{x}_k)$$

with the Broyden update

$$\mathbf{B}_{k+1} = \mathbf{B}_k + \frac{(\Delta \mathbf{x}_k - \mathbf{B}_k \Delta \mathbf{f}_k) \Delta \mathbf{x}_k^\top}{\Delta \mathbf{x}_k^\top \mathbf{B}_k \Delta \mathbf{f}_k},$$

where $\Delta \mathbf{x}_k = \mathbf{x}_{k+1} - \mathbf{x}_k$ and $\Delta \mathbf{f}_k = \mathbf{f}(\mathbf{x}_{k+1}) - \mathbf{f}(\mathbf{x}_k)$, converges to $\mathbf{x}^*$. $\square$

5.10.2 Proof

Will be written one day.

5.11 Kantorovich Theorem on Newton's Method

In this section we formulate a variant of Kantorovich Theorem, which lists sufficient, verifiable conditions to guarantee convergence of the Newton's Method.

There are many formulations of this theorem in literature, and ours does not precisely correspond to any of them. Some details are omitted to have a simple formulation and a compact proof.

Theorem 5.3 (Kantorovich Theorem On Newton's Method). Let $F : \mathbb{R}^n \rightarrow \mathbb{R}^n$ be a continuously differentiable function, and let $x_0 \in \mathbb{R}^n$ be an initial guess. Define the Jacobian matrix of F at x as $J(x)$. Assume the following conditions:

1. **Invertibility:** The Jacobian $J(x_0)$ is nonsingular.
2. **Lipschitz Continuity:** There exists a constant $L > 0$ such that

$$\|J(x) - J(y)\| \leq L\|x - y\|, \quad \text{for all } x, y \in B(x_0, \delta),$$

where $B(x_0, \delta)$ is the ball of radius δ around x_0 .

3. **Norm Bounds:** The inverse of the Jacobian satisfies

$$\|J(x_0)^{-1}\| \leq M, \quad \text{and} \quad \|J(x_0)^{-1}F(x_0)\| \leq \eta,$$

where $M > 0$ and $\eta > 0$.

4. **Radius Condition:** The following condition holds:

$$\delta \geq \frac{1}{ML} \geq 2\eta.$$

The right-most inequality is equivalent to $ML\eta \leq \frac{1}{2}$.

Under these conditions, the Newton iteration:

$$x_{k+1} = x_k - J(x_k)^{-1}F(x_k),$$

satisfies the following properties:

1. **Convergence:** There exists a limit $x^* = \lim_{k \rightarrow \infty} x_k$ which is a solution to $F(x) = 0$ and $x^* \in B(x_0, \frac{1}{LM})$.

2. **Rate of Convergence:** The convergence is quadratic and

$$\|x_k - x^*\| \leq \frac{1}{ML} \cdot 2^{1-2^p}, \quad \forall k.$$

3. **Uniqueness:** If, in addition,

$$\|J(x^*)^{-1}\| \leq M$$

then the solution x^* is the unique solution to $F(x) = 0$ in the ball $B(x_0, \frac{1}{LM})$.

□

Proof of the Kantorovich Theorem

Let $F : \mathbb{R}^n \rightarrow \mathbb{R}^n$ be a continuously differentiable function, and suppose the conditions of the theorem hold.

1. First order Taylor Expansion Remainder Bound

The Taylor formula used is:

$$F(x) = F(x_0) + J(x_0)(x - x_0) + \int_0^1 (J(x_0 + t(x - x_0)) - J(x_0))(x - x_0) dt.$$

Therefore, the remainder formula is:

$$R(x_0, x) = \int_0^1 (J(x_0 + t(x - x_0)) - J(x_0))(x - x_0) dt.$$

This makes it possible to estimate the remainder using the Lipschitz constant of J :

$$\|R(x_0, x)\| \leq \|x - x_0\| \int_0^1 \|J(x_0 + t(x - x_0)) - J(x_0)\| dt \leq \|x - x_0\| \int_0^1 Lt \|x - x_0\| dt = \frac{L}{2} \|x - x_0\|^2.$$

2. Estimating $B^{-1}A$ when A and B are Near

$$\begin{aligned} B^{-1}A &= (A + (B - A))^{-1}A = (A(I + A^{-1}(B - A)))^{-1}A \\ &= (I + A^{-1}(B - A))^{-1}, \\ \|B^{-1}A\| &\leq \frac{1}{1 - \|A^{-1}\| \|B - A\|} \quad (\text{by Neumann Series}). \end{aligned}$$

3. Recursive Estimates

Let M_k and η_k be recursively defined so that:

$$\begin{aligned} M_0 &= M, \\ \eta_0 &= \eta, \\ \|J(x_k)^{-1}\| &\leq M_k, \quad \forall k, \\ \|J(x_k)^{-1}F(x_k)\| &\leq \eta_k, \quad \forall k. \end{aligned}$$

We have

$$\begin{aligned} \|J(x_{k+1})^{-1}\| &= \|(J(x_{k+1})^{-1}J(x_k))J(x_k)^{-1}\| \leq \|(J(x_{k+1})^{-1}J(x_k))\| \|J(x_k)^{-1}\| \\ &\leq \frac{1}{1 - \|J(x_k)^{-1}\| \|J(x_{k+1}) - J(x_k)\|} \|J(x_k)^{-1}\| \leq \frac{M_k}{1 - M_k L \eta_k}. \end{aligned}$$

We also have

$$F(x_{k+1}) = \overbrace{F(x_k) + J(x_k)(x_{k+1} - x_k)}^{=0} + R(x_k, x_{k+1}) = R(x_k, x_{k+1}).$$

Therefore

$$\|J(x)_{k+1}^{-1}F(x_{k+1})\| = \|J(x)_{k+1}^{-1}R(x_k, x_{k+1})\| \leq M_{k+1} \frac{1}{2} L \|x_{k+1} - x_k\|^2 \leq M_{k+1} \frac{1}{2} L \eta_k^2.$$

In summary, we can define M_k and η_k by these recurrence relations:

$$\begin{cases} M_{k+1} = \frac{M_k}{1 - M_k L \eta_k}, \\ \eta_{k+1} = M_{k+1} \frac{1}{2} L \eta_k^2. \end{cases}$$

We note that the second equation is responsible for the quadratic convergence rate, while the first equation is sort of a nuisance, making M_k grow.

4. Convergence Considerations

Let $\Delta x_k = x_{k+1} - x_k = J(x_k)^{-1}F(x_k)$. Hence, $x_p - x_0 = \sum_{k=0}^{p-1} \Delta x_k$. The limit $x^* \stackrel{\text{def}}{=} \lim_{k \rightarrow \infty} x_k$ exists if the series $\sum_{\ell=0}^{\infty} \Delta x_\ell$ converges in norm. The series is majorized by the series $\sum_{\ell=0}^{\infty} \eta_\ell$. Thus, we need to prove that this series converges and its sum is $< \delta$, so that all inequalities are justified. After simple manipulations:

$$M_{k+1} L \eta_{k+1} = \frac{1}{2} M_{k+1} L^2 \eta_k^2 = \frac{M_{k+1}}{2M_k} (M_k L \eta_k)^2 = \frac{1}{2(1 - M_k L \eta_k)} (M_k L \eta_k)^2$$

If $M_0 L \eta_0 \leq \frac{1}{2}$ then by induction it follows that $M_{k+1} L \eta_{k+1} \leq (M_k L \eta_k)^2$ and, in particular, that

$$M_k L \eta_k \leq 2^{-2^k}, \forall k.$$

Hence, in view of $M_k \geq M_0 = M$ we have $\sum_{k=0}^{\infty} M L \eta_k \leq \sum_{k=0}^{\infty} 2^{-2^k} < 1$, which yields:

$$\sum_{k=0}^{\infty} \eta_k < \frac{1}{ML}.$$

Therefore, if $\delta \geq \frac{1}{ML}$ then we obtain quadratic convergence to $x^* \in B(x_0, \delta)$. We also have an explicit estimate:

$$\|x_k - x^*\| \leq \frac{1}{ML} \sum_{\ell=k}^{\infty} 2^{-2^\ell} < \frac{2}{ML} \cdot 2^{-2^k} = \frac{1}{ML} \cdot 2^{1-2^k}.$$

5. Uniqueness Considerations

Suppose $x^{**} \in B(x_0, \frac{1}{LM})$ is another solution. We have

$$F(x^{**}) = F(x^*) + J(x^*)(x^{**} - x^*) + R(x^*, x^{**}).$$

As $F(x^*) = F(x^{**}) = 0$, we also have $J(x^*)(x^{**} - x^*) + R(x^*, x^{**}) = 0$, and

$$x^{**} - x^* = -J(x^*)^{-1}R(x^*, x^{**}).$$

Taking the norm of both sides,

$$\|x^{**} - x^*\| \leq M \frac{L}{2} \|x^{**} - x^*\|^2.$$

If $x^{**} \neq x^*$ then $1 \leq \frac{ML}{2} \|x^{**} - x^*\|$, which implies $\|x^{**} - x^*\| \geq \frac{2}{ML}$. This is not possible, as both points belong to $B(x_0, \frac{1}{ML})$. □

5.11.1 Example Applications of Kantorovich Theorem

A 1D example

Example 5.2 (Kantorovich Theorem and Newton-Raphson Iteration). Let $F(x) = x^2 - 2$. The positive root is $x = \sqrt{2}$. For which initial guesses x_0 is the Newton-Raphson Method guaranteed to converge? $\square$

We have $J(x) = 2x$, which is a Lipschitz function with Lipschitz constant $L = 2$ on the entire line.

Step 1: Choose M and η

The optimized choices of M, η are:

$$M = \frac{1}{|J(x_0)|} = \frac{1}{2|x_0|},$$

$$\eta = \frac{|f(x_0)|}{|J(x_0)|} = \left| \frac{x_0^2 - 2}{2x_0} \right| = \left| \frac{1}{2}x_0 - \frac{1}{x_0} \right|$$

Step 2: Determine which initial guesses lead to convergence

The Radius Condition requires

$$\eta \leq \frac{1}{2ML} = \frac{1}{2 \frac{1}{2|x_0|} 2} = \frac{|x_0|}{2}.$$

Hence, we must satisfy

$$\left| \frac{1}{2}x_0 - \frac{1}{x_0} \right| \leq \frac{|x_0|}{2}.$$

This inequality holds when $\left| 1 - \frac{2}{x_0^2} \right| \leq 1$. Equivalently, $\frac{2}{x_0^2} \leq 2$, or $\boxed{|x_0| \geq 1}$.

Conclusion

When $|x_0| \geq 1$ Newton-Raphson iteration converges to x^* such that

$$|x^* - x_0| \leq \frac{1}{ML} = |x_0|.$$

In short, we show convergence to a root of the same sign as x_0 , as long as $|x_0| \geq 1$. $\square$

Remark 5.1 (On uniform Lipschitz condition). In this example F is a quadratic map. This fact allowed us to verify the Lipschitz condition on the entire domain, with a uniform Lipschitz constant, greatly simplifying verification of the assumptions of the Kantorovich Theorem. Functions such $\sin$, $\cos$, and $\tanh$ are also uniformly Lipschitz. $\square$

A 2D example

We will find the following fact useful in calculations:

Lemma 5.1 (The norm of a rank 1 matrix).

$$\|xy^\top\| = \|x\|\|y\|.$$

$\square$

Proof. If $A = xy^\top$ then

$$\|Au\| = \|x(y^\top u)\| = \|x\| |y^\top u| \leq (\|x\|\|y\|)\|u\|.$$

This shows that $\|A\| \leq \|x\|\|y\|$. If we set $u = y$, we obtain equality. Thus, $\|A\| = \|x\|\|y\|$. $\square$

Example 5.3 (Kantorovich Theorem in 2D – Reaction-Diffusion System). This example models the equilibrium state of two interacting chemical species in a reaction-diffusion system. The equations are:

$$\begin{aligned} D_1 x_1 - k_1 x_1 x_2 - R_1 &= 0, \\ D_2 x_2 + k_2 x_1 x_2 - R_2 &= 0, \end{aligned}$$

where:

- x_1, x_2 : concentrations of two interacting species (e.g., substrate and product),
- D_1, D_2 : diffusion coefficients,
- k_1, k_2 : reaction rates,
- R_1, R_2 : external sources or sinks for the species.

For concreteness, let:

$$D_1 = 1, \quad D_2 = 1, \quad k_1 = 0.1, \quad k_2 = 0.2, \quad R_1 = 1, \quad R_2 = 0.5.$$

The equations become:

$$\begin{aligned} x_1 - 0.1x_1x_2 - 1 &= 0, \\ x_2 + 0.2x_1x_2 - 0.5 &= 0. \end{aligned}$$

A reasonable initial guess for the concentrations might be $x_1 = 1, x_2 = 0.5$. Show that Newton's Method converges.

□

Step 1: Define F

Let $F(x) = (x_1 - 0.1x_1x_2 - 1, x_2 + 0.2x_1x_2 - 0.5)$.

Step 2: Find the Jacobian matrix

The Jacobian matrix for the system is:

$$J(x) = \begin{bmatrix} 1 - 0.1x_2 & -0.1x_1 \\ 0.2x_2 & 1 + 0.2x_1 \end{bmatrix}$$

Step 3: Find the Lipschitz constant

We start by rewriting J as a linear combination of constant matrices:

$$J(x) = \overbrace{\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}}^I + x_1 \overbrace{\begin{bmatrix} 0 & -0.1 \\ 0 & 0.2 \end{bmatrix}}^{J_1} + x_2 \overbrace{\begin{bmatrix} -0.1 & 0 \\ 0.2 & 0 \end{bmatrix}}^{J_2}.$$

This form emphasizes the fact that J is a linear (affine), matrix-valued function of x .

This allows us to easily compute the uniform Lipschitz constant of J because $J(x) - J(y) = (x_1 - y_1)J_1 + (x_2 - y_2)J_2$ and therefore, using Cauchy-Schwartz inequality:

$$\|J(x) - J(y)\| \leq |x_1 - y_1| \|J_1\| + |x_2 - y_2| \|J_2\| \leq \sqrt{|x_1 - y_1|^2 + |x_2 - y_2|^2} \sqrt{\|J_1\|^2 + \|J_2\|^2} = \|x - y\| \sqrt{\|J_1\|^2 + \|J_2\|^2}.$$

Hence, the Lipschitz constant is $L = \sqrt{\|J_1\|^2 + \|J_2\|^2}$. The norms of J_1 and J_2 are easy to find because they are rank 1 matrices (see Lemma 5.1).

As $J_1 = ve_2^\top$ and $J_2 = ve_1^\top$, where $v = [-0.1, 0.2]^\top$, we have $\|J_1\| = \|v\| = 0.1\sqrt{5}$. Hence

$$L \stackrel{\text{def}}{=} 0.1\sqrt{10} \approx 0.316227766016838.$$

This is a uniform Lipschitz constant.

Step 3: Estimate the inverse of the Jacobian matrix

We will use Neumann series inequality to estimate $J(x)^{-1}$:

$$\|J(x)^{-1}\| = \|(I + x_1 J_1 + x_2 J_2)^{-1}\| \leq \frac{1}{1 - \|x_1 J_1 + x_2 J_2\|} \leq \frac{1}{1 - \|x\|\sqrt{\|J_1\|^2 + \|J_2\|^2}} \leq \frac{1}{1 - \|x\|L}.$$

Therefore

$$\|J(x_0)^{-1}\| \leq M \stackrel{\text{def}}{=} \frac{1}{1 - \|x_0\|L}.$$

Step 4: Find η

We need to find $\eta \leq \frac{1}{2ML}$ such that

$$\|J(x_0)^{-1}F(x_0)\| \leq \eta.$$

We will require a stronger condition, based on the observation that $\|J(x_0)^{-1}F(x_0)\| \leq M\|F(x_0)\|$:

$$\|F(x_0)\| \leq \frac{1}{2M^2L}.$$

With out initial guess $x_0 = (1, 0.5)$ we have

$$\begin{aligned} F(x_0) &= (-0.025, 0.05), \\ \|F(x_0)\| &= 0.025\sqrt{5} \approx 0.05590169943749475. \end{aligned}$$

On the other hand

$$\frac{1}{2M^2L} = \frac{(1 - \|x_0\|L)^2}{2L} = \frac{(1 - \sqrt{1.25} \cdot 0.1\sqrt{10})^2}{2 \cdot 0.1\sqrt{10}} \approx 0.6607471950948183.$$

We can see that η can be defined as follows:

$$\eta \stackrel{\text{def}}{=} \frac{1}{2ML} \approx 1.0221218357092419.$$

Step 4: Find δ

In the absence of constraints other than the Radius Condition, any δ satisfying

$$\delta \geq \frac{1}{ML} = 2\eta$$

satisfies the requirements. In particular, $\delta = \infty$ works.

Conclusion

Quadratic convergence holds for the specified initial guess. □

Remark 5.2 (On the proximity of the initial guess to the root). In Example 5.3 we managed to quite easily satisfy the assumptions of Kantorovich Theorem. In more realistic examples it is hard to obtain valid estimates. We may want to speculatively iterate some initial condition until it is close to the solution. Eventually, the assumptions will be satisfied by replacing x_0 with some x_k . □

5.12 Exercises

Exercise 5.4. Solve the following system of equations using fixed point iteration:

$$\begin{cases} x_1 = e^{-x_1} \\ x_2 = \cos(x_2) \end{cases}$$

Use an initial guess $\mathbf{x}_0 = (0.5, 0.5)$ and a tolerance of 10^{-6} . $\square$

Exercise 5.5. Consider the system of nonlinear equations:

$$\begin{cases} x_1^3 + x_2 - 1 = 0 \\ x_1 + x_2^3 - 1 = 0 \end{cases}$$

Use Newton's method to solve this system with an initial guess $\mathbf{x}_0 = (0.5, 0.5)$. Verify the quadratic convergence by computing the error at each iteration. $\square$

Exercise 5.6. Let $\mathbf{f} : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ be defined by $\mathbf{f}(x, y) = (x^2 + y^2 - 1, x^2 - y)$. Show that $J(\mathbf{x}^*)$ is nonsingular at the solution $\mathbf{x}^*$ and apply Newton's method to approximate the solution starting from $\mathbf{x}_0 = (0.5, 0.5)$. $\square$

Exercise 5.7. For the system of equations:

$$\begin{cases} e^{x_1} - x_2 = 0 \\ x_1^2 + x_2^2 - 4 = 0 \end{cases}$$

use Newton's method to find the roots starting from an initial guess $\mathbf{x}_0 = (1, 1)$. Discuss the behavior of the convergence. $\square$

Exercise 5.8 (Secant condition in 1D). Prove that when the secant condition

$$\mathbf{B}_{\text{new}} \cdot \Delta \mathbf{F} = \Delta \mathbf{x}$$

is applied to an $F : \mathbb{R} \rightarrow \mathbb{R}$, Broyden's Method (Algorithm 5.9) reduces to the ordinary Secant Method (Algorithm 3.5). $\square$

Exercise 5.9 (Implementation Task). Implement Broyden's Good Method to solve the following system of equations:

$$\begin{cases} x_1 e^{x_2} - x_2 = 0 \\ x_1^2 + x_2^2 - 4 = 0 \end{cases}$$

Use an initial guess $\mathbf{x}_0 = (1.0, 1.0)$, an initial Jacobian approximation $B_0 = I$, and a tolerance of 10^{-6} . $\square$

Exercise 5.10. Compare the convergence rates of Newton's method and Broyden's Good Method for solving the system of equations given in Exercise 5.5. Plot the norm of the function values against the iteration number for both methods. $\square$

Exercise 5.11 (Kantorovich Conditions in 1D). The equation $x^3 - 2 = 0$ has a root $x^* = \sqrt[3]{2} \approx 1.2599210498948732$. We use Newton's Method to find this root. The initial guess is $x_0 = 1.25$. Find the largest number $\delta > 0$ that you can, such that the assumptions of Kantorovich's Theorem (stated above) are satisfied, and in particular that the Newton's Method is guaranteed to converge to $x^* \in B(x_0, \delta) = (x_0 - \delta, x_0 + \delta)$. $\square$

Exercise 5.12 (Kantorovich Conditions in 2D). The system of equations

$$\begin{cases} x_1^2 + x_2 - 1 = 0 \\ x_1 + x_2^2 - 1 = 0 \end{cases}$$

has a root $x^* = (\phi, \phi)$ where $\phi = \frac{\sqrt{5}-1}{2} \approx 0.6180339887498949$ is the golden ratio. We use Newton's Method to find this root, with the initial guess $x_0 = (0.6, 0.6)$. Find $\delta_{\min} > 0$ and $\delta_{\max} \leq \infty$ such that for $\delta \in [\delta_{\min}, \delta_{\max}]$ the assumptions of Kantorovich's Theorem are satisfied, and in particular that the Newton's Method is $B(x_0, \delta)$. $\square$



Exercise 5.13 (A One-Constant Variant of Kantorovich Theorem). Let $F : B(x_0, \delta) \rightarrow \mathbb{R}^n$ be a differentiable function and $J(x)$ be its Jacobian matrix, satisfying the following conditions:

1. $\|J(x)^{-1}J(y) - I\| \leq \delta^{-1}\|x - y\|$ for all $x, y \in B(x_0, \delta)$;
2. $\|J(x_0)^{-1}F(x_0)\| \leq \frac{\delta}{2}$.

Show that the sequence defined recursively by $x_{k+1} = x_k - J(x_k)^{-1}F(x_k)$ converges to x^* which is the unique solution to $F(x) = 0$ in $B(x_0, \delta)$. Also show that $\forall k \ \|x_k - x^*\| \leq \delta 2^{1-2^k}$. $\square$

Chapter 6

Unconstrained and Constrained Optimization

Unconstrained optimization deals with optimizing a function without any constraints. The goal is to find a point $\mathbf{x}$ that minimizes (or maximizes) the objective function $f(\mathbf{x})$.

6.1 Linearly Constrained Quadratic Optimization in Portfolio Selection

In the study of optimization, **quadratic optimization** occupies a crucial place, particularly when dealing with problems where the objective function is quadratic and the variables are continuous. Unconstrained quadratic optimization refers to the optimization of a quadratic function without any explicit constraints on the variables. Linearly constrained quadratic optimization optimizes a quadratic function on a linear or affine subspace, and is easily reduced to unconstrained optimization.

This class of problems is not only mathematically elegant but also highly applicable in various fields, including finance, engineering, and machine learning.

A general unconstrained quadratic optimization problem can be formulated as:

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top \mathbf{Q} \mathbf{x} + \mathbf{c}^\top \mathbf{x} \quad (6.1)$$

where:

- $\mathbf{x}$ is an n -dimensional vector of decision variables.
- $\mathbf{Q}$ is an $n \times n$ symmetric positive definite matrix.
- $\mathbf{c}$ is an n -dimensional vector of coefficients.

The objective is to find the vector $\mathbf{x}$ that minimizes the quadratic function $f(\mathbf{x})$.

We note that the necessary condition of a minimum is

$$\nabla f(\mathbf{x}) = \mathbf{Q} \mathbf{x} + \mathbf{c} = \mathbf{0}.$$

Thus, unconstrained quadratic optimization is equivalent to solving a linear system with a symmetric, positive definite matrix. Should a solution be found, it is automatically a minimum due to the Second Derivative Criterion.

Remark 6.1 (On alternatives to solving a linear system). In practical applications n can be large and $\mathbf{Q}$ may be a dense matrix with a large condition number. For these reasons, it may be practical to apply other techniques, such as Gradient Descent or Conjugate Gradient Method. $\square$

6.1.1 Application in Markowitz Portfolio Optimization

In finance, **Markowitz's Modern Portfolio Theory** (MPT) provides a framework for constructing portfolios that optimize expected return based on a given level of risk. The risk (variance) of a portfolio composed of multiple assets is a quadratic function of the asset weights, making it an ideal candidate for quadratic optimization techniques.

Portfolio Variance as a Quadratic Function

Consider a portfolio of n assets with weights $\mathbf{w} = [w_1, w_2, \dots, w_n]^\top$ which must sum up to 1. Note that a weight can be negative. For example we can *short-sell* a stock (sell what we do not have!) in exchange for cash, with which we can buy another stock.

The portfolio variance σ_p^2 is given by:

$$\sigma_p^2 = \mathbf{w}^\top \mathbf{\Sigma} \mathbf{w} \quad (6.2)$$

where $\mathbf{\Sigma}$ is the covariance matrix of asset returns.

Linearly Constrained Optimization Problem

The portfolio variance minimization problem is formulated as:

$$\begin{aligned} \text{minimize: } & f(\mathbf{w}) = \frac{1}{2} \mathbf{w}^\top \mathbf{\Sigma} \mathbf{w} \\ \text{subject to: } & \mathbf{1}^\top \mathbf{w} = 1. \end{aligned}$$

Here $\mathbf{1} \stackrel{\text{def}}{=} [1, 1, 1]^\top$.

Example 6.1 (A Three-Asset Example). Let's consider a portfolio with **three assets** to illustrate this concept.

1. **Expected Returns** (not directly used in variance minimization but provided for context):

- Asset 1: $E[R_1] = 8\%$
- Asset 2: $E[R_2] = 12\%$
- Asset 3: $E[R_3] = 15\%$

We capture this data as a *return vector*

$$\mathbf{r} = [0.08, 0.12, 0.15]^\top.$$

2. **The covariance matrix $\mathbf{\Sigma}$:**

$$\mathbf{\Sigma} = \begin{bmatrix} 0.02 & 0.015 & 0.020 \\ 0.015 & 0.05 & 0.025 \\ 0.020 & 0.025 & 0.10 \end{bmatrix}$$

□

Objective Function

The portfolio variance to minimize is:

$$f(\mathbf{w}) = \frac{1}{2} \mathbf{w}^\top \mathbf{\Sigma} \mathbf{w} \quad (6.3)$$

subject to the linear constraint $\mathbf{1}^\top \mathbf{w} = \sum_{j=1}^3 w_j = 1$.

6.1.2 A More Interesting Optimization Problem

Example 6.2 (A Three-Asset Example with Target Return). In the Markowitz's portfolio optimization problem, the return vector is

$$\mathbf{r} = [0.08, 0.12, 0.15]^\top$$

and the covariance matrix is

$$\Sigma = \begin{bmatrix} 0.02 & 0.015 & 0.020 \\ 0.015 & 0.05 & 0.025 \\ 0.020 & 0.025 & 0.10 \end{bmatrix}$$

Suppose now that we desire an expected return d of 10% from our portfolio. Find weight vector $\mathbf{w}$ such that the portfolio has the expected return $d = 0.1$ and minimum variance. $\square$

This leads to a constraint:

$$\mathbf{r}^\top \mathbf{w} = d$$

where $d = 0.1$. The new problem is

$$\begin{aligned} \text{minimize: } & f(\mathbf{w}) = \frac{1}{2} \mathbf{w}^\top \Sigma \mathbf{w} \\ \text{subject to: } & \mathbf{r}^\top \mathbf{w} = d \\ \text{and subject to: } & \mathbf{1}^\top \mathbf{w} = 1. \end{aligned}$$

Again, we minimize a quadratic function, but this time subject to two linear constraints. The constraints are independent unless returns of all assets are the same (thus making the problem trivial). Therefore, we can eliminate two variables from the quadratic form, thus leaving an unconstrained problem.

6.1.3 Lagrange multipliers

From vector calculus we know the method of Lagrange multipliers (see Appendix L) which tells us to introduce the Lagrange function:

$$L(\mathbf{w}, \lambda_1, \lambda_2) = \frac{1}{2} \mathbf{w}^\top \Sigma \mathbf{w} + \lambda_1 (\mathbf{r}^\top \mathbf{w} - d) + \lambda_2 (\mathbf{1}^\top \mathbf{w} - 1).$$

Subsequently, we solve the system of equations

$$\begin{cases} \frac{\partial L}{\partial \mathbf{w}} = 0, \\ \frac{\partial L}{\partial \lambda_1} = 0, \\ \frac{\partial L}{\partial \lambda_2} = 0. \end{cases} \quad \text{or} \quad \begin{cases} \Sigma \mathbf{w} + \lambda_1 \mathbf{r} + \lambda_2 \mathbf{1} = 0, \\ \mathbf{r}^\top \mathbf{w} - d = 0, \\ \mathbf{1}^\top \mathbf{w} - 1 = 0. \end{cases}$$

This is a linear system, and it has a unique solution, which is the desired minimum. This follows from the fact that Σ is positive definite because the quadratic form $f(\mathbf{w}) = \frac{1}{2} \mathbf{w}^\top \Sigma \mathbf{w}$ restricted to a linear subspace is also positive definite, and thus it has a unique minimum on that subspace.

6.1.4 Conclusion

This exploration of unconstrained quadratic optimization in the context of Markowitz portfolio selection illustrates the direct application of mathematical optimization techniques to financial problems.

6.2 Gradient Descent

Gradient Descent presented as Algorithm 6.1 is an iterative optimization algorithm used to find the minimum of a function. It works by taking steps proportional to the negative of the gradient (or approximate gradient) of the function at the current point.

The step size, also referred to as the *learning rate*, is fixed in the basic algorithm. It is common that there is no good step size that will work for all initial guesses. Therefore, in practical applications α will vary with iteration number, subject to often complex rules. Refining these rules is subject of continuing research.

Algorithm 6.1 Gradient Descent

Require: Objective function $f(\mathbf{x})$, initial guess $\mathbf{x}_0$, step size α , and tolerance ϵ

Ensure: Approximate minimum point $\mathbf{x}$

```
1: Initialize  $\mathbf{x} = \mathbf{x}_0$ 
2: while  $\|\nabla f(\mathbf{x})\| > \epsilon$  do
3:   Update  $\mathbf{x} \leftarrow \mathbf{x} - \alpha \nabla f(\mathbf{x})$ 
4: end while
5: return  $\mathbf{x}$ 
```

Example 6.3 (Distance function). Consider the function $f(x, y) = x^2 + y^2$. Using Gradient Descent with an initial guess $(x_0, y_0) = (1, 1)$, a step size $\alpha = 0.1$, and a tolerance $\epsilon = 10^{-6}$, we find the minimum point. $\square$

6.3 Newton's Method for Optimization

Newton's Method for Optimization is an iterative optimization algorithm that uses second-order information (the Hessian matrix) to find the minimum of a function.

The Newton's Method adapted to solving optimization problems is Algorithm 6.2.

Algorithm 6.2 Newton's Method for Optimization

Require: Objective function $f(\mathbf{x})$, initial guess $\mathbf{x}_0$, and tolerance ϵ

Ensure: Approximate minimum point $\mathbf{x}$

```
1: Initialize  $\mathbf{x} = \mathbf{x}_0$ 
2: while  $\|\nabla f(\mathbf{x})\| > \epsilon$  do
3:   Compute the Hessian matrix  $H = \nabla^2 f(\mathbf{x})$ 
4:   Update  $\mathbf{x} \leftarrow \mathbf{x} - H^{-1} \nabla f(\mathbf{x})$ 
5: end while
6: return  $\mathbf{x}$ 
```

Example 6.4 (Newton's Method). Consider the function $f(x, y) = x^2 + y^2$. Using Newton's Method with an initial guess $(x_0, y_0) = (1, 1)$ and a tolerance $\epsilon = 10^{-6}$, we find the minimum point. $\square$

6.4 BFGS Quasi-Newton Method and Its Limited-Memory Variant

Building upon the foundation laid by Broyden's Good Method, Algorithm 5.9 for solving nonlinear equations, we now delve into the Broyden-Fletcher-Goldfarb-Shanno (BFGS) method, a widely used algorithm for unconstrained optimization. We will also discuss its efficient counterpart for large-scale problems, the Limited-memory BFGS (L-BFGS) algorithm.

Naively, we may consider the problem of finding an optimum of a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ as that of finding a root of the equation $\nabla f(\mathbf{x}) = 0$, and therefore we can apply the root-finding algorithms to this equation. We note that $\nabla f : \mathbb{R}^n \rightarrow \mathbb{R}^n$. However, there are crucial optimizations resulting from the knowledge that we are dealing with a gradient. They will be introduced in this section.

6.4.1 BFGS Algorithm

The BFGS method presented as Algorithm 6.3 is an iterative quasi-Newton technique designed to find the minimum of a differentiable scalar function $f : \mathbb{R}^n \rightarrow \mathbb{R}$. It achieves superlinear convergence by constructing an approximation B_k to the Hessian matrix of second derivatives, which is updated at each iteration using gradient information.

Algorithm 6.3 BFGS Quasi-Newton Method

Require: Initial guess x_0 , initial Hessian approximation B_0 , tolerance ϵ , maximum iterations K

- 1: $k \leftarrow 0$
- 2: **while** $k < K$ **and** $\|\nabla f(x_k)\| > \epsilon$ **do**
- 3: Compute search direction: $p_k \leftarrow -B_k^{-1}\nabla f(x_k)$
- 4: Perform line search to find $\alpha_k > 0$ satisfying *Wolfe conditions*
- 5: Update position: $x_{k+1} \leftarrow x_k + \alpha_k p_k$
- 6: Compute difference vectors:

$$\begin{aligned} s_k &\leftarrow x_{k+1} - x_k \\ y_k &\leftarrow \nabla f(x_{k+1}) - \nabla f(x_k) \end{aligned}$$

- 7: Update Hessian approximation:

$$B_{k+1} \leftarrow B_k - \frac{B_k s_k s_k^\top B_k}{s_k^\top B_k s_k} + \frac{y_k y_k^\top}{y_k^\top s_k}$$

- 8: $k \leftarrow k + 1$
 - 9: **end while**
 - 10: **return** x_k
-

Properties of BFGS

- **Positive Definiteness:** If B_0 is positive definite and $y_k^\top s_k > 0$ (satisfied under certain conditions), all subsequent B_k are positive definite.
- **Superlinear Convergence:** Under appropriate conditions, the method exhibits superlinear convergence.
- **Robustness:** BFGS is known for its robustness and efficiency in practice, especially for medium-scale problems.

6.4.2 Sherman-Morrison-Woodbury Formula

The Sherman-Morrison-Woodbury formula is an extension of the Sherman-Morrison formula that handles rank- k updates to a matrix. If $A \in \mathbb{R}^{n \times n}$ is an invertible matrix and $U, V \in \mathbb{R}^{n \times k}$, then the inverse of the rank- k updated matrix $A + UV^\top$ (assuming it is invertible) is given by:

$$(A + UV^\top)^{-1} = A^{-1} - A^{-1}U(I_k + V^\top A^{-1}U)^{-1}V^\top A^{-1},$$

where I_k is the $k \times k$ identity matrix.

Key Properties and Insights

- **Efficiency:** Instead of directly inverting the updated matrix $A + UV^\top$, which would cost $\mathcal{O}(n^3)$, this formula leverages the existing inverse A^{-1} and the smaller matrix $I_k + V^\top A^{-1}U$, reducing computational costs to $\mathcal{O}(nk^2)$ for $k \ll n$.
- **Conditions:** The formula requires $I_k + V^\top A^{-1}U$ to be invertible, which ensures that $A + UV^\top$ is also invertible.

Application to BFGS

In the BFGS method, the update to the inverse Hessian approximation B_k^{-1} is a rank-2 update involving $s_k = x_{k+1} - x_k$ and $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$. The update formula is:

$$B_{k+1}^{-1} = \left(I - \frac{s_k y_k^\top}{y_k^\top s_k} \right) B_k^{-1} \left(I - \frac{y_k s_k^\top}{y_k^\top s_k} \right) + \frac{s_k s_k^\top}{y_k^\top s_k}.$$

This formula can be viewed as a rank-2 update because:

- The first term involves two symmetric rank-1 corrections that subtract curvature along s_k and y_k .
- The second term adds a rank-1 matrix $\frac{s_k s_k^\top}{y_k^\top s_k}$ to incorporate new curvature information.

Example 6.5 (Matrix inversion by Sherman-Morrison-Woodbury formula). Let

$$A = I, \quad U = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \text{and} \quad V = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

Compute $(A + UV^\top)^{-1}$ using the Sherman-Morrison-Woodbury formula. Verify your result by direct computation. $\square$

6.5 Rationale of the BFGS Method

Newton's method uses the second-order Taylor approximation of $f(x)$ at a point x_k :

$$f(x_k + p) \approx f(x_k) + \nabla f(x_k)^\top p + \frac{1}{2} p^\top H_k p,$$

where $H_k = \nabla^2 f(x_k)$ is the Hessian matrix at x_k . The Newton step is computed as:

$$p_k = -H_k^{-1} \nabla f(x_k).$$

However, computing and storing the Hessian H_k and its inverse can be expensive, particularly for high-dimensional problems. The BFGS method avoids this by iteratively constructing an approximation B_k to the Hessian H_k using only gradient information.

Update Formula

The BFGS update formula for the inverse Hessian approximation B_k^{-1} is:

$$B_{k+1}^{-1} = \left(I - \frac{s_k y_k^\top}{y_k^\top s_k} \right) B_k^{-1} \left(I - \frac{y_k s_k^\top}{y_k^\top s_k} \right) + \frac{s_k s_k^\top}{y_k^\top s_k}. \quad (6.4)$$

This update ensures:

- Symmetry: B_{k+1} is symmetric if B_k is symmetric.
- Positive definiteness: B_{k+1} remains positive definite if $y_k^\top s_k > 0$, which holds for a sufficiently small step size α_k .
- Secant Condition: $B_{k+1}^{-1} y_k = s_k$.

The secant condition is verified as follows:

$$\begin{aligned} \frac{s_k s_k^\top}{y_k^\top s_k} y_k &= \frac{s_k (y_k^\top s_k)}{y_k^\top s_k} = \frac{s_k (s_k^\top y_k)}{y_k^\top s_k} = s_k, \\ \left(I - \frac{y_k s_k^\top}{y_k^\top s_k} \right) y_k &= y_k - \frac{y_k (s_k^\top y_k)}{y_k^\top s_k} = y_k - y_k = 0. \end{aligned}$$

An Optimized Update Formula

Instead of using equation (6.4) we can use the expanded form which does not require creating temporary matrices. This form is

$$B_{k+1}^{-1} = B_k^{-1} + \frac{(s_k^\top y_k + y_k^\top B_k^{-1} y_k)(s_k s_k^\top)}{(s_k^\top y_k)^2} - \frac{B_k^{-1} y_k s_k^\top + s_k y_k^\top B_k^{-1}}{s_k^\top y_k} \quad (6.5)$$

Consistency of BFGS Update Rules for B_k and B_k^{-1}

The BFGS method updates the approximation B_k of the Hessian matrix and its inverse B_k^{-1} iteratively. The update rules are given by:

Update for B_k :

$$B_{k+1} = B_k + \frac{y_k y_k^\top}{y_k^\top s_k} - \frac{B_k s_k s_k^\top B_k}{s_k^\top B_k s_k}.$$

Update for B_k^{-1} :

$$B_{k+1}^{-1} = \left(I - \frac{s_k y_k^\top}{y_k^\top s_k} \right) B_k^{-1} \left(I - \frac{y_k s_k^\top}{y_k^\top s_k} \right) + \frac{s_k s_k^\top}{y_k^\top s_k}.$$

We will prove that the updates for B_k and B_k^{-1} are consistent, i.e., $B_{k+1} B_{k+1}^{-1} = I$.

Proof

Let $A = B_k$, $u = \frac{y_k}{\sqrt{y_k^\top s_k}}$, and $v = \frac{B_k s_k}{\sqrt{s_k^\top B_k s_k}}$. The update to B_{k+1} can be expressed as a rank-2 update:

$$B_{k+1} = B_k + uu^\top - vv^\top,$$

where:

$$u = \frac{y_k}{\sqrt{y_k^\top s_k}}, \quad v = \frac{B_k s_k}{\sqrt{s_k^\top B_k s_k}}.$$

The Sherman-Morrison-Woodbury formula for the inverse of a rank-2 update is:

$$(B_k + uu^\top - vv^\top)^{-1} = B_k^{-1} - \frac{B_k^{-1} u u^\top B_k^{-1}}{1 + u^\top B_k^{-1} u} + \frac{B_k^{-1} v v^\top B_k^{-1}}{1 - v^\top B_k^{-1} v}.$$

We now verify that this matches the given update for B_{k+1}^{-1} .

Step 1: Verify the Form of B_{k+1}

The update rule for B_{k+1} ensures the secant equation:

$$B_{k+1} s_k = y_k.$$

Substituting the update rule:

$$\left(B_k + \frac{y_k y_k^\top}{y_k^\top s_k} - \frac{B_k s_k s_k^\top B_k}{s_k^\top B_k s_k} \right) s_k = y_k.$$

Expanding:

$$B_k s_k + \frac{y_k (y_k^\top s_k)}{y_k^\top s_k} - \frac{B_k s_k (s_k^\top B_k s_k)}{s_k^\top B_k s_k} = y_k,$$

which simplifies to:

$$B_k s_k + y_k - B_k s_k = y_k,$$

confirming consistency.

Conclusion

The updates for B_k and B_k^{-1} are consistent, as verified by the Sherman-Morrison-Woodbury formula and the secant equation.

Rationale for the Approach

1. **Curvature Information:** By updating the approximate Hessian B_k (or its inverse B_k^{-1}) using s_k and y_k , the BFGS method incorporates second-order curvature information without requiring explicit Hessian computation.
2. **Efficiency:** The BFGS update is computationally efficient. If B_k^{-1} is stored, the update costs $\mathcal{O}(n^2)$, significantly less than $\mathcal{O}(n^3)$ for inverting the Hessian.
3. **Robustness:** The method avoids issues with ill-conditioned or singular Hessians by building approximations from gradient information, ensuring better numerical stability.
4. **Convergence:** For strongly convex quadratic functions, the exact Hessian is obtained after n updates, where n is the dimension of x . For general nonlinear functions, the method converges superlinearly under mild assumptions.

Applications

The BFGS method is widely used in applications where the objective function is smooth but computing the Hessian is infeasible. Examples include:

- Machine learning optimization problems, such as logistic regression.
- Structural optimization in engineering.
- Nonlinear least-squares problems.

6.5.1 Limited-memory BFGS (L-BFGS) Algorithm

For large-scale optimization problems, storing and updating the full Hessian approximation B_k becomes computationally infeasible due to memory constraints. The L-BFGS algorithm addresses this issue by maintaining a limited history of the updates, avoiding the need to store B_k explicitly.

Key Concepts

- **Memory Efficiency:** Instead of storing B_k , L-BFGS stores a set of the most recent m pairs $\{s_i, y_i\}$, where m is a small integer (e.g., $m = 5$ or $m = 10$).
- **Two-loop Recursion:** The search direction p_k is computed using a recursive formula that involves the stored s_i and y_i vectors.
- **Implicit Hessian Approximation:** The algorithm implicitly represents the Hessian approximation through the stored vectors without forming it explicitly.

Algorithm Outline

1. Initialization:

- Choose an initial guess x_0 .
- Set $k = 0$.
- Initialize an empty history for s_i and y_i .

2. Iteration:

(a) Compute the search direction p_k :

- Use the two-loop recursion to compute p_k based on the stored s_i and y_i .

(b) Line search:

- Find an appropriate step size α_k .
- Update the position:

$$x_{k+1} = x_k + \alpha_k p_k$$

(c) Update the history:

- Compute $s_k = x_{k+1} - x_k$ and $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$.

- If the history exceeds m pairs, discard the oldest pair.
- Add the new pair (s_k, y_k) to the history.

(d) **Increment the iteration counter:**

$$k \leftarrow k + 1$$

3. Termination:

- Stop if $\|\nabla f(x_k)\|$ is below a predefined tolerance or after a maximum number of iterations.

Advantages of L-BFGS

- **Scalability:** Suitable for problems with a large number of variables (e.g., millions).
- **Low Memory Requirement:** Requires storage proportional to $\mathcal{O}(nm)$ instead of $\mathcal{O}(n^2)$.
- **Efficiency:** Maintains good convergence properties despite the limited memory.

6.5.2 Comparison Between BFGS and L-BFGS

Aspect	BFGS	L-BFGS
Memory Usage	Stores full $n \times n$ matrix	Stores m pairs of vectors
Applicability	Small to medium-scale problems	Large-scale optimization tasks
Performance	Faster convergence per iteration	Cheaper iterations, more iterations

Table 6.1: Comparison between BFGS and L-BFGS algorithms

6.5.3 A numerical example

BFGS Algorithm for Rosenbrock Function

```

1 import numpy as np
2
3 def rosenbrock(x):
4     """Compute the Rosenbrock function."""
5     return sum(100.0 * (x[1:] - x[:-1])**2.0)**2.0 + (1 - x[:-1])**2.0
6
7 def rosenbrock_grad(x):
8     """Compute the gradient of the Rosenbrock function."""
9     grad = np.zeros_like(x)
10    grad[0] = -400 * x[0] * (x[1] - x[0]**2) - 2 * (1 - x[0])
11    for i in range(1, len(x) - 1):
12        grad[i] = 200 * (x[i] - x[i - 1]**2) - \
13                400 * x[i] * (x[i + 1] - x[i]**2) - 2 * (1 - x[i])
14    grad[-1] = 200 * (x[-1] - x[-2]**2)
15    return grad
16
17 def bfgs(x0, func, grad_func, tol=1e-6, max_iter=1000):
18     """
19     Minimize a function using the BFGS algorithm.
20
21     Parameters:
22     x0 (array-like): Initial guess.
23     func (callable): Objective function.
24     grad_func (callable): Gradient of the objective function.
25     tol (float): Tolerance for the norm of the gradient.
26     max_iter (int): Maximum number of iterations.
27

```

```

28 Returns:
29     x (array-like): Estimated position of the minimum.
30     f_val (float): Function value at the minimum.
31     """
32     x = x0.copy()
33     n = len(x)
34     I = np.eye(n)
35     H = I # Initial inverse Hessian approximation
36     g = grad_func(x)
37     for k in range(max_iter):
38         if np.linalg.norm(g) < tol:
39             print(f"Converged in {k} iterations.")
40             break
41         p = -H.dot(g)
42         # Line search parameters
43         alpha = 1
44         c = 1e-4
45         rho = 0.9
46         # Backtracking line search
47         while func(x + alpha * p) > func(x) + c * alpha * g.dot(p):
48             alpha *= rho
49         x_new = x + alpha * p
50         g_new = grad_func(x_new)
51         s = x_new - x
52         y = g_new - g
53         ys = y.dot(s)
54         if ys > 1e-10: # Avoid division by zero
55             rho_k = 1.0 / ys
56             I = np.eye(n)
57             H = (I - rho_k * np.outer(s, y)).dot(H).dot(I - rho_k * np.outer(y, s)) + \
58                 rho_k * np.outer(s, s)
59         else:
60             H = I # Reset if ys is too small
61         x = x_new
62         g = g_new
63     else:
64         print(f"Maximum iterations ({max_iter}) reached.")
65     f_val = func(x)
66     return x, f_val
67
68 # Initial guess
69 x0 = np.array([-1.2, 1.0])
70
71 # Run the BFGS algorithm
72 x_min, f_min = bfgs(x0, rosenbrock, rosenbrock_grad)

```

The code prints this result:

- Optimized parameters: [1. 1.]
- Function value at minimum: 1.2069245712078662e-17.

By inspection we confirm the validity of these results.

6.5.4 Conclusion

The BFGS algorithm is a powerful method for unconstrained optimization, providing rapid convergence by approximating the Hessian matrix. Its limited-memory variant, L-BFGS, extends its applicability to large-scale problems by economizing on memory usage while retaining the essential features that make BFGS effective. Understanding both algorithms equips us with versatile tools for tackling a wide range of optimization challenges.

Further Reading

For more details of BFGS and L-BFGS the reader may consult [15, 4].

6.6 Backtracking in Gradient Descent

Backtracking is an adaptive step size strategy used in gradient descent to ensure sufficient decrease in the function value while avoiding excessively large steps. It involves:

- Starting with an initial step size
- Iteratively shrinking the step size until certain conditions are met
- Commonly used conditions include Armijo, Wolfe, and Goldstein conditions

Backtracking ensures the robustness of gradient descent, especially in complex landscapes.

6.6.1 The Armijo Rule

The Armijo rule ensures that each step of the gradient descent sufficiently decreases the function value. The condition is:

$$f(x_k + \alpha d_k) \leq f(x_k) + c_1 \alpha \nabla f(x_k)^\top d_k$$

Where:

- c_1 is a small positive constant
- d_k is the descent direction
- α is the step size

It's typically the first condition checked in more comprehensive strategies like Wolfe conditions.

6.6.2 Wolfe Conditions

Wolfe Conditions combine the Armijo rule with a curvature condition to ensure the step size is neither too small nor too large. They consist of:

1. Sufficient Decrease (Armijo): Same as the Armijo rule
2. Curvature Condition:

$$\nabla f(x_k + \alpha d_k)^\top d_k \geq c_2 \nabla f(x_k)^\top d_k$$

Where $0 < c_1 < c_2 < 1$. These conditions help in maintaining a balanced step size throughout the optimization process.

6.6.3 Strong Wolfe Conditions

The Strong Wolfe Conditions are a stricter version of the Wolfe conditions, adding a tighter bound on the curvature condition. They are useful for more precise control of the step size, especially in highly non-linear functions. The conditions are:

1. Sufficient Decrease: Same as the Armijo rule
2. Curvature Condition:

$$|\nabla f(x_k + \alpha d_k)^\top d_k| \leq c_2 |\nabla f(x_k)^\top d_k|$$

This ensures that the step size is appropriately adjusted in steep areas of the function landscape.

6.6.4 Goldstein Conditions

The Goldstein Conditions ensure that the function value after taking a step is within a certain range, preventing too small or too large updates. The conditions are:

$$f(x_k) + c_1 \alpha \nabla f(x_k)^\top d_k \leq f(x_k + \alpha d_k) \leq f(x_k) + c_2 \alpha \nabla f(x_k)^\top d_k$$

These conditions provide a balance between convergence speed and stability.

6.7 Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent (SGD) is a variant of the gradient descent algorithm widely used for optimizing functions that can be expressed as a sum of component functions. It is particularly effective in large-scale optimization problems, such as those arising in machine learning and data science.

6.7.1 Definition

Given an objective function $f(x)$ that can be written as:

$$f(x) = \frac{1}{N} \sum_{i=1}^N f_i(x),$$

where $f_i(x)$ corresponds to the contribution of the i -th data point, the update rule for standard gradient descent is:

$$x_{k+1} = x_k - \alpha \nabla f(x_k),$$

where $\alpha > 0$ is the learning rate.

In SGD, instead of computing the full gradient $\nabla f(x_k) = \frac{1}{N} \sum_{i=1}^N \nabla f_i(x_k)$, we use an estimate based on a randomly selected component $f_{i_k}(x)$:

$$x_{k+1} = x_k - \alpha \nabla f_{i_k}(x_k),$$

where i_k is chosen uniformly at random from $\{1, 2, \dots, N\}$.

6.7.2 Advantages and Challenges

Advantages:

- Computational efficiency: Each iteration requires computing the gradient of a single component, significantly reducing the computational cost for large N .
- Faster updates: Stochastic updates allow the algorithm to explore the parameter space more quickly, often leading to faster convergence in practice.

Challenges:

- Noisy updates: Stochastic gradients introduce noise, which can lead to slower convergence or oscillations near the optimum.
- Tuning: The choice of learning rate α and batch size (if mini-batch SGD is used) significantly affects performance.

6.7.3 Applications

SGD is a cornerstone of modern machine learning algorithms, particularly for training large-scale models like neural networks. It is also commonly used in statistical estimation problems and scenarios involving streaming or online data.

6.7.4 Variants of SGD

Several enhancements to SGD have been proposed to address its challenges:

- **Mini-Batch SGD:** Uses a subset of data (batch) at each step to balance computational efficiency and gradient accuracy.
- **Momentum:** Adds a velocity term to accelerate convergence.
- **Adaptive Methods:** Techniques like AdaGrad, RMSprop, and Adam adapt the learning rate based on past gradients.

6.8 Gradient Projection Method

The Gradient Projection method is an optimization technique for solving constrained optimization problems, particularly for minimizing a function $f(x)$ over a feasible set $\mathcal{C}$, which is often defined by linear or nonlinear constraints.

Definition 3 (Feasible Set). A feasible set $\mathcal{C}$ is the subset of $\mathbb{R}^n$ defined by a set of constraints that the variables $x \in \mathbb{R}^n$ must satisfy. Formally,

$$\mathcal{C} = \{x \in \mathbb{R}^n : g_i(x) \leq 0, i = 1, \dots, m, h_j(x) = 0, j = 1, \dots, p\},$$

where:

- $g_i(x) \leq 0$ are the inequality constraints,
- $h_j(x) = 0$ are the equality constraints.

The feasible set represents all points where the constraints of the optimization problem are satisfied. $\square$

The key idea is to project the gradient descent step back onto the feasible region $\mathcal{C}$ when the unconstrained step leads outside the feasible set. This ensures that the iterates remain within the feasible region throughout the optimization process.

6.8.1 Algorithm

The method is presented as Algorithm 6.4. The algorithm uses the projection operator defined as:

$$\text{Proj}_{\mathcal{C}}(y) = \arg \min_{x \in \mathcal{C}} \|x - y\|_2^2.$$

Algorithm 6.4 Projection Method

Require: A function $f(x)$ and a feasible set $\mathcal{C}$.

Ensure: An approximation $x_k \in \mathcal{C}$ to $\arg \min_{x \in \mathcal{C}} f(x)$.

- 1: Initialize $x_0 \in \mathcal{C}$.
- 2: **while** Convergence criteria not met **do**
- 3: Choose the step size $\alpha_k > 0$ for Gradient Descent.
- 4: Perform the Gradient Descent step:

$$y_{k+1} = x_k - \alpha_k \nabla f(x_k)$$

- 5: Project the point y_{k+1} onto the feasible set $\mathcal{C}$:

$$x_{k+1} = \text{Proj}_{\mathcal{C}}(y_{k+1})$$

- 6: **end while**

- 7: **return** x_k
-

6.8.2 Example Problem

Exercise 6.1 (Gradient Projection on Simple Constraints). Minimize $f(x, y) = (x-1)^2 + (y-2)^2$ subject to $x + y \leq 2$, with $x, y \geq 0$.

1. Write down the unconstrained gradient descent step.
2. Derive the projection step for the feasible region $\mathcal{C} = \{(x, y) : x + y \leq 2, x \geq 0, y \geq 0\}$.
3. Perform two iterations of the gradient projection method starting from $(x_0, y_0) = (1, 1)$ with step size $\alpha = 0.1$.

□

Solution

★★★ **Step 1:** Gradient Descent Step ★★★ The gradient is:

$$\nabla f(x, y) = \begin{bmatrix} 2(x-1) \\ 2(y-2) \end{bmatrix}.$$

The gradient descent step is:

$$y_{k+1} = x_k - \alpha \nabla f(x_k).$$

★★★ **Step 2:** Projection Step ★★★ For the feasible set:

$$\mathcal{C} = \{(x, y) : x + y \leq 2, x \geq 0, y \geq 0\},$$

the projection ensures $x, y \geq 0$ and $x + y \leq 2$. The projection is computed as:

$$x^* = \max(0, \min(y_x, 2 - y_y)), \quad y^* = \max(0, \min(y_y, 2 - x^*)).$$

★★★ **Step 3:** Iterations ★★★ Start from $(x_0, y_0) = (1, 1)$, $\alpha = 0.1$:

• **Iteration 1:**

$$y_1 = \begin{bmatrix} 1 \\ 1.2 \end{bmatrix} \quad (\text{feasible; no projection needed}), \quad x_1 = (1, 1.2).$$

• **Iteration 2:** Gradient step:

$$y_2 = \begin{bmatrix} 1 \\ 1.36 \end{bmatrix}, \quad \text{Proj}_{\mathcal{C}}(y_2) = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

New iterate: $x_2 = (1, 1)$.

□

6.8.3 Illustrative Exercise

Exercise 6.2 (Gradient Projection on the Unit Disk). Minimize $f(x, y) = x^2 + y^2 - 2x - 3y$ subject to the constraint $x^2 + y^2 \leq 1$.

1. Compute the gradient of $f(x, y)$.
2. Implement the projection step onto the unit disk $x^2 + y^2 \leq 1$.
3. Perform three iterations starting from $(x_0, y_0) = (1, 1)$ with step size $\alpha = 0.1$.

□

Solution

★★★ Step 1: Gradient Descent Step ★★★

$$\nabla f(x, y) = \begin{bmatrix} 2x - 2 \\ 2y - 3 \end{bmatrix}, \quad y_{k+1} = x_k - \alpha \nabla f(x_k).$$

★★★ Step 2: Projection Step ★★★

The feasible set is $\mathcal{C} = \{(x, y) : x^2 + y^2 \leq 1\}$. Project onto $\mathcal{C}$:

$$\text{Proj}_{\mathcal{C}}(y_{k+1}) = \begin{cases} y_{k+1}, & \text{if } \|y_{k+1}\|_2 \leq 1, \\ \frac{y_{k+1}}{\|y_{k+1}\|_2}, & \text{otherwise.} \end{cases}$$

★★★ Step 3: Iterations ★★★

Start from $(x_0, y_0) = (1, 1)$, $\alpha = 0.1$:

- **Iteration 1:** Gradient step:

$$y_1 = \begin{bmatrix} 1 \\ 1.1 \end{bmatrix}, \quad \text{Proj}_{\mathcal{C}}(y_1) = \frac{\begin{bmatrix} 1 \\ 1.1 \end{bmatrix}}{\sqrt{2.21}} = \begin{bmatrix} 0.674 \\ 0.742 \end{bmatrix}.$$

- **Iteration 2:** Gradient step:

$$y_2 = \begin{bmatrix} 0.739 \\ 0.893 \end{bmatrix}, \quad \text{Proj}_{\mathcal{C}}(y_2) = \begin{bmatrix} 0.635 \\ 0.770 \end{bmatrix}.$$

- **Iteration 3:** Continue projection and gradient steps similarly.

6.9 Computed Tomography

The word "tomography" comes from the Greek word *tomos*, meaning "slice" or "section." Computed Tomography (CT) is a noninvasive medical procedure that uses X-rays to create cross-sectional images of the body. CT scans are commonly used for diagnosis, treatment planning, and screening. During a CT scan, the patient lies on a table that moves through a circular opening in the CT machine. An X-ray source rotates around the opening while detectors record the X-rays exiting the patient's body.

Applications of CT are not limited to medicine. Earth tomography is a technique that uses tomography to map regions of fast and slow seismic waves within the Earth's interior.

At the core of CT is a family of mathematical techniques and computer algorithms that allow the construction of a 3D model from 1-dimensional slices formed by the paths of X-rays through the patient's body. The only available information is the attenuation of X-rays as they pass through different tissues.

6.9.1 Problem Formulation

The mathematical problem of tomographic image reconstruction involves recovering an unknown image $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ from its line integrals, represented by the Radon transform Rf . The Radon transform is defined as:

$$Rf(\theta, s) = \int_{L(\theta, s)} f(x, y) ds,$$

where $L(\theta, s)$ is a line parameterized by angle θ and distance s from the origin. The measured projection data $p(\theta, s)$ obtained from an X-ray scanner can be modeled as:

$$Rf(\theta, s) = \int_{L(\theta, s)} f(x, y) ds + \varepsilon,$$

where ε denotes measurement noise. Discretizing this transform leads to a linear system of equations:

$$Af = p + \varepsilon,$$

where $A \in \mathbb{R}^{m \times n}$ is the discretized forward projection operator, $f \in \mathbb{R}^n$ is the vectorized image, and $p \in \mathbb{R}^m$ is the measured data.

Due to the ill-posed nature of the problem (sensitivity to noise and incomplete data), regularization techniques are often employed to stabilize the inversion process.

6.9.2 Tikhonov Regularization

Tikhonov regularization addresses the ill-posedness by adding a penalty term to the least squares objective, ensuring a balance between data fidelity and smoothness:

$$\min_f (\|Af - p\|_2^2 + \lambda \|f\|_2^2),$$

where $\lambda > 0$ is the regularization parameter controlling the trade-off between data fidelity and regularization. The solution can be computed analytically using:

$$\hat{f} = (A^\top A + \lambda I)^{-1} A^\top p.$$

6.9.3 Total Variation (TV) Regularization

Total Variation regularization is particularly effective for piecewise constant images, as it promotes sparsity in the gradient of the image. The objective function is:

$$\min_f (\|Af - p\|_2^2 + \lambda \|\nabla f\|_1),$$

where $\|\nabla f\|_1$ is the ℓ^1 norm of the image gradient, encouraging sparsity and reducing noise while preserving edges. The TV-regularized problem is typically solved using iterative methods due to its non-differentiability.

Both Tikhonov and TV regularization provide effective tools for stabilizing the tomographic reconstruction problem, with Tikhonov favoring smooth solutions and TV being particularly effective for preserving sharp edges.

6.9.4 Optimization Methods: BFGS and L-BFGS

For solving the regularized reconstruction problems described above, gradient-based optimization algorithms such as BFGS (Broyden-Fletcher-Goldfarb-Shanno) and its limited-memory variant L-BFGS can be used effectively.

- **Tikhonov Regularization:** BFGS and L-BFGS are well-suited due to the smooth and differentiable quadratic nature of the objective.
- **TV Regularization:** Since TV involves a non-differentiable ℓ^1 term, a proximal variant or gradient-based methods adapted for non-smooth objectives, such as Split Bregman or Chambolle-Pock algorithms, are recommended.

These algorithms provide efficient and reliable solutions for tomographic reconstruction, especially for large-scale data scenarios where computational efficiency is critical.

6.9.5 Datasets and References

The following datasets and references are commonly used for benchmarking and theoretical development in tomography and regularization research:

- LoDoPaB-CT Dataset [9]
- DeepLesion Dataset [29]
- AAPM Low-Dose CT Grand Challenge [17]
- Tikhonov Regularization Theory [27]
- Numerical Optimization Techniques [15]

6.10 Exercises

Exercise 6.3 (Basic Gradient Descent). Minimize $f(x, y) = 3x^2 + 4xy + y^2 - 6x - 8y$ using gradient descent with a constant step size of $\alpha = 0.1$. Start from $(x_0, y_0) = (1, 1)$. Perform three iterations and analyze the convergence behavior. $\square$

Exercise 6.4 (Step Size Selection). Given $f(x) = \frac{1}{2}x^\top Ax - b^\top x$, where

$$A = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 2 \end{bmatrix},$$

compute the best step size for gradient descent. $\square$

Exercise 6.5 (Convergence Analysis). Prove that gradient descent converges linearly for a strongly convex function $f(x)$ with Lipschitz-continuous gradient, and derive the convergence rate. $\square$

Exercise 6.6 (BFGS Update). Derive the BFGS update formula for the inverse Hessian approximation given a function $f(x)$ with $s_k = x_{k+1} - x_k$ and $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$. $\square$

Exercise 6.7 (Implementation Task). Write a Python or MATLAB implementation of the BFGS method for $f(x, y) = (x - 1)^4 + 2(y - 2)^2 + xy$. Test your implementation starting from $(x_0, y_0) = (0, 0)$. $\square$

Exercise 6.8 (L-BFGS Memory Usage). Explain the memory efficiency of the L-BFGS algorithm and show how storing m pairs of vectors (s_i, y_i) avoids explicit computation or storage of the Hessian matrix. $\square$

Exercise 6.9 (L-BFGS Scaling). Apply L-BFGS to minimize $f(x) = \sum_{i=1}^n (x_i - i)^2$ for $n = 10^5$. Discuss the scalability and performance compared to gradient descent. $\square$

Exercise 6.10 (Subspace Problem). Minimize $f(x) = \frac{1}{2}x^\top Ax - b^\top x$ subject to $Cx = d$, where

$$A = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 1 \end{bmatrix}, \quad d = 1.$$

Solve using the method of Lagrange multipliers. $\square$

Exercise 6.11 (Exact Solution). Show that minimizing a positive definite quadratic function $f(x)$ over a linear subspace $\mathcal{L} = \{x : Ax = b\}$ can be reduced to solving a linear system. $\square$

Exercise 6.12 (Equality Constraint). Minimize $f(x, y) = x^2 + y^2$ subject to $g(x, y) = x + 2y - 1 = 0$. Use the method of Lagrange multipliers and verify the results by directly solving the constrained problem. $\square$

Exercise 6.13 (Multiple Constraints). Minimize $f(x, y, z) = x^2 + y^2 + z^2$ subject to $x + y + z = 1$ and $x - y = 0$. Use Lagrange multipliers and compute the stationary points. $\square$

Exercise 6.14 (Economics Application). Consider the utility maximization problem $U(x, y) = x^{0.5}y^{0.5}$ subject to $px + qy = M$, where $p, q, M > 0$. Use Lagrange multipliers to derive the optimal values of x and y . $\square$

Exercise 6.15 (Second-Order Condition). Let $f(x, y, z) = x^2 + y^2 - z^2$, and $g(x, y, z) = x + y + z - 1 = 0$. Check whether the point $(1/3, 1/3, 1/3)$ is a local maximum, minimum, or saddle point using the bordered Hessian. $\square$

Exercise 6.16 (Constrained Optimization). Minimize $f(x, y) = x^3 - 3x + y^2$ subject to $g(x, y) = x^2 + y^2 - 1 = 0$. Use the bordered Hessian to classify the critical points. $\square$

Exercise 6.17 (Gradient Descent on a Manifold). Apply gradient descent to $f(x, y) = x^2 + y^2$ constrained to the unit circle $x^2 + y^2 = 1$. Show that the gradient projection method works for this problem. $\square$

Exercise 6.18 (Stochastic Gradient Descent). Compare the convergence rates of gradient descent and stochastic gradient descent for $f(x) = \frac{1}{n} \sum_{i=1}^n (x - i)^2$, with $n = 1000$. $\square$

Exercise 6.19 (Hessian Approximation in BFGS). Given the quadratic function $f(x) = \frac{1}{2}x^\top Ax - b^\top x$, where A is positive definite, prove that the exact Hessian is obtained after n iterations of the BFGS algorithm. $\square$

Exercise 6.20 (Ill-Conditioned Optimization). Minimize $f(x, y) = 10x^2 + y^2$ using gradient descent and BFGS. Discuss why BFGS is more effective for ill-conditioned problems. $\square$

Exercise 6.21 (Armijo Condition: Sufficient Decrease). Given the function $f(x) = x^2$, starting at $x_0 = 1$ with descent direction $p = -1$, determine whether a step size $\alpha = 0.5$ satisfies the Armijo condition:

$$f(x_0 + \alpha p) \leq f(x_0) + c_1 \alpha \nabla f(x_0)^\top p,$$

where $c_1 = 0.1$. $\square$

Exercise 6.22 (Goldstein Condition: Balanced Search). For the function $f(x) = x^3 - 3x^2 + 4$, starting at $x_0 = 0.5$ with descent direction $p = -1$, check whether $\alpha = 0.25$ satisfies the Goldstein condition:

$$f(x_0) + (1 - c_1) \alpha \nabla f(x_0)^\top p \leq f(x_0 + \alpha p) \leq f(x_0) + c_1 \alpha \nabla f(x_0)^\top p,$$

where $c_1 = 0.25$. $\square$

Exercise 6.23 (Wolfe Conditions: Sufficient Decrease and Curvature). Let $f(x) = \ln(x + 2)$, starting at $x_0 = 1$ with descent direction $p = -1$. Verify whether the step size $\alpha = 0.1$ satisfies both Wolfe conditions:

1. $f(x_0 + \alpha p) \leq f(x_0) + c_1 \alpha \nabla f(x_0)^\top p$,
2. $\nabla f(x_0 + \alpha p)^\top p \geq c_2 \nabla f(x_0)^\top p$,

where $c_1 = 0.1$ and $c_2 = 0.7$. $\square$

Exercise 6.24 (Strong Wolfe Conditions). Consider the function $f(x) = \sin(x)$, starting at $x_0 = 1.5$ with descent direction $p = -1$. Check whether $\alpha = 0.2$ satisfies the Strong Wolfe conditions:

1. $f(x_0 + \alpha p) \leq f(x_0) + c_1 \alpha \nabla f(x_0)^\top p$,
2. $|\nabla f(x_0 + \alpha p)^\top p| \leq c_2 |\nabla f(x_0)^\top p|$,

where $c_1 = 0.2$ and $c_2 = 0.8$. $\square$

Exercise 6.25 (Comparison of Conditions). For the quadratic function $f(x) = x^2$, start at $x_0 = 1$ with $p = -1$. Calculate the range of acceptable step sizes α for:

1. Armijo condition with $c_1 = 0.1$,
2. Goldstein condition with $c_1 = 0.2$,
3. Wolfe conditions with $c_1 = 0.1$ and $c_2 = 0.9$,
4. Strong Wolfe conditions with $c_1 = 0.1$ and $c_2 = 0.9$.

Compare the results and interpret the differences in the range of α . $\square$

Exercise 6.26 (Practical Application: Armijo vs Wolfe). Minimize $f(x) = e^x - x$ using gradient descent with backtracking line search. Experiment with both the Armijo condition and the Wolfe conditions to determine step sizes. Compare the number of iterations required to converge. $\square$

Exercise 6.27 (Geometric Interpretation). Provide a geometric interpretation of the Armijo, Wolfe, and Goldstein conditions for the quadratic function $f(x) = x^2 + 2x + 3$ starting at $x_0 = 0$ with $p = -1$. Plot the function, the tangent line, and the step sizes that satisfy each condition. $\square$

Exercise 6.28 (Strong Wolfe Condition: Robustness). Consider the function $f(x) = \cos(x)$, starting at $x_0 = 1$ with $p = -1$. Test the robustness of the Strong Wolfe condition by varying c_1 and c_2 in the ranges $c_1 \in [0.1, 0.4]$ and $c_2 \in [0.5, 0.9]$. Discuss the impact on the range of acceptable α . $\square$

Exercise 6.29 (Implementation Task). Implement a line search algorithm in Python or MATLAB that supports the Armijo, Wolfe, and Strong Wolfe conditions. Test your implementation on $f(x) = \ln(1 + x^2)$ with initial guess $x_0 = 2$. Compare the step sizes and convergence behavior for each condition. $\square$

Exercise 6.30 (Convergence Analysis). Prove that the Wolfe conditions (sufficient decrease and curvature) guarantee convergence of gradient descent for convex functions with Lipschitz-continuous gradients. Contrast this with the weaker guarantees provided by the Armijo condition alone. $\square$

Chapter 7

Interpolation and Polynomial Approximation

7.1 Introduction to Interpolation

Interpolation is a method of constructing new data points within the range of a discrete set of known data points. Polynomial approximation involves approximating a function using polynomials, which are easier to evaluate and differentiate.

7.2 Lagrange Interpolation

Lagrange Interpolation is a polynomial interpolation method that uses Lagrange basis polynomials to construct the interpolating polynomial.

7.2.1 Lagrange Polynomial

Given a set of data points $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$, the Lagrange interpolating polynomial $P(x)$ of degree n is given by:

$$P_n(x) = \sum_{i=0}^n y_i L_i(x)$$

where $L_i(x)$ are the Lagrange basis polynomials defined as:

$$L_i(x) = \prod_{\substack{0 \leq j \leq n \\ j \neq i}} \frac{x - x_j}{x_i - x_j}$$

7.2.2 Algorithm

Please refer to Algorithm 7.1.

7.2.3 Error Analysis

The error in Lagrange interpolation is given by:

$$E(x) = f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i)$$

for some ξ in the interval of interpolation.

Algorithm 7.1 Lagrange Interpolation

Require: A set of data points $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$

Ensure: The interpolating polynomial $P(x)$

- 1: Initialize $P(x) = 0$
 - 2: **for** each i from 0 to n **do**
 - 3: Compute the Lagrange basis polynomial $L_i(x)$
 - 4: Update $P(x) = P(x) + y_i L_i(x)$
 - 5: **end for**
 - 6: **return** $P(x)$
-

7.3 Newton Interpolation

Newton Interpolation is another polynomial interpolation method that uses divided differences to construct the interpolating polynomial.

7.3.1 Newton's Divided Differences

The divided differences are computed recursively as:

$$\begin{aligned} f[x_i] &= y_i \\ f[x_i, x_{i+1}] &= \frac{f[x_{i+1}] - f[x_i]}{x_{i+1} - x_i} \\ f[x_i, x_{i+1}, \dots, x_{i+k}] &= \frac{f[x_{i+1}, \dots, x_{i+k}] - f[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i} \end{aligned}$$

7.3.2 Newton Interpolating Polynomial

The Newton interpolating polynomial of degree n is given by:

$$P_n(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \dots$$

7.3.3 Algorithm

Please refer to Algorithm 7.2

Algorithm 7.2 Newton Interpolation

Require: A set of data points $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$

Ensure: The interpolating polynomial $P(x)$

- 1: Compute the divided differences table (Algorithm 7.3)
 - 2: Initialize $P(x) = f[x_0]$
 - 3: **for** each i from 1 to n **do**
 - 4: Update $P(x) = P(x) + f[x_0, x_1, \dots, x_i] \prod_{j=0}^{i-1} (x - x_j)$
 - 5: **end for**
 - 6: **return** $P(x)$
-

7.3.4 Lagrange Remainder Formula

The difference between the function and its interpolating polynomial of degree n satisfies the following identity:

$$f(x) - P_n(x) = f[x_0, x_1, \dots, x_n, x] \prod_{i=0}^n (x - x_i).$$

7.3.5 Error Analysis

The error in Newton interpolation is given by:

$$E(x) = f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i)$$

for some ξ in the interval of interpolation.

7.3.6 Chebyshev Nodes

These are special nodes $x_i \in [-1, 1]$, chosen to minimize

$$\max_{x \in [-1, 1]} \left| \prod_{i=0}^n (x - x_i) \right|.$$

They are given by

$$x_i = \cos\left(\frac{2i+1}{2n+2}\pi\right), \quad i = 0, 1, \dots, n.$$

These nodes are the roots of the *Chebyshev polynomials of the first kind*, $T_{n+1}(x)$. Typically, $T_n(x)$ is defined as:

$$T_n(x) = \cos(n\theta), \quad x = \cos \theta.$$

With this choice of interpolation nodes, the interpolation error is minimized, and the following error bound holds:

$$|f(x) - P_n(x)| \leq \frac{\max_{\xi \in [-1, 1]} |f^{(n+1)}(\xi)|}{(n+1)!} \prod_{i=0}^n |x - x_i|,$$

where $P_n(x)$ is the interpolation polynomial of degree n , and x_i are the Chebyshev nodes. Also, we have a bound

$$\max_{x \in [-1, 1]} \prod_{i=0}^n |x - x_i| = \frac{1}{2^n}.$$

It can be shown quite easily that $T_{n+1}(x) \equiv 2^n \prod_{i=0}^n (x - x_i)$, which implies the above, as $|T_n(x)| \leq 1$ if $x = \cos \theta$ for some θ , i.e., for $x \in [-1, 1]$. These considerations lead to a more explicit, uniform bound:

$$|f(x) - P_n(x)| \leq \frac{\max_{\xi \in [-1, 1]} |f^{(n+1)}(\xi)|}{2^n (n+1)!}, \quad \forall x \in [-1, 1].$$

By scaling to an arbitrary interval $[a, b]$:

$$x_i = \frac{a+b}{2} + \frac{b-a}{2} \cos\left(\frac{2i+1}{2n+2}\pi\right), \quad i = 0, 1, \dots, n,$$

$$|f(x) - P_n(x)| \leq \frac{\max_{\xi \in [-1, 1]} |f^{(n+1)}(\xi)|}{2^n (n+1)!} \left(\frac{b-a}{2}\right)^{n+1}, \quad \forall x \in [a, b].$$

7.4 Algorithm to Construct the Divided Difference Table

The divided difference table is a computational tool used in interpolation, particularly for constructing Newton's divided difference interpolation polynomial. The algorithm to construct the table is described below:

Algorithm 7.3 Construction of the divided difference table

Require: A set of $n + 1$ data points: $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$.

Ensure: The table $T_{i,k}$, $0 \leq i, k \leq n$, containing all divided differences, so that

$$T_{i,k} = f[x_i, x_{i+1}, \dots, x_{i+k}]$$

The first row contains the coefficients of the Newton interpolating polynomial.

- 1: Create a 0-based table $T_{i,k}$ with $n + 1$ rows and $n + 1$ columns.
- 2: **for** i from 0 to n **do**
- 3: $T_{i,k} \leftarrow y_k$ {Populate the first column of the table with $y_0, y_1, \dots, y_n$ }
- 4: **end for**
- 5: **for** k from 1 to n **do**
- 6: **for** i from 0 to $n - k$ **do**
- 7: Compute the divided difference as:

$$T_{i,k} = \frac{T_{i+1,k-1} - T_{i,k-1}}{x_{i+k} - x_i}.$$

- 8: **end for**
 - 9: **end for**
-

Example Layout of the Table

x_i	$f[x_i]$	$f[x_i, x_{i+1}]$	$f[x_i, x_{i+1}, x_{i+2}]$	...
x_0	y_0			
x_1	y_1			
x_2	y_2			

Example of Divided Difference Table Calculation

Suppose we are given the following data points: $(x_0, y_0) = (1, 1), (x_1, y_1) = (2, 4), (x_2, y_2) = (3, 9), (x_3, y_3) = (4, 16)$. These values are consistent with $f(x) = x^2$, but they could have been a result of an experiment. Then the divided difference table is:

x_i	$f[x_i]$	$f[x_i, x_{i+1}]$	$f[x_i, x_{i+1}, x_{i+2}]$	$f[x_i, x_{i+1}, x_{i+2}, x_{i+3}]$
1	1	$\frac{4-1}{2-1} = 3$	$\frac{5-3}{3-1} = 1$	$\frac{1-1}{4-1} = 0$
2	4	$\frac{9-4}{3-2} = 5$	$\frac{7-5}{4-2} = 1$	
3	9	$\frac{16-9}{4-3} = 7$		
4	16			

The first row entries 1, 3, 1, 0 provide the coefficients of the Newton interpolating polynomial.

Newton's Interpolating Polynomial

The coefficients of the polynomial can be obtained from the first row entries of the table. The Newton interpolating polynomial is constructed as:

$$P(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \dots$$

Finding $f(2.5)$

Using the polynomial coefficients 1, 3, 1, 0:

$$\begin{aligned}P_3(x) &= 1 + 3(x - 1) + 1(x - 1)(x - 2) + 0(x - 1)(x - 2)(x - 3), \\P_3(2.5) &= 1 + 3(2.5 - 1) + 1(2.5 - 1)(2.5 - 2) + 0(2.5 - 1)(2.5 - 2)(2.5 - 3) \\&= 1 + 3(1.5) + 1(1.5)(0.5) + 0(1.5)(0.5)(-0.5) \\&= 1 + 4.5 + 0.75 + 0 \\&= 6.25\end{aligned}$$

Thus, $P_3(2.5) = 6.25$.

We note that if $f(x) = x^2$ then $f(2.5) = 6.25$. This sheds some light on the accuracy of this 4-point approximation. The interpolating polynomial actually coincides with the function. In fact, $f(x) = P_2(x) = P_3(x)$.

7.5 Spline Interpolation

Spline interpolation uses piecewise polynomials, called splines, to interpolate data points. The most commonly used spline is the cubic spline.

7.5.1 An Overview of Cubic Splines

A cubic spline is a piecewise polynomial of degree three that is smooth and continuous at the data points. The cubic spline interpolant $S(x)$ for a set of data points $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$ is defined such that:

- $S(x_i) = y_i$ for $i = 0, 1, \dots, n$
- $S(x)$ is twice continuously differentiable

7.5.2 Algorithm

Algorithm 7.4 Cubic Spline Interpolation

Require: A set of data points $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$

Ensure: The cubic spline interpolant $S(x)$

- 1: Compute the coefficients of the cubic splines by solving the tridiagonal system
 - 2: Construct the piecewise cubic polynomial $S(x)$
 - 3: **return** $S(x)$
-

7.5.3 Error Analysis

The error in cubic spline interpolation is given by:

$$E(x) = f(x) - S(x) \approx \frac{h^4}{384} \max |f^{(4)}(\xi)|$$

where h is the maximum spacing between data points.

7.5.4 Proof of the Constant 384

To understand the origin of the constant 384 in the error analysis for cubic spline interpolation, consider the following detailed proof:

Given a function $f(x)$ and its cubic spline interpolant $S(x)$ over the interval $[x_i, x_{i+1}]$, the error at a point x in this interval can be expressed as:

$$E(x) = f(x) - S(x)$$

The error term for a cubic spline interpolation is influenced by the fourth derivative of the function $f(x)$. Specifically, the error can be derived using the fourth derivative as follows:

$$E(x) = \frac{f^{(4)}(\xi)}{4!} (x - x_i)^2 (x - x_{i+1})^2$$

where ξ is some point in the interval $[x_i, x_{i+1}]$. Here, the term $(x - x_i)^2 (x - x_{i+1})^2$ arises because cubic splines are third-degree polynomials, and the error is related to the fourth derivative of the function being interpolated.

For the maximum error over the entire interpolation range, consider the maximum value of $(x - x_i)^2 (x - x_{i+1})^2$ within the interval $[x_i, x_{i+1}]$:

$$\max_{x \in [x_i, x_{i+1}]} (x - x_i)^2 (x - x_{i+1})^2 = \left(\frac{h}{2}\right)^2 \left(\frac{h}{2}\right)^2 = \frac{h^4}{16}$$

where $h = x_{i+1} - x_i$ is the length of the interval.

Therefore, the error over the interval can be written as:

$$E(x) \leq \frac{\max |f^{(4)}(\xi)|}{24} \cdot \frac{h^4}{16} = \frac{h^4 \max |f^{(4)}(\xi)|}{384}$$

Thus, we have:

$$E(x) \leq \frac{h^4}{384} \max |f^{(4)}(\xi)|$$

This detailed derivation shows that the constant 384 arises from the combination of the factorial term $4!$ (which is 24) and the maximum value of the product $(x - x_i)^2 (x - x_{i+1})^2$ over the interval $[x_i, x_{i+1}]$.

7.5.5 A Detailed Algorithm for Cubic Splines

Given a set of n data points

$$(x_0, y_0), (x_1, y_1), \dots, (x_{n-1}, y_{n-1}),$$

the goal of cubic spline interpolation is to find a function $S(x)$ that is a cubic polynomial on each subinterval $[x_i, x_{i+1}]$ and satisfies the following conditions:

1. **Interpolation:** $S(x_i) = y_i$ for all $i = 0, \dots, n-1$.
2. **Smoothness:** The first and second derivatives $S'(x)$ and $S''(x)$ are continuous on the entire interval $[x_0, x_{n-1}]$.
3. **Natural Boundary Conditions:** $S''(x_0) = 0$ and $S''(x_{n-1}) = 0$.

On each interval $[x_i, x_{i+1}]$, the spline is expressed as:

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3.$$

It is immediate from the interpolation condition that

$$a_i = y_i.$$

Continuity of the first and second derivatives at the interior nodes leads to a system of equations for the unknowns c_i . Once these are determined (typically by solving a tridiagonal system via the Thomas algorithm), the coefficients b_i and d_i can be computed as:

$$b_i = \frac{y_{i+1} - y_i}{h_i} - \frac{h_i}{3}(2c_i + c_{i+1}), \quad d_i = \frac{c_{i+1} - c_i}{3h_i},$$

where $h_i = x_{i+1} - x_i$.

Algorithm 7.5 Compute Cubic Spline Coefficients

Require: Arrays $x_0, x_1, \dots, x_{n-1}$ and $y_0, y_1, \dots, y_{n-1}$ with $x_i < x_{i+1}$

Ensure: Arrays $a_0, a_1, \dots, a_{n-2}, b_0, b_1, \dots, b_{n-2}, c_0, c_1, \dots, c_{n-1}, d_0, d_1, \dots, d_{n-2}$

```
1: Compute  $h_i \leftarrow x_{i+1} - x_i$  for  $i = 0, \dots, n-2$ 
2: Set  $a_i \leftarrow y_i$  for  $i = 0, \dots, n-1$ 
3: Initialize arrays  $A_i, B_i, C_i, D_i$  for  $i = 1, \dots, n-2$ 
4: for  $i = 1$  to  $n-2$  do
5:    $A_i \leftarrow h_{i-1}$ 
6:    $B_i \leftarrow 2(h_{i-1} + h_i)$ 
7:    $C_i \leftarrow h_i$ 
8:    $D_i \leftarrow 3 \left( \frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}} \right)$ 
9: end for
10: Set  $c_0 \leftarrow 0$  and  $c_{n-1} \leftarrow 0$ 
11: {Solve the tridiagonal system for  $c_1, \dots, c_{n-2}$  using the Thomas algorithm}
12: for  $i = 2$  to  $n-2$  do
13:    $w \leftarrow \frac{A_i}{B_{i-1}}$ 
14:    $B_i \leftarrow B_i - w C_{i-1}$ 
15:    $D_i \leftarrow D_i - w D_{i-1}$ 
16: end for
17:  $c_{n-2} \leftarrow \frac{D_{n-2}}{B_{n-2}}$ 
18: for  $i = n-3$  to  $1$  by  $-1$  do
19:    $c_i \leftarrow \frac{D_i - C_i c_{i+1}}{B_i}$ 
20: end for
21: for  $i = 0$  to  $n-2$  do
22:    $b_i \leftarrow \frac{y_{i+1} - y_i}{h_i} - \frac{h_i(2c_i + c_{i+1})}{3}$ 
23:    $d_i \leftarrow \frac{c_{i+1} - c_i}{3h_i}$ 
24: end for
25: return  $a_0, \dots, a_{n-2}, b_0, \dots, b_{n-2}, c_0, \dots, c_{n-1}, d_0, \dots, d_{n-2}$ 
```

7.5.6 Pseudocode

Pseudocode of Algorithm 7.5 outlines in detail the steps for computing the cubic spline coefficients.

7.5.7 Conclusion

We have presented the mathematical background and a detailed algorithm for computing cubic spline interpolation. The method guarantees a smooth curve through all data points while enforcing natural boundary conditions. This section should serve as a guide for implementing cubic spline interpolation from the ground up.

7.6 Piecewise Interpolation

Piecewise interpolation involves using different interpolation methods in different segments of the data. This approach can provide more flexibility and better accuracy in certain applications.

7.6.1 Algorithm

Algorithm 7.6 Piecewise Interpolation

Require: A set of data points $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$

Ensure: The piecewise interpolant $P(x)$

- 1: **for** each segment $[x_i, x_{i+1}]$ **do**
 - 2: Choose an appropriate interpolation method (e.g., linear, quadratic, cubic)
 - 3: Compute the interpolant for the segment
 - 4: Update $P(x)$ with the interpolant for the segment
 - 5: **end for**
 - 6: **return** $P(x)$
-

7.7 Finite Difference Formulas

Finite difference methods are used to approximate derivatives of functions using discrete data points. Common finite difference formulas include forward, backward, and central differences.

7.7.1 Forward Difference Formula

The forward difference formula approximates the derivative of a function at a point using the value of the function at that point and the next point. It is given by:

$$\Delta f(x) = f(x + h) - f(x)$$

The forward difference approximation for the first derivative is:

$$f'(x) \approx \frac{f(x + h) - f(x)}{h}$$

7.7.2 Backward Difference Formula

The backward difference formula approximates the derivative of a function at a point using the value of the function at that point and the previous point. It is given by:

$$\nabla f(x) = f(x) - f(x - h)$$

The backward difference approximation for the first derivative is:

$$f'(x) \approx \frac{f(x) - f(x-h)}{h}$$

7.7.3 Central (Symmetric) Difference Formula

The central difference formula provides a more accurate approximation by using the value of the function at points on both sides of the point of interest. It is given by:

$$\delta f(x) = f(x+h) - f(x-h)$$

The central difference approximation for the first derivative is:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

7.7.4 Newton's Forward and Backward Difference Formulas

Newton's forward and backward difference formulas are used for interpolating data points and estimating derivatives using a series of finite differences.

For a function $f(x)$ with equally spaced data points $x_0, x_1, \dots, x_n$ where $x_i = x_0 + ih$:

Newton's forward difference formula for interpolation is:

$$f(x) \approx f(x_0) + \Delta f(x_0) \frac{(x-x_0)}{h} + \frac{\Delta^2 f(x_0)}{2!} \frac{(x-x_0)(x-x_1)}{h^2} + \dots$$

An alternative, compact (and equivalent) form of this is:

$$f(x_0 + ah) = \sum_{k=0}^{\infty} \binom{a}{k} \Delta^k f(x_0)$$

That is, first we find a from the equation: $x = x_0 + ah$.

Newton's backward difference formula for interpolation is:

$$f(x) \approx f(x_n) + \nabla f(x_n) \frac{(x-x_n)}{h} + \frac{\nabla^2 f(x_n)}{2!} \frac{(x-x_n)(x-x_{n-1})}{h^2} + \dots$$

Here, Δ and ∇ denote the forward and backward difference operators, respectively.

An alternative, compact (and equivalent) form of this is:

$$f(x_n - ah) = \sum_{k=0}^{\infty} (-1)^k \frac{a(a+1) \cdots (a+n-1)}{k!} \nabla^k f(x_0) = \sum_{k=0}^{\infty} \binom{-a}{k} \nabla^k f(x_0)$$

That is, first we find a from the equation: $x = x_n - ah$.

These finite difference formulas are fundamental tools for numerical differentiation and interpolation, providing approximations for derivatives and function values at non-tabulated points.

The formulas just introduced follow easily by adapting the Newton's divided difference formulas to the case of uniformly distributed nodes $x_i = x_0 + ih$, by observing the following identities:

$$f[x_0, x_1, \dots, x_n] = \frac{\Delta^n f(x_0)}{h^n},$$

$$f[x_n, x_{n-1}, \dots, x_0] = \frac{\nabla^n f(x_0)}{h^n}.$$

It should be noted that the divided difference formalism does not assume any particular ordering of the nodes, so we can reverse the order of the nodes.

Higher-Order Forward Differences

For higher-order derivatives, forward differences can be extended as follows:

$$\begin{aligned}\Delta f(x) &= f(x+h) - f(x) \\ \Delta^2 f(x) &= \Delta(\Delta f(x)) = f(x+2h) - 2f(x+h) + f(x) \\ \Delta^3 f(x) &= \Delta(\Delta^2 f(x)) = f(x+3h) - 3f(x+2h) + 3f(x+h) - f(x)\end{aligned}$$

In general, the n -th forward difference is:

$$\Delta^n f(x) = \sum_{k=0}^n (-1)^k \binom{n}{k} f(x + (n-k)h)$$

The n -th derivative can then be approximated using the forward difference:

$$f^{(n)}(x) \approx \frac{\Delta^n f(x)}{h^n}$$

7.7.5 Central Difference Formula

The central difference formula uses points on both sides of the current point to approximate derivatives, providing higher accuracy than forward or backward differences.

First-Order Central Difference

The first-order central difference approximation for the first derivative is given by:

$$f'(x) \approx \frac{f(x+h/2) - f(x-h/2)}{h}.$$

There is an alternative form with h in place of $h/2$.

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}.$$

Higher-Order Central Differences

For higher-order derivatives, central differences can be extended as follows:

$$\begin{aligned}\delta f(x) &= f(x+h/2) - f(x-h/2) \\ \delta^2 f(x) &= \delta(\delta f(x)) = f(x+h) - 2f(x) + f(x-h) \\ \delta^3 f(x) &= \delta(\delta^2 f(x)) = f(x+(3/2)h) - 3f(x+h/2) + 3f(x-h/2) - f(x-(3/2)h)\end{aligned}$$

In general, the n -th central difference is:

$$\delta^n f(x) = \sum_{k=0}^n (-1)^k \binom{n}{k} f(x + (n/2 - k)h)$$

The n -th derivative can then be approximated using the central difference:

$$f^{(n)}(x) \approx \frac{\delta^n f(x)}{h^n}$$

7.8 Verification of the Third Derivative Approximation Using Taylor Expansion

The third derivative of a function $f(x)$ can be approximated using the finite difference formula:

$$f'''(x) \approx \frac{\delta^3 f(x)}{h^3} = \frac{f(x + 3h/2) - 3f(x + h/2) + 3f(x - h/2) - f(x - 3h/2)}{(2h)^3}.$$

To verify this approximation, we expand $f(x)$ in a Taylor series about x up to order 5:

$$\begin{aligned} f(x + kh) &= f(x) + khf'(x) + \frac{(kh)^2}{2!}f''(x) + \frac{(kh)^3}{3!}f'''(x) + \frac{(kh)^4}{4!}f^{(4)}(x) + \frac{(kh)^5}{5!}f^{(5)}(x) + O(h^6), \\ f(x - kh) &= f(x) - khf'(x) + \frac{(kh)^2}{2!}f''(x) - \frac{(kh)^3}{3!}f'''(x) + \frac{(kh)^4}{4!}f^{(4)}(x) - \frac{(kh)^5}{5!}f^{(5)}(x) + O(h^6). \end{aligned}$$

Substituting these expansions into the finite difference formula, we get:

$$\frac{f(x + 3h/2) - 3f(x + h/2) + 3f(x - h/2) - f(x - 3h/2)}{h^3} = f'''(x) + \frac{h^2}{8}f^{(5)}(x) + O(h^4).$$

This gives the correct approximation for $f'''(x)$ with an error term of order h^2 , making it a second-order accurate approximation.

7.8.1 Verification Using Python and SymPy

To further verify this, we use Python's symbolic computation package SymPy to perform the Taylor expansion programmatically:

```

1  _____ Verifying a Finite Difference Formula for a High-Order Derivative _____
2  from sympy import symbols, Function, series, diff, factorial, latex
3
4  # Define symbols
5  x, h = symbols('x h')
6  f = Function('f')(x)
7
8  # Define Taylor expansions up to order 5
9  def taylor_series(k):
10     return sum([(k*h)**j*diff(f, (x, j))/factorial(j) for j in range(6)])
11
12 # Compute the finite difference expression
13 finite_difference_expr = (
14     taylor_series(3/2) - 3*taylor_series(1/2)
15     + 3*taylor_series(-1/2) - taylor_series(-3/2)
16 ) / h**3
17
18 # Expand and simplify
19 finite_difference_expansion = finite_difference_expr.series(h, 0, 6).removeO()
20 finite_difference_expansion

```

The calculation results in:

$$0.125h^2 \frac{d^5}{dx^5} f(x) + 1.0 \frac{d^3}{dx^3} f(x)$$

This Python script confirms that the leading term is $f'''(x)$. The error remains proportional to $h^2 f^{(5)}(x)$, verifying the accuracy of the finite difference approximation.

7.8.2 An Alternative Central Difference Third Derivative Approximation

The central difference approximation $f'''(x) \approx \frac{\delta^3 f(x)}{h^3}$ is not unique. Another third derivative central difference formula is given by:

$$f'''(x) \approx \frac{f(x+2h) - 2f(x+h) + 2f(x-h) - f(x-2h)}{2h^3}. \quad (7.1)$$

7.8.3 Basic Example

Consider a function $f(x) = \sin(x)$ with data points $x_i = i \cdot h$ for $i = 0, 1, 2, \dots, n$ and $h = 0.1$. To approximate the first and second derivatives at $x = 0.3$ using forward and central differences:

Forward Difference

First-order forward difference:

$$f'(0.3) = 0.955336 \approx \frac{f(0.4) - f(0.3)}{0.1} = 0.938981$$

Second-order forward difference:

$$f''(0.3) = -0.29552 \approx \frac{f(0.5) - 2f(0.4) + f(0.3)}{0.01} = -0.389094$$

Central Difference

First-order central difference:

$$f'(0.3) = 0.955336 \approx \frac{f(0.35) - f(0.25)}{0.1} = 0.954938$$

Second-order central difference:

$$f''(0.3) = -0.29552 \approx \frac{f(0.4) - 2f(0.3) + f(0.2)}{0.01} = -0.295274$$

Remark 7.1 (Central vs. forward difference for approximating derivative). The most striking in this numerical experiment is greatly increased accuracy of the estimate of $f'(x)$ by using the central difference rather than the forward difference (the difference quotient). This is because the central difference has truncation error $\mathcal{O}(h^2)$ and the difference quotient has truncation error $\mathcal{O}(h)$. $\square$

7.9 Stirling's Central Difference Interpolation Formula

Stirling's interpolation formula is given by:

$$\begin{aligned} f(x) = & y_0 + \frac{p}{1!} \frac{\overline{\Delta y_{-1} + \Delta y_0}}{2} \\ & + \frac{p^2}{2!} \Delta^2 y_{-1} \\ & + \frac{p(p^2 - 1)}{3!} \frac{\Delta^3 y_{-2} + \Delta^3 y_{-1}}{2} \\ & + \frac{p^2(p^2 - 1)}{4!} \Delta^4 y_{-2} \\ & + \frac{p(p^2 - 1)(p^2 - 4)}{5!} \frac{\Delta^4 y_{-3} + \Delta^4 y_{-2}}{2} \\ & + \dots \end{aligned}$$

The formula is the average of the forward and backward difference formulas, carefully rewritten to be convenient for calculations by hand.

Basic Example

This example is meant for by-hand calculation.

Let us consider the function $f(x) = x^3$ and interpolate at $x = 3.5$, using the given data:

x	$f(x) = x^3$
1	1
2	8
3	27
4	64
5	125

The step size $h = 1$ and taking $x_0 = 3$, we calculate the finite differences in a tabular format typically used for this type of calculation:

x	y	Δy	$\Delta^2 y$	$\Delta^3 y$	$\Delta^4 y$
1	1				
		7			
2	8		12		
		19		6	
3	27		18		0
		37		6	
4	64		24		
		61			
5	125				

The boxed quantities are the ones participating in calculating the approximation. Furthermore, if two quantities are boxed in the same column, only their average participates in the calculation.

For $x = 3.5$, we take $p = \frac{3.5-3}{1} = 0.5$. Applying Stirling's interpolation formula:

$$\begin{aligned}
 f(3.5) &\approx 27 + 0.5 \times \frac{37 + 19}{2} + \frac{0.5^2}{2!} \times 18 \\
 &\quad + \frac{0.5(0.5^2 - 1)}{3!} \times \frac{6 + 6}{2} \\
 &= 27 + 0.5 \times (37 + 19)/2 + 0.5^2/2! \times 18 + 0.5 \times (0.5^2 - 1)/3! \times (6 + 6)/2 \\
 &= 27 + 14.0 + 2.25 - 0.375 \\
 &= 42.875
 \end{aligned}$$

On the other hand $f(3.5) = 42.875$. Thus, the approximate value is exact, as it should be for a cubic function.

7.9.1 Numerical Example

This example cannot be easily computed by hand, but can be followed with a calculator (we keep approximately 10 digits after the decimal point).

Let us consider the function $f(x) = \sqrt{x}$ and interpolate at $x = 8.5$ using the given data:

x	$f(x) = \sqrt{x}$
6	2.4494897428
7	2.6457513111
8	2.8284271247
9	3.0
10	3.1622776602

The difference table is:

x	y	Δy	$\Delta^2 y$	$\Delta^3 y$	$\Delta^4 y$
1	2.4494897428				
2	2.6457513111	0.1962615683			
3	2.8284271247	0.1826758137	-0.0135857546	0.0024828162	
4	3.0	0.1715728753	-0.0111029384	0.0018077233	-0.0006750928
5	3.1622776602	0.1622776602	-0.0092952151		

The step size is $h = 1$ and taking $x_0 = 8$, we calculate the finite differences:

For $x = 8.5$, we take $p = \frac{8.5-8}{1} = 0.5$.

Applying Stirling's interpolation formula:

$$\begin{aligned}
 f(8.5) &\approx 2.8284271247 + 0.5 \times \frac{3.0 - 2.6457513111}{2} + \frac{0.5^2}{2!} \times (-0.0111029384) \\
 &\quad + \frac{0.5(0.5^2 - 1)}{3!} \times \frac{0.0024828162 + 0.0018077233}{2} \\
 &\quad + \frac{0.5^2(0.5^2 - 1)}{4!} \times (-0.0006750928).
 \end{aligned}$$

Computing term by term,

$$\begin{aligned}
 f(8.5) &\approx 2.8284271247 + 0.0885621722 + (-0.0013878673) + (-0.0001340794) + 5.2742 \cdot 10^{-6} \\
 &\approx 2.9154726244.
 \end{aligned}$$

The actual value $f(8.5) = \sqrt{8.5} = 2.9154759474$, showing that Stirling's interpolation provides a good approximation.

7.10 Finite Difference Calculus in Terms of Operator Theory

Finite difference calculus can be understood using operator theory, where discrete differentiation is expressed in terms of shift operators and difference operators. One of the consequences of establishing this approach is the explanation of the appearance of binomial coefficients in various formulas of this chapter.

7.10.1 Shift Operator and Difference Operators

Let the shift operator S_h act on a function $f(x)$ as:

$$(S_h f)(x) = f(x + h). \quad (7.2)$$

Using this, we define the forward and backward difference operators.

The **forward difference** operator is given by:

$$\Delta_h f(x) = f(x + h) - f(x) = (S_h - I)f(x), \quad (7.3)$$

where I is the identity operator.

Similarly, the **backward difference** operator is defined as:

$$\nabla_h f(x) = f(x) - f(x - h) = (I - S_h^{-1})f(x). \quad (7.4)$$

For small h , both $\Delta_h f(x)/h$ and $\nabla_h f(x)/h$ approximate the derivative $f'(x)$.

A centered difference can also be defined as:

$$\delta_h f(x) = \frac{\Delta_h + \nabla_h}{2} = \frac{S_h - S_h^{-1}}{2} f(x). \quad (7.5)$$

7.10.2 Higher-Order Differences and Binomial Expansion

Applying the forward difference operator n times gives the n -th order forward difference:

$$\Delta_h^n f(x) = \sum_{k=0}^n (-1)^{n-k} \binom{n}{k} f(x + kh). \quad (7.6)$$

This follows from expanding $(S_h - I)^n$ using the binomial theorem:

$$\Delta_h^n = (S_h - I)^n = \sum_{k=0}^n \binom{n}{k} S_h^k (-1)^{n-k}. \quad (7.7)$$

Similarly, the n -th order backward difference is:

$$\nabla_h^n f(x) = \sum_{k=0}^n (-1)^k \binom{n}{k} f(x - kh). \quad (7.8)$$

This follows from expanding $(I - S_h^{-1})^n$:

$$\nabla_h^n = (I - S_h^{-1})^n = \sum_{k=0}^n \binom{n}{k} S_h^{-k} (-1)^k. \quad (7.9)$$

7.10.3 Discrete Laplacian and Exponential Representation

The second-order difference operator (discrete Laplacian) is given by:

$$\Delta_h^2 f(x) = f(x + 2h) - 2f(x + h) + f(x) = (S_h^2 - 2S_h + I)f(x). \quad (7.10)$$

Similarly, the backward difference form is:

$$\nabla_h^2 f(x) = f(x) - 2f(x - h) + f(x - 2h) = (I - 2S_h^{-1} + S_h^{-2})f(x). \quad (7.11)$$

For small h , the shift operator can be approximated as:

$$S_h = e^{hD}, \quad S_h^{-1} = e^{-hD}, \quad (7.12)$$

where $D = d/dx$ is the continuous derivative operator. This leads to the approximation:

$$\Delta_h = e^{hD} - I \approx hD + \frac{h^2}{2}D^2 + O(h^3), \quad (7.13)$$

$$\nabla_h = I - e^{-hD} \approx hD - \frac{h^2}{2}D^2 + O(h^3). \quad (7.14)$$

Thus, finite difference operators serve as discrete analogs of differential operators in a computational setting.

7.11 Mean Value Theorem for Divided Differences

The following theorem is at the core of many error formulas when interpolating by a polynomial:

Theorem 7.1 (Mean Value Theorem for Divided Differences). Let $f : [a, b] \rightarrow \mathbb{R}$ be an n -times continuously differentiable function on an interval containing the distinct points $x_0, x_1, \dots, x_n$, where $x_0 < x_1 < \dots < x_n$. Then there exists a point $\xi \in (x_0, x_n)$ such that the divided difference satisfies:

$$f[x_0, x_1, \dots, x_n] = \frac{f^{(n)}(\xi)}{n!}. \quad (7.15)$$

□

This result is sometimes called the *Generalized Mean Value Theorem for Divided Differences*.

7.11.1 Proof

We have the following identity

$$f(x) = \overbrace{f[x_0] + \sum_{k=1}^n f[x_0, x_1, \dots, x_k] \prod_{i=0}^{k-1} (x - x_i)}^{Q_n(x)} + f[x_0, x_1, \dots, x_n, x] \prod_{i=0}^n (x - x_i)$$

where $Q_n(x)$ is the n -th Lagrange interpolating polynomial.

Let $g(x) \stackrel{\text{def}}{=} f(x) - Q_n(x)$. By definition of a Lagrange interpolating polynomial, $f(x) = Q_n(x)$ for $x = x_0, x_1, \dots, x_n$. Hence, $g(x)$ has $(n+1)$ zeros in the interval $[x_0, x_n]$. By Rolle's theorem, $g'(x)$ has n zeros (one between two consecutive zeros of $g(x)$), $g''(x)$ has $n-1$ zeros, and so on. Finally, $g^{(n)}(x)$ has at least one zero. Let ξ be that zero. Then $g^{(n)}(\xi) = 0$. On the other hand $g^{(n)}(x) = f^{(n)}(x) - n!f[x_0, x_1, \dots, x_n]$ because $f[x_0, x_1, \dots, x_n]$ is the coefficient at x^n , which is the highest power of x in $Q_n(x)$. Therefore

$$0 = g^{(n)}(\xi) = f^{(n)}(\xi) - n!f[x_0, x_1, \dots, x_n].$$

In other words,

$$f[x_0, x_1, \dots, x_n] = \frac{f^{(n)}(\xi)}{n!}.$$

7.11.2 References

The standard sources include [6, 22, 25, 3]

7.12 Exercises

Exercise 7.1. Given the data points $(1, 2), (2, 3), (3, 5)$, construct the Lagrange interpolating polynomial and evaluate it at $x = 2.5$. $\square$

Exercise 7.2. Given the data points $(1, 1), (2, 4), (3, 9)$, construct the Newton interpolating polynomial and evaluate it at $x = 2.5$. $\square$

Exercise 7.3. Given the data points $(0, 1), (1, 2), (2, 0), (3, 2)$, construct the cubic spline interpolant and evaluate it at $x = 1.5$. $\square$

Exercise 7.4. Given the data points $(1, 2), (2, 4), (3, 1), (4, 3)$, use linear interpolation for $[1, 2]$, quadratic interpolation for $[2, 3]$, and cubic interpolation for $[3, 4]$. Construct the piecewise interpolant and evaluate it at $x = 2.5$. $\square$

Exercise 7.5. For the function $f(x) = \sin(x)$, interpolate the data points $(0, 0), (\frac{\pi}{2}, 1), (\pi, 0)$ using Lagrange interpolation and compute the interpolation error at $x = \frac{\pi}{4}$. $\square$

Exercise 7.6. Given the function $f(x) = e^x$, use the first-order forward difference formula to approximate the first derivative at $x = 1$ with $h = 0.1$. Compare your result with the exact derivative. $\square$

Exercise 7.7. Given the function $f(x) = \cos(x)$, use the second-order central difference formula to approximate the second derivative at $x = \pi/4$ with $h = 0.1$. Compare your result with the exact derivative. $\square$

Exercise 7.8. Using Newton's forward difference formula, interpolate the value of the function $f(x) = \ln(x+1)$ at $x = 0.25$ given the data points $(0, 0), (0.5, \ln(1.5)), (1, \ln(2))$. $\square$

Exercise 7.9. Using Newton's central difference formula, interpolate the value of the function $f(x) = x^2$ at $x = 2.5$ given the data points $(2, 4), (3, 9), (4, 16), (5, 25)$. $\square$

Exercise 7.10. Given the function $f(x) = e^x$, use the forward difference formula to approximate the first derivative at $x = 0$ with $h = 0.1$. Compare your result with the exact derivative. $\square$

Exercise 7.11. Given the function $f(x) = \sin(x)$, use the backward difference formula to approximate the first derivative at $x = \pi/4$ with $h = 0.1$. Compare your result with the exact derivative. $\square$

Exercise 7.12. Given the function $f(x) = \ln(x)$, use the central difference formula to approximate the first derivative at $x = 2$ with $h = 0.1$. Compare your result with the exact derivative. $\square$

Exercise 7.13. For the data points $(0, 1), (0.1, 1.1052), (0.2, 1.2214), (0.3, 1.3499), (0.4, 1.4918)$, use Newton's forward difference formula to interpolate the value of the function at $x = 0.25$. $\square$

Exercise 7.14. For the data points $(0, 1), (0.1, 0.9950), (0.2, 0.9801), (0.3, 0.9553), (0.4, 0.9211)$, use Newton's backward difference formula to interpolate the value of the function at $x = 0.35$. $\square$

Exercise 7.15. Prove that the truncation error of formula (7.1) is $\mathcal{O}(h^2)$. $\square$

Chapter 8

Numerical Integration

8.1 Introduction

Numerical integration, along with numerical differentiation techniques studied in Chapter 7, are fundamental tools in numerical analysis, providing methods to approximate derivatives and integrals of functions based on discrete data points. These techniques are essential when dealing with functions that cannot be differentiated or integrated analytically.

Numerical integration involves approximating the integral of a function using discrete data points. Common numerical integration methods include the trapezoidal rule, Simpson's rule, and Gaussian quadrature.

The trapezoidal rule and Simpson's rule belong to a family known as Newton-Cotes formulas, following the idea of replacing a function by its interpolating polynomial.

8.1.1 Trapezoidal Rule

The trapezoidal rule approximates the integral of a function by dividing the area under the curve into trapezoids and summing their areas. For a function f defined on the interval $[a, b]$, the trapezoidal rule is given by:

$$\int_a^b f(x) dx \approx \frac{f(a) + f(b)}{2} \times (b - a).$$

where $h = \frac{b-a}{n}$ is the width of each subinterval. This is an approximation of the area under the graph by a trapezoid with vertices $(a, 0)$, $(b, 0)$, $(b, f(b))$, and $(a, f(a))$, counting off vertices as we traverse the boundary counterclockwise.

8.1.2 Derivation of the Single-Interval Trapezoidal Rule Error Formula

Consider the trapezoidal rule for approximating the integral of a twice continuously differentiable function f on $[a, b]$. The *single-interval trapezoidal approximation* is

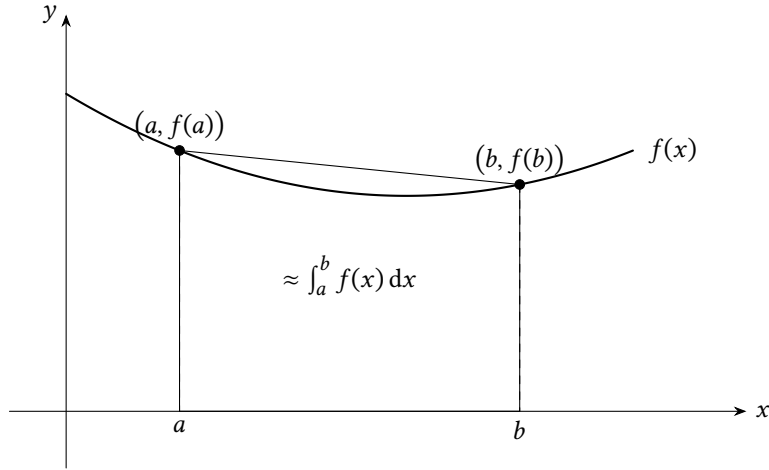
$$T(f) = \frac{b-a}{2} [f(a) + f(b)].$$

We claim that

$$\int_a^b f(x) dx - T(f) = -\frac{(b-a)^3}{12} f''(\xi) \quad \text{for some } \xi \in [a, b].$$

This term on the right side is called the *error* (or *remainder*) of the trapezoidal rule.

Figure 8.1: Approximation of an integral by the area of a trapezoid



Step 1: Interpolation by a line. Let $p(x)$ be the unique polynomial of degree 1 that interpolates f at $x = a$ and $x = b$, so

$$p(a) = f(a), \quad p(b) = f(b).$$

Then $p(x)$ can be written explicitly as

$$p(x) = f(a) \frac{b-x}{b-a} + f(b) \frac{x-a}{b-a}.$$

The trapezoidal rule $T(f)$ is precisely the integral of p over $[a, b]$:

$$\int_a^b p(x) dx = \frac{b-a}{2} [f(a) + f(b)] = T(f).$$

Step 2: Express $f(x)$ via the interpolation remainder. By the standard interpolation remainder for a degree-1 interpolant, for each $x \in [a, b]$ there exists some $\xi_x \in [a, b]$ such that

$$f(x) - p(x) = \frac{f''(\xi_x)}{2} (x-a)(x-b).$$

Hence we may write

$$f(x) = p(x) + \frac{f''(\xi_x)}{2} (x-a)(x-b).$$

Step 3: Integrate the difference $f(x) - p(x)$. The error in the trapezoidal rule is

$$\int_a^b f(x) dx - \int_a^b p(x) dx = \int_a^b [f(x) - p(x)] dx.$$

Using the interpolation remainder expression,

$$\int_a^b [f(x) - p(x)] dx = \int_a^b \frac{f''(\xi_x)}{2} (x-a)(x-b) dx.$$

Since f'' is continuous, by the Mean Value Theorem for integrals there is a single $\xi \in [a, b]$ such that

$$\int_a^b \frac{f''(\xi_x)}{2} (x-a)(x-b) dx = \frac{f''(\xi)}{2} \int_a^b (x-a)(x-b) dx.$$

Step 4: Evaluate $\int_a^b (x-a)(x-b) dx$. Let $h = b - a$. Then with the substitution $x = a + t$ (so t runs from 0 to h), we get

$$(x-a)(x-b) = t(t-h) = t^2 - ht.$$

Thus,

$$\int_a^b (x-a)(x-b) dx = \int_0^h (t^2 - ht) dt = \left[\frac{t^3}{3} - \frac{ht^2}{2} \right]_0^h = \frac{h^3}{3} - \frac{h^3}{2} = -\frac{h^3}{6}.$$

Hence,

$$\int_a^b (x-a)(x-b) dx = -\frac{(b-a)^3}{6}.$$

Step 5: Conclude the error formula. Putting these pieces together,

$$\int_a^b [f(x) - p(x)] dx = \frac{f''(\xi)}{2} \left(-\frac{(b-a)^3}{6} \right) = -\frac{(b-a)^3}{12} f''(\xi).$$

Therefore, recalling that $\int_a^b p(x) dx = T(f)$, the trapezoidal rule error is

$$\int_a^b f(x) dx - \frac{b-a}{2} [f(a) + f(b)] = -\frac{(b-a)^3}{12} f''(\xi),$$

for some $\xi \in [a, b]$, which completes the derivation.

8.1.3 Single-Step Simpson's Rule and Its Error Term

The purpose of this section is to sketch a derivation of the error term. The argument will skip some steps, and therefore is not fully rigorous. A complete proof can be found in Appendix R.

Let f be four times continuously differentiable on the interval $[a, b]$. Define

$$h = b - a, \quad m = \frac{a+b}{2}.$$

Simpson's rule approximates $\int_a^b f(x) dx$ by integrating the unique *quadratic* polynomial $p(x)$ that interpolates f at the three equally spaced points $x = a$, $x = m$, and $x = b$. Concretely,

$$p(a) = f(a), \quad p(m) = f(m), \quad p(b) = f(b).$$

The *Simpson's approximation* is then

$$S(f) = \int_a^b p(x) dx = \frac{h}{6} [f(a) + 4f(m) + f(b)].$$

Below, we derive this formula *and* its classical remainder term

$$\int_a^b f(x) dx - S(f) = -\frac{(b-a)^5}{2880} f^{(4)}(\xi) \quad \text{for some } \xi \in [a, b].$$

1. Taylor expansions about the midpoint. A clear way to see why the fourth derivative appears is to expand f in a Taylor series around the midpoint $m = \frac{a+b}{2}$. Let

$$x = m + t, \quad \text{where } t \in \left[-\frac{h}{2}, \frac{h}{2}\right].$$

Then the Taylor expansion of $f(x)$ about $x = m$ is

$$f(m+t) = f(m) + t f'(m) + \frac{t^2}{2!} f''(m) + \frac{t^3}{3!} f^{(3)}(m) + \frac{t^4}{4!} f^{(4)}(\xi_{m+t}),$$

where ξ_{m+t} is some point between m and $m+t$ (depending on t) by Taylor's theorem with remainder.

2. Exact integral of $f(x)$. We can write

$$\int_a^b f(x) dx = \int_{-h/2}^{h/2} f(m+t) dt.$$

Substitute the above Taylor expansion term by term and use the fact that odd powers of t integrate to zero over $[-\frac{h}{2}, \frac{h}{2}]$. In particular,

$$\int_{-h/2}^{h/2} t dt = 0 \quad \text{and} \quad \int_{-h/2}^{h/2} t^3 dt = 0,$$

whereas

$$\int_{-h/2}^{h/2} t^2 dt = \frac{h^3}{12}, \quad \int_{-h/2}^{h/2} t^4 dt = \frac{h^5}{80}.$$

Thus,

$$\int_a^b f(x) dx = \int_{-h/2}^{h/2} \left[f(m) + t f'(m) + \frac{t^2}{2} f''(m) + \frac{t^3}{6} f^{(3)}(m) + \frac{t^4}{24} f^{(4)}(\xi_{m+t}) \right] dt.$$

Collecting the nonzero integrals:

$$\int_a^b f(x) dx = \int_{-h/2}^{h/2} f(m) dt + \int_{-h/2}^{h/2} \frac{t^2}{2} f''(m) dt + \int_{-h/2}^{h/2} \frac{t^4}{24} f^{(4)}(\xi_{m+t}) dt.$$

Hence

$$\int_a^b f(x) dx = h f(m) + \frac{h^3}{24} f''(m) + \underbrace{\frac{1}{24} \int_{-h/2}^{h/2} t^4 f^{(4)}(\xi_{m+t}) dt}_{\text{fourth-derivative term}}.$$

Since $f^{(4)}$ is continuous, by the Mean Value Theorem for integrals there is a $\xi \in [a, b]$ such that

$$\int_{-h/2}^{h/2} t^4 f^{(4)}(\xi_{m+t}) dt = f^{(4)}(\xi) \int_{-h/2}^{h/2} t^4 dt = f^{(4)}(\xi) \frac{h^5}{80}.$$

Thus,

$$\int_a^b f(x) dx = h f(m) + \frac{h^3}{24} f''(m) + \frac{h^5}{1920} f^{(4)}(\xi).$$

3. Simpson's approximation $S(f)$. Now let us expand *Simpson's* approximation:

$$S(f) = \frac{h}{6} [f(a) + 4f(m) + f(b)].$$

We similarly use Taylor expansions (about m) for $f(a) = f(m - \frac{h}{2})$ and $f(b) = f(m + \frac{h}{2})$. One finds that *all odd-power terms* cancel in the sum $f(a) + f(b)$, so

$$f(a) + f(b) = 2f(m) + \frac{h^2}{4} f''(m) + (\text{terms involving } f^{(4)}).$$

Hence

$$f(a) + 4f(m) + f(b) = 6f(m) + \frac{h^2}{4} f''(m) + (\text{higher derivative contributions}),$$

and a short computation shows

$$S(f) = h f(m) + \frac{h^3}{24} f''(m) + \frac{h^5}{2304} [\dots f^{(4)}]$$

for some combination of values of $f^{(4)}$. More precisely, one arrives at the known result:

$$S(f) = h f(m) + \frac{h^3}{24} f''(m) + \text{a term involving } f^{(4)}.$$

4. The final error formula. Comparing the expansions for $\int_a^b f(x) dx$ and $S(f)$ reveals that *the first* mismatch occurs in the terms involving the fourth derivative. A more careful bookkeeping (or a standard reference on Newton–Cotes rules) gives the classical closed-form of the remainder:

$$\int_a^b f(x) dx - S(f) = -\frac{(b-a)^5}{2880} f^{(4)}(\xi) \quad \text{for some } \xi \in [a, b].$$

The negative sign reflects the way the interpolation parabola bends relative to the true function f .

Summary. *Single-step Simpson’s rule* exactly integrates any quadratic polynomial but introduces an error proportional to the fourth derivative of f . Explicitly,

$$S(f) = \frac{b-a}{6} \left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right], \quad \int_a^b f(x) dx - S(f) = -\frac{(b-a)^5}{2880} f^{(4)}(\xi).$$

Thus, for smooth f , Simpson’s rule attains $\mathcal{O}(h^5)$ local accuracy ($\mathcal{O}(h^4)$ in the error constant itself), making it more accurate than either the trapezoidal or midpoint rules (which depend on the second derivative).

8.2 Composite Rules for Adaptive Integration

Composite rules involve applying numerical integration methods over subintervals to improve accuracy. Adaptive integration methods dynamically adjust the size of the subintervals based on the function’s behavior to achieve the desired accuracy.

8.2.1 Composite Trapezoidal Rule

The composite trapezoidal rule divides the integration interval into smaller subintervals and applies the trapezoidal rule to each subinterval. The composite trapezoidal rule for a function f on the interval $[a, b]$ is given by:

$$\begin{aligned} \int_a^b f(x) dx &\approx \sum_{j=0}^{m-1} \frac{h}{2} (f(x_j) + f(x_{j+1})) \\ &= \frac{h}{2} \left(f(a) + 2 \sum_{j=1}^{m-1} f(x_j) + f(b) \right) \end{aligned}$$

where $x_j = a + jh$ and $h = \frac{b-a}{m}$. The second form cuts the number of function evaluations in half, which may result in significant savings.

8.2.2 Composite Simpson’s Rule

Simpson’s rule can be extended to approximate the integral of a function f over $[a, b]$ by subdividing the interval into multiple (even number of) subintervals and applying Simpson’s rule on each pair of subintervals.

Partitioning the interval. Let $2n$ be an even integer. We partition $[a, b]$ into $2n$ subintervals, each of width

$$h = \frac{b-a}{2n}.$$

Define the nodes

$$x_i = a + ih, \quad i = 0, 1, 2, \dots, 2n.$$

Hence $x_0 = a$ and $x_{2n} = b$.

Formula. The *composite Simpson's rule* approximation of $\int_a^b f(x) dx$ is given by summing up Simpson's contributions on each pair $[x_{2k}, x_{2k+2}]$. Combining them into a single formula yields

$$S_{2n}(f) = \frac{h}{3} [f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + 2f(x_4) + \cdots + 4f(x_{2n-1}) + f(x_{2n})].$$

In other words, the coefficients alternate in the pattern 1, 4, 2, 4, 2, ..., 4, 1 (with the understanding that the very first and last nodes use coefficient 1, the odd-indexed interior nodes use coefficient 4, and the even-indexed interior nodes use coefficient 2).

Error bound. If f is four times continuously differentiable on $[a, b]$, then there is some $\xi \in [a, b]$ such that

$$\int_a^b f(x) dx - S_{2n}(f) = -\frac{(b-a)^5}{180(2n)^4} f^{(4)}(\xi).$$

Equivalently, the magnitude of the error can be bounded by

$$\left| \int_a^b f(x) dx - S_{2n}(f) \right| \leq \frac{(b-a)^5}{180(2n)^4} \max_{x \in [a,b]} |f^{(4)}(x)|.$$

Remark 8.1. Compared to the *single-step* Simpson's rule, the *composite* version applies Simpson's rule on many small intervals (each of length h). This generally yields a higher accuracy for smooth functions without requiring extremely high-degree polynomials over the entire interval. Indeed, for a fixed number of subintervals $2n$, the local error on each pair of subintervals scales like $\mathcal{O}(h^5)$, and summing over n pairs gives an overall error of order $\mathcal{O}(h^4)$. $\square$

8.2.3 Gaussian Quadrature

Gaussian quadrature is a numerical integration technique that approximates the integral of a function using a weighted sum of the function values at specific points (nodes) within the integration interval. The method is particularly powerful because it can achieve exact results for polynomials of degree up to $2n - 1$ using n nodes.

Formula

For a function $f(x)$ defined on the interval $[-1, 1]$, the Gaussian quadrature formula is given by:

$$\int_{-1}^1 f(x) dx \approx \sum_{i=1}^n w_i f(x_i)$$

Here, x_i are the nodes and w_i are the corresponding weights. The nodes are the roots of the Legendre polynomials $P_n(x)$, and the weights are chosen to provide the best approximation.

8.2.4 Legendre Polynomials

Legendre polynomials are defined via Rodrigues' formula:

$$P_n(x) = \frac{1}{2^n n!} \frac{d^n}{dx^n} (x^2 - 1)^n. \quad (8.1)$$

These polynomials satisfy the orthogonality condition:

$$\int_{-1}^1 P_m(x) P_n(x) dx = \frac{2}{2n+1} \delta_{mn}. \quad (8.2)$$

The roots x_i are symmetric about $x = 0$ and lie within $(-1, 1)$.

Nodes and Weights

The nodes x_i are the roots of the n -th Legendre polynomial $P_n(x)$, and the weights w_i can be computed to ensure the quadrature rule is exact for polynomials up to degree $2n - 1$.

For commonly used values of n , the nodes and weights are as follows:

For $n = 2$ nodes:

$$x_1 = -\frac{1}{\sqrt{3}}, \quad x_2 = \frac{1}{\sqrt{3}}$$
$$w_1 = w_2 = 1$$

For $n = 3$ nodes:

$$x_1 = -\sqrt{\frac{3}{5}}, \quad x_2 = 0, \quad x_3 = \sqrt{\frac{3}{5}}$$
$$w_1 = w_3 = \frac{5}{9}, \quad w_2 = \frac{8}{9}$$

For $n = 4$ nodes:

$$x_1 = -\sqrt{\frac{3}{7} + \frac{2}{7}\sqrt{\frac{6}{5}}}, \quad x_2 = -\sqrt{\frac{3}{7} - \frac{2}{7}\sqrt{\frac{6}{5}}}$$
$$x_3 = \sqrt{\frac{3}{7} - \frac{2}{7}\sqrt{\frac{6}{5}}}, \quad x_4 = \sqrt{\frac{3}{7} + \frac{2}{7}\sqrt{\frac{6}{5}}}$$
$$w_1 = w_4 = \frac{18 - \sqrt{30}}{36}, \quad w_2 = w_3 = \frac{18 + \sqrt{30}}{36}$$

Example

Consider the function $f(x) = e^x$ and the interval $[-1, 1]$. To approximate the integral $\int_{-1}^1 e^x dx$ using Gaussian quadrature with $n = 3$:

1. The nodes are $x_1 = -\sqrt{\frac{3}{5}}$, $x_2 = 0$, and $x_3 = \sqrt{\frac{3}{5}}$.
2. The weights are $w_1 = w_3 = \frac{5}{9}$ and $w_2 = \frac{8}{9}$.

The approximate integral is given by:

$$\int_{-1}^1 e^x dx \approx \sum_{i=1}^3 w_i f(x_i) \approx \frac{5}{9}e^{-\sqrt{\frac{3}{5}}} + \frac{8}{9}e^0 + \frac{5}{9}e^{\sqrt{\frac{3}{5}}} \approx 2.3503369286800115,$$
$$\int_{-1}^1 e^x dx = e - e^{-1} \approx 2.3504023872876028.$$

This weighted sum provides a very accurate approximation to the actual integral.

Using Gaussian Quadrature

Gaussian quadrature is a powerful tool for numerical integration, particularly when dealing with integrals that can be transformed to the standard interval $[-1, 1]$. The use of appropriate weights ensures that the method is highly accurate for polynomials of degree up to $2n - 1$, making it a preferred choice for many applications in numerical analysis.

8.2.5 Derivation of Gaussian Quadrature Weights

The weights $w_i = w_i^{(n)}$ ensure that the numerical integration is exact for polynomials of degree up to $2n - 1$. They are given by:

$$w_i^{(n)} = \frac{2}{(1 - x_i^2)[P_n'(x_i)]^2}. \quad (8.3)$$

This formula is derived using Lagrange interpolation polynomials and properties of Legendre polynomials.

8.2.6 The error bound

$$\int_{-1}^1 f(x) dx = \sum_{i=1}^n w_i^{(n)} f(x_i) + \frac{2^{2n+1}(n!)^4}{(2n+1)[(2n)!]^3} f^{(2n)}(\xi), \quad -1 < \xi < 1.$$

8.2.7 Exactness for Polynomials up to Degree $2n - 1$

To prove exactness, consider a polynomial $f(x)$ of degree at most $2n - 1$:

$$f(x) = \sum_{k=0}^{2n-1} c_k P_k(x). \quad (8.4)$$

Using the orthogonality of Legendre polynomials:

$$\int_{-1}^1 P_k(x) dx = 0 \quad \text{for all } k \geq 1, \quad (8.5)$$

only the $k = 0$ term contributes to the integral, ensuring exactness. The quadrature sum also captures this integral exactly due to the choice of weights and nodes.

8.2.8 Example: Gaussian Quadrature for $n = 2$

For $n = 2$, the Legendre polynomial is:

$$P_2(x) = \frac{1}{2}(3x^2 - 1). \quad (8.6)$$

The roots are:

$$x_1 = -\frac{1}{\sqrt{3}}, \quad x_2 = \frac{1}{\sqrt{3}}. \quad (8.7)$$

The derivative is:

$$P_2'(x) = 3x, \quad (8.8)$$

which evaluates at the roots as:

$$P_2'\left(\pm \frac{1}{\sqrt{3}}\right) = \pm \sqrt{3}. \quad (8.9)$$

Using the weight formula:

$$w_1 = w_2 = \frac{2}{(1 - 1/3)(3)} = 1. \quad (8.10)$$

Thus, the Gaussian quadrature rule for $n = 2$ is:

$$\int_{-1}^1 f(x) dx \approx f(-1/\sqrt{3}) + f(1/\sqrt{3}). \quad (8.11)$$

8.2.9 References

See [12]. This citation contains a derivation of the weights $w_i^{(n)}$. Interestingly, the nodes and weights have been tabulated up to $n = 512$. It is clear that to perform the method one has to have good approximations to the roots of $P_n(x)$. Hence, the method depends on some good root-finding algorithm.

The software library GSL described in [18] provides a free implementation in C.

8.3 Richardson Extrapolation

Richardson Extrapolation is a technique to systematically improve the accuracy of a numerical method by *extrapolating* its leading error term. It is broadly used in numerical integration, numerical differentiation, and the numerical solution of differential equations.

8.3.1 Motivation and Basic Idea

Suppose a numerical method $A(h)$ approximates some exact quantity Q using a step size (or mesh width) h . We assume $A(h)$ admits an asymptotic expansion of the form

$$A(h) = Q + \alpha h^p + \mathcal{O}(h^{p+1}),$$

where α is a constant independent of h , and p is the leading order of the local error. If we halve the step size to $h/2$, the same method yields

$$A\left(\frac{h}{2}\right) = Q + \alpha \left(\frac{h}{2}\right)^p + \mathcal{O}(h^{p+1}).$$

The key insight of Richardson Extrapolation is to form a suitable linear combination of these two approximations so that the leading error term αh^p *cancels out*, producing a more accurate approximation to Q .

8.3.2 Two-Level Extrapolation Procedure

From

$$A(h) = Q + \alpha h^p + \mathcal{O}(h^{p+1}),$$

$$A\left(\frac{h}{2}\right) = Q + \alpha \left(\frac{h}{2}\right)^p + \mathcal{O}(h^{p+1}),$$

we observe:

$$A(h) - A\left(\frac{h}{2}\right) = \alpha h^p \left(1 - \frac{1}{2^p}\right) + \mathcal{O}(h^{p+1}).$$

Hence,

$$\alpha h^p = \frac{A(h) - A\left(\frac{h}{2}\right)}{1 - \frac{1}{2^p}} + \mathcal{O}(h^{p+1}).$$

Since $A(h) = Q + \alpha h^p + \mathcal{O}(h^{p+1})$, we solve for Q :

$$Q = A(h) - \alpha h^p \approx A\left(\frac{h}{2}\right) + \frac{A\left(\frac{h}{2}\right) - A(h)}{2^p - 1}.$$

Denote this new combination by $\tilde{Q}$. One can show that the leading αh^p term is eliminated, yielding a higher-order error of $\mathcal{O}(h^{p+1})$.

8.3.3 General Framework

- **Iterative Extrapolation:** By evaluating A at a sequence of step sizes $h, \frac{h}{2}, \frac{h}{4}, \dots, \frac{h}{2^k}, \dots$, one can build a triangular table of extrapolations, each step removing the next significant error term. This is the essence of *Romberg Integration* when applied to trapezoidal approximations of integrals, and *extrapolation methods* for ODEs.
- **Unknown Exponent or Coefficient:** In some scenarios, the leading error order p might be unknown. In such cases, one can estimate both p and α from the data, and then form the combination to cancel out the estimated leading term.
- **Higher-Order Terms:** The same logic can be extended to remove multiple error terms, for example $\alpha_1 h^p + \alpha_2 h^{p+1}$, by combining more refined approximations. Each extrapolation step systematically reduces the remaining dominant error.

8.3.4 Why It Works

The success of Richardson Extrapolation relies on having an *error expansion* in powers of h . When such an expansion is valid, subtracting approximations at different step sizes is an efficient way to cancel out the first (and sometimes higher) terms in that expansion. Therefore, one obtains a *boost* in the order of accuracy at a relatively small extra computational cost (assuming the method with the smaller step size has already been computed).

8.3.5 Summary

Richardson Extrapolation is a powerful method for improving the accuracy of numerical approximations. It is widely used in:

- **Numerical integration:** e.g., Romberg Integration applies Richardson to trapezoidal rule approximations.
- **Numerical differentiation:** refined difference formulas with step sizes h and $h/2$ can yield high-accuracy estimates of derivatives.
- **ODE Solvers:** extrapolation methods (e.g., Gragg's method) iterate on coarser solutions to obtain more precise solutions.

The key assumption is that the *leading order error term* in $A(h)$ behaves like αh^p . When this holds, Richardson Extrapolation systematically removes that leading term and delivers an approximation with order $\mathcal{O}(h^{p+1})$.

8.4 Romberg Integration

Romberg integration is an advanced technique that combines the trapezoidal rule with Richardson extrapolation to achieve higher accuracy in numerical integration. It recursively refines the integration result by leveraging the error estimates from the trapezoidal rule to eliminate higher-order error terms.

8.4.1 Algorithm

The Romberg integration algorithm starts with applying the trapezoidal rule to compute the integral over increasingly finer subintervals and then uses Richardson extrapolation to improve the accuracy. The algorithm can be described as follows:

8.4.2 Example

Consider the function $f(x) = e^x$ and the interval $[0, 1]$. To approximate the integral $\int_0^1 e^x dx$ using Romberg integration with a maximum depth of $n = 2$:

★★★ Step 1: Compute $R_{0,0}$ ★★★

$$R_{0,0} = \frac{1-0}{2} (e^0 + e^1) = \frac{1}{2}(1 + e) \approx 1.85914$$

Algorithm 8.1 Romberg Integration

Require: A function f to integrate, interval $[a, b]$, and maximum depth n

Ensure: An approximation $R_{n,n}$ to the integral $\int_a^b f(x) dx$

- 1: Initialize $R_{0,0} = \frac{b-a}{2} (f(a) + f(b))$
- 2: **for** $k = 1$ **to** n **do**
- 3: Compute the trapezoidal rule for 2^k intervals:

$$R_{k,0} = \frac{1}{2} R_{k-1,0} + h_k \sum_{j=1}^{2^{k-1}} f(a + (2j-1)h_k)$$

- where $h_k = \frac{b-a}{2^k}$
- 4: **for** $j = 1$ **to** k **do**
 - 5: Apply Richardson extrapolation:

$$R_{k,j} = R_{k,j-1} + \frac{R_{k,j-1} - R_{k-1,j-1}}{4^j - 1}$$

- 6: **end for**
 - 7: **end for**
 - 8: **return** $R_{n,n}$
-

*** Step 2: For $k = 1$ ***

$$h_1 = \frac{1-0}{2} = 0.5$$

$$R_{1,0} = \frac{1}{2} R_{0,0} + h_1 f\left(a + \frac{h_1}{2}\right) = \frac{1}{2} \times 1.85914 + 0.5 \times f(0.5) = 0.92957 + 0.5 \times 1.64872 \approx 1.75393$$

*** Step 3: Apply Richardson extrapolation ***

$$R_{1,1} = R_{1,0} + \frac{R_{1,0} - R_{0,0}}{4^1 - 1} = 1.75393 + \frac{1.75393 - 1.85914}{3} \approx 1.71886$$

*** Step 4: For $k = 2$ ***

$$h_2 = \frac{1-0}{4} = 0.25$$

$$R_{2,0} = \frac{1}{2} R_{1,0} + h_2 \sum_{j=1}^{2^1} f(a + (2j-1)h_2) = \frac{1}{2} \times 1.75393 + 0.25 \times (f(0.25) + f(0.75)) \approx 1.72722$$

*** Step 5: Apply Richardson extrapolation ***

$$R_{2,1} = R_{2,0} + \frac{R_{2,0} - R_{1,0}}{4^1 - 1} = 1.72722 + \frac{1.72722 - 1.75393}{3} \approx 1.71832$$

$$R_{2,2} = R_{2,1} + \frac{R_{2,1} - R_{1,1}}{4^2 - 1} = 1.71832 + \frac{1.71832 - 1.71886}{15} \approx 1.71828$$

Thus, the Romberg integration approximation $R_{2,2}$ to the integral $\int_0^1 e^x dx$ is approximately 1.71828, which is very close to the exact value $e - 1 \approx 1.71828$.

8.5 Newton-Cotes Formulas and Gaussian Quadrature

Numerical integration techniques are used to approximate definite integrals when analytical solutions are difficult or impossible to obtain. Two primary families of quadrature rules are *Newton-Cotes formulas* and *Gaussian quadrature*.

8.5.1 Newton-Cotes Quadrature

Newton-Cotes formulas approximate an integral by interpolating the function using equidistant points. The general form for an integral over $[a, b]$ using an interpolating polynomial of degree n is:

$$I = \int_a^b f(x) dx \approx \sum_{i=0}^n w_i f(x_i), \quad (8.12)$$

where the nodes x_i are equally spaced and the weights w_i are determined based on Lagrange interpolation.

Common Newton-Cotes rules include:

1. **Trapezoidal Rule** ($n = 1$): Approximates the function by a linear interpolant.
2. **Simpson's Rule** ($n = 2$): Uses a quadratic interpolant.
3. **Boole's Rule** ($n = 4$): Uses a quartic interpolant for higher accuracy.

The main drawback of Newton-Cotes methods is that their accuracy is limited by the *Runge phenomenon* — the oscillatory nature of polynomial interpolation for large n — and their reliance on evenly spaced points, which is not optimal.

8.5.2 Gaussian Quadrature

Gaussian quadrature improves accuracy by selecting both the nodes and weights optimally. The *Gauss-Legendre quadrature rule* is specifically designed for integrals of the form:

$$I = \int_{-1}^1 f(x) dx. \quad (8.13)$$

Instead of using evenly spaced points, Gaussian quadrature selects nodes as the roots of orthogonal polynomials, ensuring that polynomials of degree up to $2n - 1$ are integrated exactly with only n nodes.

For Gauss-Legendre quadrature, the nodes are the roots of the Legendre polynomial $P_n(x)$, and the corresponding weights are computed from:

$$w_i = \frac{2}{(1 - x_i^2)[P_n'(x_i)]^2}. \quad (8.14)$$

8.5.3 Gaussian Quadrature Beyond Legendre Polynomials

While Gauss-Legendre quadrature applies to integrals with uniform weight functions $w(x) = 1$, other quadrature rules exist for different weight functions:

1. **Gauss-Chebyshev**: Used for integrals with weight function $w(x) = \frac{1}{\sqrt{1-x^2}}$, useful in problems involving singularities.
2. **Gauss-Laguerre**: Applied to integrals of the form

$$\int_0^\infty e^{-x} f(x) dx, \quad (8.15)$$

using *Laguerre polynomials* $L_n(x)$.

3. **Gauss-Hermite**: Suitable for integrals with Gaussian weight functions,

$$\int_{-\infty}^\infty e^{-x^2} f(x) dx, \quad (8.16)$$

using *Hermite polynomials* $H_n(x)$.

8.5.4 Summary of Legendre, Laguerre and Hermite Polynomials

Legendre Polynomials

Legendre polynomials $P_n(x)$ are orthogonal on the interval $[-1, 1]$ with weight function $w(x) = 1$. They satisfy the recurrence relation:

$$(n+1)P_{n+1}(x) = (2n+1)xP_n(x) - nP_{n-1}(x), \quad (8.17)$$

with initial conditions:

$$P_0(x) = 1, \quad P_1(x) = x. \quad (8.18)$$

The roots of $P_n(x)$ are used as the nodes in Gauss-Legendre quadrature.

Laguerre Polynomials

Laguerre polynomials $L_n(x)$ are orthogonal on $[0, \infty)$ with weight function $w(x) = e^{-x}$. They satisfy the recurrence relation:

$$(n+1)L_{n+1}(x) = (2n+1-x)L_n(x) - nL_{n-1}(x), \quad (8.19)$$

with initial conditions:

$$L_0(x) = 1, \quad L_1(x) = 1 - x. \quad (8.20)$$

These polynomials are used in Gauss-Laguerre quadrature for integrals involving exponential decay.

Hermite Polynomials

Hermite polynomials $H_n(x)$ are orthogonal on $(-\infty, \infty)$ with weight function $w(x) = e^{-x^2}$. They satisfy the recurrence relation:

$$H_{n+1}(x) = 2xH_n(x) - 2nH_{n-1}(x), \quad (8.21)$$

with initial conditions:

$$H_0(x) = 1, \quad H_1(x) = 2x. \quad (8.22)$$

These polynomials are used in Gauss-Hermite quadrature for integrals with Gaussian weight functions.

8.6 Exercises

Exercise 8.1 (Nodes and Weights). Compute the Gauss-Legendre nodes and weights explicitly for $n = 2$ and $n = 3$. Verify that they satisfy the quadrature rule by integrating x^2 and x^4 . $\square$

Exercise 8.2 (Computational Application). Use Gauss-Legendre quadrature with 3 points to approximate:

$$I = \int_{-1}^1 e^x dx.$$

Compare the result with the exact integral and compute the relative error. $\square$

Exercise 8.3 (Change of Interval). Gauss-Legendre quadrature applies to $[-1, 1]$, but we often need to integrate over $[a, b]$. Show that the transformation

$$x = \frac{b-a}{2}t + \frac{a+b}{2}$$

maps the interval $[-1, 1]$ to $[a, b]$. Use this transformation to approximate

$$\int_1^3 \frac{1}{x} dx$$

using a 2-point Gauss-Legendre rule. $\square$

Exercise 8.4 (Weighted Integrals). Consider the integral



$$I = \int_{-1}^1 (1 - x^2)e^x dx.$$

- (a) Identify the appropriate set of orthogonal polynomials for this weight function.
- (b) Compute the nodes and weights for a 2-point quadrature rule and approximate the integral.

□

Exercise 8.5 (Adaptive Quadrature). Write a program that implements *adaptive Gaussian quadrature*, recursively subdividing the interval when the quadrature estimate does not meet a given tolerance. Test your implementation on:



$$\int_{-1}^1 \frac{dx}{1 + 25x^2}.$$

□

Exercise 8.6 (Romberg Integration). Using Romberg integration with a maximum depth of $n = 3$, approximate the integral of $f(x) = \sin(x)$ over the interval $[0, \pi]$. Compare your result with the exact integral. □

Exercise 8.7 (Romberg Integration). Approximate the integral of $f(x) = \frac{1}{1+x^2}$ over the interval $[0, 1]$ using Romberg integration with a maximum depth of $n = 4$. Compare your result with the exact integral. □

Exercise 8.8 (Romberg Integration). Using Romberg integration with a maximum depth of $n = 2$, approximate the integral of $f(x) = x^3$ over the interval $[0, 2]$. Compare your result with the exact integral. □

Exercise 8.9 (Romberg Integration). Approximate the integral of $f(x) = e^{-x^2}$ over the interval $[0, 1]$ using Romberg integration with a maximum depth of $n = 3$. Compare your result with the exact integral. □

Exercise 8.10 (Finite Difference Methods). Given the function $f(x) = e^x$, use the forward difference formula to approximate the first derivative at $x = 1$ with $h = 0.1$. Compare your result with the exact derivative. □

Exercise 8.11 (Finite Difference Methods). Given the function $f(x) = \sin(x)$, use the backward difference formula to approximate the first derivative at $x = \pi/4$ with $h = 0.1$. Compare your result with the exact derivative. □

Exercise 8.12 (Finite Difference Methods). Given the function $f(x) = \ln(x)$, use the central difference formula to approximate the first derivative at $x = 2$ with $h = 0.1$. Compare your result with the exact derivative. □

Exercise 8.13 (Trapezoidal Rule). Compute the integral of $f(x) = x^2$ over the interval $[0, 1]$ using the trapezoidal rule with $n = 4$ subintervals. Compare your result with the exact integral. □

Exercise 8.14 (Simpson's Rule). Compute the integral of $f(x) = \sin(x)$ over the interval $[0, \pi]$ using Simpson's rule with $n = 6$ subintervals. Compare your result with the exact integral. □

Exercise 8.15 (Gaussian Quadrature). Using Gaussian quadrature with $n = 3$ nodes, approximate the integral of $f(x) = \cos(x)$ over the interval $[-1, 1]$. Compare your result with the exact integral. □

Exercise 8.16 (Composite Trapezoidal Rule). Compute the integral of $f(x) = \sqrt{x}$ over the interval $[0, 4]$ using the composite trapezoidal rule with $n = 8$ subintervals. Compare your result with the exact integral. □

Exercise 8.17 (Composite Simpson's Rule). Compute the integral of $f(x) = \frac{1}{1+x^2}$ over the interval $[-1, 1]$ using the composite Simpson's rule with $n = 10$ subintervals. Compare your result with the exact integral. □

Exercise 8.18 (Orthogonality of Legendre Polynomials). Prove orthogonality of Legendre Polynomials:

$$\int_{-1}^1 P_m(x)P_n(x) dx = \frac{2}{2n+1} \delta_{mn}.$$

Use two definitions:

- (a) the recurrence relation (8.17);
 (b) the Rodrigues's Formula (8.1).

□



Exercise 8.19 (Constructing the Weights in Gauss-Legendre Quadrature). Let $P_n(x)$ be a Legendre polynomial and let x_i be its roots ($i = 1, 2, \dots, n$). Show that

- (a) the weights w_i are determined by integrating the Lagrange interpolation basis function $\ell_i(x)$ over $[-1, 1]$:

$$w_i = \int_{-1}^1 \ell_i(x) dx,$$

where

$$\ell_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}.$$

- (b) Applying the properties of orthogonal polynomials, show that the explicit formula for w_i is given by:

$$w_i = \frac{2}{(1 - x_i^2)[P'_n(x_i)]^2}.$$

□



Exercise 8.20 (Exactness of Gauss-Legendre Quadrature). Show that Gauss-Legendre quadrature with n nodes exactly integrates any polynomial of degree at most $2n - 1$. Use the fact that the Legendre polynomials form an orthogonal basis with respect to the standard inner product. □

Exercise 8.21 (Legendre Polynomials and Gram-Schmidt Process). Another way to obtain Legendre polynomials is to apply Gram-Schmidt process to the set $\{1, x, x^2, \dots, x^N\}$ in $L^2([-1, 1])$, i.e. using the inner product

$$\langle f, g \rangle = \int_{-1}^1 f(x)g(x) dx.$$

Compute the resulting polynomials for $N = 3$ and compare with the ones obtained from Rodrigues's Formula. Note the normalizing constants. □



Exercise 8.22 (Error Bound). Derive the error formula for Gauss-Legendre quadrature:

$$E_n(f) = \frac{(b-a)^{2n+1} f^{(2n)}(\xi)}{(2n+1)2^{2n}(n!)^2},$$

and explain why Gaussian quadrature performs better than Newton-Cotes formulas for smooth functions. □

Exercise 8.23 (Trapezoidal Rule and Convex Functions). Prove that the Trapezoidal Rule overestimates the value of the integral of a convex function. I.e., if $f : [a, b] \rightarrow \mathbb{R}$ is a convex function then

$$\int_a^b f(x) dx \leq \frac{b-a}{2}(f(a) + f(b)).$$

Deduce that the Composite Trapezoidal Rule also overestimates the integral. □

Exercise 8.24 (Midpoint Rule and Convex Functions). The Midpoint Rule is

$$\int_a^b f(x) dx \approx (b-a) \times f\left(\frac{a+b}{2}\right).$$

See Figure 8.2 for a graphical explanation. The area under the graph of f is approximated by a rectangle. Prove that if f is a convex function then the Midpoint Rule is an underestimate of the value of the integral, i.e.,

$$\int_a^b f(x) dx \geq (b-a) \times f\left(\frac{a+b}{2}\right).$$

Hint: Replace the rectangle with a trapezoid of equal area. □

Exercise 8.25 (Bracketed Richardson Extrapolation). Develop an algorithm based on Trapezoidal and Midpoint Rules (see Exercises 8.23 and 8.24), and Richardson Extrapolation. The algorithm must produce a *bracket* for the integral:



$$\text{lower bound} \leq \int_a^b f(x) dx \leq \text{upper bound}$$

given that f is a convex function on the interval $[a, b]$. $\square$

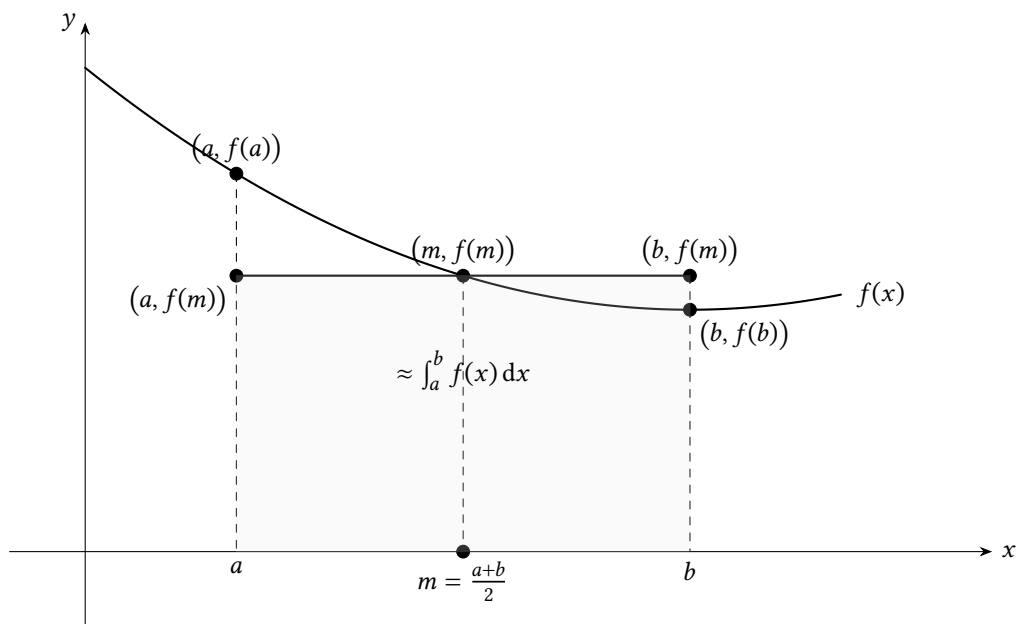


Figure 8.2: Approximation of an integral by the area of a rectangle of height $f(m)$, where $m = \frac{a+b}{2}$ — the midpoint

Chapter 9

Initial Value Problems for Ordinary Differential Equations (ODEs)

9.1 Introduction

Initial value problems (IVPs) for ordinary differential equations (ODEs) involve finding a function that satisfies a given differential equation and meets specified initial conditions. These problems arise in various fields, including physics, engineering, and biology.

The general form of an initial value problem is:

$$y'(t) = f(t, y(t)), \quad y(t_0) = y_0$$

where $y(t)$ is the unknown function, t is the independent variable, y_0 is the initial condition at $t = t_0$, and $f(t, y)$ is a given function.

Remark 9.1 (Scalars and vectors). The same theory covers scalar, vector and even Banach-space valued functions $y(t)$, with small changes, e.g., replacing absolute value with a norm in some arguments. $\square$

9.2 Euler's Method

Euler's method is a simple and intuitive numerical technique for solving IVPs. It approximates the solution by taking small steps along the curve, using the slope provided by the differential equation.

9.2.1 Algorithm

The Euler's method algorithm can be described as follows:

Algorithm 9.1 Euler's Method

Require: A function $f(t, y)$, initial condition $y(t_0) = y_0$, step size h , and number of steps n

Ensure: Approximate values of y at discrete points

- 1: Initialize $t = t_0$ and $y = y_0$
 - 2: **for** $k = 0$ **to** n **do**
 - 3: Compute $y_{k+1} = y_k + hf(t_k, y_k)$
 - 4: Update $t_{k+1} = t_k + h$
 - 5: **end for**
 - 6: **return** y
-

9.2.2 Example

Consider the differential equation:

$$y'(t) = t + y, \quad y(0) = 1$$

Using Euler's method with a step size $h = 0.1$ and $n = 10$ steps, we approximate the solution at discrete points.

The solution of this equation is

$$y(t) = (y(0) + 1)e^t - t - 1 = 2e^t - t - 1.$$

A sample implementation in Python:

```

1  import numpy as np
2
3  n = 10
4  h = .1
5  t = np.zeros(n+1)
6  y = np.zeros(n+1)
7
8  t[0] = 0.0
9  y[0] = 1.0
10
11 def f(t,y):
12     return t+y
13
14 def y_exact(t):
15     return (1+y[0])*np.exp(t)-t-1
16
17 for k in range(0,n):
18     t[k+1] = t[k] + h
19     y[k+1] = y[k] + h*f(t[k],y[k])
20
21 yy = y_exact(t)
22 for t,y,yy in zip(t,y,yy):
23     print(f"{t:.2f} {y:.4f} {yy:.4f}")

```

In Figure 9.1 we compared graphically the numerical approximation to the exact solution. The values rounded to 4 digits are:

$$y(1) = 3.4366, \quad y_{10} = 3.1875.$$

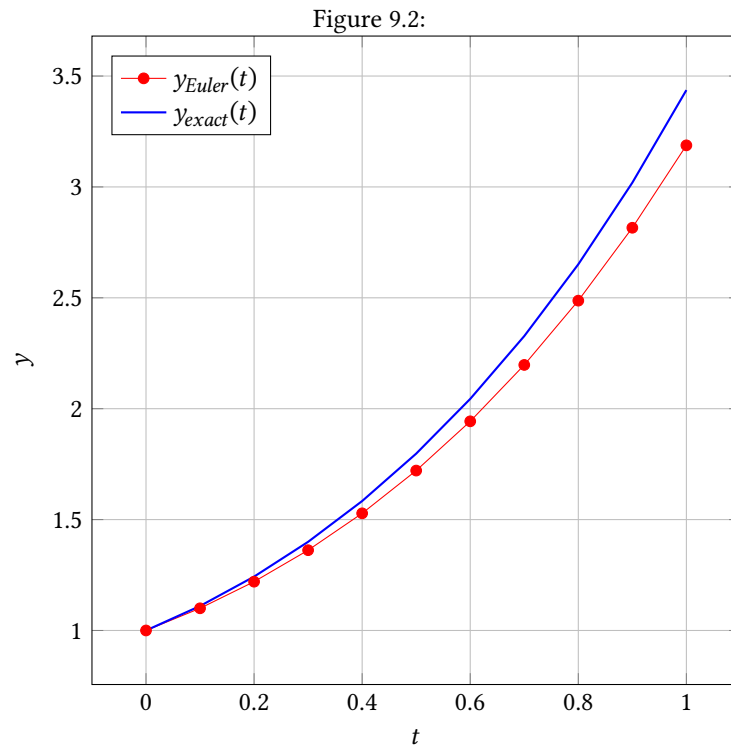
The difference is ≈ 0.25 . This reflects the fact that Euler Method is a method of order 1, i.e., the expected error is $\mathcal{O}(h)$.

Remark 9.2 (Reasons why Euler should not be used). This example shows the accuracy is poor, and this is why Euler Method should not be used. The accuracy can be improved by picking a small h , but at the expense of increasing n and computation time. Additionally, if n is large, round-off error will dominate the truncation error. Thus Euler Method cannot be salvaged. $\square$

9.3 Runge-Kutta Methods

Runge-Kutta methods are a family of iterative methods that provide highly accurate solutions to IVPs. The most commonly used is the fourth-order Runge-Kutta method (RK4).

Figure 9.1: Comparison of Euler Method to exact solution



9.3.1 Fourth-Order Runge-Kutta Method (RK4)

The RK4 method provides a good balance between accuracy and computational efficiency.

Algorithm

The RK4 algorithm can be described as follows:

Example

Consider the differential equation:

$$y'(t) = t + y, \quad y(0) = 1$$

Using the RK4 method with a step size $h = 0.1$ and $n = 10$ steps, we approximate the solution at discrete points.

A sample implementation in Python:

```

1  import numpy as np
2
3  def f(t,y):
4      return t+y
5
6  def y_exact(t):
7      return (1+y[0])*np.exp(t)-t-1
8
9  def rk4(f, t0, y0, h, n):
10     t = np.zeros(n+1)

```

Algorithm 9.2 Fourth-Order Runge-Kutta Method (RK4)

Require: A function $f(t, y)$, initial condition $y(t_0) = y_0$, step size h , and number of steps n

Ensure: Approximate values of y at discrete points

- 1: Initialize $t = t_0$ and $y = y_0$
- 2: **for** $k = 0$ **to** n **do**
- 3: Compute:

$$k_1 = hf(t_k, y_k)$$

$$k_2 = hf\left(t_k + \frac{h}{2}, y_k + \frac{k_1}{2}\right)$$

$$k_3 = hf\left(t_k + \frac{h}{2}, y_k + \frac{k_2}{2}\right)$$

$$k_4 = hf(t_k + h, y_k + k_3)$$

- 4: Update $y_{k+1} = y_k + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$
 - 5: Update $t_{k+1} = t_k + h$
 - 6: **end for**
 - 7: **return** y
-

```
11  y = np.zeros(n+1)
12  t[0] = t0
13  y[0] = y0
14  for k in range(0,n):
15      k1 = h * f(t[k], y[k])
16      k2 = h * f(t[k] + h/2, y[k] + k1/2)
17      k3 = h * f(t[k] + h/2, y[k] + k2/2)
18      k4 = h * f(t[k] + h, y[k] + k3)
19      y[k+1] = y[k] + (1/6)*(k1+2*k2+2*k3+k4)
20      t[k+1] = t[k] + h
21  return t, y
22
23  n = 10
24  h = .1
25  t0 = 0
26  y0 = 1
27  t, y = rk4(f, t0, y0, h, n)
28  yy = y_exact(t)
29
30  for t,y,yy in zip(t,y,yy):
31      print(f"{t:.2f} {y:.6f} {yy:.6f}")
```

There is no point in plotting the exact vs. approximate solution because they coincide within the resolution of the plotter. The values rounded to 6 digits are:

$$y(1) = 3.436564, \quad y_{10} = 3.436559.$$

The difference is 0.00005. This reflects the property of RK4 of being a method of order 4, i.e., the expected error is $\mathcal{O}(h^4)$.

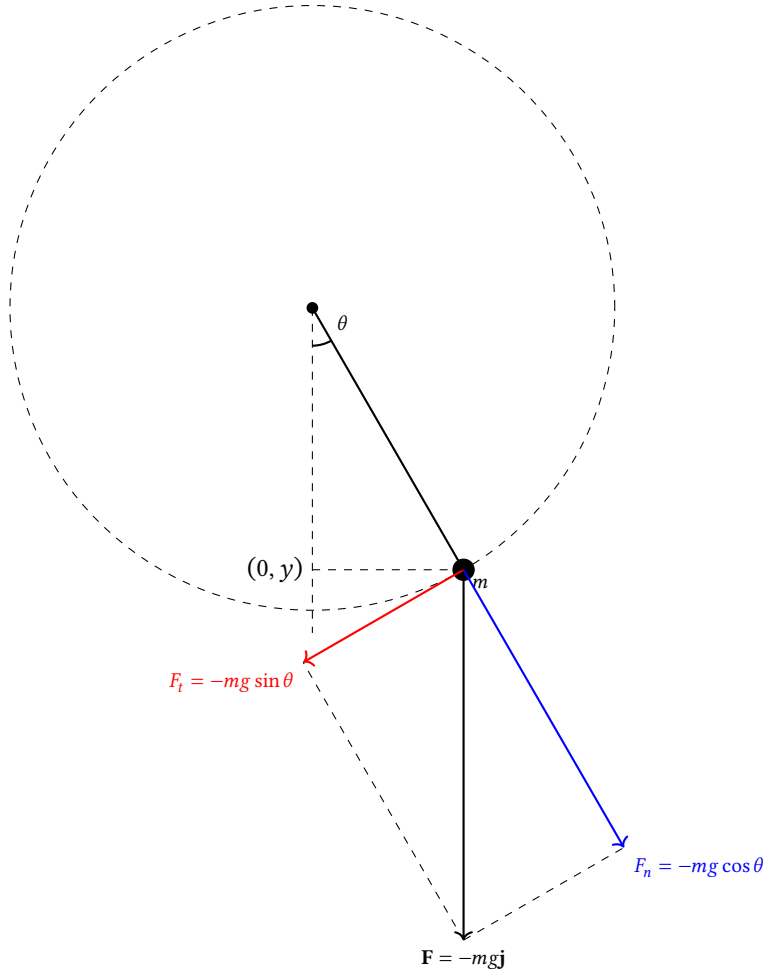


Figure 9.3: The physics of a pendulum

9.3.2 A 2-dimensional example — the pendulum

A pendulum is a device made of a weight suspended from a pivot so that it can swing freely, as shown in Figure 9.3. The second Newton's Law $ma = F$ allows us to derive an ODE describing the pendulum. We assume that the pivot is the origin of the coordinate system. The position of the pendulum is given by one number: the angle the pendulum forms with the vertical direction θ . If ℓ is the length of the pendulum then

$$x = \ell \sin \theta, y = -\ell \cos \theta.$$

Furthermore, the acceleration $a = \ell \ddot{\theta}$, where, in agreement with the conventions used in physics and mathematics, $\dot{\theta} = \frac{d\theta}{dt}$, $\ddot{\theta} = \frac{d^2\theta}{dt^2}$, etc. Therefore, the ODE expressing the equation of motion is:

$$m\ell\ddot{\theta} = F_t$$

where F_t is the component of the gravitational force along the tangent line to the circle to which the pendulum is confined. The normal component F_n is simply balanced by the counterforce of magnitude $-F_n$, exerted by the infinitely stiff, weightless rod connecting the pendulum to the pivot. Building a parallelogram of the forces, and using basic geometry we find that $F_t = -mg \sin \theta$. Therefore, the second Newton's Law yields:

$$m\ell\ddot{\theta} = -mg \sin \theta.$$

As $m > 0$, we simplify the equation to

$$\ddot{\theta} = -\frac{g}{\ell} \sin \theta.$$

By changing units, we may achieve $g/\ell = 1$. Therefore, the standard, dimensionless form of the pendulum equation is:

$$\ddot{\theta} = -\sin \theta. \quad (9.1)$$

Remark 9.3 (On solving pendulum equation in terms of elliptic functions). This equation is known to be *solvable by quadrature*, i.e., by calculating an integral and inverting a function. The integral is known as an *elliptic integral*, and the inverse function is known as an *elliptic function*. $\square$

To solve the pendulum equation (9.1) we rewrite it as a system of 2 first order equations. We introduce time dependent variables:

$$\begin{aligned} y_1 &= \theta, \\ y_2 &= \dot{\theta}. \end{aligned}$$

We have $\dot{y}_1 = y_2$ and

$$\dot{y}_2 = \ddot{\theta} = -\sin \theta = -\sin y_1.$$

Thus, we can write the pendulum equation in vector form:

$$\frac{d}{dt} \mathbf{y} = \mathbf{f}(t, \mathbf{y})$$

where

$$\mathbf{f}(t, \mathbf{y}) = (y_2, -\sin y_1).$$

The following Python code solves an IVP (initial value problem) for this ODE:

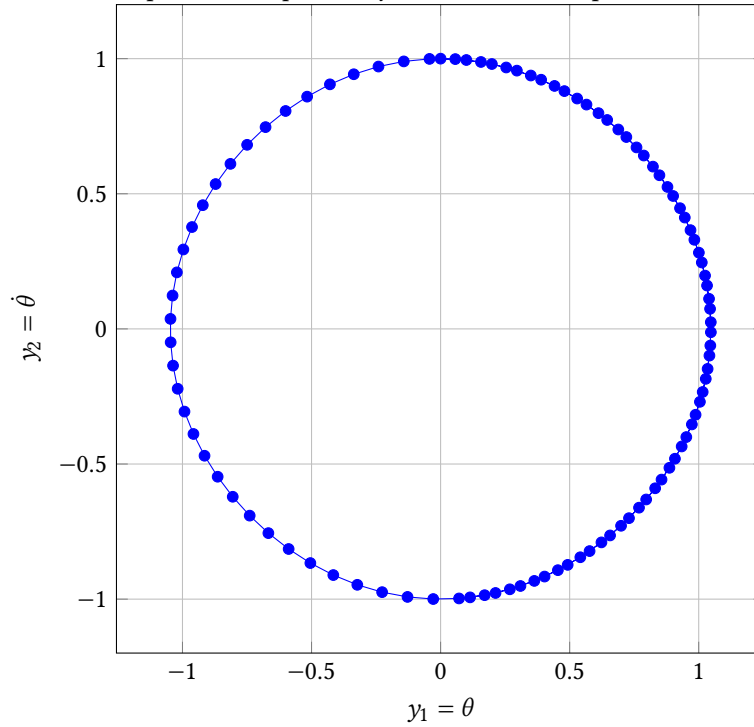
RK4 for Pendulum

```

1 import numpy as np
2
3 def f(t,y):
4     return np.array([y[1], -np.sin(y[0])])
5
6 def rk4(f, t0, y0, h, n):
7     t = np.zeros(n+1)
8     dim = y0.shape[0]
9     y = np.zeros([n+1, dim])
10    t[0] = t0
11    y[0] = y0
12    for k in range(0,n):
13        k1 = h * f(t[k], y[k])
14        k2 = h * f(t[k] + h/2, y[k] + k1/2)
15        k3 = h * f(t[k] + h/2, y[k] + k2/2)
16        k4 = h * f(t[k] + h, y[k] + k3)
17        y[k+1] = y[k] + (1/6)*(k1+2*k2+2*k3+k4)
18        t[k+1] = t[k] + h
19    return t, y
20
21 n = 100
22 h = .1
23 t0 = 0
24 y0 = np.array([0,1])
25 t, y = rk4(f, t0, y0, h, n)
26
27

```

Figure 9.4: A solution to the pendulum equation by RK4 with time step size $h = 0.1$ and $n = 100$ steps



```

28 for t,y in zip(t,y):
29     print(f"{t:.2f} {y[0]:.6f} {y[1]:.6f}")

```

In Figure 9.4 we show a single path in the phase space (time is not shown). The fact that the solution closes indicates good accuracy even with a large step $h = 0.1$.

9.4 Multistep Methods

Multistep methods use multiple past points to estimate the solution at the next step. These methods can be more efficient than single-step methods because they reuse previously computed information.

9.4.1 Adams-Bashforth Method

The Adams-Bashforth methods are explicit multistep methods. The n -step Adams-Bashforth method is given by:

$$y_{k+1} = y_k + h \sum_{j=0}^{n-1} b_j f(t_{k-j}, y_{k-j})$$

where the coefficients b_j depend on the order of the method.

Algorithm

The two-step Adams-Bashforth method can be described as follows:

Algorithm 9.3 Two-Step Adams-Bashforth Method

Require: A function $f(t, y)$, initial condition $y(t_0) = y_0$, initial values y_1 , step size h , and number of steps n

Ensure: Approximate values of y at discrete points

```
1: for  $k = 0$  to  $n-1$  do  
2:   Compute  $y_{k+1} = y_k + \frac{h}{2} (3f(t_k, y_k) - f(t_{k-1}, y_{k-1}))$   
3: end for  
4: return  $y$ 
```

9.4.2 Adams-Moulton Method

The Adams-Moulton methods are implicit multistep methods. The n -step Adams-Moulton method is given by:

$$y_{k+1} = y_k + h \sum_{j=0}^n b_j f(t_{k+1-j}, y_{k+1-j})$$

Algorithm

The one-step Adams-Moulton method (also known as the Trapezoidal Rule) can be described as follows:

Algorithm 9.4 One-Step Adams-Moulton Method (Trapezoidal Rule)

Require: A function $f(t, y)$, initial condition $y(t_0) = y_0$, step size h , and number of steps n

Ensure: Approximate values of y at discrete points

```
1: for  $k = 0$  to  $n-1$  do  
2:   Compute  $y_{k+1}$  by solving the equation:
```

$$y_{k+1} = y_k + \frac{h}{2} (f(t_k, y_k) + f(t_{k+1}, y_{k+1}))$$

```
3: end for  
4: return  $y$ 
```

9.5 Stability and Convergence

Stability and convergence are crucial properties of numerical methods for solving ODEs. A method is stable if errors do not grow exponentially, and it is convergent if the solution approaches the exact solution as the step size h decreases.

9.5.1 Stability

Stability refers to the behavior of numerical errors during the integration process. For a method to be stable, small perturbations in the initial conditions or round-off errors should not cause the solution to diverge significantly from the true solution.

Stability Region

The stability region of a numerical method is the set of complex numbers $z = h\lambda$ for which the method is stable, where λ is an eigenvalue of the Jacobian matrix of the system.

9.5.2 Convergence

A numerical method is convergent if the numerical solution y_h approaches the exact solution y as the step size h tends to zero. Convergence depends on both the consistency and stability of the method.

Lax-Richtmyer Theorem

The Lax-Richtmyer theorem states that for a consistent finite difference method, stability is a necessary and sufficient condition for convergence.

9.5.3 Example: Stability Analysis of Euler's Method

Consider the test equation $y' = \lambda y$, where λ is a complex number. Using Euler's method, the numerical solution is given by:

$$y_{k+1} = y_k + h\lambda y_k = (1 + h\lambda)y_k$$

The method is stable if $|1 + h\lambda| \leq 1$. The stability region for Euler's method is thus the set of $z = h\lambda$ such that $|1 + z| \leq 1$, which corresponds to a disk of radius 1 centered at -1 in the complex plane.

9.6 More on Runge-Kutta methods

9.6.1 Runge-Kutta 2nd Order Method (RK2)

Given the ODE $\frac{dy}{dt} = f(t, y)$ with initial condition $y(t_0) = y_0$, the RK2 method for a single step is:

$$y_{n+1} = y_n + hk_2,$$

where

$$k_1 = f(t_n, y_n),$$

$$k_2 = f\left(t_n + \frac{1}{2}h, y_n + \frac{1}{2}hk_1\right).$$

9.6.2 Runge-Kutta 3rd Order Method (RK3)

Given the ODE $\frac{dy}{dt} = f(t, y)$ with initial condition $y(t_0) = y_0$, the RK3 method for a single step is:

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 4k_2 + k_3),$$

where

$$k_1 = f(t_n, y_n),$$

$$k_2 = f\left(t_n + \frac{1}{2}h, y_n + \frac{1}{2}hk_1\right),$$

$$k_3 = f(t_n + h, y_n - hk_1 + 2hk_2).$$

9.6.3 Runge-Kutta 4th Order Method (RK4)

Given the ODE $\frac{dy}{dt} = f(t, y)$ with initial condition $y(t_0) = y_0$, the RK4 method for a single step is:

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4),$$

where

$$\begin{aligned} k_1 &= f(t_n, y_n), \\ k_2 &= f(t_n + \frac{1}{2}h, y_n + \frac{1}{2}hk_1), \\ k_3 &= f(t_n + \frac{1}{2}h, y_n + \frac{1}{2}hk_2), \\ k_4 &= f(t_n + h, y_n + hk_3). \end{aligned}$$

9.6.4 RK(F)45

Algorithm 9.5 Runge-Kutta-Fehlberg algorithm (RK45 or RKF45) with adaptive steps size

Require: $f(t, y)$, t_0 , y_0 , t_{end} , h , tolerance

Ensure: Approximate values of y at discrete points

```

1:  $t \leftarrow t_0$ 
2:  $y \leftarrow y_0$ 
3: while  $t < t_{\text{end}}$  do
4:    $k_1 \leftarrow h \cdot f(t, y)$ 
5:    $k_2 \leftarrow h \cdot f(t + \frac{1}{4}h, y + \frac{1}{4}k_1)$ 
6:    $k_3 \leftarrow h \cdot f(t + \frac{3}{8}h, y + \frac{3}{32}k_1 + \frac{9}{32}k_2)$ 
7:    $k_4 \leftarrow h \cdot f(t + \frac{12}{13}h, y + \frac{1932}{2197}k_1 - \frac{7200}{2197}k_2 + \frac{7296}{2197}k_3)$ 
8:    $k_5 \leftarrow h \cdot f(t + h, y + \frac{439}{216}k_1 - 8k_2 + \frac{3680}{513}k_3 - \frac{845}{4104}k_4)$ 
9:    $k_6 \leftarrow h \cdot f(t + \frac{1}{2}h, y - \frac{8}{27}k_1 + 2k_2 - \frac{3544}{2565}k_3 + \frac{1859}{4104}k_4 - \frac{11}{40}k_5)$ 
10:   $y_4 \leftarrow y + \frac{25}{216}k_1 + \frac{1408}{2565}k_3 + \frac{2197}{4104}k_4 - \frac{1}{5}k_5$ 
11:   $y_5 \leftarrow y + \frac{16}{135}k_1 + \frac{6656}{12825}k_3 + \frac{28561}{56430}k_4 - \frac{9}{50}k_5 + \frac{2}{55}k_6$ 
12:   $error \leftarrow |y_5 - y_4|$ 
13:  if  $error \leq \text{tolerance}$  then
14:     $t \leftarrow t + h$ 
15:     $y \leftarrow y_5$ 
16:     $h \leftarrow h \cdot \min(2, (\text{tolerance}/error)^{0.2})$ 
17:  else
18:     $h \leftarrow h \cdot \max(0.5, (\text{tolerance}/error)^{0.2})$ 
19:  end if
20:  if  $t + h > t_{\text{end}}$  then
21:     $h \leftarrow t_{\text{end}} - t$ 
22:  end if
23: end while
24: return  $y$ 

```

Remark 9.4 (On the power $0.2 = 1/5$). The theoretically correct power is $0.2 = 1/5$. This is the power used in SciPy and MATLAB. See the explanation in section 9.6.5. Some sources will have power $0.25 = 1/4$. $\square$

9.6.5 How does RK45 work?

There are two ideas in this algorithm worth noting. One is combining two algorithms, RK4 and RK5, of orders 4 and 5 respectively into one algorithm. A careful analysis to identify computations common to both algorithms led to saving computational effort by avoiding re-calculation of certain quantities.

Another idea is the method of varying the step throughout the computation. The objective is to maintain fixed local truncation error (truncation error per step) is close to tolerance. A heuristic is applied, where y_5 (RK5 approximation) is considered infinitely accurate as compared to y_4 (RK4 approximation). The RK45 method is a method of order 5 because of the update rule $y \leftarrow y_5$ selects the more accurate of the two approximations: y_4 and y_5 . Given that $y^* = y(t_n + h)$ for initial condition $y(t_n) = y_n$, we use the following heuristic for the **local truncation error**, by expanding error in a Taylor series in h and keeping only the first term of the series.

$$\begin{aligned} y_5 - y^* &= C_5 h^6, \\ y_4 - y^* &= C_4 h^5. \end{aligned}$$

This leads to

$$\text{error} = |y_5 - y_4| \approx C_4 h^5$$

because $C_5 h^6 \ll C_4 h^5$. Let $C = C_4$. We obtain the equation

$$C h^5 = \text{error}.$$

We estimate C from the current step and adjust the step:

$$C = \frac{\text{error}}{h^5}$$

In the next iteration we change the step to $\tilde{h}$ so that $C \tilde{h}^5 = \text{tolerance}$. This yields

$$\tilde{h} = \left(\frac{\text{tolerance}}{C} \right)^{1/5} = \left(\frac{\text{tolerance}}{C} \right)^{1/5} = \left(\frac{\text{tolerance}}{\text{error}/h^5} \right)^{1/5} = \left(\frac{\text{tolerance}}{\text{error}} \right)^{1/5} h.$$

This update rule for the step is implemented in the algorithm. A small addition is a rule that the step cannot increase by a factor of more than 2, and it cannot decrease by a factor smaller than 1/2.

The method can be generalized to other situations where two methods of different orders are used, e.g., in RK23 and RK56 which are common variations of RK5.

Conservative and Aggressive Choice of the Power

The RK45 method is a method of order 5 because of the update rule $y \leftarrow y_5$ which selects the more accurate of the two approximations: y_4 and y_5 . In general, if we have a pair of methods of order p and $p+1$, respectively, the correct power in the update rule for h is

$$\frac{1}{p+1}$$

to keep the error per step to be approximately constant, equal to the tolerance. In RK45, the power is $1/5 = 0.2$, resulting in increasing the step size and a faster algorithm over the power $\frac{1}{p} = 1/4$, which some implementations use. Note that this power results if we use *global truncation error* rather than local in adjusting the step. If the time of integration is of order 1 then the number of steps required is $1/h$. This requires using tolerance for local truncation error of tolerance $\cdot h$ to achieve global error tolerance. assuming that the errors are additive (which may not be true, when solutions change rapidly!). The crucial equation $C \tilde{h}^{p+1} = \text{tolerance}$ becomes

$$C \tilde{h}^{p+1} = \text{tolerance} \cdot h$$

which simplifies to:

$$C \tilde{h}^p = \text{tolerance}.$$

The remainder of the calculation remains the same, leading to the power $1/p$.

If $p = 4$, the practical impact on the efficiency of the algorithm is minimal.

In RK23, the power is $1/3$. It is important to use $\frac{1}{p+1}$ because of a significant performance impact.

9.6.6 Summary

An adaptive Runge-Kutta method combines two methods of order p and $p + 1$, optimizing calculations so that shared calculations are not duplicated.

The following algorithm excerpt is used to vary the step size:

Algorithm 9.6 Step adaptation in Runge-Kutta methods (excerpt)

```
1:  $error \leftarrow |y_{p+1} - y_p|$ 
2: if  $error \leq \text{tolerance}$  then
3:    $t \leftarrow t + h$ 
4:    $y \leftarrow y_{p+1}$ 
5:    $h \leftarrow h \cdot \min(2, (\text{tolerance}/error)^{\text{power}})$ 
6: else
7:    $h \leftarrow h \cdot \max(0.5, (\text{tolerance}/error)^{\text{power}})$ 
8: end if
```

where y_p is obtained by a method of order p and y_{p+1} is obtained by a method of order $p + 1$. The quantity power in adaptive algorithms depends on the objective:

1. If the objective is to keep truncation error per step constant, equal to the tolerance, then

$$\text{power} = \frac{1}{p + 1}.$$

2. If the objective is to keep the error per unit time equal to the tolerance then

$$\text{power} = \frac{1}{p}.$$

This impacts the interpretation of tolerance. The smaller power $1/(p + 1)$ results in a *larger step*, which may lead to a faster algorithm. On the other hand, controlling error per unit time is easy to relate to application requirements.

9.7 The Taylor Method

The Taylor method for solving ordinary differential equations (ODEs) relies on the Taylor series expansion of the solution. For a given ODE:

$$\frac{dy}{dt} = f(t, y), \quad y(t_0) = y_0$$

we aim to approximate the solution $y(t)$ at a point t_{n+1} using the values at t_n .

The Taylor method is an numerical ODE solving method in its own right, but its value has been mostly theoretical, as it requires calculating higher order derivatives using the Chain Rule. Its theoretical advantage lies in the fact that the method's order is obvious. Thus, it can be used to determine the order of other methods by comparing them with a Taylor method. The examples of this technique will be developed in this section.

9.7.1 Taylor Series Expansion

The Taylor series of $y(t)$ around t_n is given by:

$$y(t) = y(t_n) + (t - t_n)y'(t_n) + \frac{(t - t_n)^2}{2!}y''(t_n) + \frac{(t - t_n)^3}{3!}y'''(t_n) + \dots$$

For a 3rd-order Taylor method, we truncate the series after the $y'''(t_n)$ term.

9.7.2 Deriving the 3rd-Order Taylor Method

Given:

$$\frac{dy}{dt} = f(t, y)$$

The derivatives needed are:

- First derivative: $y' = f(t, y)$
- Second derivative: $y'' = \frac{d}{dt}f(t, y) = f_t + f_y \cdot f$
- Third derivative: $y''' = \frac{d}{dt}y'' = f_{tt} + f_{ty}f + f_y(f_t + f_yf) = f_{tt} + f_{ty}f + f_yf_t + f_yf^2$

where $f_t = \frac{\partial f}{\partial t}$, $f_y = \frac{\partial f}{\partial y}$, etc.

9.7.3 3rd-Order Taylor Method Formula

The formula for the 3rd-order Taylor method is:

$$y(t_{n+1}) = y(t_n) + hf(t_n, y(t_n)) + \frac{h^2}{2}y''(t_n) + \frac{h^3}{3!}y'''(t_n)$$

with $h = t_{n+1} - t_n$, and $y''(t_n)$, $y'''(t_n)$ calculated as derived previously.

9.7.4 Calculating Higher Order Derivatives

The challenge in applying the Taylor method lies in calculating higher-order derivatives of y . For the 3rd order, we explicitly derive y'' and y''' from $f(t, y)$, which can involve complex calculations depending on f .

9.7.5 Conclusion

The 3rd-order Taylor method provides a more accurate approximation than lower-order methods by including more terms from the Taylor series. However, its application depends on the ability to calculate higher-order derivatives of the given ODE.

9.7.6 A higher-order calculation

Below, $f(t, y)$ and its partials have an implied argument: (t, y) ; $y(t)$ and its derivatives have an implied argument (t) . We write d instead of ∂ for the (total) derivative over t .

$$\frac{dy}{dt} = f$$

$$\frac{d^2y}{dt^2} = f \frac{\partial f}{\partial y} + \frac{\partial f}{\partial t}$$

$$\frac{d^3y}{dt^3} = f^2 \frac{\partial^2 f}{\partial y^2} + f \left(\frac{\partial f}{\partial y} \right)^2 + \frac{\partial f}{\partial t} \frac{\partial f}{\partial y} + \frac{\partial^2 f}{\partial t^2} + 2f \frac{\partial^2 f}{\partial t \partial y}$$

etc.

To implement higher-order Taylor methods in practice, we need to be able to perform the Chain Rule automatically, either by *symbolic differentiation* or *automatic differentiation*.

9.7.7 Derivation of Runge-Kutta of order 2

Example 9.1 (Formulas for RK2). Given the ODE $\frac{dy}{dt} = f(t, y)$ with initial condition $y(t_0) = y_0$, the RK2 method for a single step is:

$$y_{n+1} = y_n + hk_2,$$

where

$$\begin{aligned} k_1 &= f(t_n, y_n), \\ k_2 &= f\left(t_n + \frac{1}{2}h, y_n + \frac{1}{2}hk_1\right). \end{aligned}$$

□

- Derive order 2 Taylor method (conventions: all partials of f are evaluated at (t, y)):

$$\begin{aligned} y' &= f(t, y) = f \\ y'' &= f_t + f_y y' = f_t + f_y f \\ y(t+h) &= y + hy' + \frac{h^2}{2}y'' + o(h^2) \\ &= y + hf + \frac{h^2}{2}(f_t + f_y f) \end{aligned}$$

- Propose a difference scheme:

$$\begin{aligned} y(t+h) &= y + hk_2, \\ k_2 &= f(t + \alpha h, y + \beta hk_1), \\ k_1 &= f(t, y) \end{aligned}$$

with constants α, β TBD.

- Expand right-hand side into a Taylor series up to order 2.

$$\begin{aligned} k_2 &= f + f_t \alpha h + f_y \beta h f + o(h) \\ y(t+h) &= y + hf + f_t \alpha h^2 + f_y \beta h^2 f + o(h^2) \\ &= y + hf + h^2(\alpha f_t + \beta f_y f) + o(h^2). \end{aligned}$$

- Match the terms with the Taylor method:

$$\alpha = \beta = \frac{1}{2}$$

- The notation is so efficient that it can handle derivations of higher-order RK methods by hand.
- Matching of coefficients leads to solving systems of non-linear equations.

Remark 9.5 (On uniqueness of RK2, different names). The above argument implies that there exists a unique RK2 method. The potential ambiguity was that different choices of parameters α and β could be made, still resulting in a method of order 2. However, as we have seen, the choice of these parameters is unique. Although the method is unique, it is also known under a different name: the Heun's Method. The method was re-discovered in different contexts. □

9.7.8 A proof that RK2 is not order 3

- The difference scheme:

$$\begin{aligned} y(t+h) &= y + hk_2, \\ k_2 &= f(t + \alpha h, y + \beta hk_1), \\ k_1 &= f(t, y) \end{aligned}$$

- Expand right-hand side into a Taylor series **up to order 3**.

$$\begin{aligned}
 k_2 &= f + f_t \alpha h + f_y \beta h f + \frac{h^2}{2} (\alpha^2 f_{tt} + 2\alpha\beta f_{ty} f + \beta^2 f_{yy} f^2) + o(h^2) \\
 y(t+h) &= y + hf + f_t \alpha h^2 + f_y \beta h^2 f \\
 &\quad + \frac{h^3}{2} (\alpha^2 f_{tt} + 2\alpha\beta f_{ty} f + \beta^2 f_{yy} f^2) + o(h^3)
 \end{aligned}$$

- The h^2 coefficient does match $y'''/3!$ because

$$y''' = f^2 f_{yy} + f f_y^2 + f_{tt} + f_t f_y + 2f f_{ty}$$

- As the match is impossible for all parameter values, there is no order 3 Runge-Kutta method with 2 stages.

9.8 Example: Solving an IVP using the Second-Order Taylor Method

Consider the initial value problem (IVP):

$$\frac{dy}{dx} = y + x, \quad y(0) = 1$$

We will solve this IVP using the second-order Taylor method over the interval $x = 0$ to $x = 1$ with a step size $h = 0.1$.

Taylor Method Derivation

The second-order Taylor method for solving the IVP $\frac{dy}{dx} = f(x, y)$ with initial condition $y(x_0) = y_0$ is given by:

$$y_{n+1} = y_n + hf(x_n, y_n) + \frac{h^2}{2} f'(x_n, y_n)$$

where $f'(x, y)$ is the partial derivative of f with respect to x :

$$f(x, y) = y + x$$

$$f'(x, y) = \frac{\partial f}{\partial x} + \frac{\partial f}{\partial y} \cdot \frac{dy}{dx}$$

Since $\frac{\partial f}{\partial x} = 1$ and $\frac{\partial f}{\partial y} = 1$:

$$f'(x, y) = 1 + 1 \cdot (y + x) = 1 + y + x$$

Step-by-Step Solution

1. **Initial Values:**

$$x_0 = 0, \quad y_0 = 1, \quad h = 0.1$$

2. **Iteration 1:**

$$y_1 = y_0 + hf(x_0, y_0) + \frac{h^2}{2} f'(x_0, y_0)$$

$$f(x_0, y_0) = y_0 + x_0 = 1 + 0 = 1$$

$$f'(x_0, y_0) = 1 + y_0 + x_0 = 1 + 1 + 0 = 2$$

$$y_1 = 1 + 0.1 \cdot 1 + \frac{0.1^2}{2} \cdot 2 = 1 + 0.1 + 0.01 = 1.11$$

3. **Iteration 2:**

$$x_1 = 0.1, \quad y_1 = 1.11$$

$$y_2 = y_1 + hf(x_1, y_1) + \frac{h^2}{2} f'(x_1, y_1)$$

$$f(x_1, y_1) = y_1 + x_1 = 1.11 + 0.1 = 1.21$$

$$f'(x_1, y_1) = 1 + y_1 + x_1 = 1 + 1.11 + 0.1 = 2.21$$

$$y_2 = 1.11 + 0.1 \cdot 1.21 + \frac{0.1^2}{2} \cdot 2.21 = 1.11 + 0.121 + 0.01105 = 1.24205$$

4. **Iteration 3:**

$$x_2 = 0.2, \quad y_2 = 1.24205$$

$$y_3 = y_2 + hf(x_2, y_2) + \frac{h^2}{2} f'(x_2, y_2)$$

$$f(x_2, y_2) = y_2 + x_2 = 1.24205 + 0.2 = 1.44205$$

$$f'(x_2, y_2) = 1 + y_2 + x_2 = 1 + 1.24205 + 0.2 = 2.44205$$

$$y_3 = 1.24205 + 0.1 \cdot 1.44205 + \frac{0.1^2}{2} \cdot 2.44205 = 1.24205 + 0.144205 + 0.01221025 = 1.39846525$$

Continue this process for more iterations until $x = 1$.

Result

After completing the iterations, you will have the approximate values of y at each step. These values can be compared with the exact solution to assess the accuracy of the method.

Comparison with Exact Solution

The exact solution to the differential equation $\frac{dy}{dx} = y + x$ with the initial condition $y(0) = 1$ is given by:

$$y(x) = 2e^x - x - 1$$

You can compare the numerical solution obtained using the Taylor method with the exact solution at each step to evaluate the accuracy.

9.9 The General Form of Explicit Runge-Kutta Methods

The family of explicit Runge-Kutta methods generalizes the RK4 method, and is given by:

$$y_{n+1} = y_n + h \sum_{i=1}^s b_i k_i,$$

where each k_i is defined as follows (the “Stages” of Explicit Runge-Kutta Methods):

$$k_1 = f(t_n, y_n),$$

$$k_2 = f(t_n + c_2 h, y_n + a_{21} k_1 h),$$

$$k_3 = f(t_n + c_3 h, y_n + (a_{31} k_1 + a_{32} k_2) h),$$

$$\vdots$$

$$k_s = f(t_n + c_s h, y_n + (a_{s1} k_1 + a_{s2} k_2 + \cdots + a_{s,s-1} k_{s-1}) h).$$

9.10 Equations of Motion for a Pendulum with an Elastic Rod

This example provides a more involved example of a mechanical system. We also use the apparatus of Lagrangian mechanics to derive the equations of motion.

9.10.1 System Description

Consider a pendulum consisting of a point mass m attached to a rod of variable length r . Unlike a traditional rigid pendulum, the rod behaves as a linear elastic spring with natural length L_0 and spring constant k . The system is subject to gravitational acceleration g and moves in a plane.

We define two generalized coordinates:

- r : the instantaneous length of the rod, which can stretch and contract.
- θ : the angular displacement of the mass from the downward vertical.

The Cartesian coordinates of the mass are given by

$$x = r \sin \theta, \quad y = -r \cos \theta.$$

9.10.2 Kinetic Energy

The kinetic energy consists of contributions from both radial and tangential motion:

$$T = \frac{1}{2}m(\dot{x}^2 + \dot{y}^2).$$

Computing the time derivatives,

$$\begin{aligned}\dot{x} &= \dot{r} \sin \theta + r \dot{\theta} \cos \theta, \\ \dot{y} &= \dot{r} \cos \theta - r \dot{\theta} \sin \theta.\end{aligned}$$

Squaring and summing,

$$T = \frac{1}{2}m(\dot{r}^2 + r^2\dot{\theta}^2).$$

9.10.3 Potential Energy

The system has two potential energy contributions:

1. **Gravitational potential energy:**

$$U_g = mgy = -mgr \cos \theta. \quad (9.2)$$

2. **Elastic potential energy** due to the stretching of the rod:

$$U_s = \frac{1}{2}k(r - L_0)^2 + \frac{\epsilon}{r}. \quad (9.3)$$

We note that we added a gravity or electric-like potential repelling the point mass from the pivot for $\epsilon > 0$. This is to avoid the point mass going through the pivot. This modification avoids singularity at $r = 0$.

Thus, the total potential energy is given by

$$U = -mgr \cos \theta + \frac{1}{2}k(r - L_0)^2 + \frac{\epsilon}{r}.$$

9.10.4 Lagrangian and Equations of Motion

The Lagrangian of the system is

$$L = T - U = \frac{1}{2}m(\dot{r}^2 + r^2\dot{\theta}^2) + mgr \cos \theta - \frac{1}{2}k(r - L_0)^2 - \frac{\epsilon}{r}.$$

Applying the Euler-Lagrange equations to r and θ :

Radial Equation:

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{r}} \right) - \frac{\partial L}{\partial r} = 0.$$

Computing derivatives,

$$m\ddot{r} = mr\dot{\theta}^2 + mg \cos \theta - k(r - L_0) + \frac{\epsilon}{r^2}.$$

Angular Equation:

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{\theta}} \right) - \frac{\partial L}{\partial \theta} = 0.$$

Computing derivatives,

$$\begin{aligned} \frac{d}{dt}(mr^2\dot{\theta}) &= -mgr \sin \theta, \\ mr^2\ddot{\theta} + 2mr\dot{r}\dot{\theta} &= -mgr \sin \theta, \\ \ddot{\theta} + 2\frac{\dot{r}\dot{\theta}}{r} &= -\frac{g}{r} \sin \theta. \end{aligned}$$

9.10.5 Final Equations of Motion

The coupled equations governing the motion of the system are:

$$\begin{aligned} \ddot{r} &= r\dot{\theta}^2 - g \cos \theta - \frac{k}{m}(r - L_0) + \frac{\epsilon}{r^2}, \\ \ddot{\theta} &= -2\frac{\dot{r}\dot{\theta}}{r} - \frac{g}{r} \sin \theta. \end{aligned} \tag{9.4}$$

These equations describe the nonlinear dynamics of the pendulum with an elastic rod, incorporating both radial and angular motion.

9.10.6 Solving an IVP — vectorized version

_____ Elastic pendulum solver with multiple initial conditions _____

```
1 import numpy as np
2 from scipy.integrate import solve_ivp
3 import matplotlib.pyplot as plt
4
5
6 def elastic_pendulum(t, Y, g, k, m, L0, epsilon):
7     r, theta, dr, dtheta = Y.reshape(4, -1)
8     d2r = r * dtheta**2 + g * np.cos(theta) - (k/m) * (r - L0) + epsilon / r**2
9     d2theta = -2 * dr * dtheta / r - (g / r) * np.sin(theta)
10    dY = np.array([dr, dtheta, d2r, d2theta])
11    # NOTE: Flattening appears optional
12    # dY = dY.flatten()
13    return dY
14
15 def energy(Y, n_ic, g, k, m, L0, epsilon):
16     r, theta, dr, dtheta = Y.reshape(4, n_ic, -1)
17     T = 0.5 * m * (dr**2 + r**2 * dtheta**2)
18     U = -m * g * r * np.cos(theta) + 0.5 * k * (r - L0)**2 + epsilon / r
19     E = T + U
20     return E
21
22
```

```

23 g, k, m, L0, epsilon = 9.81, 1.0, 1.0, 10.0, -1
24
25 initial_conditions = np.array([
26     [1.0, np.pi/6, 0.0, 0.0],
27     [1.2, np.pi/4, 0.0, 0.0],
28     [0.8, np.pi/3, 0.0, 0.0],
29     [1.1, np.pi/2, 0.0, -0.5],
30 ]).transpose()
31
32 # Check if dimensions are consistent for vectorized solver
33 assert elastic_pendulum(1, initial_conditions, g, k, m, L0, epsilon).shape \
34     == initial_conditions.shape
35
36 tmax = 100
37 n_eval = 1000
38 t_span, t_eval = (0, tmax), np.linspace(0, tmax, n_eval)
39 sol = solve_ivp(elastic_pendulum,
40                 t_span,
41                 initial_conditions.flatten(),
42                 args=(g, k, m, L0, epsilon),
43                 t_eval=t_eval,
44                 method='RK45',
45                 atol=1e-12,
46                 rtol=1e-12,
47                 vectorized=True)
48
49 fig, axes = plt.subplots(2, 1, figsize=(10, 6), sharex=True)
50 labels = ["Case 1", "Case 2", "Case 3", "Case 4"]
51
52 n_ic = initial_conditions.shape[1]
53 Y = sol.y.reshape(4, n_ic, -1)
54
55 E = energy(Y, n_ic, g, k, m, L0, epsilon)
56 for i in range(n_ic):
57     assert np.max(abs(E[i] - E[i][0])) < 1e-8
58
59 for j in range(n_ic):
60     axes[0].plot(sol.t, Y[0][j])
61     axes[1].plot(sol.t, Y[1][j])
62
63 axes[0].set_xlabel("Time (s)")
64 axes[0].set_ylabel("Radial Distance r (m)")
65 axes[1].set_xlabel("Time (s)")
66 axes[1].set_ylabel("Angle  $\theta$  (rad)")
67
68 axes[0].grid()
69 axes[1].grid()
70 plt.suptitle(
71     "Elastic Pendulum Motion for Different Initial Conditions (Vectorized)"
72 )
73 # plt.show()
74 plt.savefig("figures/generated/elastic_pendulum_vectorized.png", dpi=300, bbox_inches='tight')

```

The reader will kindly observe our care to checking that the total energy remains nearly constant with small tolerance throughout the integration. Such care is necessary when using general purpose algorithms which do not enforce the motion to stay on the hypersurface of constant energy.

The plot of the solutions produced by the code above are presented in Figure 9.5. The behavior of the solutions is much more interesting than that of the ordinary pendulum. For instance, the solution may be *aperiodic* or *chaotic*. In fact, aperiodicity and chaotic behavior is likely to occur in typical mechanical systems with two or more degrees of freedom, even if the system is conservative, such as a system obtained from the Lagrangian formalism presented in this section.

Vectorization

The code presented in this section is geared towards solving an IVP with a large number of initial conditions simultaneously. The code will perform by orders of magnitude better than solving an IVP with each of the initial conditions separately. This is not due to *multiprocessing*, i.e., use of multiple CPU working in parallel, but to *vectorization*. Vectorized code utilizes the *superscalar architecture* of a single CPU present in all modern CPUs. The arithmetic is done in an assembly-line like fashion, where several *pipelines* perform various phases of time-consuming operations, such as multiplication. Each multiplication is divided into multiple steps which can be performed in succession. The steps are distributed among typically four pipelines (think assembly lines in a car factory). Thus, a single multiplication may take 16 or 32 CPU cycles, but due to superscalar architecture is able to perform many multiplications per CPU cycle. Another form of speed up comes from the use of *vector registers*, i.e. chunks of memory in CPU which can hold, e.g., 16 floating point numbers, and perform operations in parallel. For instance, the calculation

```
for i from 1 to n do
     $z_i \leftarrow x_i \times y_i$ 
end for
```

can be performed in one CPU clock cycle, given that the vector registers x and y can hold vectors with n entries. A typical value of n could be 16 or 32. The Python ODE solver with option `vectorized=True` will map well to superscalar architecture if coded skillfully, producing impressive speedups.

A word of caution when vectorizing an ODE

The vectorized implementation of the function `elastic_pendulum` will be written differently for `vectorized=False` (default) and `vectorized=True` options. In the former case, the coordinates of the vector in phase space are stored in a row. In the latter case, we must prepare to handle a matrix with several vectors representing points in the phase space, placed *in columns* of the matrix. The code must produce a matrix of the same shape containing the derivatives.

9.10.7 References

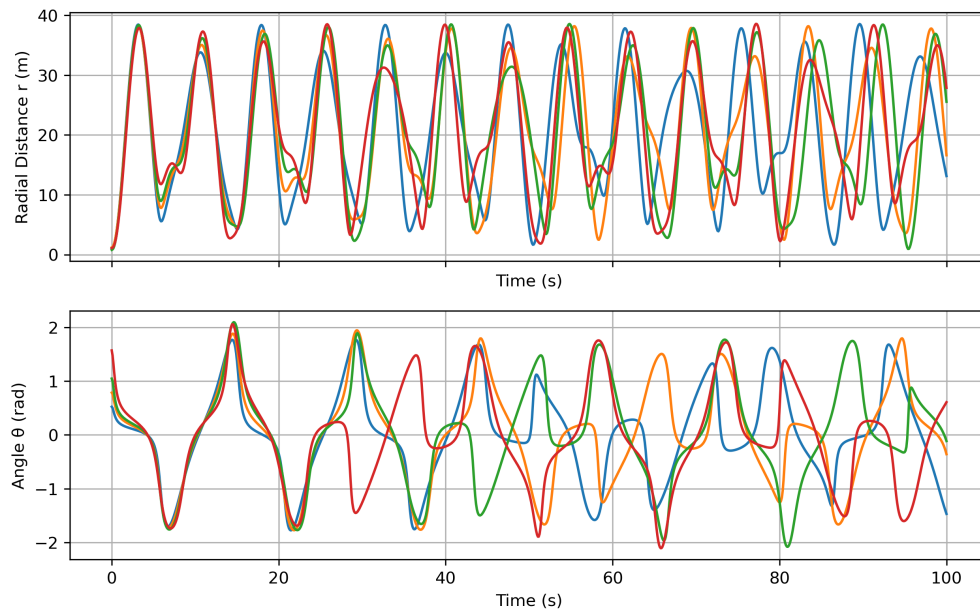
The systematic approach to mechanics based on Lagrangian formalism was introduced in the classical book by Landau and Lifshitz [8, 7].

9.11 The Lorenz System and Its Poincaré Section

The Lorenz system is a well-known example of a chaotic dynamical system. It consists of three coupled nonlinear differential equations:

$$\begin{aligned}\dot{x} &= \sigma(y - x), \\ \dot{y} &= x(\rho - z) - y, \\ \dot{z} &= xy - \beta z.\end{aligned}\tag{9.5}$$

Figure 9.5: Several solutions to the Elastic Pendulum ODE
Elastic Pendulum Motion for Different Initial Conditions (Vectorized)



where the parameters σ , ρ , and β control the system's behavior. For the classical values $\sigma = 10$, $\rho = 28$, and $\beta = 8/3$, the system exhibits chaotic dynamics.

9.11.1 Trajectory of the Lorenz System

Before analyzing the Poincaré section, we visualize a single long trajectory of the Lorenz system. The system is integrated over a long time period to show the evolution of the attractor in three-dimensional space. The trajectory exhibits the hallmark chaotic butterfly-like shape of the Lorenz attractor.

The following Python code generates the trajectory plot:

Trajectory of the Lorenz System

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  from scipy.integrate import solve_ivp
4
5  # Define the Lorenz system
6  def lorenz(t, state, sigma, rho, beta):
7      x, y, z = state
8      dxdt = sigma * (y - x)
9      dydt = x * (rho - z) - y
10     dzdt = x * y - beta * z
11     return [dxdt, dydt, dzdt]
12
13 # Define parameters
14 sigma = 10.0
15 rho = 28.0
16 beta = 8.0 / 3.0
17

```

```

18 # Initial conditions
19 initial_state = [1.0, 1.0, 1.0]
20
21 # Time span
22 t_span = (0, 100)
23 t_eval = np.linspace(*t_span, 10000)
24
25 # Solve the system
26 sol = solve_ivp(lorenz, t_span, initial_state, args=(sigma, rho, beta), t_eval=t_eval)
27
28 # Extract solution
29 x, y, z = sol.y
30
31 # Plot the Lorenz attractor
32 fig = plt.figure(figsize=(8, 6))
33 ax = fig.add_subplot(111, projection="3d")
34 ax.plot(x, y, z, lw=0.5, color="blue")
35 ax.set_xlabel("x")
36 ax.set_ylabel("y")
37 ax.set_zlabel("z")
38 ax.set_title("Lorenz Attractor")
39 plt.savefig("figures/generated/lorenz_trajectory.png", dpi=300, bbox_inches="tight")
40 plt.close()

```

The following figure illustrates the long trajectory of the system:

9.11.2 Computing the Poincaré Section

A Poincaré section is a useful tool for analyzing chaotic systems. It involves selecting a plane in phase space and recording the points where the system's trajectory intersects it. Here, we choose the plane $z = 20$ with $\dot{z} > 0$.

To compute the Poincaré section, we perform the following steps:

1. Simulate the Lorenz system using numerical integration.
2. Use event detection to find trajectory crossings.
3. Record points where the trajectory crosses the plane.
4. Plot the collected points to reveal the structure of the attractor.

In general, constructing the Poincaré section for a system

$$\dot{y} = f(y), \quad y \in \mathbb{R}^n$$

starts with picking a hypersurface (a submanifold of codimension one) in the phase space.

In our case of the Lorenz system the state space is $M = \mathbb{R}^3$. The hypersurface is $S = \{z = 20\} \subseteq M$.

The Poincaré section is a transformation $P : S \rightarrow S$ defined by the following properties:

1. For every $y_0 \in S$ there is a solution $y(t; y_0)$ to the IVP with initial condition y_0 hits $P(y_0)$ after some time $t = t(y_0) > 0$: $y(t(y_0); y_0) = P(y_0) \in S$.
2. For $t' \in (0, t(y_0))$ said solution is not in S , i.e., $y(t'; y_0) \notin S$.

Justifiably, the function $t(y_0)$ is called the *first return time* to S . The map P can be iterated. The points $P^n(y_0) \in S$ are all on the trajectory $x(t; x_0)$ and represent the consecutive visits of the trajectory to S .

ODE solvers, such as `solve_ivp` in Python, implement an *event interface*, which allows one to compute the times and values of the trajectory visits S . Below we provide the details.

Lorenz Attractor

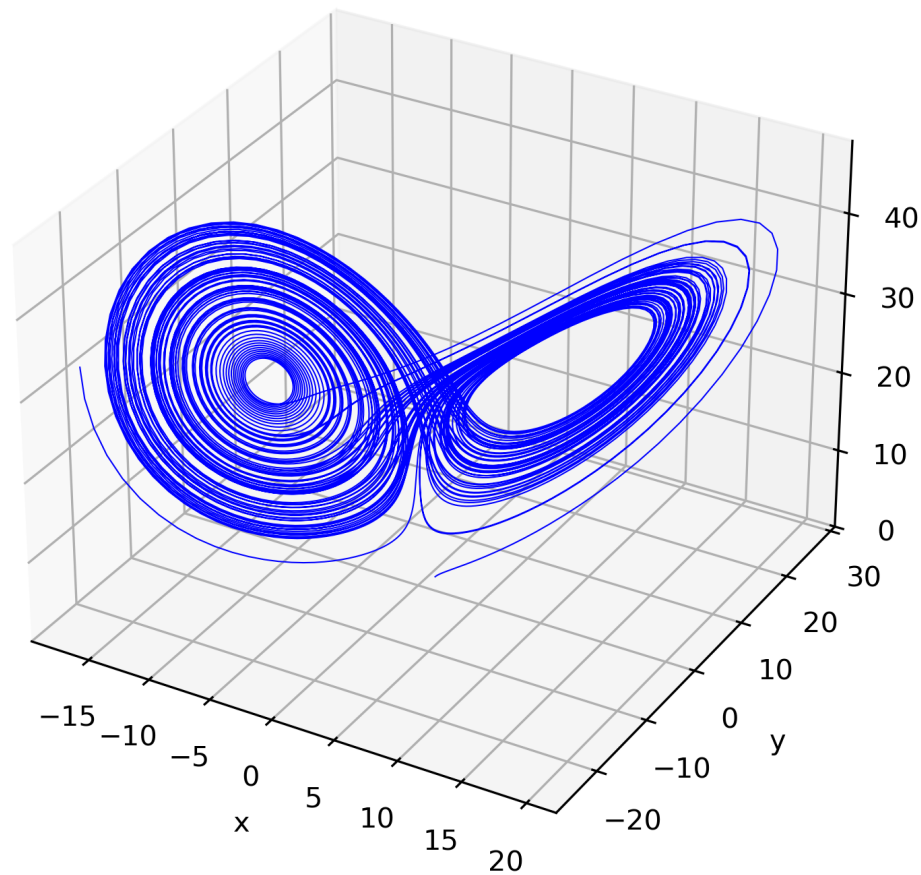


Figure 9.6: Long trajectory of the Lorenz system visualized in three dimensions.

9.11.3 Simulation and Results

The following Python code integrates the Lorenz system and extracts the Poincaré section using the event mechanism of `solve_ivp`:

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  from scipy.integrate import solve_ivp
4
5  # Define the Lorenz system
6  def lorenz(t, state, sigma, rho, beta):
7      x, y, z = state
8      dxdt = sigma * (y - x)
9      dydt = x * (rho - z) - y
10     dzdt = x * y - beta * z
11     return [dxdt, dydt, dzdt]
12
13     Poincaré Section of the Lorenz System

```

```

13 # Define parameters
14 sigma = 10.0
15 rho = 28.0
16 beta = 8.0 / 3.0
17
18 # Initial conditions
19 initial_state = [1.0, 1.0, 1.0]
20
21 # Time span
22 t_span = (0, 100)
23 t_eval = np.linspace(*t_span, 10000)
24
25 # Define an event function for Poincaré section (z = 20, crossing upwards)
26 def poincare_event(t, state, sigma, rho, beta):
27     return state[2] - 20 # z - 20
28
29 # Ensure the solver detects only crossings where dz/dt > 0
30 poincare_event.terminal = False # Do not stop integration at crossings
31 poincare_event.direction = 1 # Detect only upward crossings
32
33 # Solve the system with event detection
34 sol_events = solve_ivp(lorenz,
35                         t_span,
36                         initial_state,
37                         args=(sigma, rho, beta),
38                         t_eval=t_eval,
39                         events=poincare_event)
40
41 # Extract Poincaré section points
42 poincare_x = sol_events.y_events[0][:, 0]
43 poincare_y = sol_events.y_events[0][:, 1]
44
45 # Plot improved Poincaré section
46 plt.figure(figsize=(8, 6))
47 plt.scatter(poincare_x, poincare_y, s=1, color='blue', alpha=0.7)
48 plt.xlabel("x")
49 plt.ylabel("y")
50 plt.title("Poincaré Section of the Lorenz System (z = 20, dz/dt > 0)")
51 plt.grid(True)
52 plt.savefig("figures/generated/lorenz_poincare_event.png", dpi=300, bbox_inches='tight')
53 plt.close()

```

The resulting Poincaré section is shown in Figure 9.7.

9.12 Calculating Lyapunov Exponents

9.12.1 Introduction

Lyapunov exponents quantify the sensitivity of a dynamical system to initial conditions. They characterize the rate at which nearby trajectories converge or diverge, making them crucial for understanding chaotic behavior. This section outlines three methods for computing Lyapunov exponents:

1. The method based on the first variation equations for a system of the form $\dot{y} = f(y)$.

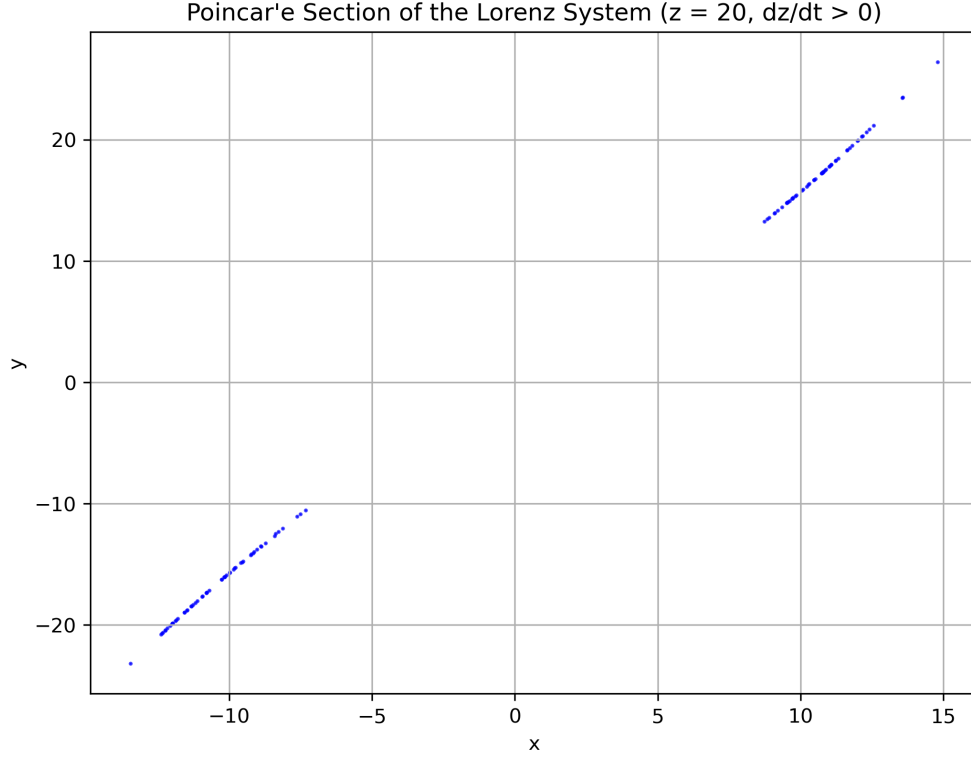


Figure 9.7: Poincaré section of the Lorenz system at $z = 20$.

2. Computing the Lyapunov exponent of a Poincaré section map (first return map to a hypersurface).
3. A direct method by studying the rate at which two nearby trajectories separate.

9.12.2 First Variation Equations Method

Consider a system of ordinary differential equations:

$$\dot{y} = f(y), \quad y \in \mathbb{R}^n.$$

To analyze the divergence of nearby trajectories, we introduce a small perturbation $\delta y(t)$ and linearize the system around a reference trajectory $y_0(t)$:

$$\dot{\delta y} = Df(y)\delta y,$$

where $Df(y)$ is the Jacobian matrix evaluated along the trajectory. We typically integrate both equations in tandem, by considering the system:

$$\begin{cases} \dot{y} = f(y), & y \in \mathbb{R}^n, \\ \dot{\delta y} = Df(y)\delta y, & \delta y \in \mathbb{R}^n. \end{cases}$$

where the state of the system is the vector $(y, \delta y) \in \mathbb{R}^{2n}$.

The largest Lyapunov exponent λ is given by:

$$\lambda = \lim_{t \rightarrow \infty} \frac{1}{t} \ln \frac{\|\delta y(t)\|}{\|\delta y(0)\|}.$$

In many situations relevant in applications there is no dependency on $y(0)$ or on $\delta y(0)$. One notable case is the Lorenz system, where the Lyapunov exponent is almost always the same, i.e., it is the same with the exception of a set of Lebesgue measure 0 of initial conditions.

This method involves integrating both the original system and the variational equations.

9.12.3 Poincaré Section Method

Instead of analyzing continuous-time trajectories, we can study the discrete-time evolution of a Poincaré map, which records the first return of trajectories to a hypersurface in phase space. If $P : S \rightarrow S$ is the Poincaré map defined on a hypersurface S , the Lyapunov exponent is computed using:

$$\lambda_P = \lim_{n \rightarrow \infty} \frac{1}{n} \ln \|DP^n(y_0)\|, \quad y_0 \in S.$$

Again, the value of the Lyapunov exponent tends to be independent of $y_0 \in S$. It should be noted that $\lambda_P \neq \lambda$. However, $\text{sgn}(\lambda_P) = \text{sgn}(\lambda)$. The ratio

$$\frac{\lambda}{\lambda_P} = \text{the average return time to } S.$$

This method provides insight into chaotic structures in reduced phase-space representations, i.e., after taking into account conserved quantities (first integrals).

9.12.4 Direct Method: Trajectory Separation

A simple yet effective approach involves simulating two nearby initial conditions and measuring their divergence over time. Given two trajectories $y_1(t)$ and $y_2(t)$, the Lyapunov exponent is estimated as:

$$\lambda = \lim_{t \rightarrow \infty} \frac{1}{t} \ln \frac{\|y_2(t) - y_1(t)\|}{\|y_2(0) - y_1(0)\|}.$$

To prevent divergence beyond the linear regime, the perturbed trajectory is periodically rescaled while maintaining the direction of separation.

9.12.5 Example: The Lyapunov Exponent of the Lorenz System

We illustrate the method based on the first variation equations of the Lorenz system (9.5):

$$\begin{cases} \frac{d}{dt}x = \sigma(y - x), \\ \frac{d}{dt}y = x(\rho - z) - y, \\ \frac{d}{dt}z = xy - \beta z, \\ \frac{d}{dt}\delta x = \sigma(\delta y - \delta x), \\ \frac{d}{dt}\delta y = \delta x(\rho - z) + x(-\delta z) - \delta y, \\ \frac{d}{dt}\delta z = \delta x y + x \delta y - \beta \delta z. \end{cases} \quad (9.6)$$

The mechanics of deriving these equations can be easily stated. The first three equations simply reproduce the Lorenz system. The last three equations are obtained by applying an operator δ to the first three equations. This operator has these two properties:

- It behaves like a differential operator, e.g., it is linear and it satisfies the product and quotient rules;
- It commutes with $\frac{d}{dt}$, e.g.,

$$\delta \frac{d}{dt}x = \frac{d}{dt}\delta x.$$

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  from scipy.integrate import solve_ivp
4
5
6  # Define the Lorenz system
7  def lorenz_var(t, extState, sigma, rho, beta):
8      x, y, z, deltaX, deltaY, deltaZ = extState
9      dxdt = sigma * (y - x)
10     dydt = x * (rho - z) - y
11     dzdt = x * y - beta * z
12     dDeltaXdt = sigma * (deltaY - deltaX)
13     dDeltaYdt = deltaX * (rho - z) + x * (-deltaZ) - deltaY
14     dDeltaZdt = deltaX * y + x * deltaY - beta * deltaZ
15     return [dxdt, dydt, dzdt, dDeltaXdt, dDeltaYdt, dDeltaZdt]
16
17
18 # Define parameters
19 sigma = 10.0
20 rho = 28.0
21 beta = 8.0 / 3.0
22
23 # Initial conditions
24 initial_state = [1.0, -1.0, 1.0, 0.0, 1.0, 0.0]
25
26 # Time span
27 t_span = (0, 300)
28 t_eval = np.linspace(*t_span, 10000)
29
30 # Solve the system with event detection
31 sol = solve_ivp(lorenz_var, t_span, initial_state, args=(sigma, rho, beta),
32                 atol=1e-6,
33                 rtol=1e-6,
34                 method='RK45',
35                 t_eval=t_eval)
36
37 t = sol.t
38 y = sol.y[:, 3, :]
39 deltaY = sol.y[3:, :]
40
41 logNormDeltaY = [
42     1/t[j]*np.log(
43         np.linalg.norm(deltaY[:, j]) /
44         np.linalg.norm(deltaY[:, 0])
45     )
46     for j in range(1, y.shape[1])
47 ]
48
49 lyapunov_exponent = logNormDeltaY[-1]
50
51 # Enable LaTeX rendering
52 plt.rcParams['text.usetex'] = True

```

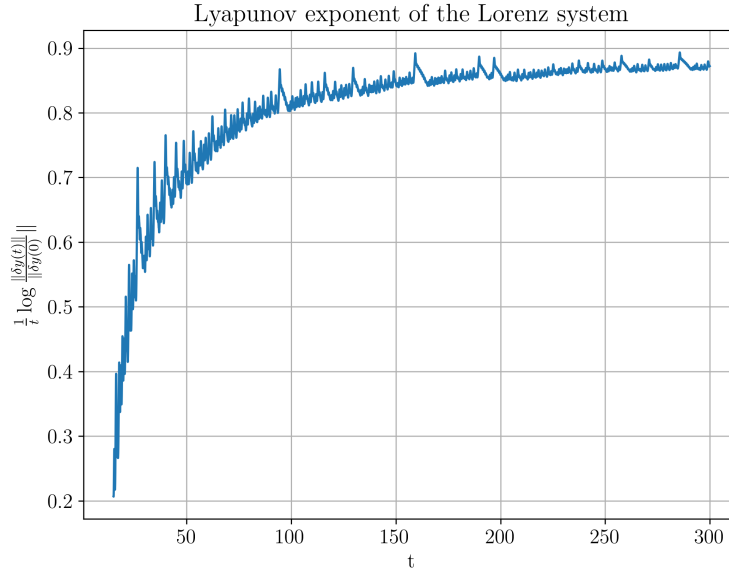


Figure 9.8: Estimating the Lyapunov exponent for the Lorenz system

```

53 plt.rcParams['font.family'] = 'serif'
54 plt.rcParams['font.size'] = '14'
55
56 burnin = 500
57
58 # Plot improved Poincaré section
59 plt.figure(figsize=(8, 6))
60 plt.plot(t[burnin+1:], logNormDeltaY[burnin:])
61 plt.xlabel(r"$t$")
62 plt.ylabel(r"$\frac{1}{t} \log \frac{\|\delta y(t)\|}{\|\delta y(0)\|}$")
63 plt.title(r"Lyapunov exponent of the Lorenz system")
64 plt.grid(True)
65 # plt.show()
66 plt.savefig("figures/generated/lorenz_lyapunov_exponent.png", dpi=300, bbox_inches='tight')
67 plt.close()

```

The plot is presented in Figure 9.8. As we can see, the value stabilizes around 0.8722272306453193. The value reported in literature is 0.906.

9.12.6 Computing the Derivative of the Poincaré Map

The derivative of the Poincaré map $P : S \rightarrow S$ is of interest. For example, we can ask to find the value of its Lyapunov exponent. The method to find the derivative consists in integrating the first variation system, up to the first return time $T = (y_0)$, and projecting $\delta y(t(y_0))$ onto the tangent space to S at $y_1 = P(y_0)$. Below is a detailed algorithm.

Consider the system

$$\dot{y}(t) = f(y(t)), \quad y(t) \in \mathbb{R}^n,$$

and a smooth Poincaré section S , defined implicitly by a scalar function $s : \mathbb{R}^n \rightarrow \mathbb{R}$, such that $S = \{y \mid s(y) = 0\}$. The Poincaré map P maps a point $y_0 \in S$ to its next intersection with S . The formula for the Lyapunov exponent of P is then

$$\lambda_P = \lim_{n \rightarrow \infty} \frac{1}{n} \log \|DP^n(y_0)\|.$$

To find the derivative $DP(y_0)$, follow Algorithm 9.7.

Algorithm 9.7 Compute derivative of the Poincaré map at y_0

Require: A vector field f , a Poincaré section S , and a point $y_0 \in S$.

Ensure: Derivative $DP(y_0)$ of the Poincaré map at y_0 .

- 1: Integrate $\dot{y}(t) = f(y(t))$ with initial condition $y(0) = y_0$ until intersection at $y(T) \in S$.
- 2: Integrate the variational equation

$$\dot{Y}(t) = Df(y(t))Y(t), \quad Y(0) = I_n,$$

up to $t = T$, obtaining $Y(T)$.

- 3: Compute the vectors $v = \nabla s(y(T))$ and $w = f(y(T))$.
- 4: Compute

$$DP(y_0) = \left(I_n - \frac{wv^\top}{v^\top w} \right) Y(T),$$

where $v = \nabla s(y(T))$.

Ensure: The Jacobian matrix $DP(y_0)$.

9.12.7 Conclusion

Each of these methods provides a different perspective on computing Lyapunov exponents, and their effectiveness depends on the system under study. The variational method is well-suited for continuous dynamical systems, the Poincaré section method is useful for discrete maps, and the direct trajectory separation method offers an intuitive approach suitable for numerical simulations.

9.13 Step Size Restriction for Convergence of RK4 on a Stiff System

Consider the initial value problem

$$\dot{x} = -x, \quad \dot{y} = -10y,$$

with random initial conditions. The exact solution is

$$x(t) = x_0 e^{-t}, \quad y(t) = y_0 e^{-10t},$$

which decays to zero as $t \rightarrow \infty$. However, the decay rates differ by an order of magnitude, suggesting that the system is stiff. We analyze how the step size h in the classical Runge-Kutta method of order 4 (RK4) must be chosen to ensure the numerical solution also decays to zero.

Stability of RK4 on Linear Equations

For a scalar linear ODE $\dot{u} = \lambda u$, the RK4 update is

$$u_{n+1} = R(h\lambda)u_n,$$

where the stability function is given by the degree-4 Taylor approximation of the exponential:

$$R(z) = 1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24}.$$

Stability of the numerical method requires $|R(h\lambda)| < 1$. The set of $z = h\lambda$ for which this holds is known as the *stability region* of RK4 (see Figure 9.9).

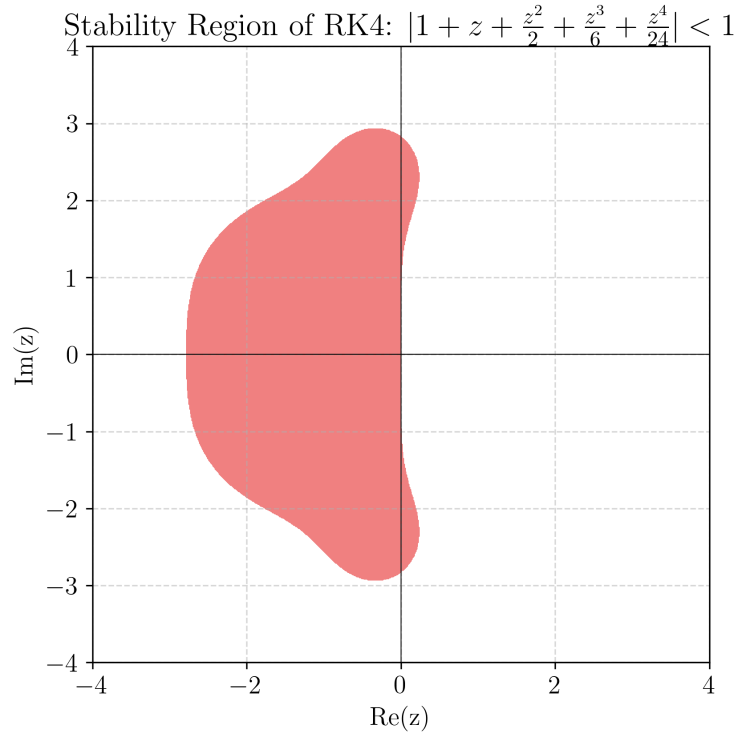


Figure 9.9: RK4 region of stability

Application to the System

Applying RK4 to each component individually, we require:

$$|R(-h)| < 1 \quad \text{and} \quad |R(-10h)| < 1.$$

The RK4 stability region contains the interval $(-2.785, 0)$ on the real axis. Therefore, we must choose h such that both $-h$ and $-10h$ lie within this interval.

The first inequality yields:

$$h < 2.785,$$

but the second, more restrictive condition gives:

$$10h < 2.785 \quad \Rightarrow \quad h < 0.2785.$$

Thus, to ensure that both components of the numerical solution decay to zero, the step size must satisfy

$$h < 0.2785.$$

Numerical Example

We solve the system with initial condition $x(0) = y(0) = 1$ up to time $T = 2$, using RK4 with various step sizes:

Step size h	$x(2)$ RK4	$y(2)$ RK4
0.1	0.1353	2.06×10^{-9}
0.3	0.1349	5.39×10^{-5}
0.5	0.1342	5.56×10^3

For $h = 0.1$, both components are accurate. At $h = 0.3$, the fast component $y(t)$ is already poorly resolved. At $h = 0.5$, the solution becomes unstable, with the y -component diverging. This confirms that for stability and convergence, the step size must satisfy $h < 0.2785$.

Integrating a Stiff System using RK4

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def rk4_step(f, u, t, h):
5     k1 = f(u, t)
6     k2 = f(u + 0.5 * h * k1, t + 0.5 * h)
7     k3 = f(u + 0.5 * h * k2, t + 0.5 * h)
8     k4 = f(u + h * k3, t + h)
9     return u + (h / 6.0) * (k1 + 2*k2 + 2*k3 + k4)
10
11 def f(U, t):
12     x, y = U
13     return np.array([-x, -10*y])
14
15 T = 2.0
16 U0 = np.array([1.0, 1.0])
17 hs = [0.1, 0.3, 0.5]
18
19 for h in hs:
20     N = int(T / h)
21     U = U0.copy()
22     for n in range(N):
23         U = rk4_step(f, U, n*h, h)
24     print(f"h = {h:.1f}, x(T) = {U[0]:.4e}, y(T) = {U[1]:.4e}")

```

A Spectral Theory Point of View

When solving a linear system $y' = Ay$, $y \in \mathbb{R}^n$, $A \in \mathbb{R}^{n \times n}$ using an explicit or implicit method with step size h , the iterative scheme takes the form

$$y_{n+1} = R(hA)y_n$$

where R is a polynomial or a rational function (for implicit methods).

If we want all solutions of the system to be bounded or go to 0 as $t \rightarrow \infty$, we must require that the spectral radius $\rho(R(hA)) < 1$.

Spectral theory (or linear algebra) teaches us that

$$\sigma(R(hA)) = R(h\sigma(A))$$

where $\sigma(A)$ denotes the spectrum of the matrix A . Thus, convergence of solutions to 0 is equivalent to the condition that the set

$$h\sigma(A) = \{h\lambda : \lambda \text{ is an eigenvalue of } A\}$$

is contained in region of absolute stability $|R(z)| < 1$ of the method, e.g., RK4. Boundedness is harder to capture because of non-diagonalizable matrices. But slower than exponential growth of solutions (which then necessarily must be polynomial) is equivalent to the set $h\sigma(A)$ being contained in the region of stability $|R(z)| \leq 1$.

A-stability and Stiffness

Explicit methods tend to behave poorly when faced with a stiff system because their region of stability is bounded, as (see Figure 9.9). This is because $R(z)$ is a polynomial which goes to infinity as $|z| \rightarrow \infty$.

Explicit methods do not have this limitation as $R(z)$ is a rational function. If $|R(z)| < 1$ for all z in the left half-plane $\Re z < 0$ then the system is called *A-stable*. For such methods, the disparate sizes of eigenvalues won't result in a blow-up of numerical solutions.

Conclusion

The system exhibits stiffness due to the disparate decay rates in the two components. The RK4 method, being an explicit integrator, requires the step size to be sufficiently small to ensure that the numerical solution lies within its stability region. Specifically, for this system, convergence to the true solution requires

$$h < 0.2785.$$

Larger step sizes may yield a qualitatively incorrect solution in which the rapidly decaying y -component becomes unstable and diverges.

9.14 Example: A Stiff 2D System with Non-Linearity

In this section, we analyze a stiff system with a stable equilibrium at the origin. We intentionally select the step size h so that $z = h\lambda$ resides outside the stability region for one of the eigenvalues of the linearized system. Our focus will be on the solution behavior given a specific initial condition.

We analyze the following nonlinear system:

Example 9.2 (Nonlinear System: Heun's Method).

$$\begin{cases} \dot{x}_1 &= -2x_1 - \frac{x_1^2}{1+x_2^4}, \\ \dot{x}_2 &= -30x_2 - \frac{x_2^2}{1+x_1^4} \end{cases} \quad (9.7)$$

We investigate the initial condition:

$$x(0) = x_0 = (1, 1).$$

We note that x_0 lies in the first quadrant.

Since 0 is a stable equilibrium for this system, does the numerical solution via Heun's Method converge to 0? $\square$

Focusing on this question, we will analyze the system, ultimately showing that the numerical solution does **not** converge to 0; rather, it exhibits periodic behavior with a period of 4.

The phase diagram is shown in Figure 9.10.

Next, we examine the numerical solution obtained using Heun's Method:

$$\begin{aligned} t_{n+1} &= t_n + h, \\ y_{n+1} &= y_n + \frac{h}{2}(f(t_n, y_n) + f(t_{n+1}, y_n + hf(t_n, y_n))). \end{aligned}$$

With $h = 0.1$, we can express Heun's Method in the predictor-corrector form:

$$\begin{aligned} y_{n+1}^p &= y_n + hf(t_n, y_n), \\ y_{n+1} &= y_n + \frac{h}{2}(f(t_n, y_n) + f(t_{n+1}, y_{n+1}^p)). \end{aligned}$$

Here, the predictor y_{n+1}^p is obtained via the Euler method, which is then refined in the corrector step, resembling a trapezoidal rule.

Heun's Method can also be described as a second-order Runge-Kutta method:

$$\begin{aligned} k_1 &= f(y), \\ k_2 &= f(y + hk_1), \\ y_{n+1} &= y_n + \frac{h}{2}(k_1 + k_2). \end{aligned}$$

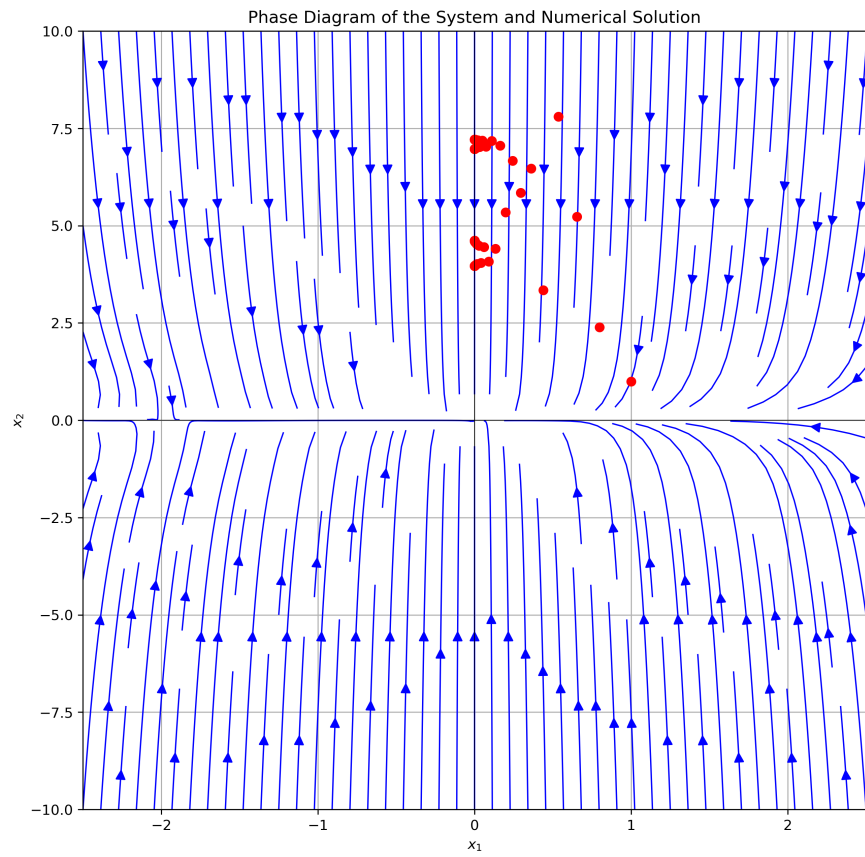


Figure 9.10: Phase diagram of the system (9.7). The numerical trajectory is marked in red.

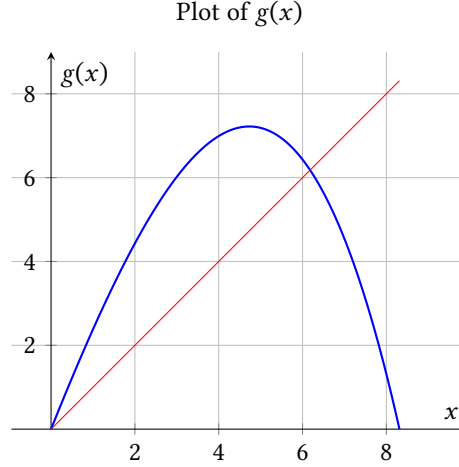


Figure 9.11: Graph of the 1D map $g(x)$.

Thus, Heun's Method serves as a case study for various approaches to iterative, explicit integration schemes. In this context, the integration scheme can be expressed as the mapping $F : \mathbb{R}^2 \rightarrow \mathbb{R}^2$:

$$F(x) = x + \frac{h}{2}(f(x) + f(x + hf(x))).$$

Notably, this mapping is differentiable but *not a diffeomorphism*. To illustrate, consider:

$$F(0, x_2) = (0, g(x_2))$$

with $g : \mathbb{R} \rightarrow \mathbb{R}$ defined as:

$$g(x) = \frac{5}{2}x - \frac{1}{10}x^2 - \frac{1}{50}x^3 - \frac{1}{2000}x^4.$$

Here, g serves as a unimodal, convex function on $[0, \infty)$ as depicted in Figure 9.11.

Key observations include:

- The x_2 -axis acts as an *invariant manifold*.
- The mapping F restricted to the x_2 -axis is folding, preserving the interval $[0, 8.31177]$, where the upper bound is the positive solution of $g(x) = 0$.

The theory surrounding such maps is complex; they can display chaotic behavior and numerous periodic points. Nevertheless, the dynamics of our specific map is straightforward as the orbit of the critical point converges to a period-4 periodic orbit. Rigorous mathematics indicates that *almost every* point in the interval $[0, 8]$ converges to this periodic behavior.

The stability function for Heun's Method is given by $R(z) = 1 + z + \frac{z^2}{2}$, while the linearized system features eigenvalues $\lambda_1 = -2$ and $\lambda_2 = -30$:

$$\begin{aligned} z_1 &= R(\lambda_1 h) = R(-2h) = 0.82, \\ z_2 &= R(\lambda_2 h) = -R(-30h) = R(-3) = 2.5. \end{aligned}$$

These correspond to the eigenvalues of the Jacobian matrix $DF(0)$; notably, $z_2 = g'(0)$, thus making 0 a *repelling fixed point* of g .

Point $(0, 0)$ acts as a *hyperbolic fixed point* of F , with one eigenvalue inside and one outside the unit circle. The Hadamard–Perron Theorem, also known as the Stable Manifold Theorem, states that for a hyperbolic fixed point of a smooth map, local, invariant stable and unstable manifolds exist, tangent to corresponding linear subspaces.

The set of initial conditions that lead to numerical convergence to 0 forms the *stable manifold* of 0, denoted $W^s(0)$. Its local definition is:

$$W_{loc}^s(0) = \{x \in B_r(0) : \lim_{n \rightarrow \infty} F^n(x) = 0\}$$

where $B_r(0)$ is the disk of radius r around 0. The Stable Manifold Theorem indicates that $W_{loc}^s(0)$ behaves like the graph of a smooth function in coordinates diagonalizing the Jacobian of F , expressed as $x_2 = h(x_1)$, for $|x_1| < r'$. For our specific example, it is evident that $W_{loc}^s(0) = B_r(0) \cap \mathbb{R} \times 0$, as x_1 -axis is invariant under F , and therefore it must contain $W_{loc}^s(0)$.

The *global stable manifold* is defined as:

$$W^s(0) = \bigcup_{n=0}^{\infty} F^{-n}(W_{loc}^s(0)).$$

This set includes all initial conditions x_0 such that $\lim_{n \rightarrow \infty} F^n(x_0) = 0$. If F were a diffeomorphism, then $W^s(0)$ would be an embedded 1D manifold. However, due to F not being 1:1, $W^s(0)$ is more complex, with multiple branches, including one starting at $(0, 8.31177)$. While this study does not analyze $W^s(0)$ in depth, algebraic singularities may arise due to the nature of our equations. Hence, $W^s(0)$ becomes a more general object, sometimes referred to as a "branched manifold" or "analytic variety."

Estimations show that the distance to the x_2 -axis decreases exponentially during iterations. Specifically, for $x'_1, x'_2 = F(x_1, x_2)$:

$$\begin{aligned} x_1^p &= x_1 \left(1 - 2h - h \frac{x_1}{1 + x_2^4} \right) \leq x_1(1 - h \cdot 2) \\ &\leq x_1(1 - 2h), \\ x'_1 &= x_1 + \frac{h}{2} \left(-2x_1 - \frac{x_1^2}{1 + x_2^4} - 2x_1^p - \frac{(x_1^p)^2}{1 + (x_2^p)^4} \right) \\ &\leq x_1 + x_1(-h) + x_1(1 - 3h)(-h) \\ &= (1 - 2h + 3h^2)x_1. \end{aligned}$$

Consequently, $0 \leq x'_1 \leq (1 - 2h + 3h^2)x_1 \leq 0.83x_1$, demonstrating exponential convergence to the x_2 -axis and implying that behavior on the x_2 -axis dictates the overall solution dynamics.

We conjecture that for $x_0 = (1, 1)$, the entire orbit $F^n(x_0)$ converges to the period-4 periodic orbit (a probabilistically sound hypothesis). This suggests that $x_0 \notin W^s(0)$, which is crucial to our analysis.

We commence a numerical study to compute "all" initial conditions yielding a numerical solution that converges to the period-4 periodic orbit. This set is termed the *basin of attraction* of the periodic orbit, presented in Figure 9.12. The fractal structure can be observed in the limbs extending to the right. Since $x_0 = (1, 1)$ lies centrally within this basin, we expect our hypothesis to hold true. Nevertheless, we will substantiate this with a **computer-assisted proof (CAP)** demonstrating that x_0 resides within this basin of attraction.

The Computer-Assisted Proof (CAP)

The proof presented in this section exemplifies the core features of computer-assisted proofs in analysis. It serves to guide students interested in crafting mathematically rigorous proofs rather than merely supporting numerical experiments. The following paragraphs, along with the Python implementation, provide substantial value.

A crucial component of a CAP is the ability of modern CPUs to adjust rounding directions. This capability allows us to monitor upper and lower bounds on the precise outcomes of computer arithmetic and the evaluation of library functions. This methodology is referred to as *interval arithmetic*. A real number x is represented by an interval $[a, b]$ where:

1. a and b are machine numbers and $a \leq b$;
2. $x \in [a, b]$.

Basin of attraction of a period-4 periodic orbit

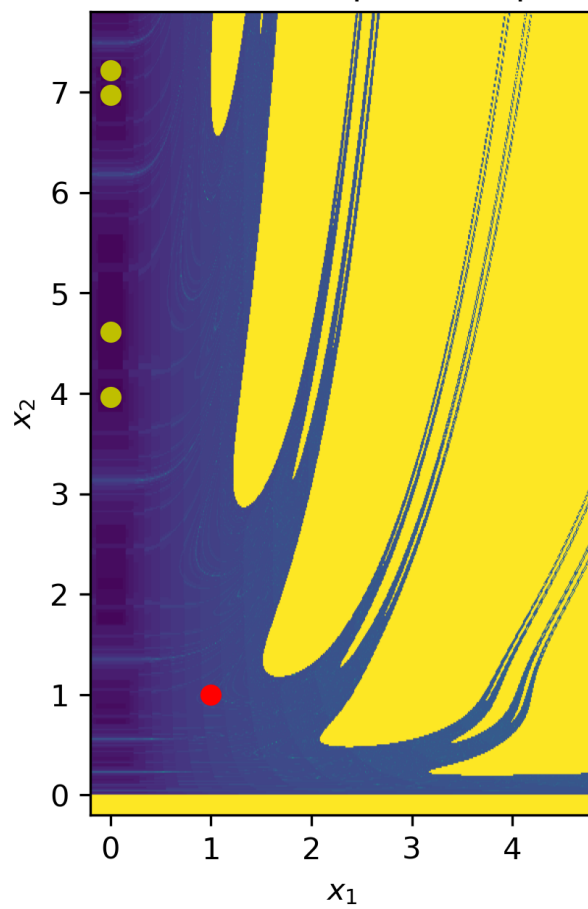


Figure 9.12: The basin of attraction of the period-4 periodic orbit. The periodic orbit is marked in yellow, with the initial condition highlighted in red.

For instance:

$$[a, b] \oplus [c, d] = [\underline{a+c}, \overline{b+d}]$$

Here, the terms "underline" and "overline" signify rounding down and up to the nearest machine number, respectively.

In a similar fashion, we define:

$$\sin([a, b]) = [c, d]$$

such that $\sin([a, b]) \subseteq [c, d]$. The values c and d are reasonably tight approximations of their optimal values, which is quite challenging!

Remark 9.6 (On the software used). We opted to utilize the Python package `mpmath`, which is based on GNU GMP (Gnu Multiprecision Library). This software is robust and has been tested over many years. We specifically employ its feature of *complex interval arithmetic*, accommodating a complex number within a small rectangle:

$$x + iy \in [a, b] + [c, d]i.$$

This means that $x \in [a, b]$ and $y \in [c, d]$ where $a \leq b$ and $c \leq d$ are all machine numbers. The selection of complex interval arithmetic was motivated by its availability in the package and the ease of mapping our 2D problem to the complex plane.

We note that the concept of a "machine number" includes "software machine numbers" of arbitrary precision, so we can theoretically avoid exhausting our precision digits. $\square$

The outline of the proof is structured as follows:

- Rewrite the map F using complex numbers.
- Start with a small interval (a box) centered on $(1, 1)$.
- Subdivide the box to keep its size under a small threshold. This generates a "cloud of boxes" rather than a single box, covering the exact result.
- Apply F to every box in the cloud and subdivide as before.
- If the box approaches the x_2 -axis, "snap" it to touch the x_2 -axis.
- If the box nears the period-4 periodic orbit, it is "absorbed" and removed from further iteration. This step is essential to prevent the number of boxes in the cloud from "blowing up."
- Cease when the entire cloud is absorbed.

```

1 from mpmath import iv, mp
2 from typing import List, NamedTuple, Tuple
3
4 iv.dps = 15
5 h = iv.mpf('0.1')
6
7 # Thresholds
8 X1_DIAM_THRESHOLD = 0.1
9 X2_DIAM_THRESHOLD = 0.1
10 SNAP_THRESHOLD = .4 # If x1_high is below this, snap to x1=0
11 SINK_EPSILON = 1.3
12
13 SINK_POINTS = [4.613677692731402, 7.214907799688287,
14               3.9654987245283035, 6.9704245174643379]
15
16 class BoxKey(NamedTuple):
17     x1_a: float; x1_b: float
18     x2_a: float; x2_b: float
19
20 def get_key(b):
21     return BoxKey(float(b.real.a), float(b.real.b), float(b.imag.a), float(b.imag.b))
22

```

```

23 def f(y):
24     x1, x2 = y.real, y.imag
25     return iv.mpc(-2*x1 - x1**2/(1+x2**4), -30*x2 - x2**2/(1+x1**4))
26
27 def F(y):
28     k1 = f(y); k2 = f(y + h * k1)
29     return y + h/2 * (k1 + k2)
30
31 def iterate_with_snap(cloud: List[iv.mpc]):
32     next_cloud = {}
33     absorbed = 0
34     snapped = 0
35
36     for box in cloud:
37         image = F(box)
38
39         # 1. Capture/Absorption check
40         x1_val = float(image.real.a)
41         x2_val = float(image.imag.mid)
42         if any(max(abs(x1_val), abs(x2_val - s)) < SINK_EPSILON for s in SINK_POINTS):
43             absorbed += 1
44             continue
45
46         # 2. "Snap to Axis" check
47         if 0 < image.real.b < SNAP_THRESHOLD:
48             snapped += 1
49             image = iv.mpc(0, image.imag)
50
51         # 3. Standard Linear Splitting
52         stack = [image]
53         while stack:
54             curr = stack.pop()
55             w1 = curr.real.b - curr.real.a
56             w2 = curr.imag.b - curr.imag.a
57
58             if w1 <= X1_DIAM_THRESHOLD and w2 <= X2_DIAM_THRESHOLD:
59                 next_cloud[get_key(curr)] = curr
60             elif w1 > X1_DIAM_THRESHOLD:
61                 # Linear split x1
62                 mid = curr.real.mid
63                 stack.extend([iv.mpc([curr.real.a, mid], curr.imag),
64                             iv.mpc([mid, curr.real.b], curr.imag)])
65             else: # w2 > X2_DIAM_THRESHOLD
66                 # Linear split x2
67                 mid = curr.imag.mid
68                 stack.extend([iv.mpc(curr.real, [curr.imag.a, mid]),
69                             iv.mpc(curr.real, [mid, curr.imag.b])])
70
71     return list(next_cloud.values()), absorbed, snapped
72
73 def prove_absorption(y0):
74     cloud = [y0]
75     total_absorbed = 0
76
77     print(f"{'Step':<5} | {'Active':<8} | {'Newly Absorbed':<8} | {'Newly snapped':<8}")
78     print("-" * 52)
79
80     for step in range(1, 251):
81         cloud, newly_absorbed, newly_snapped = iterate_with_snap(cloud)
82         total_absorbed += newly_absorbed
83         if step % 1 == 0 or not cloud:
84             print(f"{'step':<5} | {'len(cloud)':<8} | {'newly_absorbed':<16} | {'newly_snapped':<16}")
85             if not cloud:
86                 print("\nSuccess: Cloud fully absorbed.")
87                 return True

```

```

88     return False
89
90 # --- Run 1 ---
91 # Prove that the neighborhood about SINK_POINTS absorbs
92 # (in fact, is invariant)
93
94 # A half-box about zero
95 EPS = iv.mpc([0, SINK_EPSILON], [-SINK_EPSILON, SINK_EPSILON])
96 for x2 in SINK_POINTS:
97     y0 = iv.mpc([0, 0], [x2, x2]) + EPS
98     assert(prove_absorption(y0))
99
100 print("\nThe proof of invariance of sink neighborhood complete\n")
101
102 # --- Run 2 ---
103 # Prove that a trajectory of (1,1) is absorbed
104
105 y0 = iv.mpc([.78, 1.22], [.78, 1.22])
106 assert(prove_absorption(y0))
107
108 # Declare victory
109 print("\nThe proof that the numerical trajectory gets trapped"
110       "\nnear period-4 periodic orbit is complete."
111       "\n\nQED")

```

Listing 9.1: Computer-Assisted Proof that x_0 converges to period 4 orbit.

The output is:

Computer-Assisted Proof – Output			
Step	Active	Newly Absorbed	Newly snapped
1	0	1	0
Success: Cloud fully absorbed.			
Step	Active	Newly Absorbed	Newly snapped
1	0	1	0
Success: Cloud fully absorbed.			
Step	Active	Newly Absorbed	Newly snapped
1	0	1	0
Success: Cloud fully absorbed.			
Step	Active	Newly Absorbed	Newly snapped
1	0	1	0
Success: Cloud fully absorbed.			
The proof of invariance of sink neighborhood complete.			
Step	Active	Newly Absorbed	Newly snapped
1	512	0	0
2	1784	395	0
3	4480	1224	105
4	9752	3872	151

5	7344	9293	199
6	1344	7176	38
7	0	1344	0

Success: Cloud fully absorbed.

The proof that the numerical trajectory gets trapped near period-4 periodic orbit is complete.

QED
>>>

In a matter of seconds, the program demonstrated that the box

$$B = \{(x_1, x_2) \mid .78 \leq x_1, x_2 \leq 1.22\}$$

is contained in the basin of attraction of the periodic orbit. Specifically, the program proves that the set $F^7(B)$ is contained within a neighborhood of radius 0.1 of the period-4 periodic orbit approximation (using the ℓ^∞ -norm). Furthermore, the program proves that this neighborhood is an invariant set.

The result is *mathematically rigorous*, and any published journal would accept it as such.

There is considerable "margin for error" to conclude that the numerical solution gets "captured" by the periodic orbit and does not converge to 0. This holds true even amid round-off errors, which are challenging to track by other means.

A noteworthy observation is that during the computation **the cloud expanded to approximately 10,000 boxes**, making manual execution of the pseudocode with (or without) a calculator nearly impossible.

It is important to note that we used 15 decimal digits in all calculations, meaning the proof does not strictly require the extended precision capabilities of GMP.

Remark 9.7 (On decimal vs. machine numbers). In the above proof, we use decimal notation (e.g., "1.22"), which conservatively represents a machine number. The term "conservatively" may mean rounding up or down to the nearest machine number, depending on the context. $\square$

9.15 Exercises

Exercise 9.1 (Euler's Method, comparison to exact solution). Solve the initial value problem $y'(t) = t^2 + y^2$, $y(0) = 1$ using Euler's method with $h = 0.1$ for t in $[0, 1]$. Compare the numerical solution with the exact solution, if available. $\square$

Exercise 9.2 (RK4, comparison to exact solution). Solve the initial value problem $y'(t) = \cos(t) - y$, $y(0) = 0$ using the fourth-order Runge-Kutta RK4 method with $h = 0.1$ for t in $[0, 1]$. Compare the numerical solution with the exact solution, if available. $\square$

Exercise 9.3 (Two-step Adams-Bashforth Method). Solve the initial value problem $y'(t) = y - t^2 + 1$, $y(0) = 0.5$ using the two-step Adams-Bashforth method with $h = 0.1$ for t in $[0, 2]$. Compare the numerical solution with the exact solution, if available. $\square$

Exercise 9.4 (Adaptive method, plotting step size). Solve the initial value problem $y'(t) = \sin(t) - y$, $y(0) = 1$ using an adaptive step size method with an initial step size $h = 0.1$ and a tolerance $\epsilon = 10^{-5}$ for t in $[0, 1]$. Plot the step sizes used. $\square$

Exercise 9.5 (Euler's Method, stability). Analyze the stability of Euler's method for the test equation $y' = \lambda y$ with $\lambda = -2$. Determine the maximum step size h for which the method is stable. $\square$

Exercise 9.6 (Euler's Method vs. Heun's Method vs. RK4). Heun's Method (identical to RK2) is often written as a two-step *predictor-corrector method*:

$$\begin{aligned} y_{n+1}^P &= y_n + hf(t_n, y_n), \quad (\text{predictor step}) \\ y_{n+1} &= y_n + \frac{h}{2} (f(t_n, y_n) + f(t_{n+1}, y_{n+1}^P)), \quad (\text{corrector step}). \end{aligned}$$

(A family of predictor-corrector methods exists.) Implement and compare the performance of Euler's method, Heun's method and the fourth-order Runge-Kutta method for solving $y'(t) = e^t - y$, $y(0) = 1$ over the interval $[0, 2]$. $\square$

Exercise 9.7 (RK45, plotting solution). Solve the initial value problem for the differential equation

$$\frac{dy}{dx} = y - x^2 + 1, \quad y(0) = 0.5$$

using the adaptive Runge-Kutta method RK45. Integrate from $x = 0$ to $x = 2$ and plot the solution. $\square$

Exercise 9.8 (RK45, comparison to exact solution). Consider the initial value problem for the differential equation

$$\frac{dy}{dx} = -2xy, \quad y(0) = 1$$

Use the adaptive Runge-Kutta method RK45 to solve the equation from $x = 0$ to $x = 5$. Compare your numerical solution with the analytical solution $y(x) = e^{-x^2}$. $\square$

Exercise 9.9 (RK45, system of two equations). Solve the initial value problem for the system of differential equations

$$\begin{cases} \frac{dy_1}{dx} = y_2 \\ \frac{dy_2}{dx} = -y_1 \end{cases}, \quad y_1(0) = 1, \quad y_2(0) = 0$$

using the adaptive Runge-Kutta method RK45. Integrate from $x = 0$ to $x = 2\pi$ and plot the solutions $y_1(x)$ and $y_2(x)$. $\square$



Exercise 9.10 (RK45, linear equation, stability analysis). Consider the initial value problem for the differential equation

$$\frac{dy}{dx} = \sin(x) - y, \quad y(0) = 2$$

Use the adaptive Runge-Kutta method RK45 to solve the equation from $x = 0$ to $x = 10$. Analyze the stability of the solution and discuss the step size adaptation. $\square$

Exercise 9.11 (RK45, linear-fractional equation). Solve the initial value problem for the differential equation

$$\frac{dy}{dx} = \frac{x+y}{x-y}, \quad y(1) = 2$$

using the adaptive Runge-Kutta method RK45. Integrate from $x = 1$ to $x = 3$. Discuss the advantages of using an adaptive method like RK45 for this problem. $\square$

Exercise 9.12 (Taylor Method, second order, linear equation). Solve the initial value problem for the differential equation

$$\frac{dy}{dx} = y + x, \quad y(0) = 1$$

using the second-order Taylor method. Integrate from $x = 0$ to $x = 1$ with a step size of $h = 0.1$. Compare your numerical solution with the exact solution. $\square$

Exercise 9.13 (Taylor Method, third-order). Consider the initial value problem for the differential equation

$$\frac{dy}{dx} = x^2 + y^2, \quad y(0) = 0$$

Use the third-order Taylor method to solve the equation from $x = 0$ to $x = 1$ with a step size of $h = 0.1$. Compute and compare the numerical solution with the exact solution, if available. $\square$

Exercise 9.14 (Taylor Method, second-order). Solve the initial value problem for the system of differential equations

$$\begin{cases} \frac{dy_1}{dx} = y_2 \\ \frac{dy_2}{dx} = -y_1 \end{cases}, \quad y_1(0) = 1, \quad y_2(0) = 0$$

using the second-order Taylor method. Integrate from $x = 0$ to $x = \pi$ with a step size of $h = 0.1$. Plot the numerical solutions for $y_1(x)$ and $y_2(x)$. $\square$

Exercise 9.15 (Taylor Method, fourth-order). Consider the initial value problem for the differential equation

$$\frac{dy}{dx} = e^x - y, \quad y(0) = 1$$

Use the fourth-order Taylor method to solve the equation from $x = 0$ to $x = 1$ with a step size of $h = 0.1$. Compare the numerical solution with the exact solution. $\square$

Exercise 9.16 (Taylor Method, third-order). Solve the initial value problem for the differential equation

$$\frac{dy}{dx} = \sin(x) + \cos(y), \quad y(0) = 0$$

using the third-order Taylor method. Integrate from $x = 0$ to $x = 2$ with a step size of $h = 0.1$. Discuss the accuracy and stability of the numerical solution. $\square$

Exercise 9.17 (Elastic Pendulum). Write a program in Python solving the elastic pendulum equations (9.4) for a single initial condition in Python without setting the vectorization option to the solver. $\square$

Exercise 9.18 (Van der Pol oscillator). Consider the Van der Pol oscillator described by the system of equations:

$$\frac{dx}{dt} = y, \quad \frac{dy}{dt} = \mu(1 - x^2)y - x.$$

- Briefly describe what a limit cycle is, and explain why the Van der Pol oscillator has a limit cycle for $\mu > 0$.
- Numerically approximate the limit cycle of the Van der Pol oscillator for $\mu = 3$. Clearly describe your numerical approach.
- Numerically approximate the limit cycle of the Van der Pol oscillator for $\mu = 3$. Describe clearly your numerical approach.
- Numerically approximate the period of the limit cycle to within two decimal places. Clearly explain your approach.
- Define a suitable Poincaré section for this system and explain how you can numerically use it to confirm the existence and stability of the limit cycle found in the previous parts.

$\square$

Exercise 9.19 (Chaos in Periodically Forced Pendulum). Consider a periodically forced pendulum described by:

$$\dot{\theta} = \omega, \quad \dot{\omega} = -\sin(\theta) + F(t),$$

where $F(t) = \mu \sin(\omega t)$ is periodic ($\mu \neq 10, \omega > 0$). Define the Poincaré section S as $\theta = 0$. Then, $s(y) = \theta$ where $y = (\theta, \omega)$, and the tangent vectors to S satisfy $\nabla s(y) = (1, 0)^T$.

Apply Algorithm 9.7 to find an initial condition $y_0 = (\theta_0, 0)$ such that

- The Lyapunov exponent $\lambda_P(y_0) = 0$ within numerical tolerance.
- The Lyapunov exponent $\lambda_P(y_0) > 0$ within numerical tolerance.

Thus, provide evidence that non-chaotic and chaotic behavior may coexist in this system. $\square$

Chapter 10

Boundary Value Problems for ODEs

10.1 Introduction

Boundary value problems (BVPs) for ordinary differential equations (ODEs) involve finding a solution to a differential equation that satisfies boundary conditions at more than one point. These problems often arise in the modeling of physical systems, such as heat conduction, fluid flow, and structural analysis.

The general form of a boundary value problem is:

$$y''(x) = f(x, y(x), y'(x)), \quad \text{with boundary conditions} \quad y(a) = \alpha \quad \text{and} \quad y(b) = \beta$$

where $y(x)$ is the unknown function, $[a, b]$ is the interval of interest, and α and β are the boundary values.

10.2 Shooting Method

The shooting method transforms a boundary value problem into an initial value problem (IVP). It involves guessing the initial slope and iteratively refining the guess to meet the boundary conditions.

10.2.1 Algorithm

The shooting method algorithm can be described as follows:

10.2.2 Example

Consider the boundary value problem:

$$y''(x) = -y(x), \quad y(0) = 0, \quad y(\pi) = 0$$

Using the shooting method with initial guesses $y'(0) = 1$ and $y'(0) = -1$, and a tolerance $\epsilon = 10^{-5}$, we approximate the solution $y(x)$.

10.3 Finite Difference Method

The finite difference method approximates the derivatives in the differential equation using finite differences and solves the resulting system of linear equations.

10.3.1 Algorithm

The finite difference method algorithm can be described as follows:

Algorithm 10.1 Shooting Method

Require: A function $f(x, y, y')$, boundary conditions $y(a) = \alpha$, $y(b) = \beta$, initial guesses $y'(a) = s_1$ and $y'(a) = s_2$, and a tolerance ϵ

Ensure: Approximate solution $y(x)$ satisfying the boundary conditions

- 1: Solve the IVP with initial condition $y(a) = \alpha$ and $y'(a) = s_1$
- 2: Let $y_1(b)$ be the value of y at $x = b$ from the IVP solution
- 3: Solve the IVP with initial condition $y(a) = \alpha$ and $y'(a) = s_2$
- 4: Let $y_2(b)$ be the value of y at $x = b$ from the IVP solution
- 5: **while** $|y_1(b) - \beta| > \epsilon$ **do**
- 6: Compute the next guess for $y'(a)$:

$$s = s_2 - \frac{(y_2(b) - \beta)(s_2 - s_1)}{y_2(b) - y_1(b)}$$

- 7: Solve the IVP with initial condition $y(a) = \alpha$ and $y'(a) = s$
 - 8: Update $y_1(b) \leftarrow y_2(b)$, $s_1 \leftarrow s_2$, $y_2(b) \leftarrow y(b)$, and $s_2 \leftarrow s$
 - 9: **end while**
 - 10: **return** $y(x)$
-

Algorithm 10.2 Finite Difference Method

Require: A function $f(x, y, y')$, boundary conditions $y(a) = \alpha$, $y(b) = \beta$, and number of grid points n

Ensure: Approximate values of y at the grid points

- 1: Discretize the interval $[a, b]$ into n grid points x_i with step size $h = \frac{b-a}{n-1}$
- 2: **for** $i = 2$ **to** $n-1$ **do**
- 3: Approximate the second derivative:

$$y''(x_i) \approx \frac{y_{i+1} - 2y_i + y_{i-1}}{h^2}$$

- 4: Formulate the finite difference equation:

$$\frac{y_{i+1} - 2y_i + y_{i-1}}{h^2} = f(x_i, y_i, y'_i)$$

- 5: **end for**
 - 6: Solve the resulting system of linear equations for y_i
 - 7: **return** y
-

10.3.2 Example

Consider the boundary value problem:

$$y''(x) = -y(x), \quad y(0) = 0, \quad y(\pi) = 0$$

Using the finite difference method with $n = 10$ grid points, we approximate the solution $y(x)$.

10.4 Exercises

Exercise 10.1. Solve the boundary value problem $y''(x) = -y(x)$, $y(0) = 0$, $y(\pi/2) = 1$ using the shooting method. Use initial guesses $y'(0) = 1$ and $y'(0) = 0$, and a tolerance $\epsilon = 10^{-5}$. $\square$

Exercise 10.2. Solve the boundary value problem $y''(x) = y(x)$, $y(0) = 1$, $y(1) = e$ using the finite difference method with $n = 10$ grid points. Compare the numerical solution with the exact solution. $\square$

Exercise 10.3. Consider the boundary value problem $y''(x) + xy'(x) - y(x) = 0$, $y(0) = 1$, $y(1) = 0$. Solve it using the shooting method with initial guesses $y'(0) = -1$ and $y'(0) = -2$. $\square$

Exercise 10.4. Consider the boundary value problem $y''(x) - \pi^2 y(x) = 0$, $y(0) = 0$, $y(1) = \sin(\pi)$. Solve it using the finite difference method with $n = 10$ grid points. $\square$

Chapter 11

Control Theory

11.1 Introduction

Control Theory addresses a problem which merges the language of ODE and optimization.

Problem Formulation

System Dynamics

We consider a system governed by the differential equation:

$$\dot{x}(t) = f(x(t), u(t), t)$$

where $x(t)$ represents the state of the system at time t , $u(t)$ is the control input, and f is a function describing the system dynamics.

Admissible controls

The function $u(t)$ is for us to choose, as we are “in control”. However, this function is not arbitrary. The value $u(t)$ is restricted to the set of admissible controls $\mathcal{U}$. Often this set is assumed to be convex, and sometimes a convex polyhedron.

Optimization Problem

The goal is to minimize a cost function over a finite or infinite time horizon, given by:

$$\mathcal{J} = g(x(T)) + \int_{t_0}^T L(x(t), u(t), t) dt$$

where L is the running cost function, and g is the terminal cost function, and T is the terminal time. If $T = \infty$, we say that the time horizon is infinite.

11.2 Pontryagin Maximum Principle

The Pontryagin Maximum Principle (PMP) is a cornerstone of optimal control theory, providing necessary conditions for optimality in control problems. It was developed by Lev Pontryagin and his colleagues in the 1950s.

Hamiltonian

The decisive step in PMP is the introduction of the Hamiltonian:

$$\mathcal{H}(x, u, \lambda, t) = L(x, u, t) + \lambda^\top f(x, u, t)$$

where $\lambda(t)$ is the vector of costate variables.

Effective Hamiltonian

It is a function $H(x, \lambda, t)$ obtained by maximizing $\mathcal{H}$ with respect to the control:

$$H(x, \lambda, t) = \max_{u \in \mathcal{U}} \mathcal{H}(x, u, \lambda, t).$$

Necessary Conditions

The *Pontryagin Maximum Principle* states that for optimal control $u^*(t)$ and trajectory $x^*(t)$, there exists a nontrivial $\lambda(t)$ such that:

1. State and costate dynamics::

$$\dot{x}^*(t) = \frac{\partial H}{\partial \lambda}, \quad \dot{\lambda}(t) = -\frac{\partial H}{\partial x}$$

2. Hamiltonian maximization (minimization)::

$$\mathcal{H}(x^*(t), u^*(t), \lambda(t), t) = \max_{u \in \mathcal{U}} \mathcal{H}(x^*(t), u, \lambda(t), t) = H(x^*(t), \lambda(t), t).$$

3. Transversality conditions:

$$\lambda(T) = -\frac{\partial g}{\partial x}(x^*(T))$$

Remark 11.1 (On necessity of the transversality conditions). NOTE: When $x(T)$ is fixed then g has no meaning, and the transversality conditions are not necessary. In fact, **it is likely to be violated**, so it is *not a necessary condition of optimality*. They are important, however, when $x(T)$ is *free*. In other words, when we do not care where we end up at the end, as long as we follow an optimal path, and terminal cost is also optimized. When the terminal cost function g is zero (or constant), the transversality condition becomes $\lambda(T) = 0$. $\square$

While these conditions are necessary, they are not sufficient. It is hard to formulate sufficient conditions which are general enough to cover most interesting applications. Therefore one relies upon the necessary conditions to identify the potential solutions, prove their uniqueness and analyze their properties. This is usually done on case-by-case basis.

Application

The PMP transforms the optimal control problem into a boundary value problem, providing a framework for solving complex control problems.

11.3 On the Mechanics of Applying PMP

There are two seemingly different procedures to apply PMP: with and without explicitly constructing the effective Hamiltonian.

Steps Using the Effective Hamiltonian

★★★ Step 1: Construct the Hamiltonian ★★★

$$\mathcal{H}(x, u, \lambda, t) = L(x, u, t) + \lambda^\top f(x, u, t)$$

★★★ Step 2: Find optimal control ★★★

$$u^* = \arg \max_{u \in \mathcal{U}} \mathcal{H}(x, u, \lambda, t)$$

Note that $u^* = U^*(x, \lambda, t)$, where U^* is some function to be determined.

★★★ Step 3: Write down the effective Hamiltonian ★★★

$$H(x, \lambda, t) = \mathcal{H}(x, U^*(x, \lambda, t), \lambda, t)$$

★★★ Step 4: Solve the Hamiltonian system ★★★

$$\begin{cases} \dot{x} = \frac{\partial H}{\partial \lambda} \\ \dot{\lambda} = -\frac{\partial H}{\partial x} \end{cases}$$

★★★ Step 5: Define the optimal control as function of time ★★★

$$u^*(t) = U^*(x^*(t), \lambda^*(t), t)$$

Steps Working with Hamiltonian only

★★★ Step 1: Construct the Hamiltonian ★★★

$$\mathcal{H}(x, u, \lambda, t) = L(x, u, t) + \lambda^\top f(x, u, t)$$

★★★ Step 2: Write the Hamiltonian system ★★★

$$\begin{cases} \dot{x} = \frac{\partial \mathcal{H}}{\partial p} \\ \dot{\lambda} = -\frac{\partial \mathcal{H}}{\partial x} \end{cases} \iff \begin{cases} \dot{x} = f(x, u, t), \\ \dot{\lambda} = -\frac{\partial L}{\partial x}(x, u, t) - \lambda^\top \frac{\partial f}{\partial x}(x, u, t). \end{cases}$$

★★★ Step 3: Find optimal control ★★★

$$u^* = \arg \max_{u \in \mathcal{U}} \mathcal{H}(x, u, \lambda, t)$$

Note that $u^* = U^*(x, \lambda, t)$.

★★★ Step 4: Substitute the optimal control into the Hamiltonian system ★★★

$$\begin{cases} \dot{x} = f(x, U^*(x, \lambda, t), t), \\ \dot{\lambda} = -\frac{\partial L}{\partial x}(x, U^*(x, \lambda, t), t) - \lambda^\top \frac{\partial f}{\partial x}(x, U^*(x, \lambda, t), t). \end{cases}$$

Solve the system to find the optimal trajectory $(x^*(t), \lambda^*(t))$.

★★★ Step 5: Define the optimal control as function of time ★★★

$$u^*(t) = U^*(x^*(t), \lambda^*(t), t)$$

Pros and Cons of Each Approach

Writing the effective Hamiltonian and constructing the Hamiltonian system of ODE proves the Hamiltonian nature of the system ODE. The system has a mechanistic interpretation: $H(x, \lambda)$ is the energy and λ is the generalized momentum.

Working without the effective Hamiltonian may be easier to memorize and work with for a student with lesser mathematical background. The peculiar feature of this approach is that we construct the Hamiltonian system, assumint that the control is *frozen* (constant in time). Then we *unfreeze* u : we substitute u with u^* which varies with x and λ . The Hamiltonian nature of the resulting system of ODE is unclear at this point.

Equivalence of the Two Approaches

At first, the two procedures may appear different and we may surmise that they produce different results. It turns out that under practical assumptions the results are equivalent. In particular, we obtain the same system of ODE in the end.

It is easy to see that

$$\frac{\partial H}{\partial x} = \frac{\partial \mathcal{H}}{\partial x}$$

and therefore the costate equation is the same in both parts of the systems of ODE. It remains to be seen that

$$\frac{\partial H}{\partial \lambda} = \frac{\partial \mathcal{H}}{\partial \lambda} \Big|_{u=u^*}. \quad (11.1)$$

To show this, we will assume that $L(x, u, t)$ and $f(x, u, t)$ are differentiable in u and that $\mathcal{H}(x, u, \lambda, t)$ has a unique critical point $u = u^*$. It follows that

$$\frac{\partial \mathcal{H}}{\partial u} = 0$$

or, after trivial manipulations,

$$\frac{\partial L}{\partial u} + \lambda^\top \frac{\partial f}{\partial u} = 0.$$

This equation defines u implicitly in terms of λ , x , and t .

On the other hand,

$$\frac{\partial H}{\partial \lambda} = \frac{\partial \mathcal{H}}{\partial \lambda} + \overbrace{\left[\frac{\partial \mathcal{H}}{\partial u} \right]}^{=0} \frac{\partial u}{\partial \lambda} = \frac{\partial \mathcal{H}}{\partial \lambda} + \overbrace{\left(\frac{\partial L}{\partial u} + \lambda^\top \frac{\partial f}{\partial u} \right)}^{=0} \frac{\partial u}{\partial \lambda} = \frac{\partial \mathcal{H}}{\partial \lambda} = f(x, u, t).$$

This is exactly the equation (11.1), which implies the equivalence of the two approaches to applying PMP.

The scope and validity of PMP

One should ask: what happens when the assumption of smoothness and strong concavity of L with respect to u is not satisfied? The answer is that we can approximate the original problem by a regularized problem. Firstly, we assume

1. convexity of the control set $\mathcal{U}$
2. concavity of the Hamiltonian $\mathcal{H}(x, u, \lambda, t)$ with respect to u .

Under those assumptions, the approximation is possible.

The concavity in u of

$$\mathcal{H}(x, u, \lambda, t) = L(x, u, \lambda) + \lambda^\top f(x, u, t)$$

must hold **for all** λ , to guarantee **finiteness of the optimal control**. For example, if for large $\|u\|$, and some $c_1, c_2 > 0$:

$$\begin{aligned} \|f(x, u, t)\| &\leq c_1 \times \|u\|_2, \\ L(x, u, t) &\leq -c_2 \times \|u\|_2^2, \end{aligned}$$



Figure 11.1: Optimally stopping a swing

it is reasonable that this condition will be satisfied. Also, when the control is confined to a convex, bounded region $\mathcal{U}$, this also normally leads to a finite and unique

$$u^* = \arg \max_{u \in \mathcal{U}} \mathcal{H}(x, u, \lambda, t).$$

Otherwise, PMP may not work as expected.

11.4 Optimal Control of a Swinging Pendulum

Stopping a swing by applying torque

Problem Statement

- Objective: Apply torque to stop a swinging pendulum with minimum energy expenditure.
- The pendulum is modeled as a mathematical pendulum with mass m at the end of a massless rod of length l .

Dynamics of the Pendulum

The equation of motion for the pendulum subjected to torque $\tau(t)$ is:

$$\frac{d^2\theta}{dt^2} + \frac{g}{l} \sin(\theta) = \frac{\tau(t)}{ml^2}$$

where:

- $\theta(t)$ is the angular displacement,
- g is the acceleration due to gravity,
- l is the length of the rod,
- m is the mass of the pendulum,
- $\tau(t)$ is the control input (torque).

Objective Function

Minimize the energy expended by the torque, represented by the cost function:

$$J = \int_0^T \tau(t)^2 dt$$

- The goal is to find the control $\tau(t)$ that minimizes J .
- T is the time at which the pendulum comes to a stop.

We note that $T = \infty$ is allowed; this is the *infinite horizon* version of the problem. In this section we will focus primarily on the *finite horizon* version of the problem, where $T \in (0, \infty)$.

Boundary Conditions

The pendulum starts from an initial condition and must come to a stop:

- Initial angular position: $\theta(0) = \theta_0$,
- Initial angular velocity: $\frac{d\theta}{dt}(0) = \omega_0$,
- Final conditions: $\theta(T) = 0$, $\frac{d\theta}{dt}(T) = 0$.

Mathematical Formulation

- **Minimize** $J = \int_0^T \tau(t)^2 dt$
- **Subject to:**
 - Dynamics: $\frac{d^2\theta}{dt^2} + \frac{g}{l} \sin(\theta) = \frac{\tau(t)}{ml^2}$,
 - Boundary conditions at $t = 0$ and $t = T$.

11.5 Optimal Synthesis and Phase Diagrams for a Nonlinear Pendulum

In this section we present a full optimal synthesis for stopping a nonlinear pendulum by applying the minimum energy control. With the parameter choices

$$m = 1, \quad l = 1, \quad g = 1$$

the pendulum dynamics simplify to

$$\ddot{\theta}(t) + \sin \theta(t) = \tau(t).$$

The control objective is to minimize the energy expenditure,

$$J = \int_0^T \tau(t)^2 dt,$$

subject to the above boundary conditions.

Application of Pontryagin's Minimum Principle

We begin by formulating the problem in state-space form. Define the state variables

$$x_1 = \theta, \quad x_2 = \dot{\theta},$$

so that the dynamics become (we write τ for $\tau(t)$):

$$\dot{x}_1 = x_2,$$

$$\dot{x}_2 = \tau - \sin x_1.$$

Introducing the costate vector $\lambda = (\lambda_1, \lambda_2)$, the Hamiltonian for the problem is

$$\mathcal{H}(x, \lambda, \tau) = \tau^2 + \lambda_1 x_2 + \lambda_2 (\tau - \sin x_1).$$

The optimality condition obtained by minimizing the Hamiltonian with respect to the control τ yields

$$\frac{\partial \mathcal{H}}{\partial \tau} = 2\tau + \lambda_2 = 0 \implies \tau^* = -\frac{\lambda_2}{2}.$$

The effective Hamiltonian is

$$\begin{aligned} H(x, \lambda) &= \mathcal{H}(x, \lambda, \tau^*) \\ &= \frac{\lambda_2^2}{4} + \lambda_1 x_2 + \lambda_2 \left(-\frac{\lambda_2}{2} - \sin x_1 \right) \\ &= -\frac{\lambda_2^2}{4} + \lambda_1 x_2 - \lambda_2 \sin x_1. \end{aligned}$$

The costate equations are given by

$$\begin{aligned} \dot{\lambda}_1 &= -\frac{\partial H}{\partial x_1} = \lambda_2 \cos x_1, \\ \dot{\lambda}_2 &= -\frac{\partial H}{\partial x_2} = -\lambda_1. \end{aligned}$$

Thus, the necessary conditions for optimality are summarized by the two-point boundary value problem (TP-BVP):

$$\begin{cases} \dot{x}_1 = x_2, \\ \dot{x}_2 = -\frac{\lambda_2}{2} - \sin x_1, \\ \dot{\lambda}_1 = \lambda_2 \cos x_1, \\ \dot{\lambda}_2 = -\lambda_1, \end{cases}$$

with the state boundary conditions:

$$x_1(0) = \theta_0, \quad x_2(0) = \omega_0, \quad x_1(T) = 0, \quad x_2(T) = 0.$$

The costate boundary conditions are free since the final state is prescribed.

Numerical Solution and Ensemble Phase Diagrams

Since an analytical solution of the two-point boundary value problem (TPBVP) is generally not available for nonlinear systems, we solve it numerically. In the following Python code we use SciPy's `solve_bvp` function to compute the optimal state and costate trajectories for several initial conditions (different values of θ_0). This ensemble approach provides a clearer picture of how the optimal trajectories behave in the phase plane.

```

1  # ----- Stopping pendulum by applying torque -----
2  # File:      pendulum_control_synth.py
3  # -----
4  import numpy as np
5  from scipy.integrate import solve_bvp
6  import matplotlib.pyplot as plt
7
8
9  def solve_optimal(theta0, omega0, T=2.0):
10     """Solve the TPBVP for a given initial angle theta0 and initial
11     angular velocity omega0.
```

22 22 22

```
12
13 # ODE system for [x1, x2, lambda1, lambda2]
14 def odefun(t, y):
15     # y[0] = x1, y[1] = x2, y[2] = lambda1, y[3] = lambda2
16     x1 = y[0]
17     x2 = y[1]
18     lambda1 = y[2]
19     lambda2 = y[3]
20
21     dx1_dt = x2
22     # Optimal control: tau = -lambda2/2
23     dx2_dt = -lambda2/2 - np.sin(x1)
24     dlambda1_dt = lambda2 * np.cos(x1)
25     dlambda2_dt = -lambda1
26     return np.vstack((dx1_dt, dx2_dt, dlambda1_dt, dlambda2_dt))
27
28 # Boundary conditions: x1(0)=theta0, x2(0)=omega0, x1(T)=0, x2(T)=0.
29 def bcfun(ya, yb):
30     return np.array([ya[0] - theta0,      # x1(0) - theta0
31                     ya[1] - omega0,      # x2(0) - omega0
32                     yb[0] - 0.0,         # x1(T)
33                     yb[1] - 0.0])        # x2(T)
34
35 # Time grid
36 t = np.linspace(0, T, 100)
37
38 # Initial guess: simple linear interpolation for x1, constant for
39 # x2, zeros for costates.
40 y_guess = np.zeros((4, t.size))
41 y_guess[0] = theta0 * (1 - t/T)      # from theta0 to 0
42 y_guess[1] = -theta0/T               # rough guess for derivative
43 y_guess[2] = 0                       # initial guess for lambda1
44 y_guess[3] = 0                       # initial guess for lambda2
45
46 sol = solve_bvp(odefun, bcfun, t, y_guess)
47 if sol.status != 0:
48     print("BVP did not converge for theta0 =", theta0, ":", sol.message)
49 return sol
50
51
52 # Define a set of initial conditions to illustrate multiple trajectories
53 initial_thetas = [0.3, 0.4, 0.5, 0.6, 0.7]
54 initial_omegas = [0.0, 0.0, 0.0, 0.0, 0.0]
55 solutions = {}
56 T = 2.0
57
58 # Solve for each initial condition
59 for theta0, omega0 in zip(initial_thetas, initial_omegas):
60     sol = solve_optimal(theta0, omega0, T)
61     if sol.status == 0:
62         solutions[theta0] = sol
63
64 # Plot the phase diagram for all trajectories
```

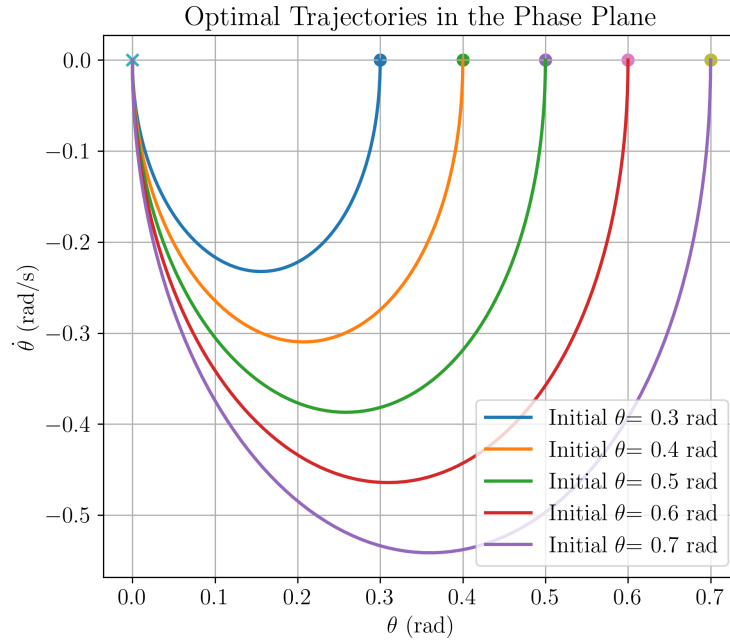


Figure 11.2: Several solutions for the problem of stopping pendulum optimally

```

65 plt.figure(figsize=(7, 6))
66 for theta0, sol in solutions.items():
67     x1 = sol.y[0]
68     x2 = sol.y[1]
69     plt.plot(x1, x2, lw=2, label=(r'Initial $\theta$' f'= {theta0:.1f} rad'))
70     # Mark initial and final points
71     plt.scatter(x1[0], x2[0], marker='o', s=50)
72     plt.scatter(x1[-1], x2[-1], marker='x', s=50)
73
74 plt.xlabel(r'$\theta$ (rad)')
75 plt.ylabel(r'$\dot{\theta}$ (rad/s)')
76 plt.title('Optimal Trajectories in the Phase Plane')
77 plt.legend()
78 plt.grid(True)
79 #plt.show()
80 plt.savefig("figures/generated/pendulum_control_synth.png", dpi=300, bbox_inches='tight')
81 plt.close()

```

The resulting plot is in Figure 11.2.

Discussion

The code above computes optimal trajectories using Pontryagin's Minimum Principle for various initial conditions (with zero initial angular velocity). Each trajectory is forced to reach the origin (0,0) at $t = 2$ seconds, and the resulting phase plane diagram provides insight into the state evolution under the minimum energy control law. By plotting several trajectories on the same diagram, one can visually appreciate the sensitivity of the synthesis with respect to different initial states.

This section thus demonstrates the practical implementation of nonlinear optimal control and provides a visual, ensemble-based understanding of the optimal state trajectories and their corresponding control profiles.

11.6 Optimal Control with Bounded Torque

In this section we consider an optimal control problem for a pendulum in which the applied torque is bounded. The dynamics of the pendulum are given by

$$\ddot{\theta}(t) + \sin \theta(t) = \tau(t),$$

and we wish to steer the system from an initial state

$$\theta(0) = \theta_0, \quad \dot{\theta}(0) = \omega_0,$$

to the equilibrium

$$\theta(T) = 0, \quad \dot{\theta}(T) = 0,$$

in minimum time while respecting the control bound

$$\tau(t) \in [-a, a].$$

As in the previous section,

$$\mathcal{H}(x, \lambda, \tau) = \tau^2 + \lambda_1 x_2 + \lambda_2 (\tau - \sin x_1) = \tau^2 + \lambda_2 \tau + \lambda_1 x_2 - \lambda_2 \sin x_1.$$

The optimality condition obtained by minimizing the Hamiltonian with respect to the control $\tau \in [-a, a]$.

Since $\mathcal{H}$ is quadratic in τ with global minimum

$$\tau_{min} = -\frac{\lambda_2}{2}$$

found in the previous section, then the optimal τ is given by:

$$\tau^* = \begin{cases} \tau_{min} & \text{if } \tau_{min} \in [-a, a], \\ a & \text{if } \tau_{min} > a, \\ -a & \text{if } \tau_{min} < -a. \end{cases}$$

As we say, τ^* is obtained by *clipping* τ_{min} to the interval $[-a, a]$. We rewrite this relationship in terms of λ_2 :

$$\tau^* = \begin{cases} -\frac{\lambda_2}{2} & \text{if } |\lambda_2| \leq 2a, \\ -a \operatorname{sgn}(\lambda_2) & \text{if } |\lambda_2| > 2a. \end{cases}$$

Thus, the effective Hamiltonian for this problem is piecewise defined:

$$H(x, \lambda) = \mathcal{H}(x, \lambda, \tau^*) = \begin{cases} -\frac{\lambda_2^2}{4} + \lambda_1 x_2 - \lambda_2 \sin x_1 & \text{if } |\lambda_2| \leq 2a, \\ a^2 - a \operatorname{sgn}(\lambda_2) + \lambda_1 x_2 - \lambda_2 \sin x_1 & \text{if } |\lambda_2| > 2a. \end{cases}$$

Although this Hamiltonian is not differentiable, this does not present any significant difficulties in calculating optimal solutions. H is continuous because clipping is a continuous operation.

Thus, the necessary conditions for optimality are summarized by the two-point boundary value problem (TP-BVP):

$$\begin{cases} \dot{x}_1 = x_2, \\ \dot{x}_2 = \begin{cases} -\frac{\lambda_2}{2} - \sin x_1, & \text{if } |\lambda_2| \leq 2a, \\ -\sin x_1, & \text{if } |\lambda_2| > 2a, \end{cases} \\ \dot{\lambda}_1 = \lambda_2 \cos x_1, \\ \dot{\lambda}_2 = -\lambda_1, \end{cases}$$

with the state boundary conditions:

$$x_1(0) = \theta_0, \quad x_2(0) = \omega_0, \quad x_1(T) = 0, \quad x_2(T) = 0.$$

The costate boundary conditions are free since the final state is prescribed.

The problem can now be formulated as that of finding a solution to a system of two non-linear equations

$$F(z) = 0$$

where $z = (z_1, z_2)$ and we require

$$\lambda_1(0) = z_1, \quad \lambda_2(0) = z_2.$$

are the costate initial conditions, and

$$F(z) = (x_1(T), x_2(T))$$

In other words, for a given initial state, we need to find a complementary costate which will result in the solution attaining the desired state after time $t = T$, precisely.

11.7 Linear-Quadratic Regulator (LQR)

We focus first on the 1-dimensional instance of the LQR problem. This is a system with one state variable $x \in \mathbb{R}$ and one control variable $u \in \mathbb{R}$ satisfying

$$\dot{x} = ax + bu, \quad 0 \leq t < \infty$$

We want to control the system (design a policy) so that the cost

$$J = \int_0^\infty qx^2 + ru^2 dt$$

is minimized ($q, r > 0$).

11.7.1 Solution of LQR in 1D by using PMP

★★★ Step 1: Define the Hamiltonian ★★★

$$H(x, u, \lambda, t) = (qx^2 + ru^2) + \lambda(ax + bu)$$

★★★ Step 2: Find the minimum u^* ★★★ This is easy, as

$$H(x, u, \lambda, t) = ru^2 + \lambda bu + \text{terms not depending on } u$$

is a quadratic function so

$$u^* = -\frac{\lambda b}{2r}$$

★★★ Step 3: Find the effective Hamiltonian ★★★

$$H(x, \lambda) = qx^2 + \lambda ax - \frac{b^2}{4r}\lambda^2$$

★★★ Step 4: Write the equations of motion ★★★

$$\begin{cases} \dot{x} = \partial_\lambda H = ax + bu^* = ax - \frac{b^2}{2r}\lambda \\ \dot{\lambda} = -\partial_x H = -2qx - \lambda a \end{cases}$$

If $\xi = (x, \lambda)$ is the phase plane vector then

$$\dot{\xi} = A\xi$$

where

$$A = \begin{bmatrix} a & -\frac{b^2}{2r} \\ -2q & -a \end{bmatrix}$$

The Hamiltonian is a quadratic function:

$$H = \frac{1}{2} \xi^\top B \xi$$

where

$$B = \begin{bmatrix} 2q & a \\ a & -\frac{b^2}{2r} \end{bmatrix}$$

★★★ **Step 5:** Find the control ★★★ From the initial state $x(0)$ we must find a matching $\lambda(0)$ so that the direction $\xi(0) = (x(0), \lambda(0))$ is an eigenvector of A with eigenvalue $\mu < 0$. Then the solution is

$$\xi(t) = e^{\mu t} \xi(0)$$

and the cost is finite. Else, the solution blows up and the cost is infinite, given $q, r > 0$.

On the similarity between A and B

There is a striking resemblance between the matrix A and B , due to the *symplectic nature* of Hamiltonian systems. Formally:

$$A = J \cdot B$$

where

$$J = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

where $J^2 = -I$ and $J^\top = -J$.

Impact of symplecticity on eigenvalue calculations

Let A, B be two $2n \times 2n$ matrices and let us make these assumptions:

- B is symmetric;
- $A = JB$, where

$$J = \begin{bmatrix} 0 & I_n \\ -I_n & 0 \end{bmatrix}$$

The Hamiltonian system with a quadratic Hamiltonian

$$H(\xi) = \frac{1}{2} \xi^\top B \xi$$

where $\xi = (q, p)$ and $B = B^\top$, is:

$$\dot{\xi} = JB\xi$$

Diagonalization of JB

The matrix $A = JB$, where B is symmetric, satisfies

$$A^\top = J^{-1}(-A)J$$

Proof: $A^\top = (JB)^\top = B^\top J^\top = B(-J) = -BJ = J^{-1}(JB)J = J^{-1}AJ$. As a corollary, matrices $-A$ and $A^\top$ are *conjugate* or *similar*. As A and $A^\top$ are similar, the *characteristic polynomial* of A and $-A$ is the same, that is $p(-\mu) = p(\mu)$. In particular, eigenvalues come in pairs $\{\mu, -\mu\}$ of the same algebraic multiplicity. As matrices A and $A^\top$ have the same Jordan canonical form, the geometric multiplicity for μ and $-\mu$ is also the same.

Remark 11.2 (Similarity vs. Congruence). Matrices A and B are *similar* when $A = S^{-1}BS$ for an invertible matrix S . Matrices A and B are *congruent* when $A = S^TBS$, where S is invertible. Similarity and congruence should not be confused. Only when S is an orthogonal matrix, they coincide. This is important, and somewhat confusing. Because a symmetric matrix *can be diagonalized by an orthogonal matrix*, we get this “coincidence”. It is important to remember that:

- Matrices are similar when they represent the same linear map in two coordinate systems.
- Matrices are congruent when they represent the same quadratic form in two coordinate systems.

□

11.8 The Direct method

In this section, you will prove the fundamental facts about Linear Quadratic Regulator (LQR). The following formulation of the problem is used:

1. Let $x \in \mathbb{R}^n$ be the state vector and $u \in \mathbb{R}^m$ be the control.
2. The state equation is

$$\dot{x}(t) = Ax(t) + Bu(t)$$

where A is an $n \times n$ matrix and B is an $n \times m$ matrix.

3. The cost function is:

$$\mathcal{J} = \frac{1}{2} \int_0^\infty (x(t)^\top Q x(t) + u(t)^\top R u(t)) dt$$

where Q is an $n \times n$ and R is $m \times m$, and both Q, R are *positive definite*.

The goal is to minimize $\mathcal{J}$ over all *policies* $u(t)$, $0 \leq t < \infty$.

★★★ **Step 1:** write the Hamiltonian: ★★★ $\mathcal{H}(x, \lambda, u)$, where $\lambda \in \mathbb{R}^n$ is the *costate*. (Recall that the font is *caligraphic*, to distinguish between the Hamiltonian and effective Hamiltonian.)

$$\mathcal{H}(x, \lambda, u) = \frac{1}{2} (x^\top Q x + u^\top R u) + \lambda^\top (Ax + Bu)$$

★★★ **Step 2:** Find optimal control for given state and costate ★★★

$$u^* = \arg \min_u \mathcal{H}(x, \lambda, u).$$

We differentiate $\mathcal{H}$ over u :

$$\partial_u \mathcal{H}(x, \lambda, u) = u^\top R + \lambda^\top B$$

We solve for u :

$$\begin{aligned} u^\top R + \lambda^\top B &= 0 \\ u^\top &= -\lambda^\top B R^{-1} \\ u &= -R^{-1} B^\top \lambda \end{aligned}$$

Hence,

$$u^* = -R^{-1} B^\top \lambda$$

★★★ **Step 3:** Find the effective Hamiltonian ★★★

$$H(x, \lambda) = \inf_u \mathcal{H}(x, \lambda, u).$$

We plug in u^* for u into $\mathcal{H}$:

$$\begin{aligned} H(x, \lambda) &= \frac{1}{2} (x^\top Q x + (-\lambda^\top B R^{-1}) R (-R^{-1} B^\top \lambda)) \\ &\quad + \lambda^\top A x - \lambda^\top B (R^{-1} B^\top \lambda) \\ &= \frac{1}{2} x^\top Q x + \lambda^\top A x - \frac{1}{2} \lambda^\top B R^{-1} B^\top \lambda. \end{aligned}$$

*** **Step 4:** Find the quadratic form representing the effective Hamiltonian *** If $\xi = (x, \lambda)$, we need to find a symmetric matrix $\tilde{H}$ such that

$$H(\xi) = \frac{1}{2} \xi^\top \tilde{H} \xi.$$

We find by inspection that

$$\tilde{H} = \begin{bmatrix} Q & A^\top \\ A & -B R^{-1} B^\top \end{bmatrix}$$

*** **Step 5:** Find the Hamiltonian system (ODE) associated with the effective Hamiltonian *** We claim that $H(x, \lambda)$ is

$$\dot{\xi} = J \tilde{H} \xi$$

where

$$J = \begin{bmatrix} 0 & I \\ -I & 0 \end{bmatrix}$$

where I is the identity matrix. Indeed, the ODE is:

$$\begin{aligned} \dot{x} &= \nabla_\lambda H = A x - B R^{-1} B^\top \lambda \\ \dot{\lambda} &= -\nabla_x H = -Q x - A^\top \lambda \end{aligned}$$

which in block form is

$$\begin{bmatrix} \dot{x} \\ \dot{\lambda} \end{bmatrix} = \begin{bmatrix} A & -B R^{-1} B^\top \\ -Q & -A^\top \end{bmatrix} \begin{bmatrix} x \\ \lambda \end{bmatrix}$$

Thus we need to check the following identity, via block matrix product:

$$\begin{bmatrix} A & -B R^{-1} B^\top \\ -Q & -A^\top \end{bmatrix} = \begin{bmatrix} 0 & I \\ -I & 0 \end{bmatrix} \begin{bmatrix} Q & A^\top \\ A & -B R^{-1} B^\top \end{bmatrix}$$

*** **Step 6:** Find the solution of Algebraic Riccati Equation *** Let us that $\tilde{H}$ is a non-singular matrix, or equivalently, that the quadratic form

$$H(\xi) = (1/2) \xi^\top \tilde{H} \xi$$

is non-degenerate. Note that non-degeneracy is necessary for stability of the system (in the control theory sense) because degeneracy implies the existence of non-zero, constant solutions (equilibria), which do not converge to zero.

Let $v_1, v_2, \dots, v_s$ (s for stable) be the eigenvectors of $J\tilde{H}$ with negative real parts, and let V_s be the matrix

$$V_s = [v_1 | v_2 | \dots | v_s]$$

It is easy to see that $s = n$, due to eigenvalues coming in pairs $\{-\mu, \mu\}$ if $\mu \in \mathbb{R}$, or quadruples $\{-\mu, -\mu^*, \mu, \mu^*\}$ if μ is complex.

Thus, V_s is $2n \times n$. You may assume for simplicity that $J\tilde{H}$ is diagonalizable, which may not be true in general.

H is non-degenerate iff $C := J\tilde{H}$ is non-singular, given that J is non-singular ($J^2 = -I$ and $J^{-1} = -J$). Thus C has eigenvalues coming in pairs, $\{\mu, -\mu\}$, due to reversibility:

$$C^\top = J(-C)J^{-1} = JCJ$$

Therefore, J maps eigenvectors of $\pm C$ to eigenvectors of $\mp C^\top$.

If C is non-diagonalizable, we need to invoke the properties of the Jordan canonical form. In fact, $C^\top$ and C are similar (conjugate). In more detail, the conjugacy is the permutation matrix that reverses the order of vectors in every *generalized eigenvector chain*. Therefore, C and $-C$ have the same Jordan canonical form (up to permutations of the Jordan blocks). We must modify the definition of V_s , replacing eigenvectors with generalized eigenvectors.

Regardless of diagonalizability, the algebraic and geometric multiplicities of μ and $-\mu$ are the same. Hence, V_s has exactly half the dimension of the entire space $\mathbb{R}^{2n}$.

Let us write

$$V_s = \begin{bmatrix} V_{s1} \\ V_{s2} \end{bmatrix}$$

where V_{sj} is a $n \times n$ matrix. Prove that if for given $x_0 \in \mathbb{R}^n$ we set

$$\lambda_0 = V_{s2} V_{s1}^{-1} x_0$$

and then pair them together, setting $\xi_0 = (x_0, \lambda_0)$, then the IVP:

$$\begin{aligned} \dot{\xi} &= J \tilde{H} \xi, \quad 0 \leq t < \infty \\ \xi(0) &= \xi_0 \end{aligned}$$

has a solution $\xi(t)$ which decays exponentially: for every $\epsilon > 0$

$$\|\xi(t)\| = o(e^{(\mu+\epsilon)t})$$

where μ is the maximum real part of a negative eigenvalue ($\mu < 0$). We have:

$$\begin{aligned} \xi(0) &= \begin{bmatrix} x_0 \\ V_{s2} V_{s1}^{-1} x_0 \end{bmatrix} \\ &= \begin{bmatrix} V_{s1} V_{s1}^{-1} x_0 \\ V_{s2} V_{s1}^{-1} x_0 \end{bmatrix} \\ &= \begin{bmatrix} V_{s1} \\ V_{s2} \end{bmatrix} V_{s1}^{-1} x_0 \\ &= V_s V_{s1}^{-1} x_0 \end{aligned}$$

That is $\xi(0)$ is in the column space of V_s , i.e., is a linear combination of eigenvectors with said eigenvalues. This implies the claimed decay.

★★★ **Step 7:** Find the cost function under optimal policy ★★★ $\mathcal{J}$ under the optimal policy $u(t)$ is a quadratic form

$$\mathcal{J} = \mathcal{J}(x_0) = \frac{1}{2} x_0^\top P x_0$$

where

$$P = V_{s2} V_{s1}^{-1}$$

is a symmetric, positive definite matrix.

This is hard to prove directly. The best way to approach this part is to review the theory, which goes into the connection between Lagrangian and Hamiltonian. The most useful conclusion is that

$$\mathcal{J} = - \int_0^\infty \lambda^\top \dot{x} dt$$

We use the relationship $\lambda = Px$ to find the antiderivative of the integrand. Also note that the Hamiltonian value is constant along the orbit, and in this particular case it is 0.

Referring to section 11.9, and the equation

$$\lambda(t) = Px(t)$$

which holds for all $t \geq 0$ (why?¹):

$$\begin{aligned}
\mathcal{J} &= - \int_0^\infty \lambda^\top \dot{x}(t) dt \\
&= - \int_0^\infty x(t)^\top P \dot{x}(t) dt \\
&= - \int_0^\infty \left(\frac{d}{dt} \frac{1}{2} x(t)^\top P x(t) \right) dt \\
&= - \frac{1}{2} x(t)^\top P x(t) \Big|_{t=0}^{t=\infty} \\
&= \frac{1}{2} x_0^\top P x_0.
\end{aligned}$$

We used the fact that $\lim_{t \rightarrow \infty} x(t) = 0$.

An algorithm to solve ARE

Following the previous discussion

Algorithm 11.1 Direct Method for Solving the Algebraic Riccati Equation

Require: Matrices $A \in \mathbb{R}^{n \times n}$, $B \in \mathbb{R}^{n \times m}$, $Q \in \mathbb{R}^{n \times n}$, $R \in \mathbb{R}^{m \times m}$

Ensure: Matrix $P \in \mathbb{R}^{n \times n}$ that solves the ARE

- 1: Construct the Hamiltonian matrix:

$$H \leftarrow \begin{bmatrix} A & -BR^{-1}B^\top \\ -Q & -A^\top \end{bmatrix}$$

- 2: Compute eigenvalues and eigenvectors of H :

$$[\text{eigvecs}, \text{eigvals}] \leftarrow \text{EigenDecomposition}(H)$$

- 3: Identify stable eigenvalues:
- 4: $\text{stable_indices} \leftarrow \{j : \text{Re}(\text{eigvals}_{jj}) < 0\}$
- 5: Extract the corresponding eigenvectors:
- 6: $\text{stable_eigvecs} \leftarrow \text{eigvecs}(:, \text{stable_indices})$
- 7: Let $n \leftarrow \text{NumberOfRows}(A)$
- 8: Partition:

$$V_{s1} \leftarrow \text{stable_eigvecs}_{1:n,:}, \quad V_{s2} \leftarrow \text{stable_eigvecs}_{n+1:2n,:}$$

- 9: Solve:

$$P \leftarrow V_{s2} V_{s1}^{-1}$$

- 10: **return** P
-

A reference implementation in MATLAB

```

1 function P = solve_are_direct_method(A, B, Q, R)
2     % Solves the Algebraic Riccati Equation (ARE) using the Direct Method.
3     %
4     % Parameters:
5     % - A, B: System matrices.
6     % - Q, R: Weight matrices for the state and input in the cost function.
7     %
8     % Returns:

```

¹Because a solution that starts in the stable subspace remains in it forever, and the equation $\lambda = Px$ is the equation of the stable subspace.

```

9      % - P: Solution to the ARE.
10
11     % Construct the Hamiltonian matrix
12     H = [A, -B * inv(R) * B'; -Q, -A'];
13
14     % Compute eigenvalues and eigenvectors
15     [eigvecs, eigvals] = eig(H);
16
17     % Filter the stable eigenvectors (eigenvalues with negative real parts)
18     stable_indices = real(diag(eigvals)) < 0;
19     stable_eigvecs = eigvecs(:, stable_indices);
20
21     % Partition the stable eigenvectors
22     n = size(A, 1); % Number of states
23     V_s1 = stable_eigvecs(1:n, :);
24     V_s2 = stable_eigvecs(n+1:end, :);
25
26     % Compute the solution to the ARE
27     P = V_s2 / V_s1;
28
29     % Return the solution
30 end

```

Listing 11.1: Matlab solution to solving algebraic Ricatti equation (ARE), using direct method

11.9 Additional Control Theory Background

The quick way to understand the form of the Hamiltonian, and why it results in an optimal solution, is to start with the corresponding Lagrangian.

The Hamiltonian (for control theory) is:

$$\mathcal{H}(x, \lambda, u, t) = f_0(x, u, t) + \lambda^\top f(x, u, t)$$

when finding the minimum cost

$$\mathcal{J} = \int_{t_0}^{t_1} f_0(x, u, t) dt$$

subject to an ODE constraint $\dot{x} = f(x, u, t)$. The Lagrangian and Hamiltonian are related by Legendre transform (*duality*):

$$\mathcal{H}(x, \lambda, u, t) = \sup_{\dot{x}} (\lambda^\top \dot{x} - \mathcal{L}(x, \dot{x}, u, t))$$

and

$$\mathcal{L}(x, \dot{x}, u, t) = \sup_{\lambda} (\lambda^\top \dot{x} - \mathcal{H}(x, \lambda, u, t))$$

Since we started with the Hamiltonian, we will use the second relation to retrieve the Lagrangian:

$$\begin{aligned} \mathcal{L}(x, \dot{x}, u, t) &= \sup_{\lambda} (\lambda^\top \dot{x} - f_0(x, u, t) - \lambda^\top f(x, u, t)) \\ &= \sup_{\lambda} (-f_0(x, u, t) + \lambda^\top (\dot{x} - f(x, u, t))) \end{aligned}$$

This formula does not make sense because it yields ∞ , unless $\dot{x} = f(x, u, t)$ is satisfied. But we could restrict λ to a nice convex set, e.g., a ball of radius ϵ , to get a finite expression.

A regularization trick

Let us modify the Hamiltonian accordingly (where we omitted the arguments that do not participate in calculations, *for clarity*):

$$\mathcal{H} = f_0 + \lambda^\top f - \frac{\epsilon}{2} \lambda^\top \lambda$$

Then the Lagrangian is

$$\mathcal{L} = \sup_{\lambda} (\lambda^\top \dot{x} - \mathcal{H})$$

The optimal value of λ is found by differentiating $\lambda^\top \dot{x} - \mathcal{H}$ over λ :

$$0 = \dot{x} - \partial_{\lambda} \mathcal{H} = \dot{x} - f + \epsilon \lambda$$

and the optimal λ is

$$\lambda^* = -\frac{1}{\epsilon}(\dot{x} - f)$$

We then find the Lagrangian:

$$\begin{aligned} \mathcal{L} &= -\frac{1}{\epsilon}(\dot{x} - f)^\top \dot{x} - f_0 + \frac{1}{\epsilon}(\dot{x} - f)^\top f + \frac{1}{2} \frac{1}{\epsilon}(\dot{x} - f)^\top (\dot{x} - f) \\ &= -\frac{1}{\epsilon}(\dot{x} - f)^\top (\dot{x} - f) - f_0 + \frac{1}{2} \frac{1}{\epsilon}(\dot{x} - f)^\top (\dot{x} - f) \\ &= -f_0 - \frac{1}{2\epsilon} \|\dot{x} - f\|^2. \end{aligned}$$

Let us consider the problem of *maximizing*

$$\int_{t_0}^{t_1} \mathcal{L}(x, \lambda, u, t) dt = - \int_{t_0}^{t_1} f_0(x, u, t) dt - \frac{1}{2\epsilon} \int_{t_0}^{t_1} \|\dot{x} - f(x, u, t)\|^2 dt$$

As the second term imposes a penalty for *not satisfying* the ODE, we thus in one shot minimize cost and strive to satisfy the ODE. Clearly, we need to pass to the limit as $\epsilon \rightarrow 0$. The Lagrangian then ceases to exist, but the Hamiltonian converges nicely (it drops its regularization term).

Conclusion

For any fixed ϵ , to minimize cost, we need to *minimize* the Hamiltonian over the control parameter u . Therefore, we expect three things:

- The optimal policy minimizes the Hamiltonian.
- The optimal state evolution to be satisfied with the effective Hamiltonian.
- We expect the regularized problem solution to converge to the solution for $\epsilon = 0$.

This is not a proof, but an *excellent heuristic* helping with understanding PMP (Pontryagin Maximum/Minimum Principle).

Optimal cost and action

The important identity, which follows from Hamiltonian mechanics and defines *the action integral*, is:

$$\int_{t_0}^{t_1} \mathcal{L}(x, \dot{x}, u, t) dt = \int_{\gamma} (\lambda^\top dx - H dt)$$

where γ is the solution (path) of the Hamiltonian system. In the specific situation (PMP) we have

$$- \int_{t_0}^{t_1} f_0(x, u, t) dt = \int_{\gamma} (\lambda^\top dx - H dt)$$

which means that the *negative cost* is the value of the action integral.

Furthermore, if the system is autonomous (no explicit time dependency in $\mathcal{L}$) then H is constant along the solution. Therefore, the negative cost is

$$\int_{t_0}^{t_1} \lambda^\top \dot{x} dt - H(t_1 - t_0)$$

If $H = 0$ then the cost is

$$\int_{t_0}^{t_1} \lambda^\top \dot{x} dt$$

The “direct” solution of the LQR problem

We move on to discuss the general LQR problem, with an arbitrary number of state variables and controls.

One method to solve LQR is known as the *direct method*. The second popular method is via the *Ricatti Equation*. The direct method has a close relationship to the methods of *Dynamical Systems*.

The Objective

To find a solution $\xi(t) = (q(t), p(t))$ of a Hamiltonian system with a quadratic Hamiltonian

$$H = \frac{1}{2} \xi^\top B \xi$$

with initial condition $q(0) = q_0$ such that

$$\lim_{t \rightarrow \infty} (q(t), p(t)) = 0$$

and the approach to 0 is exponentially fast.

Solution

Let $A = JB$ be the matrix of the Hamiltonian ODE $\dot{\xi} = A\xi$. We find a decomposition

$$\mathbb{R}^{2n} = W^s \oplus W^c \oplus W^u$$

such that W^s, W^c, W^u are the invariant subspaces corresponding to eigenvalues with negative, positive and 0 real part, respectively. We find p_0 such that $(q_0, p_0) \in W^s$. This is always possible when $W^c = \{0\}$. If not possible, we fail, else we have a solution.

Detailed Solution

★★★ **Step 1:** Construct the system matrix ★★★

$$A = \begin{bmatrix} a & -\frac{b^2}{2r} \\ -2q & -a \end{bmatrix}$$

★★★ **Step 2:** Find the characteristic polynomial ★★★

$$\det(A - \mu I) = (a - \mu)(-a - \mu) - \frac{qb^2}{r} = 0$$

$$\mu^2 - a^2 - \frac{qb^2}{r} = 0$$

$$\mu = \pm \sqrt{a^2 + \frac{qb^2}{r}}$$

★★★ Step 3: Find the relevant eigenvalue (the negative one) ★★★

$$\mu = -\sqrt{a^2 + \frac{qb^2}{r}}$$

★★★ Step 4: Find the stable eigenvector ★★★

$$-2qx + (-a - \mu)\lambda = 0$$

$$\lambda = -\frac{2q}{a + \mu}x$$

11.10 Symplecticity

The symplectic form and its preservation

Formula for a *bilinear, skew-symmetric, non-degenerate* 2-form $\omega : V \times V \rightarrow \mathbb{R}, V = \mathbb{R}^n$:

$$\omega = \sum_{i=1}^n dq_i \wedge dp_i$$

Formula on the vectors of the standard basis:

$$\omega(\partial_{q_i}, \partial_{p_j}) = \delta_{ij}$$

The form is skew-symmetric (anti-symmetric):

$$\omega(\partial_{p_i}, \partial_{q_j}) = -\omega(\partial_{q_j}, \partial_{p_i}) = -\delta_{ij}$$

All other “symplectic dot products” are 0. Symplectic product representation:

$$\omega(\xi, \eta) = \xi^\top J \eta$$

where

$$J = \begin{bmatrix} 0 & I_n \\ -I_n & 0 \end{bmatrix}$$

$J^2 = -I$, which makes it similar to the *purely imaginary unit j* (in most math books *i*). This is *not a superficial connection*. Complex manifolds (e.g., *quaternions*) are a rich source of symplectic manifolds, useful in solving mechanics problems.

Hamiltonian systems preserve symplectic form

- Let $\dot{q} = H_p, \dot{p} = -H_q$.
- The variations $(\xi, \eta) = (\delta q, \delta p)$ satisfy

$$\dot{\xi} = H_{pq}\xi + H_{pp}\eta$$

$$\dot{\eta} = -H_{qp}\xi - H_{qq}\eta$$

- The symplectic product of the variations along the solution satisfies

$$\begin{aligned} \frac{d}{dt}\omega(\xi, \eta) &= \omega(\dot{\xi}, \eta) + \omega(\xi, \dot{\eta}) \\ &= \omega(H_{pq}\xi + H_{pp}\eta, \eta) + \omega(\xi, -H_{qp}\xi - H_{qq}\eta) \\ &= H_{pq}\omega(\xi, \eta) + H_{pp}\omega(\eta, \eta) - H_{qp}\omega(\xi, \xi) - H_{qq}\omega(\xi, \eta) \end{aligned}$$

- We have $\omega(\xi, \xi) = \omega(\eta, \eta) = 0$. We also have $H_{pq} = H_{qp}$ (equality of mixed partials). Therefore

$$\frac{d}{dt}\omega(\xi, \eta) = 0$$

Q.E.D.

Hamiltonian vector field as “symplectic gradient”

The relationship between the Hamiltonian vector field $X = (H_p, H_q)$ or, more elaborately

$$X_H = H_p \frac{\partial}{\partial q} - H_q \frac{\partial}{\partial p}$$

and the Hamiltonian $H(p, q)$ is analogous to the relationship between the the gradient and a function, in the presence of a metric tensor:

$$dH(\xi)Y = \omega(X_H, Y)$$

where Y is any vector field. If we write $\omega = g_{ij}$, we can write

$$X_H = g^{ij} H_j \partial_i$$

Compare this to

$$\nabla H = g^{ij} H_j \partial_i$$

when g_{ij} is a metric tensor. The difference is that for the symplectic case $g_{ij} = -g_{ji}$ and for the metric case $g_{ij} = g_{ji}$. The symplectic tensor ω on a $2n$ -dimensional manifold M^{2n} can *always* be written in local coordinates in the form $\omega = dq \wedge dp$ by the *Darboux theorem*. The matrix form of the symplectic tensor is the matrix J . As $J^2 = -I$, J is its own inverse; in particular $g^{ij} = g_{ij}$. Useful identities:

$$X_H = J \cdot (dH)^\top, \quad X_H^\top J = dH$$

Loosely speaking, $(dH)^\top = \nabla H$ if we use the Euclidean metric structure, so

$$X_H = J \cdot \nabla H$$

which says that X_H is the “gradient of H rotated by 90 degrees”. We also mention an identity relating the Euclidean metric and the symplectic form on $\mathbb{R}^{2n}$:

$$\omega(\xi, \eta) = \langle J\xi, \eta \rangle$$

A note on ‘,’ and ‘,’

- I previously used ‘,’ to mark beginning of indices that denote the partial derivatives, this *will be changed to ‘,’*, following newer textbooks, e.g., [5].
- The symbol ‘,’ is used for the *covariant derivative* of a tensor, e.g., a vector field (see formula 5.24)

$$A^i_{;j} = \frac{\partial A^i}{\partial x^j} + A^k \Gamma^i_{kj}$$

- A more austere notation uses ‘,’ for partials:

$$A^i_{,j} = \frac{\partial A^i}{\partial x^j} + A^k \Gamma^i_{kj}$$

- Covariant differentiation is differentiation using *parallel transport*, i.e., moving a vector around a Riemannian manifold keeping as many properties of the familiar parallel translation in vector and Euclidean spaces.

Symplectic Manifold

A **symplectic manifold** is a smooth manifold, M , equipped with a closed, nondegenerate 2-form ω , known as a symplectic form. The condition that ω is closed means that its exterior derivative $d\omega = 0$, and nondegenerate means that for any nonzero vector v at any point in M , there exists another vector w such that $\omega(v, w) \neq 0$. This ensures that ω pairs vectors in the tangent space at each point in a nontrivial way, leading to the manifold having an even dimension.

Darboux's Theorem

The **Darboux Theorem**, or the **Darboux Normal Form Theorem**, states that locally, all symplectic manifolds are isomorphic to the standard symplectic space. Specifically, given any point p in a symplectic manifold (M, ω) , there exists a local coordinate chart (U, ϕ) around p such that for any point $q \in U$, the symplectic form ω can be expressed as

$$\omega = \sum_{i=1}^n dq_i \wedge dp_i$$

in these local coordinates $(q_1, p_1, \dots, q_n, p_n)$, where n is half the dimension of M . This expression is known as the standard symplectic form on $\mathbb{R}^{2n}$.

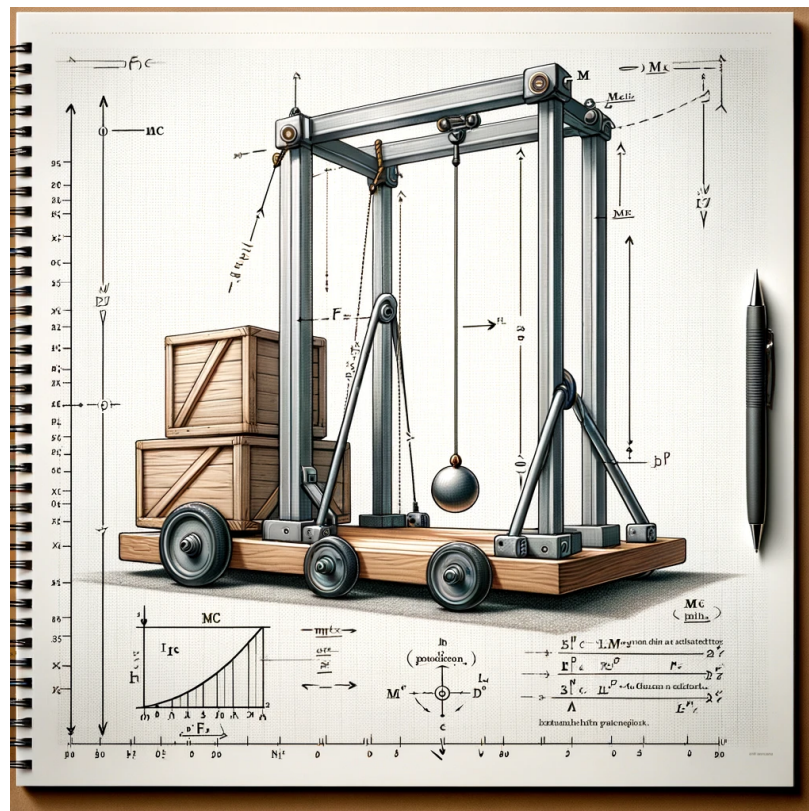
This theorem highlights that despite the global complexity and variety of symplectic structures, they all look locally like the standard symplectic space, implying that differences lie in their global properties.

11.11 Pendulum on a Cart

Introduction

The *Inverted Pendulum on a Cart* system is a classic problem in control theory. **Objective:** Stabilize the pendulum in the upright position while controlling the cart's position.

Setup Illustration



System Dynamics

The equations of motion for the system can be derived using Lagrangian mechanics. The state variables are:

- x : Cart position
- $\dot{x}$: Cart velocity
- θ : Pendulum angle from the vertical
- $\dot{\theta}$: Pendulum angular velocity

Equations of Motion

The linearized equations around the upright position are:

$$\ddot{x} = \frac{F + m_p l \dot{\theta}^2 \sin \theta - m_p g \sin \theta \cos \theta}{m_c + m_p \sin^2 \theta} \quad (11.2)$$

$$\ddot{\theta} = \frac{g \sin \theta - \ddot{x} \cos \theta}{l} \quad (11.3)$$

where:

- m_c : Mass of the cart
- m_p : Mass of the pendulum
- l : Length to the pendulum's center of mass
- g : Acceleration due to gravity
- F : Control force applied to the cart

Linear Quadratic Regulator (LQR)

The LQR controller is designed by defining a cost function that the controller minimizes. The typical cost function is

$$J = \int_0^{\infty} (\mathbf{x}^T Q \mathbf{x} + u^T R u) dt$$

where $\mathbf{x} = [x, \dot{x}, \theta, \dot{\theta}]^T$ is the state vector, u is the control input, and Q, R are the weight matrices.

Controller Design

The goal is to compute the optimal feedback gain K that minimizes the cost function, leading to the control law:

$$u = -K\mathbf{x}$$

where $\mathbf{x}$ is the state vector.

Conclusion

The LQR controller effectively stabilizes the inverted pendulum on a cart by optimizing the control force applied. This problem exemplifies the application of control theory to manage unstable systems.

11.12 Inverted Pendulum on a Cart: Controller Design

Linearization of the System

Linearization is the process of approximating a nonlinear system by a linear one around a specific operating point. For the inverted pendulum:

- Operating point: $\theta = 0$ (upright position) and $\dot{\theta} = 0$ (no angular velocity).
- Linearized equations focus on small deviations from this point.

State-Space Representation

After linearization, the system can be represented in state-space form:

$$\begin{aligned}\dot{\mathbf{x}} &= A\mathbf{x} + Bu \\ y &= C\mathbf{x} + Du\end{aligned}$$

where:

$$\mathbf{x} = [x \quad \dot{x} \quad \theta \quad \dot{\theta}]^T$$

is the state vector. The control input is $u = F$ is the force acting on the cart. The matrices A , B , C , and D are derived by linearizing the equations of motion.

Designing the LQR Controller

The Linear Quadratic Regulator (LQR) aims to minimize a cost function of the form:

$$J = \int_0^{\infty} (\mathbf{x}^T Q \mathbf{x} + u^T R u) dt$$

where Q and R are matrices defining the weight of states and control effort in the cost function.

Optimal Gain Calculation

The optimal feedback gain K is calculated by solving the Riccati equation:

$$A^T P + P A - P B R^{-1} B^T P + Q = 0$$

where P is a positive definite matrix. The control law is then:

$$u = -K\mathbf{x}$$

with $K = R^{-1} B^T P$.

Conclusion

- LQR provides a systematic approach to designing controllers for linearized systems.
- The inverted pendulum on a cart is a classic example demonstrating the application of LQR in controlling an inherently unstable system.

11.13 Choosing Q and R in LQR Control

The LQR Cost Function

The cost function in LQR control is given by:

$$J = \int_0^{\infty} (\mathbf{x}^T Q \mathbf{x} + u^T R u) dt$$

- J : Cost function to minimize.
- $\mathbf{x}$: State vector.
- u : Control input.
- Q : Weight matrix for the states.
- R : Weight matrix for the control effort.

Matrix Q

- Symmetric, positive semi-definite.
- Dimensions match the state vector ($n \times n$ for n states).
- Weights the importance of each state in the cost function.
- Higher values indicate a greater penalty for deviations in the corresponding state.

Matrix R

- Symmetric, positive definite.
- Dimensions match the control input ($m \times m$ for m inputs).
- Weights the importance of minimizing control effort.
- Higher values discourage aggressive control actions, leading to less energy use but potentially slower response.

Choosing Q and R

- Balancing control accuracy with effort/energy use.
- Initial tuning often starts with diagonal matrices for simplicity.
- Iterative process: Adjust Q and R , simulate, evaluate, and refine.
- The choice depends on specific system requirements and desired performance criteria.

Example: Inverted Pendulum

For an inverted pendulum on a cart:

- Prioritize pendulum stability (θ and $\dot{\theta}$) with higher weights in Q .
- If minimizing energy is crucial, increase the value of R to reduce control effort.

Conclusion

- The choice of Q and R is crucial for the performance of the LQR controller.
- Requires a careful balance between system performance and control effort.
- Iterative tuning and simulation are key to achieving optimal control.

11.14 Hamilton-Jacobi-Bellman Equations

Introduction

The Hamilton-Jacobi-Bellman (HJB) equation is a fundamental principle in control theory and dynamic programming. It provides a necessary condition for optimality, applicable to a broad range of optimization problems.

Hamiltonian

The Hamiltonian is given by:

$$H(x, u, \nabla V(x, t), t) = L(x, u, t) + \nabla V(x, t) \cdot f(x, u, t)$$

where $V(x, t)$ is the value function, representing the minimum cost starting from state x at time t .

Hamilton-Jacobi-Bellman Equation

The HJB equation can be formulated as:

$$-\frac{\partial V(x, t)}{\partial t} = \min_u [L(x, u, t) + \nabla V(x, t) \cdot f(x, u, t)]$$

This partial differential equation must be solved with appropriate boundary conditions to determine the optimal policy $u^*(t)$ and the value function $V(x, t)$.

11.15 Solving the Pendulum Control Problem

Overview

- Objective: Apply torque to stop a swinging pendulum with minimal energy expenditure.
- Approaches: Pontryagin Maximum Principle (PMP) and Hamilton-Jacobi-Bellman (HJB) equation.

Pontryagin Maximum Principle (PMP)

1. Construct the Hamiltonian:

$$\mathcal{H} = \tau^2 + \lambda_1 \omega + \lambda_2 \left(\frac{g}{l} \sin(\theta) + \frac{\tau}{ml^2} \right)$$

2. Optimal control condition:

$$\tau^* = -\frac{\lambda_2}{2ml^2}$$

3. System of differential equations:

- State equations and costate equations.
- Boundary conditions: $\theta(0), \omega(0), \theta(T) = 0, \omega(T) = 0$.

4. Solve the system numerically.

Hamilton-Jacobi-Bellman (HJB) Equation

1. Formulate the HJB equation for the value function $V(\theta, \omega, t)$:

$$-\frac{\partial V}{\partial t} = \min_{\tau} \left(\tau^2 + \frac{\partial V}{\partial \theta} \omega + \frac{\partial V}{\partial \omega} \left(\frac{g}{l} \sin(\theta) + \frac{\tau}{ml^2} \right) \right)$$

2. Solve the HJB equation numerically with appropriate terminal conditions.
3. Derive the optimal control policy τ^* from the minimization condition.

Challenges and Computational Methods

- Nonlinear dynamics of the pendulum pose a significant challenge.
- Both PMP and HJB approaches often require numerical methods for solution.
- The choice of method depends on specific problem constraints and available computational resources.

11.16 Energy-Realistic Control of a Swinging Pendulum

Introduction

- Objective: Modify the control problem to minimize actual energy (work) expenditure in stopping a swinging pendulum.
- Rationale: The square of torque (τ^2) provides a smooth control strategy but does not directly quantify physical work or energy.

Physical Work as Objective

- Work done by torque: $W = \int \tau(t) d\theta(t)$
- Objective function representing energy:

$$J = \int_0^T |\tau(t) \cdot \dot{\theta}(t)| dt$$

- This formulation directly ties to the physical concept of energy, penalizing the actual work done by the applied torque.

Mathematical Formulation

Minimize the energy-expended objective function:

$$\min_{\tau(t)} J = \int_0^T |\tau(t) \cdot \dot{\theta}(t)| dt$$

Subject to the pendulum dynamics:

$$\frac{d^2\theta}{dt^2} + \frac{g}{l} \sin(\theta) = \frac{\tau(t)}{ml^2}$$

And boundary conditions:

$$\begin{aligned} \theta(0) &= \theta_0, & \dot{\theta}(0) &= \omega_0 \\ \theta(T) &= 0, & \dot{\theta}(T) &= 0 \end{aligned}$$

Practical Considerations

- This energy-realistic objective may increase the complexity of the optimization problem.
- Numerical methods may be heavily relied upon due to the nonlinearity and potential non-convexity of the objective function.
- Solutions derived from this formulation are expected to be more aligned with physical efficiency and practical applicability.

11.17 Derivation of the Hamilton-Jacobi-Bellman (HJB) Equation

System Dynamics

Consider a nonlinear control system described by:

$$\frac{dx}{dt} = f(x(t), u(t), t)$$

where $x(t)$ is the state vector, $u(t)$ is the control input, and t is the time variable.

Performance Index

The goal is to find an optimal control law $u^*(t)$ that minimizes (or maximizes) the performance index J :

$$J = g(x(T)) + \int_0^T L(x(t), u(t), t) dt$$

where $g(x(T))$ is the terminal cost/reward, and $L(x(t), u(t), t)$ is the running cost/reward.

Value Function

Define the value function $V(x(t), t)$ as the optimal cost-to-go from time t to the final time T :

$$V(x(t), t) = \min_{u(t)} \left[\int_t^T L(x(\tau), u(\tau), \tau) d\tau + g(x(T)) \right]$$

subject to the system dynamics $\frac{dx}{dt} = f(x(t), u(t), t)$.

Principle of Optimality

According to the principle of optimality, the value function satisfies:

$$V(x(t), t) = \min_{u(t)} [L(x(t), u(t), t)dt + V(x(t+dt), t+dt)]$$

The principle of optimality is easily understood: to achieve optimal cost starting at $x(t)$ at time t , we need to get through point $x(t+dt)$ at time $t+dt$, at which time we must also follow the optimum path on the time interval $[t+dt, T]$.

HJB Equation

Expanding $V(x(t+dt), t+dt)$ using Taylor series approximation and taking the limit as $dt \rightarrow 0$, we obtain the HJB equation:

$$0 = \min_{u(t)} \left[L(x(t), u(t), t) + \frac{\partial V}{\partial t} + \frac{\partial V}{\partial x} f(x(t), u(t), t) \right]$$

The expression

$$\mathcal{H}(x(t), \lambda(t), u(t), t) = L(x(t), u(t), t) + \frac{\partial V}{\partial x} f(x(t), u(t), t)$$

is the familiar PMP Hamiltonian, provided that the costate equation is

$$\lambda(t) = \frac{\partial V}{\partial x}(x(t), t).$$

Therefore, λ is the *marginal cost*, i.e., the derivative of the cost-to-go.

Minimization over $u(t)$ leads to the effective Hamiltonian $H(x, \lambda, t)$, yielding the final form of HJB:

$$\frac{\partial V}{\partial t} + H\left(x, \frac{\partial V}{\partial x}, t\right) = 0.$$

Remark 11.3 (On vectors and covectors). In the above derivation, the partial $\frac{\partial V}{\partial x}$ is a *covector*, and is written as a *row vector*. It is common in engineering literature to use a gradient of $V(x, t)$, but this does not correctly reflect the nature of the covector, which is a *functional* on $\mathbb{R}^n$, not an element of $\mathbb{R}^n$. Why does it matter? Because HJB does not depend on the choice of the inner product on $\mathbb{R}^n$. In particular, a linear or even non-linear change of variables (not changing time) preserves the form of HJB. We note that

$$H : \mathbb{R}^n \times (\mathbb{R}^n)^* \times \mathbb{R} \rightarrow \mathbb{R}.$$

More generally, one can write an HJB equation on a manifold M . Then

$$H : T^*M \rightarrow \mathbb{R}$$

where T^*M is the *cotangent bundle*, of M . For example, we have the cotangent bundle of $\mathbb{R}^n$:

$$T\mathbb{R}^n = \mathbb{R}^n \times (\mathbb{R}^n)^*.$$

The mapping

$$\mathbb{R}^n \ni x \mapsto \left(x, \frac{\partial V}{\partial x} \right) \in T\mathbb{R}^n$$

is a *section* of the cotangent bundle, and, loosely speaking, when solving for $V(x, t)$, we are solving for a section of the cotangent bundle. $\square$

Time-Independent HJB Equation

When the system is time-independent and the terminal time T is free, but $x(T) = x_f$ is fixed, the cost-to-go function is time-independent. The arguments of the previous section with the obvious modifications yield:

$$H\left(x, \frac{\partial V}{\partial x}\right) = 0.$$

In particular, we have

$$H(x(T), \lambda(T)) = 0.$$

So, the Hamiltonian must be 0 in the terminal state. As the effective Hamiltonian is an integral of motion, it must be 0 for all times along the optimal solution:

$$H(x(t), \lambda(t)) = 0, \quad 0 \leq t \leq T.$$

Conclusion

The HJB equation provides a necessary condition for optimality. Its solution yields the optimal value function $V(x(t), t)$ and the corresponding optimal control law $u^*(t)$ that minimizes (or maximizes) the performance index J .

11.18 Derivation of Pontryagin Maximum Principle

The Pontryagin Maximum Principle (PMP) is a foundational result in optimal control theory. It provides necessary conditions that an optimal control must satisfy to optimize a given performance criterion.

Problem Statement

Consider a control system described by the differential equation:

$$\dot{x}(t) = f(x(t), u(t), t)$$

where $x(t) \in \mathbb{R}^n$ is the state, $u(t) \in \mathcal{U}$ is the control input, and f describes the system dynamics. The objective is to find a control $u(t)$ that optimizes a performance criterion:

$$J(u) = \int_{t_0}^{t_f} L(x(t), u(t), t) dt$$

Derivation of Pontryagin Maximum Principle

To include state equations in the optimization problem, we introduce the augmented performance criterion using the Hamiltonian:

$$\mathcal{L}(x, u, \lambda) = \int_{t_0}^{t_f} [L(x(t), u(t), t) + \lambda(t)^\top (\dot{x}(t) - f(x(t), u(t), t))] dt$$

This allows integrating the system dynamics directly into the optimization framework, facilitating the derivation of necessary conditions for optimality.

Hamiltonian and Necessary Conditions The Hamiltonian is defined as:

$$\mathcal{H}(x, u, \lambda, t) = L(x, u, t) + \lambda^\top f(x, u, t)$$

Effective Hamiltonian: Define the Hamiltonian H as:

$$H(x, \lambda, t) = \max_{u \in \mathcal{U}} \mathcal{H}(x, u, \lambda, t)$$

Costate Equation: The costate $\lambda(t)$ evolves according to:

$$\dot{\lambda}(t) = -\frac{\partial H}{\partial x}$$

Optimality Condition: The control $u(t)$ must maximize (or minimize) H for all t :

$$u^*(t) = \arg \max_{u \in \mathcal{U}} \mathcal{H}(x, u, \lambda, t)$$

Transversality Conditions: At the initial and final times, the state $x(t)$ and costate $\lambda(t)$ must satisfy certain boundary conditions.

11.19 Derivation of PMP via regularization

We restate the problem introduced at the beginning of the Chapter, slightly modifying the notation to make it more mathematically (more) rigorous, rather than following the engineering tradition as we did so far.

Introduction to understanding PMP

The quick way to understand the form of the Hamiltonian, and why it results in an optimal solution, is to start with the corresponding Lagrangian. The Hamiltonian (for control theory) is:

$$\mathcal{H}(x, \lambda, u, t) = f_0(x, u, t) + \lambda^\top f(x, u, t)$$

when finding the minimum cost

$$\mathcal{J} = \int_{t_0}^{t_1} f_0(x, u, t) dt$$

subject to an ODE constraint $\dot{x} = f(x, u, t)$.

Lagrangian, Hamiltonian and Legendre transform

The Lagrangian and Hamiltonian are related by Legendre transform (*duality*):

$$\mathcal{H}(x, \lambda, u, t) = \sup_{\dot{x}} (\lambda^\top \dot{x} - \mathcal{L}(x, \dot{x}, u, t))$$

and

$$\mathcal{L}(x, \dot{x}, u, t) = \sup_{\lambda} (\lambda^\top \dot{x} - \mathcal{H}(x, \lambda, u, t))$$

Since we started with the Hamiltonian, we will use the second relation to retrieve the Lagrangian:

$$\begin{aligned} \mathcal{L}(x, \dot{x}, u, t) &= \sup_{\lambda} (\lambda^\top \dot{x} - f_0(x, u, t) - \lambda^\top f(x, u, t)) \\ &= \sup_{\lambda} (-f_0(x, u, t) + \lambda^\top (\dot{x} - f(x, u, t))) \end{aligned}$$

This formula does not make sense because it yields ∞ , unless $\dot{x} = f(x, u, t)$ is satisfied. But we could restrict λ to a nice convex set, e.g., a ball of radius ϵ , to get a finite expression.

The regularization trick

Let us modify the Hamiltonian accordingly, where we omitted the arguments that do not participate in calculations, *for clarity*:

$$\mathcal{H} = f_0 + \lambda^\top f - \frac{\epsilon}{2} \lambda^\top \lambda$$

Then the Lagrangian is

$$\mathcal{L} = \sup_{\lambda} (\lambda^\top \dot{x} - \mathcal{H})$$

The optimal value of λ is found by differentiating $\lambda \dot{x} - \mathcal{H}$ over λ :

$$0 = \dot{x} - \partial_{\lambda} \mathcal{H} = \dot{x} - f + \epsilon \lambda$$

and the optimal λ is

$$\lambda^* = -\frac{1}{\epsilon}(\dot{x} - f)$$

We then find the Lagrangian:

$$\begin{aligned} \mathcal{L} &= -\frac{1}{\epsilon}(\dot{x} - f)^\top \dot{x} - f_0 \\ &\quad + \frac{1}{\epsilon}(\dot{x} - f)^\top f + \frac{1}{2} \frac{1}{\epsilon}(\dot{x} - f)^\top (\dot{x} - f) \\ &= -f_0 - \frac{1}{2\epsilon} \|\dot{x} - f\|^2. \end{aligned}$$

Let us consider the problem of *maximizing*

$$\int_{t_0}^{t_1} \mathcal{L}(x, \lambda, u, t) dt = - \int_{t_0}^{t_1} f_0(x, u, t) dt - \frac{1}{2\epsilon} \int_{t_0}^{t_1} \|\dot{x} - f(x, u, t)\|^2 dt$$

As the second term imposes a penalty for *not satisfying* the ODE, we thus in one shot minimize cost and strive to satisfy the ODE.

Clearly, we need to pass to the limit as $\epsilon \rightarrow 0$. The Lagrangian then ceases to exist, but the Hamiltonian converges nicely (it drops its regularization term).

Conclusion

For any fixed ϵ , to minimize cost, we need to *minimize* the Hamiltonian over the control parameter u . Therefore, we expect a number of things to be true:

- The optimal policy minimizes the Hamiltonian.
- The optimal state evolution to be satisfied with the effective Hamiltonian.
- We expect the regularized problem solution to converge to the solution for $\epsilon = 0$.

This is not a (full) proof, but an *excellent heuristic* helping with understanding PMP (Pontryagin Maximum/Minimum Principle).

Optimal cost and action

The important identity, which follows from Hamiltonian mechanics and defines *the action integral*, is:

$$\int_{t_0}^{t_1} \mathcal{L}(x, \dot{x}, u, t) dt = \int_{\gamma} (\lambda^\top dx - H dt)$$

where γ is the solution (path) of the Hamiltonian system.

In the specific situation (PMP) we have

$$-\int_{t_0}^{t_1} f_0(x, u, t) dt = \int_Y (\lambda^\top dx - H dt)$$

which means that the *negative cost* is the value of the action integral.

If the system is autonomous (no explicit time dependency in $\mathcal{L}$) then H is constant along the solution. Therefore, the negative cost is

$$\int_{t_0}^{t_1} \lambda^\top \dot{x} dt - H(t_1 - t_0)$$

If $H = 0$ then the cost is

$$\int_{t_0}^{t_1} \lambda^\top \dot{x} dt$$

11.20 Optimal Control with Saturated Input: Scalar Linear System

We consider the infinite-horizon optimal control problem for a scalar linear system with one state and one control input, subject to a bound on the control magnitude. The goal is to minimize a quadratic cost subject to the system dynamics and control constraints.

Problem Setup

The system dynamics are given by:

$$\dot{x}(t) = ax(t) + bu(t), \quad x(0) = x_0 \in \mathbb{R},$$

where $u(t) \in [-1, 1]$ is the control input. The cost functional is:

$$J(u) = \frac{1}{2} \int_0^\infty (x(t)^2 + u(t)^2) dt.$$

This is a non-quadratic control problem due to the saturation constraint $|u(t)| \leq 1$; hence it is not solvable by classical LQR techniques.

Structure of the Optimal Policy

Using Pontryagin's Maximum Principle or the Hamilton–Jacobi–Bellman (HJB) approach, one can show that the optimal control takes the feedback form

$$u^*(x) = \text{sat}(-bPx),$$

where $\text{sat}(z) = \max(-1, \min(z, 1))$, and the gain P is given by

$$P = \frac{a + \sqrt{a^2 + b^2}}{b^2}.$$

Let us define the switching threshold:

$$x_s := \frac{1}{bP},$$

so that the control is linear within the interval $|x| \leq x_s$, and saturated outside. Specifically, the policy is

$$u^*(x) = \begin{cases} -1 & x > x_s, \\ -bPx & |x| \leq x_s, \\ +1 & x < -x_s. \end{cases}$$

Value Function

The optimal cost $V(x_0)$ depends on the initial state and falls into three regimes:

1. Linear Region ($|x_0| \leq x_s$) In this case, the control remains unsaturated for all time. The value function is quadratic:

$$V(x_0) = \frac{1}{2}Px_0^2.$$

2. Saturated Region ($x_0 > x_s$) Let us define

$$A = x_0 - \frac{b}{a}, \quad x_s = \frac{1}{bP}, \quad \Delta = \frac{1}{a} \ln \left(\frac{x_s - \frac{b}{a}}{x_0 - \frac{b}{a}} \right).$$

During the time interval $0 \leq t \leq \Delta$, the control is saturated at $u(t) = -1$, and the state trajectory is:

$$x(t) = Ae^{at} + \frac{b}{a}.$$

The cost accrued during this phase is:

$$J_1 = \frac{1}{2} \left[\frac{A^2}{2a} (e^{2a\Delta} - 1) + \frac{2Ab}{a^2} (e^{a\Delta} - 1) + \left(\left(\frac{b}{a} \right)^2 + 1 \right) \Delta \right].$$

Once the state enters the linear region at time Δ , the cost-to-go is:

$$V(x_s) = \frac{1}{2b^2P}.$$

Hence the total cost is:

$$V(x_0) = J_1 + \frac{1}{2b^2P}.$$

3. Saturated Region ($x_0 < -x_s$) This case is symmetric. Define:

$$A = x_0 + \frac{b}{a}, \quad \Delta = \frac{1}{a} \ln \left(\frac{-x_s - \frac{b}{a}}{x_0 + \frac{b}{a}} \right),$$

and apply the same expressions as in the $x_0 > x_s$ case, replacing $x_0 \mapsto -x_0$ and $u(t) = +1$ for the initial phase. The value function is again:

$$V(x_0) = J_1 + \frac{1}{2b^2P},$$

with J_1 computed analogously.

Summary

The optimal control and value function for this saturated scalar system are:

$$u^*(x) = \text{sat}(-bPx),$$

$$V(x_0) = \begin{cases} \frac{1}{2}Px_0^2 & \text{if } |x_0| \leq x_s, \\ J_1(x_0) + \frac{1}{2b^2P} & \text{if } |x_0| > x_s, \end{cases}$$

where $P = \frac{a+\sqrt{a^2+b^2}}{b^2}$, $x_s = \frac{1}{bP}$, and J_1 is the explicitly integrated cost over the saturated phase.

11.21 Controlling Mathematical Pendulum

The Pendulum Setup

Equation of motion is:

$$\ddot{\theta} = \sin \theta - \alpha \dot{\theta} + u \cos \theta$$

where:

- θ is the angle measured from the *positive* vertical direction
- u is a control
- α is the friction constant Thus u is an external horizontal force.

A physical realization

One way to implement an apparatus obeying this equation would be to place a pendulum on a moving platform. The force is the acceleration of the platform. For simplicity, the dynamics of the platform is ignored, and the interaction between the platform and the pendulum is one-directional (the platform exerts a force on the pendulum). The platform is *very heavy* as compared to the pendulum and we can move it by making it accelerate. Think of a rod mounted on top of a car, where we control the forward and reverse acceleration of the car.

The Variables

1. θ - angle with vertical (upwards) direction
2. φ - angular velocity
3. $x = (\theta, \varphi)$ - the state
4. $\lambda = (\lambda_1, \lambda_2)$ - the costate
5. u - the control
6. α - the friction constant

The Hamiltonian

We introduce the following notations:

$$f_0 = (1 - \cos \theta) + \frac{1}{2}\varphi^2 + \frac{1}{2}u^2,$$

$$f_1 = \varphi,$$

$$f_2 = \sin \theta - \alpha \varphi + u \cos \theta.$$

The state equation is $\dot{x} = f(x, u)$ where $f = (f_1, f_2)$ The Hamiltonian is:

$$\begin{aligned}\mathcal{H}(x, \lambda, u) &= f_0(x, u) + \lambda^\top \cdot f(x, u) \\ &= (1 - \cos \theta) + \frac{1}{2}\varphi^2 + \frac{1}{2}u^2 + \lambda_1\varphi + \lambda_2(\sin \theta - \alpha \varphi + u \cos \theta)\end{aligned}$$

Optimal control

★★★ Step 1: Differentiate Hamiltonian over u ★★★

$$\partial_u \mathcal{H} = u + \lambda_2 \cos \theta$$

★★★ Step 2: Solve for u ★★★

$$u = u^* = -\lambda_2 \cos \theta$$

★★★ **Step 3:** Find the *effective* Hamiltonian ★★★ i.e., u^* is substituted for u :

$$\begin{aligned} H(x, \lambda) &= (1 - \cos \theta) + \frac{1}{2}\varphi^2 + \frac{1}{2}\lambda_2^2 \cos^2 \theta + \lambda_1 \varphi \\ &\quad + \lambda_2(\sin \theta - \alpha \varphi - \lambda_2 \cos \theta \cdot \cos \theta) \\ &= (1 - \cos \theta) + \frac{1}{2}\varphi^2 - \frac{1}{2}\lambda_2^2 \cos^2 \theta \\ &\quad + \lambda_1 \varphi + \lambda_2(\sin \theta - \alpha \varphi). \end{aligned}$$

Dynamical Systems view — finding stable manifold

Definition 4 (Stable Manifold). If the dynamical system is $\dot{\xi} = F(\xi)$, $\xi \in \mathbb{R}^n$ then the *stable manifold*:

$$W^s(\xi_0) = \{\xi_1 \in \mathbb{R}^n : \lim_{t \rightarrow \infty} \xi(t; \xi_1) = \xi_0\}$$

where $\xi(t; \xi_1)$ is a solution such that $\xi(0, \xi_1) = \xi_1$. $\square$

There is an *unstable manifold*:

$$W^u(\xi_0) = \{\xi_1 \in \mathbb{R}^n : \lim_{t \rightarrow -\infty} \xi(t; \xi_1) = \xi_0\}$$

The *Stable Manifold Theorem* states that the stable manifold is an actual (embedded) manifold if ξ_0 is a *hyperbolic fixed point*, i.e. $DF(\xi_0)$ has no eigenvalue with zero real part.

Mathematical basis of the algorithm

Given is the system $\dot{x} = f(x)$, $x \in \mathbb{R}^n$ and its equilibrium $x_0 = 0$. Let $\phi_t : \mathbb{R}^n \rightarrow \mathbb{R}^n$ be the *flow* of the system:

$$\phi_t(x_0) = x(t; x_0)$$

(solution of the IVP with initial condition x_0)

The flow of the system is a *1-dimensional group of diffeomorphisms*:

- $\phi_0 = id$
- $\phi_{t+s} = \phi_t \circ \phi_s$
- $\phi_{-t} = \phi_t^{-1}$

★★★ **Step 1:** Linearize the system at equilibrium ★★★ Let $A = Df(0)$

★★★ **Step 2:** Find eigenvalues λ_j and eigenvectors v_j ★★★ Suppose $\lambda_1, \lambda_2, \dots, \lambda_s$ are in the left halfplane $\text{Re} \lambda < 0$.

★★★ **Step 3:** Parameterize the stable manifold ★★★ by a map $h : \mathbb{R}^s \rightarrow \mathbb{R}^n$.

★★★ **Step 4:** Build an auxillary system ★★★

$$\dot{u} = \Lambda_s u, \quad u \in \mathbb{R}^s$$

where

$$\Lambda_s = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_s)$$

★★★ **Step 5:** conjugate the flows of the auxillary system and the main system ★★★

$$h(e^{t\Lambda_s} u) = \phi_t(h(u))$$

(a kind of non-linear diagonalization procedure). The idea is that near the origin h maps $\mathbb{R}^s$ to the eigenspace of all stable eigenvectors:

$$h(u) \approx \sum_{j=1}^s u_j v_j = V_s u$$

This leads to

$$h(u) = \phi_{-t}(h(e^{t\Lambda_s} u)) \approx \phi_{-t}(V_s e^{t\Lambda_s} u)$$

as $e^{t\Lambda_s} u \rightarrow 0$, if t is large.

★★★ Step 6: Take t to infinity ★★★ The final computational formula is:

$$h(\mathbf{u}) = \lim_{t \rightarrow \infty} \phi_{-t} \left(\sum_{j=0}^s e^{\lambda_j t} \mathbf{v}_j \right)$$

MATLAB implementation for the pendulum control problem

```

1 clear all;
2 %% Set up the Hamiltonian system symbolically
3 syms x y u a H l0 l1 l2 f0 f1 f2
4
5 % Cost function
6 f0 = 1/2*y*y + 1/2*u*u + 1-cos(x);
7 f1 = y;
8 f2 = sin(x) + u*cos(x) - a*y;
9 H = f0 + l1*f1 + l2*f2;
10
11 ustar = solve(diff(H, u),u);
12 H = subs(H, {a, u}, {0.1, ustar});
13
14 vars = { x, y, l1, l2 };
15
16 % Find the Hessian of Hamiltonian at 0
17 HessianSymbolic = hessian(H, [vars{:}]);
18 HessianAtZero = subs(HessianSymbolic,vars,{0,0,0,0});
19 HessianNumeric = double(HessianAtZero);
20
21 % Note V=W; left and right eigenvectors are the same
22 J = [zeros(2,2), eye(2); -eye(2),zeros(2,2)];
23
24 % Find the "Hamiltonian matrix"
25 C = J*HessianNumeric;
26
27 %% Solve ARE
28 % C*V = V*D; W'*C = D*C'.
29 [V,D,W] = eig(C);
30 D = diag(D);
31 ind = D < 0;
32 D = D(ind);
33 Vs = V(:,ind);
34 Vs1 = Vs(1:2,:);
35 Vs2 = Vs(3:4,:);
36 P = Vs2*inv(Vs1);
37
38 % Full Hamiltonian vector field
39 F(x,y,l1,l2) = [diff(H,l1), diff(H,l2), -diff(H,x), -diff(H,y)];
40
41 % Numerical hamiltonian in a form suitable for ODE solver; need dummy time
42 G = @(~, z) double(F(z(1),z(2),z(3),z(4)))';
43
44 Tspan = linspace(0,7,100);
45
46 x0 = [0.01,0]';
47 y0 = P*x0;
48 z0 = [x0;y0];
49
50 %% Draw trajectories, using ode23 and ode45 for comparison
51 [t, z23] = ode23(G, Tspan, z0);
52 [t, z45] = ode45(G, Tspan, z0);
53 clf;
54 hold on;
55 plot(z23(:,1),z23(:,2));
56 plot(z45(:,1),z45(:,2));

```

```

57 %xlim([-pi,pi])
58 %ylim([-2,2])
59 hold off;
60
61
62 % Shooting method - the spider web
63 psi = linspace(0,2*pi,100);
64 rad = 5;
65 % Write x as a linear combination of stable eigenvectors
66 tmax = 20;
67
68 % Solve linear equation forward, starting initial conditions on a circle
69 z = Vs * expm(tmax*diag(D)) * rad * [cos(psi); sin(psi)];
70
71 % Visualize
72 %x = z(1:2,:);
73 %plot(x(1,:),x(2,:),'-');
74
75
76 % Reverse time vector field
77 Grev = @(~, z) -double(F(z(1),z(2),z(3),z(4)))';
78
79 % Negotiate a time step
80 target_time_step = .2;
81 num_timesteps = round( tmax / target_time_step);
82 timestep = tmax / num_timesteps;
83
84 num_anglesteps = size(z,2);
85
86 Tspan = linspace(0, tmax, num_timesteps);
87 Z = zeros(num_anglesteps, num_timesteps, 4);
88
89 % Computational loop: Replace 'parfor' with 'for' to
90 % run single-threaded.
91 tic;
92 parfor j=1:num_anglesteps
93     z_start = z(:,j);
94     [t, z_traj] = ode45(Grev, Tspan, z_start);
95     Z(j, :, :) = z_traj;
96 end
97 toc;
98
99 % Visualization loop
100 clf;
101 hold on;
102 for j=1:num_anglesteps
103     z_traj = squeeze(Z(j, :, :));
104     %plot(z_traj(:,1),z_traj(:,2));
105     plot3(z_traj(:,1),z_traj(:,2),z_traj(:,3),'-o');
106     drawnow;
107 end
108 for l=1:num_timesteps
109     z_traj = squeeze(Z(:, l, :));
110     %plot(z_traj(:,1),z_traj(:,2));
111     plot3(z_traj(:,1),z_traj(:,2),z_traj(:,3),'-o');
112     drawnow;
113 end
114 xlabel("\theta");
115 ylabel("\phi");
116 zlabel("\lambda_1");
117 % Found interactively
118 view(82,-18);
119 title("Stable manifold parameterized by angle and time");
120 hold off;
121 saveas(gcf, 'figures/generated/StableManifoldParameterization.png')

```

Stable manifold parameterized by angle and time

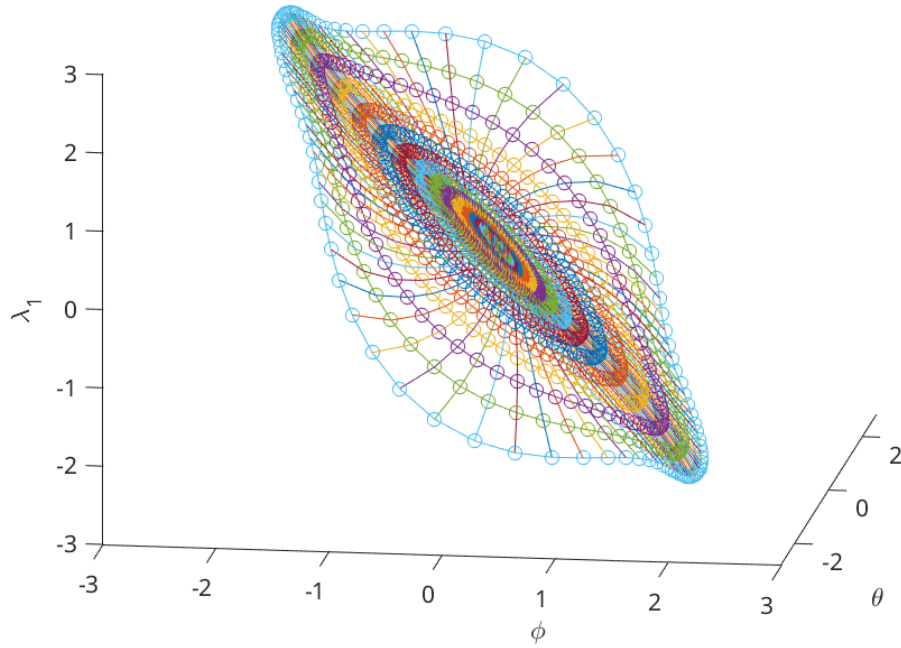


Figure 11.3: The stable manifold projected on three dimensions

A view of the stable manifold is provided in Figure 11.3.

11.22 Exercises

11.22.1 Linear-Quadratic Regulator (LQR)

Exercise 11.1 (Optimal control and stability analysis). Consider the scalar system $\dot{x}(t) = 2x(t) + 3u(t)$, with cost

$$J = \int_0^\infty (4x^2(t) + 2u^2(t)) dt.$$

Derive the optimal control $u^*(t)$ and analyze the stability. $\square$

Exercise 11.2 (Stability conditions). Given the system $\dot{x}(t) = -x(t) + 2u(t)$ and cost

$$J = \int_0^\infty (x^2(t) + u^2(t)) dt,$$

determine the equations of motion and conditions for stability. $\square$

Exercise 11.3 (Effective Hamiltonian). Consider the Hamiltonian system

$$H(x, u, \lambda, t) = x^2 + u^2 + \lambda(4x + u).$$

Find the optimal control u^* and effective Hamiltonian. $\square$

Exercise 11.4 (Explicit solution and optimality). Consider the LQR system

$$\dot{x}(t) = -x(t) + u(t), \quad J = \int_0^\infty (x^2(t) + 3u^2(t))dt.$$

Solve explicitly and discuss optimality. $\square$

Exercise 11.5 (Eigenvalue analysis of symplectic systems). Verify explicitly the similarity between $-A$ and $A^\top$ when $A = JB$ with

$$B = \begin{bmatrix} 6 & 2 \\ 2 & 1 \end{bmatrix}.$$

$\square$

11.22.2 Pontryagin Maximum Principle (PMP)

Exercise 11.6 (Basic PMP application). Solve the PMP problem:

$$\dot{x} = u, \quad J = \int_0^1 (x^2 + u^2)dt, \quad x(0) = 1, \quad x(1) = 0.$$

$\square$

Exercise 11.7 (Trajectory and control optimization). Solve the PMP problem for:

$$\dot{x} = -x + u, \quad J = \int_0^\top (x^2 + u^2)dt.$$

$\square$

Exercise 11.8 (Hamiltonian system analysis). Derive the optimal control using PMP for the system:

$$\dot{x} = x + 2u, \quad J = \int_0^\top (4x^2 + u^2)dt.$$

$\square$

Exercise 11.9 (Long-term optimality). Apply PMP to:

$$\dot{x} = 3x + u, \quad J = \int_0^\infty (2x^2 + u^2)dt, \quad x(0) = 2.$$

$\square$

Exercise 11.10 (Parameter conditions for stability). Determine general conditions on a, b, q, r ensuring stability for:

$$\dot{x} = ax + bu, \quad J = \int_0^\infty (qx^2 + ru^2)dt.$$

$\square$

Exercise 11.11 (Simulation and Analysis). For the pendulum on a cart system described in section 11.12, implement the LQR controller in a simulation environment. Two choices are: MATLAB/Simulink and ROS (Robot Operating System).

- (a) Analyze the system's response to disturbances and initial conditions.
- (b) Evaluate the effectiveness of the controller in stabilizing the pendulum and controlling the cart's position.

$\square$

11.22.3 Hamilton-Jacobi-Bellman (HJB) Equation

Exercise 11.12 (One-dimensional HJB formulation). Formulate and solve the HJB equation for:

$$\dot{x} = u, \quad J = \int_0^\infty (x^2 + u^2)dt.$$

□

Exercise 11.13 (HJB and value function). Derive and solve the HJB equation for:

$$\dot{x} = -2x + u, \quad J = \int_0^\infty (3x^2 + u^2)dt.$$

□

Exercise 11.14 (Explicit HJB PDE solution). Solve the HJB equation explicitly for the system:

$$\dot{x} = 3x + u, \quad J = \int_0^\infty (2x^2 + u^2)dt.$$

□

Exercise 11.15 (Rocket trajectory optimization). Formulate and solve the HJB PDE for:

$$\dot{x} = v, \quad \dot{v} = u, \quad J = \int_0^\top (u^2 + v^2)dt.$$

□

Exercise 11.16 (Boat navigation with current). Derive and solve the HJB PDE for:

$$\dot{x} = u, \quad J = \int_0^\top (u^2 + c^2 x^2)dt, \quad x(0) = 0, x(T) = x_f, c > 0.$$

□

Exercise 11.17 (Boat heading control). Apply HJB to control boat heading with lateral current:

$$\dot{x} = u - kx, \quad J = \int_0^\infty (\alpha x^2 + u^2)dt, \quad \alpha, k > 0.$$

□

Chapter 12

Partial Differential Equations (PDEs)

Partial Differential Equations (PDEs) are equations that involve rates of change with respect to continuous variables. They are fundamental to many fields, including physics, engineering, and finance. This chapter covers numerical methods for solving PDEs, focusing on the Finite Difference Method, Finite Element Method (FEM), Finite Volume Method (FVM), and Multigrid Methods.

12.1 Finite Difference Method

The Finite Difference Method (FDM) approximates the derivatives in PDEs using finite differences. It is widely used due to its simplicity and ease of implementation.

12.1.1 Derivation of Finite Differences

Finite differences approximate derivatives by using values at discrete grid points. For a function $u(x)$, the finite difference approximations of the first and second derivatives at point x_i are:

- Forward Difference: $u'(x_i) \approx \frac{u(x_{i+1}) - u(x_i)}{h}$
- Backward Difference: $u'(x_i) \approx \frac{u(x_i) - u(x_{i-1}))}{h}$
- Central Difference: $u'(x_i) \approx \frac{u(x_{i+1}) - u(x_{i-1}))}{2h}$
- Second Derivative: $u''(x_i) \approx \frac{u(x_{i+1}) - 2u(x_i) + u(x_{i-1}))}{h^2}$

12.1.2 Application to PDEs

Consider the heat equation $\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$. Using finite differences, we can discretize the equation on a grid with time step Δt and spatial step Δx :

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \alpha \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{(\Delta x)^2}$$

Rearranging gives the explicit update formula:

$$u_i^{n+1} = u_i^n + \frac{\alpha \Delta t}{(\Delta x)^2} (u_{i+1}^n - 2u_i^n + u_{i-1}^n) \quad (12.1)$$

Example: Heat Equation

Let's solve the heat equation $\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$ on the domain $0 \leq x \leq 1$ with Dirichlet boundary conditions $u(0, t) = 0$ and $u(1, t) = 0$. Assume $\Delta x = 0.1$, and $\Delta t = 0.005$. The initial condition is $u(x, 0) = \sin(\pi x)$.

Using the explicit update formula, we iteratively compute u_i^{n+1} for each grid point $x_i = i\Delta x$ and time step t_n .

$$\begin{aligned} u_i^{n+1} &= u_i^n + \frac{0.005}{0.1^2} \alpha (u_{i+1}^n - 2u_i^n + u_{i-1}^n) \\ &= u_i^n + 0.5\alpha (u_{i+1}^n - 2u_i^n + u_{i-1}^n) \end{aligned}$$

The boundary conditions are modeled in the discrete system by requiring

$$u_0^n = u_{10}^n = 0.$$

For fixed n , i.e., for frozen time, $\mathbf{u}^n = (u_i^n)_{i=1}^9 \in \mathbb{R}^9$. The iteration scheme is of the form,

$$\begin{aligned} \mathbf{u}^{n+1} &= \mathbf{u}^n - \lambda A \mathbf{u}^n = (I - \lambda A) \mathbf{u}^n \\ \mathbf{u}^0 &= \mathbf{u}_0, \\ \lambda &= 0.5\alpha. \end{aligned}$$

We know, for example, from Chapter 4, that the iterative scheme converges when

$$\rho(\lambda A) < 1$$

where $\rho(A)$ denotes the spectral radius of A . Hence, the necessary and sufficient condition of convergence for arbitrary initial condition $\mathbf{u}_0$ is

$$\lambda < \frac{1}{\rho(A)}.$$

This is the condition of *stability* of the numeric scheme.

The matrix A has a familiar form (the negative discrete Laplacian):

$$\begin{bmatrix} 2 & -1 & 0 & \dots & 0 & 0 \\ -1 & 2 & -1 & \dots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & -1 & 0 \\ 0 & 0 & 0 & \dots & 2 & -1 \\ 0 & 0 & 0 & \dots & -1 & 2 \end{bmatrix}$$

As $\|A\|_\infty = 3$, we have an easy bound $\rho(A) \leq 3$. Therefore, a sufficient condition of convergence is

$$\lambda < \frac{1}{3}.$$

Thus, only when $\alpha < \frac{2}{3}$ we have convergence.

For arbitrary steps Δx and Δt the condition of stability is (approximately, because we did not calculate the spectral radius exactly):

$$\alpha \frac{\Delta t}{(\Delta x)^2} < \frac{1}{3}.$$

One conclusion is that the time step must be much smaller than the space step, $\Delta t \sim (\Delta x)^2$.

Neumann Boundary Conditions

Neumann boundary conditions for the heat equation are $u_x(0, t) = u_x(1, t) = 0$. Although the PDE did not change, the discretized system is impacted by this change. We model these conditions by requiring

$$u_0^n = u_1^n, \quad u_{10}^n = u_9^n.$$

The new matrix A will have $a_{1,1} = a_{9,9} = 1$ rather than 2.

Remark 12.1 (On the physics of Dirichlet and Neumann boundary conditions). The physical interpretation Dirichlet conditions is that the boundary of the interval (rod) is kept at 0 temperature (they are submerged in a “heat bath”). The Neumann boundary conditions mean that there is no heat exchange (flux) through the boundary. That is, the boundary is insulated in some way. $\square$

12.2 Image Restoration Using Total Variation

Image restoration is recovering the true image from noisy observations. We will focus on gray-scale images represented as functions mapping locations to intensities. The challenge is distinguishing true image content from noise.

Total Variation Restoration

Total Variation (TV) Restoration is a particular method to restore true images from noisy data. One remarkable application of this algorithm was denoising the first image of a black hole.

TV balances fidelity to the observed image and smoothness. It is a variational method, where the functional to minimize is:

$$\mathcal{F}(u) = \int_D (u(x) - f(x))^2 + \lambda \|\nabla u(x)\|_p^r dx$$

where $D \subseteq \mathbb{R}^2$, and $\|\cdot\|_p$ is the L^p -norm. The regularization parameter λ is used to balance noise reduction and detail preservation, and it depends on an application and the quality of the data.

In particular, we assume that the function belongs to a Sobolev space of differentiable functions whose derivatives belong to L^p .

Typically we use Neumann boundary condition:

$$n \cdot \nabla u(x) = 0, \quad \forall x \in \partial D$$

where n is the normal vector to the boundary.

If $p = r = 2$ we obtain a simplified version of the problem where S is a differentiable, quadratic functional. This makes the theoretical analysis significantly less complicated.

If $p = 2$ and $r = 1$, we get the most studied case of TV restoration.

12.2.1 Discretization Model for Real Images

Real images are discretized into pixels. That is, we do not directly observe the underlying function $u(x)$, but only its values at grid points x_{ij} . It should be noted that u_{ij} is not a value of any function, but typically is an average of values observed in measuring a physical process. E.g., it could be the number of photons “captured” by a “pit” centered about x_{ij} by the CCD (charged-coupled device) of a digital camera.

The approach studied in this section is discretization of the functional $\mathcal{F}$ for computational implementation. We treat of the image as a matrix of pixel values. We approximate of gradients and boundary conditions in the discrete domain.

Once the functional is discretized, the problem can be solved by optimization techniques, e.g., gradient descent. The challenges lie in large-scale calculations for high-resolution images.

12.2.2 Euler-Lagrange PDE for Regularized ROF Model

The *regularized ROF*¹ model [23] is given by the functional:

$$\mathcal{F}(u) = \int_D \sqrt{\epsilon^2 + |\nabla u|^2} + \lambda \int_D (u - f)^2 dx dy$$

where u is the image to be restored, f is the observed degraded image, and λ is a regularization parameter.

For $\epsilon = 0$ we obtain the original ROF functional:

$$\mathcal{F}(u) = \|\nabla u\|_{L^1} + \lambda \|u - f\|_{L^2}.$$

The first part is the L^1 -norm of the Euclidean norm of the gradient. This is called the *total variation norm* and denoted

$$\|u\|_{TV(D)} \stackrel{\text{def}}{=} \|\nabla u\|_{L^1(D)} = \int_D \|\nabla u\| dA.$$

The objective is to minimize this functional over all functions u .

The Euler-Lagrange PDE derived from this functional is:

$$-\text{div} \left(\frac{\nabla u}{\sqrt{\epsilon^2 + |\nabla u|^2}} \right) + \lambda(u - f) = 0$$

The Neumann Boundary Conditions on a rectangular domain $[a, b] \times [c, d]$ are:

$$\begin{aligned} \frac{\partial u}{\partial x}(a, y) = 0 \quad \text{and} \quad \frac{\partial u}{\partial x}(b, y) = 0, \\ \frac{\partial u}{\partial y}(x, c) = 0 \quad \text{and} \quad \frac{\partial u}{\partial y}(x, d) = 0. \end{aligned}$$

This implies that the rate of change of u is zero in the direction normal to each edge of the rectangle. In general,

$$\frac{\partial u}{\partial \mathbf{n}} \Big|_{\partial D} = 0.$$

It is helpful to interpret the Neumann boundary condition as *no flux* condition for the gradient vector field $\mathbf{F} = \nabla u$.

12.2.3 A constant-free version of the problem

We start with the problem of minimizing total variation over u from a small L^2 -ball around f :

$$\begin{aligned} &\text{minimize} && \|u\|_{TV(D)} \\ &\text{subject to} && \|u - f\|_{L^2(D)} \leq \alpha \end{aligned}$$

Using the Lagrange multiplier version of the problem, we need to find the minimum of

$$\mathcal{L}(u, \lambda) = \|u\|_{TV(D)} + \lambda(\|u - f\|_{L^2}^2 - \alpha)$$

for a fixed α , such that $\lambda \geq 0$.

¹Rudin, Osher, Fatemi

12.2.4 1D Image Restoration with Squared Gradient Norm (Tikhonov Regularization)

This is a simplified problem formulated as follows:

- The “image” is one-dimensional: $g : [0, 1] \rightarrow \mathbb{R}$
- The functional is

$$\mathcal{F}(u) = \frac{1}{2} \int_0^1 [(u(x) - f(x))^2 + \lambda(u'(x))^2] dx$$

- The boundary conditions are

$$u'(0) = u'(1) = 0$$

We compute the derivative (differential, variation) of the functional.

- $\delta u(x)$ – a “small” function, perturbation of $u(x)$.
- $(\delta u)'(x) = (\delta u')(x)$ – or d/dx and δ commute as linear operators.

The Euler-Lagrange equations can be derived directly. We start by calculating the variation of the functional.

$$\begin{aligned} \delta \mathcal{F}(u) &= \int_0^1 [(u(x) - f(x)) \cdot (\delta q)(x) + \lambda u'(x)(\delta q)'(x)] dx \\ &= \int_0^1 (u(x) - f(x)) \cdot (\delta q)(x) dx + \lambda \int_0^1 u'(x) \cdot (\delta q)'(x) dx \\ &= \int_0^1 (u(x) - f(x)) \cdot (\delta q)(x) dx \\ &\quad + \lambda \left[u'(x) \cdot (\delta q)(x) \Big|_{x=0}^{x=1} - \int_0^1 u''(x)(\delta q)(x) dx \right] \\ &= \int_0^1 (u(x) - f(x)) \cdot (\delta q)(x) dx - \lambda \int_0^1 u''(x) \cdot (\delta q)(x) dx \\ &= \int_0^1 [(u(x) - f(x)) - \lambda u''(x)] \cdot (\delta q)(x) dx \end{aligned}$$

In the usual fashion we obtain the Euler-Lagrange equation, which is a necessary condition of optimality of $u(x)$:

$$u(x) - f(x) - \lambda u''(x) = 0.$$

The integral must be 0 for every $\delta u(x)$.

12.2.5 The difference scheme

We use the symmetric difference for discretization. Let $h = 1/N$, $x_i = ih$, $u_i = u(x_i)$, $f_i = f(x_i)$. The difference scheme yields

$$(u_i - f_i) - \lambda \frac{u_{i+1} + u_{i-1} - 2u_i}{h^2} = 0$$

subject to:

$$\begin{aligned} u_0 &= u_1 \\ u_{N-1} &= u_N. \end{aligned}$$

12.2.6 Numerical Implementation

We use an iterative method (although we could have used Gaussian elimination as well).

- Solve for u_i :

$$\left(1 + 2\frac{\lambda}{h^2}\right)u_i = f_i + \lambda(u_{i+1} + u_{i-1})$$

$$u_i = \left(1 + 2\frac{\lambda}{h^2}\right)^{-1} \left[f_i + \frac{\lambda}{h^2}(u_{i+1} + u_{i-1})\right]$$

- Special rule for u_1 and u_{N-1} : first extend by $u_0 = u_1$ and $u_N = u_{N-1}$ and then apply the above rule.

The parameters are: step size, regularization parameter, and iteration count. We observe that convergence holds if

$$\left|\frac{2\lambda/h^2}{1 + 2\lambda/h^2}\right| < 1 \iff \left|\frac{h^2}{2\lambda} + 1\right| > 1,$$

e.g., when both h and λ are positive..

12.2.7 Python Code for Image Restoration

Image restoration in 1D, quadratic optimization case

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Parameters for the numerical example
5 N = 100 # Number of points
6 x = np.linspace(0, 1, N)
7 h = x[1] - x[0] # Step size
8 lambda_reg = 0.001 # Regularization parameter
9
10 # True signal and noisy observation
11 true_signal = np.sin(2 * np.pi * x)
12 noise = np.random.normal(0, 0.1, N)
13 noisy_signal = true_signal + noise
14
15 # Initialize the restored signal for the modified problem using squared norm of the gradient
16 u_squared = np.copy(noisy_signal)
17
18 # Solving the modified problem using the Euler-Lagrange equation
19 # lambda * u''(x) + u(x) = f(x)
20 r = lambda_reg / h**2;
21 k = 1/(1 + 2*r)
22 u_squared_new = np.zeros(N)
23 for _ in range(1000): # Number of iterations
24     for i in range(1, N - 1):
25         u_prev = u_squared[i - 1] if i > 1 else u_squared[i]
26         u_next = u_squared[i + 1] if i < N - 1 else u_squared[i]
27         # Apply the Euler-Lagrange equation for the modified problem
28         u_squared_new[i] = k * (noisy_signal[i] + r * (u_prev + u_next))
29     # Update as a whole
30     u_squared = u_squared_new
31
32 # Plotting the results for the modified problem
33 plt.figure(figsize=(12, 6))
34 plt.plot(x, true_signal, label='True Signal', linestyle='dashed')
35 plt.plot(x, noisy_signal, label='Noisy Signal', alpha=0.7)
36 plt.plot(x, u_squared, label='Restored Signal (Squared Gradient Norm)')
37 plt.legend()

```

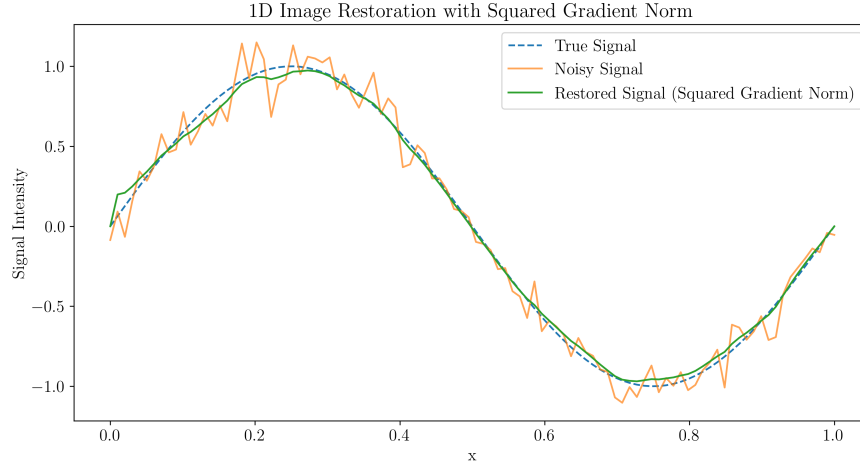


Figure 12.1: Image restoration, 1D, quadratic functional

```

38 plt.xlabel('x')
39 plt.ylabel('Signal Intensity')
40 plt.title('1D Image Restoration with Squared Gradient Norm')
41 #plt.show()
42 plt.savefig("figures/generated/image_restoration_1D_quadratic.png", dpi=300, bbox_inches='tight')
43 plt.close()

```

12.2.8 Smoothing, Denoising and the Total Variation Functional

TV functional not differentiable at $\nabla u(x) = 0$. This creates challenges for variations in image restoration. Thus, there is a need for a smoothed formulation. One way to solve to problem is to smooth the Lagrangian to bypass non-differentiability. The smoothed TV functional is

$$\mathcal{F}_\varepsilon(u) = \int_{[0,1]^2} \left[\frac{1}{2}(u(x) - f(x))^2 + \lambda \sqrt{\varepsilon^2 + \|\nabla u(x)\|_2^2} \right] dx$$

where ε is a small and positive smoothing parameter.

Euler-Lagrange Equation Derivation for Smoothed Action

For our functional, the Lagrangian L is:

$$L(u, u_x, u_y) = \frac{1}{2}(u - f)^2 + \lambda \sqrt{\varepsilon^2 + u_x^2 + u_y^2}$$

The Euler-Lagrange equation is

$$\frac{\partial L}{\partial u} - \frac{d}{dx} \left(\frac{\partial L}{\partial u_x} \right) - \frac{d}{dy} \left(\frac{\partial L}{\partial u_y} \right) = 0$$

After computing the derivatives, the equation becomes:

$$u - f - \frac{\partial}{\partial x} \left(\frac{\lambda u_x}{\sqrt{\varepsilon^2 + u_x^2 + u_y^2}} \right) - \frac{\partial}{\partial y} \left(\frac{\lambda u_y}{\sqrt{\varepsilon^2 + u_x^2 + u_y^2}} \right) = 0 \quad (12.2)$$

12.2.9 A Discrete Variant of TV denoising

The Variational Approach

An image to be restored (or smoothed) is f , and is given as an array $f_{i,j}$. The restored image is likewise only determined as an array $u_{i,j}$ of values on the grid.

We replace the continuous functional $\mathcal{F}_\epsilon$ with a discrete functional:

$$\mathcal{F}_\epsilon(u) = \frac{1}{2} \sum_{i=1}^H \sum_{j=1}^W (u_{i,j} - f_{i,j})^2 + \lambda \sum_{i=1}^H \sum_{j=1}^W \sqrt{\epsilon^2 + (u_{i+1,j} - u_{i,j})^2 + (u_{i,j+1} - u_{i,j})^2}.$$

Note that we assumed that the image is represented by a 1-based array, as in MATLAB. We assume that there is a *virtual* layer of pixels along the boundary with indices $(i, 0)$, $(i, W+1)$, $(H+1, j)$ and $(0, j)$, $0 \leq i \leq W+1$, $0 \leq j \leq H+1$.

This functional is convex and differentiable on the set of $H \times W$ arrays. satisfying Neumann-like boundary conditions:

$$u_{i,W+1} = u_{i,W}, \quad u_{i,0} = u_{i,1}, \quad u_{H+1,j} = u_{H,j}, \quad u_{0,j} = u_{1,j}.$$

Thus the virtual pixels are simply duplicates of stored pixels, and they do not need to be explicitly stored.

The gradient components of the functional $\mathcal{F}_\epsilon$ are as follows. First, we observe that the variable $u_{i,j}$ participates in exactly **three** summands (somewhat of a surprise):

$$\begin{aligned} \mathcal{F}_\epsilon(u) = & \frac{1}{2} (u_{i,j} - f_{i,j})^2 + \lambda \left\{ \sqrt{\epsilon^2 + (u_{i+1,j} - u_{i,j})^2 + (u_{i,j+1} - u_{i,j})^2} \right. \\ & + \sqrt{\epsilon^2 + (u_{i,j} - u_{i-1,j})^2 + (u_{i-1,j+1} - u_{i-1,j})^2} \\ & \left. + \sqrt{\epsilon^2 + (u_{i,j} - u_{i,j-1})^2 + (u_{i+1,j} - u_{i,j-1})^2} \right\} \\ & + \text{sum of terms not depending on } u_{i,j}. \end{aligned}$$

Therefore,

$$\begin{aligned} \frac{\partial \mathcal{F}_\epsilon(u)}{\partial u_{i,j}} = & (u_{i,j} - f_{i,j}) + \lambda \left\{ \frac{-(u_{i+1,j} - u_{i,j}) - (u_{i,j+1} - u_{i,j})}{\sqrt{\epsilon^2 + (u_{i+1,j} - u_{i,j})^2 + (u_{i,j+1} - u_{i,j})^2}} \right. \\ & + \frac{u_{i,j} - u_{i-1,j}}{\sqrt{\epsilon^2 + (u_{i,j} - u_{i-1,j})^2 + (u_{i-1,j+1} - u_{i-1,j})^2}} \\ & \left. + \frac{u_{i,j} - u_{i,j-1}}{\sqrt{\epsilon^2 + (u_{i,j} - u_{i,j-1})^2 + (u_{i+1,j-1} - u_{i,j-1})^2}} \right\}. \end{aligned}$$

Due to strict convexity of the functional, the system can be successfully solved by applying Gradient Descent. Backtracking should be used, e.g., by implementing Wolfe strong conditions. The convergence may be slow, and for large images we may opt to use GPU arrays.

For efficiency, one should utilize vectorization. Here are some techniques one should use in MATLAB:

1. One should pad the image with one row on the bottom and one column on the right (thus making a portion of the virtual pixel region non-virtual), using function `padarray`.
2. One should shift the image up, and left, thus generating extra copies of the image in memory.
3. Think of the gradient formula as applying a function to the image and those shifted copies.
4. Utilize operations such as Hadamard product (componentwise multiplication) of images.
5. Take advantage of build in vectorized functions, such as $\sqrt{\cdot}$ and arithmetical operations; you can add, subtract and divide one image by another.

6. **Avoid explicit loops in compute-intensive tasks** when using environments such as MATLAB or Python. The performance will drop by hundreds to thousands of times. E.g., looping over all pixels to shift them should be replaced with a vectorized operation, such as `circshift`.

This operation will not be quite memory efficient, but it will yield the best time performance.

Example 12.1 (The size of the memory to contain the arrays). A grayscale image which is 3012-by-2012, with 16-bit color, occupies $3012 \times 2012 \times 2 = 12120288 \text{ bytes} = 12 \text{ MB}$ of memory. Thus, we need approximately 36 megabytes of memory to compute the gradient. This is a negligible amount of memory for modern hardware. E.g., a laptop GPU may have 8GB of memory, thus allowing storing and processing hundreds of images in parallel. $\square$

12.3 Finite Element Method (FEM)

The Finite Element Method (FEM) is a powerful technique for solving PDEs, particularly on complex geometries. It involves dividing the domain into smaller subdomains (elements) and using polynomial basis functions.

12.3.1 Basic Concept

FEM approximates the solution $u(x)$ by a linear combination of basis functions $\phi_i(x)$:

$$u(x) \approx \sum_{i=1}^n u_i \phi_i(x)$$

The basis functions $\phi_i(x)$ are typically piecewise polynomials defined on each element.

12.3.2 Weak Formulation

To derive the FEM equations, we start with the weak formulation of the PDE. For the Poisson equation $-\Delta u = f$ in a domain Ω , the weak form is:

$$\int_{\Omega} \nabla u \cdot \nabla v \, d\Omega = \int_{\Omega} f v \, d\Omega \quad \forall v \in H_0^1(\Omega)$$

Substituting the finite element approximation and choosing test functions equal to basis functions, we obtain a system of linear equations:

$$K\mathbf{u} = \mathbf{f}$$

where K is the stiffness matrix and $\mathbf{f}$ is the load vector.

Example: 1D Poisson Equation

Consider the 1D Poisson equation $-\frac{d^2 u}{dx^2} = f(x)$ on the interval $0 \leq x \leq 1$ with boundary conditions $u(0) = 0$ and $u(1) = 0$. Let $f(x) = 1$.

We divide the interval into two elements and use linear basis functions $\phi_i(x)$. The weak form is:

$$\int_0^1 \frac{du}{dx} \frac{dv}{dx} \, dx = \int_0^1 f v \, dx$$

This leads to the system:

$$\begin{pmatrix} \frac{2}{h} & -\frac{1}{h} \\ -\frac{1}{h} & \frac{2}{h} \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \end{pmatrix} = \begin{pmatrix} \frac{h}{2} \\ \frac{h}{2} \end{pmatrix}$$

Solving this system gives the approximate solution $u(x)$.

12.4 Finite Volume Method (FVM)

The Finite Volume Method (FVM) is a method for solving PDEs that conserves fluxes through control volumes. It is particularly useful for conservation laws and fluid dynamics.

12.4.1 Conservation Law Formulation

FVM integrates the PDE over control volumes and applies the divergence theorem to convert volume integrals into surface integrals. For the conservation law $\frac{\partial u}{\partial t} + \nabla \cdot \mathbf{F} = 0$, integrating over a control volume V_i gives:

$$\frac{d}{dt} \int_{V_i} u dV + \int_{\partial V_i} \mathbf{F} \cdot \mathbf{n} dA = 0$$

12.4.2 Discretization

The integral form is discretized by approximating the fluxes $\mathbf{F}$ at the faces of the control volumes. For a 1D advection equation $\frac{\partial u}{\partial t} + \frac{\partial f(u)}{\partial x} = 0$, the semi-discrete form is:

$$\frac{du_i}{dt} + \frac{f(u_{i+1/2}) - f(u_{i-1/2})}{\Delta x} = 0$$

where $f(u_{i+1/2})$ is the numerical flux at the cell face.

Example: 1D Advection Equation

Consider the 1D advection equation $\frac{\partial u}{\partial t} + a \frac{\partial u}{\partial x} = 0$ with $a = 1$. Using the upwind scheme for the fluxes, the semi-discrete form is:

$$\frac{du_i}{dt} + \frac{u_i - u_{i-1}}{\Delta x} = 0$$

This can be integrated in time using the forward Euler method:

$$u_i^{n+1} = u_i^n - \frac{\Delta t}{\Delta x} (u_i^n - u_{i-1}^n)$$

Assume the initial condition $u(x, 0) = \sin(2\pi x)$ on $0 \leq x \leq 1$ with periodic boundary conditions.

12.5 Solving Burgers' Equation: Methods and Sample Code

In this section, we present both analytic and numerical approaches to solving the Burgers' equation, accompanied by Python code snippets using `pythontex`.

Exact Solution via Cole–Hopf Transform

The Cole–Hopf substitution transforms the viscous Burgers' equation:

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}$$

into the linear heat equation for a function $\phi(x, t)$, where

$$u(x, t) = -2\nu \frac{\partial_x \phi(x, t)}{\phi(x, t)}.$$

Cole–Hopf Transformation (Analytical Approach)

The Cole–Hopf transformation provides an exact solution method for the viscous Burgers' equation:

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}, \quad x \in \mathbb{R}, \quad t > 0.$$

We introduce the transformation:

$$u(x, t) = -2\nu \frac{\partial_x \phi(x, t)}{\phi(x, t)},$$

which is equivalent to writing:

$$u(x, t) = -2\nu \partial_x [\log \phi(x, t)].$$

Substituting this form into the Burgers' equation, we derive the following identity:

$$\begin{aligned} \frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} &= -2\nu \left[\frac{\phi_{xt}}{\phi} - \frac{\phi_x \phi_t}{\phi^2} \right] + \left(-2\nu \frac{\phi_x}{\phi} \right) \left(-2\nu \left[\frac{\phi_{xx}}{\phi} - \left(\frac{\phi_x}{\phi} \right)^2 \right] \right) \\ &= -2\nu \left[\frac{\partial}{\partial x} \left(\frac{\phi_t}{\phi} \right) \right] + 4\nu^2 \left[\frac{\phi_x \phi_{xx}}{\phi^2} - \left(\frac{\phi_x}{\phi} \right)^3 \right]. \end{aligned}$$

Meanwhile, computing the diffusion term:

$$\nu \frac{\partial^2 u}{\partial x^2} = \nu \cdot \left[-2\nu \left(\frac{\phi_{xxx}}{\phi} - 3 \frac{\phi_x \phi_{xx}}{\phi^2} + 2 \frac{\phi_x^3}{\phi^3} \right) \right].$$

After simplification, both sides match provided that $\phi(x, t)$ satisfies the heat equation:

$$\frac{\partial \phi}{\partial t} = \nu \frac{\partial^2 \phi}{\partial x^2}.$$

Thus, the nonlinear Burgers' equation is converted to a linear PDE. The solution procedure is:

1. Given the initial condition $u(x, 0) = u_0(x)$, define

$$\phi(x, 0) = \exp \left(-\frac{1}{2\nu} \int_{x_0}^x u_0(s) ds \right).$$

2. Solve the heat equation for $\phi(x, t)$ with appropriate boundary conditions.
3. Recover the solution $u(x, t)$ via:

$$u(x, t) = -2\nu \frac{\partial_x \phi(x, t)}{\phi(x, t)}.$$

Example: For the initial condition $u_0(x) = -\sin(\pi x)$ on the interval $[0, 1]$, one computes:

$$\phi(x, 0) = \exp \left(\frac{1}{2\nu\pi} \cos(\pi x) \right),$$

then solves the heat equation and uses the inverse transformation to obtain $u(x, t)$.

This method provides an elegant solution and highlights the interplay between **nonlinear PDEs** and **linear diffusion** via exponential transformation.

Cole-Hopf Transformation (Numerical Approach)

Below is a Python implementation for a Gaussian initial condition:

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  from scipy.special import erf
4  from scipy.integrate import simpson
5
6  # Parameters
7  nu = 0.01 # viscosity
8  L = 1.0   # domain length
9  x = np.linspace(0, L, 400)
10 t = 0.1   # evaluation time
11
12 # Initial condition for u(x, 0)
13 def u0(x):
14     return -np.sin(np.pi * x)
15
16 # Cole-Hopf solution: construct phi from u0
17 def phi_ic(x):
18     return np.exp(- (1 / (2*nu)) * np.cos(np.pi * x))
19
20 # Solve the heat equation by convolution with Gaussian kernel
21 def cole_hopf_solution(x, t, nu):
22     u = np.zeros_like(x)
23     if t == 0:
24         return u # Avoid division by zero
25     for i, xi in enumerate(x):
26         y = np.linspace(0, 1, 500)
27         kernel = np.exp(-(xi - y)**2 / (4 * nu * t)) / np.sqrt(4 * np.pi * nu * t)
28         integrand = phi_ic(y) * kernel
29         phi = simpson(integrand, x=y)
30         dphi_dx = simpson(-2*(xi - y)/(4*nu*t) * integrand, x=y)
31         u[i] = -2 * nu * dphi_dx / phi
32     return u
33
34 u_exact = cole_hopf_solution(x, t, nu)
35
36 plt.plot(x, u_exact)
37 plt.title("Exact Solution via Cole-Hopf Transform")
38 plt.xlabel("x")
39 plt.ylabel("u(x,t)")
40 plt.grid(True)
41 #plt.show()
42 plt.savefig("figures/generated/burgers-via-cole-hopf.png", dpi=300, bbox_inches='tight')
43 plt.close()

```

Numerical Method: Explicit Finite Difference Scheme

We discretize the Burgers' equation using finite differences in both time and space. The nonlinear convection and diffusion terms are treated explicitly:

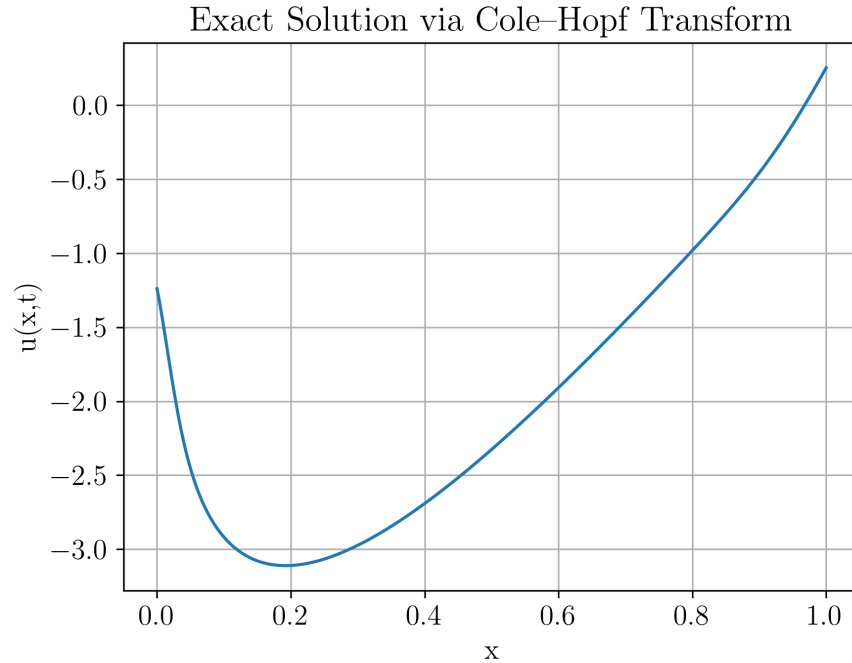


Figure 12.2: Burgers' Equation solved via Cole-Hopf Transform

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} + u_i^n \frac{u_{i+1}^n - u_{i-1}^n}{2\Delta x} = \nu \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2}.$$

Burger's Equation via Explicit FDM

```

1  # Numerical method parameters
2  nx = 101
3  nt = 500
4  dx = 1.0 / (nx - 1)
5  dt = 0.001
6  nu = 0.01
7
8  x = np.linspace(0, 1, nx)
9  u = -np.sin(np.pi * x)
10 u[0] = u[-1] = 0 # Dirichlet BCs
11
12 # Time stepping loop
13 for n in range(nt):
14     un = u.copy()
15     for i in range(1, nx-1):
16         convection = un[i] * (un[i+1] - un[i-1]) / (2*dx)
17         diffusion = nu * (un[i+1] - 2*un[i] + un[i-1]) / dx**2
18         u[i] = un[i] - dt * convection + dt * diffusion
19
20 # Plot numerical solution
21 plt.plot(x, u)
22 plt.title("Numerical Solution (Explicit FDM)")
23 plt.xlabel("x")

```

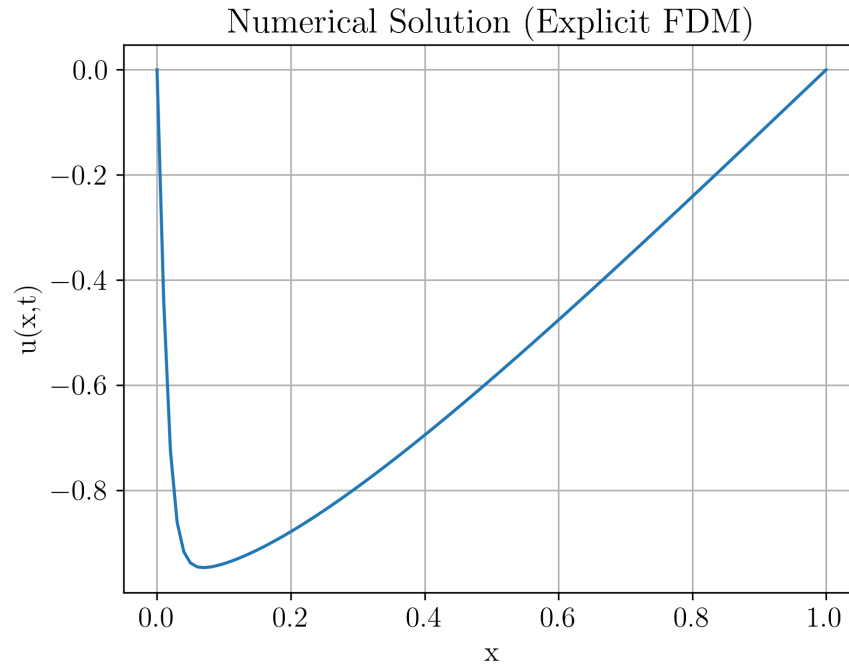


Figure 12.3: Burgers' Equation solved via Explicit FDM

```

24 plt.ylabel("u(x,t)")
25 plt.grid(True)
26 #plt.show()
27 plt.savefig("figures/generated/burgers-via-explicit-fdm.png", dpi=300, bbox_inches='tight')
28 plt.close()

```

Comparison and Notes

- The Cole–Hopf transform provides an exact solution for smooth initial data and is especially useful for illustrating the effect of viscosity.
- The finite difference scheme is easy to implement and effective for moderate viscosity and smooth solutions, but may require small time steps due to the CFL condition.
- For small ν , shock formation can occur, requiring higher-resolution methods or implicit solvers.

12.6 Multigrid Methods

Multigrid Methods are iterative techniques for solving large linear systems arising from discretized PDEs. They accelerate convergence by solving the problem on multiple scales.

12.6.1 Basic Idea

Multigrid methods use a hierarchy of grids with different resolutions. The basic steps are:

- **Smoothing:** Apply a few iterations of a simple iterative method (e.g., Gauss-Seidel) to reduce high-frequency errors.

- **Restriction:** Transfer the residual to a coarser grid.
- **Coarse Grid Correction:** Solve the error equation on the coarse grid.
- **Prolongation:** Interpolate the coarse grid correction back to the fine grid.
- **Correction:** Update the fine grid solution with the interpolated correction.

12.6.2 Multigrid Cycle

There are different types of multigrid cycles, including the V-cycle, W-cycle, and full multigrid cycle. The V-cycle is the most basic and involves recursively applying the multigrid steps on successively coarser grids.

Algorithm 12.1 Multigrid V-Cycle

Require: Initial guess u_h on the finest grid, right-hand side f_h

Ensure: Approximate solution u_h

- 1: Apply pre-smoothing (e.g., Gauss-Seidel) to u_h
 - 2: Compute the residual $r_h = f_h - A_h u_h$
 - 3: Restrict the residual $r_H = R_h^H r_h$ to the coarse grid
 - 4: Solve the coarse grid problem $A_H e_H = r_H$
 - 5: Prolongate the coarse grid correction $e_h = P_H^h e_H$ to the fine grid
 - 6: Correct the fine grid solution $u_h = u_h + e_h$
 - 7: Apply post-smoothing (e.g., Gauss-Seidel) to u_h
 - 8: **return** u_h
-

Example: Solving the Poisson Equation

Consider solving the Poisson equation $-\Delta u = f$ on a 2D domain $\Omega = [0, 1] \times [0, 1]$ with $f(x, y) = 1$. The domain is discretized using a uniform grid with $h = 0.1$.

- **Pre-smoothing:** Apply 2 iterations of Gauss-Seidel.
- **Restriction:** Compute the residual and restrict it to a coarser grid (e.g., using full-weighting).
- **Coarse Grid Correction:** Solve the error equation on the coarse grid using a direct solver or a few iterations of an iterative method.
- **Prolongation:** Interpolate the coarse grid correction back to the fine grid (e.g., using bilinear interpolation).
- **Correction:** Update the fine grid solution.
- **Post-smoothing:** Apply 2 more iterations of Gauss-Seidel.

12.6.3 Applications and Advantages

Multigrid methods are highly efficient for solving large-scale problems, particularly those arising from elliptic PDEs. They offer optimal complexity, typically $\mathcal{O}(N)$, where N is the number of unknowns.

12.7 Exercises

Exercise 12.1. Use the finite difference method to solve the heat equation $\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$ on the domain $0 \leq x \leq 1$ with $\alpha = 1$, $\Delta x = 0.1$, and $\Delta t = 0.005$. Assume the boundary conditions $u(0, t) = 0$ and $u(1, t) = 0$, and the initial condition $u(x, 0) = \sin(\pi x)$. Perform 10 time steps and plot the solution. $\square$

Exercise 12.2. Consider the 1D wave equation $\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2}$. Discretize this equation using the finite difference method with $c = 1$, $\Delta x = 0.1$, and $\Delta t = 0.01$. Assume the initial conditions $u(x, 0) = \sin(\pi x)$ and $\frac{\partial u}{\partial t}(x, 0) = 0$. Perform 20 time steps and plot the solution. $\square$

Exercise 12.3. Solve the 1D Poisson equation $-\frac{d^2u}{dx^2} = 1$ on the interval $0 \leq x \leq 1$ with boundary conditions $u(0) = 0$ and $u(1) = 0$ using the finite element method. Divide the interval into 4 elements and use linear basis functions. Compute the stiffness matrix and load vector, and solve the resulting linear system. $\square$

Exercise 12.4. Apply the finite element method to solve the 2D Poisson equation $-\Delta u = 1$ on the unit square $\Omega = [0, 1] \times [0, 1]$ with Dirichlet boundary conditions $u = 0$ on the boundary. Use a uniform grid with $h = 0.25$ and linear basis functions. Formulate the weak form, compute the stiffness matrix and load vector, and solve the resulting linear system. $\square$

Exercise 12.5. Use the finite volume method to solve the 1D advection equation $\frac{\partial u}{\partial t} + a \frac{\partial u}{\partial x} = 0$ with $a = 1$, $\Delta x = 0.1$, and $\Delta t = 0.05$. Assume the initial condition $u(x, 0) = \sin(2\pi x)$ on $0 \leq x \leq 1$ with periodic boundary conditions. Perform 10 time steps using the upwind scheme and plot the solution. $\square$

Exercise 12.6. Consider the 1D Burgers' equation $\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}$ with $\nu = 0.1$. Use the finite volume method with a uniform grid $\Delta x = 0.1$ and time step $\Delta t = 0.01$. Discretize the equation and solve for 20 time steps starting with the initial condition $u(x, 0) = -\sin(\pi x)$. $\square$

Exercise 12.7. Implement the multigrid V-cycle method to solve the 2D Poisson equation $-\Delta u = 1$ on the unit square $\Omega = [0, 1] \times [0, 1]$ with Dirichlet boundary conditions $u = 0$ on the boundary. Use a uniform grid with $h = 0.1$. Describe each step of the multigrid cycle and perform 5 V-cycles. $\square$

Exercise 12.8. Use the multigrid method to solve the 1D Poisson equation $-\frac{d^2u}{dx^2} = f(x)$ with $f(x) = \sin(\pi x)$ on the interval $0 \leq x \leq 1$. Assume Dirichlet boundary conditions $u(0) = u(1) = 0$. Implement the full multigrid cycle with 2 pre-smoothing and 2 post-smoothing Gauss-Seidel iterations. Compare the solution with the analytical solution $u(x) = \frac{\sin(\pi x)}{\pi^2}$. $\square$

Exercise 12.9. Explain the difference between the V-cycle and W-cycle in multigrid methods. Implement both cycles to solve the 2D Poisson equation $-\Delta u = 1$ on the unit square $\Omega = [0, 1] \times [0, 1]$ with Dirichlet boundary conditions $u = 0$ on the boundary. Use a uniform grid with $h = 0.2$ and compare the convergence rates. $\square$

Exercise 12.10. Solve the 2D heat equation $\frac{\partial u}{\partial t} = \Delta u$ on the unit square $\Omega = [0, 1] \times [0, 1]$ with initial condition $u(x, y, 0) = \sin(\pi x) \sin(\pi y)$ and Dirichlet boundary conditions $u = 0$ on the boundary. Use the finite difference method with $\Delta x = \Delta y = 0.1$ and $\Delta t = 0.005$. Perform 20 time steps and plot the solution at each time step. $\square$

Exercise 12.11. Implement the finite element method to solve the 2D Helmholtz equation $-\Delta u + k^2 u = f$ on a square domain with $k = 1$ and $f(x, y) = \sin(\pi x) \sin(\pi y)$. Use quadratic basis functions and a uniform grid with $h = 0.2$. Formulate the weak form, compute the stiffness matrix and load vector, and solve the resulting linear system. $\square$

Exercise 12.12. Use the finite volume method to solve the 1D diffusion equation $\frac{\partial u}{\partial t} = \nu \frac{\partial^2 u}{\partial x^2}$ with $\nu = 0.01$. Assume $\Delta x = 0.1$ and $\Delta t = 0.01$. Use the central difference scheme for spatial discretization and the forward Euler method for time integration. Assume the initial condition $u(x, 0) = \cos(\pi x)$ and perform 50 time steps. $\square$

Chapter 13

Numerical Optimization

13.1 Introduction

Numerical optimization involves finding the minimum or maximum of a function using numerical techniques. Optimization problems are pervasive in various fields, including engineering, economics, and machine learning. These problems can be categorized into unconstrained and constrained optimization problems.

13.2 Constrained Optimization

Constrained optimization deals with optimizing a function subject to constraints. Constraints can be in the form of equality constraints $g_i(\mathbf{x}) = 0$ or inequality constraints $h_i(\mathbf{x}) \leq 0$.

13.2.1 Linear Programming

Linear programming (LP) is a method for optimizing a linear objective function, subject to linear equality and inequality constraints.

Problem Formulation

A linear programming problem can be formulated as:

$$\begin{array}{ll}\text{minimize} & \mathbf{c}^T \mathbf{x} \\ \text{subject to} & A\mathbf{x} \leq \mathbf{b} \\ & \mathbf{x} \geq 0\end{array}$$

where $\mathbf{c}$ is the coefficient vector, $\mathbf{x}$ is the vector of decision variables, A is the matrix of coefficients, and $\mathbf{b}$ is the constraint vector.

Simplex Algorithm

The Simplex Algorithm is a popular method for solving linear programming problems.

Algorithm 13.1 Simplex Algorithm

Require: Coefficient matrix A , constraint vector $\mathbf{b}$, and objective function coefficients $\mathbf{c}$

Ensure: Optimal solution $\mathbf{x}$

- 1: Convert the LP problem to standard form
 - 2: Initialize the basic feasible solution
 - 3: **while** the current solution is not optimal **do**
 - 4: Determine the entering and leaving variables
 - 5: Update the basic feasible solution
 - 6: **end while**
 - 7: **return** $\mathbf{x}$
-

Example

Consider the linear programming problem:

$$\begin{aligned} \text{minimize} \quad & -x - y \\ \text{subject to} \quad & x + 2y \leq 4 \\ & 3x + y \leq 6 \\ & x \geq 0, y \geq 0 \end{aligned}$$

Using the Simplex Algorithm, we find the optimal solution.

13.2.2 Nonlinear Programming

Nonlinear programming (NLP) involves optimizing a nonlinear objective function subject to nonlinear constraints.

Problem Formulation

A nonlinear programming problem can be formulated as:

$$\begin{aligned} \text{minimize} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & g_i(\mathbf{x}) = 0, i = 1, \dots, m \\ & h_i(\mathbf{x}) \leq 0, i = 1, \dots, p \end{aligned}$$

where $f(\mathbf{x})$ is the objective function, $g_i(\mathbf{x})$ are the equality constraints, and $h_i(\mathbf{x})$ are the inequality constraints.

Karush-Kuhn-Tucker (KKT) Conditions

The KKT conditions are necessary conditions for a solution to be optimal in a nonlinear programming problem. They generalize the method of Lagrange multipliers.

- Stationarity: $\nabla f(\mathbf{x}^*) + \sum_{i=1}^m \lambda_i \nabla g_i(\mathbf{x}^*) + \sum_{i=1}^p \mu_i \nabla h_i(\mathbf{x}^*) = 0$
- Primal feasibility: $g_i(\mathbf{x}^*) = 0$ and $h_i(\mathbf{x}^*) \leq 0$
- Dual feasibility: $\mu_i \geq 0$
- Complementary slackness: $\mu_i h_i(\mathbf{x}^*) = 0$

Example

Consider the nonlinear programming problem:

$$\begin{aligned} \text{minimize} \quad & x^2 + y^2 \\ \text{subject to} \quad & x + y - 1 = 0 \\ & x \geq 0, y \geq 0 \end{aligned}$$

Using the KKT conditions, we find the optimal solution.

13.3 Exercises

Exercise 13.1. Minimize the function $f(x, y) = (x - 2)^2 + (y - 3)^2$ using Gradient Descent with an initial guess $(x_0, y_0) = (0, 0)$, step size $\alpha = 0.1$, and tolerance $\epsilon = 10^{-6}$. $\square$

Exercise 13.2. Minimize the function $f(x, y) = x^2 + y^2$ using Newton's Method with an initial guess $(x_0, y_0) = (1, 1)$ and tolerance $\epsilon = 10^{-6}$. $\square$

Exercise 13.3. Solve the linear programming problem:

$$\begin{array}{ll}\text{minimize} & -3x + y \\ \text{subject to} & 2x + y \leq 2 \\ & -x + y \leq 1 \\ & x \geq 0, y \geq 0\end{array}$$

using the Simplex Algorithm. $\square$

Exercise 13.4. Solve the nonlinear programming problem:

$$\begin{array}{ll}\text{minimize} & x^2 + y^2 \\ \text{subject to} & x + y - 1 = 0 \\ & x \geq 0, y \geq 0\end{array}$$

using the KKT conditions. $\square$

Chapter 14

Numerical Methods in HPC

14.1 Parallel Computing Fundamentals

Parallel computing involves performing multiple computations simultaneously, leveraging multiple processors to solve complex problems more efficiently.

14.1.1 Parallel Computing Models

Parallel computing models describe the architecture and communication methods used in parallel systems. Common models include:

- **Shared Memory Model:** Multiple processors access a common memory space. Communication between processors occurs through shared variables.
- **Distributed Memory Model:** Each processor has its own local memory. Processors communicate by passing messages.
- **Hybrid Model:** Combines shared and distributed memory models, often used in large-scale HPC systems.

14.1.2 Parallel Programming Basics

Parallel programming involves writing software that can execute multiple tasks simultaneously. Key concepts include:

- **Concurrency:** The ability to run multiple tasks in overlapping time periods.
- **Synchronization:** Mechanisms to ensure correct execution order of tasks.
- **Communication:** Methods for data exchange between tasks or processors.

14.2 Programming for HPC

High Performance Computing (HPC) programming requires specialized tools and libraries to manage parallelism and optimize performance.

14.2.1 OpenMP

OpenMP (Open Multi-Processing) is an API for shared-memory parallel programming in C, C++, and Fortran. It uses compiler directives to parallelize code.

Example

```
1  #include <omp.h>
2  #include <stdio.h>
3
4  int main() {
5      #pragma omp parallel
6      {
7          printf("Hello from thread %d\n", omp_get_thread_num());
8      }
9      return 0;
10 }
```

Listing 14.1: OpenMP Example

14.2.2 MPI (Message Passing Interface)

MPI is a standard for distributed-memory parallel programming. It provides functions for communication and synchronization between processes.

Example

```
1  #include <mpi.h>
2  #include <stdio.h>
3
4  int main(int argc, char** argv) {
5      MPI_Init(&argc, &argv);
6
7      int world_rank;
8      MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
9
10     int world_size;
11     MPI_Comm_size(MPI_COMM_WORLD, &world_size);
12
13     printf("Hello from rank %d out of %d processors\n", world_rank, world_size);
14
15     MPI_Finalize();
16     return 0;
17 }
```

Listing 14.2: MPI Example

14.3 GPU Programming

GPU programming leverages the massive parallelism of Graphics Processing Units (GPUs) to accelerate computation-intensive tasks.

14.3.1 CUDA

CUDA (Compute Unified Device Architecture) is a parallel computing platform and API model created by NVIDIA for GPU programming.

Example

```

1  #include <stdio.h>
2
3  __global__ void add(int *a, int *b, int *c) {
4      int index = threadIdx.x;
5      c[index] = a[index] + b[index];
6  }
7
8  int main() {
9      int a[5] = {1, 2, 3, 4, 5};
10     int b[5] = {10, 20, 30, 40, 50};
11     int c[5];
12     int *d_a, *d_b, *d_c;
13     int size = 5 * sizeof(int);
14
15     cudaMalloc((void **)&d_a, size);
16     cudaMalloc((void **)&d_b, size);
17     cudaMalloc((void **)&d_c, size);
18
19     cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
20     cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice);
21
22     add<<<1, 5>>>(d_a, d_b, d_c);
23
24     cudaMemcpy(c, d_c, size, cudaMemcpyDeviceToHost);
25
26     for (int i = 0; i < 5; i++) {
27         printf("%d + %d = %d\n", a[i], b[i], c[i]);
28     }
29
30     cudaFree(d_a);
31     cudaFree(d_b);
32     cudaFree(d_c);
33
34     return 0;
35 }

```

Listing 14.3: CUDA Example

14.3.2 OpenCL

OpenCL (Open Computing Language) is an open standard for cross-platform, parallel programming of diverse processors, including GPUs and CPUs.

Example

```

1  #include <CL/cl.h>
2  #include <stdio.h>
3
4  const char *programSource =
5  "__kernel void vecadd(__global int *A, __global int *B, __global int *C) { \n"
6      "    int idx = get_global_id(0); \n"
7      "    C[idx] = A[idx] + B[idx]; \n"
8      "} \n";
9
10 int main() {
11     int a[5] = {1, 2, 3, 4, 5};
12     int b[5] = {10, 20, 30, 40, 50};
13     int c[5];
14
15     cl_platform_id platform;
16     clGetPlatformIDs(1, &platform, NULL);
17

```

```

18  cl_device_id device;
19  clGetDeviceIDs(platform, CL_DEVICE_TYPE_GPU, 1, &device, NULL);
20
21  cl_context context = clCreateContext(NULL, 1, &device, NULL, NULL, NULL);
22
23  cl_command_queue queue = clCreateCommandQueue(context, device, 0, NULL);
24
25  cl_mem d_a = clCreateBuffer(context, CL_MEM_READ_ONLY | CL_MEM_COPY_HOST_PTR, 5 * sizeof(int), a, NULL);
26  cl_mem d_b = clCreateBuffer(context, CL_MEM_READ_ONLY | CL_MEM_COPY_HOST_PTR, 5 * sizeof(int), b, NULL);
27  cl_mem d_c = clCreateBuffer(context, CL_MEM_WRITE_ONLY, 5 * sizeof(int), NULL, NULL);
28
29  cl_program program = clCreateProgramWithSource(context, 1, (const char **)&programSource, NULL, NULL);
30  clBuildProgram(program, 1, &device, NULL, NULL, NULL);
31
32  cl_kernel kernel = clCreateKernel(program, "vecadd", NULL);
33
34  clSetKernelArg(kernel, 0, sizeof(cl_mem), &d_a);
35  clSetKernelArg(kernel, 1, sizeof(cl_mem), &d_b);
36  clSetKernelArg(kernel, 2, sizeof(cl_mem), &d_c);
37
38  size_t globalSize = 5;
39  clEnqueueNDRangeKernel(queue, kernel, 1, NULL, &globalSize, NULL, 0, NULL, NULL);
40
41  clEnqueueReadBuffer(queue, d_c, CL_TRUE, 0, 5 * sizeof(int), c, 0, NULL, NULL);
42
43  for (int i = 0; i < 5; i++) {
44      printf("%d + %d = %d\n", a[i], b[i], c[i]);
45  }
46
47  clReleaseMemObject(d_a);
48  clReleaseMemObject(d_b);
49  clReleaseMemObject(d_c);
50  clReleaseKernel(kernel);
51  clReleaseProgram(program);
52  clReleaseCommandQueue(queue);
53  clReleaseContext(context);
54
55  return 0;
56 }

```

Listing 14.4: OpenCL Example

14.4 Parallel Numerical Algorithms

Parallel numerical algorithms leverage parallel computing to solve large-scale numerical problems more efficiently.

14.4.1 Parallel Root-Finding Algorithms

Parallel root-finding algorithms can be used to find the roots of a function using multiple processors to explore different regions of the search space simultaneously.

Example: Parallel Bisection Method

14.4.2 Parallel Matrix Operations

Parallel matrix operations are essential in many scientific and engineering applications. They include matrix multiplication, inversion, and decomposition.

Algorithm 14.1 Parallel Bisection Method

Require: A function $f(x)$, interval $[a, b]$, tolerance ϵ

Ensure: Approximate root x

- 1: Split the interval $[a, b]$ into subintervals and assign each to a processor
 - 2: **for** each processor **do**
 - 3: **while** the width of the subinterval $> \epsilon$ **do**
 - 4: Compute the midpoint $c = \frac{a+b}{2}$
 - 5: **if** $f(a)f(c) < 0$ **then**
 - 6: Update $b \leftarrow c$
 - 7: **else**
 - 8: Update $a \leftarrow c$
 - 9: **end if**
 - 10: **end while**
 - 11: Return the midpoint as the root
 - 12: **end for**
 - 13: **return** the best approximation among all processors
-

Example: Parallel Matrix Multiplication

Algorithm 14.2 Parallel Matrix Multiplication

Require: Matrices A and B

Ensure: Product matrix C

- 1: Split matrix A and matrix B into submatrices and distribute them among processors
 - 2: **for** each processor **do**
 - 3: Compute the local product of submatrices
 - 4: Gather the results to form the final matrix C
 - 5: **end for**
 - 6: **return** C
-

14.4.3 Parallel ODE/PDE Solvers

Parallel solvers for ordinary differential equations (ODEs) and partial differential equations (PDEs) divide the computational domain and use multiple processors to solve the equations simultaneously.

Example: Parallel Finite Difference Method for PDEs

14.5 Exercises

Exercise 14.1. Write a parallel program using OpenMP to compute the dot product of two vectors. Compare the performance with the serial implementation. $\square$

Exercise 14.2. Implement the parallel Bisection Method for root-finding using MPI. Test the implementation on a given function and compare the results with the serial implementation. $\square$

Exercise 14.3. Write a CUDA program to perform matrix addition. Compare the performance with the CPU implementation. $\square$

Exercise 14.4. Solve the heat equation using the parallel Finite Difference Method with MPI. Visualize the solution at different time steps. $\square$

Algorithm 14.3 Parallel Finite Difference Method for PDEs

Require: PDE to solve, computational domain, grid size, time step

Ensure: Approximate solution

- 1: Discretize the computational domain and distribute subdomains among processors
 - 2: **for** each time step **do**
 - 3: **for** each processor **do**
 - 4: Update the solution using finite difference approximations
 - 5: Exchange boundary data with neighboring processors
 - 6: **end for**
 - 7: **end for**
 - 8: **return** Approximate solution
-

Chapter 15

Monte Carlo Methods

Monte Carlo methods are a broad class of computational algorithms that rely on repeated random sampling to obtain numerical results. These methods are widely used in various fields such as physics, finance, and engineering for numerical integration, optimization, and probabilistic modeling.

15.1 Random Sampling

Random sampling is a fundamental technique in Monte Carlo methods. It involves generating random samples from a given probability distribution to estimate numerical quantities, such as integrals or solutions to problems involving randomness.

15.1.1 Monte Carlo Integration

Monte Carlo integration is a technique for estimating the value of an integral using random sampling. It is particularly useful for high-dimensional integrals that are difficult to evaluate using traditional numerical methods.

Basic Algorithm

The basic algorithm for Monte Carlo integration can be described as follows:

Algorithm 15.1 Monte Carlo Integration

Require: A function $f(x)$ to integrate, the interval $[a, b]$, and the number of random samples N

Ensure: Estimate of the integral $\int_a^b f(x) dx$

- 1: Generate N random samples $x_1, x_2, \dots, x_N$ uniformly distributed in $[a, b]$
- 2: Compute the average value of $f(x)$ at the sampled points:

$$\bar{f} = \frac{1}{N} \sum_{i=1}^N f(x_i)$$

- 3: Estimate the integral:

$$\int_a^b f(x) dx \approx (b - a) \bar{f}$$

- 4: **return** Estimate of the integral
-

Example

Consider the integral:

$$I = \int_0^1 e^x dx$$

Using Monte Carlo integration with $N = 1000$ random samples, we can estimate the value of this integral.

```
1 import numpy as np
2
3 def f(x):
4     return np.exp(x)
5
6 a, b = 0, 1
7 N = 1000
8 samples = np.random.uniform(a, b, N)
9 integral_estimate = (b - a) * np.mean(f(samples))
10
11 print("Estimated integral:", integral_estimate)
```

Listing 15.1: Monte Carlo Integration in Python

15.1.2 Solving Problems Involving Randomness

Random sampling is also used to solve problems involving randomness, such as estimating the probability of events, simulating random processes, and solving stochastic differential equations.

Example: Estimating the Value of π

One classic example of using random sampling is estimating the value of π by simulating random points in a unit square and counting how many fall inside a quarter circle.

```
1 import numpy as np
2
3 N = 10000
4 points = np.random.uniform(-1, 1, (N, 2))
5 inside_circle = np.sum(points[:,0]**2 + points[:,1]**2 <= 1)
6 pi_estimate = (inside_circle / N) * 4
7
8 print("Estimated value of pi:", pi_estimate)
```

Listing 15.2: Estimating π using Monte Carlo Simulation in Python

15.2 Markov Chain Monte Carlo (MCMC)

Markov Chain Monte Carlo (MCMC) is a class of algorithms that generate a sequence of random samples from a probability distribution by constructing a Markov chain. These methods are particularly useful for sampling from complex, high-dimensional distributions.

15.2.1 Introduction to Markov Chains

A Markov chain is a stochastic process that undergoes transitions from one state to another in a state space. It is characterized by the Markov property, which states that the future state depends only on the present state and not on the sequence of states that preceded it.

Markov Property

The Markov property is a memoryless property of a stochastic process. Formally, a process $\{X_t\}$ satisfies the Markov property if

$$P(X_{t+1} = x_{t+1} \mid X_t = x_t, X_{t-1} = x_{t-1}, \dots, X_0 = x_0) = P(X_{t+1} = x_{t+1} \mid X_t = x_t)$$

Transition Matrix

For a discrete-time Markov chain with a finite state space, the transitions between states are described by a transition matrix P . The element P_{ij} represents the probability of transitioning from state i to state j :

$$P_{ij} = P(X_{t+1} = j \mid X_t = i)$$

The rows of the transition matrix sum to 1, as they represent probability distributions:

$$\sum_j P_{ij} = 1$$

Stationary Distribution

A stationary distribution π of a Markov chain is a probability distribution that remains unchanged as the chain evolves. It satisfies the equation:

$$\pi P = \pi$$

where π is a row vector and P is the transition matrix. The stationary distribution provides long-term probabilities for the states of the Markov chain.

Ergodicity

A Markov chain is ergodic if it is possible to reach any state from any other state (irreducibility) and if it does not get trapped in cycles (aperiodicity). An ergodic Markov chain has a unique stationary distribution to which it converges regardless of the initial state.

Example: Simple Random Walk

A simple random walk on the integers is an example of a Markov chain. At each time step, the process moves to the next integer with equal probability of moving up or down:

$$P(X_{t+1} = X_t + 1 \mid X_t) = \frac{1}{2}, \quad P(X_{t+1} = X_t - 1 \mid X_t) = \frac{1}{2}$$

_____ A Simple Random Walk _____

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 np.random.seed(666)
4
5 def simple_random_walk(n_steps):
6     steps = np.random.choice([-1, 1], size=n_steps)
7     return np.cumsum(steps)
8
9 n_steps = 1000
10 walk = simple_random_walk(n_steps)
11 plt.plot(walk)
12 plt.title('Simple Random Walk')
13 plt.xlabel('Step')
14 plt.ylabel('Position')
15 # plt.show()
```

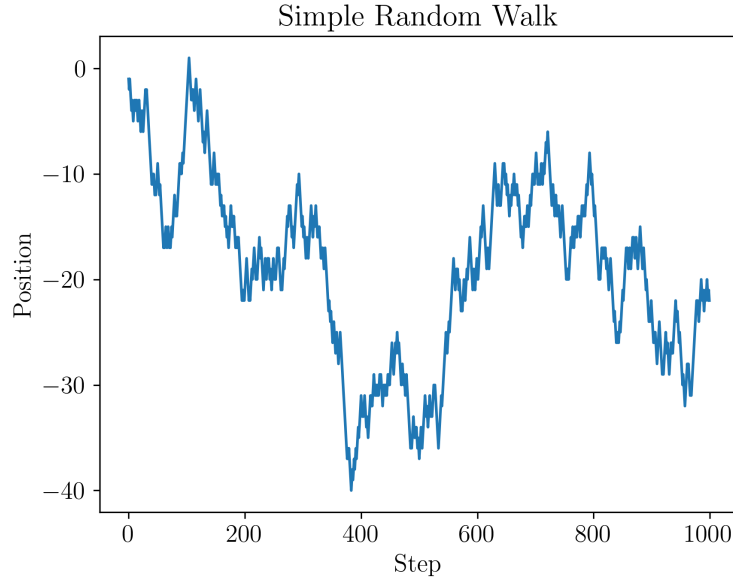


Figure 15.1: A simple random walk

```

16 plt.savefig("figures/generated/simple_random_walk.png", dpi=300, bbox_inches='tight')
17 plt.close()

```

Figure 15.1 depicts the realization of this random walk.

15.3 Reversibility and Detailed Balance

In the context of Markov chains, reversibility and detailed balance are important properties that help ensure the convergence of the Markov chain to a stationary distribution. These properties are particularly useful in the design and analysis of Markov Chain Monte Carlo (MCMC) algorithms.

15.3.1 Reversibility

A stationary Markov chain $\{X_t\}$ is said to be reversible if it satisfies the reversibility condition. Specifically, a Markov chain with transition matrix P and stationary distribution π is reversible if for all states i and j ,

$$\mathbb{P}(X_0 = i, X_1 = j) = \mathbb{P}(X_0 = j, X_1 = i).$$

Note that the set of states can be infinite, and the stationary distribution may not exist.

Physical Interpretation The process “looks the same” when played backward. The statistical properties of the reversed process are identical to the forward process. Key Distinction: Reversibility is the concept of path symmetry, while detailed balance is the specific condition that guarantees this symmetry.

15.3.2 Detailed Balance

The detailed balance condition is a stronger form of the reversibility condition. A Markov chain satisfies the detailed balance condition with respect to its stationary distribution π if for all states i and j ,

$$\pi(i)P_{ij} = \pi(j)P_{ji}.$$

The detailed balance condition ensures that the flow of probability from state i to state j is balanced by the flow of probability from state j to state i . This condition is often used to construct transition matrices that guarantee the stationary distribution π .

Key Distinction Reversibility is the concept of path symmetry, while detailed balance is the specific condition that guarantees this symmetry.

Reversibility vs. Detailed Balance

Detailed balance and reversibility are closely related but conceptually different notions for Markov chains:

- **Detailed balance** is a condition involving a distribution π and a transition kernel P . A Markov chain satisfies detailed balance with respect to π if

$$\pi(x)P(x, dy) = \pi(y)P(y, dx) \quad \text{for all } x, y \in \mathcal{X}.$$

This condition ensures that π is a stationary distribution for the chain.

- **Reversibility** is a property of a Markov chain under its stationary distribution. A chain is reversible with respect to π if the process $(X_0, X_1, X_2, \dots)$ has the same law as $(X_n, X_{n-1}, X_{n-2}, \dots)$ when $X_0 \sim \pi$. Mathematically, this is equivalent to satisfying detailed balance with respect to the stationary distribution.

Summary:

- Detailed balance is a *condition* you verify between π and P .
- Reversibility is a *property* of the Markov chain when detailed balance holds for its stationary distribution.

15.3.3 Example: Simple Random Walk on a Graph

Consider a simple random walk on an undirected graph. At each step, the walker moves to a randomly chosen neighbor of the current node. The transition matrix P of this Markov chain is given by

$$P_{ij} = \begin{cases} \frac{1}{\deg(i)} & \text{if } (i, j) \text{ is an edge,} \\ 0 & \text{otherwise,} \end{cases}$$

where $\deg(i)$ is the degree of node i .

The stationary distribution π for this Markov chain is proportional to the degree of the nodes:

$$\pi(i) = \frac{\deg(i)}{\sum_k \deg(k)}.$$

To verify that the Markov chain is reversible, we check the reversibility condition:

$$\begin{aligned} \pi(i)P_{ij} &= \frac{\deg(i)}{\sum_k \deg(k)} \cdot \frac{1}{\deg(i)} = \frac{1}{\sum_k \deg(k)}, \\ \pi(j)P_{ji} &= \frac{\deg(j)}{\sum_k \deg(k)} \cdot \frac{1}{\deg(j)} = \frac{1}{\sum_k \deg(k)}. \end{aligned}$$

Since $\pi(i)P_{ij} = \pi(j)P_{ji}$, the reversibility condition is satisfied, and the Markov chain is reversible.

15.3.4 Applications in MCMC

The detailed balance condition is crucial in the design of MCMC algorithms such as the Metropolis-Hastings algorithm. It ensures that the constructed Markov chain has the desired stationary distribution.

Metropolis-Hastings Algorithm

In the Metropolis-Hastings algorithm, the transition probability P_{ij} is constructed to satisfy the detailed balance condition with respect to the target distribution π . The acceptance ratio is given by

$$\alpha = \min \left(1, \frac{\pi(x')q(x|x')}{\pi(x)q(x'|x)} \right),$$

where $q(x'|x)$ is the proposal distribution.

By satisfying the detailed balance condition,

$$\pi(x)P(x \rightarrow x') = \pi(x')P(x' \rightarrow x),$$

the algorithm ensures that the stationary distribution of the Markov chain is the target distribution π .

The Metropolis-Hastings algorithm is listed as Algorithm 15.2:

Algorithm 15.2 Metropolis-Hastings Algorithm

Require: A target distribution $\pi(x)$, an initial state x_0 , and a proposal distribution $q(x'|x)$

Ensure: A sequence of samples $\{x_i\}$ from $\pi(x)$

- 1: Initialize x_0
- 2: **for** $i = 0$ to $N - 1$ **do**
- 3: Generate a candidate x' from the proposal distribution $q(x'|x_i)$
- 4: Compute the acceptance ratio:

$$\alpha = \min \left(1, \frac{\pi(x')q(x_i|x')}{\pi(x_i)q(x'|x_i)} \right)$$

- 5: Generate a uniform random number u from $[0, 1]$
 - 6: **if** $u \leq \alpha$ **then**
 - 7: Accept the candidate: $x_{i+1} = x'$
 - 8: **else**
 - 9: Reject the candidate: $x_{i+1} = x_i$
 - 10: **end if**
 - 11: **end for**
 - 12: **return** Sequence of samples $\{x_i\}$
-

Proof that Metropolis-Hastings Satisfies Detailed Balance

Let $\pi(x)$ be the target distribution and $q(y|x)$ be the proposal distribution.

The acceptance probability is

$$\alpha(x, y) = \min \left(1, \frac{\pi(y)q(x|y)}{\pi(x)q(y|x)} \right).$$

The transition probability is

$$P(x, dy) = \begin{cases} q(y|x)\alpha(x, y) dy, & \text{if } y \neq x, \\ 1 - \int_{z \neq x} q(z|x)\alpha(x, z) dz, & \text{if } y = x. \end{cases}$$

We want to show that detailed balance holds:

$$\pi(x)P(x, dy) = \pi(y)P(y, dx), \quad \forall x, y \in \mathcal{X}.$$

Case 1: $x \neq y$

We have:

$$\begin{aligned} \pi(x)P(x, dy) &= \pi(x)q(y|x)\alpha(x, y) dy, \\ \pi(y)P(y, dx) &= \pi(y)q(x|y)\alpha(y, x) dx. \end{aligned}$$

There are two possibilities:

- If $\frac{\pi(y)q(x|y)}{\pi(x)q(y|x)} \leq 1$, then

$$\alpha(x, y) = \frac{\pi(y)q(x|y)}{\pi(x)q(y|x)}, \quad \alpha(y, x) = 1.$$

Therefore,

$$\pi(x)q(y|x)\alpha(x, y) = \pi(y)q(x|y),$$

and

$$\pi(y)q(x|y)\alpha(y, x) = \pi(y)q(x|y).$$

Thus, $\pi(x)P(x, dy) = \pi(y)P(y, dx)$.

- If $\frac{\pi(y)q(x|y)}{\pi(x)q(y|x)} > 1$, then

$$\alpha(x, y) = 1, \quad \alpha(y, x) = \frac{\pi(x)q(y|x)}{\pi(y)q(x|y)}.$$

Thus,

$$\pi(x)q(y|x)\alpha(x, y) = \pi(x)q(y|x),$$

and

$$\pi(y)q(x|y)\alpha(y, x) = \pi(x)q(y|x).$$

Again, $\pi(x)P(x, dy) = \pi(y)P(y, dx)$.

Case 2: $x = y$

By conservation of probability, the total outgoing probability from x must be 1:

$$P(x, \{x\}) = 1 - \int_{z \neq x} q(z|x)\alpha(x, z) dz.$$

Thus, detailed balance at $x = y$ follows from the condition at $x \neq y$.

Conclusion:

The Metropolis-Hastings transition kernel satisfies detailed balance with respect to $\pi(x)$.

□

Detailed Balance Implies Stationarity

Let π be a probability distribution on the state space $\mathcal{X}$, and let $P(x, dy)$ be the transition kernel of a Markov chain.

Suppose that the detailed balance condition holds:

$$\pi(x)P(x, dy) = \pi(y)P(y, dx) \quad \text{for all } x, y \in \mathcal{X}.$$

We aim to show that π is stationary, that is,

$$\int_{\mathcal{X}} \pi(x)P(x, A) dx = \pi(A) \quad \text{for all measurable sets } A \subseteq \mathcal{X},$$

where $\pi(A) = \int_A \pi(y) dy$.

Proof. Fix any measurable set $A \subseteq \mathcal{X}$.

By definition of $P(x, A)$,

$$P(x, A) = \int_A P(x, dy),$$

so

$$\int_{\mathcal{X}} \pi(x)P(x, A) dx = \int_{\mathcal{X}} \pi(x) \left(\int_A P(x, dy) \right) dx.$$

By Fubini's theorem, we can swap the order of integration:

$$= \int_A \left(\int_{\mathcal{X}} \pi(x) P(x, dy) dx \right).$$

By the detailed balance condition,

$$\pi(x) P(x, dy) = \pi(y) P(y, dx),$$

so

$$\int_{\mathcal{X}} \pi(x) P(x, dy) dx = \pi(y) \left(\int_{\mathcal{X}} P(y, dx) \right).$$

Since $P(y, dx)$ is a probability kernel, $\int_{\mathcal{X}} P(y, dx) = 1$, hence

$$\int_{\mathcal{X}} \pi(x) P(x, dy) dx = \pi(y).$$

Thus,

$$\int_{\mathcal{X}} \pi(x) P(x, A) dx = \int_A \pi(y) dy = \pi(A).$$

This shows that π is stationary. □

Example: Sampling from a Normal Distribution

Consider sampling from a standard normal distribution $\pi(x) \sim \mathcal{N}(0, 1)$ using the Metropolis-Hastings algorithm with a proposal distribution $q(x'|x) \sim \mathcal{N}(x, \sigma^2)$.

Sampling from Standard Normal Distribution, Metropolis-Hastings Algorithm

```

1  import numpy as np
2  np.random.seed(666)
3
4  def target_distribution(x):
5      return np.exp(-0.5 * x**2) / np.sqrt(2 * np.pi)
6
7  def proposal_distribution(x, sigma=1.0):
8      return np.random.normal(x, sigma)
9
10 def metropolis_hastings(target, proposal, x0, N, sigma=1.0):
11     samples = [x0]
12     for _ in range(N):
13         x_current = samples[-1]
14         x_candidate = proposal(x_current, sigma)
15         acceptance_ratio = min(1, target(x_candidate) / target(x_current))
16         if np.random.rand() < acceptance_ratio:
17             samples.append(x_candidate)
18         else:
19             samples.append(x_current)
20     return np.array(samples)
21
22 x0 = 0.0
23 N = 10000
24 samples = metropolis_hastings(target_distribution, proposal_distribution, x0, N)
25
26 import matplotlib.pyplot as plt
27 plt.hist(samples, bins=50, density=True)

```

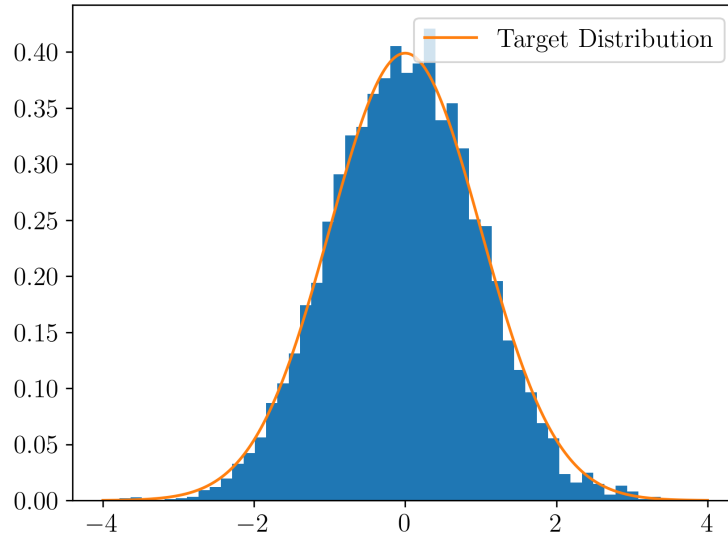


Figure 15.2: Metropolis-Hastings sampling from normal distribution

```

28 x = np.linspace(-4, 4, 1000)
29 plt.plot(x, target_distribution(x), label='Target Distribution')
30 plt.legend()
31 # plt.show()
32 plt.savefig("figures/generated/metropolis_hastings.png", dpi=300, bbox_inches='tight')
33 plt.close()

```

The result is depicted in Figure 15.2.

In this example, the acceptance ratio ensures that the detailed balance condition is satisfied, and the generated samples follow the target normal distribution.

15.4 Global Minimization Using Metropolis-Hastings

We are given the function

$$f(x) = (x - 2)^2 + \sin(5x),$$

for $x \in [-3, 5]$.

We aim to find the global minimum of $f(x)$ using a Metropolis-Hastings-style algorithm.

Algorithm

At each iteration n :

- Current position is X_n .
- Propose $Y_n = X_n + \epsilon_n$ where $\epsilon_n \sim \text{Uniform}(-\delta, \delta)$.
- If $Y_n \notin [-3, 5]$, reject and stay at X_n .
- Otherwise, accept Y_n with probability

$$\alpha(X_n, Y_n) = \min(1, \exp(-\beta(f(Y_n) - f(X_n)))).$$

Solution

Preference for downhill moves

For large β , moves that increase f are almost never accepted. Moves that decrease f are accepted with probability nearly 1. Thus, the algorithm strongly prefers downhill movement.

Acceptance behavior

- If $f(Y_n) > f(X_n)$: acceptance probability tends to 0 as $\beta \rightarrow \infty$.
- If $f(Y_n) < f(X_n)$: acceptance probability is 1.

Simulated annealing interpretation

Allowing $\beta_n \rightarrow \infty$ slowly lets the chain explore initially (tolerating uphill moves) and gradually focus on minima as time progresses.

Sample Code

Metropolis-Hastings Algorithm, simulated annealing interpretation

```
1 import random
2 import math
3 import numpy as np
4 import matplotlib.pyplot as plt
5
6
7 def simulated_annealing(f, X0, N, beta=1):
8     # Parameters
9     X = X0
10    samples = [X0]
11
12    for n in range(1, N+1):
13        Y = X + random.uniform(-delta, delta)
14
15        if Y < -3 or Y > 5:
16            accept = False
17        else:
18            delta_f = f(Y) - f(X)
19            alpha = min(1, math.exp(-beta * delta_f))
20            u = random.uniform(0, 1)
21            accept = (u < alpha)
22
23        if accept:
24            X = Y
25
26        samples.append(X)
27
28    return samples
29
30
31 def f(x):
32     return (x-2)**2 + np.sin(5*x)
33
34
35 low_x = -1
```

```

36 high_x = 3
37 betas = [1, 2, 5, 10, 20]
38 # Create the figure and subplots
39 num_plots = len(betas)+1
40 fig = plt.figure(figsize=(10, 10))
41
42 x = np.linspace(low_x, high_x, 100)
43 y = f(x)
44 plt.subplot(num_plots, 1, 1)
45 plt.plot(x, y, marker='.', linestyle='-')
46 plt.title(f"Plot of f(x)")
47
48 X0 = 0
49 delta = 0.2
50 N = 10000
51 plt_count = 0
52 for beta in betas:
53     plt_count += 1
54     samples = simulated_annealing(f, X0, N, beta)
55     # Important: Initialize for the next batch
56     X0 = samples[-1]
57     plt.subplot(num_plots, 1, 1+plt_count)
58     plt.hist(samples, bins='auto', range=(low_x, high_x))
59     plt.title(f"Histogram of {N} samples, beta={beta}")
60
61 plt.subplots_adjust(hspace=.5)
62
63 #plt.show()
64 plt.savefig("figures/generated/simulated_annealing.png", dpi=300, bbox_inches='tight')
65 plt.close()

```

Figure 15.3 demonstrates that with increasing β the limiting distribution is concentrated around the global minimum of the function. It is important that the next β segment of the sample starts where the previous segment ended. If we started with the same $X_0 = 0$ for each β , the distribution would be centered around the local minimum ≈ 1 , to the left of the global minimum ≈ 2 . In short, our samples would not penetrate the energy barrier between the local and global minimum.

Behavior with $X_0 = 0$, $\delta = 0.2$, $\beta = 10$

The chain initially explores but soon drifts toward the global minimum. After many steps, it spends most time near the minimum.

Limiting behavior

As $\beta_n \rightarrow \infty$ slowly, the limiting distribution concentrates on global minimizers of f .

15.5 Exercises

Exercise 15.1. Consider a Markov chain with transition matrix

$$P = \begin{pmatrix} 0.5 & 0.5 \\ 0.2 & 0.8 \end{pmatrix}.$$

Determine if this Markov chain satisfies the detailed balance condition with respect to its stationary distribution. $\square$

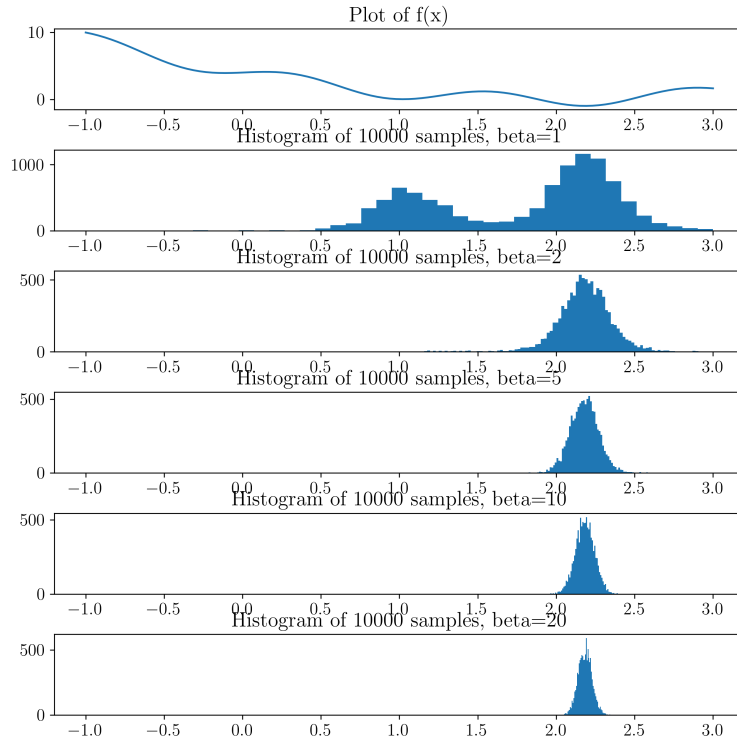


Figure 15.3: Simulated Annealing

Exercise 15.2. Verify that the simple random walk on a graph with the transition matrix defined as $P_{ij} = \frac{1}{\deg(i)}$ satisfies the detailed balance condition with the stationary distribution $\pi(i) = \frac{\deg(i)}{\sum_k \deg(k)}$. $\square$

Exercise 15.3. Implement the Metropolis-Hastings algorithm to sample from a bivariate normal distribution with mean $\mu = \mathbf{0}$ and covariance matrix $\Sigma = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 1 \end{pmatrix}$. Verify that the detailed balance condition is satisfied. $\square$

15.6 Gibbs Sampling

Gibbs sampling is another MCMC method that generates samples from a multivariate distribution by iteratively sampling from the conditional distributions of each variable.

15.6.1 Algorithm

The Gibbs sampling algorithm can be described as follows:

15.6.2 Example

Consider a bivariate normal distribution with mean $\mu = \mathbf{0}$ and covariance matrix $\Sigma = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}$.

Sampling from Bivariate Normal Distribution, Gibbs Sampling

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 np.random.seed(666)
```

Algorithm 15.3 Gibbs Sampling

Require: A target distribution $\pi(x)$, initial state x_0 , and number of variables d

Ensure: A sequence of samples $\{x_i\}$ from $\pi(x)$

- 1: Initialize x_0
 - 2: **for** $i = 0$ to $N - 1$ **do**
 - 3: **for** $j = 1$ to d **do**
 - 4: Sample x_j from the conditional distribution $\pi(x_j|x_{-j})$
 - 5: **end for**
 - 6: Store the sample $x_i = (x_1, x_2, \dots, x_d)$
 - 7: **end for**
 - 8: **return** Sequence of samples $\{x_i\}$
-

```
4
5 def gibbs_sampling(mu, Sigma, x0, N):
6     d = len(mu)
7     samples = np.zeros((N, d))
8     samples[0] = x0
9
10    for i in range(1, N):
11        x = samples[i - 1].copy()
12        for j in range(d):
13            not_j = [k for k in range(d) if k != j]
14
15            # Compute conditional mean for the j-th variable
16            mu_cond = mu[j] + Sigma[j, not_j] @ \
17                np.linalg.inv(Sigma[np.ix_(not_j, not_j)]) @ \
18                (x[not_j] - mu[not_j])
19
20            # Compute conditional variance
21            sigma_cond = Sigma[j, j] - Sigma[j, not_j] @ \
22                np.linalg.inv(Sigma[np.ix_(not_j, not_j)]) @ \
23                Sigma[not_j, j]
24
25            # Sample from conditional distribution
26            x[j] = np.random.normal(mu_cond, np.sqrt(sigma_cond))
27
28        samples[i] = x
29    return samples
30
31
32    # Parameters
33    mu = np.array([0, 0])
34    rho = 0.8
35    Sigma = np.array([[1, rho], [rho, 1]])
36    x0 = np.array([2, 2])
37    N = 1000
38
39    # Generate samples
40    samples = gibbs_sampling(mu, Sigma, x0, N)
41
42    # Plotting the samples
```

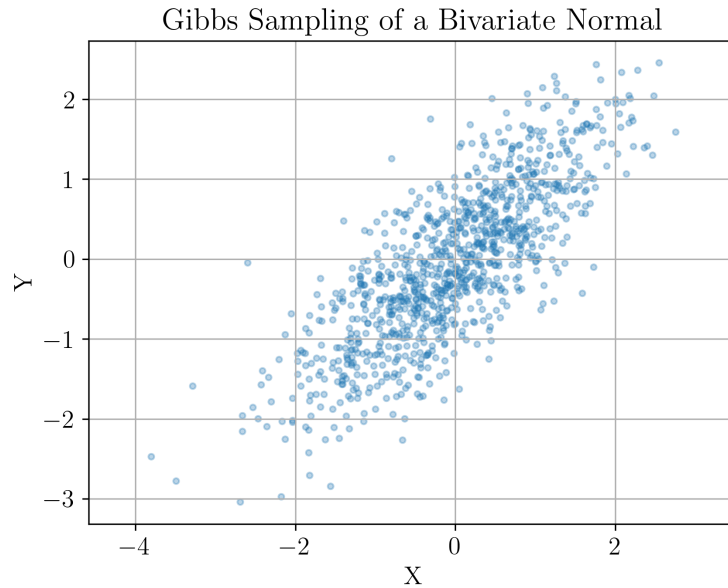


Figure 15.4: Gibbs sampling from 2D normal distribution

```

43 plt.scatter(samples[:, 0], samples[:, 1], alpha=0.3, s=10)
44 plt.title('Gibbs Sampling of a Bivariate Normal')
45 plt.xlabel('X')
46 plt.ylabel('Y')
47 plt.grid(True)
48 plt.axis('equal')
49 # plt.show()
50 plt.savefig("figures/generated/gibbs_sampling.png", dpi=300, bbox_inches='tight')
51 plt.close()

```

The result is depicted in Figure 15.4.

15.7 Importance Sampling

Importance sampling is a variance reduction technique used in Monte Carlo methods. It improves the efficiency of Monte Carlo simulations by focusing sampling efforts on the most "important" parts of the domain, thereby reducing the variance of the estimator.

15.7.1 Motivation

In Monte Carlo integration, we often aim to estimate the integral of a function $f(x)$ over a domain Ω :

$$I = \int_{\Omega} f(x) dx.$$

When $f(x)$ has significant contributions from certain regions of Ω , sampling uniformly may be inefficient. Importance sampling addresses this by sampling from a different distribution that emphasizes these important regions.

15.7.2 Basic Idea

The basic idea of importance sampling is to introduce an importance sampling distribution $g(x)$ with the same support as the original distribution, such that $g(x) > 0$ whenever $f(x) > 0$. We can rewrite the integral as:

$$I = \int_{\Omega} f(x) dx = \int_{\Omega} \frac{f(x)}{g(x)} g(x) dx.$$

By sampling from the distribution $g(x)$ instead of the uniform distribution, we obtain the importance sampling estimator:

$$\hat{I}_N = \frac{1}{N} \sum_{i=1}^N \frac{f(x_i)}{g(x_i)},$$

where $x_i \sim g(x)$ are samples drawn from the importance sampling distribution.

15.7.3 How is the variance reduced?

The estimator $\hat{I}_N$ is a random variable. Its expected value is I , the value of the integral. That is, the estimator is *unbiased*.

The quality of an estimator is typically measured as the expected squared difference between the estimator and the quantity being estimated:

$$\begin{aligned} \text{var}(\hat{I}_N) &= \frac{1}{N} \text{var}(\hat{I}_1) \\ &= \frac{1}{N} \int_{\Omega} \left| \frac{f(x)}{g(x)} - I \right|^2 g(x) dx. \end{aligned}$$

For a fixed g the only way to reduce the variance of the estimator is to increase the sample size N . But if we are also allowed to choose $g \in \mathcal{G}$ then we can achieve the minimum variance by varying g , e.g.,

$$g = \arg \min_{g \in \mathcal{G}} \int_{\Omega} \left| \frac{f(x)}{g(x)} - I \right|^2 g(x) dx.$$

If $\mathcal{G} = \{g_1(x), g_2(x)\}$ then it often (but not always!) pays to pick g for which the ratio $\frac{f(x)}{g(x)}$ varies the least. For instance, we could minimize the spread by setting:

$$g = \arg \min_{g \in \mathcal{G}} \left\{ \max_{x \in \Omega} \frac{f(x)}{g(x)} - \min_{x \in \Omega} \frac{f(x)}{g(x)} \right\}.$$

15.7.4 Choosing the Importance Distribution

The effectiveness of importance sampling depends on the choice of the importance distribution $g(x)$. A good importance distribution should be similar to the target distribution $f(x)$ in shape and should be easy to sample from. Ideally, $g(x)$ should be proportional to $|f(x)|$, but this is often impractical, so approximations are used.

15.7.5 Variance Reduction

One of the main advantages of importance sampling is variance reduction. By choosing a suitable importance distribution $g(x)$, we can significantly reduce the variance of the estimator, leading to more accurate and efficient estimates.

15.7.6 Example

Consider estimating the integral:

$$I = \int_0^1 e^x dx.$$

Instead of sampling uniformly from $[0, 1]$, we use an exponential distribution $g(x) = e^{-x}$ as the importance distribution **on the infinite interval** $[0, \infty)$.

What follows relies upon the definition of the *indicator function* 1_A of a subset $A \subseteq \Omega$, which we formulate here for convenience:

$$1_A(x) = \begin{cases} 1 & \text{if } x \in A, \\ 0 & \text{if } x \notin A. \end{cases}$$

In the context of probability theory, the indicator function is a random variable iff $A \in \Sigma$, where Ω, Σ is the underlying event space (thus Σ is the underlying σ -algebra). Any random variable assuming values $\{0, 1\}$ is called a *Bernoulli random variable*. Clearly, a random variable is a Bernoulli random variable iff it can be written as an indicator function.

Let us return to our example. The integral can be rewritten using indicator function $1_{[0,1]} : [0, \infty) \rightarrow \{0, 1\}$:

$$I = \int_0^1 \frac{e^x}{e^{-x}} e^{-x} dx = \int_0^\infty \frac{e^x}{e^{-x}} 1_{[0,1]} e^{-x} dx = \int_0^1 e^{2x} e^{-x} dx.$$

One should note that the calculation appears as if $g(x) = e^{-x}$, $0 \leq x \leq 1$ were a valid probability distribution function, which it is not. We could use instead

$$\tilde{g}(x) = \frac{e^{-x}}{Z}, \quad 0 \leq x \leq 1$$

where Z is the normalization constant:

$$Z = \int_0^1 e^{-x} dx = [-e^{-x}]_{x=0}^1 = 1 - e^{-1}.$$

The knowledge of Z is not necessary, as it cancels out in calculations. Also, sampling from $\tilde{g}$ can be regarded as a form of *rejection sampling*, i.e., we sample from the exponential distribution $g(x) = e^{-x}$, $0 \leq x < \infty$, and discard (“reject”) the samples which fall outside of $[0, 1]$. Note however, that:

$$\frac{1}{N} \sum_{i=1}^N f(x_i) = \frac{N_{\text{accept}}}{N} \left\{ \frac{1}{N_{\text{accept}}} \sum_{j=1}^{N_{\text{accept}}} f(x_{i_j}) \right\}$$

where $\{i_1, i_2, \dots, i_{N_{\text{accept}}}\}$ are all the indices for which $x_{i_j} \in [0, 1]$. We note that N_{accept} , the number of accepted samples, is a random variable: it depends on the random sample. The equivalence of averaging from $\tilde{g}(x)$ directly, and averaging from $g(x)$ and skipping samples out of the interval $[0, 1]$ relies upon the existence of the limit:

$$\lim_{N \rightarrow \infty} \frac{N_{\text{accept}}}{N} = Z = \int_0^1 g(x) dx.$$

This follows from the *Law of Large Numbers* applied to the indicator function:

$$\frac{N_{\text{accept}}}{N} = \frac{1}{N} \sum_{i=1}^N 1_{[0,1]}(x_i).$$

The convergence is (at least) in probability.

```

1 import numpy as np
2 np.random.seed(666)
3
4 # Target function
5 def f(x):
6     return np.where(x<=1, np.exp(x), 0)
7
8 # Importance sampling distribution (Exponential)
9 def g(x):
10    return np.exp(-x)
11
12 # Sample from the importance distribution
13 N = 10000
14 samples = np.random.exponential(1, N)
15
16 # Compute the importance sampling estimator
17 integral_estimate = np.mean(f(samples) / g(samples))
18
19 print("Estimated integral:", integral_estimate)

```

The approximate value of the integral is 1.6846685652735498. The exact integral is $e - 1 = 1.718281828459045$. If we were to make a choice of $g(x)$ from the set of distributions

$$\mathcal{G} = \{1_{[0,1]}, e^{-x}\}$$

we would pick the function which yields smaller value of the variance of $f(x)/g(x)$ with respect to the sampling distribution $g(x)$.

$$\int_0^1 \left| \frac{e^x}{g(x)} - (e - 1) \right|^2 g(x) dx.$$

We have:

- if $g(x) \equiv 1$:

$$\int_0^1 |e^x - (e - 1)|^2 dx = -\frac{e^2}{2} + 2e - \frac{3}{2} = 0.24203560745276542.$$

- if $g(x) = e^{-x}$ on the interval $[0, \infty)$, the mean square difference is:

$$\begin{aligned} \int_0^\infty |e^{2x} 1_{[0,1]} - (e - 1)|^2 e^{-x} dx &= \int_0^1 |e^{2x} - (e - 1)|^2 e^{-x} dx + \int_1^\infty (e - 1)^2 e^{-x} dx \\ &= \frac{e^3}{3} - e^2 + 2e - \frac{4}{3} = 3.4093531990499955. \end{aligned}$$

It should be noted that in variance considerations the fact that the true sampling distribution is supported on $[0, \infty)$ rather than on $[0, 1]$ is important. It results in an extra term. That is, limiting our integral to $[0, 1]$ would underestimate the variance. The extra variance term captures the variance in N_{accept} .

It is thus clear that the better choice is $g(x) = 1$. That is, by sampling from the exponential distribution we actually *increased* the variance, counter to our objective.

Based on the above considerations, we expect that picking $g(x)$ with more closely resembles $f(x)$ would be beneficial for the purpose of reducing variance. E.g.,

$$g(x) = 1 + x, \quad 0 \leq x \leq 1$$

would do better. The question to confirming this with calculations, and developing strategy to sample from this distribution is left to the reader.

15.7.7 Applications

Importance sampling is widely used in various fields, including:

- **Finance:** For pricing complex financial derivatives and risk management.
- **Physics:** In particle simulations and estimating rare event probabilities.
- **Machine Learning:** For training probabilistic models and Bayesian inference.

15.8 Exercises

Exercise 15.4. Implement the Monte Carlo integration algorithm to estimate the integral:

$$I = \int_0^2 x^3 dx$$

Compare your estimate with the exact value of the integral. $\square$

Exercise 15.5. Use the Metropolis-Hastings algorithm to sample from the exponential distribution $\pi(x) = \lambda e^{-\lambda x}$ for $x \geq 0$ with $\lambda = 1$. Plot the histogram of the samples and compare it with the true distribution. $\square$

Exercise 15.6. Consider the bivariate normal distribution with mean $\mu = \mathbf{0}$ and covariance matrix $\Sigma = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 1 \end{pmatrix}$. Implement the Gibbs sampling algorithm to generate samples from this distribution. Plot the samples and compare the empirical covariance matrix with the true covariance matrix. $\square$

Exercise 15.7. Write a function to perform Gibbs sampling for a trivariate normal distribution with mean $\mu = \mathbf{0}$ and a given covariance matrix Σ . Use your function to generate samples and visualize the pairwise scatter plots of the samples. $\square$

Exercise 15.8. Use importance sampling to estimate the integral:

$$I = \int_0^\infty e^{-x} dx.$$

Compare the results obtained using a uniform distribution and an exponential distribution as the importance sampling distribution. $\square$

Exercise 15.9. Consider the integral:

$$I = \int_0^2 x^3 dx.$$

Use importance sampling with a uniform distribution over $[0, 2]$ and compare the results with direct Monte Carlo integration. $\square$

Exercise 15.10. Implement an importance sampling estimator to approximate the expectation $E[e^X]$ where $X \sim \mathcal{N}(0, 1)$. Use a different normal distribution as the importance sampling distribution and compare the variance of the estimator with that obtained using simple Monte Carlo. $\square$

Exercise 15.11. Let X be a random variable with distribution function $F(x) = P(X \leq x)$.

- (a) Prove that F is an increasing, right-continuous function such that $\lim_{x \rightarrow -\infty} F(x) = 0$ and $\lim_{x \rightarrow \infty} F(x) = 1$.
- (b) Let us assume that $F(x)$ is continuous and strictly monotonic. Prove that the random variable

$$Y = F(X)$$

is uniformly distributed on the interval $[0, 1]$.

$\square$



Exercise 15.12. Let $F : \mathbb{R} \rightarrow [0, 1]$ be an increasing, right-continuous function such that $\lim_{x \rightarrow -\infty} F(x) = 0$ and $\lim_{x \rightarrow \infty} F(x) = 1$.

(a) Prove that there exists an increasing Borel function $G : (0, 1) \rightarrow \mathbb{R}$ such that

$$F \circ G = id_{(0,1)}.$$

(b) Let U be a random variable uniformly distributed on the interval $[0, 1]$. Prove that the random variable

$$X = G(U)$$

has F as its distribution function.

Carefully consider the case when F is not 1:1.

□

Exercise 15.13. Use the results of Exercise 15.12 to develop an algorithm that samples from the distribution (pdf):

$$g(x) \propto \begin{cases} 1+x & 0 \leq x \leq 1, \\ 0 & \text{otherwise.} \end{cases}$$

□

Chapter 16

Advanced Topics in HPC

16.1 Sparse Matrix Techniques

Sparse matrices contain a significant number of zero elements. Efficient storage and computation techniques are crucial for handling sparse matrices in HPC.

16.1.1 Storage Formats

Common storage formats for sparse matrices include:

- **Compressed Sparse Row (CSR)**: Stores non-zero elements and their column indices, along with row pointers.
- **Compressed Sparse Column (CSC)**: Similar to CSR but stores column pointers instead of row pointers.
- **Coordinate List (COO)**: Stores non-zero elements along with their row and column indices.

16.1.2 Sparse Matrix-Vector Multiplication

Efficient sparse matrix-vector multiplication (SpMV) is essential for many scientific and engineering applications. The CSR format is often used for this purpose due to its balance between storage efficiency and computation speed.

```
1  #include <vector>
2
3  void spmv_csr(const std::vector<double> &values, const std::vector<int> &col_indices,
4  const std::vector<int> &row_ptr, const std::vector<double> &x, std::vector<double> &y) {
5      int n = row_ptr.size() - 1;
6      for (int i = 0; i < n; ++i) {
7          y[i] = 0.0;
8          for (int j = row_ptr[i]; j < row_ptr[i+1]; ++j) {
9              y[i] += values[j] * x[col_indices[j]];
10         }
11     }
12 }
```

Listing 16.1: Sparse Matrix-Vector Multiplication in CSR Format

16.2 Advanced Eigenvalue Problems

Eigenvalue problems are fundamental in many areas of science and engineering. Advanced techniques are used to solve large-scale eigenvalue problems efficiently.

16.2.1 Arnoldi Method

The Arnoldi method is an iterative algorithm for finding a few eigenvalues and eigenvectors of large sparse matrices. It is the basis for the Arnoldi iteration used in the GMRES solver for linear systems.

16.2.2 Lanczos Method

The Lanczos method is an iterative algorithm for finding the extreme eigenvalues and eigenvectors of large sparse symmetric matrices. It reduces the matrix to a tridiagonal form, making the eigenvalue problem easier to solve.

16.3 Preconditioning Techniques

Preconditioning improves the convergence rate of iterative solvers for linear systems by transforming the system into an equivalent one that is easier to solve.

16.3.1 Jacobi Preconditioner

The Jacobi preconditioner uses the diagonal elements of the matrix. It is simple to implement and often used as a basic preconditioning technique.

```
1  #include <vector>
2
3  void jacobi_preconditioner(const std::vector<double> &A, std::vector<double> &M_inv) {
4      int n = M_inv.size();
5      for (int i = 0; i < n; ++i) {
6          M_inv[i] = 1.0 / A[i*n + i];
7      }
8  }
```

Listing 16.2: Jacobi Preconditioner

16.3.2 Incomplete LU (ILU) Preconditioner

The ILU preconditioner approximates the LU decomposition of the matrix by dropping some of the fill-in elements, balancing between accuracy and computational cost.

16.4 Monte Carlo and Stochastic Methods

Monte Carlo and stochastic methods use random sampling to estimate numerical results. These methods are useful for solving problems with high dimensionality or complex probabilistic structures.

16.4.1 Monte Carlo Integration

Monte Carlo integration estimates the value of an integral by averaging the results of random samples.

```
1  #include <cstdlib>
2  #include <cmath>
3
4  double monte_carlo_integration(int num_samples) {
5      double sum = 0.0;
6      for (int i = 0; i < num_samples; ++i) {
7          double x = static_cast<double>(rand()) / RAND_MAX;
8          sum += std::sin(x);
9      }
10     return sum / num_samples;
11 }
```

16.4.2 Stochastic Differential Equations

Stochastic differential equations (SDEs) involve differential equations with random components. Numerical methods for SDEs, such as the Euler-Maruyama method, are used to simulate their solutions.

16.5 Big Data Techniques

Big data techniques involve processing and analyzing large datasets that exceed the capabilities of traditional data processing tools.

16.5.1 MapReduce

MapReduce is a programming model for processing large datasets in parallel across a distributed cluster. It involves two main functions: map and reduce.

16.5.2 Apache Spark

Apache Spark is an open-source distributed computing system that provides an interface for programming entire clusters with implicit data parallelism and fault tolerance.

16.6 Deep Learning on GPUs

Deep learning models, particularly deep neural networks, benefit significantly from GPU acceleration due to the parallel nature of training algorithms.

16.6.1 CUDA for Deep Learning

CUDA provides libraries such as cuDNN, which are optimized for deep learning operations like convolutions and recurrent layers.

16.6.2 TensorFlow and PyTorch

TensorFlow and PyTorch are popular deep learning frameworks that support GPU acceleration, making it easier to develop and train deep learning models.

16.7 Emerging Trends and Research Topics in HPC

HPC is a rapidly evolving field with continuous advancements. Emerging trends include quantum computing and exascale computing.

16.7.1 Quantum Computing

Quantum computing leverages quantum mechanics to perform computations that are infeasible for classical computers. It has the potential to revolutionize fields such as cryptography, optimization, and material science.

16.7.2 Exascale Computing

Exascale computing refers to computing systems capable of performing at least one exaflop, or a billion billion (10^{18}) calculations per second. Exascale systems are expected to address complex scientific problems with unprecedented precision and scale.

16.8 Exercises

Exercise 16.1. Implement the CSR format for sparse matrices and write a function for sparse matrix-vector multiplication. Test your implementation on a large sparse matrix. $\square$

Exercise 16.2. Use the Lanczos method to find the largest eigenvalue of a given large sparse symmetric matrix. Compare the results with those obtained using a dense matrix solver. $\square$

Exercise 16.3. Implement the Jacobi preconditioner and use it with the Conjugate Gradient method to solve a large sparse linear system. Compare the performance with and without the preconditioner. $\square$

Exercise 16.4. Write a Monte Carlo simulation to estimate the value of π . Compare the accuracy and performance with increasing numbers of samples. $\square$

Exercise 16.5. Develop a simple deep learning model using TensorFlow or PyTorch. Train the model on a GPU and compare the training time with that on a CPU. $\square$

Exercise 16.6. Research the latest developments in quantum computing and write a report on its potential impact on HPC. $\square$

Chapter 17

Case Studies and Applications

17.1 Scientific Computing Applications

Scientific computing plays a crucial role in modern research and development, enabling the simulation, analysis, and visualization of complex systems. In this section, we explore various case studies that demonstrate the power and versatility of scientific computing. These case studies cover a range of disciplines, including physics, chemistry, biology, and engineering, showcasing the profound impact of computational methods on scientific discovery.

17.1.1 Case Study: Simulating Molecular Dynamics

Molecular dynamics (MD) simulations are a cornerstone of computational chemistry and biophysics. By modeling the interactions of atoms and molecules over time, MD simulations provide insights into the structural and functional properties of biomolecules, materials, and chemical processes.

For example, in drug design, researchers use MD simulations to predict the binding affinities of potential pharmaceutical compounds. By simulating the interaction between a drug molecule and its target protein, scientists can identify the most promising candidates for further experimental validation. This approach not only speeds up the drug discovery process but also reduces costs associated with laboratory testing.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <math.h>
4
5  // Simple MD simulation in C
6  void simulate(double *positions, double *velocities, int num_particles, double timestep, int num_steps) {
7      for (int step = 0; step < num_steps; step++) {
8          for (int i = 0; i < num_particles; i++) {
9              // Update positions and velocities
10             positions[i] += velocities[i] * timestep;
11         }
12     }
13 }
14
15 int main() {
16     int num_particles = 100;
17     double positions[100] = {0};
18     double velocities[100] = {1};
19     simulate(positions, velocities, num_particles, 0.01, 1000);
20     return 0;
21 }
```


17.1.2 Case Study: Climate Modeling

Climate modeling is essential for understanding and predicting changes in the Earth's climate system. High-performance computing (HPC) resources are employed to run sophisticated climate models that incorporate atmospheric, oceanic, and land processes.

For instance, researchers at the National Center for Atmospheric Research (NCAR) use climate models to assess the impacts of greenhouse gas emissions on global temperature and weather patterns. These models help policy-makers develop strategies to mitigate climate change and adapt to its effects.

```
1  import numpy as np
2
3  # Simple climate model in Python
4  def climate_model(temp, co2, num_years):
5      for year in range(num_years):
6          temp += 0.02 * co2
7          co2 += 2
8      return temp
9
10 initial_temp = 15 # Initial global temperature in degrees Celsius
11 initial_co2 = 400 # Initial CO2 concentration in ppm
12 final_temp = climate_model(initial_temp, initial_co2, 100)
13 print("Projected temperature in 100 years:", final_temp)
```

17.2 Optimization in Machine Learning

Optimization is a fundamental aspect of machine learning, underpinning the training of models and the fine-tuning of hyperparameters. In this section, we delve into optimization techniques and their applications in various machine learning tasks. We highlight real-world examples where optimization has led to significant improvements in model performance and computational efficiency.

17.2.1 Case Study: Gradient Descent in Deep Learning

Gradient descent is a widely used optimization algorithm in deep learning, where it is employed to minimize the loss function of neural networks. This case study explores the application of gradient descent in training deep neural networks for image recognition tasks.

For example, the MNIST dataset of handwritten digits is commonly used to benchmark neural network architectures. By using stochastic gradient descent (SGD) with backpropagation, researchers can train a convolutional neural network (CNN) to achieve high accuracy in digit recognition.

```
1  import tensorflow as tf
2  from tensorflow.keras import layers, models
3
4  # Load and preprocess the MNIST dataset
5  mnist = tf.keras.datasets.mnist
6  (train_images, train_labels), (test_images, test_labels) = mnist.load_data()
7  train_images = train_images / 255.0
8  test_images = test_images / 255.0
9
10 # Build a simple CNN
11 model = models.Sequential([
12     layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
13     layers.MaxPooling2D((2, 2)),
14     layers.Conv2D(64, (3, 3), activation='relu'),
15     layers.MaxPooling2D((2, 2)),
16     layers.Flatten(),
17     layers.Dense(64, activation='relu'),
18     layers.Dense(10, activation='softmax')
```

```

19  })
20
21  # Compile and train the model
22  model.compile(optimizer='sgd', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
23  model.fit(train_images, train_labels, epochs=5, validation_data=(test_images, test_labels))

```

17.2.2 Case Study: Hyperparameter Tuning for Support Vector Machines

Support Vector Machines (SVMs) (see Appendix K) are powerful supervised learning models used for classification and regression tasks. The performance of SVMs heavily depends on the selection of hyperparameters, such as the regularization parameter and kernel parameters.

This case study investigates the use of grid search to efficiently tune hyperparameters and achieve optimal model performance. For example, in text classification tasks, tuning the kernel and regularization parameters can significantly improve the accuracy of the SVM model.

```

1  from sklearn import datasets
2  from sklearn.model_selection import GridSearchCV
3  from sklearn.svm import SVC
4
5  # Load the iris dataset
6  iris = datasets.load_iris()
7  X, y = iris.data, iris.target
8
9  # Define the parameter grid
10 param_grid = {
11     'C': [0.1, 1, 10, 100],
12     'gamma': [1, 0.1, 0.01, 0.001],
13     'kernel': ['rbf']
14 }
15
16 # Perform grid search
17 grid = GridSearchCV(SVC(), param_grid, refit=True, verbose=2)
18 grid.fit(X, y)
19
20 # Display the best parameters
21 print("Best parameters found:", grid.best_params_)

```

17.3 Real-World Problem Solving

Applying computational techniques to solve real-world problems is a key goal of many scientific and engineering endeavors. This section presents case studies where computational methods have been successfully applied to address practical challenges across various domains, including healthcare, finance, and logistics.

17.3.1 Case Study: Optimizing Supply Chain Management

Efficient supply chain management is critical for minimizing costs and ensuring timely delivery of goods. This case study examines the application of optimization algorithms to optimize supply chain operations.

For instance, companies like Amazon use linear programming and integer programming to solve problems related to inventory management, transportation, and production scheduling. By optimizing these processes, they can reduce costs and improve customer satisfaction.

```

1  from scipy.optimize import linprog
2
3  # Simple supply chain optimization example

```

```

4  c = [1, 2, 3] # Cost coefficients
5  A = [[1, 1, 0], [0, 1, 1], [1, 0, 1]] # Inequality constraint coefficients
6  b = [10, 15, 20] # Inequality constraint bounds
7  x_bounds = (0, None) # Bounds for variables
8
9  result = linprog(c, A_ub=A, b_ub=b, bounds=[x_bounds, x_bounds, x_bounds])
10 print("Optimal solution:", result.x)

```

17.3.2 Case Study: Data-Driven Healthcare Solutions

In healthcare, data-driven approaches are transforming the way diseases are diagnosed and treated. This case study explores the use of machine learning and data analytics to develop predictive models for disease risk assessment and personalized treatment plans.

For example, predictive modeling can be used to assess the risk of patient readmission to the hospital. By analyzing patient data, such as medical history and lab results, hospitals can identify high-risk patients and implement targeted interventions to reduce readmission rates.

```

1  import pandas as pd
2  from sklearn.model_selection import train_test_split
3  from sklearn.ensemble import RandomForestClassifier
4  from sklearn.metrics import classification_report
5
6  # Load and preprocess the dataset
7  data = pd.read_csv("patient_data.csv")
8  X = data.drop("readmission", axis=1)
9  y = data["readmission"]
10
11 # Split the data into training and testing sets
12 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
13
14 # Train a random forest classifier
15 model = RandomForestClassifier(n_estimators=100)
16 model.fit(X_train, y_train)
17
18 # Evaluate the model
19 y_pred = model.predict(X_test)
20 print(classification_report(y_test, y_pred))

```

17.3.3 Case Study: Financial Portfolio Optimization

Portfolio optimization is a key aspect of financial management, where the goal is to maximize returns while minimizing risk. This case study delves into the application of optimization techniques, such as mean-variance optimization and robust optimization, to construct optimal investment portfolios.

For example, the mean-variance optimization model, developed by Harry Markowitz, helps investors balance the trade-off between risk and return. By using historical market data, investors can identify the portfolio composition that offers the highest expected return for a given level of risk.

```

1  import numpy as np
2  import pandas as pd
3  import matplotlib.pyplot as plt
4  np.random.seed(666)
5
6  # Load historical returns data
7  data = pd.read_csv("historical_returns.csv")
8  returns = data.pct_change().mean()
9  cov_matrix = data.pct_change().cov()

```

```
10
11 # Define the optimization problem
12 num_assets = len(returns)
13 weights = np.random.random(num_assets)
14 weights /= np.sum(weights)
15
16 # Calculate expected portfolio return and risk
17 expected_return = np.dot(weights, returns)
18 expected_risk = np.sqrt(np.dot(weights.T, np.dot(cov_matrix, weights)))
19
20 print("Expected portfolio return:", expected_return)
21 print("Expected portfolio risk:", expected_risk)
```


Chapter 18

Practical Considerations and Software

18.1 Programming Languages and Tools

Choosing the right programming language and tools is crucial for efficient scientific computing and software development. This section provides an overview of popular programming languages and tools used in scientific and high-performance computing (HPC).

18.1.1 Python

Python is widely used in scientific computing due to its simplicity and extensive ecosystem of libraries. It is particularly favored for data analysis, machine learning, and rapid prototyping. Key features of Python include:

- Easy-to-read syntax
- Large standard library
- Extensive support for scientific libraries such as NumPy, SciPy, and Pandas
- Integration with other languages like C/C++ and Fortran

Example of a simple numerical calculation in Python:

```
1 import numpy as np
2
3 # Calculate the sine of pi/2
4 result = np.sin(np.pi / 2)
5 print("The sine of pi/2 is:", result)
```

The result can be accessed if the above code is executed using the $\text{\LaTeX}$ *pythonex* package from CTAN or elsewhere. The same code wrapped into the pycode environment: The sine of $\pi/2$ is: 1.0 Te source of this book needs to be compiled **three times**:

```
xelatex numbook.tex
pythontex numbook.tex
xelatex numbook.tex
```

Now we can print the result: 1.0.

18.1.2 C/C++

C and C++ are powerful programming languages commonly used for performance-critical applications. They provide fine-grained control over hardware resources, making them ideal for HPC applications. Key features include:

- High performance and efficiency
- Extensive support for parallel programming (e.g., OpenMP, MPI)
- Strong standard libraries and support for numerical computation (e.g., BLAS, LAPACK)

Example of a simple program to compute the dot product of two vectors in C++:

```

1  #include <iostream>
2  #include <vector>
3
4  double dot_product(const std::vector<double>& a, const std::vector<double>& b) {
5      double result = 0.0;
6      for (size_t i = 0; i < a.size(); ++i) {
7          result += a[i] * b[i];
8      }
9      return result;
10 }
11
12 int main() {
13     std::vector<double> vec1 = {1.0, 2.0, 3.0};
14     std::vector<double> vec2 = {4.0, 5.0, 6.0};
15     std::cout << "Dot product: " << dot_product(vec1, vec2) << std::endl;
16     return 0;
17 }

```

18.1.3 MATLAB

MATLAB is a high-level language and environment designed for numerical computing, visualization, and programming. It is widely used in academia and industry for tasks such as data analysis, algorithm development, and modeling. Key features include:

- Interactive environment with integrated debugging and visualization tools
- Extensive library of built-in functions for mathematical and engineering applications
- Simulink for simulation and model-based design

Example of a simple script to plot a sine wave in MATLAB:

```

1  x = linspace(0, 2*pi, 100);
2  y = sin(x);
3  plot(x, y);
4  title('Sine Wave');
5  xlabel('x');
6  ylabel('sin(x)');

```

18.2 Numerical Libraries

Numerical libraries provide efficient and reliable implementations of mathematical functions and algorithms. This section covers some of the most widely used numerical libraries in scientific computing.

18.2.1 NumPy and SciPy

NumPy and SciPy are fundamental libraries for numerical computing in Python. NumPy provides support for arrays, matrices, and many mathematical functions, while SciPy builds on NumPy with additional functionality for optimization, integration, and statistical analysis.

Example of using NumPy and SciPy for solving a linear system of equations:

```

1 import numpy as np
2 from scipy.linalg import solve
3
4 # Coefficient matrix
5 A = np.array([[3, 1], [1, 2]])
6 # Right-hand side vector
7 b = np.array([9, 8])
8
9 # Solve the system Ax = b
10 x = solve(A, b)
11 print("Solution:", x)

```

18.2.2 BLAS and LAPACK

BLAS (Basic Linear Algebra Subprograms) and LAPACK (Linear Algebra Package) are highly optimized libraries for linear algebra operations. They are written in Fortran but have interfaces for various programming languages, including C, C++, and Python.

Example of using LAPACK to perform a matrix inversion in C:

```

1 #include <stdio.h>
2 #include <lapacke.h>
3
4 int main() {
5     int n = 3;
6     int lda = 3;
7     int ipiv[3];
8     double a[9] = {1.0, 2.0, 3.0, 0.0, 1.0, 4.0, 5.0, 6.0, 0.0};
9
10    LAPACKE_dgetrf(LAPACK_ROW_MAJOR, n, n, a, lda, ipiv);
11    LAPACKE_dgetri(LAPACK_ROW_MAJOR, n, a, lda, ipiv);
12
13    printf("Inverted matrix:\n");
14    for (int i = 0; i < n; ++i) {
15        for (int j = 0; j < n; ++j) {
16            printf("%f ", a[i*n + j]);
17        }
18        printf("\n");
19    }
20
21    return 0;
22 }

```

18.2.3 TensorFlow and PyTorch

TensorFlow and PyTorch are popular libraries for machine learning and deep learning. They provide tools for building and training neural networks, and they are widely used in both research and industry.

Example of a simple neural network in PyTorch:

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4
5 # Define a simple neural network
6 class SimpleNN(nn.Module):
7     def __init__(self):
8         super(SimpleNN, self).__init__()
9         self.fc1 = nn.Linear(2, 2)

```



```

10     self.fc2 = nn.Linear(2, 1)
11
12     def forward(self, x):
13         x = torch.relu(self.fc1(x))
14         x = torch.sigmoid(self.fc2(x))
15         return x
16
17     # Create a model instance
18     model = SimpleNN()
19     criterion = nn.BCELoss()
20     optimizer = optim.SGD(model.parameters(), lr=0.01)
21
22     # Dummy data
23     data = torch.tensor([[1.0, 2.0], [3.0, 4.0], [5.0, 6.0]])
24     target = torch.tensor([[0.0], [1.0], [0.0]])
25
26     # Training loop
27     for epoch in range(1000):
28         optimizer.zero_grad()
29         output = model(data)
30         loss = criterion(output, target)
31         loss.backward()
32         optimizer.step()
33
34     print("Trained model output:", model(data))

```

18.3 Optimization Techniques for HPC

High-performance computing often requires optimization techniques to maximize the efficiency and performance of computational tasks. This section discusses various optimization techniques used in HPC environments.

18.3.1 Parallel Computing

Parallel computing involves dividing a computational task into smaller subtasks that can be executed concurrently on multiple processors. Common parallel programming models include:

- MPI (Message Passing Interface) for distributed memory systems
- OpenMP for shared memory systems
- CUDA for GPU computing

Example of using OpenMP to parallelize a loop in C:

```

1  #include <stdio.h>
2  #include <omp.h>
3
4  int main() {
5      int n = 1000000;
6      double sum = 0.0;
7
8      #pragma omp parallel for reduction(+:sum)
9      for (int i = 0; i < n; ++i) {
10         sum += 1.0 / (i + 1);
11     }
12
13     printf("Sum: %f\n", sum);
14     return 0;
15 }

```

18.3.2 Vectorization

Vectorization involves using SIMD (Single Instruction, Multiple Data) instructions to perform the same operation on multiple data points simultaneously. Modern processors have SIMD units that can significantly speed up numerical computations.

Example of vectorized computation using NumPy in Python:

```
1 import numpy as np
2
3 # Generate large arrays
4 a = np.random.rand(1000000)
5 b = np.random.rand(1000000)
6
7 # Vectorized addition
8 c = a + b
9 print("Sum of arrays:", c)
```

18.3.3 Memory Optimization

Efficient memory usage is critical for achieving high performance in HPC applications. Techniques for memory optimization include:

- Minimizing memory access latency by optimizing data locality
- Using memory-efficient data structures
- Reducing memory footprint by avoiding unnecessary data duplication

Example of using memory-efficient data structures in Python:

```
1 import numpy as np
2
3 # Using a memory-efficient array with dtype float32
4 a = np.array([1.0, 2.0, 3.0], dtype=np.float32)
5 print("Array:", a)
6 print("Memory usage (bytes):", a.nbytes)
```


Chapter 19

Final Project Guidelines

The final project is a critical component of this course, providing an opportunity for students to apply the knowledge and skills they have acquired. This chapter outlines the guidelines for selecting, implementing, documenting, presenting, and evaluating the final project.

19.1 Project Selection

Choosing a suitable project is the first step toward a successful final project. The project should be challenging yet feasible, allowing you to demonstrate your understanding of the course material.

19.1.1 Identify Your Interests

Select a project that aligns with your interests and career goals. This will keep you motivated and engaged throughout the project. Consider areas where you have a strong foundation or where you wish to deepen your knowledge.

19.1.2 Literature Review

Conduct a thorough literature review to understand the current state of research and development in your chosen area. Identify gaps or open problems that you can address in your project. Utilize academic journals, conference proceedings, and reputable online sources for your review.

19.1.3 Define Clear Objectives

Clearly define the objectives of your project. What are you aiming to achieve? What are the key questions or problems you want to address? Well-defined objectives will guide your project and help you stay focused.

19.1.4 Feasibility Assessment

Assess the feasibility of your project. Consider the availability of resources, such as data, software tools, and computational power. Ensure that you can complete the project within the given timeframe and with the resources at your disposal.

19.1.5 Consultation with Advisors

Discuss your project ideas with your advisors or instructors. They can provide valuable feedback, help you refine your objectives, and suggest resources or methodologies you may not have considered.

19.1.6 Project Proposal

Prepare a detailed project proposal. Your proposal should include the following sections:

- **Title:** A concise and descriptive title for your project.
- **Abstract:** A brief summary of your project objectives and methodology.
- **Introduction:** Background information and the significance of your project.
- **Objectives:** Clear and specific objectives of your project.
- **Methodology:** An outline of the methods and tools you will use.
- **Timeline:** A schedule for completing the project tasks.
- **Resources:** A list of resources required for your project.

19.2 Implementation and Documentation

The implementation phase involves developing your project according to the plan outlined in your proposal. Proper documentation is essential for tracking your progress and ensuring that others can understand and replicate your work.

19.2.1 Development Environment

Set up a suitable development environment. Ensure that you have all necessary software tools and libraries installed. Use version control systems like Git to manage your code and track changes.

19.2.2 Modular Development

Implement your project in a modular fashion. Break down the project into smaller, manageable components. This will make debugging and testing easier and allow for incremental progress.

19.2.3 Testing and Validation

Thoroughly test your implementation. Validate your results to ensure accuracy and reliability. Use unit tests, integration tests, and system tests to cover different aspects of your project.

19.2.4 Documentation

Document your code and methodology comprehensively. Good documentation should include:

- **Code Comments:** Inline comments explaining the purpose and functionality of your code.
- **README File:** An overview of your project, including setup instructions, usage examples, and a summary of the project.
- **Methodology:** A detailed description of the methods and algorithms used in your project.
- **Results:** A presentation of your results, including visualizations, graphs, and tables.
- **Discussion:** An analysis of your results, discussing their significance and any limitations or challenges encountered.

19.2.5 Ethical Considerations

Consider any ethical implications of your project. Ensure that your project adheres to ethical guidelines, especially if it involves human subjects, sensitive data, or environmental impacts.

19.3 Presentation and Evaluation

Presenting your project effectively is crucial for communicating your findings and demonstrating your achievements. Your project will be evaluated based on several criteria, including the quality of your work, your understanding of the material, and your ability to communicate your results.

19.3.1 Preparation

Prepare a clear and concise presentation. Your presentation should cover the following:

- **Introduction:** Introduce yourself and your project.
- **Objectives:** Clearly state the objectives of your project.
- **Methodology:** Explain the methods and tools used in your project.
- **Results:** Present your results using visual aids such as slides, charts, and graphs.
- **Discussion:** Discuss the significance of your results and any challenges faced.
- **Conclusion:** Summarize your findings and suggest future work or improvements.

19.3.2 Visual Aids

Use visual aids to enhance your presentation. Slides should be well-organized and visually appealing. Use diagrams, charts, and images to illustrate your points. Ensure that text is readable and not overcrowded.

19.3.3 Rehearsal

Rehearse your presentation multiple times. Practice delivering your presentation within the allotted time. Rehearsing will help you become more familiar with the content and improve your delivery.

19.3.4 Delivery

Deliver your presentation confidently and clearly. Maintain eye contact with your audience and speak at a moderate pace. Be prepared to answer questions from the audience.

19.3.5 Evaluation Criteria

Your project will be evaluated based on the following criteria:

- **Originality:** The novelty and creativity of your project.
- **Relevance:** The significance and relevance of your project to the course material.
- **Methodology:** The appropriateness and rigor of your methods and tools.
- **Results:** The accuracy, clarity, and significance of your results.
- **Documentation:** The completeness and quality of your documentation.
- **Presentation:** The clarity, organization, and professionalism of your presentation.

19.3.6 Feedback

After your presentation, seek feedback from your instructors and peers. Constructive feedback will help you identify strengths and areas for improvement. Reflect on the feedback and consider how you can apply it to future projects.

Appendix A

Leveraging AI Tools for Studying Numerical Methods

In the modern era of education, AI tools such as ChatGPT have become invaluable resources for students. These tools can assist in understanding complex concepts, debugging code, locating resources, and providing advice on various aspects of programming and numerical methods. This chapter provides guidelines on how to effectively use ChatGPT and other AI tools in your studies.

A.1 Getting Assistance with Coding

AI tools like ChatGPT can help you with coding by providing explanations, debugging assistance, and code examples. Here's how you can make the most of these tools:

A.1.1 Explaining Concepts

If you encounter a concept that you don't understand, you can ask ChatGPT for an explanation. For example, if you are unsure about the Gauss-Seidel method for solving linear systems, you can ask:

```
1 Can you explain the Gauss-Seidel method for solving linear systems?
```

ChatGPT can provide a detailed explanation, breaking down the concept into understandable parts.

A.1.2 Debugging Code

When you encounter an error in your code, you can share the code with ChatGPT and describe the issue. ChatGPT can help identify the problem and suggest possible fixes. For example:

```
1 I am getting an IndexError in my Python code when trying to access an array element. Here is my code:
2 import numpy as np
3
4 arr = np.array([1, 2, 3])
5 print(arr[3])
6 What is causing this error?
```

ChatGPT can point out that the index 3 is out of bounds for an array of size 3 and suggest using a valid index.

A.1.3 Code Examples and Templates

You can request code examples or templates for specific tasks. For instance, if you need a template for implementing the Newton-Raphson method in Python, you can ask:

```
1 Can you provide a Python template for the Newton-Raphson method?
```

ChatGPT can generate a basic template for you to build upon.

A.2 Locating Resources

AI tools can assist you in finding relevant resources such as tutorials, documentation, and research papers. Here's how you can use them effectively:

A.2.1 Finding Tutorials and Documentation

If you need tutorials or documentation for a specific library or tool, you can ask ChatGPT to point you in the right direction. For example:

```
1 Where can I find documentation for the SciPy library in Python?
```

ChatGPT can provide links to the official documentation or recommend popular tutorials.

A.2.2 Research Papers and Articles

For advanced topics, you might need to refer to research papers or articles. You can ask ChatGPT for recommendations:

```
1 Can you recommend some research papers on numerical methods for solving partial differential equations?
```

ChatGPT can suggest key papers and provide summaries.

A.3 Advice on Programming Languages and Tools

Choosing the right programming languages and tools is crucial for effective learning and project development. ChatGPT can offer advice based on your needs and preferences.

A.3.1 Selecting a Programming Language

If you are unsure about which programming language to use for a particular task, you can ask ChatGPT for recommendations:

```
1 Which programming language is best for numerical simulations in scientific computing?
```

ChatGPT can provide insights into the strengths and weaknesses of various languages, such as Python, C++, and MATLAB.

A.3.2 Exploring Software Tools and Libraries

You can seek advice on the best tools and libraries for your projects. For example:

```
1 What are the best libraries for numerical optimization in Python?
```

ChatGPT can list popular libraries such as SciPy, NumPy, and TensorFlow, and explain their features.

A.4 Ethical Considerations and Best Practices

While AI tools are incredibly useful, it's important to use them responsibly and ethically. Here are some best practices:

A.4.1 Avoiding Plagiarism

Always ensure that the work you submit is your own. Use AI tools to understand concepts and get assistance, but do not copy responses verbatim without proper attribution.

A.4.2 Validating Information

Verify the information provided by AI tools with trusted sources. AI tools are powerful, but they can sometimes generate incorrect or outdated information.

A.4.3 Continuous Learning

Use AI tools as a supplement to your learning, not a replacement. Engage with the material, practice coding, and seek to understand the underlying principles.

A.4.4 Data Privacy

Be mindful of the data you share with AI tools. Avoid sharing sensitive or personal information and ensure you are complying with data privacy regulations.

A.5 Case Studies: Using ChatGPT for Numerical Methods

Here are a few case studies illustrating how ChatGPT can be used effectively in studying numerical methods:

A.5.1 Case Study: Solving Differential Equations

A student is learning about numerical methods for solving differential equations. They ask ChatGPT for an example of solving an ordinary differential equation (ODE) using the Runge-Kutta method in Python.

```
1 Can you provide an example of solving an ODE using the Runge-Kutta method in Python?
```

ChatGPT provides a detailed code example, explaining each step of the method.

A.5.2 Case Study: Matrix Operations

A student is struggling with matrix operations in C++. They ask ChatGPT for help with implementing matrix multiplication.

1

How do I implement matrix multiplication in C++?

ChatGPT provides a sample implementation and explains the logic behind the code.

A.6 Conclusion

AI tools like ChatGPT are transforming the way students learn and interact with complex subjects like numerical methods. By providing instant assistance, resources, and advice, these tools can enhance your understanding and efficiency. Remember to use them responsibly, validate the information, and continue to engage deeply with your studies.

Appendix B

IEEE 754 Floating-Point Standard: A Detailed Overview

Introduction

The IEEE 754 standard, established by the Institute of Electrical and Electronics Engineers (IEEE), provides a universally accepted framework for representing and manipulating floating-point numbers in computers. This standard ensures consistency and reliability in numerical calculations across various computing systems. Here, we delve into the key aspects of the IEEE 754 standard, accompanied by illustrative examples and exercises.

Floating-Point Formats

The IEEE 754 standard specifies multiple formats for representing floating-point numbers:

- **Single Precision (32 bits):**
 - *Sign bit*: 1 bit
 - *Exponent*: 8 bits
 - *Significand (Mantissa)*: 23 bits
- **Double Precision (64 bits):**
 - *Sign bit*: 1 bit
 - *Exponent*: 11 bits
 - *Significand (Mantissa)*: 52 bits

The general form of a floating-point number is:

$$(-1)^{\text{sign}} \times (1.\text{fraction}) \times 2^{(\text{exponent}-\text{bias})}$$

For single precision, the bias is 127, and for double precision, the bias is 1023.

Example: Single Precision Representation

Consider the number -12.375 . To represent this in IEEE 754 single precision:

1. Convert to binary: $12.375 = 1100.011_2$
2. Normalize: 1.100011×2^3
3. Sign bit: 1 (since the number is negative)

4. Exponent: $3 + 127 = 130$ (in binary: 10000010)
5. Mantissa: 100011 (remaining bits padded with zeros)

The IEEE 754 single precision representation is:

1 10000010 100011000000000000000000

Rounding Modes

The IEEE 754 standard defines several rounding modes:

- **Round to Nearest (Even):** Rounds to the nearest representable value. If equidistant, rounds to the nearest even value.
- **Round Toward Zero:** Truncates the result, effectively rounding towards zero.
- **Round Toward Positive Infinity:** Rounds up towards positive infinity.
- **Round Toward Negative Infinity:** Rounds down towards negative infinity.

Special Values

The standard specifies special values for representing non-numeric results:

- **Infinity** (∞ and $-\infty$): Represented by all exponent bits set to 1 and all fraction bits set to 0.
- **NaN (Not a Number):** Represented by all exponent bits set to 1 and a non-zero fraction.
- **Subnormal Numbers:** Represent very small values with an exponent of all zeros.

Example: Handling Special Values

Consider the following operations:

- Division by zero: $\frac{1}{0} = \infty$
- Invalid operation: $\sqrt{-1} = \text{NaN}$
- Underflow: $2^{-150} \approx$ subnormal number

Operations and Exceptions

The IEEE 754 standard defines operations for arithmetic, comparisons, and conversions, along with exceptions such as overflow, underflow, divide-by-zero, and invalid operations.

Example: Addition

Consider adding two single precision numbers:

$$a = 1.5 \quad (0\ 01111111\ 100000000000000000000000)$$

$$b = 2.5 \quad (0\ 10000000\ 010000000000000000000000)$$

Align the exponents and add the significands:

$$a + b = 4.0 \quad (0\ 10000001\ 000000000000000000000000)$$

Appendix C

Reading GPS Data in Python

To read GPS data from a GPS device on a drone using Python, you'll typically interact with the GPS module via a serial connection. Here's a step-by-step guide on how to do this:

C.1 Connect the GPS Module

Ensure your GPS module is properly connected to your drone and your computer (if necessary) via USB, UART, or another communication protocol.

C.2 Install Required Libraries

You'll need the `pyserial` library to handle serial communication and `pynmea2` to parse the NMEA sentences (the standard format for GPS data).

```
pip install pyserial pynmea2
```

C.2.1 Read and Parse GPS Data

Use the following Python script to read and parse data from the GPS module.

```
1 import serial
2 import pynmea2
3
4 # Configure the serial port
5 port = '/dev/ttyUSB0' # Change this to your serial port
6 baudrate = 9600 # Common baudrate for GPS modules
7
8 # Open the serial connection
9 ser = serial.Serial(port, baudrate, timeout=1)
10
11 while True:
12     try:
13         # Read a line of data from the serial port
14         line = ser.readline().decode('ascii', errors='replace')
15
16         # Check if the line starts with the NMEA prefix
17         if line.startswith('$'):
18             # Parse the NMEA sentence
19             msg = pynmea2.parse(line)
20
21         # Check the type of NMEA sentence
22         if isinstance(msg, pynmea2.types.talker.GGA):
```



```

23     # GGA sentences contain the fix information
24     print(f'Timestamp: {msg.timestamp}')
25     print(f'Latitude: {msg.latitude} {msg.lat_dir}')
26     print(f'Longitude: {msg.longitude} {msg.lon_dir}')
27     print(f'Altitude: {msg.altitude} {msg.altitude_units}')
28     print(f'Number of Satellites: {msg.num_sats}')
29     print(f'Horizontal Dilution of Precision: {msg.horizontal_dil}')
30 elif isinstance(msg, pynmea2.types.talker.RMC):
31     # RMC sentences contain the recommended minimum data
32     print(f'Timestamp: {msg.timestamp}')
33     print(f'Status: {msg.status}')
34     print(f'Latitude: {msg.latitude} {msg.lat_dir}')
35     print(f'Longitude: {msg.longitude} {msg.lon_dir}')
36     print(f'Speed Over Ground: {msg.spd_over_grnd} knots')
37     print(f'True Course: {msg.true_course}')
38     print(f'Date: {msg.datestamp}')
39     print(f'Magnetic Variation: {msg.mag_variation} {msg.mag_var_dir}')
40 except pynmea2.ParseError as e:
41     print(f'Failed to parse NMEA sentence: {e}')
42 except serial.SerialException as e:
43     print(f'Serial connection error: {e}')
44     break
45 except KeyboardInterrupt:
46     print("Exiting program")
47     break
48
49 # Close the serial connection
50 ser.close()

```

Appendix D

Python Virtual Environments

D.1 Introduction

Python virtual environments are a crucial tool for managing dependencies and ensuring reproducibility in software projects. By isolating the dependencies required for a project, virtual environments help avoid conflicts between different projects and provide a clean, controlled environment for development and experimentation.

D.2 Rationale for Using Virtual Environments

When working on multiple Python projects, it is common to encounter conflicting dependencies. One project might require a specific version of a library that is incompatible with the version needed by another project. Virtual environments solve this problem by creating an isolated space for each project where dependencies can be managed independently. This ensures that changes in one project do not affect others, making development more reliable and manageable.

Additionally, virtual environments help maintain a clean global Python installation, preventing it from being cluttered with packages that are only needed for specific projects. This also aids in replicating the project setup on different machines, enhancing collaboration and deployment processes.

D.3 Creating and Using Virtual Environments

To create and use virtual environments in Python, you can use the built-in `venv` module or the popular `virtualenv` package. Below are steps to create and manage a virtual environment using both tools.

D.3.1 Using `venv`

The `venv` module is included in the Python standard library starting from Python 3.3. Here is how you can create and activate a virtual environment using `venv`:

```
1  # Step 1: Create a virtual environment
2  python3 -m venv myenv
3
4  # Step 2: Activate the virtual environment
5  # On Windows
6  myenv\Scripts\activate
7
8  # On Unix or MacOS
9  source myenv/bin/activate
```

Listing D.1: Creating and activating a virtual environment using `venv`

Once activated, you will notice that the command prompt changes to indicate that you are now working within the virtual environment. Any Python packages you install using pip will be isolated to this environment.

```
1 # Step 3: Install packages within the virtual environment
2 pip install numpy scipy matplotlib
```

Listing D.2: Installing packages in the virtual environment

To deactivate the virtual environment, simply run:

```
1 deactivate
```

Listing D.3: Deactivating the virtual environment

D.3.2 Using virtualenv

The virtualenv package provides similar functionality and is compatible with both Python 2 and 3. Here are the steps to create and activate a virtual environment using virtualenv:

```
1 # Step 1: Install virtualenv (if not already installed)
2 pip install virtualenv
3
4 # Step 2: Create a virtual environment virtualenv myenv
5
6 # Step 3: Activate the virtual environment # On Windows
7 myenv\Scripts\activate
8
9 # On Unix or MacOS source myenv/bin/activate
```

Listing D.4: Creating and activating a virtual environment using virtualenv

As with venv, you can install packages and manage dependencies within this isolated environment.

D.4 Example: Setting Up a Virtual Environment for Numerical Methods

Let's walk through an example of setting up a virtual environment for a numerical methods project. Suppose you need to use libraries such as NumPy, SciPy, and Matplotlib.

```
1 # Step 1: Create a virtual environment
2 python3 -m venv numenv
3
4 # Step 2: Activate the virtual environment
5 source numenv/bin/activate
6
7 # Step 3: Install required packages
8 pip install numpy scipy matplotlib
9
10 # Step 4: Verify installation
11 python -c "import numpy; import scipy; import matplotlib; print('All packages are installed and working!')"
```

Listing D.5: Setting up a virtual environment for a numerical methods project

In this example, we created a virtual environment named numenv, activated it, and installed the necessary packages. The final step verifies that the packages are correctly installed and working.

D.5 Conclusion

Python virtual environments are an essential tool for managing project dependencies and ensuring a clean, reproducible development environment. By following the steps outlined in this appendix, you can effectively isolate and manage the dependencies for your numerical methods projects, leading to more reliable and maintainable code.

D.6 Further Reading

For more information on Python virtual environments, consider the following resources:

- Python’s official documentation on venv: [20]
- virtualenv documentation: [21]
- Python Packaging User Guide: [19]

Appendix E

The Rosenbrock Function and Its Computational Graph

The Rosenbrock function is a well-known test problem for optimization algorithms. It is defined as:

$$f(x, y) = (a - x)^2 + b(y - x^2)^2$$

where $a = 1$ and $b = 100$ are typical parameter choices. This function has a global minimum inside a long, narrow, parabolic shaped flat valley. To compute the gradient of the Rosenbrock function at a given point (x, y) , we can use automatic differentiation as implemented in the following Python code:

```
1  import numpy as np
2
3  class Variable:
4  def __init__(self, value, derivative=0.0):
5      self.value = value
6      self.derivative = derivative
7
8  def __add__(self, other):
9      if isinstance(other, Variable):
10         return Variable(self.value + other.value, self.derivative + other.derivative)
11     else:
12         return Variable(self.value + other, self.derivative)
13
14  def __mul__(self, other):
15      if isinstance(other, Variable):
16         return Variable(self.value * other.value, self.value * other.derivative + self.derivative * other.
17         value)
18     else:
19         return Variable(self.value * other, self.derivative * other)
20
21  def __pow__(self, power):
22      return Variable(self.value ** power, power * self.value ** (power - 1) * self.derivative)
23
24  def rosenbrock(x, y, a=1, b=100):
25      return (a - x)**2 + b * (y - x**2)**2
26
27  # Example usage:
28  x = Variable(1.0, 1.0)
29  y = Variable(1.0, 0.0)
30  f = rosenbrock(x, y)
31  print(f"Value of Rosenbrock function: {f.value}")
32  print(f"Derivative with respect to x: {f.derivative}")
33
34  x = Variable(1.0, 0.0)
35  y = Variable(1.0, 1.0)
```

```
35 f = rosenbrock(x, y)
36 print(f"Derivative with respect to y: {f.derivative}")
```

Listing E.1: Python code for the Rosenbrock function and its gradient calculation

Computational Graph of the Rosenbrock Function

The computational graph for the Rosenbrock function when evaluated at a point (x, y) can be represented as follows:

Figure E.1: Computational graph for the Rosenbrock function, with memory cells and operations, constructed bottom-up.

Appendix F

An automatic differentiation module in Python

In listing we include a concise but powerful implementation of automatic differentiation (AD) in Python. The main take-aways are:

- Implementing AD is not as hard as it may seem at first.
- Adding “syntactic sugar” which makes the code maximally convenient is also straightforward, using decorators.

A bigger take-away is that it is plausible to use one’s own implementation of AD over existing (commercial and public domain) implementations. The reasons could be:

- The lack of support for a particular function that we may want to use
- Simplicity and ease of debugging
- Increasing performance

Thus, we can easily add support for custom functions, or port from Python to a more efficient language, such as C++. For maximum performance, one may add support for exotic hardware which is not readily supported in more bulky software.

```
1 #-----
2 # File:      autodiff.py
3 #-----
4 # A simple autodifferentiation package
5 import numpy as np
6
7 class Variable:
8     def __init__(self, value, derivative=1.0):
9         self.value = value
10        self.derivative = derivative
11
12    def __add__(self, other):
13        if isinstance(other, Variable):
14            return Variable(self.value + other.value, self.derivative + other.derivative)
15        else:
16            return Variable(self.value + other, self.derivative)
17
18    def __radd__(self, other):
19        return self.__add__(other)
20
21    def __sub__(self, other):
22        if isinstance(other, Variable):
23            return Variable(self.value - other.value, self.derivative - other.derivative)
```



```

24         else:
25             return Variable(self.value - other, self.derivative)
26
27     def __rsub__(self, other):
28         return Variable(other - self.value, -self.derivative)
29
30
31     def __mul__(self, other):
32         if isinstance(other, Variable):
33             return Variable(self.value * other.value, self.value * other.derivative + self.derivative * other.
34             value)
35         else:
36             return Variable(self.value * other, self.derivative * other)
37
38     def __rmul__(self, other):
39         return self.__mul__(other)
40
41     def __truediv__(self, other):
42         if isinstance(other, Variable):
43             return Variable(self.value / other.value,
44                             (self.derivative * other.value - self.value * other.derivative) / (other.value ** 2)
45                             )
46         else:
47             return Variable(self.value / other, self.derivative / other)
48
49     def __rtruediv__(self, other):
50         return Variable(other / self.value, -other * self.derivative / (self.value ** 2))
51
52     def __pow__(self, power):
53         return Variable(self.value ** power, power * self.value ** (power - 1) * self.derivative)
54
55     def sin(self):
56         return Variable(np.sin(self.value), np.cos(self.value) * self.derivative)
57
58     def cos(self):
59         return Variable(np.cos(self.value), -np.sin(self.value) * self.derivative)
60
61     def exp(self):
62         return Variable(np.exp(self.value), np.exp(self.value) * self.derivative)
63
64     def to_pair(self):
65         return self.value, self.derivative
66
67     def autodiff(f):
68         def g(x):
69             xx = Variable(x)
70             yy = f(xx)
71             return yy.to_pair()
72
73         return g
74
75 if __name__ == '__main__':
76     # Example usage:
77     x = Variable(2.0)
78     y = x**2 + 3*x + 2
79
80     print(f"Value of expression: {y.value}")
81     print(f"Derivative of expression: {y.derivative}")
82
83

```

Listing F.1: A Python implementation of automatic differentiation with decorators

Syntactic sugar and decorators

It is worth noting that the implementation of the “syntactic sugar”, the decorator `@autodiff`, takes just 6 lines of code, thus illustrating the power of the idea!

Three levels of the autodiff module use

The way the code of the module is used, including modifications, will strongly depend on the experience of the user with Python. The three levels are a reflection of a module development process where consecutive phases gradually focus on the potential user of the module, maximizing the ease of use.

Level 0

The least sophisticated use is the direct use of the class `Variable`. However, the user of the module needs to understand how the class works. The user should not have to read the source code, so we would be expected to document the class in detail.

Level 1

The user (and probably the module author) may notice a *design pattern* in the code, and create a function wrap which returns a function that “knows how to differentiate itself”. This function eventually is moved to the module. This simplifies the use of the module, as only this function needs to be documented. After this is done, the user does not need to be aware of the class `Variable`.

Level 2

The user further generalizes and facilitates the use of a wrapper by applying an advanced feature of Python: decorators. The wrapper is renamed to `autodiff` and it encapsulates the semantics of automatic differentiation completely. The use becomes trivial: we simply precede the definition of the function which will acquire “autodiff powers” with the directive `@autodiff`.

```
1 from autodiff import Variable
2
3 # Level 1 Use: Simply create a Variable x holding
4 # the value and derivative and build an expression
5 # creating a Variable y holding the value and the derivative
6
7
8 x = Variable(2.0)
9 y = x**2 + 3*x + 2
10
11
12 print(f"Value of expression: {y.value}")
13 print(f"Derivative of expression: {y.derivative}")
14
15
```

Listing F.2: Level 0 use: manipulation of `Variable` class

```
1 from autodiff import Variable
2
3 # Create a function that transforms
4 # a normal function f into a function g which
5 # returns (f(x), f'(x))
6
7 f = lambda x: x**2 + 3*x + 2
8
9 def wrap(f):
```

```

10     def g(x):
11         xx = Variable(x)
12         yy = f(xx)
13         return (yy.value, yy.derivative)
14
15     return g
16
17 g = wrap(f)
18
19 y, dy = g(2.0)
20
21 print(f"Value: {y}, Derivative:{dy}")
22

```

Listing F.3: Level 1 use: creation of a wrapper function

```

1 from autodiff import autodiff
2
3 @autodiff
4 def h(x):
5     y = x**2-3*x+1
6     return y
7
8 y, dy = h(2.0)
9
10 print(f"Value: {y}, Derivative:{dy}")
11

```

Listing F.4: Level 2 use: using a decorator

Appendix G

QR Decomposition using Maxima

Introduction

In this section, we will demonstrate how to compute the QR decomposition of a matrix using the Classical Gram-Schmidt (CGS) process in Maxima. We will use the matrix

$$A = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}.$$

Maxima Code for Classical Gram-Schmidt

The following Maxima code performs the Classical Gram-Schmidt process to find the orthogonal matrix Q and upper triangular matrix R such that $A = QR$.

```
1 /* Define matrix A */
2 A : matrix([1, 1, 0], [1, 0, 1], [0, 1, 1]);
3
4 /* Step 1: Calculate q1 */
5 a1 : A[1..3, 1]; /* First column of A */
6 q1 : a1 / sqrt(transpose(a1) . a1);
7
8 /* Step 2: Calculate q2 */
9 a2 : A[1..3, 2]; /* Second column of A */
10 proj_q1_a2 : (transpose(q1) . a2) * q1; /* Projection of a2 onto q1 */
11 q2_unnorm : a2 - proj_q1_a2;
12 q2 : q2_unnorm / sqrt(transpose(q2_unnorm) . q2_unnorm); /* Normalize q2_unnorm */
13
14 /* Step 3: Calculate q3 */
15 a3 : A[1..3, 3]; /* Third column of A */
16 proj_q1_a3 : (transpose(q1) . a3) * q1;
17 proj_q2_a3 : (transpose(q2) . a3) * q2;
18 q3_unnorm : a3 - proj_q1_a3 - proj_q2_a3;
19 q3 : q3_unnorm / sqrt(transpose(q3_unnorm) . q3_unnorm); /* Normalize q3_unnorm */
20
21 /* Construct Q */
22 Q : transpose(matrix(q1, q2, q3));
23
24 /* Calculate R */
25 R : transpose(Q) . A;
26
27 /* Verify result */
28 Q . R;
```

Listing G.1: Maxima Code for QR Decomposition using Classical Gram-Schmidt

Explanation

*** Step 1: Defining the Matrix A *** We define the matrix A using Maxima's `matrix` function. In this case, we use:

$$A = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}.$$

*** Step 2: Orthogonalizing the Columns *** We extract the first column vector a_1 and normalize it to obtain q_1 . Then, we compute the projection of the second column a_2 onto q_1 and subtract it from a_2 to obtain the unnormalized q_2 . This is followed by normalizing q_2 . The same process is applied to find q_3 .

*** Step 3: Constructing Q and R *** The matrix Q is constructed by arranging the orthogonalized and normalized vectors q_1 , q_2 , and q_3 as columns. The matrix R is computed as $R = Q^T \cdot A$.

*** Step 4: Verification *** The code concludes by verifying that $Q \cdot R = A$. This serves as a check for the correctness of the decomposition.

Conclusion

The Maxima code provided uses the Classical Gram-Schmidt process to compute the QR decomposition of a given matrix. This method constructs an orthogonal matrix Q and an upper triangular matrix R , such that $A = QR$. The complete process has been implemented step-by-step using the Maxima computer algebra system.

Maxima built-in Gram-Schmidt function

Maxima supports Gram-Schmidt process, but there is no function finding QR decomposition. However, we can easily add such a function, as it is demonstrated by the code below:

```
1  /-- Mode : maxima -*/
2  load("eigen");
3  /* Put the vectors in rows */
4  A : matrix([1, 1, 0], [1, 0, 1], [0, 1, 1]);
5  /* Perform Gram-Schmidt, normalize vectors and convert to an orthogonal matrix */
6  Q : transpose(apply(matrix, map(unitvector, gramschmidt(A))));
7  /* Verify orthogonality of Q */
8  transpose(Q).Q - identfor(Q), expand;
9  /* Find R; if A = Q.R then R = transpose(Q).A */
10 R : transpose(Q).A, radcan;
11 /* Verify QR-decomposition */
12 Q.R-A, radcan;
13
14 /* Package QR-decomposition as a function */
15 qr(A) := block([Q, R],
16   Q : transpose(apply(matrix, map(unitvector, gramschmidt(A)))),
17   R : radcan(transpose(Q).A),
18   [Q, R]);
19
20 /* Try the new function out */
21 V : qr(A);
22
23 /* Verify components against the values already found */
24 is(equal(V[1], Q));
25 is(equal(V[2], R));
```

Listing G.2: QR Decomposition using built-in Gram-Schmidt

Appendix H

Horner's Scheme and Synthetic division

Horner's Scheme is an efficient algorithm to evaluate a polynomial in one variable

$$p(x) = \sum_{i=0}^n p_i x^i.$$

The direct evaluation of the polynomial would result in $2(n+1)$ multiplies and n additions. Horner's Scheme reduces the number of operations to n multiplies and n additions.

Algorithm H.1 Horner's Scheme

Require: Polynomial coefficients $p = (p_0, p_1, \dots, p_n)$, value x

Ensure: Value of the polynomial at x

```
1: Set  $pval \leftarrow p_n$ 
2: for  $i \leftarrow n$  down to 1 do
3:    $pval \leftarrow x \cdot pval + p_{i-1}$ 
4: end for
5: return  $pval$ 
```

Synthetic Division (Algorithm H.2) is a basic algorithm for dividing a polynomial $p(x)$ by a linear polynomial $x - a$ known from algebra. The product is a decomposition

$$p(x) = q(x)(x - a) + c$$

where $q(x) = \sum_{i=0}^{n-1} q_i x^i$ is the quotient and $c = p(a)$. Thus, Synthetic Division also evaluates the polynomial.

Algorithm H.2 Synthetic Division

Require: Polynomial coefficients $p = (p_0, p_1, \dots, p_n)$ value a

Ensure: Quotient coefficients $q = (q_0, q_1, \dots, q_{n-1})$ and remainder c

```
1: Set  $q_n = p_n$ 
2: for  $i \leftarrow n - 1$  down to 0 do
3:    $q_i \leftarrow p_i + a \cdot q_{i+1}$ 
4: end for
5: Set  $c \leftarrow q_0$ 
6: Pop  $q_0$  from  $q$   $\{q \leftarrow q_{1:n}\}$ 
7: return  $q$  as quotient, and  $c$  as remainder
```

Appendix I

The Modified Power Method on the Grassmannian Manifold

Introduction

The *Modified Power Method*, introduced earlier in Section 4.15, involves applying a matrix A to a pair of vectors, followed by orthogonalizing these vectors using the Gram-Schmidt process. This method aims to find an invariant subspace corresponding to the dominant eigenvectors of A . The algorithm can be interpreted as inducing a mapping on the Grassmannian manifold, searching for a fixed point.

I.1 The Algorithm

Let A be a matrix in $\mathbb{R}^{n \times n}$. The goal is to find an invariant 2-dimensional subspace under A . The modified Power Method involves the following steps:

Algorithm I.1 Modified Power Method

Require: Matrix $A \in \mathbb{R}^{n \times n}$, pair of initial vectors $\mathbf{v}_1^{(0)}, \mathbf{v}_2^{(0)}$ which are linearly independent.

Ensure: An orthonormal pair of vectors approximating the dominant eigenvectors of A .

- 1: $k \leftarrow 0$
 - 2: **repeat**
 - 3: $\mathbf{w}_1^{(k)} \leftarrow A\mathbf{v}_1^{(k)}$
 - 4: $\mathbf{w}_2^{(k)} \leftarrow A\mathbf{v}_2^{(k)}$
 - 5: **Orthogonalization:** Apply Gram-Schmidt process to $(\mathbf{w}_1^{(k)}, \mathbf{w}_2^{(k)})$, producing orthonormal vectors $(\mathbf{v}_1^{(k+1)}, \mathbf{v}_2^{(k+1)})$.
 - 6: **Normalization:** Normalize $\mathbf{v}_1^{(k+1)}$ and $\mathbf{v}_2^{(k+1)}$ to unit length.
 - 7: $k \leftarrow k + 1$
 - 8: **until** Convergence criteria met.
-

I.2 Geometric Interpretation

The Grassmannian manifold $\mathcal{G}(2, n)$ represents the collection of all 2-dimensional subspaces of $\mathbb{R}^n$. In the context of the modified Power Method:

- Each pair $(\mathbf{v}_1^{(k)}, \mathbf{v}_2^{(k)})$ defines a 2-dimensional subspace in $\mathcal{G}(2, n)$.

- The matrix A induces a transformation on the subspace at each iteration.
- The Gram-Schmidt process ensures orthogonality between the vectors, which leads to a stable iteration.

Induced Mapping on the Grassmannian

For a given pair of vectors $(\mathbf{v}_1, \mathbf{v}_2)$, the induced map T_A can be defined as:

$$T_A(\text{span}\{\mathbf{v}_1, \mathbf{v}_2\}) = \text{span}\{A\mathbf{v}_1, A\mathbf{v}_2\}.$$

The algorithm seeks a subspace such that T_A leaves it invariant, i.e.,

$$T_A(\text{span}\{\mathbf{v}_1, \mathbf{v}_2\}) = \text{span}\{\mathbf{v}_1, \mathbf{v}_2\}. \quad (\text{I.1})$$

Thus, fixed points of T_A , i.e., subspaces $V \subseteq \mathbb{R}^n$ such that $T_A(V) = V$, are exactly the *invariant subspaces* of A . An invariant subspace is usually defined by the set equation $A(V) = V$ to avoid a reference to the Grassmannian.

An algebraic form of the fixed point equation

Equation (I.1) translates to the matrix equation

$$AV = VB. \quad (\text{I.2})$$

Here $V = [\mathbf{v}_1 | \mathbf{v}_2] \in \mathbb{R}^{n \times 2}$ and $B \in \mathbb{R}^{2 \times 2}$ is the unique matrix of coefficients. Indeed, this matrix equation (I.2) is equivalent to

$$\begin{cases} A\mathbf{v}_1 = b_{11}\mathbf{v}_1 + b_{12}\mathbf{v}_2, \\ A\mathbf{v}_2 = b_{21}\mathbf{v}_1 + b_{22}\mathbf{v}_2, \end{cases}$$

and it merely expresses the fact that $A\mathbf{v}_j \in \text{span}\{\mathbf{v}_1, \mathbf{v}_2\}$. In addition, we can assume that V has orthogonal columns of unit length. This leads to

$$V^\top AV = V^\top VB = I_2 B = B.$$

which allows us to compute B easily.

By finding the eigenvalues and eigenvectors of B , an easy task since $B \in \mathbb{R}^2$, we can find two eigenvectors of A . For instance, let $B\mathbf{w} = \lambda\mathbf{w}$. Multiplying (I.2) by $\mathbf{w}$ on the right, we obtain:

$$AV\mathbf{w} = VB\mathbf{w} = V\lambda\mathbf{w} = \lambda V\mathbf{w}.$$

Hence,

$$\mathbf{v} \stackrel{\text{def}}{=} V\mathbf{w}$$

is an eigenvector of A with eigenvalue λ .

The Modified Power Method with few changes can find up to 4 (but not more!) top eigenvalues and eigenvectors by solving a polynomial equation in radicals, just as a quadratic equation is solved by the quadratic formula. If B is known already, it is likely the fastest way to find eigenvectors and eigenvalues.

I.3 A Proof of Convergence

The Modified Power Method relies upon convergence of $T_A^k(W)$ to a fixed point, given $W \in \mathcal{G}(2, n)$. However, to give this statement rigorous meaning, one must define convergence in the Grassmannian. One way to do this is to define a metric on the Grassmannian. Then the Grassmannian becomes a metric space, with the usual notion of convergence. Furthermore, the Grassmannian is a *compact* metric space, guaranteeing that every sequence has a convergent subsequence.

The details of the proof touch upon several subjects: differential topology, advanced linear algebra and general topology with emphasis on the theory of metric spaces. As a systematic exposition of these topics is beyond the scope of this book, we introduce the necessary facts in the course of the proof to the minimal extent, necessary to following the arguments. Full understanding may require consulting other resources. AI assistance is recommended.

Theorem I.1 (Convergence of the Modified Power Method). Let $A \in \mathbb{R}^{n \times n}$ have eigenvalues satisfying the inequalities

$$|\lambda_1| \geq |\lambda_2| \geq |\lambda_r| > |\lambda_{r+1}| \geq \dots \geq |\lambda_n| = 0$$

with eigenvectors $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n$. (The gap in the spectrum after the r th eigenvalue is critical!) Let $W \in \mathcal{G}(r, n)$. Then the sequence $T_A^n(W)$ converges to $V = \text{span}(\{\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_r\})$ iff

$$W \cap \text{span}(\{\mathbf{v}_{r+1}, \mathbf{v}_{r+2}, \dots, \mathbf{v}_n\}) = \{0\}. \quad (\text{I.3})$$

Furthermore, the rate of convergence is exponential. More precisely, let

$$\rho \stackrel{\text{def}}{=} \frac{|\lambda_{r+1}|}{|\lambda_r|} < 1$$

Then the distance of $T_A(W)$ from V is $\sim \rho^k$. $\square$

Proof. Let $W \in \mathcal{G}(r, n)$ be a subspace satisfying (I.3). Then $W = \text{span}\{\mathbf{w}_1, \dots, \mathbf{w}_r\}$ where $\mathbf{w}_j, j = 1, 2, \dots, r$ is some basis of W . We claim that there exist unique vectors $\mathbf{u}_j \in W, j = 1, 2, \dots, r$ such that

$$\mathbf{u}_j = \mathbf{v}_j + \sum_{i=r+1}^n c_{ij} \mathbf{v}_i, \quad j = 1, 2, \dots, r. \quad (\text{I.4})$$

To find these vectors, we first write for $j = 1, 2, \dots, r$:

$$\mathbf{w}_j = \sum_{i=1}^r d_{ij} \mathbf{v}_i + \sum_{i=r+1}^n e_{ij} \mathbf{v}_i.$$

We claim that the matrix $D \stackrel{\text{def}}{=} [d_{ij}]$ is invertible. Else, this matrix is singular and there exists a non-zero vector $\mathbf{z} = (z_1, z_2, \dots, z_r) \in \mathbb{R}^r$ such that $D\mathbf{z} = 0$. But then

$$\begin{aligned} \sum_{j=1}^r z_j \mathbf{w}_j &= \sum_{j=1}^r \sum_{i=1}^r d_{ij} z_j \mathbf{v}_i + \sum_{j=1}^r \sum_{i=r+1}^n e_{ij} z_j \mathbf{v}_i \\ &= \sum_{i=1}^r \left(\sum_{j=1}^r d_{ij} z_j \right) \mathbf{v}_i + \sum_{i=r+1}^n \left(\sum_{j=1}^r e_{ij} z_j \right) \mathbf{v}_i \\ &= \sum_{i=1}^r 0 \mathbf{v}_i + \sum_{i=r+1}^n \left(\sum_{j=1}^r e_{ij} z_j \right) \mathbf{v}_i \\ &= \sum_{i=r+1}^n \left(\sum_{j=1}^r e_{ij} z_j \right) \mathbf{v}_i. \end{aligned}$$

As the left-hand side is in W and the right-hand side is in $\text{span}\{\mathbf{v}_{r+1}, \dots, \mathbf{v}_n\}$, by (I.3)

$$\sum_{j=1}^r z_j \mathbf{w}_j = 0.$$

Due to linear independence of $\mathbf{w}_j, \mathbf{z} = 0$, which contradicts its definition.

The relationship (I.4) establishes 1:1 correspondence between subspaces $W \in \mathcal{G}(r, n)$ satisfying condition (I.3) and matrices $C = [c_{ij}], C \in \mathbb{R}^{r \times (n-r)}$.

The above construction introduces a kind-of coordinate system on a subset of the Grassmannian, giving it a structure of a *differentiable manifold*. Every coefficient c_{ij} can be regarded as a partial function $c_{ij} : \mathcal{G}(r, n) \rightarrow \mathbb{R}$. The mapping

$$\mathcal{G}(r, n) \ni W \mapsto C \in \mathbb{R}^{r \times (n-r)}$$

is called a *chart*.

A measure of the distance between V and W is the norm of the matrix C . If $V = W$, it is easy to see that $C = 0$ and thus $\|C\| = 0$. Furthermore, if $W' \in \mathcal{G}(r, n)$ is another subspace satisfying (I.3) with the corresponding matrix $C' \in R^{r \times (n-r)}$ then the distance between W and W' can be defined as

$$d_{\mathcal{V}, r}(W, W') = \|C - C'\|$$

where $\mathcal{V} = (\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n)$ is an *ordered* basis of $\mathbb{R}^n$. We emphasize the dependence on $\mathcal{V}$ and r . This is a metric, but only on a subset of $\mathcal{G}(r, n)$. Now we are ready to address convergence of the sequence $T_A^k(W)$. We have

$$A^k(\mathbf{u}_j) = \lambda_j^k \mathbf{v}_j + \sum_{i=r+1}^n c_{ij} \lambda_i^k \mathbf{v}_i.$$

Therefore,

$$\lambda_j^{-k} A^k(\mathbf{u}_j) = \mathbf{v}_j + \sum_{i=r+1}^n c_{ij} \left(\frac{\lambda_i}{\lambda_j} \right)^k \mathbf{v}_i.$$

As $T_A^k(W) = \text{span}\{\lambda_1^{-k} A^k(\mathbf{u}_1), \dots, \lambda_r^{-k} A^k(\mathbf{u}_r)\}$, and the right-hand side has the form introduced in (I.4), The matrix $C^{(k)}$ representing $T_A^k(W)$ is given by:

$$c_{ij}^{(k)} = c_{ij} \left(\frac{\lambda_i}{\lambda_j} \right)^k.$$

It follows from our assumptions that for $j = 1, 2, \dots, r$ and $i = r+1, r+2, \dots, n$:

$$\left| \frac{\lambda_i}{\lambda_j} \right| \leq \rho.$$

Therefore,

$$\|C^{(k)}\| \leq \|C\| \times \rho^k.$$

Note that we use the fact that shrinking matrix entries by a factor of ρ^k will result in shrinking the norm of the matrix by this factor. All “reasonable” matrix norms satisfy this condition, including ℓ^p -norms. (some exotic norms will not). Clearly, $\lim_{k \rightarrow \infty} d_W(T_A^k(W), V) = 0$, i.e.,

$$\lim_{k \rightarrow \infty} T_A^k(W) = V.$$

Furthermore, the convergence is exponentially fast. □

It should be noted that convergence holds for most choices of the initial subspace $W \in \mathcal{G}(r, n)$. For instance, by picking vectors $\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_r$ at random (using a random number generator), we will satisfy the convergence condition with probability 1.

I.4 Flag manifolds and QR algorithm

Below is basic information on these manifolds, used in one of the exercises to prove convergence of the QR algorithm. Some notions refer to *Lie group theory*, *representation theory* and *differential geometry* notions, which the reader would need to look up in standard sources.

Definition 5 (Flag manifold). A *flag manifold* is a *homogeneous space* that parametrizes a set of nested subspaces (a flag) in a finite-dimensional vector space. Formally, given a vector space V of dimension n over a field $\mathbb{K}$, a flag is a sequence of subspaces:

$$\{0\} \subset V_1 \subset V_2 \subset \dots \subset V_k \subset V,$$

where $\dim(V_i) = d_i$ for some integers $0 < d_1 < d_2 < \dots < d_k < n$. The *flag manifold* is the space of all such flags and can be described as a quotient of the general linear group $GL(n, \mathbb{K})$ by a parabolic subgroup. $\square$

For example, when $\mathbb{K} = \mathbb{R}$ or $\mathbb{C}$, a classic case is the *complete flag manifold*, which consists of all possible nested sequences of subspaces:

$$\{0\} \subset V_1 \subset V_2 \subset \dots \subset V_{n-1} \subset V,$$

with $\dim(V_i) = i$. The complete flag manifold $\text{fl}(V)$ has a rich structure and plays a fundamental role in algebraic geometry, representation theory, and differential geometry.

I.5 Conclusion

The modified Power Method iteratively searches for a fixed point on $\mathcal{G}(2, n)$. At each step, it applies the matrix A to a pair of vectors, followed by orthogonalization using the Gram-Schmidt process. The resulting invariant subspace corresponds to the dominant eigenvectors of A , making this method useful for approximating multiple eigenvectors and eigenvalues simultaneously. When finding several top r eigenvalues, we consider Grassmannian $\mathcal{G}(r, n)$. Likewise, the flag manifold is used to formulate the problem of convergence of the QR algorithm as a fixed point problem (Exercise I.5).

I.6 Exercises

Exercise I.1. Prove that the dimension of $\mathcal{G}(r, n)$ as a differentiable manifold, or as a topological space, is $r(n - r)$. $\square$



Exercise I.2 (Convergence almost surely). Prove that for a given matrix A the set of $W \in \mathcal{G}(r, n)$ for which convergence fails has Riemann-Lebesgue measure zero. It should be noted that there is a measure theory on differentiable manifolds, generalizing measure theory on $\mathbb{R}^n$. For instance, due to compactness, $\mathcal{G}(r, n)$ has *finite Riemann-Lebesgue volume*. In the context of this theory, the Modified Power Method converges *almost surely*. $\square$



Exercise I.3 (Projection Metric on the Grassmannian). Let $U, V \in \mathcal{G}(r, n)$ be two r -dimensional subspaces of $\mathbb{R}^n$, represented by their respective orthogonal projection matrices P_U and P_V , where:

$$P_U = Q_U Q_U^\top \quad \text{and} \quad P_V = Q_V Q_V^\top,$$

with $Q_U, Q_V \in \mathbb{R}^{n \times r}$ being orthonormal basis matrices for U and V . The *projection metric* $d_p(U, V)$ is defined as:

$$d_p(U, V) = \|P_U - P_V\|_F,$$

where $\|\cdot\|_F$ is the Frobenius norm of a matrix. Prove that $d_p(U, V)$ satisfies the axioms of a metric. **Hint:** You may use the properties of projection matrices:

$$P_U^2 = P_U, \quad P_V^2 = P_V, \quad P_U^\top = P_U, \quad P_V^\top = P_V,$$

and the subadditivity of the Frobenius norm:

$$\|A + B\|_F \leq \|A\|_F + \|B\|_F.$$

$\square$

Exercise I.4 (Proof of Convergence of the QR algorithm). Use Theorem 4.2 and induction to prove convergence of the QR-algorithm (Theorem 4.3). $\square$



Exercise I.5 (Flag manifold and the QR algorithm). Use complete flag manifolds to prove convergence of the QR algorithm under the sufficient condition stated in Theorem 4.3. The proof should follow the scheme in the proof of Theorem 4.2. In particular, given a matrix $A \in \mathbb{R}^{n \times n}$:

1. Define the induced map $T_A : \text{fl}(\mathbb{R}^n) \rightarrow \text{fl}(\mathbb{R}^n)$.
2. Rephrase the convergence of the QR algorithm as a fixed point problem for the induced map.
3. Define a suitable metric.
4. Prove exponentially fast convergence.

□

Appendix J

Spectral Decomposition and Resolvent of a Matrix

The Resolvent of a Matrix

Given a square matrix $A \in \mathbb{C}^{n \times n}$, the resolvent operator of A is defined as

$$R(\lambda; A) = (\lambda I - A)^{-1},$$

where λ is a complex number, and I is the identity matrix of the same dimension as A . This operator exists and is well-defined for all $\lambda \in \mathbb{C}$ that does not belong to the spectrum $\sigma(A)$ of A , i.e., the set of eigenvalues of A .

The resolvent is a powerful tool because it encodes information about the spectral properties and stability of A . The *spectral radius*, the *condition number*, and the behavior of iterative methods involving A are all influenced by the properties of the resolvent.

Function of a Matrix: The Cauchy Integral Formula

If f is an analytic function near the spectrum of A , the function $f(A)$ can be defined using the *Cauchy integral formula*:

$$f(A) = \frac{1}{2\pi i} \oint_{\Gamma} f(\lambda) R(\lambda; A) d\lambda,$$

where Γ is a closed contour in the complex plane that encloses all eigenvalues of A (i.e., $\sigma(A) \subset \text{int}(\Gamma)$) and does not enclose any other singularities of f . This formula provides a way to define and compute $f(A)$ directly from f and the resolvent, without explicitly knowing the eigenvalues of A .

Alternatively, if $f(z)$ has a Taylor series expansion that converges near $\sigma(A)$, we can express $f(A)$ as a power series:

$$f(A) = \sum_{k=0}^{\infty} c_k A^k,$$

where $f(z) = \sum_{k=0}^{\infty} c_k z^k$ in a neighborhood containing $\sigma(A)$.

Spectral Decomposition for a Diagonalizable Matrix

If A is diagonalizable, it can be written as $A = V\Lambda V^{-1}$, where V is the matrix of eigenvectors of A , and $\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n)$ is the diagonal matrix of eigenvalues of A . For an analytic function f near the spectrum of A , we then have

$$f(A) = V f(\Lambda) V^{-1} = V \text{diag}(f(\lambda_1), f(\lambda_2), \dots, f(\lambda_n)) V^{-1}.$$

Alternatively, we can express $f(A)$ using *spectral projectors*:

$$f(A) = \sum_{i=1}^n f(\lambda_i) P_i,$$

where each projector P_i is given by

$$P_i = \prod_{\substack{j=1 \\ j \neq i}}^n \frac{A - \lambda_j I}{\lambda_i - \lambda_j}.$$

Formula Using Eigenvectors and Eigenvalues

For a diagonalizable matrix A with distinct eigenvalues, we can express the spectral projector P_i associated with an eigenvalue λ_i directly in terms of its eigenvectors. Let $\mathbf{v}_i$ be a right eigenvector of A corresponding to λ_i , and $\mathbf{w}_i$ be the left eigenvector associated with λ_i (such that $\mathbf{w}_i^H \mathbf{v}_j = \delta_{ij}$, where δ_{ij} is the Kronecker delta). Then:

$$P_i = \frac{\mathbf{v}_i \mathbf{w}_i^H}{\mathbf{w}_i^H \mathbf{v}_i}.$$

This formula holds under the assumption that A is diagonalizable, so each eigenvalue λ_i has a complete set of linearly independent right and left eigenvectors. The projector P_i isolates the contribution of the eigenvalue λ_i in the decomposition of A .

The Resolvent and Partial Fraction Expansion

For a matrix A with distinct eigenvalues, the resolvent $(\lambda I - A)^{-1}$ can be expanded as a partial fraction:

$$(\lambda I - A)^{-1} = \sum_{i=1}^n \frac{P_i}{\lambda - \lambda_i}.$$

Each term $\frac{P_i}{\lambda - \lambda_i}$ isolates the contribution of the eigenvalue λ_i to the resolvent.

Spectral Projectors for Multiple Eigenvalues

When A has an eigenvalue λ_i with multiplicity m , the spectral projector P_i can be expressed using several methods.

Integral Representation Using the Residue Theorem

The projector P_i for λ_i with multiplicity m can be represented as an integral:

$$P_i = \frac{1}{2\pi i} \oint_{\Gamma_i} (\lambda I - A)^{-1} d\lambda,$$

where Γ_i is a contour around λ_i that encloses no other eigenvalues of A .

Limit of Derivatives for Projectors with Multiple Eigenvalues

If λ_i has multiplicity m , the projector P_i can be computed using a limit involving derivatives of the resolvent:

$$P_i = \frac{1}{(m-1)!} \lim_{\lambda \rightarrow \lambda_i} \frac{d^{m-1}}{d\lambda^{m-1}} [(\lambda - \lambda_i)^m (\lambda I - A)^{-1}].$$

This formula generalizes the single-eigenvalue case by taking into account the *higher-order pole* at $\lambda = \lambda_i$ in the resolvent.

Partial Fraction Expansion for Multiple Eigenvalues

If A has repeated eigenvalues, the partial fraction expansion of $(\lambda I - A)^{-1}$ near λ_i includes terms for each Jordan chain of eigenvectors:

$$(\lambda I - A)^{-1} = \sum_{i=1}^k \sum_{j=0}^{m_i-1} \frac{P_{i,j}}{(\lambda - \lambda_i)^{j+1}}.$$

Here:

- $\lambda_1, \lambda_2, \dots, \lambda_k$ are the distinct eigenvalues of A .
- $P_{i,0}$ is the projector onto the eigenspace associated with λ_i ,
- $P_{i,1}, P_{i,2}, \dots, P_{i,m_i-1}$ are generalized projectors for the Jordan chains.

Example: Projectors for a 2x2 Jordan Block

Consider a Jordan block

$$J = \begin{pmatrix} \lambda_i & 1 \\ 0 & \lambda_i \end{pmatrix}.$$

The projectors are:

$$P_{i,0} = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}, \quad P_{i,1} = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}.$$

The resolvent near λ_i is:

$$(\lambda I - J)^{-1} = \frac{P_{i,0}}{\lambda - \lambda_i} + \frac{P_{i,1}}{(\lambda - \lambda_i)^2}.$$

Exercises

Exercise J.1 (Resolvent and Function of a Matrix). Consider the matrix

$$A = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}.$$

1. Compute the resolvent $R(\lambda; A) = (\lambda I - A)^{-1}$.
2. Using the power series expansion of the exponential function, calculate e^A .
3. Verify that e^A can also be computed as $e^A = \sum_{i=1}^2 e^{\lambda_i} P_i$, where $\lambda_1 = 2$ and $\lambda_2 = 3$, and P_1 and P_2 are the spectral projectors for A .

□

Exercise J.2 (Spectral Decomposition with Distinct Eigenvalues). Let

$$B = \begin{pmatrix} 4 & 1 \\ 0 & 2 \end{pmatrix}.$$

1. Find the eigenvalues and eigenvectors of B .
2. Construct the spectral projectors P_1 and P_2 associated with the eigenvalues $\lambda_1 = 4$ and $\lambda_2 = 2$, using the formula $P_i = \frac{v_i w_i^H}{w_i^H v_i}$ where v_i and w_i are the right and left eigenvectors for each eigenvalue.
3. Using the spectral decomposition, write B in terms of its projectors: $B = \sum_{i=1}^2 \lambda_i P_i$.

□

Exercise J.3 (Resolvent and Partial Fraction Expansion). Given the matrix

$$C = \begin{pmatrix} 5 & 0 \\ 0 & 7 \end{pmatrix},$$

compute the resolvent $(\lambda I - C)^{-1}$ and express it as a partial fraction expansion:

$$(\lambda I - C)^{-1} = \frac{P_1}{\lambda - 5} + \frac{P_2}{\lambda - 7},$$

where P_1 and P_2 are the spectral projectors associated with the eigenvalues $\lambda_1 = 5$ and $\lambda_2 = 7$. $\square$

Exercise J.4 (Spectral Projectors for a Matrix with a Jordan Block). Consider the matrix

$$D = \begin{pmatrix} 3 & 1 \\ 0 & 3 \end{pmatrix}.$$

1. Show that D has a single eigenvalue $\lambda = 3$ with algebraic multiplicity 2 and a single Jordan block.
2. Find the generalized eigenvectors of D .
3. Using the limit formula, compute the spectral projector P_0 associated with the eigenspace and the generalized projector P_1 associated with the Jordan chain, where:

$$P_0 = \lim_{\lambda \rightarrow 3} (\lambda - 3)(\lambda I - D)^{-1}, \quad P_1 = \lim_{\lambda \rightarrow 3} \frac{d}{d\lambda} [(\lambda - 3)^2(\lambda I - D)^{-1}].$$

4. Write the resolvent $(\lambda I - D)^{-1}$ as a partial fraction expansion using P_0 and P_1 .

$\square$

Exercise J.5 (Applying a Function to a Non-Diagonalizable Matrix). Given the matrix

$$E = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix},$$

define $f(z) = e^z$ and compute $f(E) = e^E$ by using the Jordan decomposition:

1. Write E in Jordan form as $E = VJV^{-1}$, where J is the Jordan block for the eigenvalue $\lambda = 1$.
2. Compute e^J , the matrix exponential of the Jordan form.
3. Using $e^E = Ve^JV^{-1}$, find e^E .
4. Verify your result by using the power series expansion of e^E .

$\square$

Appendix K

Theory of Support Vector Machines

Introduction

Support Vector Machines (SVMs) are a cornerstone of modern machine learning, particularly effective for binary classification problems. Developed by **Vladimir Vapnik** and his colleagues, SVMs are built on the principles of structural risk minimization, aiming to find a decision boundary that generalizes well to unseen data [2, 28].

In this chapter, we will delve into the theoretical underpinnings of SVMs, exploring their geometric intuition, optimization framework, and extensions to handle nonlinear separability through kernel functions.

The Binary Classification Problem

Consider a binary classification task where we have a dataset:

$$\{(\mathbf{x}_i, y_i)\}_{i=1}^n, \quad \mathbf{x}_i \in \mathbb{R}^d, \quad y_i \in \{-1, +1\}.$$

The goal is to find a hyperplane that separates the two classes. A hyperplane in $\mathbb{R}^d$ is defined as:

$$\mathbf{w}^\top \mathbf{x} + b = 0,$$

where $\mathbf{w} \in \mathbb{R}^d$ is the weight vector and $b \in \mathbb{R}$ is the bias term.

The Maximum Margin Principle

SVMs seek the hyperplane that maximizes the *margin*, the distance between the hyperplane and the nearest data points from either class (called *support vectors*). The margin for a hyperplane is:

$$\text{Margin} = \frac{2}{\|\mathbf{w}\|},$$

where $\|\mathbf{w}\|$ is the Euclidean norm of $\mathbf{w}$. To ensure correct classification of data points, the following constraints must hold:

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad \forall i.$$

The optimization problem can then be formulated as:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2, \quad \text{subject to } y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad \forall i.$$

This formulation is a **convex quadratic programming** problem, ensuring a unique global solution.

The Dual Formulation

To solve the optimization problem, the *Lagrangian* is introduced:

$$L(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i [\mathbf{y}_i(\mathbf{w}^\top \mathbf{x}_i + b) - 1],$$

where $\alpha_i \geq 0$ are Lagrange multipliers. The *dual problem* is derived by taking the gradient of L with respect to $\mathbf{w}$ and b and setting them to zero:

$$\mathbf{w} = \sum_{i=1}^n \alpha_i \mathbf{y}_i \mathbf{x}_i, \quad \sum_{i=1}^n \alpha_i \mathbf{y}_i = 0.$$

Substituting back, the dual problem becomes:

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \mathbf{y}_i \mathbf{y}_j \mathbf{x}_i^\top \mathbf{x}_j,$$

subject to:

$$\alpha_i \geq 0, \quad \sum_{i=1}^n \alpha_i \mathbf{y}_i = 0.$$

The Kernel Trick

Real-world data is often not linearly separable. To handle this, SVMs use the *kernel trick*, which maps the input data into a higher-dimensional feature space where linear separation is possible. A kernel function $K(\mathbf{x}_i, \mathbf{x}_j)$ computes the dot product in the transformed space without explicitly performing the transformation:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j),$$

where ϕ is the mapping function. Common kernels include:

1. **Linear Kernel:** $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j$.
2. **Polynomial Kernel:** $K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^\top \mathbf{x}_j + c)^d$.
3. **Radial Basis Function (RBF) Kernel:** $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$.
4. **Sigmoid Kernel:** $K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\mathbf{x}_i^\top \mathbf{x}_j + c)$.

With kernels, the dual problem becomes:

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \mathbf{y}_i \mathbf{y}_j K(\mathbf{x}_i, \mathbf{x}_j),$$

subject to:

$$\alpha_i \geq 0, \quad \sum_{i=1}^n \alpha_i \mathbf{y}_i = 0.$$

Binary Classification with `fitcsvm`

The `fitcsvm` function in MATLAB is used to train Support Vector Machines (SVMs) for binary classification tasks. It provides a robust implementation of the SVM algorithm, offering various customization options for kernel functions, hyperparameters, and optimization techniques. This section outlines the algorithm, its usage, and the key options available in `fitcsvm`.

Algorithm Overview

The SVM algorithm implemented in `fitsvm` aims to find the hyperplane that separates the two classes with the maximum margin while minimizing misclassification errors. The optimization problem solved by `fitsvm` is:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i,$$

subject to:

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad \forall i,$$

where:

- $\mathbf{w}$ is the weight vector defining the hyperplane.
- b is the bias term.
- ξ_i are the slack variables allowing for misclassification.
- C is a regularization parameter controlling the trade-off between maximizing the margin and minimizing classification errors.

The function supports both linear and nonlinear classification by using kernel functions.

Key Options in `fitsvm`

`fitsvm` accepts various options to customize the training process. The most commonly used options include:

- **'KernelFunction'**: Specifies the kernel function used for transforming the data. Supported options include:
 - `'linear'`: $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j$.
 - `'polynomial'`: $K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^\top \mathbf{x}_j + c)^d$.
 - `'rbf'`: $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$.
 - `'gaussian'`: Equivalent to `'rbf'`.
 - `'sigmoid'`: $K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\mathbf{x}_i^\top \mathbf{x}_j + c)$.
 - Custom kernel functions can also be provided.
- **'BoxConstraint'**: Sets the regularization parameter C , which controls the trade-off between margin maximization and misclassification. Larger values of C penalize misclassification more heavily.
- **'KernelScale'**: Determines the scaling factor for the kernel function. Options include:
 - `'auto'`: Automatically estimates the kernel scale using a heuristic.
 - A positive scalar: Manually sets the kernel scale.
- **'Standardize'**: A Boolean flag that specifies whether to standardize the features (mean 0, variance 1) before training. Defaults to `false`.
- **'Solver'**: Specifies the optimization algorithm for solving the SVM problem. Supported options include:
 - `'SMO'`: Sequential Minimal Optimization (default).
 - `'ISDA'`: Iterative Single Data Algorithm.
- **'PolynomialOrder'**: Sets the degree d for the polynomial kernel function. Required when using `'polynomial'` as the `'KernelFunction'`.
- **'Nu'**: Specifies the parameter ν for Nu-SVM, a variation of SVM.
- **'IterationLimit'**: Sets the maximum number of iterations for the optimization solver.
- **'Verbose'**: Controls the display of optimization progress. Set to 1 for progress updates, and 0 for no display.
- **'Cost'**: A cost matrix specifying the penalty for misclassifying each class. Useful for handling imbalanced datasets.

Example of fitcsvm Usage

Here is an example of using `fitcsvm` to train a binary classifier with an RBF kernel:

```
1 % Generate synthetic data
2 rng(1); % For reproducibility
3 X = [randn(50,2)*0.75+ones(50,2); randn(50,2)*0.5-ones(50,2)];
4 Y = [ones(50,1); -ones(50,1)];
5
6 % Train an SVM model with RBF kernel
7 SVMModel = fitcsvm(X, Y, ...
8     'KernelFunction', 'rbf', ...
9     'BoxConstraint', 1, ...
10    'KernelScale', 'auto', ...
11    'Standardize', true);
12
13 % Visualize decision boundary
14 sv = SVMModel.SupportVectors;
15 figure;
16 gscatter(X(:,1), X(:,2), Y);
17 hold on;
18 plot(sv(:,1), sv(:,2), 'ko', 'MarkerSize', 10);
19 title('Support Vector Machine Decision Boundary');
20 hold off;
```

Listing K.1: Binary classification of synthetic data

Advantages of fitcsvm

- **Customizability:** Supports various kernel functions and optimization options.
- **Efficiency:** Uses advanced solvers like SMO for scalable training.
- **Integration:** Integrates seamlessly with MATLAB's visualization and evaluation tools.

The `fitcsvm` function provides a robust and flexible framework for training SVM classifiers, making it ideal for diverse binary classification problems.

Multiclass Classification with fitcecoc

Multiclass classification can be efficiently handled in MATLAB using the `fitcecoc` function, which implements the Error-Correcting Output Codes (ECOC) framework. ECOC extends binary classifiers, such as Support Vector Machines (SVMs) or decision trees, to multiclass problems by combining multiple binary classification tasks. This section explains the algorithm behind `fitcecoc` and its implementation.

Error-Correcting Output Codes

ECOC represents a multiclass problem with K classes using a binary coding matrix $M \in \{-1, 0, +1\}^{K \times L}$, where:

- K : Number of classes.
- L : Number of binary learners.

Each column of M defines a binary classification task:

- $+1$: A class is assigned to the positive class.
- -1 : A class is assigned to the negative class.
- 0 : A class is ignored for that task.

The binary classifiers are trained based on these assignments, and the combination of their outputs determines the final class.

Steps of the fitcecoc Algorithm

The main steps of the `fitcecoc` algorithm are:

Input Data

The input dataset is defined as:

$$\{(\mathbf{x}_i, y_i)\}_{i=1}^n, \quad \mathbf{x}_i \in \mathbb{R}^d, y_i \in \{1, 2, \dots, K\},$$

where $\mathbf{x}_i$ represents the feature vector, and y_i is the class label.

Coding Matrix Construction

`fitcecoc` creates a coding matrix M using one of the following strategies:

- **One-vs-One (default):** Each binary classifier distinguishes between two classes. M has $\binom{K}{2}$ columns.
- **One-vs-All:** Each binary classifier distinguishes one class from all others. M has K columns.
- **Custom:** Users can provide their own coding matrix.

Training Binary Learners

For each column of M :

- A binary classification problem is created by assigning +1 to the positive class, -1 to the negative class, and ignoring samples labeled 0.
- A binary learner (e.g., an SVM or decision tree) is trained on this subset.

Classification and Decoding

During prediction, each binary learner produces an output, forming a prediction vector $\hat{\mathbf{y}} = [\hat{y}_1, \hat{y}_2, \dots, \hat{y}_L]^\top$. The final class is determined by decoding $\hat{\mathbf{y}}$ using the coding matrix M . Two common decoding strategies are:

- **Hamming Distance Decoding:** Select the class with the smallest Hamming distance to $\hat{\mathbf{y}}$.
- **Loss-based Decoding:** Use a weighted loss function based on the confidence scores of the binary learners.

MATLAB Example

An example of training an ECOC classifier using SVM learners is shown below:

```
1 % Load dataset
2 load fisheriris
3 X = meas; % Features
4 Y = species; % Classes
5
6 % Train ECOC model using SVM binary learners
7 Mdl = fitcecoc(X, Y, 'Learners', 'svm');
8
9 % Predict on new data
10 [label, score] = predict(Mdl, X);
11
12 % Evaluate performance
13 confusionchart(Y, label);
```

Listing K.2: Predicting species in Fisher *iris* data

Advantages of ECOC

- **Robustness:** The redundancy in the coding matrix M makes ECOC robust to errors in individual binary classifiers.
- **Flexibility:** Any binary classifier can be used as the base learner.
- **Scalability:** ECOC handles large multiclass problems effectively.

The `fitcecoc` function provides a flexible and robust interface for multiclass classification, making it a valuable tool for machine learning applications in MATLAB.

Conclusion

Support Vector Machines elegantly combine geometric intuition with mathematical rigor, offering a versatile framework for classification and regression. Their kernelized formulation and capacity control through margin maximization make them a powerful tool in the machine learning arsenal. While their computational complexity may be a limitation for very large datasets, advances in algorithms and hardware continue to enhance their applicability.

Further reading

The kernel trick is discussed in [24].

Appendix L

Lagrange Multipliers

The Lagrange Multipliers Theorem provides a powerful method for finding extrema of functions subject to constraints, widely used across various fields in both theory and application.

The basic result concerning Lagrange multipliers is this:

Theorem L.1 (Lagrange Multipliers). Given a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ that we want to minimize and a constraint function $\mathbf{g} : \mathbb{R}^n \rightarrow \mathbb{R}^k$ such that $\mathbf{g}(\mathbf{x}) = \mathbf{0}$, assume:

1. Both f and $\mathbf{g}$ are twice continuously differentiable.
2. The gradients $\nabla g_1, \nabla g_2, \dots, \nabla g_k$, are linearly independent at the point $\mathbf{a}$.

Then, at any point $\mathbf{a}$ which is a local minimum of f subject to the constraints $\mathbf{g}(\mathbf{x}) = \mathbf{0}$, there exists a vector of Lagrange multipliers $\boldsymbol{\lambda} \in \mathbb{R}^k$ such that:

$$\nabla f(\mathbf{a}) = J_{\mathbf{g}}(\mathbf{a})^T \boldsymbol{\lambda} \quad (\text{L.1})$$

where $J_{\mathbf{g}}$ is the Jacobian matrix of $\mathbf{g}$ at $\mathbf{a}$. Vector $\boldsymbol{\lambda}$ is unique. $\square$

Proof. The map $\mathbf{g} = (g_1, g_2, \dots, g_k)$ satisfies the assumptions of the Implicit Function Theorem at $\mathbf{a}$: continuous differentiability and surjectivity of the derivative $D\mathbf{g}(\mathbf{a})$, equivalent to the linear independence of the gradients $\nabla g_i(\mathbf{a})$. Therefore, the set

$$M_r = \{\mathbf{x} \in B(\mathbf{a}, r) : g_1(\mathbf{x}) = g_2(\mathbf{x}) = \dots = g_k(\mathbf{x}) = 0\}$$

is a manifold for sufficiently small $r > 0$. It admits a parameterization

$$\mathbf{h} : \mathbb{R}^{n-k} \rightarrow \mathbb{R}^n$$

such that:

1. $\mathbf{g} \circ \mathbf{h} = \mathbf{0}$;
2. $\mathbf{h}(\mathbf{0}) = \mathbf{a}$;
3. $\mathcal{R}(D\mathbf{h}(\mathbf{0})) = \mathcal{N}(D\mathbf{g}(\mathbf{a}))$, the tangent space to M_r at $\mathbf{a}$.

The point $\mathbf{a}$ is a minimum of $f \mid M_r$ iff $\mathbf{0}$ is a local minimum of $f \circ \mathbf{h}$. The Chain Rule implies

$$d(f \circ \mathbf{h}) = (df) \circ \mathbf{h} \cdot D\mathbf{h}.$$

In particular $d(f \circ \mathbf{h})(\mathbf{0}) = df(\mathbf{a})D\mathbf{h}(\mathbf{0})$. It follows that

$$df(\mathbf{a}) \mid \mathcal{R}(D\mathbf{h}(\mathbf{0})) = df(\mathbf{a}) \mid \mathcal{N}(D\mathbf{g}(\mathbf{a})) = 0.$$

We can derive the fundamental equation (L.1) elegantly by using the following well-known criterion from linear algebra, of vanishing of a functional on a subspace:

Lemma L.1 (On vanishing of a functional). Let $\phi, \phi_i : \mathbb{R}^n \rightarrow \mathbb{R}, i = 1, \dots, k$ be linear functionals (equivalently, $\phi_i \in (\mathbb{R}^n)^*, \forall i$ where $(\mathbb{R}^n)^*$ is the dual space of $\mathbb{R}^n$). Then the following conditions are equivalent:

1. $\phi \in \text{span}\{\phi_1, \phi_2, \dots, \phi_k\}$;
2. There exist scalars $\lambda_i \in \mathbb{R}$ such that

$$\phi = \sum_{i=1}^k \lambda_i \phi_i;$$

3. For all $\mathbf{x} \in \mathbb{R}^n$:

$$\phi_1(\mathbf{x}) = \phi_2(\mathbf{x}) = \dots = \phi_k(\mathbf{x}) = 0 \implies \phi(\mathbf{x}) = 0.$$

□

When we apply the above lemma to $\phi = df(\mathbf{a})$ and $\phi_i = dg_i(\mathbf{a})$, we obtain the existence of constants λ_i such that

$$df(\mathbf{a}) = \sum_{i=1}^k \lambda_i \cdot dg_i(\mathbf{a}). \quad (\text{L.2})$$

After taking the transpose of both sides, this is exactly condition (L.1). □

Remark L.1 (No use of inner product). Condition L.2 can be expressed without using the notion of orthogonality, i.e., without an inner product. This is especially important when dealing with generalizations to infinite-dimensional spaces, such as function spaces, when inner product may not be available. □

Conclusion

- The theorem is proved by showing that any allowable variation at a local minimum must not decrease f , leading to the necessary condition involving Lagrange multipliers.
- This condition is satisfied by the existence of the Lagrange multipliers λ due to the linear independence of the gradients of the constraints.

The Lagrange function formalism

One usually introduces the *Lagrange function*

$$L(\lambda, \mathbf{x}) = f(\mathbf{x}) + \lambda^\top \mathbf{g}(\mathbf{x}) = f(x_1, x_2, \dots, x_n) + \sum_{i=1}^k \lambda_i g_i(x_1, x_2, \dots, x_n),$$

and states Theorem L.1 as

$$\nabla L(\lambda, \mathbf{a}) = 0.$$

This leads to a system of equations:

$$\begin{aligned} \nabla_{\mathbf{x}} L(\lambda, \mathbf{a}) &= \nabla f(\mathbf{a}) + \sum_{i=1}^k \lambda_i \nabla g_i(\mathbf{a}) = 0, \\ \nabla_{\lambda} L(\lambda, \mathbf{a}) = 0 &\iff \begin{cases} \frac{\partial L}{\partial \lambda_1}(\lambda, \mathbf{a}) = g_1(\mathbf{a}) = 0, \\ \frac{\partial L}{\partial \lambda_1}(\lambda, \mathbf{a}) = g_2(\mathbf{a}) = 0, \\ \dots \\ \frac{\partial L}{\partial \lambda_k}(\lambda, \mathbf{a}) = g_k(\mathbf{a}) = 0, \end{cases} \end{aligned}$$

This formulation resembles the definition of a critical point in unconstrained optimization. One can treat the Lagrange function in one of two ways:

1. As a mnemonic device to easily remember Theorem L.1.
2. As a meaningful quantity that needs to be minimized.

As it is often hard to give meaning to L , one often settles for the former.

It should be noted that, with the customary conventions, $\nabla \mathbf{g}^\top = J_{\mathbf{g}}$. In other words, the operator ∇ (*nabla*), when applied to a vector of functions $\mathbf{g} = (g_1, g_2, \dots, g_k)$, acts upon each coordinate, and the results (column vectors!) are combined into a matrix:

$$\nabla \mathbf{g} = [\nabla g_1 \mid \nabla g_2 \mid \dots \mid \nabla g_k]$$

This leads to the identity

$$\nabla_{\mathbf{x}} L = \nabla f + \nabla \mathbf{g} \cdot \boldsymbol{\lambda}.$$

We note that gradient depends on the choice of coordinates. If we want to differentiate in a coordinate-independent manner, we should use the *differential* and *Jacobi matrix*, which we can denote as either $J_{\mathbf{g}}$ or $D\mathbf{g} \in \mathbb{R}^{k \times n}$. More precisely,

$$D\mathbf{g} : \mathbb{R}^n \rightarrow \mathbb{R}^{k \times n}$$

is a matrix-valued function. Then, using $df = (\nabla f)^\top$:

$$D_{\mathbf{x}} L = df + \boldsymbol{\lambda}^\top \cdot D\mathbf{g}.$$

This last formula is a direct application of the rules of calculus (differentiation of a sum, differentiation of a composition). The period denotes matrix multiplication, or an application of a linear operator to a vector. This leads to a quick derivation of condition (L.1) by simply transposing:

$$\nabla_{\mathbf{x}} L = \nabla f + (D\mathbf{g})^\top \cdot \boldsymbol{\lambda} = \nabla f + \nabla \mathbf{g} \cdot \boldsymbol{\lambda}.$$

Also, we can see easily that $\mathbf{a}$ is a critical point iff $df(\mathbf{a})$ is in the rowspace of $D\mathbf{g}(\mathbf{a})$, i.e. $\mathcal{R}((D\mathbf{g}(\mathbf{a}))^\top)$, which is the same as the column space of $\nabla \mathbf{g}(\mathbf{a})$.

Second Derivative Test with Multiple Constraints

We will extend the well-known second derivative test taught in vector calculus, which allows one to determine whether a critical point is a local minimum, maximum or a saddle point.

General Considerations

A second derivative test, by its nature, is only capable of classifying non-degenerate critical points. For instance, when there are no constraints, the second order Taylor expansion at a critical point $\mathbf{a}$ is:

$$\begin{aligned} f(\mathbf{x}) &= f(\mathbf{a}) + df(\mathbf{a})(\mathbf{x} - \mathbf{a}) + \frac{1}{2} d^2 f(\mathbf{a})(\mathbf{x} - \mathbf{a}, \mathbf{x} - \mathbf{a}) + o(\|\mathbf{x} - \mathbf{a}\|^2) \\ &= f(\mathbf{a}) + (\mathbf{x} - \mathbf{a})^2 H(\mathbf{x} - \mathbf{a}) + o(\|\mathbf{x} - \mathbf{a}\|^2), \end{aligned}$$

where $H = \nabla^2 f(\mathbf{a})$ is the Hessian matrix at the critical point. Only when the quadratic form dominates the higher order terms the character of the critical point is determined by the Hessian. The domination can be formulated as $\mathcal{N}(H) = \{\mathbf{0}\}$, or the absence of the eigenvalue 0. The famous *Morse Lemma* states that there is a non-linear change of variables $\mathbf{x} = \mathbf{h}(\mathbf{y})$, where $\mathbf{h} : \mathbb{R}^n \rightarrow \mathbb{R}^n$ is such that $\mathbf{h}(\mathbf{0}) = \mathbf{a}$, $D\mathbf{h}(\mathbf{0}) = I_n$ and

$$f \circ \mathbf{h}(\mathbf{y}) = \frac{1}{2} \mathbf{y}^\top H \mathbf{y}$$

for $\mathbf{y}$ sufficiently close to $\mathbf{0}$.

Let us return to the situation considered in Theorem L.1. In the proof of that theorem we constructed a parameterization:

$$\mathbf{h} : \mathbb{R}^{n-k} \rightarrow \mathbb{R}^n$$

such that $\mathbf{h}(\mathbb{R}^n) = M_r$ and $\mathbf{h}(\mathbf{0}) = \mathbf{a}$, the constraint manifold. Non-degeneracy of the critical point for the critical point of $f|_{M_r}$ is defined *in coordinates*, as it is customary in working with manifolds, as non-degeneracy of $\mathbf{0}$ as the critical point of $\tilde{f} = f \circ \mathbf{h}$. Morse Lemma implies that there is a change of coordinates $\mathbf{k} : \mathbb{R}^{n-k} \rightarrow \mathbb{R}^{n-k}$ such that $\mathbf{x} = \mathbf{k}(\mathbf{y}) = \mathbf{y} + o(\|\mathbf{y}\|)$ and

$$\tilde{f} \circ \mathbf{k}(\mathbf{y}) = \frac{1}{2} \mathbf{y}^\top \tilde{H} \mathbf{y}$$

where $\tilde{H} = \nabla^2 \tilde{f}(\mathbf{0}) \in \mathbb{R}^{(n-k) \times (n-k)}$ is the Hessian of $\tilde{f}$. Replacing $\mathbf{h}$ with $\mathbf{h} \circ \mathbf{k}$, we obtain a parameterization of M_r such that $\mathbf{h}(\mathbf{0}) = \mathbf{a}$ and

$$f \circ \mathbf{h}(\mathbf{y}) = \frac{1}{2} \mathbf{y}^\top \tilde{H} \mathbf{y}.$$

Furthermore, as every quadratic form is equivalent to a sum of signed squares under a linear change of variables, we can find a parameterization such that near zero:

$$f \circ \mathbf{h}(\mathbf{y}) = - \sum_{i=1}^{\gamma} y_i^2 + \sum_{i=\gamma+1}^{n-k} y_i^2.$$

The integer $\gamma \in \{0, 1, \dots, n-k\}$ is called the *index (of inertia)* of the quadratic form $Q(\mathbf{y}) = \mathbf{y}^\top \tilde{H} \mathbf{y}$. It is obvious that γ is an algebraic invariant (the number of negative eigenvalues of the Hessian). It is less obvious, but true, that the index is also a *topological invariant*. To see this, we partition the coordinates into

$$\begin{aligned} \mathbf{y}' &= (y_1, y_2, \dots, y_\gamma) \\ \mathbf{y}'' &= (y_{\gamma+1}, y_{\gamma+2}, \dots, y_{n-k}). \end{aligned}$$

With this notation,

$$\tilde{f}(\mathbf{y}) = -\|\mathbf{y}'\|^2 + \|\mathbf{y}''\|^2.$$

Let $r > 0$ be sufficiently small and $\epsilon \in (0, r)$. Consider the set

$$D_{r,\epsilon} = \{\mathbf{y} \in \mathbb{R}^n : \|\mathbf{y}\| < r, \tilde{f}(\mathbf{y}) > -\epsilon^2\}.$$

The set is given by the inequality:

$$\|\mathbf{y}''\|^2 + \epsilon^2 < \|\mathbf{y}'\|^2 < r^2.$$

For any fixed $\mathbf{y}''$, such that $\|\mathbf{y}''\| < r$ the set of the corresponding $\mathbf{y}'$ is the region between two spheres centered at zero in $\mathbb{R}^\gamma$. As this region $D_{r,\epsilon}$ can be retracted to a sphere $S^{\gamma-1}$ and a ball in $\mathbb{R}^{n-k}$ can be retracted to a point, the set is retractible to $S^{\gamma-1}$, which captures γ in its topology (e.g., homology groups).

Test Based on Restriction to a Subspace

We should start with the note that the method presented in this section is often called *the projection method*, contrary to what is actually being done.

The method is contained in the following theorem.

Theorem L.2 (Second Derivative Test for Constrained Minimization). Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ and $\mathbf{g} : \mathbb{R}^n \rightarrow \mathbb{R}^k$ satisfy the assumptions of Theorem L.1 and let $(\lambda, \mathbf{a})$ be a stationary point of the Lagrangian

$$L(\lambda, \mathbf{x}) = f(\mathbf{x}) + \lambda^\top \mathbf{g}(\mathbf{x}).$$

Let $(\lambda, \mathbf{a})$ be a stationary point of L . A sufficient condition for $\mathbf{a}$, to be a local minimum of the constrained optimization problem of minimizing (maximizing) f subject to $\mathbf{g} = \mathbf{0}$ can be formulated as follows:

1. Let

$$H = H(\lambda, \mathbf{a}) \stackrel{\text{def}}{=} \nabla_{\mathbf{x}}^2 L(\lambda, \mathbf{a}) = \nabla^2 f(\mathbf{a}) + \lambda^\top \nabla^2 \mathbf{g}(\mathbf{a})$$

be the Hessian of L with respect to $\mathbf{x}$.

2. Let $Q : \mathbb{R}^n \rightarrow \mathbb{R}$ be the quadratic form associated with the Hessian H , i.e., $Q(\mathbf{x}) = \mathbf{x}^\top H \mathbf{x}$.
3. Let $V \subseteq \mathbb{R}^n$ be the tangent space to the constraint manifold at $\mathbf{a}$, also defined as

$$V = \mathcal{N}(D\mathbf{g}(\mathbf{a})) = \text{span}\{\nabla g_1(\mathbf{a}), \nabla g_2(\mathbf{a}), \dots, \nabla g_k(\mathbf{a})\}^\perp.$$

Then $\mathbf{a}$ is a non-degenerate local minimum (maximum) of f subject to constraint $\mathbf{g} = 0$ iff $Q|_V$ (Q restricted to V) is positive (negative) definite. $\square$

Proof. We use the notation of the proof of Theorem L.1. We focus on the case of the local minimum.

If $\mathbf{0}$ is a local minimum of $f \circ \mathbf{h}$ then clearly $\mathbf{a}$ is a local minimum of $f|_{M_r}$, and thus a solution to the constrained problem. The second derivative test known from vector calculus is a sufficient condition. It is stated as positivity of the quadratic form $d^2(f \circ \mathbf{h})(\mathbf{0})$. We have the Chain Rule for the second derivative:

$$d^2(f \circ \mathbf{h}) = ((d^2 f) \circ \mathbf{h}) D\mathbf{h} D\mathbf{h} + (df \circ \mathbf{h}) D^2 \mathbf{h}.$$

At $\mathbf{a}$ we have $df = \sum_i \lambda_i dg_i$. Therefore

$$d^2(f \circ \mathbf{h}) = (d^2 f) \circ \mathbf{h} D\mathbf{h} D\mathbf{h} - \sum_{i=1}^k \lambda_i (dg_i \circ \mathbf{h}) D^2 \mathbf{h}. \quad (\text{L.3})$$

On the other hand $g_i \circ \mathbf{h} = 0$. Differentiating twice we obtain:

$$0 = (d^2 g_i) \circ \mathbf{h} D\mathbf{h} D\mathbf{h} + (dg_i \circ \mathbf{h}) D^2 \mathbf{h}.$$

Thus

$$(dg_i \circ \mathbf{h}) D^2 \mathbf{h} = -(d^2 g_i) \circ \mathbf{h} D\mathbf{h} D\mathbf{h}.$$

Plugging this back into equation (L.3) we obtain at $\mathbf{a}$:

$$\begin{aligned} d^2(f \circ \mathbf{h}) &= ((d^2 f) \circ \mathbf{h}) D\mathbf{h} D\mathbf{h} + \sum_{i=1}^k \lambda_i ((d^2 g_i) \circ \mathbf{h}) D\mathbf{h} D\mathbf{h} \\ &= (d^2(f + \lambda^\top \mathbf{g})) \circ \mathbf{h} D\mathbf{h} D\mathbf{h} \\ &= ((d_{\mathbf{x}}^2 L) \circ \mathbf{h}) D\mathbf{h} D\mathbf{h}. \end{aligned}$$

More precisely, after substituting $\mathbf{a}$ for $\mathbf{x}$:

$$d^2(f \circ \mathbf{h})(\mathbf{0}) = (d_{\mathbf{x}}^2 L)(\lambda, \mathbf{a}) D\mathbf{h}(\mathbf{a}) D\mathbf{h}(\mathbf{a}).$$

This concludes the proof, as $\mathcal{R}(D\mathbf{h}(\mathbf{a})) = V$. $\square$

The bordered Hessian

Theorem L.2 gives a viable method for classifying critical points into local maxima, minima and saddle points. However, the method requires finding the tangent space to the constraint manifold. There is a method that does not require this step, and it resembles more the second derivative test for the unconstrained problem. It is based on the notion of a bordered Hessian.

Definition 6 (Bordered Hessian). Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be a twice continuously differentiable function, and let $g_1, g_2, \dots, g_k : \mathbb{R}^n \rightarrow \mathbb{R}$ be constraint functions with continuous second derivatives. The generalized bordered Hessian H at $(\lambda, \mathbf{x})$ is given by

$$H_B = H_B(\lambda, \mathbf{x}) \stackrel{\text{def}}{=} \left[\begin{array}{c|c} \mathbf{0}_{k \times k} & \begin{matrix} \nabla g_1(\mathbf{x})^\top \\ \vdots \\ \nabla g_k(\mathbf{x})^\top \end{matrix} \\ \hline \nabla g_1(\mathbf{x}) & \dots & \nabla g_k(\mathbf{x}) & \nabla_{\mathbf{x}}^2 L(\lambda, \mathbf{x}) \end{array} \right] = \left[\begin{array}{c|c} \mathbf{0}_{k \times k} & \nabla \mathbf{g}(\mathbf{x})^\top \\ \hline \nabla \mathbf{g}(\mathbf{x}) & \nabla_{\mathbf{x}}^2 L(\lambda, \mathbf{x}) \end{array} \right]$$

Thus, the bordered Hessian H is an $(n+k) \times (n+k)$, symmetric matrix. $\square$

Remark L.2 (On the order of variables). We write $L(\boldsymbol{\lambda}, \mathbf{x})$ and not $L(\mathbf{x}, \boldsymbol{\lambda})$ as many authors do. The reason is that with this convention the bordered Hessian is just the ordinary Hessian of L . We merely recognize the special block structure resulting from the form of L . $\square$

Criterion for Optimality with Multiple Constraints

Theorem L.3 (Second Derivative Test for Multiple Constraints). Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be a twice continuously differentiable function, and let

$$g_1, g_2, \dots, g_k : \mathbb{R}^n \rightarrow \mathbb{R}$$

be constraint functions with continuous second derivatives.

Suppose $\mathbf{x}^*$ is a stationary point for the Lagrangian of f :

$$L(\boldsymbol{\lambda}, \mathbf{x}) = f(\mathbf{x}) + \boldsymbol{\lambda}^\top \mathbf{g}(\mathbf{x}) = f(x_1, x_2, \dots, x_n) + \sum_{i=1}^k \lambda_i g_i(x_1, x_2, \dots, x_n),$$

subject to constraints

$$g_i(\mathbf{x}) = 0, \forall i,$$

and let

$$\boldsymbol{\lambda}^* = (\lambda_1^*, \lambda_2^*, \dots, \lambda_k^*)$$

be the corresponding Lagrange multipliers. Let H_B be the bordered Hessian:

$$H_B = H_B(\boldsymbol{\lambda}, \mathbf{x}) = \left[\begin{array}{c|c} \mathbf{0}_{k \times k} & \nabla \mathbf{g}(\mathbf{x})^\top \\ \hline \nabla \mathbf{g}(\mathbf{x}) & \nabla_{\mathbf{x}}^2 L(\boldsymbol{\lambda}, \mathbf{x}) \end{array} \right]$$

Let $H_B^* = H_B(\boldsymbol{\lambda}^*, \mathbf{x}^*)$ be the bordered Hessian evaluated at the critical point. Let $H_{B,i}^*$ be the leading principal minors (determinants!) of H_B^* . We consider the subsequence of $n - k$ minors with first $2k$ minors omitted:

$$H_{B,2k+1}, H_{B,2k+2}, \dots, H_{B,n+k}.$$

Then:

1. A sufficient condition for a local maximum is that the **last** $n - k$ minors alternate in sign with the smallest one having the sign $(-1)^{k+1}$.
2. A sufficient condition for a local minimum is that all of the last $n - k$ minors have the sign $(-1)^k$.

$\square$

Remark L.3 (The unconstrained case). When $k = 0$, i.e., when there are no constraints, these conditions coincide with the conditions for the unbordered Hessian to be negative definite or positive definite respectively). $\square$

Example L.1 (One constraint). Minimize $f(x_1, x_2)$ subject to $g_2(x_1, x_2) = 0$, where

$$f(x_1, x_2) = \frac{1}{2}(x_1^2 + x_2^2),$$

$$g_1(x_1, x_2) = x_1 + x_2 - 1.$$

Verify sufficiency using bordered Hessian. $\square$

Solution

We have

$$L(\lambda_1, x_1, x_2) = \frac{1}{2}(x_1^2 + x_2^2) + \lambda_1(x_1 + x_2 - 1),$$

$$\nabla L(\lambda_1, x_1, x_2) = (x_1 + x_2 - 1, x_1 + \lambda_1, x_2 + \lambda_1),$$

$$H_B = \nabla^2 L = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

The principal minors of the bordered Hessian are:

$$H_{B,1} = |0| = 0,$$

$$H_{B,2} = \begin{vmatrix} 0 & 1 \\ 1 & 1 \end{vmatrix} = -1,$$

$$H_{B,3} = \begin{vmatrix} 0 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \end{vmatrix} = -2.$$

Only the last $n - k = 2 - 1 = 1$ minors matter. In our case, $H_3 = -2$ is the only minor of consequence, and its sign is $-1 = (-1)^k$. Thus, the stationary point is a local minimum. $\square$

Example L.2 (Two constraints). Minimize

$$f(x_1, x_2, x_3) = x_1 + x_2 + 2x_3$$

on a circle given implicitly by the constraints:

$$g_1(x_1, x_2, x_3) = x_1^2 + x_2^2 + x_3^2 - 1,$$

$$g_2(x_1, x_2, x_3) = x_1 - x_2 = 0,$$

$\square$

Solution

*** Step 1: Form the Lagrange function *** It is a function of 5 variables:

$$L(\lambda_1, \lambda_2, x_1, x_2, x_3) = x_1 + x_2 + 2x_3 + \lambda_1(x_1^2 + x_2^2 + x_3^2 - 1) + \lambda_2(x_1 - x_2).$$

*** Step 2: Find the gradient components *** In vector form,

$$\nabla_{\mathbf{x}} L(\lambda_1, \lambda_2, x_1, x_2, x_3) = (1 + 2\lambda_1 x_1 + \lambda_2, 1 + 2\lambda_1 x_2 - \lambda_2, 2 + 2\lambda_1 x_3).$$

$$\nabla_{\lambda} L(\lambda_1, \lambda_2, x_1, x_2, x_3) = (x_1^2 + x_2^2 + x_3^2 - 1, x_1 - x_2).$$

*** Step 3: Find critical points *** We solve a system of nonlinear equations:

$$\begin{cases} 1 + 2\lambda_1 x_1 + \lambda_2 &= 0, \\ 1 + 2\lambda_1 x_2 - \lambda_2 &= 0, \\ 2 + 2\lambda_1 x_3 &= 0, \\ x_1^2 + x_2^2 + x_3^2 - 1 &= 0, \\ x_1 - x_2 &= 0. \end{cases}$$

We solve this system by elimination, which is not very enlightening but an arduous task even for this minimalistic example. It is appropriate to utilize symbolic computation. For example, a Python program¹ to do this is below:

¹This code was generated by ChatGPT.

```

1 import sympy as sp
2
3 # Define the variables
4 x1, x2, x3, lambda1, lambda2 = sp.symbols('x1 x2 x3 lambda1 lambda2')
5
6 # Define the equations
7 equations = [
8     1 + 2 * lambda1 * x1 + lambda2, # Equation 1
9     1 + 2 * lambda1 * x2 - lambda2, # Equation 2
10    2 + 2 * lambda1 * x3,           # Equation 3
11    x1**2 + x2**2 + x3**2 - 1,      # Equation 4
12    x1 - x2                         # Equation 5
13 ]
14
15 # Solve the system of equations
16 solutions = sp.solve(equations, [lambda1, lambda2, x1, x2, x3], dict=True)
17
18 # Display the solutions
19 for sol in solutions:
20     print(sol)
21

```

Listing L.1: Solving a non-linear system with Python and SymPy

The raw Python output is:

```

{lambda1: -sqrt(6)/2, lambda2: 0, x1: sqrt(6)/6, x2: sqrt(6)/6, x3: sqrt(6)/3}
{lambda1: sqrt(6)/2, lambda2: 0, x1: -sqrt(6)/6, x2: -sqrt(6)/6, x3: -sqrt(6)/3}

```

Thus the system of equations has the following two solutions, in standard notation:

$$\lambda_1 = \mp \frac{\sqrt{6}}{2}, \quad \lambda_2 = 0, \quad x_1 = \pm \frac{\sqrt{6}}{6}, \quad x_2 = \pm \frac{\sqrt{6}}{6}, \quad x_3 = \pm \frac{\sqrt{6}}{3},$$

where the choice of signs is either all top or all bottom.

On general grounds, one solution is a maximum and the other is a minimum. We may resolve the question which one is which by either evaluating the function f at both (not generalizable to higher dimensions!) or use the bordered Hessian (generalizable).

★★★ **Step 4:** Form the bordered Hessian ★★★ The bordered Hessian is, for general $\mathbf{x}$ and λ :

$$H_B = \left[\begin{array}{cc|ccc} 0 & 0 & 2x_1 & 2x_2 & 2x_3 \\ 0 & 0 & 1 & -1 & 0 \\ \hline 2x_1 & 1 & 2\lambda_1 & 0 & 0 \\ 2x_2 & -1 & 0 & 2\lambda_1 & 0 \\ 2x_3 & 0 & 0 & 0 & 2\lambda_1 \end{array} \right].$$

★★★ **Step 5:** Find the relevant principal minors ★★★ As $n - k = 3 - 2 = 1$, only one determinant, $\det H_B = H_{B,5}$ needs to be found. Using cofactor expansion repeatedly we find that:

$$\det(H_B) = 8\lambda_1 (x_1^2 + 2x_1x_2 + x_2^2 + 2x_3^2) = 8\lambda_1 ((x_1 + x_2)^2 + 2x_3^2).$$

Thus, the sign of $\det(H_B)$ is the same as the sign of λ_1 .

Conclusion

As $\lambda_1 = \mp \frac{\sqrt{6}}{2}$, the top choice of the sign results in a minimum and the bottom in a minimum. This is confirmed by evaluating f at both critical points. $\square$

Exercise L.1 (Full automation). Expand the Python and SymPy code presented in Listing L.1 to completely automate the process of computing and classifying critical points of the given optimization problem with constraints. In particular, the second derivative test should be implemented. A larger project would be to generalize to arbitrary n and k , and arbitrary symbolic f and $\mathbf{g}$. $\square$

A Proof of the Bordered Hessian Criterion

Theorem L.2 reduces the proof to the case when

1. f is a quadratic function;
2. The constraints g_i are linear functions.

Indeed, let H be the Hessian $\nabla^2 L(\lambda, \mathbf{a})$ and let $\mathbf{v}_i = \nabla g_i(\mathbf{a})$, $\forall i$. We need to show that $\mathbf{0}$ is a minimum, maximum or saddle point, when minimizing

$$Q(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top H \mathbf{x}$$

subject to

$$V^\top \mathbf{x} = 0,$$

where

$$V = [\mathbf{v}_1 \mid \mathbf{v}_2 \mid \dots \mid \mathbf{v}_k].$$

The bordered Hessian is

$$H_B = \left[\begin{array}{c|c} \mathbf{0}_{k \times k} & V^\top \\ \hline V & H \end{array} \right].$$

Our proof uses a variant of Gaussian Elimination in which both row and column operations are employed. After each row operation a column operation follows using the same multipliers. We note that a column operation corresponds to a linear change of variables. Row and column exchanges are allowed and are performed in tandem as well. When we apply the process to H_B operating on the rows and columns, starting with row or column k , and ending with row or column $2k$. The resulting matrix has the format:

$$\left[\begin{array}{c|c|c} \mathbf{0}_{k \times k} & D_k & \mathbf{0}_{k \times (n-k)} \\ \hline D_k & H_{11} & H_{12} \\ \hline \mathbf{0}_{(n-k) \times k} & H_{12}^\top & H_{22} \end{array} \right]$$

where $D_k = \text{diag}(d_1, d_2, \dots, d_k)$ is a diagonal matrix containing the k pivots of V . The matrix D_k is the *reduced row echelon form* of V . We note that $d_i \neq 0$ because $\mathbf{v}_i$ are linearly independent.

We claim that

$$\det \left(\left[\begin{array}{c|c} \mathbf{0}_{k \times k} & D_k \\ \hline D_k & H_{11} \end{array} \right] \right) = (-1)^k \prod_{i=1}^k d_i^2.$$

The proof follows from moving the last k columns before the first k columns, which results in a lower-triangular matrix with determinant $\prod_{i=1}^k d_i^2$. As this requires k row exchanges, each flipping the sign of the determinant, our claim follows. Thus, as row and column operations preserve the determinant,

$$\text{sgn } H_{B,2k} = (-1)^k.$$

Following the modified Gaussian Elimination to the end, but prohibiting rows exchanges, we obtain a matrix of the form:

$$\left[\begin{array}{c|c|c} \mathbf{0}_{k \times k} & D_k & \mathbf{0}_{k \times (n-k)} \\ \hline D_k & H_{11} & \mathbf{0}_{(n-k) \times (n-k)} \\ \hline \mathbf{0}_{(n-k) \times k} & \mathbf{0}_{(n-k) \times (n-k)} & D_{n-k} \end{array} \right]$$

where D_{n-k} is the diagonal matrix containing the remaining $n-k$ pivots. It is clear that after all changes of variables, the equations of the constraint manifold are simply $x_1 = x_2 = \dots = x_k = 0$. In the new coordinates, Q restricted to the constraint manifold is represented by D_{n-k} . In particular, the index (the number of negative eigenvalues) for D_{n-k} is equal to the number of negative numbers amongst

$$(-1)^k H_{B,2k+1}, (-1)^k H_{B,2k+2}, \dots, (-1)^k H_{B,k+n}.$$

If all numbers are positive, Q restricted to the constraint manifold is positive definite and we have a local minimum. If they alternate, starting with a minus, we have local maximum. This concludes the proof.

Exercise L.2 (Detecting degeneracy). The context for this exercise is the proof of this section. Show that if we encounter a zero pivot, or equivalently, zero minor, then the critical point is degenerate. Conclude that we need not worry about this case, as we are only concerned with classifying non-degenerate critical points. $\square$

Exercise L.3 (Gaussian Elimination for bordered Hessian). Design and implement the modified Gaussian Elimination algorithm that finds the principal minors of H_B . Calculate the approximate number of floating point operations (FLOPS) required. $\square$

Exercise L.4 (Index of bordered Hessian when $k = n$). Let $H \in \mathbb{R}^k$ and $V \in \mathbb{R}^{k \times k}$ be a non-singular matrix. Prove that the symmetric matrix

$$H_B = \left[\begin{array}{c|c} \mathbf{0}_{k \times k} & V^\top \\ \hline V & H \end{array} \right]$$

has index k . In other words, the number of positive and negative eigenvalues is the same and equals k . In particular, H_B is non-singular and $\text{sgn det } H_B = (-1)^k$. $\square$

References

See [1].



Appendix M

Envelope Theorem (Milgrom–Segal formulation)

A central object in comparative statics is the *value function*

$$V(t) = \sup_{x \in X} f(x, t),$$

where X is an arbitrary choice set and t is a real parameter. The classical envelope theorem is often proved under differentiability in x and regularity conditions that yield first-order conditions and differentiable policy functions. Milgrom and Segal [14] give a modern formulation that applies to *arbitrary* choice sets and does not require differentiability in x nor smooth dependence of optimizers on t .

M.1 Setup

Fix an interval $I = [a, b] \subset \mathbb{R}$. Let $f : X \times I \rightarrow \mathbb{R}$ be such that for every $x \in X$, the map $t \mapsto f(x, t)$ is absolutely continuous on I . Let $f_t(x, t)$ denote its (a.e.) derivative with respect to t . Define the argmax correspondence

$$X^*(t) = \arg \max_{x \in X} f(x, t) \quad (\text{assumed nonempty for each } t \in I).$$

The value function is

$$V(t) = \sup_{x \in X} f(x, t) = \max_{x \in X} f(x, t),$$

where the last equality holds because $X^*(t) \neq \emptyset$.

M.2 The Milgrom–Segal envelope theorem

Theorem M.1 (Envelope theorem for arbitrary choice sets [14, Thm. 2]). Assume there exists an integrable function $b \in L^1(I)$ such that

$$|f_t(x, t)| \leq b(t) \quad \text{for all } x \in X \text{ and for a.e. } t \in I.$$

Then the value function V is absolutely continuous on I , hence differentiable for a.e. $t \in I$. Moreover, the one-sided derivatives satisfy, for a.e. $t \in I$,

$$D^+V(t) = \sup_{x \in X^*(t)} f_t(x, t), \quad D^-V(t) = \inf_{x \in X^*(t)} f_t(x, t).$$

In particular, at every point t where V is differentiable,

$$V'(t) \in \left[\inf_{x \in X^*(t)} f_t(x, t), \sup_{x \in X^*(t)} f_t(x, t) \right].$$

If $f_t(x, t)$ is constant over $x \in X^*(t)$ (for example, if $X^*(t)$ is a singleton), then

$$V'(t) = f_t(x^*(t), t) \quad \text{for any } x^*(t) \in X^*(t)$$

for a.e. $t \in I$. $\square$

M.3 Interpretation

The theorem shows that first-order changes in $V(t) = \max_x f(x, t)$ depend only on the *direct* partial derivative f_t evaluated at optimizers. No differentiability (or even continuity) of an optimizer selection $t \mapsto x^*(t)$ is required. This is particularly useful in problems with kinks, constraints, or discrete choices, where optimizers can jump while the value function remains absolutely continuous.

M.4 Multidimensional parameter version

Let $T \subset \mathbb{R}^k$ be an open set and let $t = (t_1, \dots, t_k) \in T$ be a vector of parameters. Consider the value function

$$V(t) = \sup_{x \in X} f(x, t),$$

where X is an arbitrary choice set and $f : X \times T \rightarrow \mathbb{R}$.

M.4.1 Regularity assumptions

Assume:

- For each $x \in X$, the function $t \mapsto f(x, t)$ is absolutely continuous on every line segment in T .
- For each coordinate $j = 1, \dots, k$, the partial derivative

$$\frac{\partial f}{\partial t_j}(x, t)$$

exists for a.e. t and satisfies the uniform bound

$$\left| \frac{\partial f}{\partial t_j}(x, t) \right| \leq b_j(t), \quad b_j \in L^1_{\text{loc}}(T),$$

independently of $x \in X$.

- The argmax correspondence

$$X^*(t) = \arg \max_{x \in X} f(x, t)$$

is nonempty for all $t \in T$.

M.4.2 Directional envelope theorem

Theorem M.2 (Multidimensional envelope theorem [14]). For any direction $h \in \mathbb{R}^k$, the directional derivative of V exists for a.e. $t \in T$ and satisfies

$$DV(t; h) = \sup_{x \in X^*(t)} \nabla_t f(x, t) \cdot h,$$

where

$$\nabla_t f(x, t) = \left(\frac{\partial f}{\partial t_1}(x, t), \dots, \frac{\partial f}{\partial t_k}(x, t) \right).$$

$\square$

M.4.3 Gradient form

If the maximizer is unique for a.e. t , or if $\nabla_t f(x, t)$ is constant over $x \in X^*(t)$, then V is differentiable a.e. and

$$\nabla V(t) = \nabla_t f(x^*(t), t).$$

In general, when multiple maximizers exist, the gradient of V is set-valued:

$$\nabla V(t) \in \text{co}\{\nabla_t f(x, t) : x \in X^*(t)\},$$

where co denotes the convex hull. Thus V is almost everywhere *Clarke differentiable*, and the envelope theorem identifies its generalized gradient.

M.5 Notes and related references

For classical treatments of envelope results (typically under smoothness and interiority conditions), see standard microeconomic or optimization texts such as [10, 26].

Appendix N

Constrained Minimization with Inequality Constraints

A vector $x \in \mathbb{R}^n$ is called a *positive vector* iff $x_j > 0$ for $j = 1, 2, \dots, n$. We simply write $x > 0$ when x is positive. Similarly, x is called *non-negative* if $x_j \geq 0$ $j = 1, 2, \dots, n$, and write $x \geq 0$. Note that $x > 0$ is not equivalent to $x \geq 0 \wedge x \neq 0$, unless $n \leq 1$.

N.1 Problem formulation

We consider optimization problems of the form

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{subject to} \quad g_j(x) \leq 0, \quad j = 1, \dots, m, \quad (\text{N.1})$$

where $f : \mathbb{R}^n \rightarrow \mathbb{R}$ and $g_j : \mathbb{R}^n \rightarrow \mathbb{R}$ are continuously differentiable. The feasible set is

$$\mathcal{F} = \{x \in \mathbb{R}^n : g_j(x) \leq 0 \quad \forall j\}.$$

A recurring model problem throughout this chapter is the box-constrained quadratic program

$$\min_{x \in \mathbb{R}^n} \frac{1}{2} x^T Q x + c^T x, \quad 0 \leq x_i \leq 1, \quad (\text{N.2})$$

where Q is symmetric. This problem lies at the intersection of convex analysis, numerical linear algebra, and algorithm design.

N.2 Convex and nonconvex settings

N.2.1 Convex problems

If $Q \succeq 0$ in (N.2), or more generally if f is convex and each g_j is convex, then (N.1) is a convex optimization problem. In this case:

- Any local minimizer is a global minimizer.
- The Karush–Kuhn–Tucker conditions are both necessary and sufficient.
- Strong duality holds under Slater’s condition.

Convexity allows one to focus on algorithmic efficiency rather than global optimality concerns.

N.2.2 Nonconvex problems

If Q is indefinite or f is otherwise nonconvex, the problem may admit multiple local minimizers. In this setting:

- KKT conditions are necessary but not sufficient.
- Algorithms typically converge only to stationary points.
- Initialization and globalization strategies become critical.

Many methods described in this chapter apply to both settings, but their guarantees differ substantially.

N.3 Lagrangian and KKT conditions

Define the Lagrangian

$$\mathcal{L}(x, \lambda) = f(x) + \sum_{j=1}^m \lambda_j g_j(x), \quad \lambda_j \geq 0.$$

Under appropriate constraint qualifications, a point x^* is a local minimizer of (N.1) if and only if there exists $\lambda^* \geq 0$ such that:

$$\begin{aligned} g_j(x^*) &\leq 0, \\ \lambda_j^* &\geq 0, \\ \nabla f(x^*) + \sum_j \lambda_j^* \nabla g_j(x^*) &= 0, \\ \lambda_j^* g_j(x^*) &= 0. \end{aligned}$$

The last condition, complementary slackness, encodes the fact that only active constraints influence stationarity.

N.4 Box constraints and componentwise optimality

For the box constraints $0 \leq x_i \leq 1$, introduce multipliers $\alpha_i, \beta_i \geq 0$. Stationarity becomes

$$Qx^* + c - \alpha^* + \beta^* = 0,$$

with

$$\alpha_i^* x_i^* = 0, \quad \beta_i^* (x_i^* - 1) = 0.$$

This leads to the componentwise optimality conditions

$$x_i^* = \begin{cases} 0, & (Qx^* + c)_i > 0, \\ 1, & (Qx^* + c)_i < 0, \\ \in (0, 1), & (Qx^* + c)_i = 0. \end{cases}$$

These inequalities underpin both active-set and projected-gradient algorithms.

N.5 Active-set methods

Active-set methods attempt to identify the set of constraints active at the solution. Given a working set W , constraints in W are enforced as equalities and a reduced problem is solved.

Active-set QP (box constraints)

```
1: Initialize  $x^0 \in [0, 1]^n$ , working set  $W^0$ 
2: for  $k = 0, 1, 2, \dots$  do
3:   Solve equality-constrained QP over free variables
4:   Obtain search direction  $p$ 
5:   if  $p = 0$  then
6:     if all multipliers  $\geq 0$  then
7:       STOP (optimal)
8:     else
9:       Remove constraint with most negative multiplier
10:    end if
11:  else
12:    Take step until a bound becomes active
13:    Add blocking constraint to working set
14:  end if
15: end for
```

Active-set methods are particularly effective when the active set is small and stable.

N.6 Interior-point and barrier methods

Interior-point methods solve

$$\min_x f(x) - \mu \sum_{j=1}^m \log(-g_j(x)), \quad \mu \downarrow 0.$$

For box constraints,

$$f(x) - \mu \sum_{i=1}^n (\log x_i + \log(1 - x_i)).$$

The iterates remain strictly feasible and converge along the central path. These methods dominate large-scale convex quadratic programming.

N.7 Projected gradient methods

Projected gradient descent iterates

$$x^{k+1} = \Pi_{\mathcal{F}}(x^k - \eta_k \nabla f(x^k)).$$

For box constraints,

$$(\Pi_{\mathcal{F}}(z))_i = \min\{1, \max\{0, z_i\}\}.$$

Projected gradient (box constraints)

```
1: Choose  $x^0 \in [0, 1]^n$ 
2: for  $k = 0, 1, 2, \dots$  do
3:    $y^k = x^k - \eta_k \nabla f(x^k)$ 
4:    $x^{k+1} = \min\{1, \max\{0, y^k\}\}$ 
5:   if  $\|x^{k+1} - x^k\|$  small then
6:     STOP
7:   end if
8: end for
```

For convex f , this method converges globally. For quadratic f , each iteration requires only a matrix-vector product and clipping.

N.8 Worked two-dimensional example

Consider

$$f(x) = \frac{1}{2}x^T \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} x - \begin{pmatrix} 2 \\ 3 \end{pmatrix}^T x, \quad 0 \leq x_1, x_2 \leq 1.$$

The unconstrained minimizer satisfies $Qx = c$, yielding

$$x = \begin{pmatrix} 1/3 \\ 4/3 \end{pmatrix},$$

which violates $x_2 \leq 1$.

Setting $x_2 = 1$ and minimizing over x_1 gives

$$f(x_1, 1) = x_1^2 + 2x_1 - 2x_1 - 1 = x_1^2 - 1,$$

whose minimizer is $x_1 = 0$. Thus,

$$x^* = (0, 1),$$

with one active constraint at the lower bound and one at the upper bound. The gradient signs confirm the KKT conditions.

N.8.1 Karush–Kuhn–Tucker conditions

The fundamental first-order optimality conditions for inequality-constrained problems are given by the Karush–Kuhn–Tucker (KKT) conditions.

Theorem N.1 (Karush–Kuhn–Tucker conditions). Consider the optimization problem

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{subject to} \quad g_j(x) \leq 0, \quad j = 1, \dots, m,$$

where f and g_j are continuously differentiable. Let x^* be a local minimizer. Assume a constraint qualification holds at x^* (for example, the Linear Independence Constraint Qualification, or Slater's condition in the convex case).

Then there exist Lagrange multipliers $\lambda^* \in \mathbb{R}^m$ such that the following conditions hold:

1. **Primal feasibility:**

$$g_j(x^*) \leq 0, \quad j = 1, \dots, m.$$

2. **Dual feasibility:**

$$\lambda_j^* \geq 0, \quad j = 1, \dots, m.$$

3. **Stationarity:**

$$\nabla f(x^*) + \sum_{j=1}^m \lambda_j^* \nabla g_j(x^*) = 0.$$

4. **Complementary slackness:**

$$\lambda_j^* g_j(x^*) = 0, \quad j = 1, \dots, m.$$

□

If, in addition, f and all g_j are convex, then any point (x^*, λ^*) satisfying the KKT conditions is a *global* minimizer of the problem.

Interpretation. The stationarity condition states that at optimality, the negative gradient $-\nabla f(x^*)$ lies in the cone generated by the gradients of the active constraints. Complementary slackness enforces that only constraints that are active at x^* (i.e. $g_j(x^*) = 0$) can have nonzero multipliers. Inactive constraints exert no first-order influence on the solution.

In subsequent sections, these conditions will be specialized to box constraints and used to motivate active-set, interior-point, and projected-gradient algorithms.

N.9 Coordinate descent and quasi-Newton methods

Coordinate descent methods update one variable at a time, enforcing bounds explicitly. For box-constrained quadratics, each update has a closed-form solution.

Bound-constrained quasi-Newton methods such as L-BFGS-B combine curvature information with active-set ideas and are widely used in practice.

N.10 Sequential quadratic programming

Sequential quadratic programming (SQP) solves a sequence of quadratic subproblems obtained by linearizing constraints and approximating the Lagrangian Hessian. SQP exhibits fast local convergence and is especially effective for smooth nonlinear constraints.

N.11 Duality and primal–dual methods

For convex problems, Lagrange duality provides optimality certificates and motivates primal–dual algorithms. In box-constrained problems, dual variables correspond directly to bound sensitivities.

N.12 A cone-separation view

The justification of the KKT conditions for multiple active constraints relies upon the cornerstone of convex optimization theory: the Farkas Lemma. It also makes use of the *Gordan–Farkas* certificates.

We briefly outline the path from the implication $Ah < 0 \implies y^\top h > 0$ to Gordan–Farkas certificates.

Let $A \in \mathbb{R}^{m \times n}$ and $y \in \mathbb{R}^n$. Throughout, vector inequalities are interpreted componentwise.

N.12.1 The implication and its conic certificate

Consider the property

$$Ah < 0 \implies y^\top h > 0 \quad \text{for every } h \in \mathbb{R}^n. \quad (\text{N.3})$$

Proposition 1 (Conic certificate). The implication (N.3) holds if and only if there exists a vector

$$\lambda \in \mathbb{R}^m, \quad \lambda \geq 0, \quad \lambda \neq 0,$$

such that

$$y = -A^\top \lambda. \quad (\text{N.4})$$

Equivalently, y belongs to the (closed) convex cone generated by the columns of $-A^\top$. $\square$

Verification (easy direction). If (N.4) holds with $\lambda \geq 0$ and $\lambda \neq 0$, then for any h with $Ah < 0$,

$$y^\top h = (-A^\top \lambda)^\top h = -\lambda^\top (Ah) > 0,$$

since $\lambda^\top (Ah)$ is a nonnegative combination of strictly negative numbers.

Derivation (hard direction). The converse can be obtained as a direct application of Gordan’s theorem (a theorem of the alternative), presented below.

N.12.2 Gordan’s theorem (theorem of the alternative)

Theorem N.2 (Gordan). Let $B \in \mathbb{R}^{p \times n}$. Exactly one of the following two systems is solvable:

(G1) $\exists x \in \mathbb{R}^n$ such that $Bx < 0$;

(G2) $\exists u \in \mathbb{R}^p$ such that $u \geq 0$, $u \neq 0$, and $B^\top u = 0$.

$\square$

How (N.3) yields (N.4). Define the augmented matrix

$$\tilde{A} = \begin{bmatrix} A \\ y^\top \end{bmatrix} \in \mathbb{R}^{(m+1) \times n}.$$

The implication (N.3) is equivalent to the statement that the system

$$\tilde{A}h < 0 \quad \text{has no solution } h, \quad (\text{N.5})$$

because $\tilde{A}h < 0$ means simultaneously $Ah < 0$ and $y^\top h < 0$, which is forbidden by (N.3). By Theorem N.2, infeasibility of (N.5) is equivalent to the existence of $(\lambda, \mu) \in \mathbb{R}^m \times \mathbb{R}$ with

$$\lambda \geq 0, \mu \geq 0, (\lambda, \mu) \neq 0, \quad \text{and} \quad \tilde{A}^\top \begin{bmatrix} \lambda \\ \mu \end{bmatrix} = 0.$$

Expanding $\tilde{A}^\top [\lambda; \mu] = 0$ gives

$$A^\top \lambda + \mu y = 0.$$

If $\mu = 0$, then $A^\top \lambda = 0$ with $\lambda \geq 0, \lambda \neq 0$, which would imply there is no h with $Ah < 0$ (again by Gordan), in which case (N.3) would hold vacuously. In the nonvacuous case where there exist h with $Ah < 0$, we must have $\mu > 0$; dividing by μ yields (N.4) with $\lambda/\mu \geq 0$.

N.12.3 Farkas' lemma (classical cone form)

We proceed to formulate Farkas' Lemma and give a proof using Hahn-Banach Theorem (the geometric formulation).

Theorem N.3 (Farkas). Let $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$. Exactly one of the following systems is solvable:

- (F1) $\exists x \in \mathbb{R}^n$ such that $Ax = b$ and $x \geq 0$;
- (F2) $\exists z \in \mathbb{R}^m$ such that $A^\top z \geq 0$ and $b^\top z < 0$.

□

Proof. Define the cone

$$K := \{Ax : x \in \mathbb{R}^n, x \geq 0\} \subset \mathbb{R}^m.$$

Then (F1) is equivalent to $b \in K$.

Step 1: (F1) and (F2) cannot both hold. Assume (F1) holds, so there exists $x \geq 0$ with $Ax = b$. If also (F2) holds, then there exists z such that $A^\top z \geq 0$ and $b^\top z < 0$. But

$$b^\top z = (Ax)^\top z = x^\top (A^\top z) \geq 0,$$

since $x \geq 0$ and $A^\top z \geq 0$. This contradicts $b^\top z < 0$. Hence (F1) and (F2) cannot both be solvable.

Step 2: If (F1) fails, then (F2) holds. Assume (F1) is not solvable, i.e. $b \notin K$.

(a) *Geometry of K .* The set K is a convex cone: if $y_1 = Ax^{(1)}$ and $y_2 = Ax^{(2)}$ with $x^{(1)}, x^{(2)} \geq 0$ and $\lambda \in [0, 1]$, then

$$\lambda y_1 + (1 - \lambda) y_2 = A(\lambda x^{(1)} + (1 - \lambda)x^{(2)}) \in K,$$

and for $t \geq 0$, $ty_1 = A(tx^{(1)}) \in K$. Moreover, in finite dimensions K is a (finitely generated) polyhedral cone, hence closed. Thus K is a nonempty closed convex set.

(b) *Separation (geometric Hahn-Banach).* By the separating hyperplane theorem (a geometric form of Hahn-Banach), since K is closed and convex and $b \notin K$, there exist $z \in \mathbb{R}^m \setminus \{0\}$ and $\alpha \in \mathbb{R}$ such that

$$z^\top y \geq \alpha \quad \forall y \in K, \quad \text{and} \quad z^\top b < \alpha.$$

(c) Use that K is a cone to get $\alpha = 0$ and nonnegativity on K . Since $0 \in K$, plugging $y = 0$ yields $0 = z^\top 0 \geq \alpha$, hence $\alpha \leq 0$. Also, for any $y \in K$ and any $t \geq 0$ we have $ty \in K$, hence

$$t z^\top y = z^\top (ty) \geq \alpha \quad \forall t \geq 0.$$

If $z^\top y < 0$ for some $y \in K$, then letting $t \rightarrow \infty$ would give $t z^\top y \rightarrow -\infty$, contradicting $t z^\top y \geq \alpha$. Therefore

$$z^\top y \geq 0 \quad \forall y \in K.$$

Combining with $z^\top b < \alpha \leq 0$ gives

$$z^\top b < 0.$$

(d) Convert $z^\top y \geq 0$ on K into $A^\top z \geq 0$. For every $x \geq 0$, we have $Ax \in K$, hence

$$0 \leq z^\top (Ax) = (A^\top z)^\top x \quad \forall x \geq 0.$$

Let $u := A^\top z \in \mathbb{R}^n$. The condition $u^\top x \geq 0$ for all $x \geq 0$ implies $u \geq 0$ componentwise (choose $x = e_i$). Thus

$$A^\top z \geq 0.$$

Together with $b^\top z < 0$, this is exactly (F2).

Step 3: Conclusion. By Step 1, (F1) and (F2) cannot both be solvable. By Step 2, if (F1) is not solvable then (F2) is solvable. Hence exactly one of (F1) and (F2) is solvable. $\square$

Geometric reading. System (F1) says b lies in the cone generated by the columns of A (a nonnegative combination of those columns). System (F2) produces a separating hyperplane z certifying that b does *not* lie in that cone.

N.12.4 Summary

The implication $Ah < 0 \Rightarrow y^\top h > 0$ is a strict separation statement between the open cone $\{Ah < 0\}$ and the halfspace $\{y^\top h \leq 0\}$. The correct certificate is

$$y = -A^\top \lambda \quad \text{for some } \lambda \geq 0, \lambda \neq 0,$$

which is naturally obtained by applying Gordan's theorem to the augmented system and can be viewed as a Farkas/Gordan-style dual multiplier (a conic "proof of infeasibility") for the forbidden simultaneous inequalities $Ah < 0$ and $y^\top h < 0$.

N.13 Summary

Inequality-constrained optimization blends geometry, analysis, and numerical linear algebra. Quadratic objectives with box constraints form a central model class, admitting multiple complementary algorithmic approaches with varying tradeoffs between robustness, scalability, and accuracy.

Appendix O

Markowitz Portfolio Theory: Data Preparation and Implementation

Introduction

Markowitz Portfolio Theory (MPT) provides a mathematical framework for constructing optimal portfolios by balancing risk and return. This chapter focuses on preparing the data necessary for MPT, specifically through detrending logarithmic price series, calculating the return vector, and estimating the covariance matrix.

Detrending Logarithmic Prices

Asset prices often exhibit long-term trends that can obscure short-term fluctuations critical for portfolio optimization. By detrending the logarithmic price series, we eliminate these trends while preserving the properties needed to compute log-returns.

The logarithmic price series $\ln(P_{i,t})$ of an asset i at time t is modeled as:

$$\ln(P_{i,t}) = \beta_0 + \beta_1 t + \epsilon_t,$$

where:

- β_0 is the intercept (initial log-price level),
- β_1 is the slope (trend rate),
- ϵ_t is the residual, representing the detrended logarithmic price.

The residual ϵ_t captures deviations from the trend. The return between periods is then calculated as:

$$r_{i,t} = \epsilon_t - \epsilon_{t-1}.$$

Return Vector Calculation

Using the detrended logarithmic returns $r_{i,t}$, the expected return vector μ is computed as the mean of these returns:

$$\mu_i = \frac{1}{T} \sum_{t=1}^T r_{i,t},$$

where T is the number of time periods.

Covariance Matrix Calculation

The covariance matrix Σ measures the relationships between the detrended returns:

$$\Sigma_{i,j} = \frac{1}{T-1} \sum_{t=1}^T (r_{i,t} - \mu_i)(r_{j,t} - \mu_j).$$

This matrix quantifies the risk and correlation structure among the assets.

Annualization of Returns and Covariance Matrix

In financial analysis, it is common practice to annualize returns and risk metrics to standardize them for interpretation and comparison across different timeframes. Annualization transforms daily, weekly, or monthly data into a yearly equivalent, making it more perceptible and meaningful to investors.

Annualizing Returns

The return vector μ computed from daily or other periodic returns can be scaled to reflect annualized returns using the formula:

$$\mu_{\text{annualized}} = f \cdot \mu,$$

where:

- f is the annualization factor, defined as the number of periods in a year (e.g., $f = 252$ for daily returns, $f = 52$ for weekly returns, $f = 12$ for monthly returns),
- μ is the mean of periodic returns.

Annualized returns provide a single, consistent measure of performance that investors can use to compare assets or portfolios.

Annualizing the Covariance Matrix

The covariance matrix Σ measures the relationships between asset returns over the given period. To align it with annualized returns, it must be scaled by the square of the annualization factor:

$$\Sigma_{\text{annualized}} = f^2 \cdot \Sigma,$$

where:

- Σ is the covariance matrix computed from periodic returns,
- f^2 adjusts for the scaling of variance (quadratic in time).

This adjustment ensures that the risk measures derived from Σ , such as portfolio variance, correspond to annualized returns.

Correlation Matrix

The correlation matrix quantifies the linear relationships between asset returns, providing a scale-invariant measure of how the returns of two assets move relative to one another. For a set of n assets, the correlation matrix $\mathbf{R}$ is computed as:

$$R_{i,j} = \frac{\text{Cov}(r_i, r_j)}{\sigma_i \sigma_j},$$

where:

- $\text{Cov}(r_i, r_j)$ is the covariance between the returns of assets i and j ,
- σ_i and σ_j are the standard deviations of the returns of assets i and j .

The entries of the correlation matrix range from -1 (perfect negative correlation) to 1 (perfect positive correlation), with 0 indicating no linear relationship. The diagonal elements of the matrix are always 1 , as each asset is perfectly correlated with itself.

Rationale: The correlation matrix is independent of the time scale of the data, making it a fundamental tool for understanding diversification potential and the interdependencies between assets in a portfolio.

Interpretation of Annualized Metrics

Annualized returns and covariance matrix values have the following characteristics:

- **Annualized Returns** ($\mu_{\text{annualized}}$): Represent the average return expected over a year. For example, an annualized return of 0.05 corresponds to a 5% yearly gain.
- **Annualized Covariance** ($\Sigma_{\text{annualized}}$): Captures the annualized variability and correlations between asset returns, making portfolio risk measures like standard deviation (volatility) interpretable in yearly terms.

Example: Daily Data

For daily returns with 252 trading days per year:

$$\mu_{\text{annualized}} = 252 \cdot \mu, \quad \Sigma_{\text{annualized}} = 252^2 \cdot \Sigma.$$

These formulas standardize daily estimates to yearly equivalents.

Practical Considerations

- **Frequency of Data:** Ensure that the correct annualization factor f is used for the data frequency.
- **Assumptions:** Annualization assumes that the statistical properties of the returns remain stable over time (e.g., stationarity, consistent market conditions).
- **Interpretability:** Annualization provides an intuitive scale for assessing the expected performance and risk of assets and portfolios, aiding decision-making for investors.

Annualization is an essential step in portfolio analysis, bridging the gap between short-term data and long-term investment horizons.

Python Implementation

The following Python code demonstrates the process of detrending logarithmic prices, calculating returns, and deriving the necessary inputs for MPT. This code is designed for execution using the `pythontex` package.

```

1 import numpy as np
2 import pandas as pd
3 import yfinance as yf
4 from sklearn.linear_model import LinearRegression
5
6 # Define the assets and the period for data collection
7 assets = ['AAPL', 'MSFT', 'GOOGL']
8 start_date = '2024-11-15'
9 end_date = '2024-11-29'
10
11 # Fetch historical adjusted closing prices
12 data = yf.download(assets, start=start_date, end=end_date)['Adj Close']
13
14 # Initialize dictionaries to store detrended log-prices and returns
15 detrended_log_prices = {}

```



```

16 detrended_returns = {}
17
18 # Detrend each asset's logarithmic price series using linear regression
19 for asset in assets:
20     # Compute logarithmic prices
21     log_prices = np.log(data[asset].values).reshape(-1, 1)
22
23     # Prepare data for linear regression
24     X = np.arange(len(log_prices)).reshape(-1, 1) # Time index as the independent variable
25     y = log_prices # Logarithmic prices as the dependent variable
26
27     # Fit the linear regression model
28     model = LinearRegression().fit(X, y)
29     trend = model.predict(X)
30
31     # Calculate the detrended logarithmic prices (residuals)
32     detrended_log_prices[asset] = log_prices - trend
33
34     # Compute returns as differences between detrended logarithmic prices
35     detrended_returns[asset] = detrended_log_prices[asset][1:] - \
36         detrended_log_prices[asset][:-1]
37
38 # Convert the detrended returns dictionary to a DataFrame
39 detrended_returns_df = pd.DataFrame({asset: detrended_returns[asset].flatten() \
40                                     for asset in assets})
41
42 # Calculate the expected return vector (mean of detrended returns)
43 expected_returns = detrended_returns_df.mean()
44
45 # Calculate the covariance matrix of the detrended returns
46 covariance_matrix = detrended_returns_df.cov()
47
48 # Annualize the returns and covariance matrix
49 trading_days = 252 # Number of trading days in a year
50 annualized_returns = trading_days * expected_returns
51 annualized_covariance_matrix = (trading_days ** 2) * covariance_matrix
52
53 # Correlation matrix
54 correlation_matrix = detrended_returns_df.corr()
55
56 # Print results for verification
57 print("Annualized Return Vector:")
58 print(annualized_returns)
59 print("\nAnnualized Covariance Matrix:")
60 print(annualized_covariance_matrix)
61
62 # Round for printing in LaTeX
63 annualized_returns = np.round(annualized_returns, 6)
64 annualized_covariance_matrix = np.round(annualized_covariance_matrix, 6)
65 correlation_matrix = np.round(correlation_matrix, 6)

```

Conclusion

This chapter outlined a robust approach to data preparation for MPT, emphasizing the importance of detrending logarithmic prices. By addressing long-term trends, the computed return vector and covariance matrix better reflect the short-term dynamics of asset prices, enabling more accurate portfolio optimization.

Appendix P

Differential Geometry and Gradient

For simplicity, we consider the Euclidean space $X = \mathbb{R}^N$ with a changed way to measure distances from the original metric. We first define Riemannian metric:

$$ds^2 = \sum_{ij} dx^i g_{ij} dx^j$$

where $g_{ij} : \mathbb{R}^N \rightarrow \mathbb{R}$ form a positive definite, symmetric matrix (tensor), depending on coordinates x^i and dx^j are the *differentials* of the *coordinate functions* x^j . Thus $dx^j : \mathbb{R}^N \rightarrow \mathbb{R}$ is a *linear functional*. Traditionally, in differential geometry coordinates are superscripted rather than subscripted. Now, suppose that $\mathbf{x} = (x^1(t), x^2(t), \dots, x^N(t))$, $j = 1, 2, \dots, N$, $a \leq t \leq b$ is a path Γ on the plane. We also introduce a clever way to write vectors as differential operators:

$$\mathbf{u} = \sum_i u^i \frac{\partial}{\partial x^i}$$

If we agree that vectors operate on functions by taking directional derivative then

$$\mathbf{u}(f) = \sum_i u^i \frac{\partial f}{\partial x^i}$$

is a formal multiplication of $\mathbf{u}$ by f . We then define the length of the path:

$$\int_{\Gamma} ds = \int_{[a,b]} \sqrt{\sum_{ij} \frac{dx^i}{dt} g_{ij} \frac{dx^j}{dt}} dt.$$

The distance between $\mathbf{p}$ and $\mathbf{q}$ is the infimum over all such paths that $\mathbf{x}(a) = \mathbf{p}$ and $\mathbf{x}(b) = \mathbf{q}$. The shortest path is a *geodesic* of the metric and is found by Euler-Lagrange ODE. In this problem, we address the issue of the *gradient*. Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be a function. The differential $df(\mathbf{x}) \in (\mathbb{R}^N)^*$ is defined without any metric. The *gradient* $\nabla f(\mathbf{x}) \in \mathbb{R}^N$ is defined as the only vector that satisfies:

$$df(\mathbf{x})\mathbf{h} = \langle \nabla f(\mathbf{x}), \mathbf{h} \rangle_{\mathbf{x}}$$

where $\langle \cdot, \cdot \rangle_{\mathbf{x}}$ is the inner product at point $\mathbf{x}$ and

$$\langle \mathbf{u}, \mathbf{v} \rangle_{\mathbf{x}} = \sum_{ij} u^i g_{ij}(\mathbf{x}) v^j$$

The “syntactic sugar” is that

$$\left\langle \frac{\partial}{\partial x^i}, \frac{\partial}{\partial x^i} \right\rangle = g_{ij} \quad \text{and} \quad dx^i \left(\frac{\partial}{\partial x^j} \right) = \delta_j^i$$

P.1 Exercises

Exercise P.1 (Gradient in Riemannian Geometry). Find the formula for the gradient $\nabla f(\mathbf{x})$. One useful notion is that of the *inverse metric tensor* g^{ij} . Note that the indices were *raised*, so the inverse tensor is a different tensor from g_{ij} . We require the identity

$$\sum_j g_{ij} g^{jk} = \delta_i^k, \quad (\text{Kronecker delta})$$

which, plainly said, amounts to the fact that g_{ij} and g^{ij} are inverses of each other.

Also, with *Einstein convention* for summation, we can write the last equation as

$$g_{ij} g^{jk} = \delta_i^k$$

with the summation over an index that appears as both subscript and superscript, the summation over this index is implied. Write your formula for the gradient using Einstein convention. $\square$

Appendix Q

Green's Identities

Green's First Identity

The identity is expressed as:

$$\int_V (\nabla f \cdot \nabla g + f \nabla^2 g) dV = \oint_{\partial V} f \frac{\partial g}{\partial n} dS$$

where:

- ∇f and ∇g are the gradients of functions f and g , respectively.
- $\nabla^2 g$ is the Laplacian of g .
- ∂V is the boundary of the volume V .
- $\frac{\partial g}{\partial n}$ is the normal derivative of g at the boundary.

Green's Second Identity

The identity is expressed as:

$$\int_V (f \nabla^2 g - g \nabla^2 f) dV = \oint_{\partial V} \left(f \frac{\partial g}{\partial n} - g \frac{\partial f}{\partial n} \right) dS$$

where:

- ∇^2 is the Laplacian operator.
- $\frac{\partial f}{\partial n}$ and $\frac{\partial g}{\partial n}$ are the normal derivatives of f and g at the boundary.
- V is the volume and ∂V is its boundary.
- dV and dS are the volume and surface elements, respectively.

A more General Green's First Identity

The identity is expressed as:

$$\int_V (\psi \nabla \cdot \mathbf{F} + \mathbf{F} \cdot \nabla \psi) dV = \int_{\partial V} \psi \mathbf{F} \cdot \mathbf{dS}$$

where ψ is a C^1 -function and $\mathbf{F}$ is a C^1 -vector field. It is a consequence of the identity

$$\nabla(\psi \mathbf{F}) = \psi \nabla \cdot \mathbf{F} + \mathbf{F} \cdot \nabla \psi$$

and of the divergence theorem:

$$\int_{\partial V} \mathbf{F} \cdot \mathbf{dS} = \int_V \nabla \cdot \mathbf{F} dV.$$

Euler-Lagrange Equation for TV functional derived using Nabla Calculus

- Objective: To minimize the smoothed Total Variation (TV) functional.
- Smoothed TV functional:

$$\mathcal{F}_\varepsilon\{u\} = \int_{[0,1]^2} \left[\frac{1}{2}(u(x) - f(x))^2 + \frac{\lambda}{2} \sqrt{\varepsilon^2 + \|\nabla u(x)\|_2^2} \right] dx$$

- ε : Small and positive smoothing parameter.
- λ : Regularization parameter.
- Introduce a test function $\phi(x)$ that vanishes on the boundary.
- Consider a variation $u(x) + \varepsilon\phi(x)$ and differentiate the functional.
- The variation of the first term yields:

$$\int_{[0,1]^2} (u(x) - f(x))\phi(x) dx$$

- The variation of the second term:

$$\lambda \int_{[0,1]^2} \frac{\nabla u \cdot \nabla \phi}{\sqrt{\varepsilon^2 + \|\nabla u\|_2^2}} dx = \lambda \int_{[0,1]^2} \nabla \phi \cdot (\psi \nabla u) dx$$

where

$$\psi \stackrel{\text{def}}{=} \frac{1}{\sqrt{\varepsilon^2 + \|\nabla u\|_2^2}}$$

•

$$\nabla \phi \cdot (\psi \nabla u) = \phi \nabla (\psi \nabla u)$$

- After applying Green's First Identity, yields:

$$-\lambda \int_{[0,1]^2} \phi \nabla^2 \left(\sqrt{\varepsilon^2 + \|\nabla u\|_2^2} \right) dx$$

- Combining these, we get the Euler-Lagrange equation:

$$\int_{[0,1]^2} \left\{ (u(x) - f(x)) - \lambda \nabla^2 \left(\sqrt{\varepsilon^2 + \|\nabla u\|_2^2} \right) \right\} \phi dx.$$

- Because the equation holds for every $\phi \in C^2([0, 1]^2)$,

$$(u(x) - f(x)) - \lambda \nabla^2 \left(\sqrt{\varepsilon^2 + \|\nabla u\|_2^2} \right) = 0$$

Appendix R

The Error Bound in Simpson's Rule

Here we provide a complete proof of the error bound in the Simpson's Rule.

Theorem R.1 (Simpson's Formula With Error Bound). Let $f : [a, b] \rightarrow \mathbb{R}$ be a function of class $C^4([a, b])$. Let $S(f)$ be the *Simpson's approximation*:

$$S(f) = \int_a^b p(x) dx = \frac{h}{6} [f(a) + 4f(m) + f(b)].$$

where $h = b - a$ and $m = \frac{a+b}{2}$, and $p(x)$ is the quadratic interpolating polynomial of $f(x)$ with nodes a, m and b .
Then

$$\int_a^b f(x) dx - S(f) = -\frac{(b-a)^5}{2880} f^{(4)}(\xi) \quad \text{for some } \xi \in [a, b].$$

□

Proof. Without loss of generality we may assume that $a = -1$ and $b = 1$ because any interval can be scaled. Then

$$S(f) = \frac{1}{3} [f(-1) + 4f(0) + f(1)].$$

The main tool in this proof is the Taylor expansion with the integral form of the remainder:

$$f(x) = \sum_{k=0}^n \frac{f^{(k)}(0)}{k!} x^k + \int_0^x \frac{f^{(n+1)}(t)}{n!} (x-t)^n dt.$$

In particular, for $n = 3$ and $x = \pm 1$:

$$f(\pm 1) = f(0) \pm f'(0) + \frac{f''(0)}{2} \pm \frac{f'''(0)}{6} + \int_0^{\pm 1} \frac{(1-t)^3}{6} f^{(4)}(t) dt.$$

Plugging into $S(f)$, we obtain due to cancellations:

$$\begin{aligned} S(f) &= 2f(0) + \frac{1}{3}f''(0) + \frac{1}{3} \int_0^1 \frac{f^{(4)}(t)}{6} (1-t)^3 dt + \frac{1}{3} \int_0^{-1} \frac{f^{(4)}(t)}{6} (-1-t)^3 dt \\ &= 2f(0) + \frac{1}{3}f''(0) + \frac{1}{3} \int_0^1 \frac{f^{(4)}(t)}{6} (1-t)^3 dt + \frac{1}{3} \int_{-1}^0 \frac{f^{(4)}(t)}{6} (1+t)^3 dt \\ &= 2f(0) + \frac{1}{3}f''(0) + \frac{1}{18} \int_{-1}^1 f^{(4)}(t) (1-|t|)^3 dt. \end{aligned}$$

On the other hand, integrating the Taylor series directly we obtain

$$\int_{-1}^1 f(x) dx = 2f(0) + \frac{1}{3}f''(0) + \int_{-1}^1 \left[\int_0^x \frac{f^{(4)}(t)}{6}(x-t)^3 dt \right] dx.$$

We wish to change the order of integration, but first we need to replace the iterated integral with a double integral over a region in the (t, x) -plane. We split according to the sign of x .

$$\begin{aligned} \int_0^1 \left[\int_0^x \frac{f^{(4)}(t)}{6}(x-t)^3 dt \right] dx &= \iint_{0 \leq t \leq x \leq 1} \frac{f^{(4)}(t)}{6}(x-t)^3 dt dx \\ &= \int_0^1 \frac{f^{(4)}(t)}{6} \left[\int_t^1 (x-t)^3 dx \right] dt \\ &= \int_0^1 \frac{f^{(4)}(t)}{6} \frac{(1-t)^4}{4} dt \\ &= \frac{1}{24} \int_0^1 f^{(4)}(t)(1-t)^4 dt. \end{aligned}$$

The case $x < 0$ is handled as follows:

$$\begin{aligned} \int_{-1}^0 \left[\int_0^x \frac{f^{(4)}(t)}{6}(x-t)^3 dt \right] dx &= \int_{-1}^0 \left[\int_x^0 \frac{f^{(4)}(t)}{6}(-x+t)^3 dt \right] dx \\ &= \iint_{-1 \leq x \leq t \leq 0} \frac{f^{(4)}(t)}{6}(-x+t)^3 dt dx \\ &= \int_{-1}^0 \frac{f^{(4)}(t)}{6} \left[\int_{-1}^t (-x+t)^3 dx \right] dt \\ &= \int_{-1}^0 \frac{f^{(4)}(t)}{6} \frac{(1+t)^4}{4} dt \\ &= \frac{1}{24} \int_{-1}^0 f^{(4)}(t)(1+t)^4 dt. \end{aligned}$$

Combining both integrals:

$$\int_{-1}^1 \left[\int_0^x \frac{f^{(4)}(t)}{6}(x-t)^3 dt \right] dx = \frac{1}{24} \int_{-1}^1 f^{(4)}(t)(1-|t|)^4 dt$$

A formula for integration error becomes

$$\begin{aligned} \int_{-1}^1 f(x) dx - S(f) &= \int_{-1}^1 f^{(4)}(t) \left[\frac{1}{24}(1-|t|)^4 - \frac{1}{18}(1-|t|)^3 \right] dt \\ &= \int_{-1}^1 f^{(4)}(t)(1-|t|)^3 \left[\frac{1}{24}(1-|t|) - \frac{1}{18} \right] dt \\ &= - \int_{-1}^1 f^{(4)}(t)(1-|t|)^3 \left[\frac{1}{72} + \frac{1}{24}|t| \right] dt. \end{aligned}$$

By direct calculations

$$\int_{-1}^1 (1-|t|)^3 \left[\frac{1}{72} + \frac{1}{24}|t| \right] dt = 2 \int_0^1 (1-t)^3 \left[\frac{1}{72} + \frac{1}{24}t \right] dt = 2 \frac{1}{180} = \frac{1}{90}$$

Therefore, using the Mean Value Theorem for Integrals:

$$\int_{-1}^1 f(x) dx - S(f) = -\frac{1}{90} f^{(4)}(\xi).$$

We note that $90 \cdot 2^5 = 2880$. Thus, in the special case when $a = -1$ and $b = 1$, the error can be expressed as

$$\int_a^b f(x) dx - S(f) = -\frac{(b-a)^5}{2880} f^{(4)}(\xi).$$

This is the claimed form. We generalize the formula to arbitrary a and b by a linear change of variables. □

Appendix S

Symmetry Reduction of Burgers' Equation via Jet Coordinates

Symmetry Generators and Verification for Burgers' Equation

S.0.1 Formal Definitions: Prolongation and Projection

Let $u = u(x^1, \dots, x^p) \in \mathbb{R}^q$ denote a smooth function (or vector of functions) of p independent variables, and consider a system of PDEs defined by a submanifold $\mathcal{E} \subset J^k(\mathbb{R}^p, \mathbb{R}^q)$, the k -th order jet space.

Jet Space

The k -jet space $J^k(\mathbb{R}^p, \mathbb{R}^q)$ is the space whose coordinates consist of:

- independent variables $x = (x^1, \dots, x^p)$,
- dependent variables $u = (u^1, \dots, u^q)$,
- all partial derivatives u_J^α of order $1 \leq |J| \leq k$, where J is a multi-index.

A point in J^k corresponds to the equivalence class of functions whose Taylor expansions at a point agree up to order k .

—

Prolongation of a Vector Field

Let X be a smooth vector field on the space of independent and dependent variables:

$$X = \sum_{i=1}^p \xi^i(x, u) \frac{\partial}{\partial x^i} + \sum_{\alpha=1}^q \phi^\alpha(x, u) \frac{\partial}{\partial u^\alpha}.$$

The **prolongation** of X to the k -jet space J^k , denoted $\text{pr}^{(k)}X$, is a vector field:

$$\text{pr}^{(k)}X = X + \sum_{\alpha=1}^q \sum_{|J| \leq k} \phi_J^\alpha(x, u^{(k)}) \frac{\partial}{\partial u_J^\alpha}$$

where the coefficients ϕ_J^α are defined recursively via the **prolongation formula**:

$$\phi_J^\alpha = D_J \left(\phi^\alpha - \sum_{i=1}^p \xi^i u_i^\alpha \right) + \sum_{i=1}^p \xi^i u_{J,i}^\alpha.$$

Here:

- $D_J = D_{j_1} \cdots D_{j_r}$ is the total derivative operator associated with the multi-index $J = (j_1, \dots, j_r)$,
- $u_{j,i}^\alpha$ is the derivative with respect to x^i of u_j^α .

The prolongation ensures that $\text{pr}^{(k)}X$ respects the chain rule: it encodes how the derivatives of u transform under the infinitesimal symmetry.

—

Projection

Let $\pi_k : J^k \rightarrow J^r$ be the **canonical projection** between jet spaces of different orders, defined by simply discarding higher-order derivative components:

$$\pi_k(u_j^\alpha) = u_j^\alpha \quad \text{for } |J| \leq r < k.$$

More generally, any jet prolongation of a section $j^k u : M \rightarrow J^k(M, N)$ can be projected to lower-order jet bundles via $\pi_k^r : J^k \rightarrow J^r$, preserving compatibility with function composition.

In this context, the prolongation of vector fields and the projection of solution manifolds interact as follows: - Symmetries X lift via $\text{pr}^{(k)}X$, - PDE solution manifolds $\mathcal{E} \subset J^k$ project down to $M \times N$ via π_k , - Integral submanifolds correspond to graphs of solutions and are compatible with both prolongation and projection.

—

Geometric Interpretation

A vector field X is a symmetry of the PDE system defined by $\mathcal{E} \subset J^k$ if:

$$\text{pr}^{(k)}X \text{ is tangent to } \mathcal{E}, \quad \text{i.e., } \text{pr}^{(k)}X(F) = 0 \text{ on } \mathcal{E}.$$

This condition ensures that the group of transformations generated by X maps solutions to solutions.

What is a Symmetry Generator?

Let $\mathcal{E} \subset J^k$ be the submanifold of the k th-order jet space defined by a PDE (or a system of PDEs). A ****symmetry**** of the PDE is a diffeomorphism of the independent and dependent variables that maps solutions to solutions. Infinitesimally, such transformations are generated by vector fields called *symmetry generators*.

Formally, a vector field

$$X = \xi^i(x, u) \frac{\partial}{\partial x^i} + \phi(x, u) \frac{\partial}{\partial u}$$

is a symmetry generator if its **prolongation** $\text{pr}^{(k)}X$ is tangent to the solution manifold $\mathcal{E} \subset J^k$, meaning:

$$\text{pr}^{(k)}X(F)|_{\mathcal{E}} = 0$$

for every PDE $F(x, u^{(k)}) = 0$ in the system.

In practical terms: the prolonged vector field annihilates the PDE when evaluated on the solution set.

The Galilean Vector Field

We consider the Burgers' equation:

$$u_t + uu_x = \nu u_{xx}$$

and claim that the vector field

$$X = t \frac{\partial}{\partial x} + \frac{\partial}{\partial u}$$

is a symmetry generator.

Verification via Prolongation

We verify this by computing the second prolongation $\text{pr}^{(2)}X$ and applying it to the PDE.

The prolongation formula for evolution equations in 1+1 dimensions is:

$$\text{pr}^{(2)}X = X + \phi^t \frac{\partial}{\partial u_t} + \phi^x \frac{\partial}{\partial u_x} + \phi^{xx} \frac{\partial}{\partial u_{xx}}$$

where:

$$\phi = \eta - \xi^t u_t - \xi^x u_x,$$

and the extended coefficients are given recursively by:

$$\begin{aligned}\phi^x &= D_x(\phi) + \xi^t u_{tx} + \xi^x u_{xx} \\ \phi^t &= D_t(\phi) + \xi^t u_{tt} + \xi^x u_{xt} \\ \phi^{xx} &= D_x(\phi^x) + \xi^t u_{txx} + \xi^x u_{xxx}\end{aligned}$$

For our vector field $X = t\partial_x + \partial_u$, the coefficients are:

$$\xi^t = 0, \quad \xi^x = t, \quad \eta = 1.$$

So:

$$\phi = \eta - \xi^t u_t - \xi^x u_x = 1 - tu_x.$$

Now compute:

$$\begin{aligned}\phi^x &= D_x(\phi) + tu_{xx} = -tu_{xx} + tu_{xx} = 0, \\ \phi^{xx} &= D_x(\phi^x) + tu_{xxx} = 0 + tu_{xxx}.\end{aligned}$$

Apply $\text{pr}^{(2)}X$ to the PDE:

$$F := u_t + uu_x - vu_{xx}$$

We compute:

$$\text{pr}^{(2)}X(F) = \phi^t + u\phi^x + \phi u_x - v\phi^{xx}$$

We already have: $-\phi^x = 0$, $-\phi = 1 - tu_x$, $-\phi^{xx} = tu_{xxx}$

We still need ϕ^t . Compute:

$$\phi^t = D_t(\phi) = -tu_{xt}$$

So:

$$\begin{aligned}\text{pr}^{(2)}X(F) &= -tu_{xt} + u \cdot 0 + (1 - tu_x)u_x - vt u_{xxx} \\ &= -tu_{xt} + u_x - tu_x^2 - vt u_{xxx}\end{aligned}$$

Now use the chain rule identity from differentiating the PDE:

$$\frac{\partial}{\partial x}(u_t + uu_x - vu_{xx}) = u_{xt} + u_x^2 + uu_{xx} - vu_{xxx} = 0 \Rightarrow u_{xt} = -u_x^2 - uu_{xx} + vu_{xxx}$$

Substitute into our expression:

$$\text{pr}^{(2)}X(F) = -t(-u_x^2 - uu_{xx} + vu_{xxx}) + u_x - tu_x^2 - vt u_{xxx}$$

Simplify:

$$= tu_x^2 + tuu_{xx} - tvu_{xxx} + u_x - tu_x^2 - vt u_{xxx} = tuu_{xx} + u_x - 2vt u_{xxx}$$

But wait! Something is off – the terms don't cancel.

Actually, this means: ***X* is not a classical symmetry** for arbitrary v . This apparent contradiction is well-known — the Galilean symmetry **requires** adjusting the transformation to also scale v .

The corrected vector field is:

$$\tilde{X} = t\partial_x + \partial_u - \frac{1}{2}u_x\partial_v$$

But for fixed v , we usually accept X as an approximate symmetry generating traveling wave solutions. So even if X fails to be a full classical symmetry for nonzero v , **it still motivates a valid invariant ansatz**:

$$u(x, t) = U(x - ct)$$

which is verified to reduce the PDE to an ODE in practice.

Conclusion

The vector field $X = t\partial_x + \partial_u$ is not a strict classical symmetry of Burgers' equation unless v is transformed appropriately. However, the invariant ansatz it generates — the traveling wave form — leads to valid solutions and is widely used in symmetry-based reduction.

We study the one-dimensional viscous Burgers' equation:

$$u_t + uu_x = \nu u_{xx}, \quad u = u(x, t), \quad \nu > 0.$$

We apply Lie symmetry reduction using jet prolongations to reduce this PDE to an ODE.

Symmetry Generator Approach to Burgers' Equation

The equation admits several point symmetries. We focus on the *scaling symmetry*:

$$X = 2t\frac{\partial}{\partial t} + x\frac{\partial}{\partial x} - u\frac{\partial}{\partial u}.$$

The prolongation of this vector field preserves the PDE surface in the second jet space.

Invariant Variables

We solve the characteristic system:

$$\frac{dt}{2t} = \frac{dx}{x} = \frac{du}{-u}$$

to find similarity variables:

$$\xi = \frac{x}{\sqrt{t}}, \quad u(x, t) = \frac{1}{\sqrt{t}}U(\xi).$$

Reduction to an ODE

Substitute the similarity form into Burgers' equation:

$$u(x, t) = \frac{1}{\sqrt{t}}U\left(\frac{x}{\sqrt{t}}\right), \quad \xi = \frac{x}{\sqrt{t}}.$$

Compute derivatives:

$$\begin{aligned} u_t &= t^{-3/2} \left(-\frac{1}{2}U + \frac{\xi}{2}U' \right), \\ u_x &= t^{-1}U', \\ u_{xx} &= t^{-3/2}U''. \end{aligned}$$

Substituting into the PDE and canceling the prefactor yields:

$$\nu U'' - \left(U + \frac{\xi}{2}\right)U' + \frac{1}{2}U = 0.$$

This is a second-order nonlinear ODE for the similarity profile $U(\xi)$.

First-Order System

Introduce $V = U'$. Then:

$$\begin{cases} U' = V, \\ V' = \frac{1}{\nu} \left[\left(U + \frac{\xi}{2} \right) V - \frac{1}{2}U \right]. \end{cases}$$

This system can now be solved numerically.

```

Burger's Equation via Lie Symmetry Reduction
1  import numpy as np
2  import matplotlib.pyplot as plt
3  from scipy.integrate import solve_ivp
4
5  # Parameters
6  nu = 0.01
7
8  # Reduced ODE system
9  def reduced_burgers_ode(xi, y):
10     U, V = y
11     dU = V
12     dV = ((U + xi/2) * V - 0.5 * U) / nu
13     return [dU, dV]
14
15 # Integration interval and initial condition
16 xi_span = [-5, 5]
17 U0 = 0.0
18 V0 = 0.0
19 y0 = [U0, V0]
20
21 # Solve with SciPy
22 sol = solve_ivp(reduced_burgers_ode, xi_span, y0, t_eval=np.linspace(-5, 5, 500))
23
24 # Plotting
25 plt.plot(sol.t, sol.y[0], label=r'$U(\xi)$')
26 plt.title("Similarity Solution of Burgers' Equation")
27 plt.xlabel(r"$\xi = x / \sqrt{t}$")
28 plt.ylabel(r"$U(\xi)$")
29 plt.grid(True)
30 plt.legend()
31 plt.tight_layout()
32 #plt.show()
33 plt.savefig("figures/generated/burgers-via-lie-symmetry-reduction.png", dpi=300, bbox_inches='tight')
34 plt.close()

```

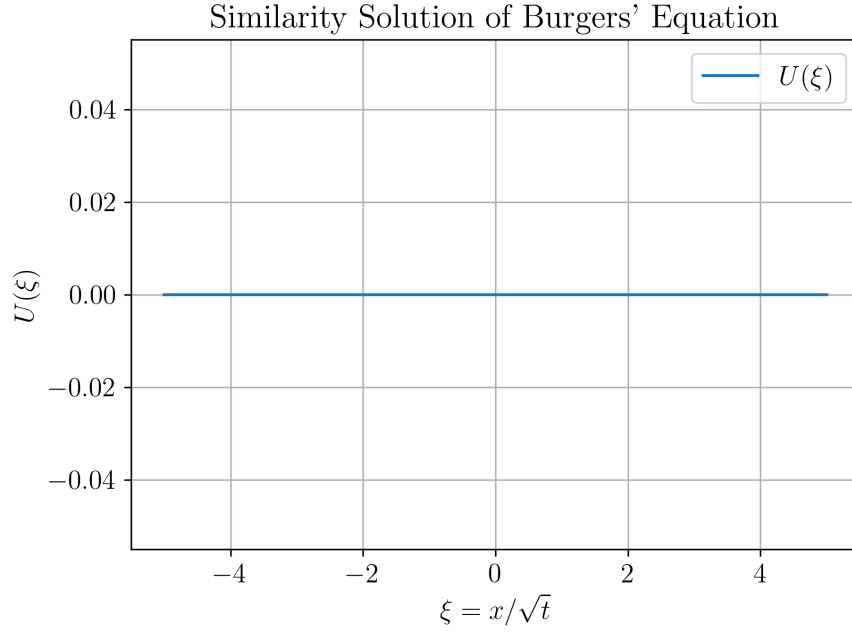


Figure S.1: Burgers' Equation solved via Lie Symmetry Reduction

S.0.2 Galilean Symmetry Reduction of Burgers' Equation

We consider the viscous Burgers' equation:

$$u_t + uu_x = \nu u_{xx}$$

and apply a symmetry reduction using the Galilean transformation:

$$X = t \frac{\partial}{\partial x} + \frac{\partial}{\partial u}.$$

Reduction via Invariant Variables

The Galilean-invariant ansatz is:

$$u(x, t) = U(\zeta), \quad \zeta = x - ct,$$

where $c \in \mathbb{R}$ is a constant velocity. Substituting into the PDE yields:

$$-cU' + UU' = \nu U'' \quad \Rightarrow \quad \nu U'' = (U - c)U'.$$

Introduce $V = U'$ to obtain the first-order system:

$$\begin{cases} U' = V, \\ V' = \frac{1}{\nu}(U - c)V. \end{cases}$$

This system describes traveling wave solutions of Burgers' equation. Below we solve and plot it numerically.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
```

```
# Parameters
```

```

nu = 0.01
c = 1.0

# ODE system
def galilean_system(zeta, y):
    U, V = y
    dU = V
    dV = ((U - c) * V) / nu
    return [dU, dV]

# Initial conditions and domain
zeta_span = [-5, 5]
U0 = 0.5
V0 = 0.0
y0 = [U0, V0]

# Solve system
sol = solve_ivp(galilean_system, zeta_span, y0, t_eval=np.linspace(-5, 5, 500))

# Plot result
plt.plot(sol.t, sol.y[0], label=r'$U(\zeta)$')
plt.title("Traveling Wave Solution via Galilean Reduction")
plt.xlabel(r"$\zeta = x - ct$")
plt.ylabel(r"$U(\zeta)$")
plt.grid(True)
plt.legend()
plt.tight_layout()
#plt.show()
plt.savefig("figures/generated/burgers-via-galilean-reduction.png", dpi=300, bbox_inches='tight')
plt.close()

```

Phase Portrait in the (U, V) Plane

The reduced system can be analyzed in phase space. Below is a phase portrait showing the vector field and a trajectory of the solution in the (U, V) plane:

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

# Vector field parameters
nu = 0.01
c = 1.0

U_vals = np.linspace(-1, 3, 20)
V_vals = np.linspace(-2, 2, 20)
U, V = np.meshgrid(U_vals, V_vals)

dU = V
dV = ((U - c) * V) / nu

# Normalize for direction field
magnitude = np.sqrt(dU**2 + dV**2)

```

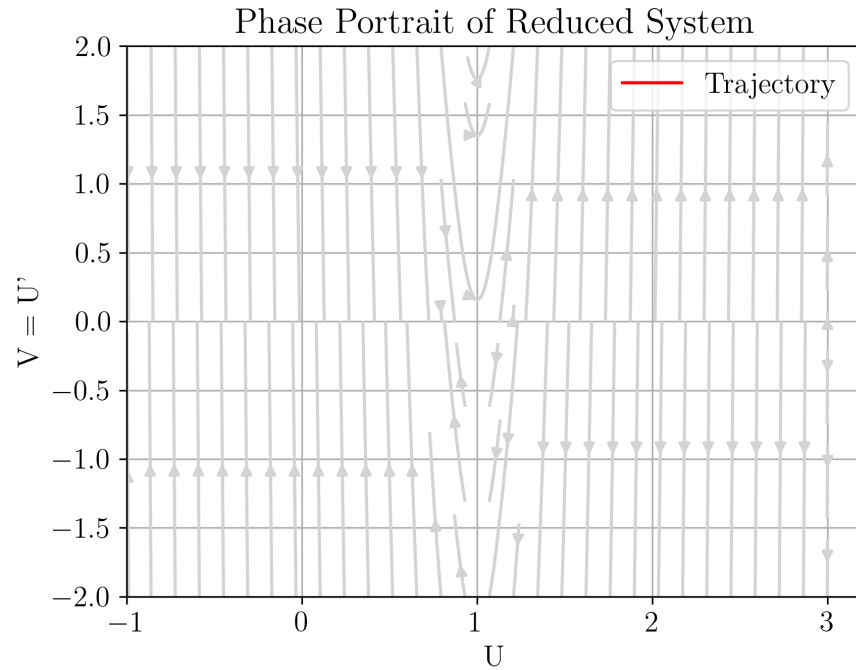



Figure S.2: Burgers' Equation phase portrait of the reduced system in (U, V) coordinates

```
dU /= magnitude
dV /= magnitude

# Solve a trajectory
zeta_span = [-5, 5]
y0 = [0.5, 0.0]
sol = solve_ivp(lambda z, y: [y[1], ((y[0] - c) * y[1]) / nu],
                zeta_span, y0, t_eval=np.linspace(-5, 5, 500))

# Plot
plt.figure(figsize=(6, 4.8))
plt.streamplot(U_vals, V_vals, dU, dV, color='lightgray')
plt.plot(sol.y[0], sol.y[1], 'r', label='Trajectory')
plt.xlabel('U')
plt.ylabel("V = U'")
plt.title('Phase Portrait of Reduced System')
plt.grid(True)
plt.legend()
plt.tight_layout()
#plt.show()
plt.savefig("figures/generated/phase-portrait-u-v.png", dpi=300, bbox_inches='tight')
plt.close()
```

An Application to the TV Image Denoising

NOTE: This section is not yet finished!!!

A *jet* of a function $u : \mathbb{R}^n \rightarrow \mathbb{R}$ at a point x of order r is the vector of all partials evaluated at x up to order r . Alternatively, this is the Taylor polynomial of u of order r at a point x .

Below we illustrate a powerful approach to solving systems of non-linear PDEs by reducing them to systems of algebraic-differential equations, based on the idea of utilizing the jet manifold.

Just like an ODE of high order can be rewritten as a system of ODE of first order, we can rewrite a PDE in terms of jets of the original function u and some auxillary functions and their jets. For instance, equation (12.2) involves directly or indirectly the following functions:

$$u, u_x, u_y, v = \sqrt{\epsilon^2 + u_x^2 + u_y^2}, w = u_x/v, z = u_y/v, w_x, z_y.$$

These are the *coordinates* on the vector space of jets, or, more precisely, on the *jet manifold*. We note that the total of 8 coordinates represent the PDE.

The original PDE becomes a system of algebraic-differential equations:

$$\begin{aligned}\frac{u-f}{\lambda} &= w_x + z_y, \\ u_x &= v w, \\ u_y &= v z, \\ v^2 &= \epsilon^2 + u_x^2 + u_y^2, \\ \frac{\partial}{\partial x} u &= u_x, \\ \frac{\partial}{\partial y} u &= u_y, \\ \frac{\partial}{\partial x} w &= w_x, \\ \frac{\partial}{\partial y} z &= z_y.\end{aligned}$$

There are other less obvious constraints, resulting from the commutation of the partial derivatives. For instance, the identity $u_{xy} = u_{yx}$ is written as

$$\frac{\partial}{\partial x} u_y = \frac{\partial}{\partial y} u_x$$

This consistency condition results in differentiating already existing constraints $u_x = v w$ and $u_y = v z$:

$$\frac{\partial}{\partial x} (v w) = \frac{\partial}{\partial y} (v z)$$

Performing product rule, we obtain:

$$v_x w + v w_x = v_y z + v z_y.$$

Thus, the complete system is:

$$\left\{ \begin{array}{l} \frac{u-f}{\lambda} = w_x + z_y, \\ u_x = v w, \\ u_y = v z, \\ v^2 = \epsilon^2 + u_x^2 + u_y^2, \\ v_x w + v w_x = v_y z + v z_y, \\ \partial_x u = u_x, \\ \partial_y u = u_y, \\ \partial_x w = w_x, \\ \partial_y z = z_y, \\ \partial_x v = v_x, \\ \partial_y v = v_y, \\ \partial_x w = w_x, \\ \partial_y w = w_y. \end{array} \right.$$

The technique used in deriving this system is called *jet prolongation*. That is, the original coordinates x, y, u were prolonged by adding certain jet coordinates $u_x, u_y, u_{xx}, u_{xy}, u_{yy}, \dots$, in order to build a constraint manifold and first-order differential equations on that manifold, to faithfully represent the original problem.

The number of jet coordinates is 8 and there are 5 algebraic constraints, thus reducing the dimension of the constraint manifold to 3. The two differential operators ∂_x, ∂_y give rise to 2 *vector fields*, say, X and Y , on the constraint manifold. These vector fields *commute*: $\partial_x \partial_y = \partial_y \partial_x$. Following those vector fields and their linear combinations gives rise to a 2-dimensional submanifold of the constraint manifold. Projecting on the 0-th jet subspace x, y, z yields the graph of $u = u(x, y)$.

We shall now elaborate on the way the vector fields X and Y are constructed. They must be tangent to the constraint manifold and project onto ∂_x and ∂_y , respectively. For every constraint, we define differential forms ω_j , $j = 1, 2, 3, 4$, which are 0 on the tangent space to the constraint manifold:

$$\begin{aligned} \omega_1 &= d\left(\frac{u-f}{\lambda} - w_x - z_y\right) = \frac{u_x dx + y_y dy - f_x dx - f_y dy}{\lambda} - dw_x - dz_y, \\ \omega_2 &= d(u_x - v w) = du_x - w dv - v dw, \\ \omega_3 &= d(u_y - v z) = du_y - z dv - v dz, \\ \omega_4 &= d(v^2 - \epsilon^2 - u_x^2 - u_y^2) = 2v dv - 2u_x du_x - 2u_y du_y, \\ \omega_5 &= d(v_x w + v w_x - v_y z - v z_y) = w dv_x - v_x dw - w_x dv - v dw_x - z dv_y - v_y dz - z_y dv - v dz_y. \end{aligned}$$

In general,

$$X = X_x \partial_x + X_u \partial_u + X_{u_x} \partial_{u_x} + X_{u_y} \partial_{u_y} + \dots + X_{z_y} \partial_{z_y}.$$

Equations $\omega_i(X) = 0$ must hold. This allows us to find the coefficients $X_x, X_u, X_{u_x}, X_{u_y}, \dots$

Using any integration scheme for ODE, we can generate solutions in the x and y direction, but we must be mindful of the fact that the truncation error will result in divergence from satisfying the algebraic constraints. To avoid this problem, we need to augment every integration step with a projection step, to keep the constraints satisfied. If the domain in the xy -plane is simple, it is not hard to satisfy the boundary conditions. For instance, if the domain is a rectangle, integrating along x and y directions results in a boundary value problem on an interval.

Remark S.1 (On difficulty of prolongation). It is a *lucky accident* that prolongation for the TV PDE can be performed with ease. General algebraic techniques exists (a subject of *non-commutative algebra*) which allow performing prolongation in a systematic way. Typically a low-order PDE may require significantly higher-order jets to represent the original problem. $\square$

Appendix T

An Analysis of Kahan's Algorithm

Kahan's algorithm is listed as Algorithm 2.1.

Lemma T.1 (Exact telescoping identity for Kahan). *Run Kahan's compensated summation on $x_1, \dots, x_n$ with $s_0 = c_0 = 0$:*

$$y_k = \text{fl}(x_k - c_{k-1}), \quad t_k = \text{fl}(s_{k-1} + y_k), \quad c_k = \text{fl}((t_k - s_{k-1}) - y_k), \quad s_k = t_k.$$

Write the local rounding errors in the usual model $\text{fl}(a \circ b) = (a \circ b)(1 + \delta)$, $|\delta| \leq u$, as

$$\begin{aligned} y_k &= (x_k - c_{k-1}) + \varepsilon_k^{(1)}, & |\varepsilon_k^{(1)}| &\leq u(|x_k| + |c_{k-1}|), \\ t_k &= (s_{k-1} + y_k) + \varepsilon_k^{(2)}, & |\varepsilon_k^{(2)}| &\leq u(|s_{k-1}| + |y_k|), \\ c_k &= ((t_k - s_{k-1}) - y_k) + \varepsilon_k^{(3)}, & |\varepsilon_k^{(3)}| &\leq u|\varepsilon_k^{(2)}|. \end{aligned}$$

Then with $S = \sum_{i=1}^n x_i$ the final sum s_n satisfies the exact identity

$$s_n - S = \varepsilon_n^{(2)} + \sum_{k=1}^n \varepsilon_k^{(1)} - 2 \sum_{k=1}^{n-1} \varepsilon_k^{(3)}. \quad (\text{T.1})$$

□

Proof. Let $a_k := s_k + c_k$. Since $c_k = ((t_k - s_{k-1}) - y_k) + \varepsilon_k^{(3)} = \varepsilon_k^{(2)} + \varepsilon_k^{(3)}$ and $s_k = t_k$,

$$a_k = s_{k-1} + y_k + 2\varepsilon_k^{(2)} + \varepsilon_k^{(3)}.$$

Hence

$$a_k - a_{k-1} = (y_k - c_{k-1}) + 2\varepsilon_k^{(2)} + \varepsilon_k^{(3)} = x_k + \varepsilon_k^{(1)} + 2(\varepsilon_k^{(2)} - \varepsilon_{k-1}^{(2)}) + (\varepsilon_k^{(3)} - 2\varepsilon_{k-1}^{(3)}).$$

Summing $k = 1$ to n (with $\varepsilon_0^{(2)} = \varepsilon_0^{(3)} = 0$) and using $a_0 = 0$, $a_n = s_n + c_n$, $c_n = \varepsilon_n^{(2)} + \varepsilon_n^{(3)}$ yields (T.1). □

Theorem T.1 (Kahan/Compensated Summation: explicit bound). Let u be the unit roundoff, and define

$$\gamma_m := \frac{m u}{1 - m u} \quad (m \geq 1), \quad \text{and} \quad \gamma := \gamma_1 = \frac{u}{1 - u}.$$

Assume $nu < 1$ (no under/overflow, round-to-nearest model). Let $\hat{s} = s_n$ be Kahan's result on $x_1, \dots, x_n$, and $S = \sum_{i=1}^n x_i$, and set $A := \sum_{i=1}^n |x_i|$. Then

$$|\hat{s} - S| \leq (2\gamma + \gamma^2)A + \frac{\gamma(5 + 3\gamma + \gamma^2)(1 + \gamma)\gamma_n}{1 - \gamma_n(1 + \gamma)^2} A. \quad (\text{T.2})$$

In particular, the right-hand side is finite as long as $\gamma_n(1 + \gamma)^2 < 1$ (which holds whenever nu is modest). □

Proof. From Lemma T.1,

$$E := |\hat{s} - S| \leq |\varepsilon_n^{(2)}| + \sum_{k=1}^n |\varepsilon_k^{(1)}| + 2 \sum_{k=1}^{n-1} |\varepsilon_k^{(3)}|.$$

Introduce the shorthand

$$E_1 := \sum_{k=1}^n |\varepsilon_k^{(1)}|, \quad E_2 := \sum_{k=1}^n |\varepsilon_k^{(2)}|, \quad E_3 := \sum_{k=1}^n |\varepsilon_k^{(3)}|, \quad Y := \sum_{k=1}^n |y_k|.$$

Single-operation bounds give

$$|\varepsilon_k^{(1)}| \leq \gamma(|x_k| + |c_{k-1}|), \quad |\varepsilon_k^{(2)}| \leq \gamma(|s_{k-1}| + |y_k|), \quad |\varepsilon_k^{(3)}| \leq \gamma|\varepsilon_k^{(2)}|.$$

Hence $E_3 \leq \gamma E_2$ and

$$E_1 \leq \gamma A + \gamma \sum_{k=1}^n |c_{k-1}| \leq \gamma A + \gamma \sum_{k=1}^n (|\varepsilon_{k-1}^{(2)}| + |\varepsilon_{k-1}^{(3)}|) \leq \gamma A + \gamma(E_2 + E_3) \leq \gamma A + \gamma(1 + \gamma)E_2. \quad (1)$$

Also $|\varepsilon_n^{(2)}| \leq \gamma(|s_{n-1}| + |y_n|) \leq \gamma(Y + E_2)$, because $|s_{n-1}| \leq \sum_{i < n} |y_i| + \sum_{i < n} |\varepsilon_i^{(2)}| \leq Y + E_2$. Next,

$$Y = \sum_{k=1}^n |x_k - c_{k-1} + \varepsilon_k^{(1)}| \leq A + \sum_{k=1}^n |c_{k-1}| + E_1 \leq A + E_2 + E_3 + E_1 \leq (1 + \gamma)A + (1 + \gamma)E_2 + E_1. \quad (2)$$

For the addition-error pileup we use a standard “geometric” bound:

$$E_2 = \sum_{k=1}^n |\varepsilon_k^{(2)}| \leq \gamma \sum_{k=1}^n (|s_{k-1}| + |y_k|) \leq \gamma(nY + (n-1)E_2),$$

since $|s_{k-1}| \leq \sum_{i < k} |y_i| + \sum_{i < k} |\varepsilon_i^{(2)}|$. Rearranging,

$$(1 - \gamma(n-1))E_2 \leq \gamma nY \quad \Rightarrow \quad E_2 \leq \gamma_n Y, \quad \gamma_n = \frac{nu}{1 - nu}. \quad (3)$$

Now assemble the pieces. Using (1)–(3) and $E_3 \leq \gamma E_2$,

$$\begin{aligned} E &\leq \gamma(Y + E_2) + E_1 + 2E_3 \\ &\leq \gamma((1 + \gamma)A + (1 + \gamma)E_2 + E_1) + \gamma E_2 + E_1 + 2\gamma E_2 \\ &= \underbrace{(2\gamma + \gamma^2)A}_{\text{A-term}} + \underbrace{\gamma(2 + 2\gamma + \gamma^2)E_2}_{\text{E}_2\text{-term}} + \underbrace{(1 + \gamma)E_1}_{\text{E}_1\text{-term}}. \end{aligned}$$

Insert (1) into the last term and collect the coefficients of E_2 :

$$E \leq (2\gamma + \gamma^2)A + \gamma(5 + 3\gamma + \gamma^2)E_2.$$

Finally, apply (3) and (2):

$$E_2 \leq \gamma_n Y \leq \gamma_n((1 + \gamma)A + (1 + \gamma)^2 E_2),$$

so that

$$(1 - \gamma_n(1 + \gamma)^2)E_2 \leq \gamma_n(1 + \gamma)A \quad \Rightarrow \quad E_2 \leq \frac{\gamma_n(1 + \gamma)}{1 - \gamma_n(1 + \gamma)^2} A.$$

Substitute this into the previous bound for E to obtain (T.2). □

Corollary T.1 (Simplified constants). If $u \leq \frac{1}{4}$ and $nu \leq \frac{1}{10}$, then

$$|\hat{s} - S| \leq (2u + (6n + 4)u^2) \sum_{i=1}^n |x_i|$$

□

Appendix U

Conditional Expectation and Related Concepts

U.1 What Can We Condition On?

In elementary probability, conditioning is introduced via events:

$$P(A | B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) > 0.$$

However, this is only the beginning. In modern probability, we may condition on:

- events (B),
- values of a random variable ($X = x$),
- a σ -algebra ($\mathcal{F}$),
- another random variable (Y) (equivalently, $\sigma(Y)$).

The correct unifying object is *conditional expectation*.

U.2 Conditional Expectation with Respect to a σ -Algebra

Let $(\Omega, \mathcal{A}, P)$ be a probability space and $X \in L^1(\Omega)$. Let $\mathcal{F} \subset \mathcal{A}$ be a sub- σ -algebra.

Definition 7 (Conditional Expectation). The conditional expectation of X given $\mathcal{F}$, denoted

$$\mathbb{E}(X | \mathcal{F}),$$

is any $\mathcal{F}$ -measurable random variable satisfying

$$\int_F \mathbb{E}(X | \mathcal{F}) dP = \int_F X dP \quad \text{for all } F \in \mathcal{F}.$$

□

Interpretation.

$\mathbb{E}(X | \mathcal{F})$ is the “best approximation” of X using only the information contained in $\mathcal{F}$.

U.3 Finite Partition Case

If $\mathcal{F} = \sigma\{B_1, \dots, B_k\}$ where B_i is a partition of Ω , then

$$\mathbb{E}(X \mid \mathcal{F}) = \sum_{i=1}^k \left(\frac{1}{P(B_i)} \int_{B_i} X(\omega) dP(\omega) \right) \mathbf{1}_{B_i}.$$

Equivalently, for $\omega \in B_i$,

$$\mathbb{E}(X \mid \mathcal{F})(\omega) = \frac{1}{P(B_i)} \int_{B_i} X(\omega) dP(\omega).$$

Thus, the conditional expectation is piecewise constant on the atoms of $\mathcal{F}$.

U.4 General Case via Approximation

If $\mathcal{F}$ is not generated by a finite partition, we approximate it by a sequence of finite σ -algebras:

$$\mathcal{F}_n = \sigma\{B_1^{(n)}, \dots, B_{m_n}^{(n)}\}, \quad \mathcal{F}_n \uparrow \mathcal{F}.$$

Then

$$\mathbb{E}(X \mid \mathcal{F}) = \lim_{n \rightarrow \infty} \mathbb{E}(X \mid \mathcal{F}_n),$$

where the limit holds in L^1 (and almost surely under mild conditions).

This is closely related to martingale convergence (Doob's theorem).

U.5 Conditioning on a Random Variable

If Y is a random variable, we define

$$\mathbb{E}(X \mid Y) := \mathbb{E}(X \mid \sigma(Y)).$$

Thus, $\mathbb{E}(X \mid Y)$ is a function of Y , i.e.,

$$\mathbb{E}(X \mid Y)(\omega) = g(Y(\omega))$$

for some measurable function (g).

U.6 Density-Based Formulas

If (X, Y) admit a joint density $f_{X,Y}(x, y)$, then

$$f_{X|Y}(x \mid y) = \frac{f_{X,Y}(x, y)}{f_Y(y)}, \quad f_Y(y) = \int f_{X,Y}(x, y) dx.$$

The conditional expectation becomes

$$\mathbb{E}(X \mid Y = y) = \int x f_{X|Y}(x \mid y) dx.$$

This agrees with the abstract definition via $(\mathbb{E}(Y))$.

U.7 Conditional Probability as Conditional Expectation

A key identity is:

$$\mathbb{P}(A) = \mathbb{E}(\mathbf{1}_A),$$

and more generally,

$$\mathbb{P}(A \mid \mathcal{F}) = \mathbb{E}(\mathbf{1}_A \mid \mathcal{F}).$$

Thus, conditional probability is a special case of conditional expectation.

Similarly,

$$\mathbb{P}(A \mid Y) = \mathbb{E}(\mathbf{1}_A \mid Y).$$

U.8 Key Properties

For integrable random variables X, Z and $\mathcal{F}$ -measurable Z :

- Linearity:

$$\mathbb{E}(aX + bZ \mid \mathcal{F}) = a\mathbb{E}(X \mid \mathcal{F}) + bZ.$$

- Tower property:

$$\mathbb{E}(\mathbb{E}(X \mid \mathcal{F}) \mid \mathcal{G}) = \mathbb{E}(X \mid \mathcal{G}), \quad \mathcal{G} \subset \mathcal{F}.$$

- Taking out what is known:

$$\mathbb{E}(ZX \mid \mathcal{F}) = Z, \mathbb{E}(X \mid \mathcal{F}).$$

- Law of total expectation:

$$\mathbb{E}(X) = \mathbb{E}(\mathbb{E}(X \mid \mathcal{F})).$$

U.9 Hilbert Space Viewpoint (Optional but Insightful)

If $X \in L^2(\Omega)$, then $\mathbb{E}(X \mid \mathcal{F})$ is the orthogonal projection of X onto the closed subspace $L^2(\mathcal{F})$.

Thus,

$$\mathbb{E}((X - \mathbb{E}(X \mid \mathcal{F}))Z) = 0 \quad \text{for all } Z \in L^2(\mathcal{F}).$$

U.10 Connection to Markov Processes

Let (X_t) be a Markov chain. Then

$$\mathbb{P}(X_t = i_t \mid X_{t-1}, \dots, X_0) = P(X_t = i_t \mid X_{t-1}).$$

Hence,

$$\mathbb{P}(X_0 = i_0, \dots, X_t = i_t) = \pi_{i_0} \prod_{s=1}^t p_{i_{s-1}, i_s},$$

where π is the initial distribution and $P = (p_{ij})$ is the transition matrix.

This is a structured example of conditioning with respect to increasing σ -algebras.

U.11 Path Space and Shift Map

The natural probability space for a Markov chain is

$$\Omega = (i_0, i_1, i_2, \dots),$$

the space of infinite paths.

Define coordinate projections

$$\pi_j(\omega) = i_j,$$

and the shift map

$$S((i_0, i_1, i_2, \dots)) = (i_1, i_2, i_3, \dots).$$

Cylinder sets of the form

$$\{\omega : i_0 = a_0, \dots, i_k = a_k\}$$

generate the natural σ -algebra.

This viewpoint connects probability with dynamical systems and ergodic theory.

U.12 Closing Remark

Conditional expectation is the central organizing principle of probability. It unifies:

- conditional probability,
- regression,
- filtering,
- Markov processes,
- martingales.

If probability theory were a city, conditional expectation would be its transit system: everything routes through it.

U.13 Worked Examples

Example U.1 (Conditioning on a Finite Partition). Let $\Omega = \{1, 2, 3, 4\}$ with uniform probability, and define

$$X(\omega) = \omega.$$

Let $\mathcal{F} = \sigma(1, 2, 3, 4)$.

Then

$$\mathbb{E}(X \mid \mathcal{F})(\omega) = \begin{cases} \frac{1+2}{2} = \frac{3}{2}, & \omega \in \{1, 2\}, \\ \frac{3+4}{2} = \frac{7}{2}, & \omega \in \{3, 4\}. \end{cases}$$

□

Interpretation: We replace X by its average on each atom of the partition. The function becomes “blocky” — a coarse-grained version of X .

Example U.2 (Conditioning on a Random Variable). Let (X, Y) have joint density

$$f_{X,Y}(x, y) = 2, \quad 0 < x < y < 1.$$

First compute the marginal:

$$f_Y(y) = \int_0^y 2 \, dx = 2y.$$

Thus,

$$f_{X|Y}(x|y) = \frac{2}{2y} = \frac{1}{y}, \quad 0 < x < y.$$

Then

$$\mathbb{E}(X | Y = y) = \int_0^y x \frac{1}{y} dx = \frac{1}{y} \cdot \frac{y^2}{2} = \frac{y}{2}.$$

Conclusion:

$$\mathbb{E}(X | Y) = \frac{Y}{2}.$$

□

Interpretation Given $(Y=y)$, (X) is uniform on $((0,y))$.

Example U.3 (Conditional Probability via Expectation). Let $A \subset \Omega$. Then

$$\mathbb{P}(A | \mathcal{F}) = \mathbb{E}(\mathbf{1}_A | \mathcal{F}).$$

In Example (U.1), take $A = \{1, 3\}$. Then

$$\mathbb{P}(A | \mathcal{F})(\omega) = \begin{cases} \frac{1}{2}, & \omega \in 1, 2, \\ \frac{1}{4}, & \omega \in 3, 4. \end{cases}$$

□

Interpretation Conditional probability is just averaging the indicator.

Example U.4 (Tower Property in Action). Let $\mathcal{G} \subset \mathcal{F}$. Then

$$\mathbb{E}(\mathbb{E}(X | \mathcal{F}) | \mathcal{G}) = \mathbb{E}(X | \mathcal{G}).$$

Concrete instance Refine the partition

$$\{1, 2, 3, 4\} \rightarrow \{1, 2\}, \{3, 4\} \rightarrow \{1\}, \{2\}, \{3, 4\}.$$

Conditioning twice collapses to conditioning once on the coarser information. □

Moral Conditioning is a projection; projecting twice does nothing new.

Example U.5 (Markov Chain Factorization). Let X_t be a Markov chain with initial distribution π and transition matrix P .

Then

$$\mathbb{P}(X_0 = i_0, \dots, X_t = i_t) = \pi_{i_0} \prod_{s=1}^t p_{i_{s-1}, i_s}.$$

□

Interpretation The entire joint law is built from repeated conditioning:

$$\mathbb{P}(X_s | X_{s-1}, \dots, X_0) = \mathbb{P}(X_s | X_{s-1}).$$

The past collapses to the present—memory gets compressed into one state.

U.14 Exercises

Exercise U.1 (Finite Partition). Let $\Omega = \{1, 2, 3, 4, 5, 6\}$ with uniform probability, and $X(\omega) = \omega^2$. Let

$$\mathcal{F} = \sigma(\{1, 2, 3\}, \{4, 5, 6\}).$$

Compute $\mathbb{E}(X \mid \mathcal{F})$. $\square$

Exercise U.2 (Indicator Trick). Let $A, B \in \mathcal{A}$ with $\mathbb{P}(B) > 0$. Show that

$$\mathbb{E}(\mathbf{1}_A \mid \sigma(B)) = \mathbb{P}(A \mid B)\mathbf{1}_B + \mathbb{P}(A \mid B^c)\mathbf{1}_{B^c}.$$

$\square$

Exercise U.3 (Conditioning on a Function). Let $X \sim \text{Uniform}(-1, 1)$ and $Y = X^2$. Compute $\mathbb{E}(X \mid Y)$. *Hint: symmetry is your ally.* $\square$

Exercise U.4 (Tower Property). Let $\mathcal{G} \subset \mathcal{F}$. Prove the tower property:

$$\mathbb{E}(\mathbb{E}(X \mid \mathcal{F}) \mid \mathcal{G}) = \mathbb{E}(X \mid \mathcal{G}).$$

$\square$

Exercise U.5 (Best Approximation). Let $X \in L^2$. Show that $\mathbb{E}(X \mid \mathcal{F})$ minimizes

$$\mathbb{E}[(X - Z)^2]$$

over all $\mathcal{F}$ -measurable Z .

$\square$

Exercise U.6 (Markov Chain). Let (X_t) be a Markov chain with transition matrix P . Show that

$$\mathbb{E}(f(X_{t+1}) \mid X_t) = \sum_j f(j)p_{X_t, j}.$$

$\square$

Exercise U.7 (Law of Total Expectation). Let Y be discrete. Show that

$$\mathbb{E}(X) = \sum_y \mathbb{E}(X \mid Y = y)P(Y = y).$$

$\square$

Exercise U.8 (Reconstruction). Let $\mathcal{F} = \sigma(Y)$. Show that there exists a measurable function g such that

$$\mathbb{E}(X \mid \mathcal{F}) = g(Y).$$

Interpretation Conditioning on Y produces a function of Y . $\square$

Appendix V

Answers and Hints For Selected Exercises

Many of the exercises are solved with few digits (typically, 1–3) of precision. The instructor may request more digits, so that the student is required to implement the solutions in code.

Solution To Exercise 2.18: Let u denote the unit roundoff. We use the standard model for each floating-point operation $\text{fl}(\circ)$: $\text{fl}(x \circ y) = (x \circ y)(1 + \delta)$ with $|\delta| \leq u$, and assume no overflow/underflow.

(1) A 2×2 determinant. For $D = uz - vw$ we compute

$$\widehat{uz} = (uz)(1 + \delta_1), \quad \widehat{vw} = (vw)(1 + \delta_2), \quad \widehat{D} = \text{fl}(\widehat{uz} - \widehat{vw}) = ((uz)(1 + \delta_1) - (vw)(1 + \delta_2))(1 + \delta_3).$$

Rearrange as

$$\widehat{D} = (u(1 + \eta_{11})z(1 + \eta_{12}) - v(1 + \eta_{21})w(1 + \eta_{22}))(1 + \delta_3),$$

with all $\eta_{ij}, \delta_3 = O(u)$. Thus $\widehat{D} = \det\left(\begin{bmatrix} u & v \\ w & z \end{bmatrix} + \Delta\right)$ for some Δ whose entries satisfy $|\Delta| \leq cu\|u \ v; w \ z\|$. Hence a 2×2 determinant is backward stable.

(2) *Numerator and denominator.* Apply (1) to each determinant appearing in Cramer's rule: there exist perturbations ΔA and ΔA_i with $\|\Delta A\|/\|A\|, \|\Delta A_i\|/\|A_i\| = O(u)$ such that

$$\widehat{\det A} = \det(A + \Delta A), \quad \widehat{\det A_i} = \det(A_i + \Delta A_i).$$

(3) *Division.* Floating-point division satisfies $\text{fl}(p/q) = (p/q)(1 + \delta)$, $|\delta| \leq u$. Therefore the computed $\hat{x}_i = \widehat{\det A_i} / \widehat{\det A}$ equals the exact quotient of slightly perturbed determinants.

(4) *Backward stability.* Because the map $A \mapsto \det A$ is multilinear in the columns, the perturbations in numerator and denominator correspond to $O(u)$ relative perturbations in the columns of A and b . One can bundle these into a single matrix perturbation ΔA with $\|\Delta A\|/\|A\| = O(u)$ such that the computed $\hat{x}$ satisfies $(A + \Delta A)\hat{x} = b$. (A direct construction uses that Cramer's rule expresses each x_i as a ratio of cofactors; perturbing those cofactors by $O(u)$ corresponds to $O(u)$ perturbations of the underlying columns.)

(5) *Remark.* Backward stability does not guarantee small *forward* error when A is ill-conditioned: $\|\hat{x} - x\|/\|x\| \leq \kappa(A)O(u)$. The argument also does not scale: for $n \times n$, Cramer's rule requires $O(n!)$ work and suffers from severe cancellation in large determinants, whereas Gaussian elimination with partial pivoting is $O(n^3)$ and widely backward stable in practice.

Solution To Exercise 3.23:

(1) Use continuity and monotonicity (2) $A = -2b - 2a + 4$, $B = b^2 + 4ab - 6b + a^2 - 6a$, $C = -2ab^2 + 2b^2 - 2a^2b + 8ab + 2a^2$, $D = a^2b^2 - 2ab^2 + b^2 - 2a^2b - 2ab + a^2$. (3) Very complicated, have a CAS or AI write it out.

Solution To Exercise 4.35: To reduce the matrix

$$A = \begin{pmatrix} 4 & 1 & 2 & 3 \\ 1 & 3 & 4 & 2 \\ 2 & 4 & 5 & 1 \\ 3 & 2 & 1 & 2 \end{pmatrix}$$

to tridiagonal form using Householder transformations, we perform the following steps.

Step 1: Apply H_1 to Zero Out $A_{2,1} = 1$, $A_{3,1} = 2$, and $A_{4,1} = 3$

1. Define vector x :

$$x = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}.$$

2. Calculate $\|x\|$:

$$\|x\| = \sqrt{1^2 + 2^2 + 3^2} = \sqrt{14} \approx 3.7.$$

3. Construct v :

$$v = x - \|x\|e_1 = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} - \begin{pmatrix} 3.7 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} -2.7 \\ 2 \\ 3 \end{pmatrix}.$$

4. Normalize v :

$$\|v\| = \sqrt{(-2.7)^2 + 2^2 + 3^2} \approx 4.5,$$

$$u = \frac{v}{\|v\|} = \frac{1}{4.5} \begin{pmatrix} -2.7 \\ 2 \\ 3 \end{pmatrix} \approx \begin{pmatrix} -0.6 \\ 0.44 \\ 0.67 \end{pmatrix}.$$

5. Construct H_1 :

$$H_1 = I - 2uu^T \approx I - 2 \begin{pmatrix} -0.6 \\ 0.44 \\ 0.67 \end{pmatrix} \begin{pmatrix} -0.6 & 0.44 & 0.67 \end{pmatrix}.$$

Applying H_1 to A results in an intermediate matrix $A^{(1)}$:

$$A^{(1)} \approx \begin{pmatrix} 4.0 & -3.2 & 0 & 0 \\ -3.2 & 3.0 & 1.8 & 0 \\ 0 & 1.8 & 5.6 & 1.2 \\ 0 & 0 & 1.2 & 1.6 \end{pmatrix}.$$

Step 2: Apply H_2 to Zero Out $A_{3,2} = 1.8$

1. Define vector x :

$$x = \begin{pmatrix} 1.8 \\ 1.2 \end{pmatrix}.$$

2. Calculate $\|x\|$:

$$\|x\| = \sqrt{1.8^2 + 1.2^2} \approx 2.2.$$

3. Construct v :

$$v = x - \|x\|e_1 = \begin{pmatrix} 1.8 \\ 1.2 \end{pmatrix} - \begin{pmatrix} 2.2 \\ 0 \end{pmatrix} = \begin{pmatrix} -0.4 \\ 1.2 \end{pmatrix}.$$

4. Normalize v :

$$\|v\| = \sqrt{(-0.4)^2 + 1.2^2} \approx 1.3,$$

$$u = \frac{v}{\|v\|} = \frac{1}{1.3} \begin{pmatrix} -0.4 \\ 1.2 \end{pmatrix} \approx \begin{pmatrix} -0.31 \\ 0.92 \end{pmatrix}.$$

5. Construct H_2 :

$$H_2 = I - 2uu^\top \approx I - 2 \begin{pmatrix} -0.31 \\ 0.92 \end{pmatrix} \begin{pmatrix} -0.31 & 0.92 \end{pmatrix}.$$

Applying H_2 to $A^{(1)}$ yields the tridiagonal matrix $A^{(2)}$:

$$A^{(2)} \approx \begin{pmatrix} 4.0 & -3.2 & 0 & 0 \\ -3.2 & 3.0 & 2.1 & 0 \\ 0 & 2.1 & 5.6 & -0.8 \\ 0 & 0 & -0.8 & 1.6 \end{pmatrix}.$$

Final Result

The tridiagonal form of A , with numerical values approximated to two significant digits, is:

$$A^{(2)} \approx \begin{pmatrix} 4.0 & -3.2 & 0 & 0 \\ -3.2 & 3.0 & 2.1 & 0 \\ 0 & 2.1 & 5.6 & -0.8 \\ 0 & 0 & -0.8 & 1.6 \end{pmatrix}.$$

Solution To Exercise 4.36: To reduce the matrix

$$A = \begin{pmatrix} 4 & 3 & 2 & 0 \\ 3 & 5 & 1 & 2 \\ 2 & 1 & 6 & 4 \\ 0 & 2 & 4 & 3 \end{pmatrix}$$

to tridiagonal form using Givens rotations, we will zero out the subdiagonal elements $A_{3,1}$ and $A_{4,2}$ step by step.

Step 1: Zero Out $A_{3,1} = 2$

To zero out the element $A_{3,1} = 2$, we apply a Givens rotation to the first and third rows. We calculate the rotation parameters c and s as follows:

$$c = \frac{4}{\sqrt{4^2 + 2^2}} = \frac{4}{\sqrt{20}} = \frac{2}{\sqrt{5}}, \quad s = \frac{2}{\sqrt{20}} = \frac{1}{\sqrt{5}}.$$

The Givens rotation matrix $G_{13}(\theta_1)$ is:

$$G_{13}(\theta_1) = \begin{pmatrix} \frac{2}{\sqrt{5}} & 0 & \frac{1}{\sqrt{5}} & 0 \\ 0 & 1 & 0 & 0 \\ -\frac{1}{\sqrt{5}} & 0 & \frac{2}{\sqrt{5}} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Applying G_{13} to A from both sides, we obtain:

$$A^{(1)} = G_{13}^\top A G_{13} = \begin{pmatrix} 4.4 & 3 & 0 & 0 \\ 3 & 5 & 1.6 & 2 \\ 0 & 1.6 & 5.6 & 4 \\ 0 & 2 & 4 & 3 \end{pmatrix}.$$

Step 2: Zero Out $A_{4,2} = 2$

Next, we zero out the element $A_{4,2} = 2$ by applying another Givens rotation $G_{24}(\theta_2)$ to the second and fourth rows. The rotation parameters are:

$$c = \frac{5}{\sqrt{5^2 + 2^2}} = \frac{5}{\sqrt{29}}, \quad s = \frac{2}{\sqrt{29}}.$$

The Givens rotation matrix $G_{24}(\theta_2)$ is:

$$G_{24}(\theta_2) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \frac{5}{\sqrt{29}} & 0 & \frac{2}{\sqrt{29}} \\ 0 & 0 & 1 & 0 \\ 0 & -\frac{2}{\sqrt{29}} & 0 & \frac{5}{\sqrt{29}} \end{pmatrix}.$$

Applying G_{24} to $A^{(1)}$, we obtain:

$$A^{(2)} = G_{24}^\top A^{(1)} G_{24} = \begin{pmatrix} 4.4 & 3.47 & 0 & 0 \\ 3.47 & 5 & 1.6 & 0 \\ 0 & 1.6 & 5.6 & 4 \\ 0 & 0 & 4 & 3 \end{pmatrix}.$$

Result

After these two Givens rotations, the matrix A is in tridiagonal form:

$$A^{(2)} = \begin{pmatrix} 4.4 & 3.47 & 0 & 0 \\ 3.47 & 5 & 1.6 & 0 \\ 0 & 1.6 & 5.6 & 4 \\ 0 & 0 & 4 & 3 \end{pmatrix}.$$

Solution To Exercise 6.29: The general Gradient Descent algorithm with backtracking can be implemented as follows:

```

1  import numpy as np
2  def backtracking_gradient_descent(x0, alpha0, f, df, strategy, \
3                                  c1=0.01, c2=0.9, beta=0.5, max_iter=128):
4
5      """
6      Minimize a function using Gradient Descent algorithm with backtracking.
7      Three methods are implemented: Armijo rule (sufficient descent),
8      Wolfe condition (sufficient curvature), strong Wolfe condition and Goldstein.
9
10     Parameters:
11         x0 (array-like):    Initial guess.
12         alpha0 (float):     Initial learning rate.
13         f (callable):       Objective function.
14         df (callable):      Gradient of the objective function.
15         strategy (string):  The name of the strategy: "armijo", "wolfe",
16                             "strong_wolfe" or "goldstein".
17         c1 (float):         The c1 constant for Armijo or Goldstein condition.
18         c2 (float):         The c2 constant for Wolfe or Goldstein condition.
19         beta (float):       The factor by which to reduce alpha.
20         max_iter (int):     Maximum number of iterations.

```

```

21 Returns:
22     x (array-like): Estimated position of the minimum.
23     alpha (float): The final learning rate.
24     iter (int) : The number of iterations upon exit.
25 """
26 x = x0
27 alpha = alpha0
28 for iter in range(max_iter): # Maximum number of iterations
29     grad = df(x)
30     d = -grad # Descent direction is the negative gradient
31     while True:
32         x_new = x + alpha*d
33         lhs = f(x_new)
34         rhs = f(x) + c1*alpha*np.dot(grad, d)
35         # Check conditions based on the strategy
36         if strategy == "armijo" and lhs <= rhs:
37             break
38         elif strategy == "wolfe" and lhs <= rhs and \
39             np.dot(df(x_new), d) >= c2*np.dot(grad, d):
40             break
41         elif strategy == "strong_wolfe" and lhs <= rhs \
42             and abs(np.dot(df(x_new), d)) <= c2*abs(np.dot(grad, d)):
43             break
44         elif strategy == "goldstein" and \
45             lhs >= f(x) + c1*alpha*np.dot(grad, d) and \
46             lhs <= f(x) + c2*alpha*np.dot(grad, d):
47             break
48         else:
49             alpha *= beta # Reduce alpha
50             if alpha < 1e-10: # Avoid excessively small alpha
51                 break
52     x = x_new
53     return x, alpha, iter # Return final position and alpha
54
55 def f(x):
56     return np.log(1+x**2)
57 def df(x):
58     return 2*x/(1+x**2)
59
60 x0 = 2
61 alpha0 = 1 # Initial step size
62 max_iter = 64 # Iterations
63 # List of strategies
64 strategies = ["armijo", "wolfe", "strong_wolfe", "goldstein"]
65
66 # Perform backtracking for each strategy and print the results
67 results = {strategy: backtracking_gradient_descent(x0, alpha0, f, df, \
68     strategy, max_iter=max_iter) for strategy in strategies}

```


Conditions	x	α	Number of iterations
Armijo	$-9.45964358487743e - 10$	$5.293955920339377e - 23$	63
Wolfe	1.9999999998137357	$6.310887241768095e - 30$	63
Strong Wolfe	1.9999999998137357	$6.310887241768095e - 30$	63
Goldstein	1.9999999998137357	$1.1102230246251565e - 16$	63

As we can see, only Armijo condition leads to progress towards the obvious minimum $x^* = 0$. This is due to the fact that $f(x)$ is a concave function and the *sufficient curvature* condition is never satisfied. The algorithm stops when α drops below the set threshold. x ends up very close to $x_0 = 2$. To confirm concavity near x_0 , we find:

$$f''(x) = \frac{1}{1+x^2} - \frac{4x^2}{(1+x^2)^2} = \frac{2(x-1)(x+1)}{(1+x^2)^2}.$$

Clearly $f'(x) \geq 0$ only if $|x| \leq 1$ and $f''(x_0) = f''(2) = -\frac{6}{25}$.

It is evident that for $x_0 \in (-1, 1)$ the picture changes:

Comparison of Backtracking Strategies

```

1 x0 = 0.75
2 alpha0 = 1 # Initial step size
3 max_iter = 64 # Iterations
4 # List of strategies
5 strategies = ["armijo", "wolfe", "strong_wolfe", "goldstein"]
6
7 # Perform backtracking for each strategy and print the results
8 results = {strategy: backtracking_gradient_descent(x0, alpha0, f, df, \
9             strategy, max_iter=max_iter) for strategy in strategies}

```

For $x_0 = 0.75$ our table looks like this:

Conditions	x	α	Number of iterations
Armijo	$8.965209224032593e - 10$	$2.6469779601696886e - 23$	63
Wolfe	$8.965209224032593e - 10$	$2.6469779601696886e - 23$	63
Strong Wolfe	$-3.3976609080916536e - 19$	$1.0097419586828951e - 28$	63
Goldstein	0.7499999997764827	$2.7755575615628914e - 17$	63

It looks like, except for Goldstein, all methods are quite effective.

Solution To Exercise L.1: We write this somewhat general piece of code in Python, which solves the case of $k = n-1$ constraints. This requires only the determinant of the bordered Hessian to classify the critical points:

Classification of Critical Points using Bordered Hessian

```

1 import sympy as sp
2
3 # Define the variables
4 x1, x2, x3, lambda1, lambda2 = sp.symbols('x1 x2 x3 lambda1 lambda2')
5
6 # Define the Lagrange function as a symbolic expression
7 L = x1 + x2 + 2*x3 + lambda1 * (x1**2 + x2**2 + x3**2 - 1) + lambda2 * (x1 - x2)
8
9 # Extract variables from L
10 v = list(sp.ordered(L.free_symbols))
11
12 # Define the equations
13 equations = [sp.diff(L, vv) for vv in v]
14
15 # Solve the system of equations

```

```

16 solutions = sp.solve(equations, v, dict=True)
17
18 # Display the solutions
19 for sol in solutions:
20     print(sol)
21
22 # Define gradient
23 # gradient = lambda f, v: Matrix([f]).jacobian(v)
24
25 # Compute Hessian, its determinant and factor
26 HB5 = sp.hessian(L,v).det().factor()
27
28 # Display the signs of the determinant
29 for sol in solutions:
30     print(sp.sign(HB5.subs(sol)))

```

Python output for the determinant of H_B is, in $\text{\LaTeX}$:

$$8\lambda_1 (x_1^2 + 2x_1x_2 + x_2^2 + 2x_3^2)$$

The program also prints the signs of the determinant evaluated at the critical points.

Solution To Exercise 7.3: Here is the Python program which solves the cubic spline interpolation problem:

```

1 def compute_cubic_spline_coeffs(x, y):
2     """
3     Given lists of x and y values, compute the coefficients for the natural
4     cubic spline. Returns a list of tuples for each interval [x[i], x[i+1]]:
5     (a, b, c, d, x_start, x_end)
6     where the spline on the interval is:
7     S(x) = a + b*(x - x_start) + c*(x - x_start)**2 + d*(x - x_start)**3.
8     """
9     n = len(x)
10    # Compute interval lengths h[i] = x[i+1] - x[i]
11    h = [x[i+1] - x[i] for i in range(n-1)]
12
13    # Build the tridiagonal system for the c coefficients.
14    # The system is defined for the interior points i = 1, 2, ..., n-2.
15    # For each interior i, we have:
16    # h[i-1]*c[i-1] + 2*(h[i-1]+h[i])*c[i] + h[i]*c[i+1] =
17    # 3*((y[i+1]-y[i])/h[i] - (y[i]-y[i-1])/h[i-1])
18    # With natural boundary conditions: c[0] = c[n-1] = 0.
19
20    # These lists hold the lower, main, and upper diagonals and the RHS.
21    L = [] # lower diagonal (length n-2)
22    D = [] # main diagonal (length n-2)
23    U = [] # upper diagonal (length n-3)
24    B = [] # right-hand side (length n-2)
25
26    for i in range(1, n-1):
27        L.append(h[i-1])
28        D.append(2 * (h[i-1] + h[i]))

```

```

29     if i < n - 1 - 1:
30         U.append(h[i])
31     B.append(3 * ((y[i+1] - y[i]) / h[i] - (y[i] - y[i-1]) / h[i-1]))
32
33     # Solve the tridiagonal system using the Thomas algorithm.
34     # Forward elimination:
35     for i in range(1, len(D)):
36         factor = L[i-1] / D[i-1]
37         D[i] = D[i] - factor * U[i-1]
38         B[i] = B[i] - factor * B[i-1]
39
40     # Back substitution:
41     c_internal = [0] * (n - 2) # these are c[1] to c[n-2]
42     c_internal[-1] = B[-1] / D[-1]
43     for i in range(len(D) - 2, -1, -1):
44         c_internal[i] = (B[i] - U[i] * c_internal[i+1]) / D[i]
45
46     # Assemble full c coefficients (with natural boundary conditions)
47     c = [0] + c_internal + [0]
48
49     # Compute b and d coefficients for each interval.
50     b = [0] * (n - 1)
51     d = [0] * (n - 1)
52     a = y[:] # a[i] is simply y[i]
53     for i in range(n - 1):
54         b[i] = (y[i+1] - y[i]) / h[i] - h[i] * (2*c[i] + c[i+1]) / 3
55         d[i] = (c[i+1] - c[i]) / (3 * h[i])
56
57     # Pack the coefficients and the corresponding interval endpoints.
58     coeffs = []
59     for i in range(n - 1):
60         coeffs.append((a[i], b[i], c[i], d[i], x[i], x[i+1]))
61
62     return coeffs
63
64
65 def spline_eval(coeffs, x_val):
66     """
67     Evaluate the cubic spline at a given x value using the computed coefficients.
68     If x_val lies outside the provided intervals, it extrapolates using the first
69     or last interval.
70     """
71     # Find the interval that contains x_val.
72     for a, b, c, d, x_start, x_end in coeffs:
73         if x_start <= x_val <= x_end:
74             dx = x_val - x_start
75             return a + b * dx + c * dx**2 + d * dx**3
76
77     # If x_val is outside the range, extrapolate using the first or last interval.
78     if x_val < coeffs[0][4]:
79         a, b, c, d, x_start, _ = coeffs[0]
80         dx = x_val - x_start
81         return a + b * dx + c * dx**2 + d * dx**3

```

```

82     else:
83         a, b, c, d, x_start, _ = coeffs[-1]
84         dx = x_val - x_start
85         return a + b * dx + c * dx**2 + d * dx**3
86
87
88 def main():
89     # Example data points.
90     x = [0, 1, 2, 3]
91     y = [1, 2, 0, 2]
92
93     # Compute spline coefficients.
94     coeffs = compute_cubic_spline_coeffs(x, y)
95
96     # Display the computed coefficients for each interval.
97     print("Spline coefficients for each interval [x[i], x[i+1]]:")
98     for i, (a, b, c, d, x_start, x_end) in enumerate(coeffs):
99         print("Interval [{:.2f}, {:.2f}]: a = {:.4f}, b = {:.4f}, c = {:.4f}, d = {:.4f}"
100               .format(x_start, x_end, a, b, c, d))
101
102     # Evaluate the spline at several points between 0 and 3.
103     print(r"\nEvaluated spline values:")
104     num_points = 31 # for x values 0, 0.1, 0.2, ..., 3.0
105     step = (x[-1] - x[0]) / (num_points - 1)
106     for i in range(num_points):
107         x_val = x[0] + i * step
108         s_val = spline_eval(coeffs, x_val)
109         print("S({:.2f}) = {:.4f}".format(x_val, s_val))
110
111 if __name__ == "__main__":
112     main()

```

Here is the output:

```

Spline coefficients for each interval [x[i], x[i+1]]:
Interval [0.00, 1.00]: a = 1.0000, b = 2.0667, c = 0.0000, d = -1.0667
Interval [1.00, 2.00]: a = 2.0000, b = -1.1333, c = -3.2000, d = 2.3333
Interval [2.00, 3.00]: a = 0.0000, b = -0.5333, c = 3.8000, d = -1.2667
\nEvaluated spline values:
S(0.00) = 1.0000
S(0.10) = 1.2056
S(0.20) = 1.4048
S(0.30) = 1.5912
S(0.40) = 1.7584
S(0.50) = 1.9000
S(0.60) = 2.0096
S(0.70) = 2.0808
S(0.80) = 2.1072
S(0.90) = 2.0824
S(1.00) = 2.0000
S(1.10) = 1.8570
S(1.20) = 1.6640
S(1.30) = 1.4350

```

```

S(1.40) = 1.1840
S(1.50) = 0.9250
S(1.60) = 0.6720
S(1.70) = 0.4390
S(1.80) = 0.2400
S(1.90) = 0.0890
S(2.00) = 0.0000
S(2.10) = -0.0166
S(2.20) = 0.0352
S(2.30) = 0.1478
S(2.40) = 0.3136
S(2.50) = 0.5250
S(2.60) = 0.7744
S(2.70) = 1.0542
S(2.80) = 1.3568
S(2.90) = 1.6746
S(3.00) = 2.0000

```

Solution To Exercise 9.17:

————— Solution of Elastic Pendulum equation —————

```

1  import numpy as np
2  from scipy.integrate import solve_ivp
3  import matplotlib.pyplot as plt
4
5
6  def elastic_pendulum(t, Y, g, k, m, L0, epsilon):
7      r, theta, dr, dtheta = Y
8      d2r = r * dtheta**2 + g * np.cos(theta) - (k/m) * (r - L0) + epsilon / r**2
9      d2theta = -2 * dr * dtheta / r - (g / r) * np.sin(theta)
10     return np.array([dr, dtheta, d2r, d2theta])
11
12
13 g, k, m, L0, epsilon = 9.81, 1.0, 1.0, 10.0, -1
14
15 y0 = np.array([1.0, np.pi/6, 0.0, 0.0])
16
17 tmax = 100
18 n_eval = 1000
19 t_span, t_eval = (0, tmax), np.linspace(0, tmax, n_eval)
20 sol = solve_ivp(elastic_pendulum,
21                 t_span,
22                 y0,
23                 args=(g, k, m, L0, epsilon),
24                 t_eval=t_eval,
25                 method="RK45",
26                 rtol=1e-6,
27                 atol=1e-6,
28                 vectorized=False)
29
30 fig, axes = plt.subplots(2, 1, figsize=(10, 6), sharex=True)
31
32 axes[0].plot(sol.t, sol.y[0])

```

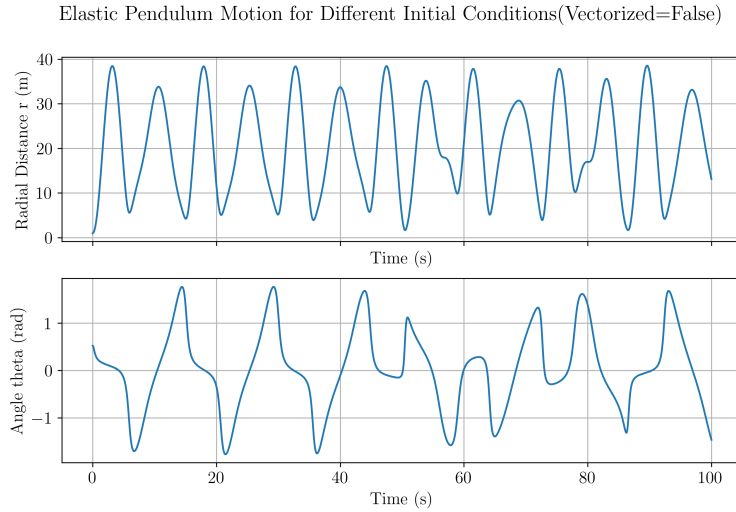


Figure V.1: Unvectorized Elastic Pendulum

```

33 axes[1].plot(sol.t, sol.y[1])
34
35 axes[0].set_xlabel("Time (s)")
36 axes[0].set_ylabel("Radial Distance r (m)")
37 axes[1].set_xlabel("Time (s)")
38 axes[1].set_ylabel("Angle theta (rad)")
39 # axes[0].legend()
40 axes[0].grid()
41 axes[1].grid()
42 plt.suptitle(("Elastic Pendulum Motion for Different Initial Conditions"
43              "(Vectorized=False)"
44              ))
45 # plt.show()
46 plt.savefig("figures/generated/unvectorized_elastic_pendulum.png", dpi=300, bbox_inches="tight")
47
48 plt.close()

```

See Figure V.1.

Solution To Exercise P.1: Let us consider a test vector

$$\mathbf{h} = \sum_i h^i \partial_i$$

As we noted

$$df(\mathbf{x})\mathbf{h} = \sum_i h^i \frac{\partial f}{\partial x^i}$$

Let

$$\nabla f(\mathbf{x}) = \sum_i u^i \frac{\partial}{\partial x^i}$$

The inner product

$$\begin{aligned}\langle \nabla f(\mathbf{x}), \mathbf{h} \rangle &= \left\langle \sum_i u^i \frac{\partial}{\partial x^i}, \sum_i h^i \frac{\partial}{\partial x^i} \right\rangle \\ &= \left\langle \sum_i u^i \frac{\partial}{\partial x^i}, \sum_j h^j \frac{\partial}{\partial x^j} \right\rangle \\ &= \sum_{ij} u^i h^j g_{ij}\end{aligned}$$

Hence, for every $\mathbf{h}$, in particular, when $h^j = \delta_k^j$ for fixed k .

$$\begin{aligned}\sum_{ij} u^i h^j g_{ij} &= \sum_j h^j \frac{\partial f}{\partial x^j} \\ \sum_i u^i g_{ik} &= \frac{\partial f}{\partial x^k}\end{aligned}$$

Multiplying by g^{kl} and summing over k :

$$\begin{aligned}\sum_{i,k} u^i g_{ik} g^{kl} &= \sum_k g^{kl} \frac{\partial f}{\partial x^k} \\ \sum_{i,k} u^i \delta_i^l &= \sum_k g^{kl} \frac{\partial f}{\partial x^k} \\ u^l &= \sum_k g^{kl} \frac{\partial f}{\partial x^k} \\ \nabla f &= \sum_l u^l \frac{\partial}{\partial x^l} = \sum_{kl} g^{kl} \frac{\partial f}{\partial x^k} \frac{\partial}{\partial x^l}\end{aligned}$$

Also, you may read a blog article on this subject¹. The subject is of significance importance in physics, and is likely to gain more attention in machine learning.

Thus, the formula for the gradient is:

$$\nabla f = \sum_{kl} g^{kl} \frac{\partial f}{\partial x^k} \frac{\partial}{\partial x^l}$$

Below there are a couple more alternative answers which illustrate some points.

There is an alternative answer using Einstein's summation convention:

$$\nabla f = g^{kl} f_{;k} \partial_l$$

where along the way we introduced the convention that subscripts after “;” refer to partial derivatives. Yet another form would compute the *components* of ∇f :

$$(\nabla f)^l = g^{kl} f_{;k}$$

which underscores the difference between a tensor (∇f) and its components $(\nabla f)^l$. We can write $\nabla f = (\nabla f)^l \partial_l$. Note that there cannot be $(\nabla f)_l$ in the answer as ∇f is a *vector*, or a purely *contravariant tensor* with one index. However, the *index lowering* operation on a tensor would yield:

$$(\nabla f)_m = (\nabla f)^l g_{lm} = g_{lm} g^{kl} f_{;k} = \delta_{mk} f_{;k} = f_{;m}$$

¹ See Gradient of a function in Riemannian manifold.

Therefore, $(\nabla f)_m$ is the same as $(df)_m$, which is a covariant tensor (*covector*), the derivative of f . This is the way of “getting back” df from ∇f . Note that we have a different take on the equation $df(\mathbf{x})\mathbf{h} = \langle \nabla f(\mathbf{x}), \mathbf{h} \rangle$:

$$\begin{aligned} df(\mathbf{x})\mathbf{h} &= (f_i dx^i)(h^j \partial_j) \\ &= f_{;i} h^j dx^i(\partial_j) \\ &= f_{;i} h^j \delta_j^i = f_{;i} h^i \end{aligned}$$

which is obviously true!

Solution To Exercise 15.11: Clearly, Y takes on values in $[0, 1]$. We need to prove that for any $y \in [0, 1]$

$$P(Y \leq y) = y.$$

Based on strict monotonicity, F is 1:1 and

$$(0, 1) \subseteq F(\mathbb{R}) \subseteq [0, 1].$$

There exists an inverse

$$F^{-1} : F(\mathbb{R}) \rightarrow \mathbb{R}.$$

We have for $y \in (0, 1)$:

$$\begin{aligned} P(Y \leq y) &= P(F(X) \leq y) = P(X \leq \inf F^{-1}(y)) \\ &= F(F^{-1}(y)) = y. \end{aligned}$$

By continuity of probability, this equation also holds for $y \in \{0, 1\}$. □

Solution To Exercise 15.12: We first define G . The first attempt is to define $G = F^{-1}$ by the equation

$$G(F(x)) = x.$$

But G is not always well-defined because if

$$F(x_1) = F(x_2)$$

and $x_1 \neq x_2$, we obtain a contradiction $G(F(x_1)) = x_1 \neq x_2 = G(F(x_2))$. Thus, G assigns two distinct values to the same argument. Hence, this definition only works if F is 1:1 (injective). Also, a function that is not 1:1, does not have the left inverse.

An onto (surjective function) has a (non-unique) right inverse, so no contradiction results, as G is required to be a right inverse. Else, we observe that, due to monotonicity and continuity, for every $u \in (0, 1)$ the preimage $F^{-1}(u)$ is a non-empty interval. Furthermore, because F is right-continuous, for every the preimage $F^{-1}(u)$ is either a point $a(u)$, or semi-closed interval of the form $[a(u), b(u))$. We define

$$G(u) \stackrel{\text{def}}{=} a(u) = \inf F^{-1}(u).$$

In other words, $G(u)$ is the smallest x such that $F(x) = u$.

This function is increasing and 1:1. It is a Borel function. To show this we need to prove that $G^{-1}((-\infty, x])$ is a Borel set. However, $G(u) \leq x \iff F(x) \geq u$, so the preimage is an interval (a Borel set).

Finally, let $X = G(U)$. We have

$$\begin{aligned} P(X \leq x) &= P(G(U) \leq x) = P(F(G(U)) \leq F(x)) \\ &= P(U \leq F(x)) = F(x) \end{aligned}$$

This proved the claim that F is the distribution function of $X = G(U)$. □

Bibliography

- [1] Alpha C. Chiang. *Fundamental Methods of Mathematical Economics*. 3rd. New York: McGraw-Hill, 1984. ISBN: 9780070108134. URL: https://books.google.com/books/about/Fundamental_methods_of_mathematical_econ.html?id=6gcoAQAAMAAJ.
- [2] Corinna Cortes and Vladimir Vapnik. “Support-vector networks”. In: *Machine learning* 20.3 (1995), pp. 273–297.
- [3] G. Dahlquist and Å. Björck. *Numerical Methods in Scientific Computing*. SIAM, 2008.
- [4] John E. Dennis and Robert B. Schnabel. *Numerical Methods for Unconstrained Optimization and Nonlinear Equations*. SIAM, 1996.
- [5] Daniel A. Fleisch. *A Student’s Guide to Vectors and Tensors*. Student’s Guides. Cambridge University Press, 2011.
- [6] E. Isaacson and H. B. Keller. *Analysis of Numerical Methods*. Wiley, 1966.
- [7] L.D. Landau and E.M. Lifshitz. *Mechanics*. 3rd. Vol. 1. Course of Theoretical Physics. Translated from the Russian. Oxford: Butterworth-Heinemann, 1976.
- [8] L.D. Landau and E.M. Lifshitz. *Механика*. Russian. Vol. 1. Курс теоретической физики. Moscow: Гостехиздат, 1940.
- [9] Jakob Leuschner, Felix Löffler, and Andreas Maier. “The LoDoPaB-CT Dataset: A Benchmark Dataset for Low-Dose Computed Tomography Reconstruction”. In: *arXiv preprint arXiv:1910.01113* (2019).
- [10] Andreu Mas-Colell, Michael D. Whinston, and Jerry R. Green. *Microeconomic Theory*. New York: Oxford University Press, 1995.
- [11] Encyclopedia of Mathematics. *Cantor ternary function*. 2024. URL: http://encyclopediaofmath.org/index.php?title=Cantor_ternary_function&oldid=35600.
- [12] Encyclopedia of Mathematics. *Gauss Quadrature Formula*. 2025. URL: https://encyclopediaofmath.org/index.php?title=Gauss_quadrature_formula.
- [13] MathWorks. *MATLAB Automatic Differentiation*. 2023. URL: <https://www.mathworks.com/help/deeplearning/ug/deep-learning-with-automatic-differentiation-in-matlab.html>.
- [14] Paul Milgrom and Ilya Segal. “Envelope Theorems for Arbitrary Choice Sets”. In: *Econometrica* 70.2 (2002), pp. 583–601.
- [15] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer Science & Business Media, 2006.
- [16] OpenAI. *ChatGPT: A Conversational AI Model*. 2023. URL: <https://chat.openai.com/>.
- [17] American Association of Physicists in Medicine. *AAPM Low-Dose CT Grand Challenge*. 2016. URL: <https://www.aapm.org/GrandChallenge/LCDose>.
- [18] GNU Operating System Project. *GSL — GNU Scientific Library*. 2025. URL: <https://www.gnu.org/software/gsl/>.
- [19] Python. *Python: Pip, Packaging and Virtual Environments*. 2023. URL: <https://packaging.python.org/guides/installing-using-pip-and-virtual-environments/>.

- [20] Python. *Python: Virtual Environments*. 2023. URL: <https://docs.python.org/3/library/venv.html>.
- [21] Python. *Python: Virtual Environments*. 2023. URL: <https://virtualenv.pypa.io/en/latest/>.
- [22] A. Ralston and P. Rabinowitz. *A First Course in Numerical Analysis*. Dover, 2001.
- [23] Leonid I. Rudin, Stanley Osher, and Emad Fatemi. “Nonlinear total variation based noise removal algorithms”. In: *Physica D: Nonlinear Phenomena* 60.1 (1992), pp. 259–268. ISSN: 0167-2789. DOI: [https://doi.org/10.1016/0167-2789\(92\)90242-F](https://doi.org/10.1016/0167-2789(92)90242-F). URL: <https://www.sciencedirect.com/science/article/pii/016727899290242F>.
- [24] Bernhard Schölkopf and Alexander J Smola. *Learning with Kernels*. MIT Press, 2002.
- [25] J. Stoer and R. Bulirsch. *Introduction to Numerical Analysis*. Springer, 2002.
- [26] Rangarajan K. Sundaram. *A First Course in Optimization Theory*. Cambridge: Cambridge University Press, 1996.
- [27] Andrey N. Tikhonov. “Regularization of incorrectly posed problems”. In: *Soviet Mathematics* (1963).
- [28] Vladimir Vapnik. *Statistical Learning Theory*. Wiley, 1998.
- [29] Ke Yan, Xiaosong Wang, Le Lu, and Ronald M Summers. *DeepLesion: A large-scale CT lesion dataset*. Elsevier, 2018.

List of Algorithms

2.1	Kahan's Algorithm	31
3.1	Bisection Method	47
3.2	Bisection Method with Explicit Loop Counter and Tolerance	48
3.3	Fixed-Point Iteration	50
3.4	Newton-Raphson Method	55
3.5	Secant Method	57
3.6	False Position Method	59
4.1	Gaussian Elimination without Pivoting	66
4.2	Gaussian Elimination with Partial Pivoting	66
4.3	Gaussian Elimination with Complete Pivoting	67
4.4	Back Substitution	67
4.5	LU Decomposition	68
4.6	Gaussian Elimination with Simulated Row Exchanges	69
4.7	PA=LU Decomposition	71
4.8	Jacobi Method	79
4.9	Gauss-Seidel Method	79
4.10	Successive Over-Relaxation (SOR)	80
4.11	Power Method	84
4.12	Power Method in 2D	96
4.13	QR Algorithm	98
4.14	Householder Transformation	100
4.15	Givens Rotation	103
4.16	Householder QR (thin Q)	104
4.17	QR Algorithm with Shifts	107
4.18	Inverse Power Method with Shifts	107
5.1	Fixed Point Iteration	121
5.2	Newton's Method	122
5.3	Broyden's Good Method	126
6.1	Gradient Descent	140
6.2	Newton's Method for Optimization	140
6.3	BFGS Quasi-Newton Method	141
6.4	Projection Method	149
7.1	Lagrange Interpolation	156
7.2	Newton Interpolation	156
7.3	Construction of the divided difference table	158
7.4	Cubic Spline Interpolation	159
7.5	Compute Cubic Spline Coefficients	161
7.6	Piecewise Interpolation	162
8.1	Romberg Integration	183
9.1	Euler's Method	189
9.2	Fourth-Order Runge-Kutta Method (RK4)	192

9.3	Two-Step Adams-Bashforth Method	196
9.4	One-Step Adams-Moulton Method (Trapezoidal Rule)	196
9.5	Runge-Kutta-Fehlberg algorithm (RK45 or RKF45) with adaptive steps size	198
9.6	Step adaptation in Runge-Kutta methods (excerpt)	200
9.7	Compute derivative of the Poincaré map at y_0	217
10.1	Shooting Method	232
10.2	Finite Difference Method	232
11.1	Direct Method for Solving the Algebraic Riccati Equation	250
12.1	Multigrid V-Cycle	289
13.1	Simplex Algorithm	292
14.1	Parallel Bisection Method	299
14.2	Parallel Matrix Multiplication	299
14.3	Parallel Finite Difference Method for PDEs	300
15.1	Monte Carlo Integration	301
15.2	Metropolis-Hastings Algorithm	306
15.3	Gibbs Sampling	313
H.1	Horner's Scheme	363
H.2	Synthetic Division	363
I.1	Modified Power Method	365

Index

- (Generalized) Mean Value Theorem for Divided Differences, 165
- A-stability, 209
- adaptive methods, 42
- adjoint operator, 90
- AI Tools
 - ChatGPT, 325
 - choosing programming language and tools, 326
 - code examples and templates, 326
 - coding assistance, 325
 - debugging code, 325
 - finding documentation, 326
 - finding tutorials, 326
- Apache Spark, 307
- area element, 79
- Armijo rule, 141, 142
- ASCII code, 26
- backtracking, 141
- backward stability, 30
 - view point, 30
- band matrix, 81
- BFGS, 136
 - L-BFGS, 136
 - secant condition, 138, 139
- block structure
 - discrete Laplace operator, 81
- bordered Hessian, 148, 369
 - generalized, 369
- Broyden's Good Method, 123–125, 131, 136
- Broyden-Fletcher-Goldfarb-Shanno, 136
- byte, 26
- Cantor ternary function, 47
- Cantor set
 - middle-third Cantor set, 47
- Cauchy integral formula, 355
- Cauchy sequence, 47
- central processing unit, CPU, 21
- Chambolle-Pock algorithm, 147
- ChatGPT, 17
- Chebyshev nodes, 153
- computer arithmetic, 25
 - error mitigation, 36
 - floating-point, 25
 - IEEE 754 standard, 25
 - machine epsilon
 - definition, 29
 - machine epsilon, 29
 - precision, 28
 - rounding modes, 28
 - unit in the last place, ulp, 37
- computer cluster, 22
- contraction mapping, 49
- contractionan mapping, 49
- convex quadratic programming, 359
- cotangent bundle, 251
- CUDA, 22
- Devil's Staircase, 47
- differentiable manifold
 - chart, 352
 - Grassmannian, 351
- differential geometry, 352
- Dirichlet boundary conditions, 79
- earth tomography, 146
- error
 - absolute, 40
 - amplification, 40
 - propagation, 40
 - relative, 40
 - round-off, 40
 - truncation, 40
 - types of, 40
- error analysis
 - backward, 41
 - forward, 42
- Euclidean norm, 71
- Farkas lemma, 383
 - classical cone form, 384
- finite difference scheme
 - Laplace operator, 79
- Frobenius norm, 71
- function of a matrix, 355

- Gauss-Legendre quadrature, 180
- Gaussian elimination, 63
 - with simulated row exchanges, 67
 - without pivoting, 64
- Gaussian Quadrature, 179
- golden ratio, 54
- Goldstein conditions, 141, 142
- Gordan's Theorem, 383
- Gordan–Farkas certificate, 383
- GPS, 115
 - trilateration, 115, 116
- GPU, 22
- Gram-Schmidt process
 - modified, MGS, 86
- Gram-Schmidt process
 - classical, CGS, 85
- hardware architectures, 21
- Hermite polynomials, 180, 181
- hermitian transpose, 97
- Heun's Method, 197, 218
- Hilbert matrix, 74
 - zero-based, 74
- homogenous space, 352
- Horner's Scheme, 347
- HPC, 21
 - importance of, 22
- IEEE 754 standard
 - precision and accuracy, 331
- IEEE 754 standard, 329
 - floating-point formats, 329
 - rounding modes, 330
 - single precision, 329
 - special values, 330
- ill-conditioned, 41
- index of inertia, 368
- indicator function, 300
- invariant interval, 49
- invariant set, 55
- invariant subspace, 350
- Jordan canonical form, 84
- Kronecker delta, 356
- Lagrange multipliers, 135, 370
- Laguerre polynomials, 180, 181
- Laplace operator, 79
 - finite difference approximation, 79
- Law of Large Numbers, 301
- least squares method
 - normal equations, 73

- Legendre polynomials, 181
- length element, 79
- Lie group theory, 352
- linearization
 - one-dimensional, 52
- loop invariant, 46
- Lorenz system, 202
- LU decomposition, 44, 63, 66
- Lyapunov exponent, 204–206
 - for Poincaré section, 205
- machine learning, 22
- machine number
 - definition, 28
- MapReduce, 307
- marginal cost, 251
- Markov chain
 - detailed balance, 292
 - reversibility, 291
- memory
 - distributed, 22
 - shared, 22
- Morse Lemma, 367
- MPI, 22, 284
 - function, 284
- MPI, Message Passing Interface, 22
- multiprocessing, 201
- Neumann series, 76
- Newton's backward difference formula, 159
- Newton's forward difference formula, 159
- Newton-Cotes formulas, 179
- nibble, 26
- normed space, 71
- numerical method, 21
 - consistency, 41
 - convergence, 41
 - reliability, 41
 - robust, 42
 - stability, 41
- ODE
 - aperiodic solutions, 201
 - chaotic solutions, 201
- one-based matrix, 74
- OpenCL, 22
- OpenMP, 22
- PA=LU decomposition, 66, 69
 - example, 70
- parallel processing, 21
- parallelizing a loop
 - OpenMP, 318

- PCA, 106
- pipeline architecture, 201
- pivoting
 - partial, 44, 63
- Poincaré section, 202
 - computing, 202
 - constructing, 203
- point charge, 78
- point geometry, 50
- Poisson equation, 78
- pole of the resolvent
 - higher order, 356
- Pontryagin Maximum Principle, 225, 226
 - Hamiltonian maximization, 226
 - state and costate dynamics, 226
 - transversality conditions, 226
- positive definite operator
 - negative Laplace operator, 78
- Power Method
 - Complex Conjugate Shifted Power Method, 92
- power method
 - modified, 349
- Principal Component Analysis, PCA, 61
- processing core
 - multiple, 21
- pythontex, 315
- PyTorch, 307
- reaction-diffusion system, 129
- reduced row echelon form, 373
- rejection sampling, 301
- representation theory, 352
- residue theorem, 356
- resolvent operator, 355
 - partial fraction expansion
 - multiple eigenvalues, 357
 - simple eigenvalues, 356
- root finding
 - bracket, 45
- root finding, 45
 - bisection method, 45
 - Newton-Raphson method, 53
 - Regula Falsi, 56
 - secand method, 54
- Rosenbrock function, 339
 - the computational graph, 340
 - the formula, 339
 - the gradient, 339
- rounding
 - Banker's rounding, 28
 - MATLAB rounding, 28
- secant method, 54
- series
 - Taylor, 40
- Sherman-Morrison formula, 122
- Sherman-Morrison-Woodbury formula, 138, 139
- SIMD, 319
- single-interval trapezoidal approximation, 169
- singular value, 105
- singular values, 74
- sparse matrix, 305
 - compressed sparse column format, CSC, 305
 - compressed sparse row format, CSR, 305
 - coordinate list format, COO, 305
- spectral decomposition
 - diagonalizable matrix, 355
- spectral projectors, 356
 - multiple eigenvalues, 356
- spectrum, 76
- Split Bregman algorithm, 147
- stencil
 - 5-point, 79
- Stirling's interpolation formula, 162
- stochastic differential equation, 307
- supercomputers, 21
- superscalar architecture, 201
- Support Vector Machines, 311, 359
- SVD
 - compact form, 107
 - full form, 107
- syntactic sugar
 - automatic differentiation, 57
- Synthetic Division, 347
- TensorFlow, 307
- the first Green's identities, 78
- Thomas algorithm, 157
- thread of execution
 - multiple, 21
- tolerance, 42
- two-ladder problem
 - classical, 49
- unitary matrix, 97
- validation, 42
- Van der Pol oscillator, 219
- vector length element, 79
- vectorization, 201, 319
 - NumPy, 319
- verification, 42
- well-conditioned problem, 41
- Wolfe conditions, 141, 142
 - BFGS, 137

curvature, 149
strong, 142, 149
sufficient decrease, 149

zero-based matrix, 74